Spring Handbook

Spring Framework 7 · Spring Boot 4 · DI & AOP · MVC & REST · Data JPA · Security · Testing · WebFlux · Messaging · Observability · Cloud · Interview prep

19 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The Spring Ecosystem and Your First App**
   - 1.1 What Spring is
   - 1.2 What Spring Boot adds
   - 1.3 The Spring ecosystem
   - 1.4 History and versions
   - 1.5 Creating a project with start.spring.io
   - 1.6 Project layout and the build file
   - 1.7 Starters
   - 1.8 The main class and `@SpringBootApplication`
   - 1.9 Running the application
   - 1.10 DevTools
   - 1.11 Your first REST endpoint
   - 1.12 Testing the first app
   - 1.13 In the interview
   - Chapter TL;DR
2. **Chapter 2 — IoC and Dependency Injection**
   - 2.1 Inversion of Control and Dependency Injection
   - 2.2 The container: `BeanFactory` vs `ApplicationContext`
   - 2.3 Defining beans
   - 2.4 Injection types
   - 2.5 How Spring picks a bean
   - 2.6 Bean scopes
   - 2.7 The bean lifecycle
   - 2.8 Post-processors: the container's extension points
   - 2.9 Circular dependencies
   - 2.10 Lazy initialization
   - 2.11 `@Conditional` and profiles
   - 2.12 Application events
   - 2.13 Null-safety with JSpecify (Framework 7)
   - 2.14 Programmatic registration with `BeanRegistrar` (Framework 7)
   - 2.15 In the interview — quick round
   - Chapter TL;DR
3. **Chapter 3 — Configuration and Auto-configuration**
   - 3.1 Externalized configuration and the `Environment`
   - 3.2 `application.properties` vs `application.yaml`
   - 3.3 Precedence: which value wins?
   - 3.4 Environment variables and relaxed binding
   - 3.5 `@Value`
   - 3.6 `@ConfigurationProperties` — type-safe configuration
   - 3.7 Profiles, profile-specific files and profile groups
   - 3.8 Importing configuration: `spring.config.import`
   - 3.9 Placeholders, random values and other sources
   - 3.10 How auto-configuration works
   - 3.11 Seeing what Boot decided
   - 3.12 Overriding and excluding auto-configuration
   - 3.13 Writing your own auto-configuration and starter
   - 3.14 Boot 4 modularization
   - 3.15 In the interview — quick round
   - Chapter TL;DR
4. **Chapter 4 — AOP and Proxies**
   - 4.1 Cross-cutting concerns and the AOP vocabulary
   - 4.2 Writing aspects with `@Aspect`
   - 4.3 Pointcut expressions
   - 4.4 How Spring AOP works: proxies
   - 4.5 Proxy limitations — proved
   - 4.6 Spring features built on proxies
   - 4.7 Framework 7 core resilience: `@Retryable` and `@ConcurrencyLimit`
   - 4.8 Spring AOP vs AspectJ
   - 4.9 Common mistakes (whole chapter)
   - 4.10 In the interview
   - 4.11 Chapter TL;DR
5. **Chapter 5 — Spring MVC and REST APIs**
   - 5.1 REST in one page
   - 5.2 How a request flows: the `DispatcherServlet`
   - 5.3 Controllers and request mappings
   - 5.4 Reading input and writing output
   - 5.5 JSON with Jackson 3
   - 5.6 Validation
   - 5.7 Error handling and Problem Details (RFC 9457)
   - 5.8 Content negotiation
   - 5.9 API versioning (Framework 7)
   - 5.10 CORS and file upload
   - 5.11 Filters vs interceptors (vs AOP)
   - 5.12 Documenting the API: OpenAPI with springdoc
   - 5.13 Putting it together: design checklist and mistakes
   - 5.14 In the interview
   - 5.15 Chapter TL;DR
6. **Chapter 6 — REST Clients and HTTP Interfaces**
   - 6.1 Choosing a client
   - 6.2 `RestClient` basics
   - 6.3 Error handling
   - 6.4 Timeouts and the underlying HTTP library
   - 6.5 Interceptors
   - 6.6 HTTP service interfaces (`@HttpExchange`)
   - 6.7 Client-side API versioning and retries
   - 6.8 Testing clients
   - 6.9 `RestTemplate` and `WebClient` in brief
   - 6.10 Common mistakes, real-world examples, interview
   - 6.11 Chapter TL;DR
7. **Chapter 7 — Data Access with JDBC and Transactions**
   - 7.1 The data-access stack
   - 7.2 DataSource and HikariCP
   - 7.3 JdbcClient and JdbcTemplate
   - 7.4 Schema and data initialization
   - 7.5 Exception translation: DataAccessException
   - 7.6 Testing JDBC code with @JdbcTest
   - 7.7 How a Spring transaction works
   - 7.8 @Transactional attributes
   - 7.9 Propagation
   - 7.10 Rollback rules
   - 7.11 Isolation levels
   - 7.12 readOnly and timeout
   - 7.13 Transaction pitfalls
   - 7.14 Programmatic transactions: TransactionTemplate
   - 7.15 @TransactionalEventListener
   - 7.16 Interview questions
   - Chapter TL;DR
8. **Chapter 8 — Spring Data JPA and Hibernate**
   - 8.1 JPA, Hibernate and Spring Data — who does what
   - 8.2 Setting up
   - 8.3 Entities and ids
   - 8.4 The persistence context and dirty checking
   - 8.5 Relationships
   - 8.6 Fetch types and LazyInitializationException
   - 8.7 The N+1 problem
   - 8.8 Repositories

- 8.9 Derived queries
- 8.10 @Query: JPQL, native SQL and @Modifying
- 8.11 Pagination and sorting
- 8.12 Projections
- 8.13 Auditing
- 8.14 Locking: optimistic and pessimistic
- 8.15 Open Session in View
- 8.16 Schema migrations with Flyway
- 8.17 Testing JPA code
- 8.18 Performance checklist
- 8.19 Interview questions
- Chapter TL;DR

9. **Chapter 9 — Spring Data beyond JPA, and caching**
   - 9.1 The Spring Data family
   - 9.2 Spring Data JDBC and aggregates
   - 9.3 Spring Data REST
   - 9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase
   - 9.5 Spring Data MongoDB (not run — needs Docker)
   - 9.6 Spring Data Redis (not run — needs Docker)
   - 9.7 Testing MongoDB and Redis with Testcontainers (not run)
   - 9.8 The caching abstraction
   - 9.9 Choosing a data technology
   - 9.10 Interview questions
   - Chapter TL;DR
10. **Chapter 10 — Spring Security 1: Fundamentals**
   - 10.1 Vocabulary and what the starter does by default
   - 10.2 Architecture: the filter chain
   - 10.3 Configuring a chain: the lambda DSL in Spring Security 7
   - 10.4 Authentication: users, providers, form login and Basic
   - 10.5 Password storage: hashing with `DelegatingPasswordEncoder`
   - 10.6 Authorizing requests: roles, authorities, 401 vs 403
   - 10.7 Method security
   - 10.8 CSRF protection
   - 10.9 CORS
   - 10.10 Sessions, the security context and logout
   - 10.11 Security headers
   - 10.12 Testing security with `spring-security-test`
   - 10.13 Common mistakes (whole chapter)
   - 10.14 In the interview
   - Chapter 10 TL;DR
11. **Chapter 11 — Spring Security 2: Tokens and OAuth2**
   - 11.1 From sessions to tokens: bearer tokens and JWT
   - 11.2 OAuth 2.0 and OpenID Connect in one picture
   - 11.3 A JWT resource server in Spring
   - 11.4 Minting tokens with `NimbusJwtEncoder`
   - 11.5 Mapping claims to authorities
   - 11.6 Testing token security
   - 11.7 Opaque tokens and introspection
   - 11.8 Multi-tenancy: several issuers in one API
   - 11.9 OAuth2 login: "Sign in with ..."
   - 11.10 Calling other services: client credentials and token relay
   - 11.11 Refresh tokens, lifetimes, revocation and logout
   - 11.12 Where should a browser keep tokens? The BFF pattern
   - 11.13 Storing keys and secrets
   - 11.14 Spring Authorization Server
   - 11.15 Rate limiting and brute-force protection
   - 11.16 Common mistakes (whole chapter)
   - 11.17 In the interview
   - 11.18 Security fundamentals beyond Spring
   - Chapter 11 TL;DR
12. **Chapter 12 — Testing Spring Applications**
   - 12.1 The application under test
   - 12.2 The test pyramid in a Spring app
   - 12.3 Test dependencies in Boot 4
   - 12.4 Unit tests: no Spring at all
   - 12.5 How the Spring TestContext Framework works
   - 12.6 `@SpringBootTest`: the whole application
   - 12.7 Test slices
   - 12.8 `MockMvcTester`: AssertJ-style MockMvc
   - 12.9 `RestTestClient` (Framework 7.0)
   - 12.10 `@MockitoBean` and `@MockitoSpyBean`
   - 12.11 Test context caching
   - 12.12 `@DynamicPropertySource`: properties known only at runtime
   - 12.13 Full-stack tests on a real port
   - 12.14 Testcontainers and `@ServiceConnection`
   - 12.15 Profiles and properties in tests
   - 12.16 Common mistakes — checklist
   - 12.17 In the interview
   - 12.18 Types of API testing — and the Spring tool for each
   - 12.19 Chapter TL;DR
13. **Chapter 13 — Reactive Programming and Spring WebFlux**
   - 13.1 Why reactive? The cost of waiting
   - 13.2 Reactive Streams: the contract
   - 13.3 Project Reactor: `Mono` and `Flux`
   - 13.4 Operators
   - 13.5 Testing with `StepVerifier` and virtual time
   - 13.6 Error handling
   - 13.7 Backpressure in practice
   - 13.8 Schedulers and threads
   - 13.9 Hot vs cold, sinks and context
   - 13.10 Spring WebFlux: annotated controllers
   - 13.11 Functional endpoints
   - 13.12 Full-stack WebFlux tests and Server-Sent Events
   - 13.13 `WebClient`: calling other services without blocking
   - 13.14 R2DBC: reactive SQL
   - 13.15 Virtual threads vs WebFlux: a measured comparison
   - 13.16 Common mistakes — checklist
   - 13.17 In the interview
   - 13.18 Chapter TL;DR
14. **Chapter 14 — Messaging, Async and Scheduling**
   - 14.1 The map: four ways to do work "somewhere else"
   - 14.2 Application events: synchronous vs `@Async` listeners
   - 14.3 `@TransactionalEventListener`: events tied to a commit
   - 14.4 `@Async` and executors
   - 14.5 `@Async` on virtual threads
   - 14.6 Exceptions in async methods
   - 14.7 Scheduling with `@Scheduled`
   - 14.8 Messaging concepts you need first
   - 14.9 Kafka in five minutes
   - 14.10 Setting up Spring Kafka (with JSON)
   - 14.11 Producing: `KafkaTemplate`, keys and partitions
   - 14.12 Consuming: `@KafkaListener`, concurrency and consumer groups
   - 14.13 Testing with `@EmbeddedKafka`
   - 14.14 Error handling: `DefaultErrorHandler` and the dead-letter topic
   - 14.15 Non-blocking retries with `@RetryableTopic`
   - 14.16 Producer guarantees: idempotence and transactions
   - 14.17 The transactional outbox
   - 14.18 RabbitMQ with Spring AMQP (configuration, not run)
   - 14.19 Common mistakes (whole chapter)
   - 14.20 In the interview
   - Chapter TL;DR
15. **Chapter 15 — Observability and Production**

- 15.1 Observability in one page
- 15.2 Spring Boot Actuator
- 15.3 Health, health groups and Kubernetes probes
- 15.4 The info endpoint
- 15.5 Metrics with Micrometer
- 15.6 The Observation API and `@Observed`
- 15.7 Distributed tracing
- 15.8 Structured logging
- 15.9 Graceful shutdown
- 15.10 Virtual threads
- 15.11 Faster startup: extracted jars, CDS, AOT cache, native images
- 15.12 Production configuration checklist
- 15.13 Common mistakes (whole chapter)
- 15.14 In the interview
- Chapter TL;DR

16. **Chapter 16 — Spring Cloud and Microservices**

- 16.1 The map: microservice problems and Spring answers
- 16.2 Release trains and versions
- 16.3 Centralized configuration: Spring Cloud Config
- 16.4 Service discovery: Eureka, Consul, Kubernetes
- 16.5 Client-side load balancing with Spring Cloud LoadBalancer
- 16.6 Declarative clients: OpenFeign vs HTTP interface clients
- 16.7 API gateway: Spring Cloud Gateway
- 16.8 Resilience: circuit breaker, retry, rate limiter
- 16.9 Distributed tracing across services
- 16.10 Contract testing
- 16.11 Messaging between services
- 16.12 Common mistakes (whole chapter)
- 16.13 In the interview
- Chapter TL;DR

17. **Chapter 17 — Spring AI and What's New in Boot 4 / Framework 7**

- 17.1 What Spring AI is
- 17.2 Setting up a project
- 17.3 `ChatModel` and `ChatClient`
- 17.4 Prompts and prompt templates
- 17.5 Advisors
- 17.6 Chat memory
- 17.7 Structured output
- 17.8 Tool calling
- 17.9 RAG: retrieval-augmented generation
- 17.10 MCP: the Model Context Protocol
- 17.11 Production concerns
- 17.12 What's new: how to read the rest of this chapter
- 17.13 Spring Boot 3.x → 4.x migration table
- 17.14 Spring Framework 6 → 7
- 17.15 Spring Security 6 → 7
- 17.16 What Boot 4.1 adds (June 2026)
- 17.17 In the interview — quick round
- Chapter TL;DR

18. **Chapter 18 — Spring Interview Questions**

- 18.1 How to use this chapter
- 18.2 Core container and dependency injection
- 18.3 Configuration and auto-configuration
- 18.4 AOP and proxies
- 18.5 Spring MVC, REST and HTTP clients
- 18.6 Data access, JPA and transactions
- 18.7 Caching
- 18.8 Security
- 18.9 Testing
- 18.10 Reactive and WebFlux
- 18.11 Messaging, async and scheduling
- 18.12 Production and observability
- 18.13 Microservices and Spring Cloud
- 18.14 Spring AI and the Boot 4 / Framework 7 changes
- 18.15 Proofs: five tricky answers, run
- 18.16 "Explain your Spring project" — a walkthrough template
- 18.17 Debugging scenarios
- Chapter TL;DR

19. **Chapter 19 — Cheat Sheet and Glossary**

- 19.1 How to use this chapter
- 19.2 Versions at a glance
- 19.3 Annotations cheat sheet
- 19.4 Starters in Boot 4.1
- 19.5 Key properties cheat sheet
- 19.6 Transactions: `@Transactional` at a glance
- 19.7 Bean scopes and the lifecycle
- 19.8 Spring Security 7 DSL snippets
- 19.9 Testing annotations and slices
- 19.10 Reactor operators quick table
- 19.11 Actuator endpoints
- 19.12 Which tool for which job
- 19.13 Boot 3 → 4 quick migration checklist
- 19.14 Interview index
- 19.15 Proofs: the facts on these sheets, run
- 19.16 Glossary A–Z
- Chapter TL;DR

Chapter 1 — The Spring Ecosystem and Your First App

Spring is the most widely used framework for building server-side Java applications (programs that run on a server and answer other programs, rather than apps on your own screen): web APIs, back-office systems, batch jobs (programs that process a large pile of data in one run, often at night), message consumers (programs that react to messages arriving on a queue).

> **Key term — API / REST API:** An **API** (Application Programming Interface) is the set of operations one program offers to other programs. A **web API** offers them over HTTP, the protocol browsers use. A **REST API** is a web API in a common style: each thing (an order, a user) has a URL, and you act on it with standard HTTP methods such as `GET` (read) and `POST` (create), usually exchanging JSON. See System Design Fundamentals for HTTP basics. This chapter explains what Spring and Spring Boot actually are, where they came from, which versions are current, and then walks you through building, running, packaging and testing a first REST API with **Spring Boot 4.1.1** on **Java 25**.

You only need basic Java for this book. When a Java feature matters (records, annotations, lambdas, Maven), this book links to the Java Handbook instead of teaching it again.

What you'll learn

1. What a framework is, and what problem Spring solves (§1.1).
2. What Spring Boot adds on top of Spring (§1.2).
3. The main projects of the Spring ecosystem and where each is covered in this book (§1.3).
4. History and versions: Framework 7, Boot 4, Java 17 baseline, Jakarta EE 11 (§1.4).
5. Creating a project with start.spring.io (§1.5) and reading its layout and `pom.xml` (§1.6).
6. Starters — what they are and what Boot 4 changed about them (§1.7).
7. The main class, `@SpringBootApplication`, and what `SpringApplication.run` does (§1.8).
8. Running the app: IDE, `mvn spring-boot:run`, the executable jar, reading the startup log (§1.9).
9. DevTools for fast restarts (§1.10).
10. Your first REST endpoint (§1.11) and your first two tests (§1.12).

All runnable code is in `code/hello-boot/`: a Maven project whose tests were run with `mvn -q -B -ntp verify` on JDK 25. Outputs below are real (long log lines trimmed with `...`).

1.1 What Spring is

What it is

A **library** is code you call (for example, a JSON parser). A **framework** is code that calls *you*: it owns the program's structure and plugs your classes into it at the right moments. This reversal is called **Inversion of Control (IoC)** — "don't call us, we'll call you".

Spring Framework is an open-source (its source code is public and free to use) Java framework whose core is an **IoC container**: an object that creates your application's objects, connects them to each other, configures them, and manages their whole life. Objects managed by the container are called **beans**. Around that core, Spring offers modules for web applications (Spring MVC — the classic web framework where each request gets its own thread; WebFlux — a non-blocking alternative), data access (**JDBC** — Java Database Connectivity, Java's standard low-level API for talking to SQL databases; and **transactions** — groups of database changes that succeed or fail as one unit), messaging, testing, scheduling and more.

Analogy: building a restaurant yourself means hiring every cook, buying every oven, and telling each cook where the ovens are. With Spring you write a list — "I need a cook who needs an oven" — and a manager (the container) hires them, hands each cook an oven, opens the doors in the morning and closes up at night.

The problem it solves

Without a container, every class builds its own dependencies with `new`:

```
// Hard-wired: OrderService decides WHICH repository and WHICH mailer it uses.
class OrderService {
    private final OrderRepository repo = new JdbcOrderRepository(new HikariDataSource(...));
    private final Mailer mailer = new SmtpMailer("smtp.example.com", 25);
}
```

This causes three problems:

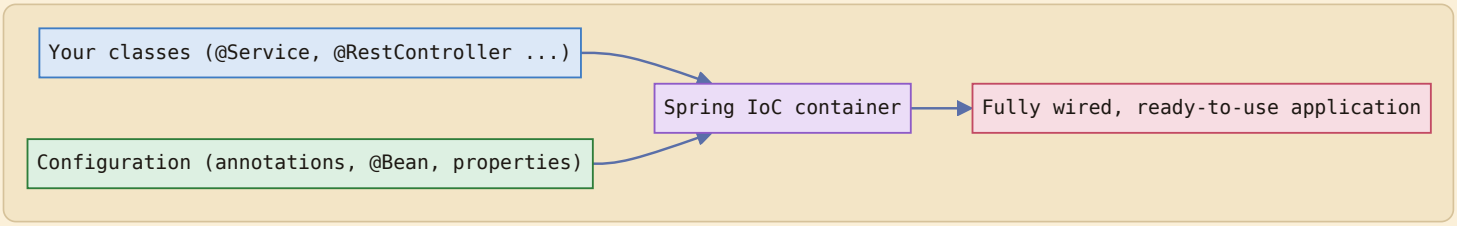
Problem	Why it hurts
Tight coupling	Swapping `JdbcOrderRepository` for another implementation means editing `OrderService`.
Hard to test	A unit test of `OrderService` now needs a real database and a real mail server.
Scattered configuration	Host names, pool sizes and credentials are buried in constructors all over the code.

Spring's answer is **Dependency Injection (DI)**: a class *declares* what it needs (usually as constructor parameters) and the container *supplies* it. The class depends on an interface; the container picks the implementation. Chapter 2 covers DI in depth; the design principle behind it is the Dependency Inversion Principle — see LLD §4.10 and the Hollywood Principle in LLD Chapter 5.

Beyond DI, Spring gives you ready-made, battle-tested answers for the plumbing every server application needs: HTTP request handling, JSON conversion (**JSON** is a plain-text data format like `{"name":"Ada"}`, the usual way web APIs exchange data), database transactions, security, scheduling, caching, and consistent exception types.

How it works (bird's-eye view)

1. You write ordinary Java classes (often called POJOs — "Plain Old Java Objects") and mark the ones Spring should manage, for example with `@Service` (an **annotation**: a `@`-label on a class or method that tools and frameworks can read at runtime).
2. At startup the container reads that **configuration metadata** (annotations, `@Bean` methods, or, in old projects, XML).
3. It creates each bean, injecting the other beans it needs.
4. It may wrap beans in **proxies** (stand-in objects that sit in front of the real object, intercept calls, and pass them on) to add behaviour such as transactions or security (Chapter 4).
5. Your application runs; the container calls your code when HTTP requests, messages or timers arrive.
6. At shutdown the container calls clean-up callbacks and destroys the beans.



Real-world examples

Spring (usually via Spring Boot) runs large parts of the back ends of banks, retailers, airlines, streaming services and public-sector systems. Netflix, for example, has publicly described moving its Java services to Spring Boot. It is also the most common Java stack in job descriptions and back-end interviews.

TL;DR

Spring is a framework built around an IoC container that creates and wires your objects (beans) through dependency injection, plus modules for web, data, security and more.

1.2 What Spring Boot adds

What it is

Spring Boot is a project built *on top of* Spring Framework that makes Spring applications quick to create and ready for production. It does not replace Spring — every Boot application *is* a Spring application. Boot adds:

Feature	What it means	Where in this book
Starters	One dependency per capability (`spring-boot-starter-webmvc`) that brings a tested set of libraries.	§1.7
Dependency management	Boot picks compatible versions of ~hundreds of libraries, so your `pom.xml` has no `<version>` tags.	§1.6
Auto-configuration	Boot looks at your classpath (the jars your app includes, §1.6) and settings and creates sensible beans for you (a web server, a JSON mapper, a `DataSource` — an object that hands out database connections...). Your own beans always win.	Chapter 3
Embedded server	Tomcat (or Jetty) — a web server that receives HTTP requests — runs *inside* your app. No separate server to install; you run `java -jar app.jar`.	§1.9
Externalized configuration	Settings come from `application.properties`/`.yaml`, environment variables (named values set by the operating system or container, e.g. `SERVER_PORT=9090`), command-line arguments... with a clear precedence order.	Chapter 3
Production features	Health checks ("is the app OK?" URLs), metrics (numbers like request counts), tracing (following one request across services), structured logging, graceful shutdown (Actuator).	Chapter 15
Testing support	Slices (tests that start only one layer of the app) like `@WebMvcTest`, `@SpringBootTest`, test starters.	Chapter 12

Analogy: Spring Framework is a professional kitchen full of equipment. Spring Boot is the same kitchen delivered pre-assembled, with the ovens already connected and a sensible default menu — you can still move any piece of equipment.

Why it matters

Before Boot (2014), a Spring web app needed a WAR file (Web Application Archive — a zip of your app that you copy into a separately installed server), an external Tomcat, XML or Java configuration for the `DispatcherServlet` (Spring MVC's central request dispatcher, Chapter 5), a view resolver (maps a view name to an HTML template), a JSON converter, a data source, and a careful choice of library versions that worked together. Boot reduced that to a generated project and a `main` method, while keeping every default overridable.

Spring vs Spring Boot

Question	Spring Framework	Spring Boot
What is it?	The core framework: IoC container, MVC, transactions...	An opinionated (it makes sensible default choices for you) layer that configures Spring for you
Can you use one without the other?	Yes — plain Spring works without Boot	No — Boot always uses Spring Framework
Configuration	You declare the infrastructure beans yourself	Auto-configured; you override only what you need
Server	Deploy a WAR to an external server (traditional)	Embedded server inside an executable jar
Versions	You choose each library version	Managed by Boot's BOM (version list, §1.6)
Typical use today	Libraries, special setups	Almost every new application

Common mistakes

- Thinking "Spring Boot" is a different framework with different annotations. `@Service`, `@Autowired`, `@Transactional`, `@RestController` are all Spring *Framework* annotations; Boot mostly adds `@SpringBootApplication`, `@ConfigurationProperties` and test annotations.
- Believing auto-configuration is magic you cannot inspect. It is ordinary `@Configuration` classes guarded by conditions, and Boot can print a report of every decision (Chapter 3).

In the interview

Q: What is the difference between Spring and Spring Boot? *30-second answer:* Spring Framework provides the IoC container and modules like MVC and transactions. Spring Boot sits on top: it adds starters for dependency sets, a managed version BOM, auto-configuration that creates beans based on the classpath and properties, an embedded server so the app runs as a plain jar, externalized configuration and production features like Actuator. Boot removes boilerplate (repetitive setup code that is the same in every project); it does not replace Spring.

- *Follow-up: Can you override what Boot configures?* Yes. Auto-configured beans are usually guarded by `@ConditionalOnMissingBean`, so defining your own bean of that type makes Boot back off; many settings are also just properties.
- *Follow-up: Does Boot make the app slower?* Startup does some extra condition evaluation, but the runtime code path is the same Spring code. Boot also offers AOT (ahead-of-time: doing work at build time instead of at startup) processing and native images (the app compiled into a stand-alone machine-code executable that needs no JVM — the Java Virtual Machine, the program that normally runs compiled Java code) for faster startup (Chapter 15).

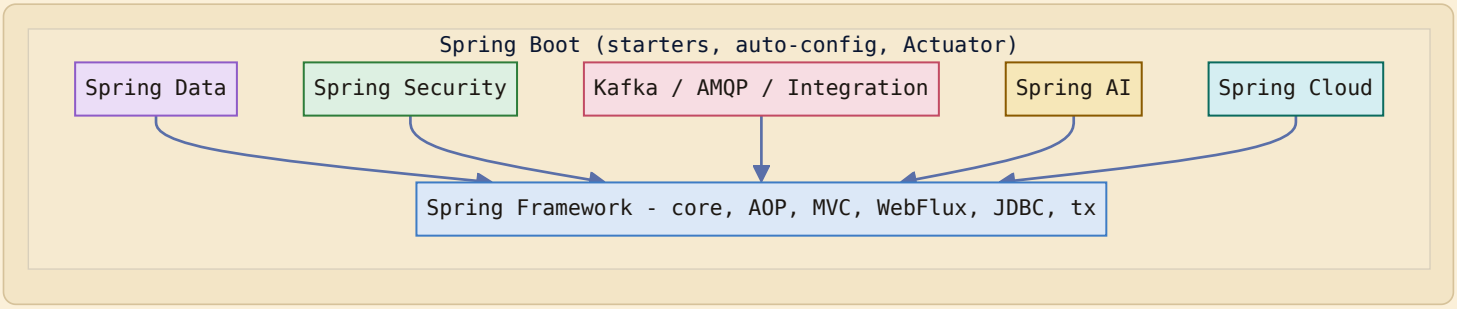
TL;DR

Spring Boot = Spring Framework + starters + managed versions + auto-configuration + embedded server + externalized config + production features.

1.3 The Spring ecosystem

"Spring" is also an umbrella name for a family of projects maintained by the Spring team (part of Broadcom since its acquisition of VMware). They share a release rhythm and are all version-managed by Spring Boot, so you rarely pick their versions yourself.

Project	What it gives you	Chapter
Spring Framework	IoC container, AOP (Aspect-Oriented Programming — adding cross-cutting behaviour such as logging around methods, Chapter 4), MVC, WebFlux, JDBC, transactions, testing	2, 4, 5, 7, 13
Spring Boot	Starters, auto-configuration, embedded servers, Actuator	1, 3, 12, 15
Spring Data	Repository interfaces (you declare the data-access methods; Spring writes the code) for JPA, JDBC, MongoDB, Redis, R2DBC...	8, 9
Spring Security	Authentication (who are you?), authorization (what may you do?), OAuth2 (a standard for delegated login/access), JWT (a signed token carrying user claims)	10, 11
Spring Cloud	Config server, service discovery (services finding each other's addresses), gateway (single entry point in front of many services), load balancing (spreading requests over several copies)	16
Spring for Apache Kafka / Spring AMQP	Messaging with Kafka and RabbitMQ (message brokers: servers that store and forward messages between programs)	14
Spring Batch	Large, restartable batch jobs	— (mentioned in 14)
Spring Integration	Enterprise integration patterns (channels, adapters)	—
Spring Session	Store HTTP sessions (per-user server-side state across requests) in Redis or a database	10
Spring for GraphQL	GraphQL (an API query language where the client picks the fields it wants) servers on Spring	—
Spring Modulith	Structure a monolith (one app deployed as a single unit) into verified modules	16
Spring AI	LLM (large language model) chat clients, embeddings (text as number vectors), RAG (retrieval-augmented generation), tool calling	17



Real-world stack example. A typical online-shop order service (one small, independently deployed app in a larger system) is: Spring Boot + Spring MVC (REST API) + Spring Data JPA (PostgreSQL, a popular open-source SQL database) + Spring Security (JWT) + Spring for Apache Kafka (publish "order placed" events) + Actuator/Micrometer (Micrometer is the metrics library Spring uses). This book builds each of those layers.

1.4 History and versions

A short timeline

Year	Release	What changed (in one line)
2002	Rod Johnson's book *Expert One-on-One J2EE Design and Development*	Showed a lighter alternative to EJB (Enterprise JavaBeans, the heavyweight component model of J2EE — the old name of enterprise Java); its sample code became Spring
2004	Spring Framework 1.0	IoC container, AOP, transaction abstraction, XML configuration
2007	Spring 2.5	Annotation-driven config: `@Autowired`, `@Component`, `@Controller`
2009	Spring 3.0	Java 5 baseline, `@Configuration`/`@Bean` Java config, SpEL (Spring Expression Language, small `#{...}` expressions), REST support in MVC
2013	Spring 4.0	Java 8 support, `@Conditional` (the foundation of auto-configuration)
2014	Spring Boot 1.0	Starters, auto-configuration, embedded servers
2017–18	Spring 5.0 / Boot 2.0	Java 8 baseline (minimum supported version), reactive stack (WebFlux; non-blocking, stream-based code)
2022	Spring 6.0 / Boot 3.0	**Java 17 baseline**, `javax.*` → `jakarta.*`, AOT and GraalVM (a JDK that can compile Java to native executables) native images
Nov 2025	**Spring Framework 7.0** (13 Nov) / **Spring Boot 4.0** (20 Nov)	Jakarta EE 11 (the standard enterprise Java APIs), JSpecify null-safety (annotations marking what may be `null`), modular Boot jars, Jackson 3, API versioning, HTTP service clients, JUnit 6
Jun 2026	Spring Boot 4.1.0 (10 Jun)	gRPC (a fast binary remote-call protocol from Google) support, Jackson config updates, observability updates
Aug 2026	Spring Boot 4.1.1 (20 Aug)	Bug fixes and dependency upgrades — **the version used in this book**

The current generation — what Boot 4.1.1 gives you

The versions below come from Boot 4.1.1's dependency BOM (`spring-boot-dependencies-4.1.1.pom`), so they are exactly what your project gets:

Component	Version in Boot 4.1.1	Notes
Java	**17 minimum**, 25 recommended	Framework 7 keeps the Java 17 baseline; this book uses Java 25 (LTS — long-term support release) — see Java Ch. 17
Spring Framework	7.0.9	
Jakarta EE	11 (Servlet 6.1 — the Java web-server API; JPA 3.2 — the Java object/database mapping API; Bean Validation 3.1 — rules like `@NotNull`)	
Tomcat / Jetty	11.0.24 / 12.1.12	Undertow is no longer supported (it lacks Servlet 6.1)
Jackson	3.1.5 (`tools.jackson` packages)	Jackson 2 still available via a separate module, deprecated
Hibernate ORM (object-relational mapper: maps Java objects to database tables) / Validator	7.4.5 / 9.1.3	
Spring Security	7.1.1	
Spring Data	2026.0.1 release train (a named set of modules released together)	
JUnit Jupiter	6.0.3	JUnit 4 support is deprecated in Framework 7
Mockito (fake objects for tests) / AssertJ (readable test assertions)	5.23.0 / 3.27.7	
Kotlin (a JVM language)	2.3.21 (2.2 minimum)	
GraalVM (native images)	25+	

`javax` → `jakarta`

Java EE was handed to the Eclipse Foundation in 2017 and renamed **Jakarta EE**; for trademark reasons its packages were renamed from `javax.*` to `jakarta.*` (Jakarta EE 9, 2020). Since Spring 6 / Boot 3, Spring uses only the `jakarta` names:

`jakarta.servlet.http.HttpServletRequest`, `jakarta.persistence.Entity`, `jakarta.annotation.PostConstruct`, `jakarta.validation.constraints.NotNull`. Framework 7 also removed the last support for `javax.annotation` and `javax.inject`. If a tutorial imports `javax.persistence`, it is for Boot 2 or older.

How versions relate

- **Boot pins everything.** You choose one number — the Boot version — and the BOM chooses compatible versions of Spring Framework, Spring Data, Security, Tomcat, Jackson, Hibernate, JUnit, etc.
- **Release cadence.** Boot ships a new minor version (4.0, 4.1, ... — new features, no breaking changes) roughly every six months, and patch releases (4.1.1, 4.1.2 — bug fixes only) regularly in between.
- **Support.** Each Boot minor line gets roughly 13 months of free open-source support, then commercial support. Stay on a supported minor; check the "Support" section at spring.io/projects/spring-boot before planning an upgrade.
- **Version names.** `-M1`, `-M2` are milestones, `-RC1` release candidates, a plain `4.1.1` is GA (generally available). `-SNAPSHOT` versions are nightly builds — never use them in production.

Common mistakes

- Adding `<version>` to a Spring dependency in a Boot project. It overrides the tested version and can break compatibility. Let the parent manage it.
- Mixing `javax.*` and `jakarta.*` imports after copying old code.
- Following a Boot 2/3 tutorial for Boot 4 without checking renamed starters (§1.7) and Jackson 3 package names.

Self-check

1. Which Java version is the minimum for Spring Framework 7? → Java 17 (25 is recommended).
2. Why does Spring 6+ use `jakarta.persistence` instead of `javax.persistence`? → Java EE moved to Eclipse as Jakarta EE and the packages were renamed.
3. In a Boot project, who decides the Tomcat version? → Boot's dependency BOM (`spring-boot-dependencies`).

TL;DR

This book uses Spring Boot 4.1.1 → Spring Framework 7.0.9, Jakarta EE 11, Tomcat 11, Jackson 3, JUnit 6, on Java 25. Pick the Boot version; Boot picks the rest.

1.5 Creating a project with start.spring.io

What it is

Spring Initializr (at **start.spring.io**) is a web service that generates a ready-to-build Spring Boot project as a zip. IntelliJ IDEA (an IDE — Integrated Development Environment, i.e. a code editor with build, run and debug tools), VS Code (Spring Boot Extension Pack) and Eclipse (Spring Tools) all call the same service from their "New project" wizards.

How it works

1. Open start.spring.io.
2. **Project**: *Gradle - Groovy* (the site's default), *Gradle - Kotlin*, or *Maven*. These are **build tools**: programs that download your dependencies (libraries your code uses), compile, test and package the project. This book uses Maven.
3. **Language**: Java (Kotlin and Groovy are also offered).
4. **Spring Boot**: pick the latest GA version without a suffix — **4.1.1** at the time of writing. Avoid `SNAPSHOT` and `M`/`RC` versions.
5. **Project metadata**: *Group* (`com.example`, usually your reversed domain), *Artifact* (`hello-boot`, also the jar name), *Package name* (`com.example.hello` — your main class goes here), *Packaging* (`Jar` — use `War` only for deploying to an external server), *Java* (17, 21, 25 or 27; this book uses 25).
6. **Dependencies**: search and add. For the first app: *Spring Web* (id `web`) and *Spring Boot DevTools* (id `devtools`).
7. Click **Generate**, unzip, and open the folder in your IDE.

You can do the same from a terminal (the service has a small HTTP API) with `curl`, a command-line program that sends HTTP requests:

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d bootVersion=4.1.1 -d javaVersion=25 \
  -d groupId=com.example -d artifactId=hello-boot -d packageName=com.example.hello \
  -d dependencies=web,devtools -o hello-boot.zip
```

For Boot 4.1.1, choosing *Spring Web* produces `spring-boot-starter-webmvc` and a matching test starter `spring-boot-starter-webmvc-test` — exactly the `pom.xml` shown in §1.6 (checked against the service's generated `pom.xml`).

Common mistakes

- Choosing a Java version newer than the JDK installed on your machine (the build fails with `release version 25 not supported`). Check with `java -version`, see Java §1.4.

- Adding every dependency "just in case". Each starter can switch on auto-configuration — e.g. adding *Spring Data JPA* without a database makes startup fail with "Failed to configure a DataSource". Add what you use.
- Putting classes in a package *outside* the main class's package (see §1.8).

1.6 Project layout and the build file

The layout

```
hello-boot/
├── pom.xml                          ← build file (Maven)
├── mvnw, mvnw.cmd, .mvn/            ← Maven Wrapper: builds without installing Maven (generated by Initializr)
└── src/
    ├── main/
    │   ├── java/com/example/hello/
    │   │   ├── HelloApplication.java   ← the main class (§1.8)
    │   │   ├── HelloController.java    ← REST endpoint (§1.11)
    │   │   ├── GreetingService.java
    │   │   └── Greeting.java
    │   └── resources/
    │       ├── application.properties  ← configuration (Chapter 3)
    │       ├── static/                 ← optional: files served as-is (HTML, CSS, JS)
    │       └── templates/              ← optional: server-side templates (Thymeleaf)
    └── test/java/com/example/hello/
        ├── HelloControllerTest.java
        └── HelloApplicationTests.java
```

This is the standard Maven layout (`src/main/java`, `src/main/resources`, `src/test/java`) — explained in Java §18.1–18.2. Everything under `src/main/resources` ends up on the **classpath** (the list of places the JVM loads classes and files from), which is why Boot can find `application.properties` there.

The `pom.xml`

```xml
<!-- code/hello-boot/pom.xml -->
<parent>                                            <!-- 1 -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
  <relativePath/>
</parent>

<groupId>com.example</groupId>
<artifactId>hello-boot</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
  <java.version>25</java.version>                   <!-- 2 -->
</properties>

<dependencies>
  <dependency>                                      <!-- 3 -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>                                      <!-- 4 -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>                                      <!-- 5 -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>                                        <!-- 6 -->
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

1. `spring-boot-starter-parent` — a parent POM (Project Object Model, Maven's name for a `pom.xml`; a child POM inherits settings from its parent) that (a) imports the dependency BOM (**Bill of Materials**: a POM that lists tested versions for hundreds of libraries), (b) sets UTF-8 encoding, (c) compiles with `-parameters` (so Spring can read constructor parameter names), and

(d) pre-configures plugins (add-ons that give Maven extra build steps: surefire for running tests, resources filtering, the Boot plugin).

2. `java.version` — the parent turns this into `maven.compiler.release`, so `javac --release 25` is used.
3. **The web starter** — no `<version>`: the parent manages it.
4. **DevTools** — `runtime` scope + `optional` keeps it out of other projects that depend on yours; the Boot plugin also leaves it out of the packaged jar by default.
5. **The test starter** — only on the test classpath.
6. `spring-boot-maven-plugin` — adds goals (named Maven tasks) such as `spring-boot:run`, and during `package` it **repackages** the normal jar (a zip of compiled classes) into an executable "fat jar" — one jar that also contains all its dependency jars (§1.9).

If you cannot use the parent (your company has its own parent POM), import the BOM instead:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>4.1.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

You then configure the compiler (`-parameters`) and the Boot plugin's `repackage` goal yourself.

Gradle equivalent (what Initializr generates):

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}
java { toolchain { languageVersion = JavaLanguageVersion.of(25) } }
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') { useJUnitPlatform() }
```

Commands: `./gradlew bootRun`, `./gradlew bootJar`, `./gradlew test`.

Self-check

1. Why do the dependencies have no `<version>`? → The parent imports Boot's BOM, which manages them.
2. What does `spring-boot-maven-plugin` do during `mvn package`? → Repackages the jar into an executable fat jar with all dependencies and a launcher.
3. Which Maven folder should `application.properties` go in? → `src/main/resources`.

1.7 Starters

What it is

A **starter** is a dependency that contains (almost) no code; it exists only to pull in a curated, version-aligned set of libraries for one capability. Adding `spring-boot-starter-webmvc` gives you Spring MVC, an embedded Tomcat, Jackson for JSON, logging, and the Boot auto-configuration that wires them together.

How it works

This is the real runtime dependency tree of `hello-boot` (`mvn dependency:tree -Dscope=runtime`):

```
com.example:hello-boot:jar:0.0.1-SNAPSHOT
+- org.springframework.boot:spring-boot-starter-webmvc:jar:4.1.1:compile
|  +- org.springframework.boot:spring-boot-starter:jar:4.1.1:compile
|  |  +- org.springframework.boot:spring-boot-starter-logging:jar:4.1.1:compile
|  |  |  +- ch.qos.logback:logback-classic:jar:1.5.38:compile
|  |  |  +- org.apache.logging.log4j:log4j-to-slf4j:jar:2.25.5:compile
|  |  |  \- org.slf4j:jul-to-slf4j:jar:2.0.18:compile
|  |  +- jakarta.annotation:jakarta.annotation-api:jar:3.0.0:compile
|  |  \- org.yaml:snakeyaml:jar:2.6:compile
|  +- org.springframework.boot:spring-boot-starter-jackson:jar:4.1.1:compile
|  |  \- org.springframework.boot:spring-boot-jackson:jar:4.1.1:compile
|  |     \- tools.jackson.core:jackson-databind:jar:3.1.5:compile
|  +- org.springframework.boot:spring-boot-starter-tomcat:jar:4.1.1:compile
|  |  +- org.springframework.boot:spring-boot-starter-tomcat-runtime:jar:4.1.1:compile
|  |  |  +- org.apache.tomcat.embed:tomcat-embed-core:jar:11.0.24:compile
|  |  |  ...
|  |  \- org.springframework.boot:spring-boot-tomcat:jar:4.1.1:compile
|  +- org.springframework.boot:spring-boot-http-converter:jar:4.1.1:compile
|  |  \- org.springframework:spring-web:jar:7.0.9:compile
|  \- org.springframework.boot:spring-boot-webmvc:jar:4.1.1:compile
|     +- org.springframework.boot:spring-boot-servlet:jar:4.1.1:compile
|     \- org.springframework:spring-webmvc:jar:7.0.9:compile
\- org.springframework.boot:spring-boot-devtools:jar:4.1.1:runtime (optional)
   +- org.springframework.boot:spring-boot:jar:4.1.1:compile
   |  +- org.springframework:spring-core:jar:7.0.9:compile
   |  |  +- commons-logging:commons-logging:jar:1.3.6:compile
   |  |  \- org.jspecify:jspecify:jar:1.0.1:compile
   |  \- org.springframework:spring-context:jar:7.0.9:compile
   \- org.springframework.boot:spring-boot-autoconfigure:jar:4.1.1:compile
```

Notice the Boot 4 pattern: each technology has a **Boot module** holding its auto-configuration (`spring-boot-webmvc`, `spring-boot-tomcat`, `spring-boot-jackson`) and a **starter** that combines that module with the library itself.

Boot 4 modular starters

Boot 3 shipped all auto-configuration in one big `spring-boot-autoconfigure` jar. **Boot 4 split it into focused modules**, one per technology, and renamed some starters to match. In Boot 4.1.1 the core `spring-boot-autoconfigure` jar lists only 12 auto-configurations; web MVC's live in `spring-boot-webmvc`, Tomcat's in `spring-boot-tomcat`, and so on (Chapter 3 shows this).

Capability	Boot 4 starter	Boot 3 name (if different)
Servlet web / REST (Spring MVC + Tomcat)	`spring-boot-starter-webmvc`	`spring-boot-starter-web`
Reactive web (WebFlux + Netty)	`spring-boot-starter-webflux`	
Blocking HTTP client (`RestClient`)	`spring-boot-starter-restclient`	part of `-web`
Reactive HTTP client (`WebClient`)	`spring-boot-starter-webclient`	part of `-webflux`
JSON (Jackson 3)	`spring-boot-starter-jackson`	`spring-boot-starter-json` (still available in 4.x)
Bean Validation	`spring-boot-starter-validation`	
JDBC / JPA	`spring-boot-starter-jdbc` / `spring-boot-starter-data-jpa`	
Security	`spring-boot-starter-security`	
OAuth2 resource server	`spring-boot-starter-security-oauth2-resource-server`	`spring-boot-starter-oauth2-resource-server`
AspectJ-based AOP	`spring-boot-starter-aspectj`	`spring-boot-starter-aop`
Kafka	`spring-boot-starter-kafka`	(previously just `spring-kafka`)
Actuator	`spring-boot-starter-actuator`	
Core only (no web)	`spring-boot-starter`	
Testing, generic	`spring-boot-starter-test`	
Testing, per technology	`spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-security-test`, ...	— (new in Boot 4)

Rules of thumb for Boot 4:

- **Every technology starter has a `-test` companion** that brings the test slice for it (`@WebMvcTest` lives in `spring-boot-webmvc-test`). Each test starter already includes `spring-boot-starter-test`.
- Some old names still exist in 4.1 as **deprecated** aliases (`spring-boot-starter-web`, `spring-boot-starter-oauth2-resource-server`) so upgrades don't break immediately; others are gone (`spring-boot-starter-aop` → use `spring-boot-starter-aspectj`). Prefer the new names.

- For a quick upgrade from Boot 3, `spring-boot-starter-classic` and `spring-boot-starter-test-classic` bring the old "everything on the classpath" behaviour while you fix imports.
- Third-party starters are named `<name>-spring-boot-starter` (e.g. `mybatis-spring-boot-starter`); only official ones start with `spring-boot-starter-` (Chapter 3 builds one).

Common mistakes

- Adding both `spring-boot-starter-webmvc` and `spring-boot-starter-webflux` and being surprised the app is a *servlet* app. When both are present, Boot chooses Spring MVC.
- Using `spring-boot-starter-test` alone and then finding `@WebMvcTest` missing in Boot 4 — add `spring-boot-starter-webmvc-test`.
- Excluding a transitive library (one you get indirectly, because a dependency of yours depends on it) by hand to "slim down" and breaking auto-configuration. Use the starter meant for the job (e.g. exclude `spring-boot-starter-tomcat` and add `spring-boot-starter-jetty` to switch servers).

Self-check

1. What code does a starter contain? → Essentially none; it's a curated dependency list.
2. What is the Boot 4 name of the old `spring-boot-starter-web`? → `spring-boot-starter-webmvc`.
3. Where does `@WebMvcTest` come from in Boot 4? → `spring-boot-webmvc-test`, via `spring-boot-starter-webmvc-test`.

TL;DR

Starters = one dependency per capability. Boot 4 split auto-configuration into per-technology modules, renamed a few starters (`web` → `webmvc`, `aop` → `aspectj`) and added a `-test` starter for each technology.

1.8 The main class and `@SpringBootApplication`

```java
// code/hello-boot/src/main/java/com/example/hello/HelloApplication.java
package com.example.hello;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication            // = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan
public class HelloApplication {

    public static void main(String[] args) {
        SpringApplication.run(HelloApplication.class, args);   // creates and starts the ApplicationContext
    }
}
```

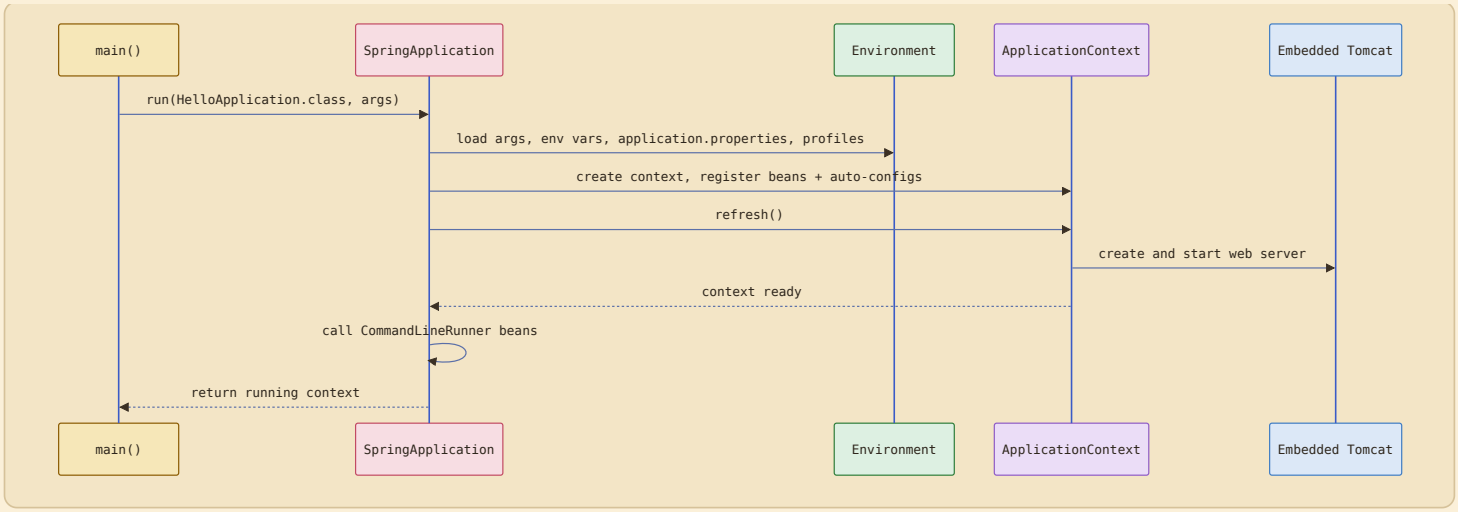
What `@SpringBootApplication` means

It is a shortcut for three annotations:

Annotation	What it does
`@SpringBootConfiguration`	Marks this class as a configuration source (a specialised `@Configuration`). Tests use it to find your app's configuration.
`@EnableAutoConfiguration`	Turns on auto-configuration: Boot loads candidate configuration classes and applies those whose conditions match (Chapter 3).
`@ComponentScan`	Scans **this class's package and all sub-packages** for `@Component`, `@Service`, `@Repository`, `@Controller`, `@RestController`, `@Configuration` classes and registers them as beans.

What `SpringApplication.run` does

1. Decides the application type from the classpath: **servlet** (Spring MVC present), **reactive** (only WebFlux present) or **none**.
2. Prepares the `Environment`: reads command-line arguments, environment variables, `application.properties`/`.yaml`, and activates profiles (Chapter 3).
3. Prints the banner.
4. Creates the right `ApplicationContext` (the IoC container; for a servlet app a web-server-aware one).
5. Registers bean definitions: your scanned classes plus auto-configurations whose conditions pass.
6. **Refreshes** the context: creates all singleton beans (beans with exactly one shared instance — the default), and for a web app starts the embedded Tomcat.
7. Runs any `CommandLineRunner`/`ApplicationRunner` beans (code you want to run once at startup).
8. Returns the running context; the JVM keeps running while Tomcat's threads are alive.



Where to put the main class

Put it in the **root package** of your code (`com.example.hello`) and every other class in that package or below (`com.example.hello.orders`, `com.example.hello.web`...). Component scanning starts from the main class's package, so a class in `com.example.other` is silently **not** a bean.

Common mistakes

- Main class in a sub-package (`com.example.hello.app`) while controllers live in `com.example.hello.web` → endpoints return 404 (the HTTP status code for "not found") because the controllers were never scanned.
- Placing the main class in the *default* (unnamed) package — Boot warns and scanning every jar on the classpath breaks startup.
- Writing logic in `main` after `run(...)` expecting it to run "before the server starts". Use a `CommandLineRunner` or an event listener (Chapter 2).

In the interview

Q: What does `@SpringBootApplication` do? It combines `@SpringBootConfiguration` (this class is configuration), `@EnableAutoConfiguration` (apply Boot's conditional auto-configurations) and `@ComponentScan` (register annotated classes from this package downwards).

- *Follow-up: How would you scan a package outside the main package?* Move the code under the root package (best), or use `@SpringBootApplication(scanBasePackages = "...")` / `@Import` for specific classes.
- *Follow-up: How do you exclude one auto-configuration?* `@SpringBootApplication(exclude = SomeAutoConfiguration.class)` or the property `spring.autoconfigure.exclude`.

TL;DR

`@SpringBootApplication` = configuration + auto-configuration + component scan from this package down. `SpringApplication.run` builds the environment, creates the container, registers and creates beans, and starts the embedded server.

1.9 Running the application

Three ways to run

Way	Command	When
IDE	Run `HelloApplication.main`	Daily development, debugging with breakpoints
Maven plugin	`./mvnw spring-boot:run` (or `mvn spring-boot:run`)	Terminal development; picks up `src/main/resources` directly
Executable jar	`./mvnw package` then `java -jar target/hello-boot-0.0.1-SNAPSHOT.jar`	Production, containers, CI (continuous integration: a server that builds and tests every change)

Pass settings on the command line in either form:

```
mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=9090"
java -jar target/hello-boot-0.0.1-SNAPSHOT.jar --server.port=9090 --spring.profiles.active=dev
```

The startup log, line by line

Running the packaged jar with JDK 25 (`java -jar target/hello-boot-0.0.1-SNAPSHOT.jar --server.port=18081`):

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.1.1)

INFO 91361 --- [hello-boot] [main] c.e.hello.HelloApplication : Starting HelloApplication v0.0.1-SNAPSHOT using Java 25.0.2 with PID 91361
...
INFO 91361 --- [hello-boot] [main] c.e.hello.HelloApplication : No active profile set, falling back to 1 default profile: "default"
INFO 91361 --- [hello-boot] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 18081 (http)
INFO 91361 --- [hello-boot] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.24]
INFO 91361 --- [hello-boot] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 376 ms
INFO 91361 --- [hello-boot] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 18081 (http) with context path '/'
INFO 91361 --- [hello-boot] [main] c.e.hello.HelloApplication : Started HelloApplication in 0.728 seconds (process running for 0.929)
```

(Timestamps removed and logger names shortened for width.)

Log part	Meaning
`91361`	Process ID
`[hello-boot]`	`spring.application.name` from `application.properties`
`[main]`	Thread name (a thread is one independent path of execution inside the process)
`No active profile set...`	No profile chosen, so only `application.properties` applies (profiles: Chapter 3)
`Tomcat initialized/started on port ...`	The embedded server is listening
`Started HelloApplication in 0.728 seconds`	The context is fully refreshed and ready

When the first request arrives you also see `Initializing Spring DispatcherServlet 'dispatcherServlet'` — the front controller (one object that receives every request and routes it) of Spring MVC is created lazily (only when first needed) on the first request (Chapter 5). Pressing Ctrl+C (or `kill <pid>`) shows:

```
INFO ... [ionShutdownHook] o.s.boot.tomcat.GracefulShutdown : Commencing graceful shutdown. Waiting for active requests to complete
INFO ... [tomcat-shutdown] o.s.boot.tomcat.GracefulShutdown : Graceful shutdown complete
```

Graceful shutdown is on by default: the server stops accepting new requests and lets in-flight ones (already being processed) finish (up to `spring.lifecycle.timeout-per-shutdown-phase`, default 30 s) before the context closes (Chapter 15).

Inside the executable jar

`mvn package` produced a 19.9 MB jar. Its layout (from `unzip -l`) and manifest (`MANIFEST.MF`, a small text file inside a jar that tells Java which class to start):

```
META-INF/MANIFEST.MF
org/springframework/boot/loader/...        ← Boot's launcher classes
BOOT-INF/classes/com/example/hello/*.class ← your code
BOOT-INF/classes/application.properties
BOOT-INF/lib/*.jar                         ← 34 dependency jars, stored unchanged
BOOT-INF/classpath.idx, BOOT-INF/layers.idx
```

```
Main-Class: org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.example.hello.HelloApplication
Spring-Boot-Version: 4.1.1
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
```

A plain JVM cannot load classes from jars *nested* inside a jar. So `java -jar` starts Boot's **`JarLauncher`** (the `Main-Class`), which sets up a class loader (the JVM component that finds and loads `.class` files) able to read `BOOT-INF/lib/*.jar`, then calls your **`Start-Class`**. `layers.idx` groups files into layers (dependencies change rarely, your classes often) so Docker images (packaged, runnable snapshots of an app plus its OS files, run as **containers**) can cache them — see Chapter 15. General jar packaging is covered in Java §18.12.

Common mistakes

- `Web server failed to start. Port 8080 was already in use.` Another process (often a previous run) holds the port. Stop it or set `server.port`. `server.port=0` picks a random free port.
- Running `java -jar` on the *original* jar (`*.jar.original`) — it has no launcher and fails with "no main manifest attribute".
- Expecting DevTools to work with `java -jar`: packaged apps disable it (§1.10).

Self-check

1. Which class is the manifest's `Main-Class` in a Boot fat jar? → `org.springframework.boot.loader.launch.JarLauncher`; your class is `Start-Class`.

2. How do you change the port without editing files? → `--server.port=9090` on the command line (or env var `SERVER_PORT`).
3. What does graceful shutdown do? → Stops accepting requests, waits for active ones to finish, then closes the context.

1.10 DevTools

What it is

Spring Boot DevTools (`spring-boot-devtools`) is a development-only module that makes the edit–run loop faster:

Feature	What it does
Automatic restart	When compiled classes change on the classpath, the application restarts in about a second.
Development property defaults	Turns off caches that get in the way while developing (template caching, static-resource caching), enables the H2 console (web UI for H2, a small in-memory database), etc.
Global settings	Personal settings in `~/.config/spring-boot/spring-boot-devtools.properties` apply to all your apps.
Remote DevTools	Opt-in restart of a remote app over HTTP; a security risk — only on trusted networks.
LiveReload	Browser auto-refresh; **disabled by default since Boot 4.0 and deprecated in 4.1** (no replacement).

How restart works

1. DevTools uses **two class loaders**: a *base* loader for the jars that don't change (Spring, Tomcat, Jackson...) and a *restart* loader for your own classes.
2. It watches your compiled output folders (e.g. `target/classes`).
3. When something changes there — because your IDE compiled on save (or IntelliJ's *Build Project*), or you ran `mvn compile` — DevTools throws away only the restart class loader and starts the context again.
4. Because the heavy base loader stays warm, a restart is much faster than a cold start.

Changes under `static/`, `public/`, `templates/` and `META-INF/resources` do **not** trigger a restart (they don't need one). Tune with `spring.devtools.restart.exclude` / `additional-exclude`, or disable with `spring.devtools.restart.enabled=false`.

When it is off

DevTools considers an app "in production" — and disables itself — when it is launched with `java -jar` from a packaged jar or from a special class loader. It is also turned off in tests: the test log of this chapter's project shows `Restart disabled due to context in which it is running`.

Common mistakes

- Expecting restarts after *editing* a `.java` file without compiling — DevTools watches compiled classes. Enable "build project automatically" in your IDE.
- Shipping DevTools to production by removing `<optional>true</optional>` or configuring the plugin to include it.
- Confusing restart with hot-swapping (replacing a class's code inside the running JVM without restarting): restart re-creates the whole context, so in-memory state (sessions, caches) is lost. For true class reloading, tools like JRebel exist.

TL;DR

DevTools restarts the app quickly when compiled classes change (two class loaders), applies developer-friendly defaults, and switches itself off for packaged jars and tests.

1.11 Your first REST endpoint

What we build

`GET /hello?name=Ada` → `{"message":"Hello, Ada!","length":11}`. Three small classes show the typical layering: a **controller** (HTTP), a **service** (logic) and a **record** (the JSON shape).

```
// Greeting.java — the response body
public record Greeting(String message, int length) {}
```

A **record** is a compact immutable data class (Java 16 — see Java §6.8). Jackson 3 (the JSON library) serializes (converts an object into text or bytes for sending) its components as JSON fields.

```java
// GreetingService.java
@Service                                    // a bean: Spring creates one instance and manages it
public class GreetingService {
    public Greeting greet(String name) {
        String message = "Hello, " + name + "!";
        return new Greeting(message, message.length());
    }
}
```

```java
// HelloController.java
@RestController                             // a bean whose method results are written to the HTTP response body
public class HelloController {

    private final GreetingService service;

    HelloController(GreetingService service) { // constructor injection: Spring passes the GreetingService bean
        this.service = service;
    }

    @GetMapping("/hello")                   // handles GET /hello and GET /hello?name=Ada
    Greeting hello(@RequestParam(defaultValue = "World") String name) {
        return service.greet(name);
    }
}
```

What each piece does:

Piece	Role
`@RestController`	`@Controller` + `@ResponseBody`: the return value becomes the response body (converted to JSON), not a view name.
Constructor parameter	Spring sees `HelloController` needs a `GreetingService` and passes the singleton bean. No `@Autowired` needed with a single constructor (Chapter 2).
`@GetMapping("/hello")`	Maps HTTP GET requests for path `/hello` to this method.
`@RequestParam(defaultValue = "World")`	Reads the query parameter `name` (the `?name=Ada` part of the URL); if absent, uses `"World"`.
Return type `Greeting`	Jackson (auto-configured by Boot) writes it as JSON with `Content-Type: application/json` (the HTTP header naming the body's format).

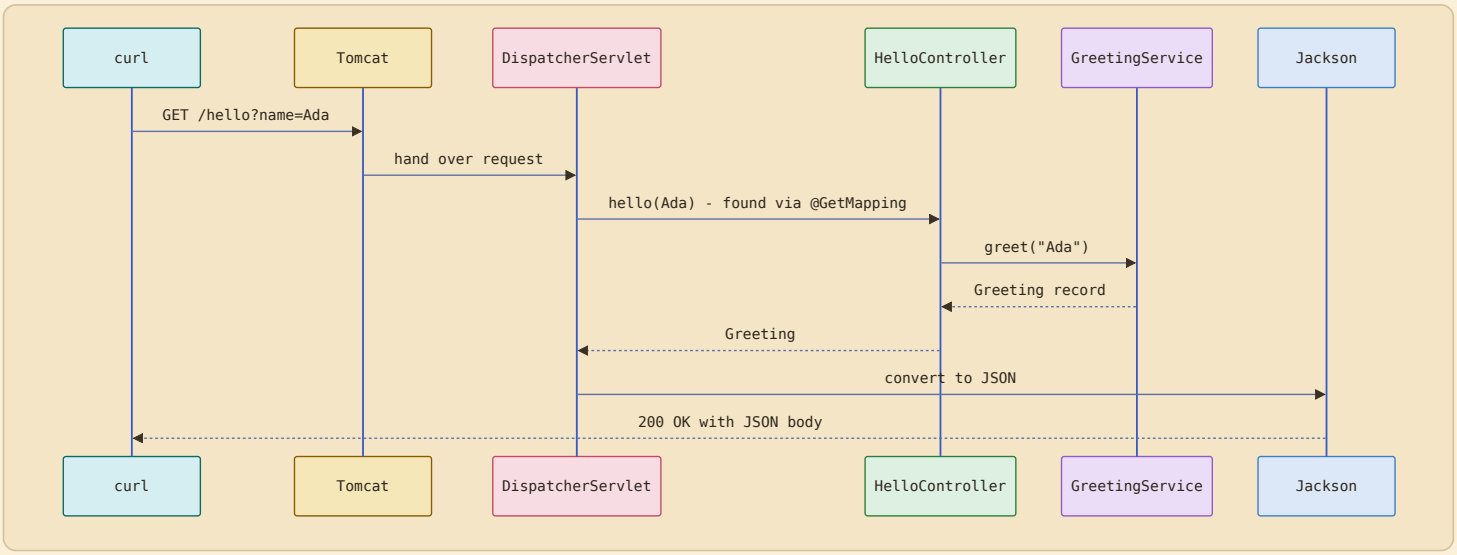
Calling it

With the jar running on port 18081:

```
$ curl -s 'localhost:18081/hello?name=Ada'
{"message":"Hello, Ada!","length":11}
$ curl -s localhost:18081/hello
{"message":"Hello, World!","length":13}
$ curl -s -i localhost:18081/hello | head -3
HTTP/1.1 200
Content-Type: application/json
Content-Length: 39
$ curl -s localhost:18081/nope
{"timestamp":"2026-09-28T23:24:50.396Z","status":404,"error":"Not Found","path":"/nope"}
```

The last response comes from Boot's default error handling (`/error` endpoint). Chapter 5 replaces it with RFC 9457 `ProblemDetail` responses (an RFC is a published internet standard; RFC 9457 defines a JSON shape for error responses).

What happened during a request



Common mistakes

- Using `@Controller` instead of `@RestController` for a JSON API → Spring treats `"Hello"` as a *view name* and fails with a template error.
- Putting business logic in the controller. Keep controllers thin (parse input, call a service, return output) so logic is testable without HTTP.
- Returning mutable entities straight from the database — prefer small response records (Chapter 8 shows why).

1.12 Testing the first app

Spring Boot tests are ordinary JUnit tests (JUnit is Java's standard unit-testing framework) with extra annotations that start (part of) the Spring container. Two styles cover most needs; Chapter 12 goes much deeper. JUnit basics are in Java §18.4.

Test 1 — a web *slice* with `MockMvcTester`

```
// HelloControllerTest.java
@WebMvcTest(HelloController.class)        // only the web layer; no real server
@Import(GreetingService.class)             // the service is not part of the web slice, so add it
class HelloControllerTest {

    @Autowired
    MockMvcTester mvc;                     // AssertJ-flavoured MockMvc (Framework 6.2+)

    @Test
    void greetsWorldByDefault() {
        var result = mvc.get().uri("/hello").exchange();
        System.out.println("GET /hello          -> " + result.getResponse().getStatus() + " " + contentOf(result));
        assertThat(result).hasStatusOk()
                .bodyJson().extractingPath("$.message").isEqualTo("Hello, World!");
    }

    @Test
    void greetsByName() {
        var result = mvc.get().uri("/hello?name=Ada").exchange();
        System.out.println("GET /hello?name=Ada -> " + result.getResponse().getStatus() + " " + contentOf(result));
        assertThat(result).hasStatusOk()
                .bodyJson().extractingPath("$.length").isEqualTo(11);
    }
    // contentOf(...) reads the response body as a String
}
```

- `@WebMvcTest` starts a *slice*: only MVC infrastructure, controllers, JSON and filters — no services, repositories or web server. That is why `GreetingService` is added with `@Import` (the more common alternative is mocking it — replacing it with a fake whose answers the test scripts — with `@MockitoBean`, Chapter 12).
- `MockMvcTester` sends requests straight to the `DispatcherServlet` in memory and offers AssertJ assertions such as `hasStatusOk()` and `bodyJson().extractingPath(...)` (a JSONPath expression; `$` is the document root).

Test 2 — the whole app on a real port with `RestTestClient`

```java
// HelloApplicationTests.java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
class HelloApplicationTests {

    @LocalServerPort int port;
    @Autowired RestTestClient client;      // Framework 7 test client, pre-bound to the random port

    @Test
    void realHttpCall() {
        String body = client.get().uri("/hello?name=Spring")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType("application/json")
                .expectBody(String.class).returnResult().getResponseBody();
        System.out.println("Tomcat on random port " + (port > 0 ? "OK" : "?") + ", body = " + body);
        assertThat(body).isEqualTo("{\"message\":\"Hello, Spring!\",\"length\":14}");
    }
}
```

- `@SpringBootTest` starts the full application context, like `main` does. With `RANDOM_PORT` it also starts Tomcat on a free port, so the test makes real HTTP calls.
- `RestTestClient` (new in Spring Framework 7) is a fluent test client for blocking apps; `@AutoConfigureRestTestClient` (from the `spring-boot-resttestclient` module, included by the webmvc test starter) binds it to the running server.
- Test class names must end in `Test`, `Tests` or `TestCase` for Maven Surefire (the plugin that runs unit tests) to run them by default — a class named `*IT` (integration test) is only run by the Failsafe plugin, which this project does not configure.

Output

`mvn -q -B -ntp verify` (Spring banners and log lines removed):

```
Tomcat on random port OK, body = {"message":"Hello, Spring!","length":14}
GET /hello?name=Ada -> 200 {"message":"Hello, Ada!","length":11}
GET /hello          -> 200 {"message":"Hello, World!","length":13}
```

(JUnit does not guarantee the order of test methods, so the two `GET` lines can swap.) The build ends with exit code 0: all three tests passed.

Trade-offs

	`@WebMvcTest` + `MockMvcTester`	`@SpringBootTest(RANDOM_PORT)` + `RestTestClient`
Starts	Web layer only, no server	Whole app + real Tomcat
Speed	Fast	Slower
Catches	Mapping, JSON, validation, controller logic	Everything incl. filters, server config, real HTTP
Use for	Most controller tests	A few end-to-end "smoke" tests (quick checks that the whole app starts and basically works)

Self-check

1. Why did `HelloControllerTest` need `@Import(GreetingService.class)`? → `@WebMvcTest` does not scan `@Service` classes.
2. Which annotation starts the full context on a random port? → `@SpringBootTest(webEnvironment = RANDOM_PORT)`.
3. Why might a test named `HelloApplicationIT` never run with `mvn verify`? → Surefire only picks `*Test`/`*Tests`/`*TestCase` by default; `*IT` needs Failsafe.

1.13 In the interview

Q: Walk me through what happens when you run a Spring Boot jar. `java -jar` starts `JarLauncher` from the manifest; it builds a class loader for `BOOT-INF/lib` jars and calls the `Start-Class` main method. `SpringApplication.run` detects a servlet app, prepares the `Environment` (args, env vars, config files, profiles), creates the `ApplicationContext`, registers scanned beans and matching auto-configurations, refreshes the context — creating singletons and starting embedded Tomcat — then runs `CommandLineRunner`s.

- *Follow-up: Why an embedded server?* One self-contained artifact per service: same jar in dev, CI and production, easy to containerise, no server-version drift.
- *Follow-up: Can you still build a WAR?* Yes: packaging `war`, extend `SpringBootServletInitializer`, mark the server starter `provided`.

Q: What is a starter? A dependency with no code that brings a curated, version-aligned set of libraries for one capability, plus the Boot module holding that capability's auto-configuration. In Boot 4 they are modular: `spring-boot-starter-webmvc`, plus a `-test` companion per technology.

Q: What changed in Spring Boot 4 / Framework 7? Java 17 baseline (25 recommended), Jakarta EE 11 (Servlet 6.1, Tomcat 11), modularized Boot jars and renamed starters, Jackson 3 (`tools.jackson`), JUnit 6, JSpecify null-safety annotations, API versioning in MVC/WebFlux, HTTP service client auto-configuration, `RestTestClient`, core retry/concurrency-limit annotations, Undertow removed. Details in Chapter 17.

Q: `@WebMvcTest` vs `@SpringBootTest`? Slice vs whole app. `@WebMvcTest` loads only MVC components and is fast; `@SpringBootTest` loads everything and, with a web environment, can start a real server.

Chapter TL;DR

- **Spring Framework** = IoC container that creates and wires your objects (beans) + modules for web, data, transactions, messaging.
- **Spring Boot** = Spring + starters + managed versions (BOM) + auto-configuration + embedded server + externalized config + Actuator.
- **Current generation**: Framework 7 (Nov 2025) and Boot 4 (Nov 2025); this book uses **Boot 4.1.1 → Framework 7.0.9**, Jakarta EE 11, Tomcat 11, Jackson 3, JUnit 6, on **Java 25** (Java 17 minimum).
- Generate projects at **start.spring.io**; the `pom.xml` inherits `spring-boot-starter-parent`, has version-less starters and the `spring-boot-maven-plugin`.
- **Boot 4 starters are modular**: `spring-boot-starter-webmvc` (was `-web`), a `-test` starter per technology.
- `@SpringBootApplication` = `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` (from the main class's package down).
- Run with the IDE, `mvn spring-boot:run`, or `java -jar` on the fat jar (`JarLauncher` → your `Start-Class`). Graceful shutdown is on by default.
- **DevTools** restarts fast on recompilation and disables itself for packaged jars and tests.
- A REST endpoint is a `@RestController` method with `@GetMapping`; return a record and Jackson writes JSON.
- Test controllers with `@WebMvcTest` + `MockMvcTester`; smoke-test the whole app with `@SpringBootTest(RANDOM_PORT)` + `RestTestClient`.

Next: Chapter 2 — IoC and Dependency Injection opens up the container you just used.

Chapter 2 — IoC and Dependency Injection

Everything in Spring — web controllers (classes that handle HTTP requests), transactions (groups of database changes that succeed or fail together), security, Spring Data repositories (data-access objects Spring generates for you) — sits on one foundation: the **IoC container** that creates your objects and connects them. If you understand the container, the rest of Spring stops feeling like magic. This chapter opens it up: how beans are defined, how dependencies are found and injected, scopes, the full bean lifecycle, post-processors, circular dependencies, lazy beans, conditions and profiles, events, and two Framework 7 additions — JSpecify null-safety and programmatic bean registration.

What you'll learn

1. Inversion of Control and Dependency Injection, with and without Spring (§2.1).
2. `BeanFactory` vs `ApplicationContext` (§2.2).
3. Four ways to define beans: component scanning, `@Bean` methods (full vs lite `@Configuration`), `@Import`, `BeanRegistrar` (§2.3).
4. Constructor, setter and field injection — and why constructor wins (§2.4).
5. How Spring picks a bean: by type, `@Primary`, `@Fallback`, `@Qualifier`, collections, optional dependencies (§2.5).
6. Scopes: singleton, prototype, request, session (§2.6).
7. The lifecycle: `@PostConstruct`, `InitializingBean`, init/destroy methods, `SmartLifecycle` (§2.7).
8. `BeanPostProcessor` and `BeanFactoryPostProcessor` — the container's extension points (§2.8).
9. Circular dependencies — why they fail and how to fix them (§2.9).
10. Lazy initialization (§2.10), `@Conditional` and profiles (§2.11), application events (§2.12).
11. JSpecify null-safety (§2.13) and `BeanRegistrar` (§2.14), both new in Framework 7.

Runnable code: `code/ioc-container/` (plain Spring containers inside JUnit tests — JUnit is Java's standard testing framework — so each demo gets its own isolated context) and `code/web-scopes/` (request and session scopes). Both built with `mvn -q -B -ntp verify` on Spring Boot 4.1.1 / Framework 7.0.9 / JDK 25. Each demo writes to a small `Trace` log and the tests print it, so the outputs below show the *order* in which Spring did things.

2.1 Inversion of Control and Dependency Injection

What it is

- **Dependency**: an object another object needs to do its job. A `PriceCalculator` needs a `TaxPolicy`.
- **Inversion of Control (IoC)**: instead of your code creating and controlling its collaborators, an outside party (the framework) does it and calls your code.
- **Dependency Injection (DI)**: the specific form of IoC where an object receives its dependencies from outside — through its constructor, a setter or a field — instead of creating them with `new` or looking them up.
- **IoC container**: the Spring object that performs DI for a whole application. Objects it manages are **beans**.

Analogy: a film crew. An actor (your class) doesn't hire their co-stars; the casting director (container) reads the script (configuration), casts everyone, and puts them on set together.

The problem it solves

```java
// code/ioc-container/src/main/java/com/example/ioc/basics/PriceCalculator.java
public class PriceCalculator {
    private final TaxPolicy taxPolicy;

    public PriceCalculator(TaxPolicy taxPolicy) {       // the dependency is handed in, not created here
        this.taxPolicy = taxPolicy;
        Trace.add("PriceCalculator created with " + taxPolicy.getClass().getSimpleName());
    }

    public long totalCents(long netCents) { return netCents + taxPolicy.taxCents(netCents); }
}
```

`PriceCalculator` depends on the *interface* `TaxPolicy`, not on `FlatTaxPolicy`. That already gives you testability (pass a fake `TaxPolicy` in a **unit test** — a small automated test of one class in isolation) and flexibility (swap implementations without editing the class). DI is first a *design style*; the container just automates the wiring. Someone still has to call the constructors:

```java
// Without Spring — you wire by hand (fine for 3 classes, painful for 300):
PriceCalculator calc = new PriceCalculator(new FlatTaxPolicy());
```

```java
// With Spring — you describe the wiring once:
@Configuration
public class ShopConfig {
    @Bean TaxPolicy taxPolicy() { return new FlatTaxPolicy(); }

    @Bean PriceCalculator priceCalculator(TaxPolicy taxPolicy) {   // parameter = dependency
        return new PriceCalculator(taxPolicy);
    }
}
// ...and ask the container for the finished object:
try (var ctx = new AnnotationConfigApplicationContext(ShopConfig.class)) {
    PriceCalculator calc = ctx.getBean(PriceCalculator.class);
}
```

Why the container is worth it

Hand wiring	Spring container
You decide creation order	Spring works out the order from the dependency graph (the map of which object needs which)
One object per `new`; sharing needs care	Singletons shared automatically; other scopes available
No central place to add cross-cutting behaviour (concerns like logging or security that touch many classes)	Can wrap beans in proxies (stand-in objects that intercept calls before passing them on): transactions, security, caching (Chapter 4)
Configuration values passed by hand	Values injected from properties/environment (Chapter 3)
Start-up/shut-down hooks by hand	Lifecycle callbacks and graceful shutdown

When not to use it

Not every object should be a bean. **Beans are the long-lived "service" objects** — services, repositories, controllers, clients, configuration. Data objects (a `Greeting`, an `Order` entity — a class mapped to a database table — or a DTO, Data Transfer Object: a plain class that just carries data between layers) are created with `new` in your code, many times per request. Making them beans adds nothing.

The design ideas behind DI — the Dependency Inversion Principle and "program to an interface" — are covered in LLD §4.10 and LLD Chapter 5.

TL;DR

DI = objects receive dependencies instead of creating them. Spring's IoC container automates DI for the whole app and manages each bean's life.

2.2 The container: `BeanFactory` vs `ApplicationContext`

What it is

Spring has two container interfaces:

- `BeanFactory` — the basic container: holds **bean definitions** (recipes: class, scope, dependencies, init method...) and creates beans from them on request.
- `ApplicationContext` — extends `BeanFactory` and adds everything a real application needs. It is what you always use; Spring Boot creates one for you.

Feature	`BeanFactory`	`ApplicationContext`
Create and inject beans	Yes	Yes
Singletons created	**Lazily**, on first `getBean`	**Eagerly**, at startup (`refresh()`) — errors show up immediately
Registers `BeanPostProcessor`s automatically	No (manual)	Yes — so `@Autowired`, `@PostConstruct`, `@Transactional` work
Application events (`publishEvent`, `@EventListener`)	No	Yes
`Environment`, profiles, property sources	No	Yes
i18n (internationalization: showing text in the user's language) messages (`MessageSource`)	No	Yes
Resource loading (`classpath:`, `file:`)	No	Yes
Typical implementations	`DefaultListableBeanFactory`	`AnnotationConfigApplicationContext`; in Boot: servlet/reactive web-server contexts

Eager vs lazy — proved

```
// ContainerBasicsTest.java (excerpt)
try (var ctx = new AnnotationConfigApplicationContext(ShopConfig.class)) {
    Trace.add("-- context refreshed, now calling getBean");
    PriceCalculator calc = ctx.getBean(PriceCalculator.class);
    Trace.add("total for 1000 = " + calc.totalCents(1000));
    assertThat(ctx.getBean(PriceCalculator.class)).isSameAs(calc);   // singleton
}

var bf = new DefaultListableBeanFactory();
bf.registerBeanDefinition("taxPolicy", new RootBeanDefinition(FlatTaxPolicy.class));
var calcDef = new RootBeanDefinition(PriceCalculator.class);
calcDef.setAutowireMode(RootBeanDefinition.AUTOWIRE_CONSTRUCTOR);
bf.registerBeanDefinition("priceCalculator", calcDef);
Trace.add("-- definitions registered, nothing created yet");
bf.getBean(PriceCalculator.class);
```

Output:

```
--- ApplicationContext (eager) ---
 1. FlatTaxPolicy created
 2. PriceCalculator created with FlatTaxPolicy
 3. -- context refreshed, now calling getBean
 4. total for 1000 = 1200
--- Bare BeanFactory (lazy) ---
 1. -- definitions registered, nothing created yet
 2. FlatTaxPolicy created
 3. PriceCalculator created with FlatTaxPolicy
```

Notice three things: the context created both beans *before* anyone asked; Spring created `FlatTaxPolicy` first because `PriceCalculator` needs it; and `getBean` twice returned the same instance (the default **singleton** scope).

How startup works (`refresh()`)

1. **Load bean definitions** — from `@Configuration` classes, component scanning, `@Import`, registrars.
2. **Run `BeanFactoryPostProcessor`s** — they may read or change *definitions* (e.g. resolve `${...}` placeholders — markers replaced by a property value) before any bean exists (§2.8).
3. **Register `BeanPostProcessor`s** — objects that will get a chance to modify every bean.
4. **Instantiate all non-lazy singletons** — for each: construct, inject, run init callbacks, apply post-processors (§2.7).
5. **Finish** — start `SmartLifecycle` beans, publish `ContextRefreshedEvent`.

In Spring Boot, `SpringApplication.run` does all this (Chapter 1 §1.8).

Common mistakes

- Calling `ctx.getBean(...)` all over your code ("service locator" style). Use injection; `getBean` belongs in tests, `main` methods and framework code.
- Creating a second `ApplicationContext` inside a bean. You get a duplicate set of beans that are not the ones the rest of the app uses.

In the interview

Q: BeanFactory vs ApplicationContext? `BeanFactory` is the minimal container: definitions + lazy creation. `ApplicationContext` extends it with eager singleton creation, automatic post-processor registration (needed for annotations), events, the `Environment`/profiles, i18n and resource loading. Applications use `ApplicationContext`.

- *Follow-up: Why is eager creation good?* Configuration errors (missing bean, bad property) fail at startup instead of on the first request in production.

TL;DR

Use `ApplicationContext`. It creates singletons eagerly in dependency order, and it's what makes annotations, events and profiles work.

2.3 Defining beans

There are four ways to tell the container about beans. Most apps use the first two.

1. Component scanning with stereotype annotations

Stereotype annotations are labels that say what *role* a class plays (service, repository, controller). Annotate the class; Spring finds it while scanning packages (`@ComponentScan`, included in `@SpringBootApplication`).

Annotation	Meaning	Extra behaviour
`@Component`	Generic bean	—
`@Service`	Business logic	None (documents intent)
`@Repository`	Data access	Persistence exceptions (errors thrown by database libraries) translated to Spring's `DataAccessException` (when a translation post-processor is present — Boot/Spring Data set it up)
`@Controller`	Spring MVC controller (returns views)	Detected as a request handler
`@RestController`	`@Controller` + `@ResponseBody`	Return values written as the body
`@Configuration`	Holds `@Bean` methods	Proxied in full mode (below)

All of them are meta-annotated with `@Component` (their own declaration carries `@Component`, so they count as one); that's why scanning finds them.

```
@Repository public class InvoiceRepository {}

@Service public class InvoiceService {
    private final InvoiceRepository repository;
    public InvoiceService(InvoiceRepository repository) { this.repository = repository; }
}

@Component("renderer") public class PdfRenderer {}   // explicit bean name

public class NotABean {}                              // no annotation -> ignored
```

```
scanned beans: invoiceRepository, invoiceService, renderer
```

Bean names. The default is the simple class name with a lower-case first letter (`InvoiceService` → `invoiceService`); `@Component("renderer")` overrides it. For a *nested* class Spring uses the outer class too: `Ambiguous.DiskStorage` becomes `ambiguous.DiskStorage` (you'll see this in §2.5's output).

2. `@Bean` methods

Write a method that builds the object; Spring calls it once (for a singleton) and registers the result. Use `@Bean` for **classes you can't annotate** (third-party libraries: `ObjectMapper` — Jackson's JSON converter, `DataSource` — a source of database connections, a cloud SDK — Software Development Kit, a vendor's client library — client) and when construction needs logic.

```
@Configuration
public class ShopConfig {
    @Bean TaxPolicy taxPolicy() { return new FlatTaxPolicy(); }                 // bean name = method name
    @Bean PriceCalculator priceCalculator(TaxPolicy taxPolicy) { return new PriceCalculator(taxPolicy); }
}
```

`@Bean` attributes: `name`/`value` (names and aliases), `initMethod`, `destroyMethod`, `autowireCandidate`, `defaultCandidate` (6.2). Combine with `@Scope`, `@Lazy`, `@Primary`, `@Profile`, `@Conditional...` on the same method.

Full vs lite `@Configuration` — proved

A `@Bean` method in a `@Configuration` class can call *another* `@Bean` method. What that call returns depends on the mode:

```
@Configuration                      // "full" mode: proxyBeanMethods = true (the default)
public class FullConfig {
    @Bean AuditLog auditLog() { return new AuditLog(); }
    @Bean OrderService orderService() { return new OrderService(auditLog()); }       // inter-bean call
    @Bean PaymentService paymentService() { return new PaymentService(auditLog()); } // inter-bean call
}

@Configuration(proxyBeanMethods = false)   // "lite" mode: no CGLIB subclass
public class LiteConfig { /* identical methods */ }

@Configuration(proxyBeanMethods = false)   // lite mode done right: ask for dependencies as parameters
public class LiteSafeConfig {
    @Bean AuditLog auditLog() { return new AuditLog(); }
    @Bean OrderService orderService(AuditLog log) { return new OrderService(log); }
    @Bean PaymentService paymentService(AuditLog log) { return new PaymentService(log); }
}
```

Output (`AuditLog` numbers each instance it creates):

```
full: config class = FullConfig$$SpringCGLIB$$0
full: order.log=#1 payment.log=#1, AuditLogs created=1
lite: config class = LiteConfig
lite: order.log=#2 payment.log=#3, AuditLogs created=3
lite+params: order.log=#1 payment.log=#1, AuditLogs created=1
```

How it works:

1. In **full mode**, Spring generates a **CGLIB subclass** of your configuration class at startup (`FullConfig$$SpringCGLIB$$0`). CGLIB is a bytecode library that creates subclasses at runtime (**bytecode** is the compiled form of Java that the JVM — Java Virtual Machine — executes).
2. The subclass overrides each `@Bean` method: "if the bean already exists in the container, return it; otherwise call the real method once". So `auditLog()` called from two methods still yields the one singleton.
3. In **lite mode** (`proxyBeanMethods = false`, or `@Bean` methods in a plain `@Component`), there is no subclass: `auditLog()` is an ordinary Java call and creates a new, *unmanaged* object each time — 3 `AuditLog`s, two of which Spring knows nothing about (no lifecycle callbacks, no proxies).
4. **Parameters** avoid the problem in both modes: Spring injects the container's bean.

	Full (`@Configuration`)	**Lite (`proxyBeanMethods = false` or `@Component`)**
Inter-bean method calls	Return the managed singleton	Create new unmanaged objects
Startup cost	Generates a CGLIB subclass	None — slightly faster, friendlier to AOT (ahead-of-time build processing)/native images (apps compiled to a stand-alone executable)
Class may be `final` / methods `private`?	No (must be subclassable)	Yes
Recommended style	Works either way if you use parameters	Spring Boot's own auto-configurations all use lite mode

Rule of thumb: declare dependencies as `@Bean` method parameters and never call `@Bean` methods directly — then the mode doesn't matter.

3. `@Import`

`@Import(SomeConfig.class)` pulls in another configuration class (or a plain class as a bean, or an `ImportSelector` / `ImportBeanDefinitionRegistrar` / `BeanRegistrar`) without component scanning. Auto-configuration and `@Enable...` annotations are built on it (Chapter 3).

4. Programmatic registration — `BeanRegistrar` (Framework 7)

For beans whose number or type depends on runtime logic (loops, ifs over properties). See §2.14.

Older projects also define beans in **XML** (`<bean id="..." class="...">`). It still works but new code uses annotations.

Common mistakes

- Annotating an interface with `@Service` — scanning registers only concrete classes. Annotate the implementation.
- Calling a `@Bean` method from a lite configuration and getting a second instance (shown above).
- Two beans with the same name (e.g. two `@Bean` methods named `dataSource` in different configs). Spring Boot disables bean-definition overriding by default, so startup fails with `BeanDefinitionOverrideException` — rename one.

In the interview

Q: `@Component` vs `@Bean`? `@Component` goes on *your* class and is found by scanning; `@Bean` goes on a factory *method* in a configuration class and is used for third-party classes or when creation needs code. Both produce beans that behave the same.

- *Follow-up: What does `proxyBeanMethods = false` change?* No CGLIB subclass; calls between `@Bean` methods become plain Java calls and create new objects. Safe when dependencies are passed as parameters, and cheaper at startup.
- *Follow-up: Difference between `@Service` and `@Component`?* None technically; `@Service` documents intent. `@Repository` adds exception translation.

TL;DR

Scan your own classes (`@Service`, `@Repository`...), use `@Bean` for others, pass dependencies as `@Bean` method parameters, and know that full `@Configuration` is a CGLIB subclass.

2.4 Injection types

What it is

Spring can inject a dependency in three places:

```java
// Constructor injection (preferred)
@Service
public class AlertService {
    private final Notifier notifier;
    public AlertService(Notifier notifier) { this.notifier = notifier; }   // no @Autowired needed
}

// Setter (method) injection and field injection
@Component
public class OtherStyles {
    @Autowired @Qualifier("push")
    Notifier fieldInjected;                  // field injection: needs reflection, can't be final

    Notifier setterInjected;

    @Autowired
    void setNotifier(@Qualifier("email") Notifier n) {   // setter/method injection
        this.setterInjected = n;
    }
}
```

`@Autowired` tells Spring "inject here". Since Spring 4.3, a class with **exactly one constructor** doesn't need it — Spring uses that constructor automatically. With several constructors, mark the one to use with `@Autowired`. The standard `jakarta.inject.Inject` annotation works the same way if `jakarta.inject-api` is on the classpath (Framework 7 dropped the old `javax.inject`).

Trade-offs

	Constructor	Setter	Field
Fields can be `final` (immutable)	Yes	No	No
Object is complete after `new`	Yes	No — may be half-initialised	No
Unit test without Spring	Just call the constructor	Call setters	Needs reflection (code inspecting/changing objects at runtime) or Spring
Required dependency obvious	Yes (it's in the signature)	No	No
Circular dependency possible	No (fails fast, §2.9)	Yes	Yes
Too many dependencies visible	Yes — a 9-parameter constructor is a design smell (a hint that the design is off) you can see	Hidden	Hidden
Good for	**Almost everything**	Optional dependencies, re-configuration	Test classes (`@Autowired MockMvcTester mvc`)

Common mistakes

- Field injection in production code — convenient but hides dependencies and makes plain unit tests awkward.
- Adding `@Autowired` to a single constructor "just in case" — harmless but noise.
- Using a dependency *inside the constructor* of a class that uses field injection → `NullPointerException` (the error thrown when you use a reference that is `null`), because fields are injected after construction.

In the interview

Q: Why is constructor injection recommended? Dependencies become `final`, the object is fully built when constructed, required dependencies are explicit, unit tests just call `new`, and circular dependencies fail fast instead of hiding. The Spring team recommends it.

- *Follow-up: When would you use setter injection?* For truly optional dependencies or values that can change after startup.

TL;DR

Use constructor injection with `final` fields; skip `@Autowired` on a single constructor; keep field injection for tests.

2.5 How Spring picks a bean

What it is

To inject a parameter of type `Notifier`, Spring looks for **candidate beans assignable to that type** (beans whose class is that type, implements it, or extends it). The rules when there are zero, one, or many candidates:

1. **Exactly one candidate** → inject it.
2. **Several candidates** → narrow down in this order:
    1. A `@Qualifier("name")` on the injection point selects the bean with that name (or that qualifier).
    2. Otherwise, a bean marked `@Primary` wins.
    3. Otherwise, if all but one are marked `@Fallback` (Framework 6.2), the remaining one wins.
    4. Otherwise, a bean whose **name equals the parameter name** wins (needs the `-parameters` compiler flag — an option that keeps parameter names in the compiled class — which Boot's parent sets).
    5. Otherwise → `NoUniqueBeanDefinitionException`. (`@jakarta.annotation.Priority` on the candidate classes is also consulted — highest priority wins — but it is rarely used in application code.)
3. **No candidate** → `NoSuchBeanDefinitionException` — unless the injection point is optional (`Optional<T>`, `ObjectProvider<T>`, `@Nullable`, `@Autowired(required = false)`).
4. **Collection/array/Map of the type** → inject *all* candidates: `List<Notifier>` ordered by `@Order`/`Ordered`, `Map<String, Notifier>` keyed by bean name.

Example

```
public final class Notifiers {
    @Component("sms") @Primary @Order(1)
    public static class SmsNotifier implements Notifier { public String send(String m) { return "SMS: " + m; } }

    @Component("email") @Order(2)
    public static class EmailNotifier implements Notifier { public String send(String m) { return "EMAIL: " + m; } }

    @Component("push") @Order(3)
    public static class PushNotifier implements Notifier { public String send(String m) { return "PUSH: " + m; } }
}

@Service
public class AlertService {
    public AlertService(Notifier defaultNotifier,                    // by type -> the @Primary one
                        @Qualifier("email") Notifier emailNotifier,  // by name
                        List<Notifier> all,                          // all, sorted by @Order
                        Map<String, Notifier> byName,                // bean name -> bean
                        Optional<AuditTrail> audit,                  // absent -> Optional.empty()
                        ObjectProvider<AuditTrail> auditProvider) {  // lazy, optional lookup
        ...
    }
}
```

Output:

```
by type (@Primary): SMS: hi
@Qualifier(email):  EMAIL: hi
List (by @Order):   [SMS: hi, EMAIL: hi, PUSH: hi]
Map keys:           [email, push, sms]
optional AuditTrail present? false
```

And the failure case — two `Storage` beans, no tie-breaker:

```
@Component public static class DiskStorage implements Storage {}
@Component public static class CloudStorage implements Storage {}
@Component public static class Backup { public Backup(Storage storage) {} }
```

```
startup failed: No qualifying bean of type 'com.example.ioc.ambiguous.Ambiguous$Storage' available:
expected single matching bean but found 2: ambiguous.CloudStorage,ambiguous.DiskStorage
```

The exception is `UnsatisfiedDependencyException` wrapping `NoUniqueBeanDefinitionException` — the context never starts.

The tie-breakers compared

Tool	Where it goes	Meaning	Use when
`@Primary`	On a bean	"Default choice for this type"	One implementation is the normal one
`@Fallback` (6.2+)	On a bean	"Use me only if nothing else qualifies"	A default that others should override
`@Qualifier("x")`	On the injection point (and optionally the bean)	"I want exactly this one"	A specific consumer needs a specific implementation
Custom qualifier annotation	e.g. `@Qualifier @interface Fast {}`	Type-safe qualifier	Avoid string names spread across code
`List<T>` / `Map<String,T>`	Injection point	"Give me all of them"	Strategy/plugin patterns, broadcast
`ObjectProvider<T>`	Injection point	Lazy/optional/multiple lookup: `getIfAvailable()`, `getIfUnique()`, `orderedStream()`	Optional beans, prototypes (§2.6)

`List<Notifier>` + `Map` is how you implement the **Strategy pattern** with Spring: each strategy is a bean, the consumer receives them all and picks by key — see LLD §11.3.

Common mistakes

- Adding a second implementation of an interface later (e.g. a test stub) and breaking every injection point that relied on "only one". Decide on `@Primary` or qualifiers early.
- Qualifier string typos — they fail at startup (good), but a custom qualifier annotation avoids them entirely.
- Expecting `@Order` to change which single bean is injected. It only orders collections; use `@Primary`/`@Qualifier` for single injection.

In the interview

Q: Two beans implement the same interface. How does Spring decide? By type first; with several candidates, `@Qualifier` on the injection point, then `@Primary`, then `@Fallback`, then a bean named like the parameter; otherwise startup fails with `NoUniqueBeanDefinitionException`. Or inject `List<T>`/`Map<String,T>` to get all.

- *Follow-up: How do you inject an optional dependency?* `Optional<T>`, `ObjectProvider<T>.getIfAvailable()`, a `@Nullable` parameter (§2.13), or `@Autowired(required = false)` on a setter.

Self-check

1. `@Primary` on `SmsNotifier`, `@Qualifier("email")` on a parameter — which is injected there? → `email`; the qualifier is more specific.
2. What order does `List<Notifier>` have? → `@Order`/`Ordered` values, lowest first; unordered beans come last.
3. What happens with zero candidates for a required dependency? → `NoSuchBeanDefinitionException` at startup.

TL;DR

Type first, then `@Qualifier` > `@Primary` > `@Fallback` > parameter name. Collections get everything. Optional via `Optional`/`ObjectProvider`/`@Nullable`.

2.6 Bean scopes

What it is

A **scope** decides *how many instances* of a bean exist and *how long* each lives.

Scope	Instances	Lives until	Available in
`singleton` (default)	One per container	Context closes	All apps
`prototype`	New one **per injection/lookup**	Garbage collected (freed automatically by the JVM once nothing references it) — Spring forgets it after handing it out	All apps
`request`	One per HTTP request	Request ends	Web apps
`session`	One per HTTP session (one user's series of requests, tied together by a cookie)	Session expires	Web apps
`application`	One per `ServletContext` (the web app as a whole, inside the server)	App stops	Web apps
`websocket`	One per WebSocket (a long-lived two-way connection between browser and server) session	Socket closes	WebSocket apps

Declare with `@Scope("prototype")` (or `ConfigurableBeanFactory.SCOPE_PROTOTYPE`), `@RequestScope`, `@SessionScope`, `@ApplicationScope`.

> **Spring singleton ≠ GoF Singleton.** (GoF = "Gang of Four", the authors of the classic *Design Patterns* book.) The GoF Singleton pattern enforces one instance *per class loader* via a private constructor. A Spring singleton is one instance *per container and bean definition* — you could define two beans of the same class. See LLD §8.8.

Singleton vs prototype — and the classic trap

```
@Component @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class ShoppingCart { public final int id = SEQ.incrementAndGet(); @PreDestroy void close() {...} }

@Component
public class PriceList { public final int id = SEQ.incrementAndGet(); @PreDestroy void close() {...} }

@Service                                          // singleton that depends on a prototype
public class CheckoutService {
    private final ShoppingCart injectedOnce;               // injected ONE time, at creation
    private final ObjectProvider<ShoppingCart> carts;       // asks the container each time
    public int cartIdFromField() { return injectedOnce.id; }
    public int cartIdFromProvider() { return carts.getObject().id; }
}
```

Output:

```
singleton: PriceList #1, #1
prototype: ShoppingCart #2, #3
prototype injected into singleton: #1, #1 (stuck!)
via ObjectProvider:                #4, #5 (fresh)
--- destroy callbacks on close ---
 1. PriceList#1 @PreDestroy
```

What this proves:

1. `getBean(PriceList.class)` always returns instance #1; `getBean(ShoppingCart.class)` returns a new one each time.
2. **A prototype injected into a singleton is created once** — when the singleton is built — and then reused forever. The singleton is created only once, so injection happens only once.
3. The fix is to *look up* a new instance when needed: `ObjectProvider<T>.getObject()` (shown), `@Lookup` method injection, or a scoped proxy.
4. **Spring does not call destroy callbacks on prototypes.** Only `PriceList`'s `@PreDestroy` ran. If a prototype holds resources (files, connections), the code that asked for it must clean up.

Request and session scopes — proved

A singleton controller can't hold "the current request's object" in a field... unless that field is a **scoped proxy**: a stand-in object that, on every method call, finds the real instance for the *current* request or session.

```java
@Component @RequestScope          // one instance per HTTP request; a scoped proxy is injected into singletons
public class RequestInfo { private final int id = SEQ.incrementAndGet(); public int id() { return id; } }

@Component @SessionScope          // one instance per HTTP session
public class Cart { private final List<String> items = new ArrayList<>(); ... }

@RestController                   // a singleton...
public class CartController {
    private final Cart cart;              // ...holding proxies that find the right
    private final RequestInfo request;    //    session/request instance on every call

    @PostMapping("/cart/{item}")
    String add(@PathVariable String item) {
        cart.add(item);
        return "request#" + request.id() + " cart=" + cart.items() + " proxy=" + cart.getClass().getSimpleName().contains("CGLIB");
    }
}
```

The test posts twice with Alice's session and once with Bob's (`MockHttpSession` objects):

```
alice: request#1 cart=[book] proxy=true
alice: request#2 cart=[book, pen] proxy=true
bob:   request#3 cart=[mug] proxy=true
```

Every request got a new `RequestInfo`; Alice's two requests shared one `Cart`, Bob got his own — through the same singleton controller. `@RequestScope` and `@SessionScope` create class-based (CGLIB) proxies by default (`proxyMode = TARGET_CLASS`); with plain `@Scope("request")` you must set `proxyMode` yourself.

Thread safety

A singleton bean is shared by **all request threads at once**. Keep singletons **stateless** (holding no data that changes between calls — only `final` references to other beans and configuration). Per-request data belongs in method parameters/local variables, or in request-scoped beans. Concurrency basics: Java Chapter 14.

Common mistakes

- A mutable field (`private int counter`, a `HashMap` cache) in a singleton service → race conditions under load.

  > **Key term — race condition:** a bug where the result depends on the exact timing of several threads touching the same data. Two threads both read `counter = 5`, both add 1, both write `6` — one increment is lost. It appears only under concurrent load, so it often slips through tests.

- Expecting a new prototype on each method call when it was injected once (shown above).
- Injecting a request-scoped bean into a singleton without a proxy → `Scope 'request' is not active for the current thread` at startup.
- Using session scope for data in a horizontally scaled app (one that runs several copies behind a load balancer) without session replication (copying session data so every copy can see it — Spring Session, Chapter 10; see System Design Fundamentals).

In the interview

Q: What are Spring's bean scopes? singleton (default, one per container), prototype (new per lookup, not destroyed by Spring), and web scopes request, session, application, websocket.

- *Follow-up: Prototype inside singleton?* Injected once, so effectively a singleton for that consumer. Use `ObjectProvider`, `@Lookup`, or a scoped proxy.
- *Follow-up: Are singletons thread-safe?* Spring doesn't make them thread-safe; they're shared, so design them stateless.

TL;DR

Singleton by default, shared by all threads — keep it stateless. Prototype = new per lookup, no destroy callback. Web scopes use proxies so singletons can reach the current request/session object.

2.7 The bean lifecycle

What it is

From creation to destruction Spring calls a fixed sequence of hooks. You rarely need more than `@PostConstruct` and `@PreDestroy`, but knowing the order explains many "why is my field null?" questions.

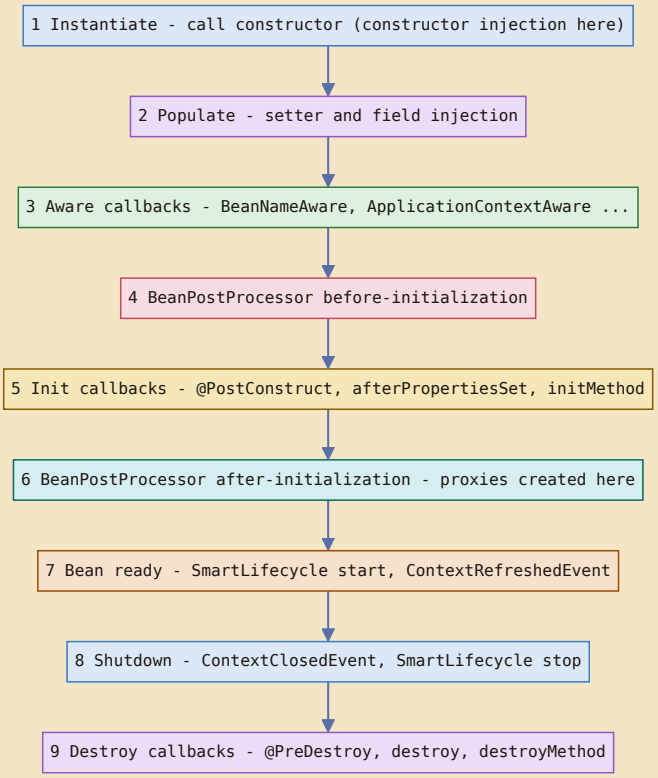
This bean implements *every* hook and records when it runs:

```java
public class CacheWarmer implements BeanNameAware, InitializingBean, DisposableBean {
    public CacheWarmer()                          { Trace.add("constructor"); }
    @Autowired void setDependency(Dependency d)   { Trace.add("setter injection"); }
    @Override public void setBeanName(String n)   { Trace.add("BeanNameAware.setBeanName(" + n + ")"); }
    @PostConstruct void postConstruct()           { Trace.add("@PostConstruct"); }
    @Override public void afterPropertiesSet()    { Trace.add("InitializingBean.afterPropertiesSet"); }
    void customInit()                             { Trace.add("@Bean(initMethod) customInit"); }

    @PreDestroy void preDestroy()                 { Trace.add("@PreDestroy"); }
    @Override public void destroy()               { Trace.add("DisposableBean.destroy"); }
    void customDestroy()                          { Trace.add("@Bean(destroyMethod) customDestroy"); }
}

@Configuration
public class LifecycleConfig {
    @Bean static TracingPostProcessor tracingPostProcessor() { return new TracingPostProcessor(); } // static!
    @Bean Dependency dependency() { return new Dependency(); }
    @Bean(initMethod = "customInit", destroyMethod = "customDestroy")
    CacheWarmer cacheWarmer() { return new CacheWarmer(); }
    @Bean Poller poller() { return new Poller(); }                // a SmartLifecycle bean

    @EventListener void onRefresh(ContextRefreshedEvent e) { Trace.add("ContextRefreshedEvent"); }
    @EventListener void onClose(ContextClosedEvent e)      { Trace.add("ContextClosedEvent"); }
}
```

Output (context created, then closed):

```
--- bean lifecycle ---
 1. constructor
 2. setter injection
 3. BeanNameAware.setBeanName(cacheWarmer)
 4. BeanPostProcessor.before(cacheWarmer)
 5. @PostConstruct
 6. InitializingBean.afterPropertiesSet
 7. @Bean(initMethod) customInit
 8. BeanPostProcessor.after(cacheWarmer)
 9. SmartLifecycle.start (Poller)
10. ContextRefreshedEvent
11. ----- application running -----
12. ContextClosedEvent
13. SmartLifecycle.stop (Poller)
14. @PreDestroy
15. DisposableBean.destroy
16. @Bean(destroyMethod) customDestroy
```

How it works



1. **Instantiate** — the constructor runs; constructor-injected dependencies are available here.

2. **Populate properties** — setter and field injection.
3. **`Aware` callbacks** — the bean can receive container objects: its name, the `BeanFactory`, the `ApplicationContext`, the `Environment`. (Prefer injecting these normally; `Aware` interfaces tie code to Spring.)
4. **`BeanPostProcessor.postProcessBeforeInitialization`** for every post-processor. `@PostConstruct` itself is implemented by one (`CommonAnnotationBeanPostProcessor`), which is why our custom "before" ran just before it.
5. **Initialization**, in this order: `@PostConstruct` → `InitializingBean.afterPropertiesSet()` → `@Bean(initMethod)`.
6. **`postProcessAfterInitialization`** — the bean may be *replaced* here, typically by a proxy (transactions, AOP, §2.8).
7. After **all** singletons are ready: `SmartLifecycle.start()` (by phase), then `ContextRefreshedEvent` (then, in Boot, `ApplicationStartedEvent` and `ApplicationReadyEvent`).
8. On `close()` (or JVM shutdown hook — code the JVM runs when the process is asked to stop — which Boot registers): `ContextClosedEvent`, `SmartLifecycle.stop()` in reverse phase order.
9. **Destruction**, in this order: `@PreDestroy` → `DisposableBean.destroy()` → `@Bean(destroyMethod)`. Beans are destroyed in reverse dependency order.

Which hook to use

Hook	Use for	Notes
`@PostConstruct` / `@PreDestroy` (`jakarta.annotation`)	Validate state, warm a cache, open/close a resource	**Default choice** — standard, no Spring interface
`InitializingBean` / `DisposableBean`	Framework or library code	Couples your class to Spring
`@Bean(initMethod, destroyMethod)`	Third-party classes you can't annotate	`destroyMethod` is inferred: a public `close()` / `shutdown()` is called automatically
`SmartLifecycle`	Things that must *run* while the app is up: message listeners, pollers, schedulers	`start` / `stop`, `getPhase()` orders them (lower starts first, stops last), `isAutoStartup()`
`ApplicationReadyEvent` listener (Boot)	"Do X once the app is fully started"	Runs after runners; the server already accepts traffic
`CommandLineRunner` / `ApplicationRunner` (Boot)	One-off startup tasks with access to args	Run before `ApplicationReadyEvent`

Common mistakes

- Doing heavy work (remote calls, long cache loads) in constructors or `@PostConstruct` — it slows startup and a failure kills the app. Consider lazy loading or `ApplicationReadyEvent`.
- Calling `@Transactional` methods from `@PostConstruct` and expecting a transaction: the proxy is applied in step 6, *after* `@PostConstruct` (step 5), and you are calling `this` anyway (self-invocation, Chapter 4).
- Expecting `@PreDestroy` on prototype beans (§2.6).
- Killing the JVM with `kill -9` (a forced kill the process cannot intercept): no shutdown hook runs, so no destroy callbacks.

In the interview

Q: Describe the Spring bean lifecycle. Instantiate → inject → `Aware` callbacks → BPP before-init → `@PostConstruct` → `afterPropertiesSet` → custom init → BPP after-init (proxies) → ready; on shutdown `@PreDestroy` → `destroy()` → custom destroy.

- *Follow-up: Where does a `@Transactional` proxy come from?* A `BeanPostProcessor`'s after-initialization step wraps the bean.
- *Follow-up: `@PostConstruct` vs constructor?* In the constructor, setter/field-injected dependencies and post-processing haven't happened yet; `@PostConstruct` runs after all injection.

TL;DR

Construct → inject → aware → BPP-before → `@PostConstruct` → `afterPropertiesSet` → initMethod → BPP-after (proxies) → lifecycle start → ... → `@PreDestroy` → `destroy` → destroyMethod.

2.8 Post-processors: the container's extension points

What it is

Two interfaces let code customise the container itself:

	`BeanFactoryPostProcessor` (BFPP)	`BeanPostProcessor` (BPP)
Works on	Bean **definitions** (recipes)	Bean **instances**
Runs	Once, after definitions are loaded, **before any normal bean is created**	For every bean, before and after its init callbacks
Can	Change scopes, class names, property values; register more definitions	Modify the bean, or **return a different object** (proxy/wrapper)
Spring's own examples	`PropertySourcesPlaceholderConfigurer` (resolves `${...}`), `ConfigurationClassPostProcessor` (processes `@Configuration`, `@ComponentScan`, `@Import`)	`AutowiredAnnotationBeanPostProcessor` (`@Autowired`, `@Value`), `CommonAnnotationBeanPostProcessor` (`@PostConstruct`, `@PreDestroy`), the AOP auto-proxy creator (`@Transactional`, `@Cacheable`, `@Async`)

In other words, most of Spring's annotation "magic" is ordinary post-processors.

Example — a BPP that swaps beans for logging proxies

```
@Target(ElementType.TYPE) @Retention(RetentionPolicy.RUNTIME)
public @interface LogCalls {}

@Component @LogCalls
public class FriendlyGreeter implements Greeter { public String greet(String name) { return "Hi " + name; } }

@Component
public class LogCallsPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) {
        if (!bean.getClass().isAnnotationPresent(LogCalls.class)) return bean;
        return Proxy.newProxyInstance(                          // JDK dynamic proxy over the bean's interfaces
                bean.getClass().getClassLoader(), bean.getClass().getInterfaces(),
                (proxy, method, args) -> {
                    Trace.add("-> " + beanName + "." + method.getName() + "(" + args[0] + ")");
                    Object result = method.invoke(bean, args);  // call the real object
                    Trace.add("<- " + result);
                    return result;
                });
    }
}

@Component
public class CountingFactoryPostProcessor implements BeanFactoryPostProcessor {
    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory bf) {
        Trace.add("BeanFactoryPostProcessor: friendlyGreeter defined=" + bf.containsBeanDefinition("friendlyGreeter")
                + ", created=" + bf.containsSingleton("friendlyGreeter"));
    }
}
```

Output (test gets the `Greeter` bean and calls `greet("Ada")`):

```
--- post-processors ---
 1. BeanFactoryPostProcessor: friendlyGreeter defined=true, created=false
 2. bean class = jdk.proxy2.$Proxy45
 3. -> friendlyGreeter.greet(Ada)
 4. <- Hi Ada
```

1. The BFPP ran when the `friendlyGreeter` **definition** existed but no **instance** had been created.
2. The BPP returned a proxy, so the container stored and injected the *proxy* (`jdk.proxy2.$Proxy45`), not `FriendlyGreeter`. Every caller now goes through the logging code — this is exactly how Spring AOP adds transactions and caching (Chapter 4). The Proxy pattern itself: LLD §9.7.
3. The proxy here is a **JDK dynamic proxy** (Java's built-in way to create, at runtime, an object that implements given interfaces and routes every call to one handler). Because it implements only the *interfaces*, `ctx.getBean(FriendlyGreeter.class)` would now fail — one reason to inject interfaces, and one reason Spring often uses class-based CGLIB proxies instead.

Rules

- Declare BFPP and BPP `@Bean` methods `static` (as in `LifecycleConfig` above). They must be created very early; a non-static `@Bean` method forces Spring to create the whole configuration class early too, so that class's own `@Autowired`/`@Value` may not be processed. Spring logs an info/warning message when this happens.
- A BPP should not depend on ordinary beans: anything it injects is created before all BPPs are registered, and so is "not eligible for getting processed by all BeanPostProcessors" (e.g. won't get transactional proxies). Spring logs exactly that phrase.
- Order several post-processors with `Ordered`/`PriorityOrdered` or `@Order`.

TL;DR

BFPP edits recipes before any bean exists; BPP sees (and can replace) every bean. `@Autowired`, `@Value`, `@PostConstruct` and all proxy-based features are implemented as post-processors.

2.9 Circular dependencies

What it is

A **circular dependency** is a cycle in the dependency graph: `OrderService` needs `CustomerService`, which needs `OrderService` (or a longer loop A → B → C → A).

What happens — proved

```
// Constructor cycle
@Component class OrderService    { OrderService(CustomerService customers) {} }
@Component class CustomerService { CustomerService(OrderService orders) {} }

// Field cycle
@Component class OrderService    { @Autowired CustomerService customers; }
@Component class CustomerService { @Autowired OrderService orders; }

// Cycle broken with @Lazy
@Component class OrderService    { OrderService(@Lazy CustomerService customers) {...} }
@Component class CustomerService { CustomerService(OrderService orders) {...} }
```

Output:

```
constructor cycle -> BeanCurrentlyInCreationException: Error creating bean with name 'ctorCycle.CustomerService': Requested bean is currently
in creat...
field cycle, plain Spring -> OK, orders.customers.orders == orders? true
field cycle, Spring Boot -> BeanCurrentlyInCreationException
@Lazy cycle -> OK, injected class = LazyCycle$CustomerService$$SpringCGLIB$$0, call = customers
```

How it works

1. **Constructor cycles are impossible to satisfy.** To call `new OrderService(customers)` Spring needs a finished `CustomerService`, whose constructor needs a finished `OrderService`... Spring detects the loop and throws `BeanCurrentlyInCreationException`.
2. **Field/setter cycles can be satisfied in plain Spring**: Spring constructs `OrderService` (empty), registers an *early reference* to it in an internal cache, then creates `CustomerService` and injects that early reference, then finishes `OrderService`. (The internal "singleton caches" are sometimes called the three-level cache in interviews.)
3. **Spring Boot forbids that by default** (`spring.main.allow-circular-references=false`, since Boot 2.6). A cycle usually means the classes are too entangled, and early references can also skip proxies (an early reference is the raw object, not the transactional proxy). Boot's startup report prints the cycle as a small ASCII diagram.
4. **`@Lazy` on an injection point** injects a proxy instead of the real bean; the real `CustomerService` is looked up on the first method call. That breaks the creation-time cycle — note the `$$SpringCGLIB$$` class name.

How to fix a cycle (best first)

1. **Redesign.** Usually one of the classes does two jobs. Extract the shared logic into a third bean both can depend on, or move a method to where its data lives.
2. **Use events** (§2.12): `OrderService` publishes `OrderPlaced`; `CustomerService` listens. No compile-time dependency either way.
3. **Inject lazily**: `@Lazy` on the parameter, or `ObjectProvider<CustomerService>` and call `getObject()` when needed.
4. **Last resort**: `spring.main.allow-circular-references=true` — hides the design problem; avoid.

In the interview

Q: How does Spring handle circular dependencies? Constructor cycles fail with `BeanCurrentlyInCreationException`. Field/setter cycles can be resolved with early singleton references, but Spring Boot disallows them by default since 2.6. Fix by redesigning (extract a third class), decoupling with events, or `@Lazy`/`ObjectProvider`.

- *Follow-up: Why is constructor injection "good" here?* It makes cycles fail fast at startup instead of silently producing half-initialised or unproxied objects.

TL;DR

Constructor cycles always fail; field cycles work in plain Spring but Boot rejects them. Refactor first; `@Lazy` or events if you must.

2.10 Lazy initialization

What it is

By default the `ApplicationContext` creates every singleton at startup. A **lazy** bean is created on first use instead.

```
@Component @Lazy                          // not created at startup; created on first use
public class ReportGenerator { public ReportGenerator() { Trace.add("ReportGenerator created (lazy)"); } }

@Component                                // eager by default
public class RateCache { public RateCache() { Trace.add("RateCache created (eager)"); } }
```

```
--- @Lazy ---
 1. RateCache created (eager)
 2. -- context started
 3. ReportGenerator created (lazy)
--- spring.main.lazy-initialization=true ---
 1. -- context started
 2. RateCache created (eager)
```

The second run started the same classes with Spring Boot and the property `spring.main.lazy-initialization=true`, which makes **every** bean lazy — even `RateCache`, which has no `@Lazy`. `@Lazy(false)` on a bean opts it back out.

Trade-offs

Eager (default)	Lazy
Slower startup	Faster startup
Misconfiguration fails **at startup**	Misconfiguration fails **on first request** — maybe in production
First request is fast	First request pays the creation cost
Memory used up front	Memory only for used beans

When to use it: individual rarely used, expensive beans; global lazy init only in development or for tests/CLI tools (command-line programs) where startup time dominates. Keep production eager so broken configuration can't get past startup. (For fast production startup, Chapter 15 covers AOT and CDS — Class Data Sharing, a JVM cache of pre-loaded classes.)

2.11 `@Conditional` and profiles

What it is

- `@Conditional(SomeCondition.class)` — register a bean *only if* a `Condition` returns true. A `Condition` sees the environment (properties and profiles), the registry (the bean definitions registered so far) and the classpath (the jars and classes available to the app). This is the building block of Spring Boot's auto-configuration (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty` — Chapter 3).
- `@Profile("dev")` — a ready-made condition: register only when that **profile** is active. A profile is a named set of beans/settings for one environment (`dev`, `test`, `prod`). Expressions are allowed: `"!dev"`, `"prod & eu"`, `"dev | test"`.

```
public class OnFeatureFlag implements Condition {
    @Override
    public boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata) {
        return context.getEnvironment().getProperty("features.fraud-check", Boolean.class, false);
    }
}

@Configuration
public class PaymentBeans {
    @Bean @Profile("dev")                 // only when profile "dev" is active
    PaymentGateway fakeGateway() { return () -> "fake gateway (dev)"; }

    @Bean @Profile("!dev")                // any time "dev" is NOT active
    PaymentGateway httpGateway() { return () -> "HTTP gateway (real)"; }

    @Bean @Conditional(OnFeatureFlag.class)  // custom condition
    FraudChecker fraudChecker() { return new FraudChecker(); }
}
```

The test starts two contexts — one with profile `dev` and `features.fraud-check=true`, one with neither:

```
profile dev : fake gateway (dev), fraudChecker beans = 1
no profile  : HTTP gateway (real), fraudChecker beans = 0
```

How to activate profiles

`spring.profiles.active=dev` in a property file, `--spring.profiles.active=dev` on the command line, env var `SPRING_PROFILES_ACTIVE=dev`, `@ActiveProfiles("test")` in tests, or `context.getEnvironment().setActiveProfiles("dev")` in plain Spring. With no active profile, the `default` profile is active. Profile-specific *property files* (`application-dev.yaml`) and profile groups are in Chapter 3.

Common mistakes

- Using profiles for things that are just values (URLs, timeouts). Keep one bean and change the *property* per environment; reserve profiles for genuinely different beans (a fake payment gateway in dev).
- `@Profile("prod")` on a bean that tests need → tests fail with "no bean". Provide an alternative for the non-prod case (`"!prod"`).
- Writing a custom `Condition` in a Boot app when a `@ConditionalOnProperty` already does it.

TL;DR

`@Conditional` = register the bean only if a condition matches; `@Profile` = the environment-name condition. Boot's auto-configuration is built from these.

2.12 Application events

What it is

The container includes a small **publish/subscribe** mechanism (senders "publish" messages without knowing who receives them; receivers "subscribe" to the kinds they care about). Any bean can publish an event object through `ApplicationEventPublisher`; any bean method annotated `@EventListener` whose parameter type matches receives it. Publishers don't know who listens — this is the **Observer pattern** (LLD §11.1) built into Spring. Since Spring 4.2 an event can be any object (it no longer has to extend `ApplicationEvent`); records are ideal.

```
public record OrderPlaced(String orderId, long cents) {}
public record StockReserved(String orderId) {}

@Service
public class OrderService {
    private final ApplicationEventPublisher events;
    public OrderService(ApplicationEventPublisher events) { this.events = events; }

    public void place(String id, long cents) {
        Trace.add("OrderService: saving " + id + " on " + Thread.currentThread().getName());
        events.publishEvent(new OrderPlaced(id, cents));   // listeners run NOW, same thread
        Trace.add("OrderService: publishEvent returned");
    }
}

@Component
public class OrderListeners {
    @EventListener @Order(1)
    void sendEmail(OrderPlaced e) { Trace.add("  email listener for " + e.orderId() + " on " + Thread.currentThread().getName()); }

    @EventListener @Order(2)
    StockReserved reserveStock(OrderPlaced e) {          // a non-void return value is published as a new event
        Trace.add("  stock listener for " + e.orderId());
        return new StockReserved(e.orderId());
    }

    @EventListener(condition = "#event.cents > 10000")    // SpEL filter: only big orders
    void alertBigOrder(OrderPlaced event) { Trace.add("  BIG order alert for " + event.orderId()); }

    @EventListener
    void onStock(StockReserved e) { Trace.add("  shipping listener sees StockReserved(" + e.orderId() + ")"); }
}
```

Output for `place("A-1", 2_500)` then `place("B-2", 50_000)`:

```
--- events ---
 1. OrderService: saving A-1 on main
 2.   email listener for A-1 on main
 3.   stock listener for A-1
 4.   shipping listener sees StockReserved(A-1)
 5. OrderService: publishEvent returned
 6. OrderService: saving B-2 on main
 7.   email listener for B-2 on main
 8.   stock listener for B-2
 9.   shipping listener sees StockReserved(B-2)
10.   BIG order alert for B-2
11. OrderService: publishEvent returned
```

How it works

1. `publishEvent` finds all listeners whose parameter type matches the event (generics — type parameters like `List<String>` — are respected).
2. It calls them **synchronously** (the caller waits until they finish)**, on the publisher's thread** (a thread is one independent path of execution in the program) (`main` above), in `@Order` sequence; unordered listeners run after ordered ones.
3. A listener's return value (if not `void`/`null`) is published as a new event — `StockReserved` was handled *before* the remaining `OrderPlaced` listener ran.
4. `condition` is a SpEL (Spring Expression Language — small expressions Spring evaluates at runtime) expression over `#event` (or the parameter name); `false` skips the listener.
5. Only after all listeners finish does `publishEvent` return. **An exception in a listener propagates to the publisher** (is thrown on up to the code that called `publishEvent`).

Variations and built-in events

Need	Use	Chapter
Run the listener on another thread	`@Async` + `@EventListener` (needs `@EnableAsync`)	14
Run only after the publisher's DB transaction commits (is permanently saved)	`@TransactionalEventListener(phase = AFTER_COMMIT)`	7
React to container stages	`ContextRefreshedEvent`, `ContextClosedEvent` (Framework); `ApplicationStartingEvent`, `ApplicationEnvironmentPreparedEvent`, `ApplicationPreparedEvent`, `ApplicationStartedEvent`, `ApplicationReadyEvent`, `ApplicationFailedEvent`, `WebServerInitializedEvent` (Boot)	2.7, 15
Events between services	A message broker (a server that stores and forwards messages between programs, e.g. Kafka), not application events — they are in-memory, single JVM	14

Trade-offs

Direct method call	Application event
Explicit, easy to follow in the IDE	Decoupled — publisher doesn't know listeners
Compile-time dependency	No dependency; breaks cycles (§2.9)
Obvious error handling	Synchronous by default: slow/failing listeners slow/fail the publisher
—	Lost if the JVM crashes (not durable — not saved anywhere that survives a crash)

Common mistakes

- Assuming events are asynchronous. They aren't unless you add `@Async`.
- Using in-memory events as a reliable integration mechanism; for "must not be lost" use the transactional outbox (save the event in the same database transaction as the data, then publish it from there) + a broker (Chapter 14, microservices notes).
- Sending emails in a plain listener inside a transaction that later rolls back (is cancelled, undoing its changes) — use `@TransactionalEventListener`.

TL;DR

`publishEvent` + `@EventListener` = in-process Observer. Synchronous, same thread, ordered with `@Order`, filterable with SpEL, return values become new events.

2.13 Null-safety with JSpecify (Framework 7)

What it is

`NullPointerException` is Java's most common runtime error. Java has no built-in way to say "this may be null". **JSpecify** (`org.jspecify.annotations`) is a standard set of annotations agreed by several vendors (Google, JetBrains, Uber, Spring, ...):

Annotation	Meaning
`@NullMarked` (on a package, class or module)	"In here, every type is non-null unless marked otherwise"
`@Nullable`	This field/parameter/return value may be `null`
`@NonNull`	Explicitly non-null (rarely needed inside `@NullMarked`)
`@NullUnmarked`	Opt a part back out

Spring Framework 7 and Boot 4 annotate their whole codebase with JSpecify, replacing Spring's own `org.springframework.lang.Nullable`/`NonNull` (deprecated since 7.0). IDEs (IntelliJ IDEA) and build-time checkers such as **NullAway** (an Error Prone plugin — Error Prone is Google's extra compiler checks) read the annotations and warn when you might dereference `null` (call a method or read a field through a `null` reference); Kotlin sees Spring APIs with correct nullable/non-null types. `jspecify` comes transitively (as a dependency of a dependency) with `spring-core`.

Example

```
// package-info.java
@NullMarked
package com.example.ioc.nullsafety;
import org.jspecify.annotations.NullMarked;

// ReportService.java
@Service
public class ReportService {
    private final @Nullable Translator translator;   // may be absent

    // Spring treats a parameter annotated with any @Nullable as optional: it injects null if no bean.
    public ReportService(@Nullable Translator translator) { this.translator = translator; }

    public String title(String raw) {
        return translator == null ? raw : translator.translate(raw);   // the null check the annotation asks for
    }

    public @Nullable String findTitle(int id) { return id == 1 ? "Q3 sales" : null; }  // callers see null is possible
}
```

There is no `Translator` bean in the context:

```
no Translator bean -> title = Q3 sales, findTitle(2) = null
```

How it works

1. `@NullMarked` on the package makes every unannotated type in it non-null by *contract*.
2. `@Nullable` marks the exceptions. Tools, not the JVM, enforce the contract — at runtime these annotations do nothing by themselves.
3. **Spring's container reads them for injection**: an injection point annotated with an annotation named `Nullable` is treated as optional, so a missing bean gives `null` instead of a startup failure (like `Optional<T>`).
4. Add NullAway to your build to turn the contract into compile errors (optional; Spring itself does this).

Common mistakes

- Thinking `@Nullable` adds runtime checks — it doesn't; it documents and enables tooling.
- Mixing old `org.springframework.lang.Nullable` with JSpecify in new code. Use `org.jspecify.annotations`.
- Putting `@Nullable` before a qualified type in the wrong place: JSpecify annotations are *type-use* annotations (they attach to a type, wherever a type is written, rather than to a declaration), so an array or nested type is written `String @Nullable []` or `Outer.@Nullable Inner`.

TL;DR

Framework 7 / Boot 4 use JSpecify: `@NullMarked` package + `@Nullable` exceptions. Tools check it; Spring also treats `@Nullable` injection points as optional.

2.14 Programmatic registration with `BeanRegistrar` (Framework 7)

What it is

`@Bean` methods register a *fixed* set of beans. When the set depends on runtime data — "one client bean per configured channel" — Framework 7's `BeanRegistrar` lets you register beans with ordinary code (loops, `if`s), in a form that also works with AOT/native images.

```java
public class ChannelRegistrar implements BeanRegistrar {
    public record Channel(String name) {}
    public record Router(java.util.List<Channel> channels) {}

    @Override
    public void register(BeanRegistry registry, Environment env) {
        String[] names = env.getProperty("channels", String[].class, new String[] {"email"});
        for (String n : names) {                                    // one bean per configured channel
            registry.registerBean(n + "Channel", Channel.class, spec -> spec.supplier(ctx -> new Channel(n)));
        }
        registry.registerBean("router", Router.class, spec -> spec
                .lazyInit()
                .supplier(ctx -> new Router(ctx.beanProvider(Channel.class).orderedStream().toList())));
    }
}

@Configuration
@Import(ChannelRegistrar.class)   // a BeanRegistrar is activated with @Import
public class RegistrarConfig {}
```

With `channels=email,sms,push`:

```
registered: emailChannel, smsChannel, pushChannel, router
router -> Router[channels=[Channel[name=email], Channel[name=sms], Channel[name=push]]]
```

The `spec` offers the same options as annotations: `primary()`, `fallback()`, `lazyInit()`, `prototype()`, `scope(...)`, `order(...)`, `description(...)`, `supplier(...)` (whose context can fetch other beans with `bean(...)` / `beanProvider(...)`). Use it for libraries and dynamic setups; for normal application code, `@Component` and `@Bean` remain simpler.

2.15 In the interview — quick round

Question	Short answer
What is IoC? DI?	IoC: the framework controls object creation and calls your code. DI: objects receive dependencies instead of building them; Spring's container does it.
BeanFactory vs ApplicationContext?	Minimal lazy container vs full container: eager singletons, post-processors, events, environment, i18n. (§2.2)
`@Component` vs `@Bean`?	Class annotation found by scanning vs factory method for third-party classes/complex creation. (§2.3)
Full vs lite configuration?	Full = CGLIB subclass makes inter-bean calls return singletons; lite = plain calls. Pass dependencies as parameters. (§2.3)
Why constructor injection?	Immutable, complete objects, explicit, testable, fails fast on cycles. (§2.4)
Two beans of one type?	`@Qualifier`, `@Primary`, `@Fallback`, parameter name, or inject `List` / `Map`. (§2.5)
Default scope? Thread-safe?	Singleton; shared across threads, so keep it stateless. (§2.6)
Prototype in singleton?	Injected once; use `ObjectProvider` / `@Lookup` /scoped proxy. (§2.6)
Lifecycle order?	Construct → inject → aware → BPP before → `@PostConstruct` → `afterPropertiesSet` → init-method → BPP after → ... → `@PreDestroy` → `destroy` → destroy-method. (§2.7)
BPP vs BFPP?	Instances vs definitions. `@Autowired` / `@Transactional` are BPPs; `${}` resolution is a BFPP. (§2.8)
Circular dependency?	Constructor cycles fail; Boot rejects field cycles by default; refactor, events, or `@Lazy`. (§2.9)
`@Profile` vs `@Conditional`?	`@Profile` is a specific condition on active profiles; `@Conditional` takes any `Condition`. (§2.11)
Are events async?	No — synchronous on the publisher's thread unless `@Async`. (§2.12)
What's new in the Framework 7 container?	JSpecify null-safety, `BeanRegistrar`, `javax.inject` / `javax.annotation` support removed. (§2.13–2.14)

Chapter TL;DR

- **DI**: classes declare dependencies (constructor parameters); the `ApplicationContext` creates beans in dependency order, eagerly, at startup.
- **Define beans** by scanning (`@Component`, `@Service`, `@Repository`, `@Controller`), with `@Bean` methods (third-party classes), `@Import`, or Framework 7's `BeanRegistrar`.
- **Full** `@Configuration` is a CGLIB subclass (inter-bean calls return singletons); **lite** mode is plain Java. Pass dependencies as parameters and it doesn't matter.
- **Constructor injection** with `final` fields is the default choice.

- **Picking a bean**: type → `@Qualifier` → `@Primary` → `@Fallback` → parameter name; `List`/`Map` for all; `Optional`/`ObjectProvider`/`@Nullable` for optional.
- **Scopes**: singleton (shared, stateless!), prototype (new per lookup, no destroy callbacks), request/session (via scoped proxies).
- **Lifecycle**: construct → inject → aware → BPP-before → `@PostConstruct` → `afterPropertiesSet` → init-method → BPP-after (proxies) → `SmartLifecycle.start` → `ContextRefreshedEvent` ... `@PreDestroy` → `destroy` → destroy-method.
- **Post-processors** implement Spring's annotations; BPPs can replace beans with proxies. Declare them `static`.
- **Circular dependencies**: constructor cycles fail, Boot rejects field cycles — redesign, use events, or `@Lazy`.
- **Lazy init** trades startup errors for faster startup; `@Conditional`/`@Profile` choose beans per environment.
- **Events**: `publishEvent` + `@EventListener`, synchronous and ordered.
- **Framework 7**: JSpecify `@NullMarked`/`@Nullable` everywhere; `BeanRegistrar` for programmatic registration.

Next: Chapter 3 — Configuration and Auto-configuration shows where property values come from and how Boot uses conditions to configure beans for you.

Chapter 3 — Configuration and Auto-configuration

A real application runs in many places — your laptop, CI (the continuous-integration server that builds and tests every change), staging (a production-like test environment), production (the live system real users hit) — with different database URLs, passwords, timeouts and feature flags (on/off switches for features). Spring Boot's answer is **externalized configuration**: the same jar reads its settings from outside, from sources with a well-defined priority. Boot then uses those settings, together with what is on the classpath (the jars and classes available to the app at runtime), to **auto-configure** beans for you. This chapter covers both halves and ends by writing your own auto-configuration and starter.

What you'll learn

1. The `Environment` and property sources — how Spring sees configuration (§3.1).
2. `application.properties` vs `application.yaml`, file locations and multi-document files (§3.2).
3. The precedence order of all sources, proved (§3.3), and environment variables with relaxed binding (§3.4).
4. `@Value` (§3.5) vs type-safe `@ConfigurationProperties` with records, conversion and validation (§3.6).
5. Profiles, profile-specific files and profile groups (§3.7), and `spring.config.import` (§3.8).
6. Random values, placeholders and other sources (§3.9).
7. How auto-configuration works: `AutoConfiguration.imports`, `@ConditionalOn...` (§3.10).
8. Reading the conditions report (`--debug`, the conditions endpoint) (§3.11) and overriding/excluding auto-configuration (§3.12).
9. Writing a custom starter and testing it with `ApplicationContextRunner` (§3.13).
10. Boot 4's modularization and what it means for you (§3.14).

Runnable code: `code/config-demo/` (a non-web Boot app started in tests with different arguments, fake environment variables and profiles) and `code/greeter/` (a three-module build: auto-configuration module, starter, and an app using it). Built with `mvn -q -B -ntp verify` on Spring Boot 4.1.1 / JDK 25; all outputs are real.

Background you may want first: the container, `@Conditional` and profiles in Chapter 2 (§2.11). The architecture idea behind this chapter — keep config out of the artifact — is the *externalized configuration* pattern in the microservices notes.

3.1 Externalized configuration and the `Environment`

What it is

Externalized configuration means settings live *outside* the compiled code, so one build artifact runs everywhere and only its configuration changes. (This is factor III, "Config", of the Twelve-Factor App methodology — a well-known list of twelve rules for building cloud-friendly services.)

In Spring, all configuration is reached through one object, the `Environment`:

- a **property source** (`PropertySource`) is one named set of key/value pairs — a file, the OS environment, the command line, a map in a test;
- the `Environment` holds an **ordered list** of property sources and the active **profiles**;
- `environment.getProperty("shop.name")` asks each source *in order* and returns the **first** value found.

Analogy: a stack of transparent sheets on an overhead projector. Each sheet (property source) writes some values; you read the projection from the top, so a value on a higher sheet hides the same key on lower sheets.

Why it matters

- **No rebuilds per environment** — the same jar goes from test to production.
- **Secrets stay out of Git** (the version-control system that stores your source code) — passwords arrive through environment variables (named values the operating system passes to a process) or mounted files (files a platform places into the app's file system at startup).
- **Operations can change behaviour** (timeouts, pool sizes such as how many database connections to keep open, feature flags) without a code change.

How it works in Boot

1. `SpringApplication.run` builds the `Environment` *before* creating any bean.
2. It adds sources: command-line arguments, `SPRING_APPLICATION_JSON`, Java system properties (key/value settings passed to the JVM with `-Dkey=value`), OS environment variables, `random.*`, then the **config data** files (`application.properties`/`.yaml` and profile-specific variants, imports).
3. Beans read values through `@Value`, `@ConfigurationProperties` or `Environment` injection.
4. Auto-configuration conditions (`@ConditionalOnProperty`) read the same `Environment`.

3.2 `application.properties` vs `application.yaml`

Syntax

Both formats hold the same data; pick one per project. The book's demo uses YAML (a human-friendly configuration format that uses indentation to show nesting):

```yaml
# config-demo/src/main/resources/application.yaml (first part)
spring:
  application:
    name: config-demo
shop:
  name: Corner Shop
  checkout-timeout: 45s              # kebab-case in files; binds to checkoutTimeout
  max-upload: 10MB
  currencies: [USD, EUR]
  discounts:
    gold: 15
    silver: 5
  payment:
    retries: 2
```

The same in `.properties`:

```properties
spring.application.name=config-demo
shop.name=Corner Shop
shop.checkout-timeout=45s
shop.max-upload=10MB
shop.currencies=USD,EUR          # or shop.currencies[0]=USD / shop.currencies[1]=EUR
shop.discounts.gold=15
shop.discounts.silver=5
shop.payment.retries=2
```

	`.properties`	`.yaml` / `.yml`
Structure	Flat `a.b.c=value` lines	Indentation-based tree
Lists	`x=a,b` or `x[0]=a`	`[a, b]` or `- a` lines
Multiple documents in one file	`#---` separator line	`---` separator line
Pitfalls	Long repeated prefixes	Indentation errors; unquoted values like `on`, `no`, `010` may be read as booleans (true/false)/octal (base-8 numbers) — quote them
Loadable by `@PropertySource`	Yes	**No**
Both in the same folder	`.properties` wins over `.yaml`	

Use **kebab-case** keys in files (`checkout-timeout` — lower-case words joined by dashes), the recommended canonical (standard, official) form.

Where Boot looks

Boot loads `application.properties`/`.yaml` from these locations; **later ones override earlier ones**:

1. The classpath root (`src/main/resources/application.yaml` ends up here)
2. The classpath `/config` package
3. The current directory
4. The `config/` subdirectory of the current directory
5. Immediate child directories of `config/` (e.g. `config/db/application.yaml`)

So an operator can drop a `config/application.yaml` next to the jar to override what's packaged inside it. Change the file name with `spring.config.name`, replace the search locations with `spring.config.location`, or add locations with `spring.config.additional-location` (these three must be set early: as command-line args, system properties or env vars).

Multi-document files

A single file can contain several documents; a document can be **activated** conditionally:

```yaml
# ... first document above ...
---
# a second document inside the same file, used only when "prod" is active
spring:
  config:
    activate:
      on-profile: prod
shop:
  payment:
    gateway-url: https://pay.prod.example.test
    retries: 5
```

`spring.config.activate.on-profile` takes a profile expression (`prod | staging`); `spring.config.activate.on-cloud-platform` (e.g. `kubernetes` — Kubernetes is the most common platform for running containers across many machines) is the other activation property. Later documents override earlier ones.

3.3 Precedence: which value wins?

The full order

From **lowest** to **highest** priority (a higher item overrides lower ones), per the Boot 4.1 reference:

#	Source	Example
1	Default properties	`SpringApplication.setDefaultProperties(...)`
2	`@PropertySource` on `@Configuration` classes	`@PropertySource("classpath:legacy.properties")`
3	**Config data** files (table below)	`application.yaml`, `application-prod.yaml`
4	`RandomValuePropertySource`	`${random.uuid}`
5	**OS environment variables**	`SHOP_NAME=...`
6	**Java system properties**	`java -Dshop.name=... -jar app.jar`
7	JNDI (Java Naming and Directory Interface, a lookup service in classic app servers) attributes	(app servers)
8–9	`ServletContext` / `ServletConfig` init parameters	(WAR — web-archive — deployments to an external server)
10	`SPRING_APPLICATION_JSON`	`SPRING_APPLICATION_JSON='{"shop":{"name":"X"}}'`
11	**Command-line arguments**	`java -jar app.jar --shop.name=...`
12	`properties` attribute of tests	`@SpringBootTest(properties = "shop.name=T")`
13	`@DynamicPropertySource`	Testcontainers (a library that starts real databases in Docker for tests) URLs (Chapter 12)
14	`@TestPropertySource`	
15	DevTools global settings	`~/.config/spring-boot/`

Within **config data** (row 3), again lowest → highest:

1. `application.*` packaged **inside** the jar
2. `application-{profile}.*` inside the jar
3. `application.*` **outside** the jar (current dir, `config/`)
4. `application-{profile}.*` outside the jar

Memory aid: *specific beats general* (profile file > plain file; outside > inside; command line > everything the app ships with), and tests override everything.

Proved

The test starts the real application five times, each time adding one higher source (fake environment variables are injected through a custom `StandardEnvironment`; see `Apps.java`):

```java
// PrecedenceTest.java (simplified)
shopName(Map.of());                                                  // 1
shopName(Map.of(), "--spring.profiles.active=dev");                  // 2
shopName(Map.of("SHOP_NAME", "Env Shop"), "--spring.profiles.active=dev");   // 3
System.setProperty("shop.name", "SysProp Shop");
shopName(Map.of("SHOP_NAME", "Env Shop"));                           // 4
shopName(Map.of("SHOP_NAME", "Env Shop"), "--shop.name=CLI Shop");           // 5 (system property still set)
```

```
1 application.yaml only          -> Corner Shop
2 + application-dev.yaml         -> Corner Shop (DEV)
3 + env SHOP_NAME                -> Env Shop
4 + -Dshop.name (system prop)    -> SysProp Shop
5 + --shop.name (command line)   -> CLI Shop
```

And this is the `Environment`'s actual search order when started with `--spring.profiles.active=dev` (first wins):

```
Environment property sources (first wins):
  configurationProperties
  commandLineArgs
  systemProperties
  systemEnvironment
  random
  Config resource 'class path resource [application-dev.yaml]' via location 'optional:classpath:/'
  Config resource 'class path resource [owner.properties]' via location 'classpath:owner.properties'
  Config resource 'class path resource [application.yaml]' via location 'optional:classpath:/' (document #0)
  applicationInfo
  defaultProperties
```

Notes on this list:

- `configurationProperties` at the top is not a real source; it's Boot's adapter that exposes all the others to the binder (Boot's component that copies property values into Java objects) with relaxed names.
- `owner.properties` sits *above* `application.yaml` because `application.yaml` imports it (§3.8): **an imported file overrides the file that imports it**.
- The profile file `application-dev.yaml` is above both.

Common mistakes

- Setting a value in `application.yaml` and wondering why it's ignored — an environment variable (often from a Kubernetes manifest — the YAML file describing how to run the app — or a `.env` file) with the same relaxed name overrides it.
- Putting secrets in `application.yaml` in Git. Use env vars, mounted files (`configtree:`, §3.8), or a secret manager (a dedicated service that stores and hands out passwords and keys).
- Expecting a value set in `@PropertySource` to beat `application.yaml` — `@PropertySource` is near the bottom (row 2).

In the interview

Q: A property is set in `application.yaml`, an environment variable and a command-line argument. Which wins? The command-line argument; then system properties, then environment variables, then profile-specific files, then `application.yaml`. Test properties beat all of them.

- *Follow-up: How do you see where a value came from?* The Actuator (Boot's production-monitoring module, Chapter 15) `env` endpoint (a URL the app exposes, here `/actuator/env/shop.name`) lists every source that defines it, highest first; failure messages from binding also show the *origin* (e.g. `"shop.name" from property source "commandLineArgs"`, §3.6).

TL;DR

Command line > system properties > env vars > profile files > `application.*`; outside the jar > inside; imported file > importer. Tests override all.

3.4 Environment variables and relaxed binding

What it is

Environment variable names can't contain dots or dashes on most systems, so Boot maps them by rule. **Relaxed binding** means Boot accepts several spellings of the same property when binding to `@ConfigurationProperties`:

Form	Example	Use in
Kebab case (canonical, recommended)	`shop.checkout-timeout`	`.properties` / `.yaml`
Camel case (words joined, each new word capitalized)	`shop.checkoutTimeout`	Also accepted in files
Underscore	`shop.checkout_timeout`	Also accepted in files
Upper case with underscores	`SHOP_CHECKOUTTIMEOUT`	**Environment variables**

Environment-variable rule: replace `.` with `_`, **remove** `-`, convert to upper case. List index: `SHOP_CURRENCIES_0=USD`. So `shop.payment.retries` → `SHOP_PAYMENT_RETRIES`, `spring.profiles.active` → `SPRING_PROFILES_ACTIVE`.

Proved

```
Apps.start(Map.of("SHOP_PAYMENT_RETRIES", "4", "SHOP_CHECKOUTTIMEOUT", "2m"))
```

```
SHOP_PAYMENT_RETRIES=4, SHOP_CHECKOUTTIMEOUT=2m -> retries=4, checkoutTimeout=PT2M
```

Common mistakes

- Assuming `SHOP_CHECKOUT_TIMEOUT` (an underscore where the dash was) is wrong or ignored. It isn't: for backward compatibility (so older setups keep working) Boot's binder also accepts this **legacy** spelling and binds it to `shop.checkout-timeout` (proved in ch19 — `SHOP_CHECKOUT_TIMEOUT=3m -> PT3M`). The real mistake is using two spellings for the same property: they are separate environment variables, so a search for `SHOP_CHECKOUTTIMEOUT` misses the value set by the other one. Stick to the documented form (drop the dash).
- Using camelCase keys in `@Value("${shop.checkoutTimeout}")`. `@Value` only gets relaxed matching if the placeholder uses the canonical kebab-case form: `@Value("${shop.checkout-timeout}")`.

3.5 `@Value`

What it is

`@Value` injects one value into a field, constructor parameter or `@Bean` method parameter.

```java
// config-demo/src/main/java/com/example/config/ValueStyle.java
@Component
public class ValueStyle {
    ValueStyle(@Value("${shop.name}") String name,                              // required: startup fails if missing
               @Value("${shop.support-email:help@example.test}") String email, // default after the colon
               @Value("#{${shop.payment.retries:3} + 1}") int maxAttempts,      // SpEL: #{...}
               @Value("${shop.owner}") String owner) {                          // comes from an imported file
        ...
    }
}
```

```
@Value                  -> name=Corner Shop, supportEmail=help@example.test, maxAttempts=3, owner=Ada
```

Syntax	Meaning
`${key}`	Property placeholder; startup fails with `Could not resolve placeholder` if missing
`${key:default}`	Default when missing (`${key:}` = empty string)
`#{expression}`	SpEL (Spring Expression Language) — arithmetic, method calls, other beans: `#{@clock.millis()}`
`#{${key} * 2}`	Placeholder resolved first, then SpEL evaluated

Conversion to `int`, `boolean`, `Duration`, enums (Java types with a fixed set of named values), etc. is automatic; comma-separated strings convert to arrays/lists.

When not to use it

For more than two or three related settings, use `@ConfigurationProperties`: `@Value` strings are scattered, untyped until runtime, not validated, and have no IDE (code editor such as IntelliJ IDEA) auto-completion.

3.6 `@ConfigurationProperties` — type-safe configuration

What it is

`@ConfigurationProperties("shop")` binds **every property under a prefix** into one Java object. With a **record** (a compact Java class that just holds data; fields are fixed once created), the object is immutable (cannot change after creation) and bound through its constructor ("constructor binding").

```java
// config-demo/src/main/java/com/example/config/ShopProperties.java
@ConfigurationProperties("shop")
@Validated
public record ShopProperties(
        @NotBlank String name,
        @DefaultValue("30s") Duration checkoutTimeout,
        @DefaultValue("5MB") DataSize maxUpload,
        @DefaultValue("USD") List<String> currencies,
        Map<String, Integer> discounts,
        @Valid @DefaultValue Payment payment) {

    public record Payment(
            @NotBlank @DefaultValue("https://pay.example.test") String gatewayUrl,
            @Min(0) @Max(5) @DefaultValue("3") int retries) {
    }
}

@SpringBootApplication
@ConfigurationPropertiesScan          // registers every @ConfigurationProperties class in this package tree
public class ConfigDemoApplication { ... }
```

Bound from the YAML in §3.2:

```
@ConfigurationProperties -> ShopProperties[name=Corner Shop, checkoutTimeout=PT45S, maxUpload=10485760B, currencies=[USD, EUR], discounts={gold=15, silver=5}, payment=Payment[gatewayUrl=https://pay.example.test, retries=2]]
```

Any bean can now take `ShopProperties` as a constructor parameter and read `props.payment().retries()`.

How it works

1. **Register** the class: `@ConfigurationPropertiesScan` on the application (scans like component scanning), or `@EnableConfigurationProperties(ShopProperties.class)` on a configuration class (typical in auto-configurations).
2. At startup Boot's **`Binder`** walks the constructor parameters (or setters, for JavaBean-style classes), builds each property name from the prefix + relaxed form of the parameter name, and looks it up in the `Environment`.
3. **Conversion**: strings become the parameter type — numbers, booleans, enums, `List`, `Map`, nested records, and these special types:

Type	Accepted values	Notes
`java.time.Duration`	`45s`, `5m`, `2h`, `1d`, `500ms`, `PT45S` (ISO-8601, the international standard text format for dates and durations), plain `45`	Plain numbers default to milliseconds; change with `@DurationUnit(ChronoUnit.SECONDS)`
`org.springframework.util.unit.DataSize`	`10MB`, `512KB`, `1GB`, plain `1024`	Plain numbers are bytes; `@DataSizeUnit` changes it. `10MB` = 10 × 1024 × 1024 = `10485760B`
`java.time.Period`	`1y3d`, `P1Y3D`	

4. **Defaults**: `@DefaultValue("...")` on a constructor parameter supplies a value when the property is absent. A bare `@DefaultValue` on a nested record (as on `payment`) means "create it even if no `shop.payment.*` property exists" — otherwise the nested object would be `null`. `gatewayUrl` came from its default because only `retries` was configured.
5. **Validation**: with `@Validated` on the class and a Bean Validation provider (a library implementing the standard `@NotBlank`/`@Min`-style rules) on the classpath (`spring-boot-starter-validation` → Hibernate Validator), constraints are checked at startup. Put `@Valid` on nested objects so their constraints are checked too.

Validation failure — proved

```java
Apps.start("--shop.payment.retries=9", "--shop.name= ")
```

Boot's failure analyzer (a component that turns a startup exception into a readable explanation) prints this (real output) and the app does not start:

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Binding to target com.example.config.ShopProperties failed:

    Property: shop.name
    Value: " "
    Origin: "shop.name" from property source "commandLineArgs"
    Reason: must not be blank

    Property: shop.payment.retries
    Value: "9"
    Reason: must be less than or equal to 5


Action:

Update your application's configuration
```

The root exception is `BindValidationException`. **Failing at startup is the point**: a bad production setting is caught during deployment, not at 3 a.m. when the first payment retries.

IDE auto-completion — configuration metadata

Adding `spring-boot-configuration-processor` as an annotation processor (a plug-in that runs inside the Java compiler and can generate files) generates `META-INF/spring-configuration-metadata.json` at compile time. IDEs use it for auto-completion and documentation of `shop.*` keys. Its descriptions come from the record's Javadoc (documentation comments `/** ... */`) `@param` tags. Generated for this project (first entries):

```
{"name": "shop.checkout-timeout", "type": "java.time.Duration", "description": "how long a checkout may take", "sourceType":
"com.example.config.ShopProperties", "defaultValue": "30s"}
{"name": "shop.currencies", "type": "java.util.List<java.lang.String>", "description": "accepted currency codes", "sourceType":
"com.example.config.ShopProperties", "defaultValue": "USD"}
```

```
<!-- pom.xml: since JDK 23, javac no longer runs processors found on the classpath by default -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
...
<plugin>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-configuration-processor</artifactId>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

Other shapes

- **JavaBean style** (class with a no-arg constructor, getters and setters) is still supported and needed when you want mutable properties; records are simpler.
- A class with several constructors marks the binding one with `@ConstructorBinding`.
- **Third-party classes**: put `@ConfigurationProperties("pool")` on a `@Bean` method that returns the object; Boot binds `pool.*` onto its setters.

`@Value` vs `@ConfigurationProperties`

	`@ConfigurationProperties`	`@Value`
Granularity	A whole group under a prefix	One value
Type safety	Typed object, checked at startup	Converted per injection point
Relaxed binding	Yes	Limited (use kebab-case placeholders)
Validation	`@Validated` + Jakarta constraints	No
IDE metadata / auto-complete	Yes (processor)	No
SpEL	No	Yes
Default values	`@DefaultValue` / field initialisers	`${key:default}`
Best for	Any real settings of your app or library	A single value, or when you need SpEL

Common mistakes

- Forgetting to register the class (`@ConfigurationPropertiesScan` or `@EnableConfigurationProperties`) → "No qualifying bean of type ShopProperties". Adding `@Component` instead does **not** work for a record: constructor binding only happens for classes registered by scanning or `@EnableConfigurationProperties`; as a plain component Spring would try to inject beans of type `String`, `Duration`... and fail.
- Expecting `@DefaultValue`/field defaults to be visible via `@Value` or `Environment.getProperty` — they aren't; they exist only in the bound object.
- A typo in a key (`shop.chekout-timeout`) is silently ignored — metadata-based IDE warnings help, and so does validation of required fields.
- Leaving out `@Valid` on nested objects, so their constraints never run.

In the interview

Q: `@Value` vs `@ConfigurationProperties`? `@ConfigurationProperties` binds a prefix into a typed, validated object with relaxed binding and IDE metadata — use it for groups of settings. `@Value` injects single values and supports SpEL.

- *Follow-up: How do you make configuration immutable?* Use a record (constructor binding) — no setters.
- *Follow-up: How do you fail fast on bad config?* `@Validated` + constraints; Boot refuses to start and prints each invalid property with its origin.

TL;DR

Group settings into a `@ConfigurationProperties` **record**, register it with `@ConfigurationPropertiesScan`, use `Duration`/`DataSize` types, `@DefaultValue` for defaults, `@Validated` to fail fast, and the configuration processor for IDE help.

3.7 Profiles, profile-specific files and profile groups

What it is

A **profile** is a named environment ("dev", "test", "prod"). Activating it (Chapter 2 §2.11) does two things:

1. registers beans marked `@Profile("...")`, and
2. loads **profile-specific configuration**: `application-{profile}.properties`/`.yaml` files and documents with `spring.config.activate.on-profile`.

```yaml
# application-dev.yaml — loaded only when "dev" is active; overrides application.yaml
shop:
  name: Corner Shop (DEV)
  checkout-timeout: 5m
```

Profile groups

A **profile group** gives one name to several profiles, so operators activate a single word:

```yaml
spring:
  profiles:
    group:
      production: [prod, metrics]     # --spring.profiles.active=production turns on both
```

```
Apps.start("--spring.profiles.active=production")
```

```
active profiles = [production, prod, metrics]
payment         = Payment[gatewayUrl=https://pay.prod.example.test, retries=5]
```

Activating `production` also activated `prod`, so the `on-profile: prod` document in `application.yaml` applied (gateway URL and 5 retries).

Rules worth knowing

Property	Meaning
`spring.profiles.active`	The profiles to activate (comma-separated). Last one listed wins on conflicts between their files.
`spring.profiles.default`	Profiles active when none are set (default: `default`)
`spring.profiles.include`	Profiles always added on top of the active ones
`spring.profiles.group.<name>`	Expand one profile into several

- `spring.profiles.active`, `.default` and `.include` may only appear in **non-profile-specific** documents — you can't activate a profile from inside `application-dev.yaml` (Boot fails with `InvalidConfigDataPropertyException`). Define `spring.profiles.group` there too.
- Profile-specific files always override the non-specific file in the same location.
- In tests: `@ActiveProfiles("test")`.

Common mistakes

- One profile per developer machine (`application-john.yaml`) committed to Git. Use an ignored `config/application.yaml` or env vars.
- Building different jars per environment — the whole point is one jar plus a profile.
- Deeply layered profiles (`dev,eu,feature-x,local`) that nobody can reason about. Keep a small, documented set.

TL;DR

Profiles select beans *and* configuration. `application-{profile}.yaml` overrides `application.yaml`; groups activate several profiles with one name.

3.8 Importing configuration: `spring.config.import`

What it is

`spring.config.import` pulls more configuration into the `Environment` from inside a config file (or from the command line):

```
spring:
  config:
    import:
      - classpath:owner.properties              # must exist
      - optional:file:./local-overrides.yaml    # ignored if missing
```

```
shop.owner from owner.properties = Ada
```

The optional `./local-overrides.yaml` doesn't exist and the app starts anyway.

Location form	Example	Use
`classpath:`	`classpath:owner.properties`	Split a large config into files
`file:`	`file:/etc/myapp/extra.yaml`	Files on the host/volume (storage attached to a container)
`optional:` prefix	`optional:file:./local.yaml`	Don't fail if missing (without it: `ConfigDataLocationNotFoundException`)
`configtree:`	`optional:configtree:/etc/secrets/`	**Kubernetes secrets/ConfigMaps (Kubernetes objects holding secret or plain settings) mounted as files**: each file name becomes a key, its content the value (`/etc/secrets/db/password` → `db.password`)
Extension hint	`file:/etc/app/conf[.yaml]`	Location without a recognisable extension
Other schemes (via libraries)	`configserver:http://...` (Spring Cloud Config), `vault://` (Spring Cloud Vault), `aws-secretsmanager:` / `aws-parameterstore:` (Spring Cloud AWS)	Central config / secret stores (Chapter 16)

Rules: relative locations are resolved relative to the importing file; an imported file **overrides** the file that imports it (seen in §3.3's list); imports are processed once, even if listed twice.

3.9 Placeholders, random values and other sources

- **Placeholders inside property files**: `app.description=${spring.application.name} runs on port ${server.port:8080}`. Refer to other keys in canonical kebab-case.
- **Random values** (row 4 of the precedence table), handy for test secrets or instance ids (`uuid` = universally unique identifier, a random 128-bit id):

  ```
  my.secret=${random.value}
  my.id=${random.uuid}
  my.number=${random.int}
  my.port=${random.int[1024,65536]}
  my.small=${random.int(10)}
  ```

  Each placeholder is evaluated independently at startup; they are not secure random tokens (values generated by a cryptographically strong, unguessable generator) for production secrets.
- `SPRING_APPLICATION_JSON`: a JSON blob in one env var or system property, flattened into properties — convenient where only one variable can be set.
- `@PropertySource("classpath:legacy.properties")` on a `@Configuration` class: plain Spring's way to add a file. Low priority, `.properties` only (no YAML, no profiles). In Boot apps prefer `spring.config.import`.
- **Injecting** `Environment`: `env.getProperty("shop.name")`, `env.getProperty("shop.payment.retries", Integer.class, 3)`, `env.matchesProfiles("prod")`. Useful in conditions and registrars (Chapter 2 §2.14).

3.10 How auto-configuration works

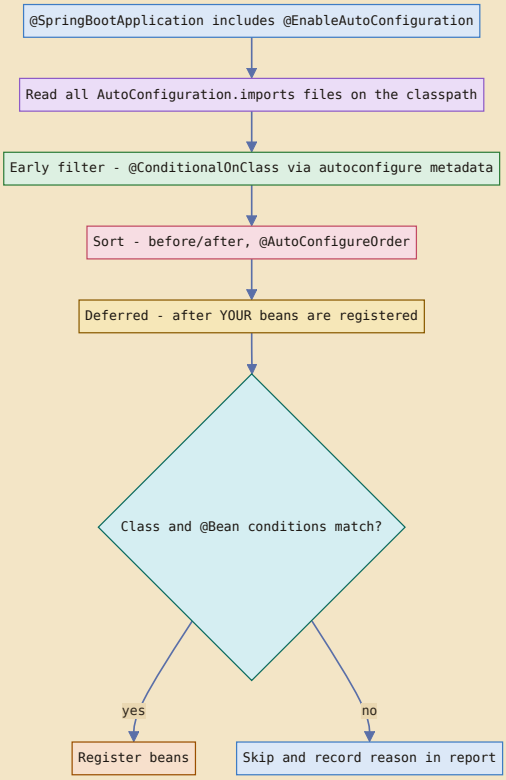
What it is

Auto-configuration is Boot's mechanism for registering beans you'd otherwise have to write yourself — a `DispatcherServlet`, an embedded Tomcat, a Jackson `JsonMapper`, a `DataSource` (hands out database connections), a transaction manager (the object that starts, commits and rolls back transactions) — *only when it makes sense*: the required library is on the classpath, you haven't defined your own bean, and the relevant property allows it.

Each auto-configuration is just a `@Configuration` class (marked `@AutoConfiguration`) whose class and `@Bean` methods carry **conditions**. There's no special magic beyond Chapter 2's `@Conditional` and `@Import`.

How it works

1. `@SpringBootApplication` includes `@EnableAutoConfiguration`, which imports `AutoConfigurationImportSelector`.
2. The selector reads every file named `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` on the classpath. Each jar that provides auto-configuration ships one, listing class names one per line.
3. **Early filtering**: using pre-computed `META-INF/spring-autoconfigure-metadata.properties` (generated by `spring-boot-autoconfigure-processor`), Boot drops candidates whose `@ConditionalOnClass` can't match — without even loading those classes. This keeps startup fast.
4. Candidates are **sorted** by `@AutoConfiguration(before/after = ...)` and `@AutoConfigureOrder`.
5. It is a **deferred** import: auto-configurations are processed **after all your own configuration and scanned beans** are registered. That's what makes `@ConditionalOnMissingBean` reliable — Boot can see whether *you* already defined the bean.
6. Each remaining class's conditions are evaluated; matching classes and `@Bean` methods register beans. Every decision is recorded in the **condition evaluation report** (§3.11).



Here is what Boot 4.1.1's web MVC module ships (from `spring-boot-webmvc-4.1.1.jar`):

```
org.springframework.boot.webmvc.autoconfigure.DispatcherServletAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.WebMvcObservationAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.actuate.endpoint.web.WebMvcHealthEndpointExtensionAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.actuate.web.mappings.WebMvcMappingsAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.error.ErrorMvcAutoConfiguration
```

The condition annotations

All in `org.springframework.boot.autoconfigure.condition`:

Annotation	Matches when	Typical use
`@ConditionalOnClass` / `@ConditionalOnMissingClass`	A class is / isn't on the classpath	"Configure Jackson only if Jackson is present"
`@ConditionalOnBean` / `@ConditionalOnMissingBean`	A bean of a type/name exists / doesn't exist	**Back off** if the user defined their own
`@ConditionalOnSingleCandidate`	Exactly one (or one primary) bean of a type	"Create a `JdbcTemplate` if there's one `DataSource`"
`@ConditionalOnProperty`	A property has a value (`havingValue`), or exists; `matchIfMissing`	Feature switches
`@ConditionalOnBooleanProperty` (Boot 3.5+)	A boolean property is `true` (by default); `matchIfMissing`	`spring.jmx.enabled` (JMX = Java Management Extensions, a built-in JVM monitoring interface)
`@ConditionalOnResource`	A resource exists	`classpath:META-INF/build-info.properties`
`@ConditionalOnWebApplication` / `@ConditionalOnNotWebApplication`	Servlet or reactive web app / not	Web-only beans
`@ConditionalOnWarDeployment` / `@ConditionalOnNotWarDeployment`	Deployed as WAR / not	
`@ConditionalOnThreading`	Platform (classic OS) or virtual (lightweight, JVM-managed) threads selected (`spring.threads.virtual.enabled`)	Executors — thread pools that run tasks (Chapter 14)
`@ConditionalOnCloudPlatform`	Running on Kubernetes, Cloud Foundry (another app-hosting platform), ...	Platform tweaks
`@ConditionalOnJava`	JVM version range	
`@ConditionalOnExpression`	A SpEL expression is true	Last resort — can trigger early bean creation

Common mistakes

- Using `@ConditionalOnBean`/`@ConditionalOnMissingBean` in *your own* (non-auto) configuration classes. They only see beans registered *so far*, so the result depends on processing order. They are reliable in auto-configurations because those run last.
- Putting `@ConditionalOnClass(SomeLib.class)` on a `@Bean` *method* whose return type is from that library. If the class is missing, loading the configuration class itself can fail. Put the condition on a nested `@Configuration` class instead (the Boot docs' recommended pattern).

In the interview

Q: How does Spring Boot auto-configuration work? `@EnableAutoConfiguration` imports a deferred selector that loads class names from every `META-INF/spring/...AutoConfiguration.imports` file, filters by `@ConditionalOnClass` early using generated metadata, sorts them, and — after user configuration is registered — evaluates their conditions. Matching configurations register beans; most beans are `@ConditionalOnMissingBean`, so user beans take priority.

- *Follow-up: Where were auto-configurations listed before?* In `META-INF/spring.factories` under `EnableAutoConfiguration`; the `.imports` file arrived in Boot 2.7 and `spring.factories` stopped working for auto-configuration in Boot 3.0.
- *Follow-up: Why are auto-configurations deferred?* So `@ConditionalOnMissingBean` can see the user's beans and back off.

TL;DR

Auto-configuration = ordinary `@Configuration` classes listed in `AutoConfiguration.imports`, applied last, guarded by `@ConditionalOn...` so they fill gaps and back off when you define your own beans.

3.11 Seeing what Boot decided

The `--debug` report

Start any Boot app with `--debug` (or `debug=true` in `application.properties`) and Boot prints the **CONDITIONS EVALUATION REPORT**. Real excerpt from `java -jar config-demo.jar --debug`:

```
============================
CONDITIONS EVALUATION REPORT
============================

Positive matches:
-----------------

   AopAutoConfiguration matched:
      - @ConditionalOnBooleanProperty (spring.aop.auto=true) matched (OnPropertyCondition)

   TaskExecutionAutoConfiguration matched:
      - @ConditionalOnClass found required class 'org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor' (OnClassCondition)

   ValidationAutoConfiguration matched:
      - @ConditionalOnClass found required class 'jakarta.validation.executable.ExecutableValidator' (OnClassCondition)
      - @ConditionalOnResource found location classpath:META-INF/services/jakarta.validation.spi.ValidationProvider (OnResourceCondition)

   ValidationAutoConfiguration#defaultValidator matched:
      - @ConditionalOnMissingBean (types: jakarta.validation.Validator; SearchStrategy: all) did not find any beans (OnBeanCondition)
   ...

Negative matches:
-----------------

   AopAutoConfiguration.AspectJAutoProxyingConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class 'org.aspectj.weaver.Advice' (OnClassCondition)

   JmxAutoConfiguration:
      Did not match:
         - @ConditionalOnBooleanProperty (spring.jmx.enabled=true) did not find property 'spring.jmx.enabled' (OnPropertyCondition)
      Matched:
         - @ConditionalOnClass found required class 'org.springframework.jmx.export.MBeanExporter' (OnClassCondition)

   TaskExecutorConfigurations.TaskExecutorConfiguration#applicationTaskExecutorVirtualThreads:
      Did not match:
         - @ConditionalOnThreading did not find VIRTUAL (OnThreadingCondition)
   ...

Exclusions:
-----------

    None

Unconditional classes:
----------------------

    org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration
    org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration
    ...
```

How to read it:

- **Positive matches** — applied. `Class#method` entries are individual `@Bean` methods.
- **Negative matches** — skipped, with the failing condition (and any that matched). This is where you look when "Boot didn't create X": the missing class, bean or property is named.
- **Exclusions** — auto-configurations you excluded explicitly.
- **Unconditional classes** — auto-configurations with no class-level condition.

Note `ValidationAutoConfiguration` matched only because `spring-boot-starter-validation` put Hibernate Validator on the classpath — that's auto-configuration reacting to a dependency.

The same report in code and over HTTP

The report is a bean-factory object you can query, which the demo does in a test:

```
var report = ConditionEvaluationReport.get(ctx.getBeanFactory());
var outcomes = report.getConditionAndOutcomesBySource().get("...JmxAutoConfiguration");
```

```
TaskExecutionAutoConfiguration  MATCHED
   - @ConditionalOnClass found required class 'org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor'
JmxAutoConfiguration  DID NOT MATCH
   - @ConditionalOnClass found required class 'org.springframework.jmx.export.MBeanExporter'
   - @ConditionalOnBooleanProperty (spring.jmx.enabled=true) did not find property 'spring.jmx.enabled'
AopAutoConfiguration  MATCHED
   - @ConditionalOnBooleanProperty (spring.aop.auto=true) matched
auto-configuration classes evaluated: 7
```

With **Actuator** (Chapter 15), `GET /actuator/conditions` returns the same report as JSON (expose it with `management.endpoints.web.exposure.include=conditions`; keep it off public networks). `/actuator/configprops` shows all bound `@ConfigurationProperties` beans and `/actuator/env` the property sources.

3.12 Overriding and excluding auto-configuration

In order of preference:

1. **Set a property.** Most auto-configurations are tuned by properties (`server.port`, `spring.jackson.*`, `spring.datasource.*`). The reference's "Common Application Properties" appendix lists them all; your IDE completes them from the metadata.
2. **Define your own bean.** Auto-configured beans are `@ConditionalOnMissingBean`, so your bean of the same type replaces Boot's (proved with the starter in §3.13).
3. **Use a customizer.** Many auto-configurations accept customizer beans to *adjust* rather than replace (e.g. `JsonMapperBuilderCustomizer` for Jackson 3, `WebServerFactoryCustomizer` for Tomcat).
4. **Exclude an auto-configuration entirely:**

```
@SpringBootApplication(exclude = JmxAutoConfiguration.class)
```

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jmx.JmxAutoConfiguration
```

5. **Remove the dependency.** If you don't use it, don't put the starter on the classpath.

Common mistakes

- Excluding by class name copied from a Boot 3 tutorial — in Boot 4 most auto-configuration classes moved to per-module packages (e.g. `org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration`, previously `org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration`). Check the report for the current name.
- Adding `@EnableWebMvc` to a Boot app "to configure MVC" — it tells Boot you want full manual control and switches off Boot's MVC auto-configuration (Chapter 5).

3.13 Writing your own auto-configuration and starter

Why

Companies write starters so every service gets the same client library, tracing setup (recording a request's path through several services) or security defaults with one dependency — for example an internal `payments-client-spring-boot-starter`. Open-source examples: `mybatis-spring-boot-starter`, `springdoc-openapi-starter-webmvc-ui` (generates OpenAPI — a standard machine-readable API description — and a docs UI).

The structure

The Boot 4.1 reference recommends two modules (merge them into one `acme-spring-boot-starter` if the auto-configuration is simple):

Module	Name pattern	Contains
Auto-configure module	`acme-spring-boot`	`@AutoConfiguration` classes, `@ConfigurationProperties`, the `.imports` file
Starter	`acme-spring-boot-starter`	No code; depends on the auto-configure module, the library, and `spring-boot-starter`
(Optional) test starter	`acme-spring-boot-starter-test`	Test support

Never start your module names with `spring-boot` — that namespace is reserved for official modules — and use your own property prefix (`acme.*`), not `spring.*`/`server.*`.

Our example wires a tiny `GreetingService` library:

```
greeter/
├── pom.xml                        ← aggregator: builds the modules together
└── modules/
    ├── autoconfig/                ← auto-configure module (artifactId greeting-spring-boot)
    │   ├── src/main/java/com/example/greeting/core/GreetingService.java
    │   ├── src/main/java/com/example/greeting/autoconfigure/GreetingProperties.java
    │   ├── src/main/java/com/example/greeting/autoconfigure/GreetingAutoConfiguration.java
    │   └── src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
    ├── starter/pom.xml            ← dependencies only (artifactId greeting-spring-boot-starter)
    └── demo/                      ← an application that uses the starter
```

(In real life `GreetingService` would be a separate library jar; it's in the same module here to keep the demo small.)

The code

```
/** The "library" the starter wires up. It knows nothing about Spring. */
public class GreetingService {
    private final String template;
    public GreetingService(String template) { this.template = template; }
    public String greet(String name) { return template.formatted(name); }
}

/**
 * @param enabled  whether to auto-configure a GreetingService
 * @param template message template; %s is replaced by the name
 */
@ConfigurationProperties("greeting")
public record GreetingProperties(
        @DefaultValue("true") boolean enabled,
        @DefaultValue("Hello, %s!") String template) {
}

@AutoConfiguration                                          // a @Configuration(proxyBeanMethods = false) for Boot
@ConditionalOnClass(GreetingService.class)                  // 1. only if the library is on the classpath
@ConditionalOnBooleanProperty(name = "greeting.enabled", matchIfMissing = true) // 2. unless switched off
@EnableConfigurationProperties(GreetingProperties.class)    // 3. bind greeting.* into GreetingProperties
public class GreetingAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                               // 4. back off if the user defined their own
    GreetingService greetingService(GreetingProperties props) {
        return new GreetingService(props.template());
    }
}
```

```
# src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.greeting.autoconfigure.GreetingAutoConfiguration
```

The auto-configure module's `pom.xml` depends on `spring-boot-autoconfigure` and declares two annotation processors: `spring-boot-configuration-processor` (property metadata) and `spring-boot-autoconfigure-processor`, which generated this file for Boot's early class filtering (§3.10 step 3):

```
# target/classes/META-INF/spring-autoconfigure-metadata.properties (generated)
com.example.greeting.autoconfigure.GreetingAutoConfiguration=
com.example.greeting.autoconfigure.GreetingAutoConfiguration.ConditionalOnClass=com.example.greeting.core.GreetingService
```

The starter's `pom.xml` is just:

```
<artifactId>greeting-spring-boot-starter</artifactId>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>com.example.greeting</groupId>
    <artifactId>greeting-spring-boot</artifactId>
    <version>${project.version}</version>
  </dependency>
</dependencies>
```

Testing auto-configuration with `ApplicationContextRunner`

`ApplicationContextRunner` builds a tiny, throw-away context per test with exactly the configuration, properties and classpath you specify — much faster and more precise than `@SpringBootTest`. Its AssertJ (a fluent test-assertion library) assertions (`hasSingleBean`, `doesNotHaveBean`, `hasBean`) read naturally.

```
private final ApplicationContextRunner runner = new ApplicationContextRunner()
        .withConfiguration(AutoConfigurations.of(GreetingAutoConfiguration.class));

@Test void createsTheBeanByDefault() {
    runner.run(ctx -> assertThat(ctx).hasSingleBean(GreetingService.class));
}
@Test void propertiesCustomiseIt() {
    runner.withPropertyValues("greeting.template=Welcome back, %s.").run(ctx -> ...);
}
@Test void canBeSwitchedOff() {
    runner.withPropertyValues("greeting.enabled=false").run(ctx -> assertThat(ctx).doesNotHaveBean(GreetingService.class));
}
@Configuration(proxyBeanMethods = false)
static class UserConfig { @Bean GreetingService myGreeting() { return new GreetingService("Yo %s"); } }

@Test void backsOffWhenTheUserDefinesOne() {
    runner.withUserConfiguration(UserConfig.class).run(ctx ->
            assertThat(ctx).hasSingleBean(GreetingService.class).hasBean("myGreeting"));
}
@Test void skippedWhenLibraryIsMissing() {
    runner.withClassLoader(new FilteredClassLoader(GreetingService.class))   // pretend the jar is absent
          .run(ctx -> assertThat(ctx).doesNotHaveBean("greetingService"));
}
```

Output (each test prints what it saw):

```
default         -> Hello, Ada!
with property   -> Welcome back, Ada.
enabled=false   -> GreetingService beans: 0
user bean       -> Yo Ada (bean name: myGreeting)
class missing   -> has greetingService? false
```

All five behaviours are proved: default bean, property binding, the on/off switch, backing off to the user's bean, and skipping when the library is absent.

Using the starter

The demo app has one dependency on `greeting-spring-boot-starter`, one property, and **no** `@Bean` or `@Import` for `GreetingService`:

```
# modules/demo/src/main/resources/application.properties
greeting.template=Good morning, %s!
```

```
@SpringBootTest
class DemoApplicationTests {
    @Autowired GreetingService greetings;          // provided by the starter's auto-configuration
    ...
}
```

```
app says: Good morning, Ada!
GreetingAutoConfiguration matched? true
```

Checklist for a good starter

1. Guard the class with `@ConditionalOnClass` of the library's key type; guard beans with `@ConditionalOnMissingBean`.
2. Put `@ConditionalOnClass` for *optional* integrations on nested `@Configuration(proxyBeanMethods = false)` classes.
3. Use `@ConfigurationProperties` with your own prefix and Javadoc on every property (it becomes IDE help).
4. Order relative to Boot's auto-configurations with `@AutoConfiguration(after = ...)` when you depend on their beans.
5. Test with `ApplicationContextRunner` (and `WebApplicationContextRunner` for web-specific parts).
6. Keep the starter free of code and of optional dependencies.

In the interview

Q: How would you create a custom Spring Boot starter? Write an `@AutoConfiguration` class with `@ConditionalOnClass`/`@ConditionalOnMissingBean`/property conditions and a `@ConfigurationProperties` record, list it in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, package it as `acme-spring-boot`, and publish an `acme-spring-boot-starter` pom that depends on it and the library. Test with `ApplicationContextRunner`.

- *Follow-up: How does a user override your bean?* By defining a bean of the same type; `@ConditionalOnMissingBean` makes yours back off.

3.14 Boot 4 modularization

What changed

Up to Boot 3.5, one jar — `spring-boot-autoconfigure` — held auto-configuration for *every* supported technology (156 auto-configuration classes in 3.5.6). **Boot 4 split it into one module per technology.** In 4.1.1, `spring-boot-autoconfigure` itself lists only 12 core auto-configurations (AOP, task execution/scheduling, JMX, SSL (encrypted connections), lifecycle, message source, property placeholders...). The rest moved to modules such as:

Module jar	Auto-configurations (4.1.1)	Example
`spring-boot-webmvc`	6	`org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration`
`spring-boot-tomcat`	5	`...tomcat.autoconfigure.servlet.TomcatServletWebServerAutoConfiguration`
`spring-boot-jackson`	1	`...jackson.autoconfigure.JacksonAutoConfiguration`
`spring-boot-jdbc`	9	DataSource, JdbcTemplate/JdbcClient, transactions
`spring-boot-actuator-autoconfigure`	24	Actuator endpoints
`spring-boot-micrometer-metrics`	28	Metrics

Why it matters to you

- **Smaller classpath, faster and more predictable startup**: only modules for technologies you use are present, so fewer candidates to filter and no accidental auto-configuration from a stray transitive library (one pulled in indirectly by another dependency).
- **Clearer ownership**: each module has its own packages, its own starter and its own `-test` starter (`@WebMvcTest` now lives in `spring-boot-webmvc-test`, `@DataJpaTest` in `spring-boot-data-jpa-test`).
- **Migration impact**: imports of auto-configuration classes (in `exclude = ...`, `@ImportAutoConfiguration`, `@AutoConfiguration(after = ...)`) and of test-slice annotations change package. The quick path is `spring-boot-starter-classic` / `spring-boot-starter-test-classic`, which restore the Boot 3-style "everything on the classpath" behaviour while you move to the modular starters. Full migration notes: Chapter 17.

> **TL;DR**
>
> Boot 4 = one auto-configuration module + starter + test starter per technology. The mechanism (imports file + conditions) is unchanged; class packages moved.

3.15 In the interview — quick round

Question	Short answer
Name the main configuration sources in priority order.	Command line > `SPRING_APPLICATION_JSON` > system properties > env vars > `random.*` > profile files (outside > inside jar) > `application.*` > `@PropertySource` > defaults. Tests on top. (§3.3)
How does an env var map to a property?	`.`→`_`, drop `-`, upper-case: `SHOP_CHECKOUTTIMEOUT` → `shop.checkout-timeout`. (§3.4)
`@Value` vs `@ConfigurationProperties`?	Single value + SpEL vs typed, validated, relaxed, documented group. (§3.6)
How do you validate configuration?	`@Validated` + Jakarta constraints (+ `@Valid` for nested); app fails to start with each bad property and origin. (§3.6)
Profile-specific config?	`application-{profile}.yaml`, `on-profile` documents; groups activate several profiles. (§3.7)
Kubernetes secrets as config?	`spring.config.import=optional:configtree:/etc/secrets/` (or env vars). (§3.8)
How does auto-configuration work?	`.imports` files → early class filter → sort → deferred → conditions → beans; `@ConditionalOnMissingBean` lets you override. (§3.10)
Why didn't Boot create bean X?	Run with `--debug` or `/actuator/conditions`; read the negative match. (§3.11)
How do you turn off one auto-configuration?	`exclude =` on `@SpringBootApplication` or `spring.autoconfigure.exclude`. (§3.12)
How do you write a starter?	`@AutoConfiguration` + conditions + properties in `acme-spring-boot`, listed in `.imports`; `acme-spring-boot-starter` pom; test with `ApplicationContextRunner`. (§3.13)
What did Boot 4 modularize?	Auto-configuration split into per-technology modules with their own starters and test starters. (§3.14)

Chapter TL;DR

- All configuration flows through the **`Environment`**: an ordered list of property sources; the first source with a key wins.
- **Precedence** (high → low): tests > command line > `SPRING_APPLICATION_JSON` > system properties > env vars > profile-specific files > `application.*` (outside jar > inside; imported file > importer) > `@PropertySource` > defaults.
- **Env vars** use relaxed binding: `SHOP_PAYMENT_RETRIES` → `shop.payment.retries`; drop dashes.
- Prefer **`@ConfigurationProperties`** **records** (with `@ConfigurationPropertiesScan`, `@DefaultValue`, `Duration`/`DataSize`, `@Validated`) over scattered `@Value`s. Add the configuration processor for IDE metadata.
- **Profiles** select beans and config files; **profile groups** activate several at once; `spring.config.activate.on-profile` makes multi-document files conditional.
- **`spring.config.import`** adds files, optional files, and `configtree:` directories (Kubernetes secrets).
- **Auto-configuration** = `@AutoConfiguration` classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, processed after your beans, guarded by `@ConditionalOnClass`/`OnMissingBean`/`OnProperty`/`OnBooleanProperty`...
- Debug it with `--debug` (conditions report) or `/actuator/conditions`; override with properties, your own bean, customizers, or `exclude`.
- A **custom starter** = auto-configure module + code-free starter pom; test with `ApplicationContextRunner`.
- **Boot 4** split auto-configuration into per-technology modules (`spring-boot-webmvc`, `spring-boot-jdbc`, ...), each with a starter and a `-test` starter.

Next: Chapter 4 — AOP and Proxies explains the proxies you met in Chapter 2 and how `@Transactional`, `@Cacheable` and `@Async` are built on them.

Chapter 4 — AOP and Proxies

Many things an application must do are not part of any single business rule: logging, timing, transactions (groups of database changes that are saved together or not at all), caching (keeping recent results in memory so repeat requests are fast), security checks, retries (trying a failed call again). If you write that code by hand inside every service method, the business logic drowns in it and one forgotten line becomes a bug. **Aspect-Oriented Programming (AOP)** lets you write such a concern once and declare *where* it applies. Spring implements AOP with **proxies** — stand-in objects that wrap your beans — and the same proxy mechanism is what makes `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize` and the new Framework 7 `@Retryable` work.

Understanding proxies is the single most useful piece of "Spring internals" knowledge: it explains the most common Spring bug of all (an annotation that silently does nothing) and it comes up in almost every Spring interview.

What you'll learn

1. The problem AOP solves and its vocabulary: aspect, advice, pointcut, join point, target, proxy, weaving.
2. Writing aspects with `@Aspect`: the five advice types, binding arguments, ordering several aspects.
3. The pointcut expression language (`execution`, `within`, `@annotation`, `bean`, ...).
4. How Spring creates proxies: JDK dynamic proxies (Java's built-in runtime proxies, which implement interfaces) vs CGLIB subclasses (classes generated at runtime that extend your class), Boot's default, Framework 7's `@Proxyable`.
5. Proxy limitations, **proved by tests**: self-invocation, `final` methods, `private` methods — and the fixes.
6. Which Spring features are proxy-based and what that means for you.
7. Framework 7 core resilience (keeping an app working when things it depends on fail): `@Retryable`, `@ConcurrencyLimit`, `RetryTemplate`.
8. Spring AOP vs full AspectJ weaving, and when not to use AOP at all.

All code lives in `code/aop-proxies/` — a Spring Boot 4.1.1 / Framework 7.0 project on Java 25. Every claim marked "proved" is checked by a JUnit (Java's standard test framework) test; the output shown is real `mvn -q verify` output (Spring's log lines trimmed as `...`).

> **Prerequisites.** Beans, the `ApplicationContext` and `BeanPostProcessor` are explained in Chapter 2. The Proxy *design pattern* itself (without Spring) is covered in the LLD handbook: Proxy Pattern.

4.1 Cross-cutting concerns and the AOP vocabulary

The problem it solves

Imagine an `OrderService` with ten methods, and three requirements: log each call, measure how long it takes, and run it in a database transaction. Without AOP every method looks like this:

```
public String placeOrder(String item, int qty) {
    log.info("placeOrder {} {}", item, qty);          // logging
    long start = System.nanoTime();                    // timing
    TransactionStatus tx = txManager.getTransaction(def);  // transaction
    try {
        String id = /* ... the 3 lines that actually matter ... */;
        txManager.commit(tx);
        return id;
    } catch (RuntimeException e) {
        txManager.rollback(tx);
        throw e;
    } finally {
        metrics.record("placeOrder", System.nanoTime() - start);
    }
}
```

The real logic is three lines; the rest is copied into every method. Such code is called a **cross-cutting concern**: a requirement that *cuts across* many classes instead of living in one. Two symptoms give it away:

- **Scattering** — the same code is spread over many classes.
- **Tangling** — each method mixes several unrelated concerns.

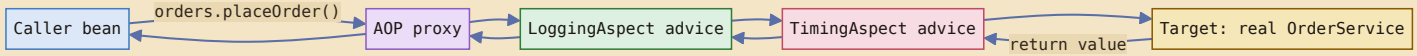
What it is

AOP is a way to modularise cross-cutting concerns. You write the concern once in an **aspect**, and describe with a **pointcut** which methods it applies to. The framework then applies it for you.

Analogy: airport security. Passengers (method calls) go to many different gates (business methods), but every one of them passes through the same security checkpoint first. Gates do not each run their own security; the checkpoint is defined once and placed in the path to all of them.

The vocabulary

Term	Meaning	In this chapter's code
Join point	A point during execution where extra behaviour *could* be added. In Spring AOP this is always **a method execution on a bean**.	the call `orders.placeOrder("book", 2)`
Pointcut	A predicate (a yes/no test) that selects join points.	`execution(public * ch04.aop.OrderService.*(..))`
Advice	The code to run at the selected join points, plus *when* (before, after, around...).	`LoggingAspect.before(...)`
Aspect	A class that bundles pointcuts and advice for one concern.	`LoggingAspect`, `TimingAspect`
Target object	The real object being advised.	the plain `OrderService` instance
AOP proxy	The object Spring creates that wraps the target and runs the advice. Other beans receive the proxy.	`OrderService$$SpringCGLIB$$0`
Weaving	Linking aspects with targets. Spring does it **at runtime by creating proxies**; AspectJ (the original, full AOP tool for Java) can do it at compile time (when `javac` builds the code) or class-load time (when the JVM first loads a class).	done by a `BeanPostProcessor` at startup
Advisor	Spring's internal pairing of one advice with one pointcut. You rarely write these by hand.	—
Introduction	Adding a new interface to an advised object. Rarely used.	—



The caller *thinks* it holds an `OrderService`. It actually holds the proxy, which runs the advice chain and eventually calls the real object.

Self-check

1. What are the two symptoms of a cross-cutting concern? → Scattering (same code in many places) and tangling (several concerns mixed in one method).
2. What is a join point in Spring AOP? → Only a method execution on a Spring bean (not field access, not constructors).
3. Pointcut vs advice? → The pointcut says *where*, the advice says *what* and *when*.

4.2 Writing aspects with `@Aspect`

How to set it up

1. Add the starter. In **Boot 4** it is `spring-boot-starter-aspectj` (it was called `spring-boot-starter-aop` up to Boot 3.x and was renamed to make clear that it brings AspectJ's annotation support):

   ```
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-aspectj</artifactId>
   </dependency>
   ```

2. Boot's `AopAutoConfiguration` sees AspectJ on the classpath (the set of jars the app runs with) and switches on `@EnableAspectJAutoProxy` for you (property `spring.aop.auto`, default `true`). In a plain Spring (non-Boot) app you add `@EnableAspectJAutoProxy` to a `@Configuration` class yourself.
3. Write a class annotated **both** `@Aspect` and `@Component` (or register it with `@Bean`). `@Aspect` alone is *not* a stereotype (one of the role annotations like `@Component`/`@Service` that scanning looks for) — component scanning ignores it, and an aspect that is not a bean never runs.

The five kinds of advice

Annotation	Runs	Can it change the outcome?	Typical use
`@Before`	before the method	only by throwing an exception (the method then never runs)	argument checks, audit "who called"
`@AfterReturning`	after a normal return	can read the return value (bind with `returning=`), cannot replace it	log results, publish an event
`@AfterThrowing`	after the method throws	can read the exception (`throwing=`); the exception still propagates (continues up to the caller) unless the advice throws a different one	translate/log errors
`@After`	always, like `finally`	no	release resources, clear context
`@Around`	wraps the call; *you* call `proceed()`	yes: skip the call, change arguments, change or replace the return value, catch exceptions, retry	timing, caching, transactions, retries

Rule of thumb from the Spring docs: use the **least powerful** advice that does the job. A `@Before` cannot forget to call `proceed()`; an `@Around` can.

Example: logging aspect

The business class knows nothing about logging (`OrderService.java`):

```
@Service
public class OrderService {
    public String placeOrder(String item, int quantity) {
        return "order-" + item + "-x" + quantity;
    }
    public void cancel(String orderId) {
        if (orderId.startsWith("unknown")) {
            throw new IllegalArgumentException("no such order: " + orderId);
        }
    }
    @Timed("monthly-report")
    public String monthlyReport() throws InterruptedException {
        Thread.sleep(20);                  // pretend to do slow work
        return "report ready";
    }
}
```

The aspect (`LoggingAspect.java`) writes into a small `CallLog` bean so a test can check the exact order:

```
@Aspect                 // "this class contains advice + pointcuts"
@Component              // ...and it must also be a bean, or Spring never sees it
@Order(1)               // lower number = outer layer = runs first on the way in
public class LoggingAspect {
    private final CallLog log;
    public LoggingAspect(CallLog log) { this.log = log; }

    // A named, reusable pointcut: every public method of OrderService
    @Pointcut("execution(public * ch04.aop.OrderService.*(..))")
    void orderServiceMethods() {}

    @Before("orderServiceMethods()")
    public void before(JoinPoint jp) {
        log.add("before " + jp.getSignature().getName() + " args=" + Arrays.toString(jp.getArgs()));
    }

    @AfterReturning(pointcut = "orderServiceMethods()", returning = "result")
    public void afterReturning(JoinPoint jp, Object result) {
        log.add("afterReturning " + jp.getSignature().getName() + " -> " + result);
    }

    @AfterThrowing(pointcut = "orderServiceMethods()", throwing = "ex")
    public void afterThrowing(JoinPoint jp, Exception ex) {
        log.add("afterThrowing " + jp.getSignature().getName() + " -> " + ex.getMessage());
    }

    @After("orderServiceMethods()")   // like 'finally': runs on success AND on exception
    public void after(JoinPoint jp) {
        log.add("after " + jp.getSignature().getName());
    }
}
```

What each part does:

- `@Pointcut` on an empty method gives the expression a **name** (`orderServiceMethods()`) that the advice annotations reuse. The method body is never executed.

- `JoinPoint` gives access to the method signature (its name, parameter types and return type), the arguments (`getArgs()`), the target (`getTarget()`) and the proxy (`getThis()`).
- `returning = "result"` binds the return value to the parameter named `result`. If you declare the parameter as `String` instead of `Object`, the advice only matches methods returning a `String`.
- `throwing = "ex"` does the same for the exception; the parameter type also filters (here: only `Exception` and subclasses).

The test (`AspectsTest.java`) calls the service and prints the log:

```
@Test
void adviceRunsAroundASuccessfulCall() {
    String result = orders.placeOrder("book", 2);
    System.out.println("result = " + result);
    log.lines().forEach(l -> System.out.println("  " + l));
    assertThat(log.lines()).containsExactly(
            "before placeOrder args=[book, 2]",
            "afterReturning placeOrder -> order-book-x2",
            "after placeOrder");
}

@Test
void adviceRunsWhenTheMethodThrows() {
    assertThatThrownBy(() -> orders.cancel("unknown-7"))
            .isInstanceOf(IllegalArgumentException.class);   // the aspect does not swallow it
    log.lines().forEach(l -> System.out.println("  " + l));
    assertThat(log.lines()).containsExactly(
            "before cancel args=[unknown-7]",
            "afterThrowing cancel -> no such order: unknown-7",
            "after cancel");
}
```

Output:

```
result = order-book-x2
  before placeOrder args=[book, 2]
  afterReturning placeOrder -> order-book-x2
  after placeOrder
  before cancel args=[unknown-7]
  afterThrowing cancel -> no such order: unknown-7
  after cancel
```

Notice that `@After` ran **after** `@AfterReturning`/`@AfterThrowing`. Since Spring Framework 5.2.7, advice methods of the *same* aspect on the *same* join point have a fixed precedence — `@Around`, `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing` from highest to lowest — and an `@After` method is effectively invoked after the returning/throwing advice of that aspect, just like a `finally` block.

`@Around` advice and custom annotations

`@Around` receives a `ProceedingJoinPoint`. Calling `proceed()` continues to the next advice or the real method; not calling it skips the method entirely. A very common pattern is to trigger an aspect with your own annotation:

```
@Retention(RetentionPolicy.RUNTIME)  // must be visible at runtime for the pointcut to see it
@Target(ElementType.METHOD)
public @interface Timed {
    String value() default "";       // optional label
}
```

```
@Aspect
@Component
@Order(2)               // inner layer: runs after LoggingAspect's @Before
public class TimingAspect {
    private final CallLog log;
    public TimingAspect(CallLog log) { this.log = log; }

    // Matches any bean method annotated with @Timed and binds the annotation to 'timed'
    @Around("@annotation(timed)")
    public Object time(ProceedingJoinPoint pjp, Timed timed) throws Throwable {
        long start = System.nanoTime();
        log.add("timer start [" + timed.value() + "]");
        try {
            return pjp.proceed();          // call the next advice / the real method
        } finally {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.add("timer stop [" + timed.value() + "] took>=20ms? " + (ms >= 20));
        }
    }
}
```

- `@annotation(timed)` uses the *parameter name* `timed`; its type `Timed` tells AspectJ which annotation to look for, and the annotation instance is passed in, so the advice can read `timed.value()`.
- The advice **must return** what `proceed()` returned (or something else deliberately). An `@Around` method declared `void`, or one that forgets `return`, makes every advised method return `null` — a classic bug.

- Always declare `throws Throwable` and let exceptions pass through unless you intend to handle them.

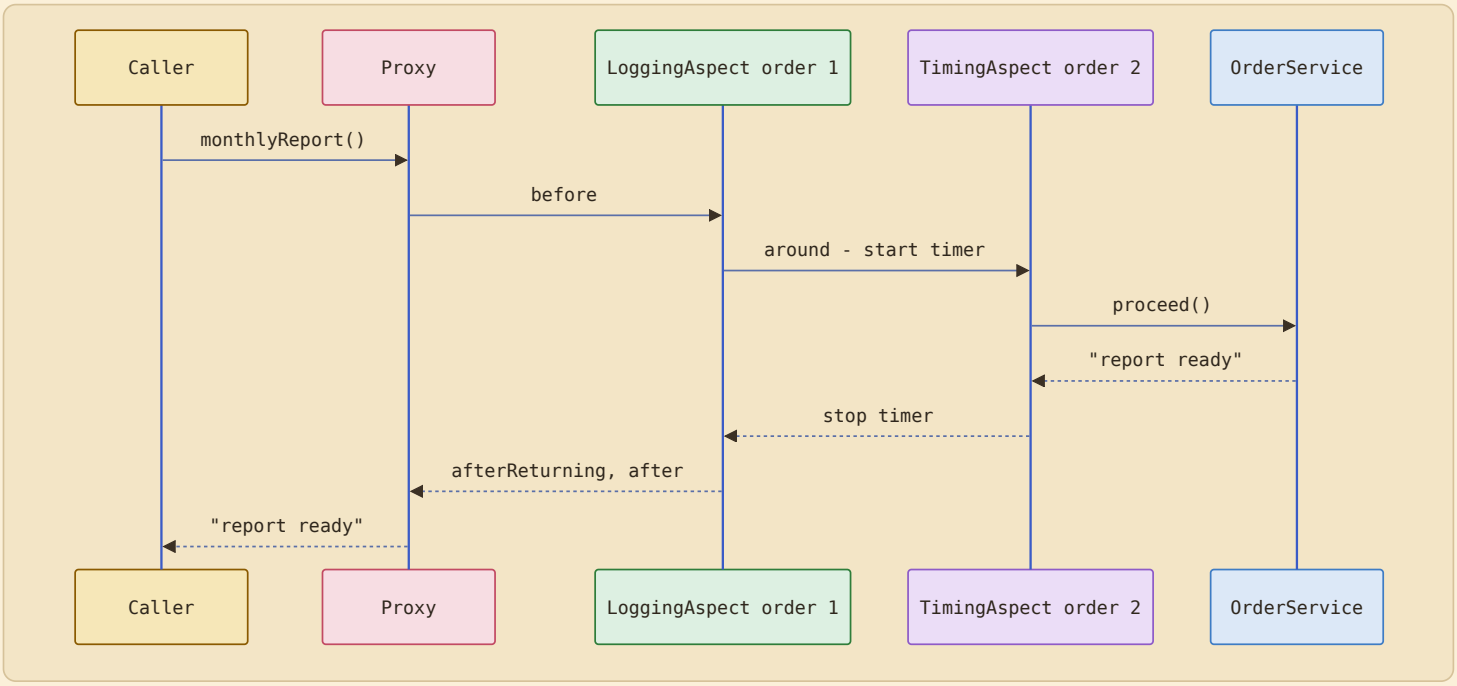
Ordering several aspects

When two aspects advise the same method, Spring orders them with `@Order` (or by implementing `Ordered`). Lower value = higher precedence = **outer** layer: it runs first on the way in and last on the way out. Without an explicit order the sequence is undefined.

```
@Test
void twoAspectsNestByOrder() throws Exception {
    orders.monthlyReport();
    log.lines().forEach(l -> System.out.println("  " + l));
    assertThat(log.lines()).containsExactly(
            "before monthlyReport args=[]",        // @Order(1) LoggingAspect: outer
            "timer start [monthly-report]",        // @Order(2) TimingAspect: inner
            "timer stop [monthly-report] took>=20ms? true",
            "afterReturning monthlyReport -> report ready",
            "after monthlyReport");
}
```

Output:

```
  before monthlyReport args=[]
  timer start [monthly-report]
  timer stop [monthly-report] took>=20ms? true
  afterReturning monthlyReport -> report ready
  after monthlyReport
```



This matters in real systems. Spring's own infrastructure advisors (transactions, caching, security) have orders too; for example, if you write a retry aspect, it must be *outside* the transaction aspect so each retry gets a fresh transaction.

Common mistakes

- **Forgetting** `@Component` on the aspect → nothing happens, no error.
- `@Around` **without** `return pjp.proceed()` → methods return `null`, or the real method never runs.
- **Swallowing exceptions** in `@Around` → callers think a failed operation succeeded, and `@Transactional` further out does not roll back.
- **Pointcut too broad** (`execution(* *(..))`) → Spring tries to proxy every bean, including infrastructure beans; startup slows or fails with odd errors. Always restrict to your packages.
- **Relying on unordered aspects** → inconsistent behaviour between runs or environments.

Self-check

1. Which advice type can prevent the target method from running without throwing? → Only `@Around` (by not calling `proceed()`).
2. Does `@AfterThrowing` stop the exception? → No. The exception continues to the caller unless the advice itself throws another one.
3. Aspect A has `@Order(1)` and B `@Order(2)`. Whose `@Before` runs first? → A's; A is the outer layer.
4. Why must `Timed` have `RetentionPolicy.RUNTIME`? → Spring inspects annotations with reflection (Java's API for examining classes and methods while the program runs) at runtime; `CLASS`/`SOURCE` retention makes them invisible, so the pointcut never matches.

4.3 Pointcut expressions

What it is

A pointcut expression is a tiny query language (from AspectJ) that selects methods. Spring AOP supports a subset of AspectJ's **pointcut designators** (PCDs):

Designator	Matches method executions ...	Example
`execution(...)`	whose signature matches a pattern (the main one)	`execution(* com.shop.service.*.*(..))`
`within(type-pattern)`	declared in matching types/packages	`within(com.shop..*)`
`this(Type)`	where the **proxy** is an instance of `Type`	`this(com.shop.Auditable)`
`target(Type)`	where the **target** is an instance of `Type`	`target(com.shop.Repository)`
`args(...)`	whose runtime arguments match (and can bind them)	`args(id, ..)`
`@annotation(A)`	on methods annotated with `A`	`@annotation(com.shop.Timed)`
`@within(A)`	in classes annotated with `A`	`@within(org.springframework.stereotype.Service)`
`@target(A)`	where the target's runtime class has `A`	`@target(com.shop.Audited)`
`@args(A)`	whose argument types carry annotation `A`	`@args(com.shop.Sensitive)`
`bean(name)`	on beans with a matching name — **Spring-only**, not AspectJ	`bean(*Service)`

The `execution` pattern, piece by piece

```
execution( modifiers?  return-type  declaring-type?.method-name(params)  throws? )
execution( public      *            com.shop.service..*Service.find*(Long, ..) )
```

Part	Meaning in the example
`public`	optional; only public methods (with CGLIB proxies, protected/package-private — no modifier, visible only inside the same package — methods can also be matched)
`*`	any return type
`com.shop.service..`	the package `com.shop.service` **or any sub-package** (`..` in a type pattern)
`*Service`	any type whose name ends in `Service`
`find*`	method names starting with `find`
`(Long, ..)`	first parameter `Long`, then zero or more of anything (`..` in a parameter list). `()` = no params, `(*)` = exactly one param of any type

Combining and reusing

Pointcuts combine with `&&`, `||`, `!` and can reference named pointcuts, even from other classes:

```
public class CommonPointcuts {
    @Pointcut("within(com.shop..web..*)")     public void webLayer() {}
    @Pointcut("within(com.shop..service..*)") public void serviceLayer() {}
    @Pointcut("execution(* com.shop..*Repository.*(..))") public void dataAccess() {}
}

@Before("com.shop.aop.CommonPointcuts.serviceLayer() && args(id, ..)")
public void checkId(Long id) { /* id is bound from the first argument */ }
```

The chapter's `TracingAspect` shows a `within` pointcut that excludes the aspect's own package class:

```
@Before("within(ch04.proxy..*) && !within(ch04.proxy.TracingAspect)")
public void trace(JoinPoint jp) { log.add("trace " + jp.getSignature().toShortString()); }
```

Performance tip

AspectJ matching is done once per bean method at startup and then cached, so runtime cost is small — but **kinded** designators (`execution`) and **scoping** designators (`within`) are cheapest to evaluate. Contextual ones (`this`, `target`, `args`, `@annotation`) are more expensive on their own. A good pointcut includes a `within(...)` scope whenever possible.

Common mistakes

- `com.shop.*` matches only types directly in `com.shop`; use `com.shop..*` for sub-packages.
- `execution(* save(*))` matches exactly one parameter, not "any parameters" — that is `(..)`.

- Putting a custom trigger annotation only on an **interface** method and expecting `@annotation(...)` to match: matching is done against the method actually executed on the bean class, so put custom trigger annotations on the implementation method to be safe. (Spring's own annotations such as `@Transactional` use their own, more forgiving lookup.)

Self-check

1. `execution(* com.acme.*.*(..))` — does it match a method in `com.acme.billing.InvoiceService`? → No; `.*.` matches one package level. Use `com.acme..*.*(..)`.
2. Which PCD is Spring-specific? → `bean(...)`.
3. What does `args(id, ..)` do in addition to matching? → Binds the first argument to the advice parameter named `id`.

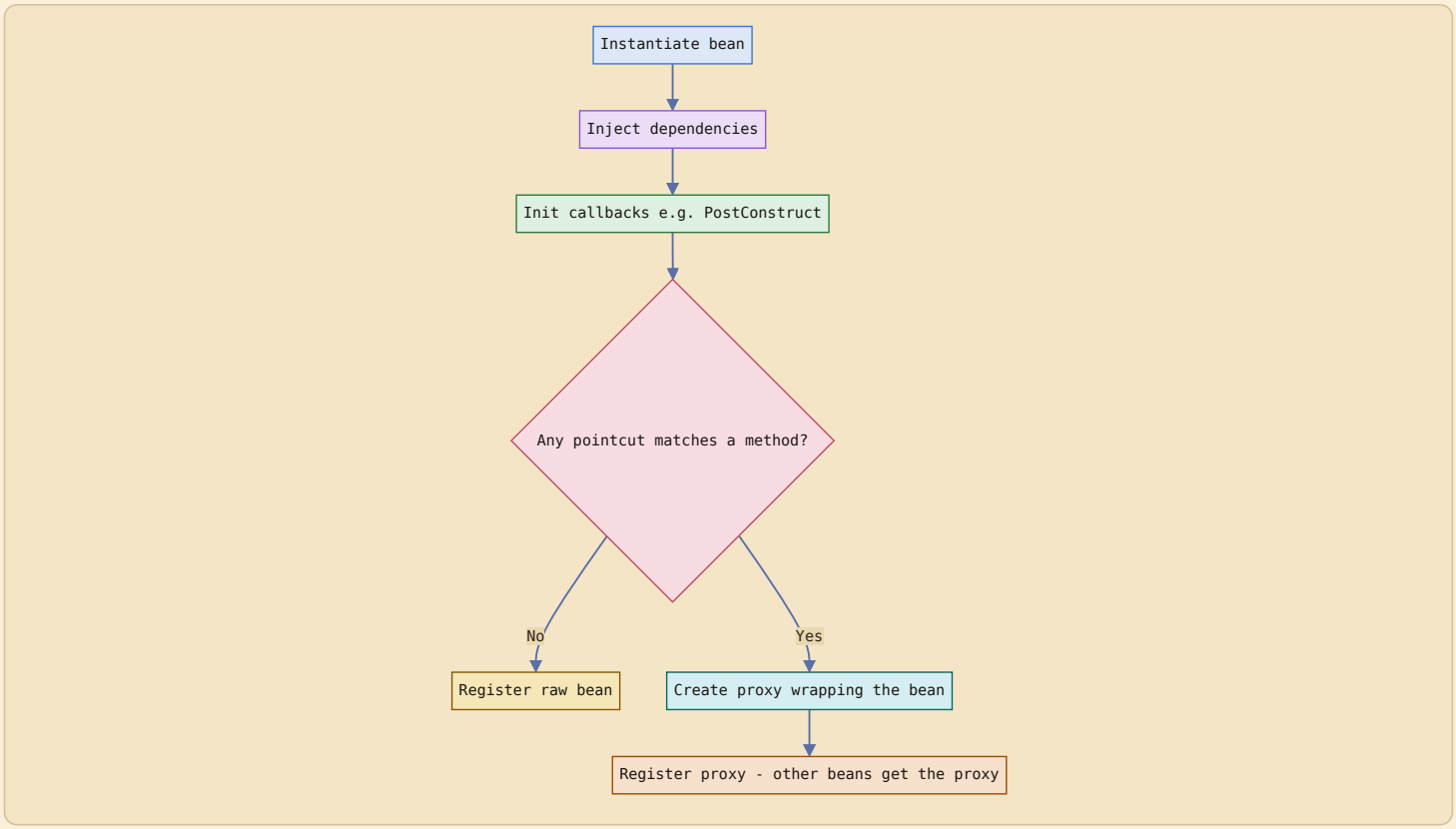
4.4 How Spring AOP works: proxies

What it is

A **proxy** is an object that has the same type as your bean (so callers cannot tell the difference) but forwards each call through the advice chain to the real object. Spring creates proxies **at startup**, in a `BeanPostProcessor` (see Chapter 2). For `@Aspect` support that post-processor is `AnnotationAwareAspectJAutoProxyCreator`.

How it works

1. The container creates the real bean (constructor, dependency injection, `@PostConstruct`).
2. In `postProcessAfterInitialization`, the auto-proxy creator asks: *does any advisor's pointcut match any method of this bean's class?*
3. If no: the raw bean is registered. If yes: it builds a proxy around the bean with the matching advisors, and **the proxy** is what gets registered in the context and injected everywhere.
4. At call time the proxy builds a chain of interceptors (advice objects, each of which can act before and after passing the call on) for that method and walks it; the last link calls the real method on the target.



Two kinds of proxy

	JDK dynamic proxy	CGLIB proxy
Built with	`java.lang.reflect.Proxy` (part of the JDK — Java Development Kit, the standard Java tools and libraries)	bytecode (compiled JVM instructions) generation of a **subclass** (Spring ships a repackaged CGLIB)
Requires	the bean implements at least one interface	the class is not `final` and has a usable constructor
The proxy *is a*	implementation of the interfaces only	subclass of the bean's class
Can be injected as the concrete class?	**No** — `BeanNotOfRequiredTypeException`	Yes
Methods it can intercept	interface methods (public)	non-final, non-private methods (public, protected, package-private within the same package)
Class name you'll see	`jdk.proxy2.$Proxy83`	`OrderService$$SpringCGLIB$$0`

Which one does Spring pick?

- **Plain Spring Framework:** JDK proxy if the bean implements an interface, otherwise CGLIB (unless you set `proxyTargetClass = true`).
- **Spring Boot:** `spring.aop.proxy-target-class` defaults to `true`, so Boot uses **CGLIB for everything**, even beans with interfaces. Boot chose this so that injecting a bean by its concrete class never fails. Set the property to `false` to get the plain-Spring behaviour.
- **Framework 7:** `@Proxyable` on a bean class or `@Bean` method overrides the global default for that one bean: `@Proxyable(ProxyType.INTERFACES)`, `@Proxyable(ProxyType.TARGET_CLASS)`, or `@Proxyable(interfaces = ...)` to expose only some interfaces. Framework 7 also merges the proxy settings of the different `@Enable...` annotations into one global default instead of each deciding separately.

Proved in `ProxyTypesTest.java`:

```
static String describe(Object bean) {
    String kind = AopUtils.isJdkDynamicProxy(bean) ? "JDK dynamic proxy"
                : AopUtils.isCglibProxy(bean) ? "CGLIB proxy" : "not a proxy";
    return kind + " -> " + bean.getClass().getName();
}

@Test
void whatKindOfProxyDidSpringCreate() {
    System.out.println("OrderService (no interface):       " + describe(orders));
    System.out.println("DefaultGreetingService (interface): " + describe(greeting));
    System.out.println("@Proxyable(INTERFACES) bean:        " + describe(interfaceOnly));
    assertThat(AopUtils.isCglibProxy(orders)).isTrue();
    assertThat(AopUtils.isCglibProxy(greeting)).isTrue();     // Boot default: proxy-target-class=true
    assertThat(AopUtils.isJdkDynamicProxy(interfaceOnly)).isTrue();
    assertThat(interfaceOnly).isNotInstanceOf(DefaultGreetingService.class);
}
```

where the third bean is:

```
@Service
@Proxyable(ProxyType.INTERFACES)       // ask for a JDK dynamic proxy
public class InterfaceOnlyGreetingService implements GreetingService { ... }
```

Output:

```
OrderService (no interface):       CGLIB proxy -> ch04.aop.OrderService$$SpringCGLIB$$0
DefaultGreetingService (interface): CGLIB proxy -> ch04.proxy.DefaultGreetingService$$SpringCGLIB$$0
@Proxyable(INTERFACES) bean:        JDK dynamic proxy -> jdk.proxy2.$Proxy83
```

A proxy by hand: `java.lang.reflect.Proxy`

To demystify it, here is a JDK dynamic proxy without Spring. You give the JDK a list of interfaces and an `InvocationHandler`; every call on the proxy lands in `invoke(proxy, method, args)`:

```
GreetingService target = new DefaultGreetingService();
InvocationHandler handler = (proxy, method, args) -> {
    System.out.println("  [handler] before " + method.getName());
    Object result = method.invoke(target, args);      // delegate to the real object
    System.out.println("  [handler] after  " + method.getName());
    return result;
};
GreetingService proxy = (GreetingService) Proxy.newProxyInstance(
        GreetingService.class.getClassLoader(), new Class<?>[] {GreetingService.class}, handler);
String out = proxy.greet("Ada");
System.out.println("result = " + out + ", proxy class = " + proxy.getClass().getSimpleName());
```

Output:

```
  [handler] before greet
  [handler] after  greet
result = Hello, Ada, proxy class = $Proxy137
```

Spring's JDK proxies are exactly this, with an `InvocationHandler` that runs the advisor chain.

Spring's `ProxyFactory`

Spring exposes its proxy machinery as `ProxyFactory`, handy in tests or libraries. The advice here is an AOP Alliance (a small, shared set of AOP interfaces several frameworks agreed on) `MethodInterceptor` — the low-level interface all Spring advice is converted into:

```
MethodInterceptor upper = inv -> ((String) inv.proceed()).toUpperCase();

ProxyFactory jdk = new ProxyFactory(new DefaultGreetingService());
jdk.addInterface(GreetingService.class);   // interfaces known + proxyTargetClass=false -> JDK
jdk.addAdvice(upper);
GreetingService p1 = (GreetingService) jdk.getProxy();

ProxyFactory cglib = new ProxyFactory(new DefaultGreetingService());
cglib.setProxyTargetClass(true);           // force a subclass
cglib.addAdvice(upper);
DefaultGreetingService p2 = (DefaultGreetingService) cglib.getProxy();
```

Output:

```
factory 1: JDK dynamic proxy -> jdk.proxy2.$Proxy83 says HELLO, BOB
factory 2: CGLIB proxy -> ch04.proxy.DefaultGreetingService$$SpringCGLIB$$1 says HELLO, BOB
```

(The JDK reuses one generated proxy class per interface set, which is why `$Proxy83` appears twice. The numbers in `$Proxy83`/`$Proxy137` depend on how many proxy classes the JVM generated before, so your run may show different ones.)

Trade-offs

Concern	JDK proxy	CGLIB proxy
Inject by concrete class	fails	works
`final` classes / methods	irrelevant (interface-based)	class cannot be proxied / method silently not advised
Startup cost	tiny	generates a class per bean (small, and cached)
Constructor	not involved	Spring uses Objenesis (a library that creates objects without calling a constructor), so the target's constructor is **not** run again for the proxy
Recommended	when you program to interfaces and want only interface methods exposed	Boot's default; works for classes without interfaces

Self-check

1. In a Boot app, `PaymentService implements Payments` is `@Transactional`. What proxy do you get? → CGLIB (Boot's `spring.aop.proxy-target-class=true`).
2. Why can injecting `PaymentServiceImpl` fail in plain Spring? → The bean is a JDK proxy implementing only `Payments`; it is not a `PaymentServiceImpl`.
3. Who creates Spring's proxies and when? → A `BeanPostProcessor` (an auto-proxy creator) after the bean is initialised, at startup.

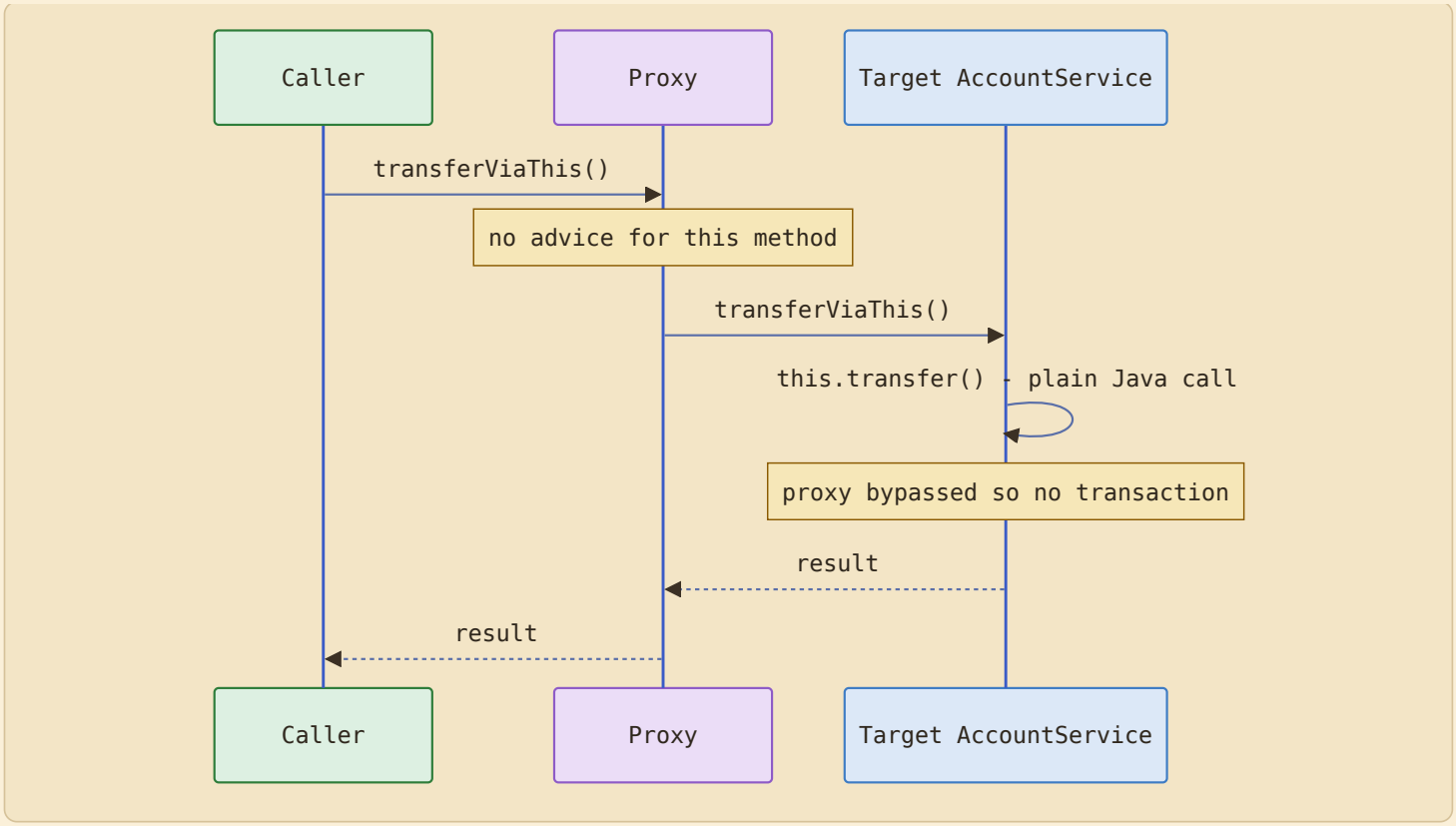
4.5 Proxy limitations — proved

Because the advice lives in the proxy, **advice runs only when a call passes through the proxy**. Every limitation below follows from that one sentence.

Self-invocation

What it is

Inside a bean, `this` refers to the **target object**, not the proxy. A call `this.someMethod()` — or just `someMethod()` — is an ordinary Java call that never touches the proxy, so no advice runs on it: no transaction, no caching, no async, no retry, no security check.



Proof 1 — `@Transactional`

```
@Service
public class AccountService {
    private final ObjectProvider<AccountService> self;   // lazy handle to *our own proxy*

    public AccountService(ObjectProvider<AccountService> self) { this.self = self; }

    @Transactional
    public String transfer() {
        // true only if a transaction proxy started a transaction before we got here
        return "transaction active = " + TransactionSynchronizationManager.isActualTransactionActive();
    }
    public String transferViaThis()      { return this.transfer(); }              // skips the proxy
    public String transferViaSelfProxy() { return self.getObject().transfer(); }  // goes through it
}
```

`TransactionSynchronizationManager.isActualTransactionActive()` reports whether a real transaction is bound to the current thread (a thread is one independent path of execution; Spring tracks the transaction per thread). The project has H2 (a small in-memory database) + `spring-boot-starter-jdbc`, so Boot configured a `DataSourceTransactionManager`.

Proof 2 — `@Cacheable`

```
@Cacheable("prices")                    // proxy checks the "prices" cache before calling us
public int price(String sku) {
    realCalls.incrementAndGet();         // counts how often the real method body runs
    return sku.length() * 100;
}
public int priceTwiceViaThis(String sku) {
    return this.price(sku) + this.price(sku);   // self-invocation: cache never consulted
}
```

Proof 3 — `@Async`

```
@Async                                  // proxy hands the call to a TaskExecutor thread
public CompletableFuture<String> build() {
    return CompletableFuture.completedFuture(Thread.currentThread().getName());
}
public CompletableFuture<String> buildViaThis() {
    return this.build();                // runs on the CALLER's thread
}
```

The test (`SelfInvocationTest.java`) asserts all three. Output:

```
caller -> proxy -> transfer():        transaction active = true
caller -> proxy -> this.transfer():   transaction active = false
caller -> proxy -> self.transfer():   transaction active = true
2 external calls -> real method ran 1 time(s)
2 internal calls -> real method ran 2 time(s)
caller thread:          main
@Async through proxy:   task-1
@Async via this.build(): main
```

`task-1` is a thread of Boot's auto-configured `applicationTaskExecutor` (a thread pool: a set of reusable worker threads that run submitted tasks). Via `this`, the "async" (meant to run in the background on another thread) method ran synchronously (the caller waited, on its own thread) on `main` — no error, no warning.

Fixes, best first

1. **Move the method to another bean.** If `transfer()` must be transactional on its own, it probably belongs in a separate service anyway. Clearest, no Spring tricks.
2. **Call through the proxy by self-injection** — inject your own bean lazily (`ObjectProvider<T>`, or a field `@Autowired @Lazy MyService self`). Shown above; it works but surprises readers, so comment it.
3. `AopContext.currentProxy()` — requires `@EnableAspectJAutoProxy(exposeProxy = true)` and couples code to Spring AOP; the Spring docs discourage it.
4. **Put the annotation on the outer method** (the entry point) instead of the inner one, if that is what you really meant.
5. **AspectJ weaving** (section 4.8) — advice is woven into the bytecode, so `this` calls are advised too. Rarely worth the build complexity.

`final` methods on CGLIB proxies

A CGLIB proxy is a subclass; a subclass **cannot override a `final` method**. So a call to a `final` method runs the method body **on the proxy object itself**. Worse: Spring creates proxy instances without running your constructor, so the proxy's own fields are `null`.

```
@Service
public class PricingService {
    private final List<String> currencies;
    public PricingService() { this.currencies = List.of("EUR", "USD"); }

    public String currencies()             { return "currencies=" + currencies; }
    public final String currenciesFinal()  { return "currencies=" + currencies; }
}
```

`TracingAspect` advises every method of this class, so the bean is proxied. The test:

```
String normal = pricing.currencies();
String viaFinal = pricing.currenciesFinal();
System.out.println("normal method: " + normal);
System.out.println("final method:  " + viaFinal);
System.out.println("advice log:    " + log.lines());
assertThat(viaFinal).isEqualTo("currencies=null");           // proxy's own, never-initialised field
```

Output:

```
normal method: currencies=[EUR, USD]
final method:  currencies=null
advice log:    [trace PricingService.currencies()]
```

The final method was not advised **and** read `null` instead of `[EUR, USD]`. In real code this shows up as a mysterious `NullPointerException` on an injected field that "definitely was injected". Kotlin classes are `final` by default, which is why Kotlin (a JVM language) Spring projects use the `kotlin-spring` (all-open) compiler plugin, which makes annotated classes non-final automatically.

A `final` **class** cannot be subclassed at all; if it needs advice and has no interface, startup fails with an error saying CGLIB cannot subclass it.

Other things proxies cannot advise

Case	Why	What happens
`private` methods	not visible to a subclass, not on an interface	advice silently ignored
`static` methods	not called on an instance	ignored
Calls inside the **constructor** or `@PostConstruct`	proxy does not exist yet / you are on the target	ignored (e.g. `@Transactional` in `@PostConstruct` has no transaction)
Objects created with `new`	not Spring beans, never proxied	ignored
Field access	Spring AOP only supports method execution join points	not possible (needs AspectJ)
Methods only on the class, with a JDK proxy	the proxy implements only interfaces	not reachable through the proxy at all

For `@Transactional` specifically: since Framework 6.0, `protected` and package-private methods can be transactional with class-based (CGLIB) proxies; with interface-based proxies they must be `public`, and `private` never works.

In the interview

30-second answer: "Spring AOP is proxy-based. At startup a BeanPostProcessor wraps any bean that matches a pointcut in a proxy — CGLIB by default in Boot, JDK dynamic proxy for interface-only proxying — and other beans get the proxy. Advice runs only when a call goes through the proxy. So `this.method()` calls, private or final methods, and calls from constructors skip `@Transactional`, `@Cacheable`, `@Async` and custom aspects. The fix is usually to move the method to another bean, or to call through an injected reference to the proxy."

- *"My `@Transactional` method doesn't roll back — why?"* → Top causes: self-invocation; method is private/final; exception is checked (a *checked* exception must be declared with `throws`, e.g. `IOException`; an *unchecked* one extends `RuntimeException`; rollback defaults to unchecked exceptions only — see Chapter 7); exception caught inside the method; bean created with `new`.
- *"How can you see if a bean is proxied?"* → `AopUtils.isAopProxy(bean)` / `isCglibProxy` / `isJdkDynamicProxy`, or print `bean.getClass()` and look for `$$SpringCGLIB$$` or `$Proxy`.
- *"JDK vs CGLIB?"* → JDK: interface-based, from the JDK, cannot be cast to the class. CGLIB: runtime subclass, can't handle final classes/methods. Boot defaults to CGLIB via `spring.aop.proxy-target-class=true`.

Self-check

1. Why does `this.transfer()` skip the transaction? → `this` is the target, not the proxy; the transaction interceptor lives in the proxy.
2. What did the `final` method return in the proof and why? → `currencies=null` — it ran on the CGLIB proxy instance, whose fields were never initialised.
3. Name two fixes for self-invocation. → Extract the method into another bean; inject and call your own proxy (`ObjectProvider<Self>` or `@Lazy` self-reference).
4. Does `@Async` on a method called from `@PostConstruct` of the same bean run asynchronously? → No; that call is on the target, not through the proxy.

4.6 Spring features built on proxies

Many annotations in this book are "AOP you did not write yourself". They are processed by infrastructure advisors, not `@Aspect` classes, but they obey **all** the rules of section 4.5.

Annotation	Switched on by	Interceptor / processor	Chapter
`@Transactional`	Boot auto-config (`@EnableTransactionManagement` in plain Spring)	`TransactionInterceptor`	7
`@Cacheable`, `@CachePut`, `@CacheEvict`	`@EnableCaching`	`CacheInterceptor`	9
`@Async`	`@EnableAsync`	`AsyncExecutionInterceptor`	14
`@Retryable`, `@ConcurrencyLimit` (Framework 7)	`@EnableResilientMethods`	`RetryAnnotationBeanPostProcessor`, `ConcurrencyLimitBeanPostProcessor`	4.7 below
`@PreAuthorize`, `@PostAuthorize`, `@Secured`	`@EnableMethodSecurity`	`AuthorizationManagerBeforeMethodInterceptor`, ...	10
`@Validated` on a class + constraint annotations on method parameters	Boot auto-config (`MethodValidationPostProcessor`)	`MethodValidationInterceptor`	5
`@Observed` (Micrometer, Spring's metrics library)	an `ObservedAspect` bean	an `@Aspect`	15
Spring Data repository interfaces	`@EnableJpaRepositories` / Boot	JDK proxy generated for the interface	8
`@Configuration` classes (full mode)	always	CGLIB subclass that intercepts `@Bean` method calls (not AOP advice, same technique)	2

Order matters when several apply. For example, `@Retryable` + `@Transactional` on the same method: if the transaction interceptor is outside the retry interceptor, all attempts share one transaction that is already marked rollback-only (doomed to be undone, whatever happens next) after the first failure. Put retries on a caller one layer up, or set the orders explicitly (`@EnableTransactionManagement(order = ...)`, `@EnableResilientMethods(order = ...)`).

Self-check

1. Which annotation turns on `@Cacheable`? → `@EnableCaching`.
2. Does `@PreAuthorize` protect a `private` method? → No — it is proxy-based like the others.

4.7 Framework 7 core resilience: `@Retryable` and `@ConcurrencyLimit`

What it is

Spring Framework **7.0** (November 2025) added a small resilience toolkit to the core framework, in the packages `org.springframework.resilience.annotation` and `org.springframework.core.retry`:

- `@Retryable` — call a method again when it throws, with back-off delay.
- `@ConcurrencyLimit` — allow at most N threads inside a method at a time (a *bulkhead*: a limit that stops one overloaded part from sinking the whole app, named after a ship's watertight walls).
- `RetryTemplate` / `RetryPolicy` — the same retry logic, programmatically.

They replace most uses of the separate **Spring Retry** project; Boot 4 removed dependency management for Spring Retry. They are **not** a full resilience library: there is no circuit breaker (stops calling a failing service for a while so it can recover), rate limiter (caps calls per second) or time limiter (gives up after a deadline) — for those use Resilience4j (a popular Java resilience library) (see Chapter 16 and the circuit breaker notes).

Why it matters

Remote calls fail *transiently* (briefly, then work again): a timeout, a 503 (HTTP "service unavailable") during a deploy (rolling out a new version), a dropped connection. Retrying once or twice with a short, growing delay turns many such failures into successes. And with **virtual threads** (very cheap threads managed by the JVM, so you can have millions) (Java book 15.4) you no longer have a thread-pool size that naturally caps how many callers hit a slow downstream service (a service *you* call) at once — `@ConcurrencyLimit` puts that cap back where you need it.

How to switch it on

Add `@EnableResilientMethods` to a configuration class. As of Boot 4.1 there is no auto-configuration for it, so without the annotation `@Retryable` is silently ignored — like every other proxy feature.

```
@SpringBootApplication
@EnableCaching          // turns on the @Cacheable proxy
@EnableAsync            // turns on the @Async proxy
@EnableResilientMethods // Framework 7: turns on @Retryable and @ConcurrencyLimit
public class AopApplication { ... }
```

(Alternatively register a `RetryAnnotationBeanPostProcessor` and/or `ConcurrencyLimitBeanPostProcessor` bean.)

`@Retryable` attributes (Framework 7.0)

Attribute	Default	Meaning
`includes` / `value`	empty = all exceptions	exception types to retry (also matches **nested causes**)
`excludes`	empty	exception types never to retry
`predicate`	none	a `MethodRetryPredicate` class for custom decisions, applied after includes/excludes
`maxRetries`	`3`	retries **after** the first call → total attempts = 1 + `maxRetries`
`delay`	`1000` ms	wait before the first retry
`multiplier`	`1.0`	each delay = previous × multiplier (exponential back-off — waits that double or grow each time — when > 1)
`maxDelay`	`Long.MAX_VALUE`	cap for the growing delay
`jitter`	`0`	random variation applied to each delay, so many clients don't retry in lock-step (all at the same instant)
`timeout`	`0` = none	total time budget for all attempts including delays (since 7.0.2)
`timeUnit`	`MILLISECONDS`	unit for the numeric delay attributes
`...String` variants	—	`maxRetriesString`, `delayString`, ... accept `${...}` placeholders and SpEL

It works on methods returning `Mono`/`Flux` (Project Reactor's types for 0–1 and 0–N values that arrive later) too (the retry is applied to the reactive pipeline), and publishes a `MethodRetryEvent` for each failed attempt, which you can observe with an `@EventListener`.

Example: a flaky client

```
@Service
public class InventoryClient {
    // up to 3 retries (4 attempts in total), waiting 10 ms, then 20 ms, then 40 ms
    @Retryable(includes = TransientException.class, maxRetries = 3, delay = 10, multiplier = 2)
    public int stockFor(String sku) {
        attempts.add("attempt " + (attempts.size() + 1));
        if (failuresLeft-- > 0) {
            throw new TransientException("inventory timed out");
        }
        return 42;
    }

    // IllegalArgumentException is not in 'includes', so it is never retried
    @Retryable(includes = TransientException.class, maxRetries = 3, delay = 10)
    public int stockForBadSku(String sku) {
        attempts.add("attempt " + (attempts.size() + 1));
        throw new IllegalArgumentException("bad sku " + sku);
    }
}
```

The tests (`ResilienceTest.java`) make it fail twice (recovers), ten times (gives up), and with a non-retryable exception:

```
inventory.failNextCalls(2);
int stock = inventory.stockFor("SKU-1");           // succeeds on attempt 3

inventory.failNextCalls(10);
assertThatThrownBy(() -> inventory.stockFor("SKU-1"))
        .isInstanceOf(TransientException.class);    // the LAST original exception
assertThat(inventory.attempts()).hasSize(4);        // 1 call + 3 retries
```

Output:

```
stock = 42 after [attempt 1, attempt 2, attempt 3]
gave up after [attempt 1, attempt 2, attempt 3, attempt 4]
IllegalArgumentException -> [attempt 1]
```

When all attempts fail, the caller receives the **last original exception** (here a `TransientException`), not a wrapper — so existing `catch` blocks keep working.

`@ConcurrencyLimit`

```
@ConcurrencyLimit(2)                        // default policy BLOCK: extra callers wait
public void print(String doc) throws InterruptedException {
    int now = inside.incrementAndGet();
    maxInside.accumulateAndGet(now, Math::max);
    Thread.sleep(50);                        // hold the "printer" for a while
    inside.decrementAndGet();
}

@ConcurrencyLimit(limit = 1, policy = ConcurrencyLimit.ThrottlePolicy.REJECT)  // 7.0.3+
public void printUrgent(String doc) throws InterruptedException {
    Thread.sleep(100);
}
```

- `value` / `limit`: the maximum concurrent invocations **per bean instance**. `1` makes the method behave like a lock (only one thread may be inside at a time) on the bean.
- `limitString`: placeholder/SpEL variant, e.g. `"${printer.max-concurrent}"`.
- `policy` (since 7.0.3): `BLOCK` (default) makes extra callers wait; `REJECT` throws `InvocationRejectedException` (a subclass of `RejectedExecutionException`) immediately.

The test starts 6 virtual threads calling `print` and a second caller of `printUrgent` while the first is still inside:

```
6 callers, max inside print() at once = 2
second caller -> InvocationRejectedException
```

Programmatic: `RetryTemplate`

Use it when the retry scope is a few lines inside a method, or when policies are chosen at runtime:

```
RetryPolicy policy = RetryPolicy.builder()
        .maxRetries(4)
        .delay(Duration.ofMillis(5))
        .multiplier(2)
        .includes(TransientException.class)
        .build();
RetryTemplate retry = new RetryTemplate(policy);

String result = retry.execute(() -> {
    if (calls.incrementAndGet() < 3) throw new TransientException("busy");
    return "done on call " + calls.get();
});
```

Output:

```
RetryTemplate -> done on call 3
```

`execute(...)` throws a checked `RetryException` on exhaustion that holds every failure (`getExceptions()`, `getLastException()`, `getRetryCount()`); `invoke(...)` instead rethrows the last original exception unchecked. A `RetryListener` can observe each attempt (for logging or metrics).

Trade-offs: which retry tool?

	Framework 7 `@Retryable` / `RetryTemplate`	Spring Retry (`org.springframework.retry`)	Resilience4j
Where	built into `spring-context` / `spring-core`	separate library, no longer managed by Boot 4	separate library + Spring Boot integration
Retry + back-off + jitter	yes	yes	yes
Recovery method (`@Recover`)	no — catch the exception at the caller	yes	fallback methods
Circuit breaker, rate limiter, time limiter	no	basic circuit breaker only	yes, all
Bulkhead	`@ConcurrencyLimit`	no	semaphore (a counter of free permits) and thread-pool bulkheads
Metrics	via `RetryListener` / `MethodRetryEvent`	listeners	built-in Micrometer metrics
Choose when	you need simple retries / limits and want no extra dependency	legacy code	you need circuit breaking and rich resilience policies

When not to use it

- **Non-idempotent operations** (charging a card, sending an email) — a retry after a timeout may do the work twice.

  > **Key term — idempotent:** an operation is idempotent if doing it twice has the same effect as doing it once. "Set the price to 10" is idempotent; "add 10 to the balance" is not. An **idempotency key** is a unique id the caller sends with a request so the server can recognise a repeat and skip it. See System Design Fundamentals. Retry only operations that are safe to repeat, or make them idempotent first (idempotency keys). See Retry pattern notes.

- **Errors that won't go away** — validation errors, 4xx responses (HTTP status codes 400–499, meaning the *request* was wrong), `NullPointerException`. Always set `includes` narrowly.
- **Retry at every layer** — if the gateway (the entry-point server in front of your services), the client library and your service each retry 3 times, one user request can become 64 downstream calls ("retry storm"). Retry at one layer.
- **Inside a transaction** — see section 4.6.

Common mistakes

- Forgetting `@EnableResilientMethods` → no retries, no error.
- Self-invocation → `this.stockFor()` is not retried (it is proxy-based).
- `maxRetries = 3` read as "3 attempts" → it is 4 attempts.
- No jitter on a fleet of instances (many running copies of the same app) → synchronised retry waves after an outage.

Self-check

1. With `maxRetries = 2`, how many times can the method run? → 3 (1 + 2).
2. What does the caller get after the last failed attempt of `@Retryable`? → The last original exception.
3. How does `@ConcurrencyLimit` behave at the limit by default? → `BLOCK`: extra callers wait for a free slot. With `policy = REJECT` they get `InvocationRejectedException`.
4. Is there a circuit breaker in Framework 7 core? → No; use Resilience4j (or Spring Cloud Circuit Breaker).

TL;DR

`@EnableResilientMethods` + `@Retryable(includes = ..., maxRetries, delay, multiplier, jitter)` for retries; `@ConcurrencyLimit(n)` to cap concurrent callers; `RetryTemplate` + `RetryPolicy.builder()` programmatically. All are proxy-based.

4.8 Spring AOP vs AspectJ

What it is

AspectJ is the original, full AOP language for Java. Spring borrowed its annotations and pointcut syntax (that is why the imports are `org.aspectj.lang.annotation.*`), but by default Spring does **not** use the AspectJ weaver — it uses its own proxies and only uses AspectJ's library to *parse and match* pointcut expressions.

	Spring AOP (default)	AspectJ weaving
How advice is applied	runtime proxy around a bean	bytecode modified at compile time (`ajc`), post-compile, or load time (LTW — load-time weaving — with a Java agent, a helper that can change classes as the JVM loads them)
Join points	method execution on Spring beans only	method call and execution, constructor, field get/set, static initialisers, ... on **any** object
Self-invocation, `final`, `private`	not advised	advised
Objects created with `new`	not advised	advised
Setup	add the starter	AspectJ compiler plugin or `-javaagent`, extra build complexity
Performance	a few extra method calls per advised invocation	practically zero overhead (code is inlined — copied straight into the method)
Good for	95% of enterprise needs: transactions, logging, security	fine-grained or domain-object advice, very hot paths (code that runs extremely often)

`@Transactional`, `@Cacheable` and `@Async` can be switched to AspectJ mode (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)` plus the weaver), which removes the self-invocation limitation. Very few teams do this; extracting a bean is simpler.

When not to use AOP at all

- **Business logic** — if a rule matters to the domain (e.g. "orders over 1000 need approval"), write it explicitly in the service. Hidden advice makes code hard to follow and test.

- **When a plain decorator (a wrapper object that adds behaviour), filter or interceptor fits** — HTTP concerns belong in servlet filters (code that runs on every web request before it reaches Spring MVC) / `HandlerInterceptor` (Chapter 5); client concerns in `ClientHttpRequestInterceptor` (Chapter 6).
- **Very broad pointcuts** — "log every method in the app" produces noise and cost; use tracing/metrics tools (Chapter 15).

Real-world examples

- **Spring itself**: transactions, caching, security, async and retries as shown above.
- **Micrometer `@Timed` / `@Observed`** — aspects that record metrics/traces around annotated methods.
- **Audit logging** in banking/insurance apps: an `@Audited` annotation + `@AfterReturning` advice writes who-did-what records.
- **Multi-tenant routing** (one app serving many customer organisations, "tenants", with separate data): an `@Around` aspect sets a tenant id in a `ThreadLocal` (a variable with a separate value per thread)/scoped value before repository calls and clears it after.
- **Feature flags / rate limits** on specific service methods via custom annotations.

4.9 Common mistakes (whole chapter)

Symptom	Likely cause	Fix
Aspect never runs	aspect is not a bean; starter missing; pointcut doesn't match	add `@Component`; add `spring-boot-starter-aspectj`; test the pointcut
`@Transactional` / `@Cacheable` / `@Async` / `@Retryable` ignored	self-invocation; private/final method; bean made with `new`; missing `@Enable...`	extract to another bean; make method public and non-final; inject the bean; enable the feature
`NullPointerException` on an injected field inside a method	`final` method on a CGLIB proxy runs on the proxy	remove `final`
`BeanNotOfRequiredTypeException`	JDK proxy injected where the concrete class is expected	inject the interface, or use CGLIB (Boot default)
Advised method returns `null`	`@Around` advice doesn't return `proceed()`'s result	`return pjp.proceed();`
Slow startup / weird errors after adding an aspect	pointcut matches infrastructure beans	scope with `within(com.yourapp..*)`
Retry happens inside a failing transaction	transaction interceptor wraps the retry interceptor	retry at a higher layer or set advisor orders

---

4.10 In the interview

Q: What is AOP and how does Spring implement it? 30-second answer: AOP modularises cross-cutting concerns such as logging, transactions and security. You write an aspect containing advice and a pointcut; Spring applies it at runtime by wrapping matching beans in proxies created by a `BeanPostProcessor`. Callers get the proxy, which runs the advice chain and then the real method.

- *Follow-up: name the advice types.* → `@Before`, `@AfterReturning`, `@AfterThrowing`, `@After`, `@Around`. Only `@Around` controls whether the method runs and what it returns.
- *Follow-up: how do you order two aspects?* → `@Order` / `Ordered`; lower value is the outer layer.
- *Follow-up: what's a join point in Spring AOP?* → Only method execution on a Spring bean.

Q: JDK dynamic proxy vs CGLIB — which does Spring Boot use? CGLIB by default for all beans (`spring.aop.proxy-target-class=true`), which allows injecting by class. Plain Spring uses JDK proxies when interfaces exist. Framework 7 adds `@Proxyable` to override per bean.

- *Follow-up: limits of CGLIB?* → No `final` classes; `final`/`private` methods are not advised; `final` methods even run on the uninitialised proxy instance.

Q: Why might `@Async`/`@Transactional` not work? → Self-invocation (most common), not a Spring bean, private/final method, missing `@EnableAsync`/transaction manager, or for transactions: checked exception / exception swallowed.

Q: What's new for resilience in Spring Framework 7? → `@Retryable` (with delay, multiplier, jitter, maxDelay, includes/excludes, reactive support), `@ConcurrencyLimit` (bulkhead, BLOCK or REJECT), enabled with `@EnableResilientMethods`; plus `RetryTemplate`/`RetryPolicy` in `spring-core`. No circuit breaker — use Resilience4j for that.

- *Follow-up: what's the danger of retries?* → Duplicate side effects on non-idempotent operations and retry storms; retry only transient errors, add jitter, retry in one layer.

Q: Spring AOP vs AspectJ? → Spring AOP: proxies, method execution on beans only, easy. AspectJ: bytecode weaving, any join point, no self-invocation problem, more build complexity.

4.11 Chapter TL;DR

- **AOP** = write a cross-cutting concern once (an **aspect**: **advice** + **pointcut**) and apply it to many **join points** (in Spring: bean method executions).
- `@Aspect` **+** `@Component`, `spring-boot-starter-aspectj` (Boot 4 name). Five advice types; `@Around` must `return pjp.proceed()`. Order aspects with `@Order` (lower = outer).
- Pointcuts: `execution(...)`, `within(...)`, `@annotation(...)`, `args(...)`, `bean(...)`; `..` means "any sub-package" or "any parameters".
- Spring AOP is **proxy-based**. Boot uses **CGLIB** subclasses by default; JDK proxies implement interfaces only; Framework 7's `@Proxyable` overrides per bean.
- **Advice runs only through the proxy**: self-invocation, `private`, `final`, `static`, constructor/`@PostConstruct` calls and `new` objects are not advised — proved for `@Transactional`, `@Cacheable`, `@Async` and a `final` method.
- `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize`, `@Validated`, `@Retryable` are all proxy features with the same rules.
- Framework 7 **resilience**: `@EnableResilientMethods`, `@Retryable` (attempts = 1 + `maxRetries`, last exception rethrown), `@ConcurrencyLimit` (BLOCK/REJECT), `RetryTemplate`. No circuit breaker in core.
- Next: Chapter 5 — Spring MVC and REST.

Chapter 5 — Spring MVC and REST APIs

Spring MVC is Spring's web framework for the classic, thread-per-request (each incoming request is handled from start to end by one thread, which waits while it does I/O) Servlet stack (the standard Java web-server API, implemented by servers such as Tomcat and Jetty). Almost every Spring Boot service that exposes an HTTP/JSON API is built on it. This chapter builds a small but complete **book catalogue API** and uses it to explain how a request travels through Spring MVC, how to map URLs to methods, read input, validate it, return the right status codes and error bodies, negotiate formats, version the API (new in Framework 7), handle CORS (letting browser pages from other sites call your API, §5.10) and file uploads, page through results, document the API with OpenAPI (a standard, machine-readable API description, §5.12), and plug in filters and interceptors (code that runs before/after every request, §5.11).

What you'll learn

1. REST in one page: resources, HTTP methods, status codes, idempotency.
2. The `DispatcherServlet` request flow and the components involved.
3. Controllers and request mappings (`@RestController`, `@GetMapping`, path patterns).
4. Reading input: path variables, query parameters, headers, request bodies; and writing output with `ResponseEntity`.
5. JSON with **Jackson 3** (Boot 4's default) — what changed from Jackson 2.
6. Validation with Jakarta Bean Validation: `@Valid`, `@Validated`, groups, parameter validation.
7. Error handling with `@RestControllerAdvice` and **Problem Details (RFC 9457)**.
8. Content negotiation (`Accept`, `produces`, 406) and pagination.
9. **API versioning** — first-class in Framework 7 / Boot 4.
10. CORS, file upload, filters vs interceptors, OpenAPI with springdoc, HATEOAS in brief.

Code: `code/mvc-rest/` (the book API) and `code/api-versioning/` — Spring Boot 4.1.1, Framework 7.0, Java 25. All behaviour is proved with `@WebMvcTest` + `MockMvcTester` tests (a test that starts only the web layer and sends fake requests to it in memory); output shown is real. Testing in depth is Chapter 12; the reactive alternative (WebFlux) is Chapter 13; calling other APIs is Chapter 6.

The project's single web dependency is the Boot 4 starter:

```
<!-- Spring MVC + embedded Tomcat + Jackson 3 (was spring-boot-starter-web before Boot 4) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

5.1 REST in one page

What it is

REST (Representational State Transfer) is an architectural style for HTTP APIs. In practice a "REST API" means:

- The API exposes **resources** (things) at **URLs**: `/api/books`, `/api/books/3`.
- Clients act on them with standard **HTTP methods**.
- Data travels as **representations** (usually JSON) whose format is agreed with headers (`Content-Type`, `Accept`). An HTTP **header** is a `Name: value` line of metadata sent before the body of a request or response.
- Each request carries everything needed to handle it (**stateless** server — it keeps no memory of earlier requests from that client).
- Results are signalled with **HTTP status codes**.

Methods

Method	Meaning	Safe? (no change)	Idempotent? (repeat = same effect)	Typical response
`GET /books`	list/search	yes	yes	`200` + body
`GET /books/3`	read one	yes	yes	`200`, or `404`
`POST /books`	create (server picks the id)	no	**no** — two POSTs create two books	`201 Created` + `Location` header
`PUT /books/3`	replace the whole resource	no	yes	`200` + body, or `204`
`PATCH /books/3`	partial update	no	not necessarily	`200` / `204`
`DELETE /books/3`	delete	no	yes	`204 No Content`, then `404`

Idempotency matters for retries: a client may safely retry a timed-out `PUT` or `DELETE`, but not a `POST` without an idempotency key (a unique id sent with the request so the server can spot and ignore a duplicate; see System Design Fundamentals) (see Chapter 6).

Status codes you will actually use

Code	Name	When
200	OK	success with a body
201	Created	a POST created something; add `Location`
204	No Content	success, nothing to return (DELETE)
400	Bad Request	malformed JSON, invalid fields, bad parameter types
401	Unauthorized	not authenticated (Chapter 10)
403	Forbidden	authenticated but not allowed; also a rejected CORS request
404	Not Found	no such resource / URL
405	Method Not Allowed	URL exists, method doesn't (response has `Allow`)
406	Not Acceptable	cannot produce any format the client accepts
409	Conflict	state conflict, e.g. duplicate or version clash
413	Content Too Large	upload too big
415	Unsupported Media Type	request body format not supported
422	Unprocessable Content	some teams use it for semantic validation errors (Spring uses 400 by default)
500	Internal Server Error	unexpected bug
503	Service Unavailable	overloaded / dependency down

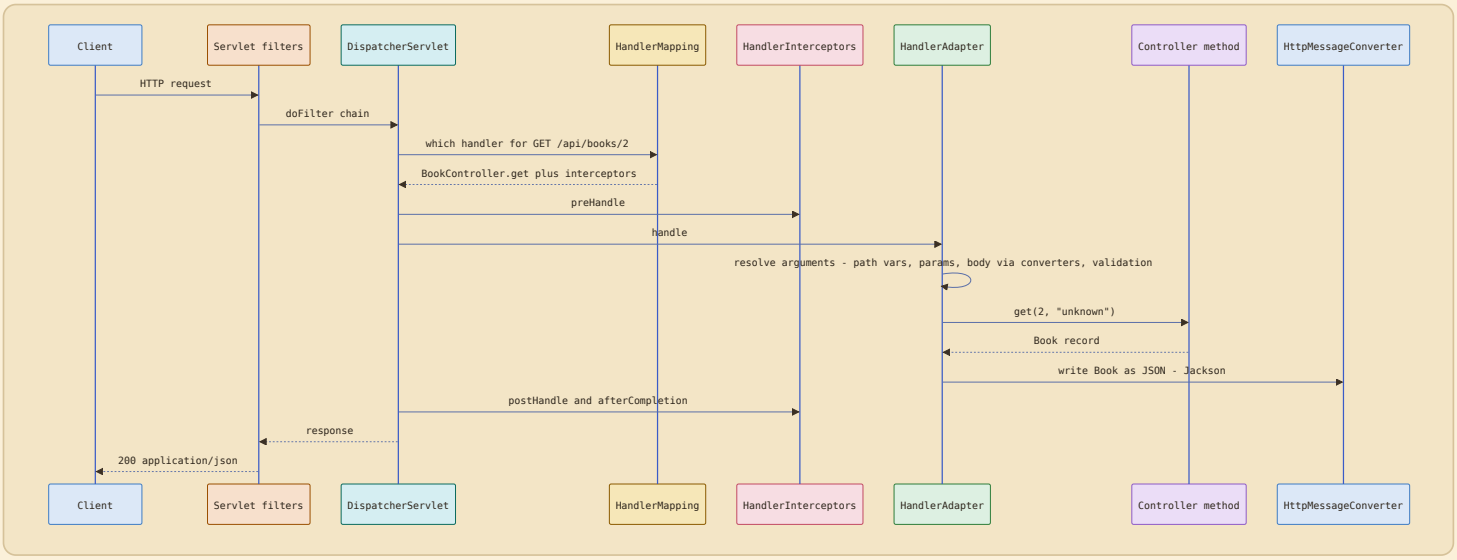
5.2 How a request flows: the `DispatcherServlet`

What it is

Spring MVC is built around one servlet (a Java class that a web server calls to handle HTTP requests), the `DispatcherServlet` — a **front controller**: every request enters through it, and it delegates each step to pluggable components. Analogy: a hotel reception desk. Guests (requests) do not wander to rooms; reception looks up who handles the request, passes it on, and hands back the answer in a form the guest understands.

Boot registers the `DispatcherServlet` at `/` automatically and starts an embedded **Tomcat 11** (Jakarta Servlet 6.1).

How it works



1. **Servlet filters** run first (security, request ids (a unique tag per request for tracing it in logs), compression). They are outside Spring MVC.
2. The `DispatcherServlet` asks its `HandlerMapping`**s** which handler matches. For annotated controllers, `RequestMappingHandlerMapping` compares path, HTTP method, `consumes`/`produces`, params, headers and — in Framework 7 — the **API version**.
3. `HandlerInterceptor.preHandle` runs for the chosen handler.
4. A `HandlerAdapter` invokes the method. Before that, `HandlerMethodArgumentResolver`**s** build each argument: `@PathVariable` from the URL, `@RequestParam` from the query string (the `?page=1&size=2` part of a URL), `@RequestBody` via an `HttpMessageConverter` (Jackson for JSON), plus validation.

5. The return value is handled by a `HandlerMethodReturnValueHandler`: for `@ResponseBody` / `@RestController` it picks an `HttpMessageConverter` based on the `Accept` header (content negotiation) and writes the body.
6. If anything throws, `HandlerExceptionResolver`s turn the exception into a response (`@ExceptionHandler` methods, `@ResponseStatus`, Spring's defaults).
7. `postHandle` / `afterCompletion` interceptor callbacks, then filters unwind.

For server-rendered HTML (Thymeleaf, a template engine that fills HTML templates with data), step 5 instead returns a view name that a `ViewResolver` renders; this book focuses on JSON APIs.

Self-check

1. What is a front controller? → One entry point that receives every request and delegates to handlers — the `DispatcherServlet`.
2. Which component turns a Java object into JSON? → An `HttpMessageConverter` (the Jackson one), chosen by content negotiation.
3. Do servlet filters know which controller method will run? → No — they run before the `DispatcherServlet` resolves the handler; interceptors do know.

5.3 Controllers and request mappings

What it is

A **controller** is a bean whose methods handle requests. `@RestController` = `@Controller` + `@ResponseBody`: every method's return value is written straight into the response body (as JSON), not treated as a view name.

```
@RestController                          // = @Controller + @ResponseBody on every method
@RequestMapping("/api/books")            // common path prefix for the class
public class BookController {

    private final BookRepository repo;

    public BookController(BookRepository repo) { this.repo = repo; }

    @GetMapping
    public PageResponse<Book> list(@RequestParam(defaultValue = "0") @Min(0) int page,
                                   @RequestParam(defaultValue = "2") @Min(1) @Max(50) int size,
                                   @RequestParam(required = false) String author) { ... }

    @GetMapping("/{id:\\d+}")      // regex: only digits, so "/export" never lands here
    public Book get(@PathVariable long id,
                    @RequestHeader(name = "X-Client", defaultValue = "unknown") String client) {
        return repo.findById(id).orElseThrow(() -> new BookNotFoundException(id));
    }

    @PostMapping
    public ResponseEntity<Book> create(
            @Validated({BookRequest.OnCreate.class, Default.class}) @RequestBody BookRequest request) { ... }

    @PutMapping("/{id:\\d+}")
    public Book update(@PathVariable long id, @Validated @RequestBody BookRequest request) { ... }

    @DeleteMapping("/{id:\\d+}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable long id) { ... }
}
```

Mapping annotations

Annotation	Same as
`@GetMapping`	`@RequestMapping(method = GET)`
`@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`	the other methods
`@RequestMapping` on the class	prefix + shared settings for all methods

Useful attributes (all mapping annotations):

Attribute	Narrows the match to requests ...	Example
`path` / `value`	with this URL pattern	`"/{id}"`
`consumes`	whose `Content-Type` matches	`consumes = "application/json"`
`produces`	whose `Accept` allows this type	`produces = "text/csv"`
`params`	having / lacking a parameter	`params = "format=short"`
`headers`	having a header value	`headers = "X-Beta=true"`
`version` (Framework 7)	with a matching API version	`version = "1.1+"`

Path patterns

Spring MVC matches with `PathPattern` (the default since Framework 6 / Boot 3):

Pattern	Matches
`/books/{id}`	one segment captured as `id`
`/books/{id:\d+}`	only if the segment matches the regex (regular expression: a text pattern; `\d+` = one or more digits)
`/files/**`	anything below `/files/`
`/files/{*path}`	the rest of the path, captured (must be at the end)
`/books/*.csv`	`*` = any characters within one segment

Two version-specific facts:

- **Trailing slashes** no longer match by default (since Framework 6.0): `/api/books/` is *not* `/api/books`.
- When several patterns match, the **most specific** wins (fewer wildcards — `*`/`**` placeholders that match anything — or variables, longer literal text).

A real pitfall caught by the tests

The first version of this controller mapped `@GetMapping("/{id}")`. A request `GET /api/books/export` with `Accept: application/xml` could not be served by the two `/export` methods (they produce JSON and CSV only), so Spring fell back to the next matching pattern — `/{id}` with `id = "export"` — and answered **`400 "Failed to convert 'id'"`** instead of **`406 Not Acceptable`**. Constraining the variable with a regex, `/{id:\d+}`, removed the ambiguity. Use regex constraints (or distinct paths) when literal sub-paths share a level with variables.

> **Self-check**

1. `@RestController` vs `@Controller`? → `@RestController` adds `@ResponseBody` so return values become the response body; `@Controller` returns view names unless a method has `@ResponseBody`.
2. Does `GET /api/books/` hit a method mapped to `/api/books` in Boot 4? → No; trailing-slash matching is off by default since Framework 6.
3. How do you restrict `{id}` to digits? → `{id:\d+}` (in Java source `"{id:\\d+}"`).

5.4 Reading input and writing output

Method arguments

Annotation / type	Source	Notes
`@PathVariable long id`	URL segment `{id}`	type conversion is automatic; failure → 400
`@RequestParam int page`	query string / form field	`required` defaults to `true`; `defaultValue` makes it optional; `Optional<T>` also works
`@RequestParam Map<String,String>`	all query params	
`@RequestHeader("X-Client") String c`	request header	`defaultValue`, `required` as above
`@CookieValue`	cookie (small named value the browser stores and sends back on every request)	
`@RequestBody BookRequest r`	body, via `HttpMessageConverter`	JSON → record/class with Jackson
`@RequestPart MultipartFile f`	part of a `multipart/form-data` (the body format browsers use to upload files) body	section 5.10
`@ModelAttribute` / no annotation on a class	query params / form fields bound to an object's properties	handy for search filters
`HttpServletRequest`, `Principal` (the logged-in user), `Locale` (the client's language/region), `UriComponentsBuilder`	framework-provided	

Names: with `@PathVariable long id` Spring uses the parameter name `id`. That needs the `-parameters` compiler flag, which `spring-boot-starter-parent` enables; outside Boot's parent, write `@PathVariable("id")`.

Return values

Return type	Result
any object / record / `List`	`200` + body serialised (turned into JSON text) by a message converter
`void` + `@ResponseStatus(NO_CONTENT)`	`204`, no body
`ResponseEntity<T>`	full control: status, headers, body
`ProblemDetail` / `ErrorResponse`	RFC 9457 error body (section 5.7)
`String` in a `@RestController`	written as text (`text/plain` or what `produces` says)
`CompletableFuture<T>`, `Callable<T>`, `DeferredResult<T>`, `SseEmitter`, `StreamingResponseBody`	async/streaming responses (SSE — Server-Sent Events, a one-way stream of messages from server to browser — is covered in Chapter 13)

`ResponseEntity` for 201 + `Location`

```
@PostMapping
public ResponseEntity<Book> create(
        @Validated({BookRequest.OnCreate.class, Default.class}) @RequestBody BookRequest request) {
    Book saved = repo.save(request);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{id}").buildAndExpand(saved.id()).toUri();
    return ResponseEntity.created(location).body(saved);
}
```

`ServletUriComponentsBuilder.fromCurrentRequest()` starts from the URL the client used, so the `Location` is correct behind different hosts (if you run behind a proxy — e.g. a load balancer that receives the client's request and forwards it to your app — set `server.forward-headers-strategy=framework` so `X-Forwarded-*` headers, which carry the client's original host and protocol, are honoured).

The tests

The tests use `MockMvcTester` (Framework 6.2+), the AssertJ-style (AssertJ is a fluent test-assertion library) front end for MockMvc: it calls the `DispatcherServlet` in-process with mock (fake, test-only) request/response objects — no network, no Tomcat.

```java
@WebMvcTest                        // web slice: controllers, advice, filters, WebMvcConfigurers, Jackson
@Import(BookRepository.class)      // @Repository beans are not part of the slice, so add ours
class BookApiTest {
    @Autowired MockMvcTester mvc;

    @Test
    void createReturns201WithLocation() throws Exception {
        MvcTestResult r = mvc.post().uri("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        {"title":"Release It! 2nd ed","author":"Michael T. Nygard",
                         "isbn":"9781680502398","year":2018}""")
                .exchange();
        show("POST /api/books", r);
        System.out.println("  Location = " + r.getResponse().getHeader("Location"));
        assertThat(r).hasStatus(201).hasHeader("Location", "http://localhost/api/books/6");
    }
    // list, get, update, delete tests ...
}
```

Output (all CRUD — Create, Read, Update, Delete — tests):

```
GET /api/books?page=1&size=2 -> 200 {"content":[{"id":3,"title":"Refactoring","author":"Martin
Fowler","isbn":"9780134757599","published_year":2018,"addedOn":"2026-01-15"},{"id":4,"title":"Domain-Driven Design","author":"Eric
Evans","isbn":"9780321125217","published_year":2003,"addedOn":"2026-01-15"}],"page":1,"size":2,"totalElements":5,"totalPages":3}
GET /api/books/2 -> 200 {"id":2,"title":"Effective Java","author":"Joshua
Bloch","isbn":"9780134685991","published_year":2018,"addedOn":"2026-01-15"}
  X-Request-Id = req-15
  X-Handler    = BookController#get
POST /api/books -> 201 {"id":6,"title":"Release It! 2nd ed","author":"Michael T.
Nygard","isbn":"9781680502398","published_year":2018,"addedOn":"2026-01-15"}
  Location = http://localhost/api/books/6
PUT /api/books/1 -> 200 {"id":1,"title":"Clean Code (2nd ed)","author":"Robert C.
Martin","isbn":"9780132350884","published_year":2025,"addedOn":"2026-01-15"}
DELETE /api/books/1 -> 204
GET /api/books/1 after delete -> 404
```

(`X-Request-Id` and `X-Handler` come from the filter and interceptor in section 5.11. The number in `req-15` is a counter shared by all tests in the cached context, so it depends on test order and may differ in your run.)

Pagination

`list` returns one **page** plus the numbers a client needs to build page links:

```java
public record PageResponse<T>(List<T> content, int page, int size, long totalElements, int totalPages) {}
```

- Use **zero-based** (first page is `0`) `page` and a bounded `size` (`@Max(50)`) so nobody can request a million rows.
- Return a **stable DTO** (Data Transfer Object: a simple class/record shaped for the API) like this one rather than serialising Spring Data's `Page` implementation directly — its JSON shape is not a stable API (Spring Data now warns about it and offers `@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)`). With Spring Data you would accept a `Pageable` argument (`?page=1&size=2&sort=title,asc`) — see Chapter 8.
- For very large or fast-changing data, **keyset (cursor) pagination** (`?after=lastSeenId` — "give me the rows after the last one I saw") is faster and more stable than `OFFSET` (SQL's "skip the first N rows", which still has to read them) — see the SQL book, Indexes and query performance.

Self-check

1. How do you return 201 with a `Location` header? → `ResponseEntity.created(uri).body(saved)`.
2. What status does a missing required `@RequestParam` produce? → 400 (`MissingServletRequestParameterException`).
3. What does `MockMvcTester` avoid? → Starting a server; it drives the `DispatcherServlet` directly.

5.5 JSON with Jackson 3

What it is

Jackson is the JSON library behind Spring MVC's JSON converter. **Spring Boot 4 uses Jackson 3** by default (Boot 4.1.1 manages Jackson 3.1.x). Jackson 3 is a new major version (one allowed to break compatibility) with new coordinates (the Maven group/artifact names that identify a library), so code and configuration written for Jackson 2 need small changes.

What changed from Jackson 2

Topic	Jackson 2 (Boot 3)	Jackson 3 (Boot 4)
Maven group / Java package	`com.fasterxml.jackson.*`	`tools.jackson.*`
Annotations (`@JsonProperty`, `@JsonIgnore`, ...)	`com.fasterxml.jackson.annotation`	**unchanged**: still `com.fasterxml.jackson.annotation`
Java baseline	Java 8	Java 17
Main mapper	`ObjectMapper` (mutable)	`JsonMapper` (a format-specific `ObjectMapper`), **immutable**, built with `JsonMapper.builder()`
Exceptions	checked (must be declared or caught) `JsonProcessingException`	unchecked (extends `RuntimeException`) `JacksonException` (`DatabindException`, `StreamReadException`, ...)
`java.time`, `Optional`, parameter names	separate modules (`jsr310`, `jdk8`, ...)	built in
Dates	timestamps by default (Boot turned this off)	ISO strings by default (`WRITE_DATES_AS_TIMESTAMPS` disabled)
Unknown JSON properties	fail by default (Boot turned this off)	ignored by default
Property order	declaration order	alphabetical by default, but creator (constructor/record) properties first
Renamed types	`JsonSerializer`, `JsonDeserializer`, `SerializerProvider`, `Module`	`ValueSerializer`, `ValueDeserializer`, `SerializationContext`, `JacksonModule`

Spring Boot 4 changes that follow from it:

Boot 3	Boot 4
`Jackson2ObjectMapperBuilderCustomizer`	`JsonMapperBuilderCustomizer`
`@JsonComponent`, `@JsonMixin`	`@JacksonComponent`, `@JacksonMixin`
`spring.jackson.read.*` / `spring.jackson.write.*`	`spring.jackson.json.read.*` / `spring.jackson.json.write.*`
define an `ObjectMapper` bean to customise	define/customise a `JsonMapper` bean
—	deprecated `spring-boot-jackson2` module to keep Jackson 2 while migrating
`HttpMessageConverters` bean	deprecated; use `ServerHttpMessageConvertersCustomizer` / `ClientHttpMessageConvertersCustomizer`

Example

The API record uses an annotation from the unchanged annotations package:

```
import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.LocalDate;

public record Book(
        Long id,
        String title,
        String author,
        String isbn,
        @JsonProperty("published_year") int year,   // rename one JSON field
        LocalDate addedOn) {
}
```

The test injects Boot's `JsonMapper`:

```java
@SpringBootTest
class JacksonTest {
    @Autowired JsonMapper mapper;   // Jackson 3 (package tools.jackson), configured by Boot 4

    @Test
    void serializeAndDeserializeARecord() {
        Book b = new Book(7L, "Effective Java", "Joshua Bloch", "9780134685991", 2018, LocalDate.of(2026, 1, 15));
        String json = mapper.writeValueAsString(b);          // no checked exception in Jackson 3
        System.out.println("mapper class: " + mapper.getClass().getName());
        System.out.println("JSON: " + json);

        Book back = mapper.readValue("""
                {"id":8,"title":"T","author":"A","isbn":"1","published_year":2001,
                 "addedOn":"2026-02-01","somethingNew":true}""", Book.class);
        System.out.println("read back (unknown field ignored): " + back);
    }

    @Test
    void jacksonExceptionsAreUnchecked() {
        assertThatThrownBy(() -> mapper.readValue("{not json", Book.class))
                .isInstanceOf(JacksonException.class)
                .isInstanceOf(RuntimeException.class) ... ;
    }
}
```

Output:

```
mapper class: tools.jackson.databind.json.JsonMapper
JSON: {"id":7,"title":"Effective Java","author":"Joshua Bloch","isbn":"9780134685991","published_year":2018,"addedOn":"2026-01-15"}
read back (unknown field ignored): Book[id=8, title=T, author=A, isbn=1, year=2001, addedOn=2026-02-01]
bad JSON -> StreamReadException (a RuntimeException: true)
```

Notice: `LocalDate` became `"2026-01-15"` with no extra module; the record kept its declaration order (record components are creator properties, which are sorted first); the unknown `somethingNew` was ignored. In the error responses later in this chapter, `ProblemDetail`'s fields *do* appear alphabetically (`detail, instance, status, title, type`) — that is Jackson 3's new default ordering for regular properties.

Customising

```java
@Bean
JsonMapperBuilderCustomizer jsonCustomizer() {
    return builder -> builder.enable(SerializationFeature.INDENT_OUTPUT);   // tools.jackson.databind
}
```

or with properties, e.g. `spring.jackson.serialization.indent-output=true`, `spring.jackson.default-property-inclusion=non_null`. (`spring.jackson.json.read.*` / `json.write.*` hold the JSON-format-specific `JsonReadFeature`/`JsonWriteFeature` switches.)

Common mistakes

- Mixing Jackson 2 and 3 imports (`com.fasterxml.jackson.databind.ObjectMapper` vs `tools.jackson.databind...`): Spring MVC uses the Jackson 3 mapper, so a Jackson 2 `ObjectMapper` bean you define is simply not used for HTTP. (Some libraries, e.g. springdoc's swagger-core, still bring Jackson 2 for their own use — that is fine.)
- Exposing JPA entities (classes mapped to database tables by JPA, Java's object-database mapping standard) directly as JSON → lazy-loading errors (related data loaded only on first access, after the database session is closed), infinite recursion (two objects that point at each other are serialised forever), leaking columns. Use DTOs/records (Chapter 8).
- Relying on field order in clients — JSON objects are unordered by definition.

5.6 Validation

What it is

Jakarta Bean Validation (the standard Java API for validation rules; Hibernate Validator is the implementation; add `spring-boot-starter-validation`) lets you declare rules as annotations on fields/record components and parameters. Spring MVC runs them before your method executes, so the controller only ever sees valid input.

```java
public record BookRequest(
        @NotBlank @Size(max = 100) String title,
        @NotBlank String author,
        // required only when creating (group OnCreate); an update may leave it out
        @NotBlank(groups = OnCreate.class)
        @Pattern(regexp = "\\d{13}", message = "must be 13 digits") String isbn,
        @Min(1450) @Max(2100) int year) {

    /** Validation group marker: rules that apply only on create. */
    public interface OnCreate {}
}
```

Common constraints: `@NotNull`, `@NotBlank` (not null and not only whitespace), `@NotEmpty`, `@Size`, `@Min`/`@Max`, `@Positive`, `@Email`, `@Pattern`, `@Past`/`@Future`, `@Valid` (cascade into a nested object or list elements). Custom constraints: an annotation + a `ConstraintValidator`.

Three ways validation is triggered

Where	How to trigger	Failure exception	Default status
`@RequestBody` / `@ModelAttribute` object	`@Valid` or `@Validated` on the parameter	`MethodArgumentNotValidException`	400
Constraints directly on `@RequestParam`, `@PathVariable`, `@RequestHeader` (e.g. `@Max(50) int size`)	automatic — **built-in method validation** since Framework 6.1, no `@Validated` needed	`HandlerMethodValidationException`	400
Any Spring bean method (services)	`@Validated` on the **class** + constraints on parameters	`ConstraintViolationException` (AOP proxy, see Chapter 4)	500 unless you handle it

Do **not** put `@Validated` on the controller class in Framework 6.1+: it switches that controller to the AOP-based validation of the third row, and you get `ConstraintViolationException` instead of the web-aware exceptions.

Validation groups

A **group** is a marker interface (an empty interface used only as a label) that labels constraints so you can validate different subsets in different situations. Constraints without `groups` belong to `jakarta.validation.groups.Default`. `@Valid` cannot select groups; Spring's `@Validated({...})` can:

```
@PostMapping                    // create: Default rules + OnCreate rules (isbn required)
public ResponseEntity<Book> create(
        @Validated({BookRequest.OnCreate.class, Default.class}) @RequestBody BookRequest request) { ... }

@PutMapping("/{id:\\d+}")       // update: only Default rules (isbn optional)
public Book update(@PathVariable long id, @Validated @RequestBody BookRequest request) { ... }
```

Test output (the same body without `isbn`):

```
POST without isbn -> 400 errors={"isbn":"must not be blank"}
PUT  without isbn -> 200
```

Invalid body and invalid query parameter:

```
POST invalid book -> 400 application/problem+json
   {"detail":"Request body has 3 invalid field(s)","instance":"/api/books","status":400,"title":"Bad Request","errors":{"isbn":"must be 13
digits","title":"must not be blank","year":"must be greater than or equal to 1450"}}
GET /api/books?size=500 -> 400 application/problem+json
   {"detail":"Validation failure","instance":"/api/books","status":400,"title":"Bad Request","errors":{"size":"must be less than or equal to
50"}}
```

The `errors` maps are added by the exception handler in the next section.

Common mistakes

- Forgetting `@Valid`/`@Validated` on `@RequestBody` → annotations on the record are ignored.
- Forgetting `@Valid` on a nested field (`@Valid Address address`) → nested object not validated.
- Missing `spring-boot-starter-validation` → no validator on the classpath, constraints silently ignored.
- Using `@NotNull` on a primitive `int` (a plain value type that can never be `null`) → never null; use a wrapper `Integer` (the object version, which can be `null`) if "missing" must be detected.
- Validating only in the UI → every API must validate its own input.

Self-check

1. How do you validate `size` ≤ 50 on a `@RequestParam` in Framework 7? → Put `@Max(50)` on the parameter; built-in method validation handles it (400 `HandlerMethodValidationException`).
2. Why `@Validated` rather than `@Valid` for groups? → `@Valid` (Jakarta) has no groups attribute; Spring's `@Validated` does.
3. What is the default group? → `jakarta.validation.groups.Default` — every constraint without `groups`.

5.7 Error handling and Problem Details (RFC 9457)

The problem it solves

Without a plan, errors leak as stack traces (the list of method calls active when an exception happened), HTML error pages, or a different JSON shape per endpoint. Clients need **one predictable error format**. **RFC 9457 "Problem Details for HTTP APIs"** (July

2023, replacing RFC 7807) defines it: media type (the format label used in `Content-Type`) `application/problem+json` with members

Member	Meaning
`type`	URI (a string that identifies a resource; URLs are one kind) identifying the problem type (default `about:blank`)
`title`	short human summary of the type
`status`	HTTP status code
`detail`	explanation of this occurrence
`instance`	URI of this occurrence (Spring fills the request path)
extensions	any extra members, e.g. `bookId`, `errors`

Spring's model: `ProblemDetail` (the body), `ErrorResponse` (an exception that carries a `ProblemDetail`; all Spring MVC exceptions implement it), `ErrorResponseException` (a base class for your own).

How it works

1. Controller code throws.
2. `ExceptionHandlerExceptionResolver` looks for an `@ExceptionHandler` for that type — first in the controller, then in `@ControllerAdvice` / `@RestControllerAdvice` beans (global handlers).
3. Otherwise `ResponseStatusExceptionResolver` (for `@ResponseStatus` or `ResponseStatusException`) and `DefaultHandlerExceptionResolver` (Spring MVC's own exceptions) set a status.
4. Anything still unhandled goes to the servlet container's (the web server, here Tomcat) error page, which Boot maps to `/error` → `BasicErrorController`, producing Boot's JSON (`timestamp`, `status`, `error`, `path`).

The cleanest setup: one `@RestControllerAdvice` that **`extends`** `ResponseEntityExceptionHandler`. The base class already maps every Spring MVC exception (400, 404, 405, 406, 413, 415, 500 ...) to a `ProblemDetail`; you add handlers for your own exceptions and override the hooks you want to enrich.

```
@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(BookNotFoundException.class)
    public ProblemDetail bookNotFound(BookNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Book not found");
        pd.setType(URI.create("https://api.example.com/problems/book-not-found"));
        pd.setProperty("bookId", ex.bookId());          // extra member in the JSON
        return pd;
    }

    // Customise the built-in 400 for @Valid/@Validated @RequestBody failures: add field errors
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers, HttpStatusCode status, WebRequest request) {
        ProblemDetail pd = ex.getBody();                // already a ProblemDetail (status 400)
        Map<String, String> errors = new TreeMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(fe -> errors.put(fe.getField(), fe.getDefaultMessage()));
        pd.setDetail("Request body has " + errors.size() + " invalid field(s)");
        pd.setProperty("errors", errors);
        return ResponseEntity.status(status).headers(headers).body(pd);
    }

    // Customise the built-in 400 for constraint violations on @RequestParam/@PathVariable/... (Framework 6.1+)
    @Override
    protected ResponseEntity<Object> handleHandlerMethodValidationException(
            HandlerMethodValidationException ex, HttpHeaders headers, HttpStatusCode status, WebRequest request) {
        ProblemDetail pd = ex.getBody();
        Map<String, String> errors = new TreeMap<>();
        ex.getParameterValidationResults().forEach(result -> result.getResolvableErrors().forEach(err ->
                errors.put(result.getMethodParameter().getParameterName(), err.getDefaultMessage())));
        pd.setProperty("errors", errors);
        return ResponseEntity.status(status).headers(headers).body(pd);
    }
}
```

Test output — our own exception and several framework errors, all in the same format:

```
GET /api/books/99 -> 404 application/problem+json
   {"detail":"Book 99 does not exist","instance":"/api/books/99","status":404,"title":"Book not
found","type":"https://api.example.com/problems/book-not-found","bookId":99}
GET /api/books?page=abc -> 400 application/problem+json
   {"detail":"Failed to convert 'page' with value: 'abc'","instance":"/api/books","status":400,"title":"Bad Request"}
PATCH /api/books/1 -> 405 application/problem+json
   {"detail":"Method 'PATCH' is not supported.","instance":"/api/books/1","status":405,"title":"Method Not Allowed"}
POST text/plain -> 415 application/problem+json
   {"detail":"Content-Type 'text/plain;charset=UTF-8' is not supported.","instance":"/api/books","status":415,"title":"Unsupported Media
Type"}
POST broken JSON -> 400 application/problem+json
   {"detail":"Failed to read request","instance":"/api/books","status":400,"title":"Bad Request"}
```

The 405 response also carries an `Allow` header listing the supported methods, and the 415 an `Accept` header listing supported body types. Note the generic `detail` for broken JSON: Spring deliberately does not echo parser internals back to clients.

Other options

Option	Use when
`spring.mvc.problemdetails.enabled=true`	you have **no** advice of your own and just want Problem Details for Spring MVC's exceptions (Boot registers a `ResponseEntityExceptionHandler` for you; default `false`)
`@ResponseStatus(HttpStatus.NOT_FOUND)` on an exception class	quick and simple, but no custom body
`throw new ResponseStatusException(HttpStatus.CONFLICT, "...")`	one-off errors without a custom class
extend `ErrorResponseException`	your exception carries its own `ProblemDetail` (status, type, extra properties)
`@ExceptionHandler` inside one controller	an error specific to that controller
`@RestControllerAdvice(basePackages = ...)` / `assignableTypes`	different handlers for different API areas

Common mistakes

- **Catching exceptions in controllers** and returning `200` with an `"error"` field — breaks HTTP semantics, caching and client retries.
- **Leaking internals** (`ex.getMessage()` from a SQL exception, stack traces) in `detail`. Log them server-side; return a safe message.
- **A generic `@ExceptionHandler(Exception.class)` returning 400** — bugs are 500s. Keep a catch-all that returns 500 and logs.
- Two advices handling the same exception without `@Order` — which wins is undefined.

In the interview

30-second answer: "I use a single `@RestControllerAdvice` that extends `ResponseEntityExceptionHandler`, so every Spring MVC exception already produces an RFC 9457 `ProblemDetail` with `application/problem+json`. I add `@ExceptionHandler` methods for domain exceptions — e.g. `NotFound` → 404 with a `type` URI and extra properties — and override `handleMethodArgumentNotValid` to include field errors. Unexpected exceptions become 500s with a safe message and are logged with a correlation id (an id attached to every log line of one request, so you can find them all)."

- *What's RFC 9457?* → Problem Details for HTTP APIs; it obsoletes RFC 7807; fields `type`, `title`, `status`, `detail`, `instance` plus extensions.
- *Order of lookup?* → `@ExceptionHandler` in the controller, then in `@ControllerAdvice` beans, then `@ResponseStatus`/`ResponseStatusException`, then Spring defaults, finally Boot's `/error`.

5.8 Content negotiation

What it is

Content negotiation is how client and server agree on a representation. The client sends `Accept: text/csv` (what it wants back) and `Content-Type` (what it is sending). Spring matches `Accept` against each method's `produces` and the available `HttpMessageConverter`s; `Content-Type` against `consumes`.

```java
@GetMapping(value = "/export", produces = MediaType.APPLICATION_JSON_VALUE)
public List<Book> exportJson() { return repo.findAll(null); }

@GetMapping(value = "/export", produces = "text/csv")
public String exportCsv() {
    return repo.findAll(null).stream()
            .map(b -> b.id() + "," + b.title() + "," + b.year())
            .collect(Collectors.joining("\n", "id,title,year\n", "\n"));
}
```

Output:

```
Accept: text/csv -> text/csv;charset=UTF-8
   id,title,year
   1,Clean Code,2008
   2,Effective Java,2018
Accept: application/json -> application/json starts [{"id":1,"title":"Clean Code","author":"...
Accept: application/xml -> 406
```

- No `Accept` header (or `*/*`) → the first producible type wins.
- XML would need a converter: add `tools.jackson.dataformat:jackson-dataformat-xml` (Jackson 3) and Boot registers it automatically.
- Query-parameter negotiation (`?format=csv`) can be turned on with `spring.mvc.contentnegotiation.favor-parameter=true` (parameter name `format`, changeable with `spring.mvc.contentnegotiation.parameter-name`). Negotiation by path extension (`/books.json`) was deprecated in Framework 5.3 and is not available with the modern `PathPattern` matching.

5.9 API versioning (Framework 7)

What it is

Public APIs change. **API versioning** lets old clients keep the old contract while new clients use a new one. Before Framework 7 every team invented its own scheme (`/v1/...` prefixes, custom header conditions). **Spring Framework 7.0 made it first-class**: a version is resolved from each request, parsed, validated, and used as one more mapping condition, with support on the client side (`RestClient`, `WebClient`, HTTP interfaces — Chapter 6) and in tests.

How it works

1. A **resolver** reads the version from the request: header, query parameter, path segment, or media-type parameter.
2. A **parser** turns it into a comparable version (default: semantic versioning `major.minor.patch` — major for breaking changes, minor for new features, patch for fixes — so `1.1` = `1.1.0`).
3. The version is **validated**: if missing and no default → 400 `MissingApiVersionException`; if not supported → 400 `InvalidApiVersionException`.
4. Mappings with a `version` attribute are matched: `"1.0"` = exactly that version; `"1.1+"` = a **baseline** — 1.1 and higher, until a more specific mapping for a higher version exists. Mappings without `version` match any version.
5. An optional **deprecation handler** adds standard `Deprecation` (RFC 9745), `Sunset` (RFC 8594) and `Link` headers to responses for deprecated versions.

Configuration

In Boot 4 via properties (or `WebMvcConfigurer.configureApiVersioning(ApiVersionConfigurer)` in plain Spring):

```properties
# Framework 7 API versioning, configured through Boot 4 properties
spring.mvc.apiversion.use.header=X-API-Version
spring.mvc.apiversion.default=1.0
# versions used in @GetMapping(version=...) are detected automatically; list extra ones here
spring.mvc.apiversion.supported=1.2
```

Property	Meaning
`spring.mvc.apiversion.use.header` / `.use.query-parameter` / `.use.path-segment` / `.use.media-type-parameter`	where the version comes from
`spring.mvc.apiversion.default`	version assumed when a request has none
`spring.mvc.apiversion.required`	reject requests without a version
`spring.mvc.apiversion.supported`	extra supported versions
`spring.mvc.apiversion.detect-supported`	detect supported versions from mappings

Beans of type `ApiVersionResolver`, `ApiVersionParser` and `ApiVersionDeprecationHandler` are picked up by Boot's MVC auto-configuration for full control. Our deprecation handler:

```
@Bean
ApiVersionDeprecationHandler deprecationHandler() {
    var handler = new StandardApiVersionDeprecationHandler();
    handler.configureVersion("1.0")
            .setDeprecationDate(ZonedDateTime.of(2026, 6, 1, 0, 0, 0, 0, ZoneOffset.UTC))
            .setSunsetDate(ZonedDateTime.of(2027, 1, 1, 0, 0, 0, 0, ZoneOffset.UTC))
            .setDeprecationLink(URI.create("https://api.example.com/docs/migrate-to-v2"));
    return handler;
}
```

The controller:

```
@RestController
@RequestMapping("/api/greeting")
public class GreetingController {
    public record GreetingV1(String message) {}
    public record GreetingV1_1(String message, String language) {}
    public record GreetingV2(String text, String lang) {}

    @GetMapping(version = "1.0")            // exactly version 1.0
    public GreetingV1 v1()     { return new GreetingV1("Hello"); }

    @GetMapping(version = "1.1+")           // baseline: 1.1 and every later version...
    public GreetingV1_1 v1_1() { return new GreetingV1_1("Hello", "en"); }

    @GetMapping(version = "2.0")            // ...until a more specific mapping takes over
    public GreetingV2 v2()     { return new GreetingV2("Hello", "en"); }
}
```

Output of `ApiVersioningTest`:

```
X-API-Version: none -> 200 {"message":"Hello"}
X-API-Version: 1.0  -> 200 {"message":"Hello"}
X-API-Version: 1.1  -> 200 {"message":"Hello","language":"en"}
X-API-Version: 1.2  -> 200 {"message":"Hello","language":"en"}
X-API-Version: 2.0  -> 200 {"text":"Hello","lang":"en"}
X-API-Version: 3.0  -> 400
Deprecation: @1780272000
Sunset: Fri, 1 Jan 2027 00:00:00 GMT
Link: <https://api.example.com/docs/migrate-to-v2>; rel="deprecation"; type="text/html"
```

- No header → default `1.0`. `1.2` has no exact mapping, so the `1.1+` baseline serves it (1.2 is supported because we listed it). `3.0` is unsupported → 400.
- `Deprecation: @1780272000` is the RFC 9745 format: `@` + Unix seconds (seconds since 1 Jan 1970 UTC; here 2026-06-01T00:00Z).

Trade-offs: where to put the version

Strategy	Example	Pros	Cons
Path segment	`/api/v2/books`	visible, easy to test in a browser, cache-friendly	"the same resource" has several URLs
Header	`X-API-Version: 2.0`	clean URLs	invisible in logs/links unless you add it; needs `Vary` (a header telling caches which request headers change the response) for caches
Query parameter	`?version=2.0`	easy to try	mixes with filters, easy to drop
Media type parameter	`Accept: application/json;version=2.0`	"pure" HTTP content negotiation	least discoverable, harder tooling

Whatever you choose, prefer **additive, backwards-compatible changes** (new optional fields that old clients can ignore) and bump the version only for breaking changes (changes that make existing clients fail).

5.10 CORS and file upload

CORS

What it is

Browsers enforce the **same-origin policy** (an *origin* is scheme + host + port, e.g. `https://shop.example.com`): JavaScript on `https://shop.example.com` may not read responses from `https://api.example.com` unless the API opts in with **CORS** (Cross-Origin Resource Sharing) headers. For "non-simple" requests (e.g. JSON `POST`, custom headers) the browser first sends a **preflight** `OPTIONS` request asking permission.

CORS protects **users' browsers**, not your server: `curl` and server-to-server calls ignore it. It is not authentication.

How to configure

Globally (preferred):

```
@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
            .allowedOrigins("https://shop.example.com")   // the browser app's origin
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .maxAge(3600);                                // preflight may be cached 1 h
}
```

or per controller/method with `@CrossOrigin(origins = "...")`. With Spring Security, also enable `http.cors(...)` so the security filter chain lets preflights through (Chapter 10).

Test output (preflights from an allowed and an unknown origin):

```
preflight from shop.example.com -> 200 Allow-Origin=https://shop.example.com Allow-Methods=GET,POST,PUT,DELETE Max-Age=3600
preflight from evil.example.net -> 403 Allow-Origin=null
```

Common mistakes

- `allowedOrigins("*")` together with `allowCredentials(true)` — not allowed by the CORS spec (Spring rejects it); use explicit origins or `allowedOriginPatterns`.
- Configuring CORS in MVC but not in Spring Security → preflight gets 401/403 from security first.

File upload

Multipart (`multipart/form-data`) requests carry files plus form fields. Boot configures multipart support; limits:

```
spring.servlet.multipart.max-file-size=1MB       # default 1MB
spring.servlet.multipart.max-request-size=2MB    # default 10MB
```

```
@PostMapping("/api/uploads")
public UploadResult upload(@RequestPart("file") MultipartFile file,
                           @RequestParam(defaultValue = "") String note) throws IOException {
    String text = new String(file.getBytes(), StandardCharsets.UTF_8);
    return new UploadResult(file.getOriginalFilename(), file.getSize(),
            file.getContentType(), (int) text.lines().count(), note);
}
```

Output:

```
POST /api/uploads -> 200 {"fileName":"books.csv","bytes":36,"contentType":"text/csv","lines":3,"note":"weekly import"}
```

- Exceeding the limit in a running server raises `MaxUploadSizeExceededException`, which `ResponseEntityExceptionHandler` maps to **413**. (MockMvc does not parse real multipart streams, so the limit is not exercised by this test.)
- **Security:** never trust `getOriginalFilename()` as a path (path traversal: `../../etc/passwd`); generate your own storage names, check type and size, and stream large files (`file.getInputStream()` / `transferTo`) rather than loading them into memory. Large files usually go straight to object storage (a file store such as Amazon S3) via pre-signed URLs (temporary upload links signed by your server) instead of through your API.

5.11 Filters vs interceptors (vs AOP)

What it is

Three places to put code that runs around requests:

	Servlet `Filter`	Spring MVC `HandlerInterceptor`	AOP aspect (Ch. 4)
Runs	before/after the `DispatcherServlet`, for every request (also static files, errors)	inside the `DispatcherServlet`, only for requests mapped to a handler	around bean method calls
Knows the controller method?	no	yes (`HandlerMethod`)	yes (join point)
Can wrap/replace request & response objects	yes	no	no
Typical use	security (Spring Security is a filter chain), request ids/MDC (Mapped Diagnostic Context: per-thread values added to every log line), compression, CORS	per-endpoint auth checks, locale, audit, timing by handler	business-level cross-cutting concerns
Register	`@Component` implementing `Filter` (often `OncePerRequestFilter`) or `FilterRegistrationBean`	`WebMvcConfigurer.addInterceptors`	`@Aspect` `@Component`

The chapter's filter adds a correlation id to every response:

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class RequestIdFilter extends OncePerRequestFilter {
    private final AtomicLong counter = new AtomicLong();

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = req.getHeader("X-Request-Id");
        if (id == null) id = "req-" + counter.incrementAndGet();
        res.setHeader("X-Request-Id", id);   // echo it so clients and logs can correlate
        chain.doFilter(req, res);            // continue to the next filter / the DispatcherServlet
    }
}
```

The interceptor records which controller method was chosen — something a filter cannot know:

```
public class HandlerNameInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
        if (handler instanceof HandlerMethod hm) {
            res.setHeader("X-Handler", hm.getBeanType().getSimpleName() + "#" + hm.getMethod().getName());
        }
        return true;                         // false would stop the request here
    }
}
```

registered with `registry.addInterceptor(new HandlerNameInterceptor()).addPathPatterns("/api/**")`. The earlier output showed both headers: `X-Request-Id = req-15`, `X-Handler = BookController#get`.

Interceptor callbacks: `preHandle` (before the method; return `false` to stop), `postHandle` (after the method, **before** the response is written for views — for `@ResponseBody` methods the body is already written, so headers added here are too late), `afterCompletion` (always, even on exceptions — use for cleanup/metrics).

`@WebMvcTest` includes `Filter` beans, `WebMvcConfigurer`s and interceptors, which is why the tests see both headers.

5.12 Documenting the API: OpenAPI with springdoc

What it is

OpenAPI (formerly Swagger) is a machine-readable description of an HTTP API: paths, parameters, bodies, responses. Tools generate client SDKs (ready-made client libraries), contract tests (tests that check the real service matches the published description) and interactive docs from it. **springdoc-openapi** generates the document from your Spring MVC controllers at runtime. Its **3.x** line targets **Spring Boot 4** (2.x is for Boot 3); this project uses 3.1.1.

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-api</artifactId>   <!-- -ui adds Swagger UI -->
  <version>3.1.1</version>
</dependency>
```

- `/v3/api-docs` serves the JSON document (`/v3/api-docs.yaml` for YAML).
- With `springdoc-openapi-starter-webmvc-ui`, Swagger UI appears at `/swagger-ui.html`.
- It reads your mappings, parameter annotations and Bean Validation constraints (`@Max(50)` becomes `maximum: 50`). Add detail with `io.swagger.v3.oas.annotations` (`@Operation`, `@ApiResponse`, `@Schema`).

The test fetches the document:

```
@SpringBootTest                // full context: springdoc is not part of the @WebMvcTest slice
@AutoConfigureMockMvc          // Boot 4: @SpringBootTest no longer adds MockMvc on its own
class OpenApiTest { ... mvc.get().uri("/v3/api-docs") ... }
```

Output:

```
openapi: 3.1.0
  /api/books/{id} [get, put, delete]
  /api/uploads [post]
  /api/books [get, post]
  /api/books/export [get]
```

springdoc logs a warning that `/v3/api-docs` is enabled by default — in production either secure it or set `springdoc.api-docs.enabled=false` (and `springdoc.swagger-ui.enabled=false`).

Code-first vs design-first: springdoc is *code-first* (document generated from code). Many API-platform teams go *design-first*: write the OpenAPI file, review it, then generate server interfaces/clients with `openapi-generator`. Both are fine; the key is that the published spec is tested against the real service.

HATEOAS in brief

HATEOAS (Hypermedia as the Engine of Application State) means responses include **links** to related actions (`"_links": {"self": ..., "next": ...}`), so clients navigate the API like a website instead of hard-coding URLs. **Spring HATEOAS** (`spring-boot-starter-hateoas`) provides `EntityModel`, `CollectionModel`, `PagedModel` and `linkTo(methodOn(...))` builders, rendering HAL JSON (a common JSON convention for embedding links). It is valuable for long-lived public APIs; most internal JSON APIs skip it and rely on OpenAPI documentation instead.

5.13 Putting it together: design checklist and mistakes

Area	Do
URLs	plural nouns (`/books`, `/books/{id}`), no verbs; nested only for true ownership (`/orders/{id}/items`)
Methods & codes	follow section 5.1; `201 + Location` on create, `204` on delete
DTOs	separate request/response records; never expose entities
Validation	Bean Validation on every input; groups for create vs update
Errors	one `@RestControllerAdvice`, RFC 9457 `ProblemDetail`, no internals
Lists	always paginate, cap `size`, stable JSON shape
Versioning	pick one strategy; Framework 7 `version` attribute; deprecate with headers
Security	authentication/authorization (Ch. 10), CORS explicit origins, upload limits
Docs	OpenAPI published and tested
Observability (being able to see what a running system is doing, via logs, metrics and traces)	request id filter, metrics per endpoint (Ch. 15)
Threads	Spring MVC is thread-per-request; with `spring.threads.virtual.enabled=true` Tomcat uses virtual threads (cheap JVM-managed threads, so thousands can wait at once), giving high concurrency without reactive (non-blocking, callback/stream-based) code (Ch. 13)

Functional endpoints. Besides annotated controllers, Spring MVC offers `RouterFunction` / `HandlerFunction` ("WebMvc.fn"): routes are declared as code (`route().GET("/api/books/{id}", handler::get).build()`). Same `DispatcherServlet`, same converters; choose it if you prefer explicit routing over annotations. It also supports API versioning.

Common mistakes (whole chapter)

Symptom	Cause	Fix
404 for `/api/books/`	trailing slash no longer matched	call without slash or map both
400 "Failed to convert 'id'" for a literal path	`/{id}` pattern catches `/export`	regex `{id:\d+}`
Validation ignored	missing `@Valid` / `@Validated` or validation starter	add them
`ConstraintViolationException` → 500 from a controller	`@Validated` on the controller class	remove it; rely on built-in method validation
Errors in five different JSON shapes	ad-hoc try/catch in controllers	central advice + `ProblemDetail`
Browser "CORS error", curl works	missing CORS config (or blocked by security)	`addCorsMappings` + `http.cors()`
Header set in `postHandle` missing	`@ResponseBody` already wrote the response	set it in `preHandle` or a filter
`@SpringBootTest` test can't autowire `MockMvc` (Boot 4)	MockMvc no longer auto-added	`@AutoConfigureMockMvc`

5.14 In the interview

Q: Walk me through what happens when a request hits a Spring MVC app. Filters → `DispatcherServlet` → `HandlerMapping` finds the controller method (path, method, consumes/produces, version) → interceptors' `preHandle` → `HandlerAdapter` resolves arguments (message converters for bodies, conversion, validation) → method runs → return value written by a message converter chosen by `Accept` → exceptions go to `HandlerExceptionResolver`s (`@ExceptionHandler`/advice) → `afterCompletion` → filters.

- *Filter vs interceptor?* → Filter: servlet level, every request, can wrap request/response, doesn't know the handler. Interceptor: MVC level, knows the `HandlerMethod`, only mapped requests.
- *`@Controller` vs `@RestController`?* → The latter adds `@ResponseBody`.

Q: How do you validate input and report errors? → Bean Validation annotations on DTOs + `@Valid` / `@Validated` (groups via `@Validated`); constraints on parameters are validated automatically since 6.1; a global `@RestControllerAdvice` extending `ResponseEntityExceptionHandler` returns RFC 9457 `ProblemDetail`s with field errors.

Q: PUT vs PATCH vs POST? Which are idempotent? → POST creates (not idempotent); PUT replaces (idempotent); PATCH partially updates (not necessarily idempotent); GET/DELETE idempotent.

Q: How would you version an API in Spring Boot 4? → Framework 7 API versioning: configure a resolver (`spring.mvc.apiversion.use.header=X-API-Version` or path segment), set default/supported versions, use `@GetMapping(version = "1.1+")` baselines, add a deprecation handler that sends `Deprecation` / `Sunset` / `Link` headers; clients use `ApiVersionInserter`.

Q: What changed with JSON in Boot 4? → Jackson 3: `tools.jackson` packages (annotations unchanged), immutable `JsonMapper`, unchecked exceptions, java.time built in, ISO dates by default; Boot's customizer is `JsonMapperBuilderCustomizer`, `@JsonComponent` → `@JacksonComponent`, JSON read/write properties moved under `spring.jackson.json.*`.

Q: What is CORS and does it secure my API? → A browser mechanism that lets a page from one origin read responses from another if the server allows it via headers and preflight. It doesn't secure the API against non-browser clients — you still need authentication and authorization.

5.15 Chapter TL;DR

- `spring-boot-starter-webmvc` (Boot 4 name) gives Spring MVC on embedded Tomcat with Jackson 3.
- Request flow: **filters** → **`DispatcherServlet`** → **HandlerMapping** → **interceptors** → **argument resolution (converters + validation)** → **controller** → **message converter (by `Accept`)** → **exception resolvers**.
- `@RestController` + `@GetMapping/@PostMapping/...`; `@PathVariable`, `@RequestParam`, `@RequestHeader`, `@RequestBody`; `ResponseEntity.created(uri)` for 201; `@ResponseStatus(NO_CONTENT)` for 204. Constrain path variables (`{id:\d+}`) to avoid ambiguous matches.
- **Jackson 3**: `tools.jackson.*`, annotations unchanged, immutable `JsonMapper`, unchecked `JacksonException`, java.time built in.
- **Validation**: `@Valid` / `@Validated` on bodies (groups via `@Validated`), parameter constraints validated automatically (6.1+), don't put `@Validated` on controllers.
- **Errors**: one `@RestControllerAdvice extends ResponseEntityExceptionHandler`, returning **RFC 9457 `ProblemDetail`** (`application/problem+json`).
- **Content negotiation** via `Accept` + `produces`; 406 if impossible. Paginate with a bounded `size` and a stable DTO.
- **API versioning** (Framework 7): `spring.mvc.apiversion.*`, `version = "1.0"` / `"1.1+"`, 400 for unsupported versions, `Deprecation` / `Sunset` headers.
- **CORS** in `addCorsMappings` (+ Spring Security); **uploads** with `MultipartFile` and size limits; **filters** vs **interceptors**; **springdoc 3.x** for OpenAPI on Boot 4.
- Next: Chapter 6 — REST clients and HTTP interfaces.

Chapter 6 — REST Clients and HTTP Interfaces

Chapter 5 built an API. Real services also **call** APIs: a payment provider, another team's microservice (a small, independently deployed service that owns one business area), a search engine. Calling over the network brings problems you never have with a local method call — slowness, timeouts (giving up after a set wait), partial failures (some calls or servers fail while others work), error bodies, versioning, retries. This chapter covers Spring's HTTP clients and how to use them safely:

What you'll learn

1. The four Spring HTTP clients — `RestClient`, `RestTemplate`, `WebClient`, HTTP interfaces — and which to choose (including `RestTemplate`'s deprecation timeline).
2. `RestClient` in depth: requests, bodies, `retrieve()` vs `exchange()`, `ResponseEntity`, generics.
3. Error handling: Spring's exception hierarchy, status handlers, reading Problem Details from error bodies, mapping to your own exceptions.
4. Timeouts and the underlying HTTP library (the code that actually opens connections and sends bytes: the JDK's built-in client, Apache HttpClient, Jetty, Reactor Netty) — Boot 4's `spring.http.clients.*`.
5. Interceptors (code that wraps every outgoing request) for cross-cutting concerns — things every call needs, such as headers, logging, tracing (following one request across services).
6. **HTTP service interfaces** (`@HttpExchange`) and **Boot 4's HTTP service client auto-configuration** (`@ImportHttpServices`, `spring.http.serviceclient.*`).
7. Client-side **API versioning** (Framework 7).
8. **Retries** with Framework 7 `@Retryable` — and when not to retry.
9. Testing clients: `@RestClientTest` + `MockRestServiceServer`, and tests against a real local server.

Code: `code/rest-clients/` — Spring Boot 4.1.1, Framework 7.0, Java 25. To make the tests hit a *real* HTTP server without the internet, the project also contains a small "remote" catalog API (`CatalogServerController`) that the test starts on a free local port; imagine it belongs to another team. All output shown is real.

```
<!-- RestClient.Builder, RestTemplateBuilder, HTTP service client auto-configuration (Boot 4 starter) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient</artifactId>
</dependency>
```

(An app that already has `spring-boot-starter-webmvc` for its own endpoints still adds `spring-boot-starter-restclient` for the client side — Boot 4's starters are modular.)

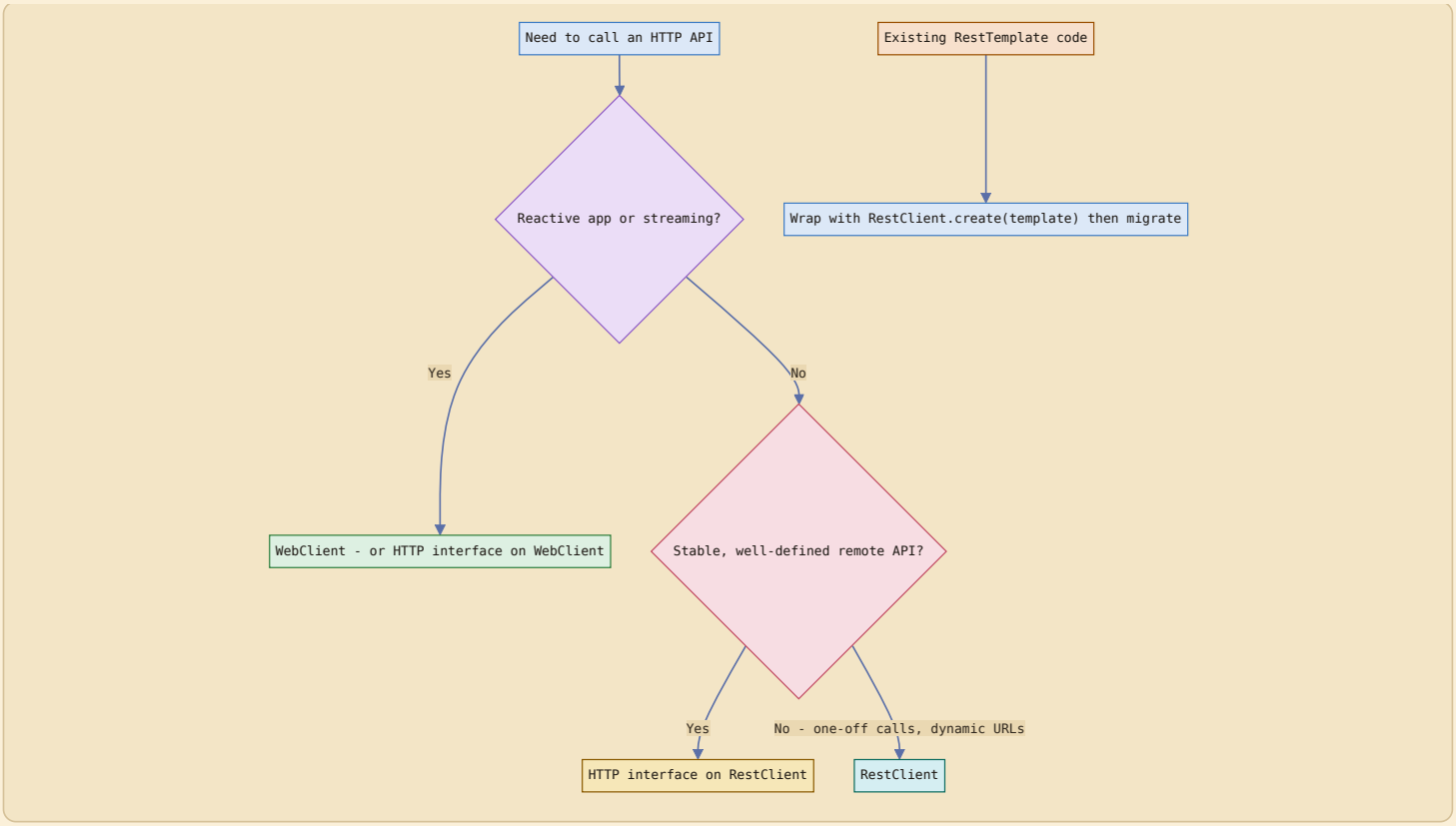
6.1 Choosing a client

What they are

Client	Style	Since	Status (Framework 7)	Use for
`RestClient`	synchronous (the calling thread waits for the response), **fluent** (calls chained with dots: `get().uri(...).retrieve()`) API	Framework 6.1 (2023)	the recommended blocking client	new code in Spring MVC / non-reactive apps
`RestTemplate`	synchronous, template methods (`getForObject`, `exchange`, ...)	Spring 3.0 (2009)	**deprecation announced**: intent to deprecate in 7.0, planned `@Deprecated` in 7.1 (Nov 2026), removal in 8.0	existing code only; migrate to `RestClient`
`WebClient`	non-blocking, reactive (the thread is freed while waiting; results arrive later as `Mono` = 0–1 value or `Flux` = 0–N values)	Framework 5.0	recommended reactive client	WebFlux apps, streaming, reactive pipelines (Chapter 13)
HTTP interface (`@HttpExchange`)	**declarative** (you describe *what* to call, not *how*): an annotated Java interface, Spring generates the implementation	Framework 6.0; Boot auto-config in 4.0	recommended for calling well-defined APIs	typed clients for other services; runs on `RestClient` or `WebClient`

Why `RestClient` replaced `RestTemplate`: the same underlying machinery (request factories, message converters, interceptors) but a modern fluent API instead of dozens of overloaded methods (methods with the same name but different parameter lists), and one place for all new features (API versioning, `requiredBody`, status handlers). The Spring team's plan (spring.io blog, "The state of HTTP clients in Spring", September 2025) keeps `RestTemplate` working through the 7.x line — so no panic, but no new code on it.

Do you need `WebClient` in a Spring MVC app? Not any more. Before `RestClient`, people pulled in WebFlux just to use `WebClient` and called `.block()`. With `RestClient` — and **virtual threads** (lightweight threads managed by the JVM, so millions can wait at once) making blocking calls cheap (Java book 15.4) — the blocking client is the simpler choice for MVC apps.



Self-check

1. Which client should new blocking code use? → `RestClient` (directly or behind an HTTP interface).
2. When is `RestTemplate` scheduled to be removed? → Framework 8.0; formally deprecated from 7.1.
3. Do you need WebFlux to use HTTP interfaces? → No; they run on `RestClient` too.

6.2 `RestClient` basics

How it works

1. Get a **builder**. In Boot, inject `RestClient.Builder` — a **prototype** bean (a new instance for every injection point) pre-configured with Boot's message converters (Jackson 3, which turns Java objects into JSON and back), the chosen HTTP library, timeouts from `spring.http.clients.*`, and (when Micrometer, Spring's metrics library, is present) observability (metrics and traces for each call). Each injection point gets a fresh builder, so configuring one does not affect others. (Outside Boot: `RestClient.builder()` or `RestClient.create("https://...")`.)
2. Set shared settings (`baseUrl`, `defaultHeader`, `defaultStatusHandler`, `requestInterceptor`, `requestFactory`) and `build()`. A `RestClient` is **immutable** (cannot change after it is built) **and thread-safe** (many threads may use it at the same time) — build it once (e.g. in a constructor) and reuse it.
3. Per call: choose the method (`get()`, `post()`, `put()`, `patch()`, `delete()`, `method(...)`), the `uri(...)`, headers, body.
4. Finish with **`retrieve()`** (then `body(...)`, `toEntity(...)`, `toBodilessEntity()`) or **`exchange(...)`**.

Example: body, entity and generic lists

```
RestClient rest = builder.baseUrl(baseUrl()).build();

BookDto book = rest.get().uri("/catalog/books/{id}", 1).retrieve().body(BookDto.class);

ResponseEntity<List<BookDto>> entity = rest.get()
        .uri(b -> b.path("/catalog/books").queryParam("author", "Martin Fowler").build())
        .retrieve()
        .toEntity(new ParameterizedTypeReference<List<BookDto>>() {});
```

Output:

```
body(BookDto.class) -> BookDto[id=1, title=Release It!, author=Michael T. Nygard]
toEntity(List) -> 200 OK 2 books
```

What each part does:

- `uri("/catalog/books/{id}", 1)` — a **URI template**; variables are URL-encoded (special characters like spaces turned into `%20`-style codes) for you. Never build URLs by string concatenation with user input.

- `uri(b -> b.path(...).queryParam("author", "Martin Fowler").build())` — the `UriBuilder` form for query parameters (the space is encoded).
- `body(BookDto.class)` — deserialises (turns the JSON text back into an object) the JSON body into your record. Your DTO (Data Transfer Object: a plain class/record that just carries data) only needs the fields you use; unknown fields are ignored (Jackson 3 default).
- `ParameterizedTypeReference<List<BookDto>>` — Java erases generics (type parameters such as `<BookDto>`) at runtime, so `List.class` would give you a list of maps; the anonymous subclass (the `{}` at the end creates a one-off subclass) captures the full type.
- `toEntity(...)` returns `ResponseEntity<T>`: status + headers + body. `toBodilessEntity()` when you only care about the status.
- `requiredBody(...)` (Framework 7.0.4+) is like `body(...)` but throws if the body is empty instead of returning `null`.

A hand-written client class

Wrap the calls in a small class so the rest of your code talks in domain terms (the language of the business — books, orders — rather than HTTP) (`CatalogClient.java`):

```
@Component
public class CatalogClient {

    private final RestClient rest;

    public CatalogClient(RestClient.Builder builder, @Value("${catalog.base-url}") String baseUrl) {
        this.rest = builder                                    // Boot's pre-configured, prototype builder
                .baseUrl(baseUrl)
                .defaultHeader("X-Client", "catalog-client")
                .defaultStatusHandler(status -> status.is5xxServerError(), (request, response) -> {
                    throw new CatalogUnavailableException(
                            "catalog answered " + response.getStatusCode().value() + " for " + request.getURI().getPath());
                })
                .build();
    }

    /** 404 becomes Optional.empty(); 5xx becomes CatalogUnavailableException (default status handler). */
    public Optional<BookDto> findBook(long id) {
        try {
            return Optional.ofNullable(rest.get()
                    .uri("/catalog/books/{id}", id)           // URI template: id is encoded for us
                    .retrieve()                               // status handlers apply to retrieve()
                    .body(BookDto.class));
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }

    /** exchange(): full control over the raw response - status handlers are NOT applied. */
    public String describe(long id) {
        return rest.get()
                .uri("/catalog/books/{id}", id)
                .exchange((request, response) -> {
                    if (response.getStatusCode().isError()) {
                        return "status " + response.getStatusCode().value();
                    }
                    BookDto b = response.bodyTo(BookDto.class);
                    return b.title() + " by " + b.author();
                });
    }

    public ResponseEntity<BookDto> create(String title, String author) {
        return rest.post()
                .uri("/catalog/books")
                .body(new BookDto(null, title, author))      // serialised to JSON by Jackson
                .retrieve()
                .toEntity(BookDto.class);                     // status + headers + body
    }
}
```

Output:

```
findBook(2)   -> Optional[BookDto[id=2, title=Refactoring, author=Martin Fowler]]
findBook(999) -> Optional.empty
describe(1)   -> Release It! by Michael T. Nygard
describe(999) -> status 404
create        -> 201 CREATED Location=/catalog/books/102 body=BookDto[id=102, title=Designing Data-Intensive Applications, author=Martin
Kleppmann]
```

`retrieve()` vs `exchange()`

	`retrieve()`	`exchange((req, res) -> ...)`
Error statuses	turned into exceptions (default or your status handlers)	**you** inspect the status; **status handlers are not applied**
Body handling	`body`, `toEntity`, `toBodilessEntity`	`res.bodyTo(...)`, `res.getHeaders()`, anything
Use when	almost always	different body types per status, streaming, very custom handling

A lesson from writing this chapter: the first version of `findBook` used `exchange()` and treated 404 specially, assuming the 5xx (HTTP status codes 500–599: server-side failures) `defaultStatusHandler` would still fire. It does not — `exchange()` bypasses status handlers — so a 503 silently became `Optional.empty()`. The `@RestClientTest` in section 6.8 caught it. If you use `exchange()`, handle **every** status yourself.

Self-check

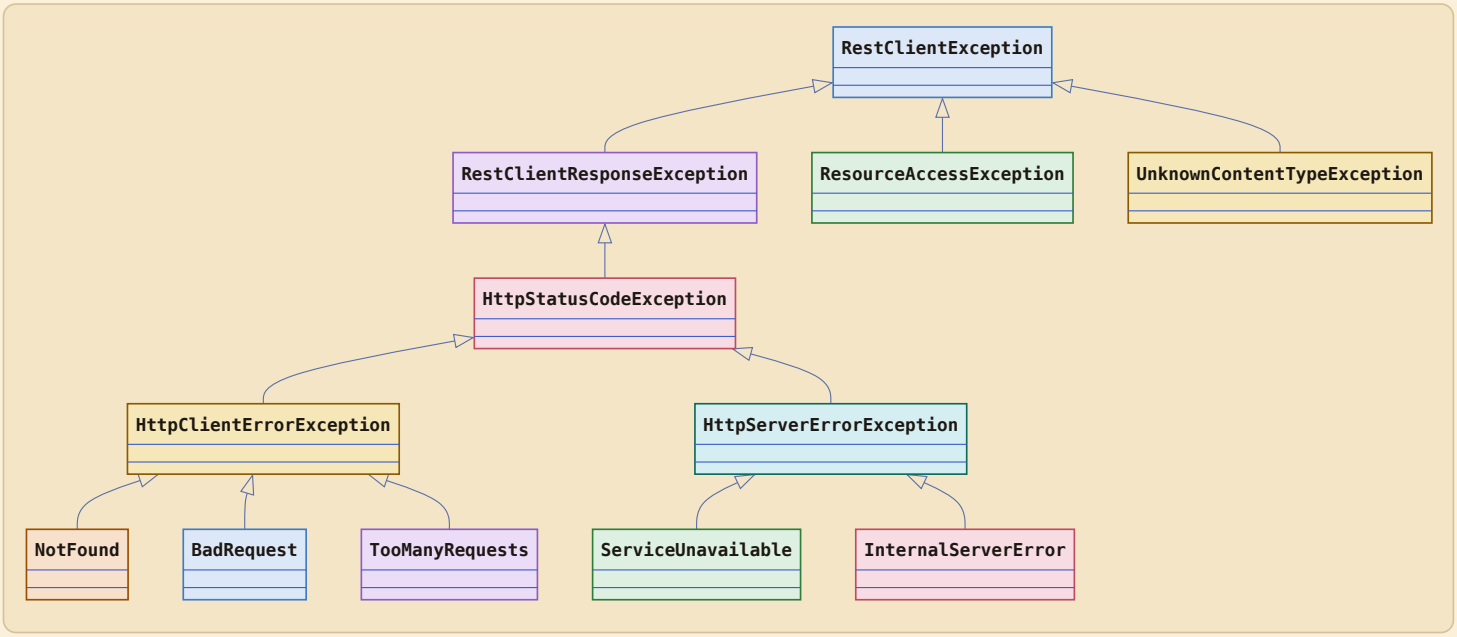
1. Why inject `RestClient.Builder` instead of calling `RestClient.builder()` in Boot? → Boot's builder carries the auto-configured converters, HTTP library, timeouts and observability.
2. Is a `RestClient` safe to share between threads? → Yes, it is immutable and thread-safe once built.
3. How do you deserialise a `List<BookDto>`? → `body(new ParameterizedTypeReference<List<BookDto>>() {})`.

More request features (quick reference)

Need	`RestClient` call
Header on one request	`.header("X-Tenant", "acme")`, `.headers(h -> h.setBearerAuth(token))`
Accept / content type	`.accept(MediaType.APPLICATION_JSON)`, `.contentType(MediaType.APPLICATION_JSON)`
Form post (`application/x-www-form-urlencoded`)	`.contentType(APPLICATION_FORM_URLENCODED).body(multiValueMap)`
Multipart upload	`.body(new MultipartBodyBuilder().part("file", resource).build())` with `contentType(MULTIPART_FORM_DATA)`
Download a large file	`.exchange((req, res) -> { res.getBody().transferTo(out); return null; })` — stream, don't buffer
Plain text	`.retrieve().body(String.class)`
Per-request attributes for interceptors/observations	`.attribute("name", value)`

6.3 Error handling

Spring's client exception hierarchy



Exception	Meaning	Retry?
`HttpClientErrorException` (4xx) with nested types `.NotFound`, `.BadRequest`, `.Conflict`, `.TooManyRequests`, ...	the **request** was wrong or not allowed	no (except 429 "too many requests" after waiting the `Retry-After` header's delay, and 408 "request timeout")
`HttpServerErrorException` (5xx) with `.InternalServerError`, `.BadGateway`, `.ServiceUnavailable`, `.GatewayTimeout`	the server failed	often yes (transient — likely to pass on its own), if the operation is idempotent (safe to repeat: doing it twice has the same effect as once — see System Design Fundamentals)
`ResourceAccessException`	I/O (input/output, here network) problem: connection refused, **timeout**, DNS (the system that turns host names into IP addresses)	often yes
`UnknownContentTypeException`	no converter for the response type	no — a bug or contract change (the other side changed its API)

All are **unchecked** (you are not forced to catch or declare them: `RestClientException extends RuntimeException`). `RestClientResponseException` gives you `getStatusCode()`, `getResponseHeaders()`, `getResponseBodyAsString()`, and **`getResponseBodyAs(Class)`** — which can decode an RFC 9457 error body directly:

```
assertThatThrownBy(() -> rest.get().uri("/catalog/books/{id}", 999).retrieve().body(BookDto.class))
        .isInstanceOf(HttpClientErrorException.NotFound.class)
        .satisfies(e -> {
            var ex = (HttpClientErrorException) e;
            ProblemDetail pd = ex.getResponseBodyAs(ProblemDetail.class);   // read the error body
            System.out.println("404 -> " + ex.getClass().getSimpleName()
                    + ", problem title='" + pd.getTitle() + "', detail='" + pd.getDetail() + "'");
        });
```

Output:

```
404 -> NotFound, problem title='Book not found', detail='No book 999'
```

Mapping to your own exceptions

Callers of `CatalogClient` should not need to know it uses HTTP. Translate at the edge:

- **Per request:** `.retrieve().onStatus(HttpStatusCode::is4xxClientError, (req, res) -> { throw new ... })`.
- **For every request of a client:** `builder.defaultStatusHandler(predicate, handler)` — used above to turn any 5xx into `CatalogUnavailableException`:

```
503 -> CatalogUnavailableException: catalog answered 503 for /catalog/flaky
```

- **Expected "not found":** return `Optional.empty()` (as `findBook` does) instead of throwing.

Common mistakes

- Catching `Exception` around a client call and returning `null` — hides outages; log and translate instead.
- Logging full response bodies of errors — may contain personal data or secrets.
- Treating every 4xx as retryable — a 400 will fail the same way every time.
- Forgetting that **no timeout = wait forever**-ish (next section).

Self-check

1. A call times out. Which exception? → `ResourceAccessException` (its cause is the library's timeout exception, e.g. `HttpTimeoutException` for the JDK client).
2. How do you read an RFC 9457 error body from a failed call? → `ex.getResponseBodyAs(ProblemDetail.class)` on the `RestClientResponseException`.
3. Where should HTTP exceptions be translated into domain exceptions? → At the client edge (status handlers or the client class), so business code never sees HTTP types.

6.4 Timeouts and the underlying HTTP library

What it is

`RestClient` does not open sockets (an operating-system network connection endpoint) itself; it delegates to a **`ClientHttpRequestFactory`** wrapping a real HTTP library. Two timeouts matter:

- **Connect timeout** — how long to wait for the TCP (the protocol that sets up a reliable connection) and TLS (the encryption layer behind `https`) connection to be established.
- **Read timeout** — how long to wait for response data once connected.

Why it matters: without timeouts, a slow downstream service (one that *you* call) makes your request threads pile up waiting, until your service also stops answering — a **cascading failure** (one failure spreading to the services that depend on it). Every remote call needs a timeout shorter than your own callers' timeout. See also Circuit breaker and Bulkhead.

Which library does Boot pick?

Boot 4 detects, in this order, the first one on the classpath:

Priority	Library	Factory	Add dependency
1	Apache HttpClient 5	`HttpComponentsClientHttpRequestFactory`	`org.apache.httpcomponents.client5:httpclient5`
2	Jetty HttpClient	`JettyClientHttpRequestFactory`	`org.eclipse.jetty:jetty-client`
3	Reactor Netty	`ReactorClientHttpRequestFactory`	`io.projectreactor.netty:reactor-netty-http`
4	JDK `java.net.http.HttpClient`	`JdkClientHttpRequestFactory`	nothing (built in)
5	`HttpURLConnection`	`SimpleClientHttpRequestFactory`	nothing

This project has none of the first three, so it uses the **JDK HttpClient**. Force a choice with `spring.http.clients.imperative.factory=jdk|http-components|jetty|reactor|simple`. (For `WebClient` the equivalent is `spring.http.clients.reactive.connector`.)

Configuring timeouts

Globally, for every auto-configured client (Boot 4 property names — the Boot 3.x `spring.http.client.*` names are deprecated):

```
spring.http.clients.connect-timeout=1s
spring.http.clients.read-timeout=3s
spring.http.clients.redirects=dont-follow     # or follow / follow-when-possible
```

Per client, in code, with Boot's `ClientHttpRequestFactoryBuilder` and `HttpClientSettings`:

```
var factory = ClientHttpRequestFactoryBuilder.jdk()
        .build(HttpClientSettings.defaults()
                .withConnectTimeout(Duration.ofSeconds(1))
                .withReadTimeout(Duration.ofMillis(300)));
RestClient rest = builder.baseUrl(baseUrl()).requestFactory(factory).build();
rest.get().uri("/catalog/slow").retrieve().body(String.class);   // server needs 1.5 s
```

Output:

```
slow endpoint -> ResourceAccessException caused by HttpTimeoutException after ~300 ms
```

For HTTP service groups (section 6.6) the timeouts can be set per group: `spring.http.serviceclient.<group>.read-timeout=2s`.

Choosing values: measure the downstream's normal latency (how long one call takes; p99 = the time that 99% of calls stay under) and set the read timeout a bit above it; keep the total (timeout × attempts + back-off — the waits between retries) below what *your* caller will wait. Connection **pooling** (keeping open connections and reusing them) (Apache/Jetty/Netty pool by default; the JDK client keeps connections alive) avoids paying TCP/TLS setup on every call.

Self-check

1. Connect vs read timeout? → Connect: establishing the connection. Read: waiting for response data after connecting.
2. Which HTTP library does Boot use if you add nothing? → The JDK `java.net.http.HttpClient`.
3. Boot 4 property for a global read timeout? → `spring.http.clients.read-timeout` (the Boot 3 name `spring.http.client.read-timeout` is deprecated).

6.5 Interceptors

A `ClientHttpRequestInterceptor` wraps every request a client sends — the client-side twin of a servlet filter. Uses: auth headers (tokens — strings proving who the caller is), correlation ids (one id carried through all calls of a request so logs can be joined), logging, metrics.

```
RestClient rest = builder.baseUrl(baseUrl())
        .requestInterceptor((request, body, execution) -> {
            request.getHeaders().add("X-Trace-Id", "trace-42");          // add a header
            var response = execution.execute(request, body);            // do the call
            log.add(request.getMethod() + " " + request.getURI().getPath() + " -> " + response.getStatusCode().value());
            return response;
        })
        .build();
```

Output:

```
interceptor: GET /catalog/books/1 -> 200
interceptor: GET /catalog/books/3 -> 200
```

- You rarely need interceptors for **tracing**: when Micrometer observability is set up (Actuator, Chapter 15), Boot instruments the auto-configured builder with an observation customizer that records `http.client.requests` metrics and propagates trace headers (passes the current trace id on to the next service). This only works if you build from the injected builder — `RestClient.create()` skips it.
- For OAuth2 bearer tokens (access tokens sent as `Authorization: Bearer ...`; OAuth2 is the standard protocol for issuing them), Spring Security provides `OAuth2ClientHttpRequestInterceptor` (Chapter 11).
- Don't read the response body in an interceptor unless you buffer it (`BufferingClientHttpRequestFactory`); a body can normally be read only once.
- To customise **all** builders Boot creates, declare a `RestClientCustomizer` bean.

6.6 HTTP service interfaces (`@HttpExchange`)

What it is

An **HTTP service interface** is a Java interface whose methods are annotated with the HTTP method, path and parameters. Spring generates a **proxy** that implements it by turning each call into an HTTP request (the same proxy idea as Chapter 4). It is Spring's built-in answer to Feign-style declarative clients (Feign/OpenFeign is a popular library that does the same from annotated interfaces) — no extra library. Analogy: a TV remote. You press "channel 5"; you do not care which infrared codes are sent.

```
@HttpExchange("/catalog")
public interface CatalogApi {

    @GetExchange("/books/{id}")
    BookDto get(@PathVariable long id);

    @GetExchange("/books/{id}")
    ResponseEntity<BookDto> getEntity(@PathVariable long id);   // status + headers too

    @GetExchange("/books")
    List<BookDto> byAuthor(@RequestParam String author);

    @PostExchange("/books")
    ResponseEntity<BookDto> create(@RequestBody BookDto book);

    @GetExchange(url = "/greeting", version = "2.0")   // Framework 7: API version per method
    String greetingV2();
}
```

Annotation	Purpose
`@HttpExchange` (type)	shared URL prefix, content type, accept, headers, version
`@GetExchange`, `@PostExchange`, `@PutExchange`, `@PatchExchange`, `@DeleteExchange`	method + path
`@PathVariable`, `@RequestParam`, `@RequestHeader`, `@RequestBody`, `@RequestPart`, `@CookieValue`	the same annotations as server controllers, now describing the request to send
`URI` parameter	override the full URL dynamically

Return types: a body type (`BookDto`, `List<BookDto>`), `ResponseEntity<T>`, `ResponseEntity<Void>`, `HttpHeaders`, `void`; on `WebClient`, also `Mono<T>`/`Flux<T>`. Errors surface as the same `HttpClientErrorException`/`HttpServerErrorException` as `RestClient`.

Boot 4 auto-configuration: `@ImportHttpServices`

Before Framework 7 / Boot 4 you created each proxy yourself. Now you declare **groups** of interfaces; each group gets its own `RestClient` configured from properties:

```
@SpringBootApplication
@EnableResilientMethods                                  // for @Retryable (Framework 7)
@ImportHttpServices(group = "catalog", types = CatalogApi.class)   // creates a CatalogApi bean
public class ClientsApplication { ... }
```

```
# Boot 4: HTTP service client group "catalog" (the @HttpExchange interface)
spring.http.serviceclient.catalog.base-url=http://localhost:8080
spring.http.serviceclient.catalog.read-timeout=2s
spring.http.serviceclient.catalog.default-header.X-Client=handbook-ch06
# the group's RestClient sends the API version (from @GetExchange(version=...)) in this header
spring.http.serviceclient.catalog.apiversion.insert.header=X-API-Version
```

- `types = ...` lists interfaces; `basePackages`/`basePackageClasses` scans a package instead. Repeat the annotation for several groups (one per remote service).
- `clientType = WEB_CLIENT` makes a group use `WebClient`.
- For configuration beyond properties, declare a `RestClientHttpServiceGroupConfigurer` bean: `groups -> groups.filterByName("catalog").forEachClient((group, builder) -> builder.requestInterceptor(...))`.

Then inject and call it like any bean (`HttpInterfaceTest.java`):

```
@Autowired CatalogApi catalogApi;     // generated by @ImportHttpServices + Boot auto-config

BookDto one = catalogApi.get(1);
ResponseEntity<BookDto> e = catalogApi.getEntity(2);
ResponseEntity<BookDto> created = catalogApi.create(new BookDto(null, "Accelerate", "Nicole Forsgren"));
String v2 = catalogApi.greetingV2();
```

Output:

```
bean class     -> $ProxyN
get(1)         -> BookDto[id=1, title=Release It!, author=Michael T. Nygard]
byAuthor(...)  -> [BookDto[id=2, title=Refactoring, author=Martin Fowler], BookDto[id=3, title=Patterns of Enterprise Application
Architecture, author=Martin Fowler]]
getEntity(2)   -> 200 OK server saw X-Client=handbook-ch06
create(...)    -> 201 CREATED BookDto[id=101, title=Accelerate, author=Nicole Forsgren]
greetingV2()   -> v2: hello
get(999)       -> NotFound
```

The bean is a JDK dynamic proxy (`$Proxy...`). `X-Client=handbook-ch06` shows the per-group `default-header` property reached the server, and `greetingV2()` shows the method's `version = "2.0"` was sent in the `X-API-Version` header configured for the group.

What Boot does for you, by hand

```
RestClient restClient = RestClient.builder().baseUrl(baseUrl()).build();
HttpServiceProxyFactory factory = HttpServiceProxyFactory
        .builderFor(RestClientAdapter.create(restClient)).build();
CatalogApi manual = factory.createClient(CatalogApi.class);
```

```
manual proxy get(3) -> Patterns of Enterprise Application Architecture
```

Useful in plain Spring, in tests, or in libraries.

Trade-offs

	HTTP interface	Hand-written `RestClient` class	OpenFeign (Spring Cloud)
Boilerplate	minimal	more	minimal
Custom per-call logic (fallbacks, `Optional` on 404)	needs a wrapper or default methods (interface methods with a body)	natural	via fallbacks (alternative answers returned when the call fails)
Dependencies	none beyond Spring	none	Spring Cloud OpenFeign (in maintenance mode — bug fixes only, no new features; Spring recommends HTTP interfaces)
Config per remote service	`spring.http.serviceclient.<group>.*` (Boot 4)	your own properties	`spring.cloud.openfeign.client.config.*`
Can share the interface with the server?	technically, but couples client and server release cycles (neither can ship a change without the other)	—	same caveat

Common approach: an HTTP interface for the raw calls + a thin service class on top for domain translation (`Optional`, own exceptions, retries).

Self-check

1. What does `@ImportHttpServices(group = "catalog", types = CatalogApi.class)` do? → Registers a `CatalogApi` proxy bean backed by a `RestClient` for group `catalog`, configured from `spring.http.serviceclient.catalog.*`.
2. Which class creates the proxy under the hood? → `HttpServiceProxyFactory` with a `RestClientAdapter` (or `WebClientAdapter`).
3. How would you add an interceptor to only one group? → A `RestClientHttpServiceGroupConfigurer` bean using `groups.filterByName("catalog").forEachClient(...)`.

6.7 Client-side API versioning and retries

API versioning (Framework 7)

When the server versions its API (Chapter 5, section 5.9), the client must send the version. Framework 7 adds this to every client: an `ApiVersionInserter` decides *how* (header, query param, path segment, media-type parameter), `defaultApiVersion` sets the usual version, and each request can override it with `.apiVersion(...)`:

```
RestClient rest = builder.baseUrl(baseUrl())
        .apiVersionInserter(ApiVersionInserter.useHeader("X-API-Version"))  // how to send it
        .defaultApiVersion("1.0")                                           // unless overridden
        .build();
String v1 = rest.get().uri("/catalog/greeting").retrieve().body(String.class);
String v2 = rest.get().uri("/catalog/greeting").apiVersion("2.0").retrieve().body(String.class);
```

Output:

```
default version -> v1: hello
apiVersion(2.0) -> v2: hello
```

HTTP interfaces put the version on the method (`@GetExchange(version = "2.0")`) and configure the inserter per group (`spring.http.serviceclient.<group>.apiversion.insert.header=...`), as shown in 6.6. Server-side version properties (`spring.mvc.apiversion.*`) are **not** applied to clients; configure them separately.

Retries

What it is

A **retry** repeats a failed call hoping the failure was transient. Framework 7's `@Retryable` (Chapter 4, section 4.7) makes this one annotation:

```
@Service
public class StockService {
    private final RestClient rest;

    public StockService(RestClient.Builder builder, @Value("${catalog.base-url}") String baseUrl) {
        this.rest = builder.baseUrl(baseUrl).build();
    }

    @Retryable(includes = org.springframework.web.client.HttpServerErrorException.class,
               maxRetries = 3, delay = 20, multiplier = 2, jitter = 5)
    public String checkStock() {
        return rest.get().uri("/catalog/flaky").retrieve().body(String.class);
    }
}
```

The `/catalog/flaky` endpoint answers 503 twice, then 200:

```
checkStock() -> ok on call 3
```

Rules for safe retries

1. **Only transient failures:** 5xx (especially 502/503/504), timeouts/`ResourceAccessException`, 429 (respect `Retry-After`). Never 400/401/403/404/409/422.
2. **Only idempotent operations** — GET, PUT, DELETE — or POSTs protected by an **idempotency key** header that the server uses to de-duplicate (the way payment APIs such as Stripe do).
3. **Exponential back-off + jitter** (waits that grow each time, plus a small random amount) so thousands of clients don't hammer a recovering service in sync.
4. **Bound the total time**: attempts × read timeout + delays must fit your own deadline (`@Retryable(timeout = ...)` since 7.0.2 caps the total).
5. **Retry in one layer only**, and combine with a **circuit breaker** (a switch that stops calling a failing service for a while; Resilience4j is a common library for it) so a dead dependency is not retried forever — see Chapter 16 and Retry pattern.
6. `@Retryable` is proxy-based: it must be on a Spring bean method called from **another** bean.

Self-check

1. Should you retry a `POST /payments` after a read timeout? → Only if the request carries an idempotency key the server honours; otherwise you may charge twice.
2. With `maxRetries = 3`, how many HTTP calls at most? → 4.
3. Does the server's `spring.mvc.apiversion.use.header` configure your clients? → No; configure `apiVersionInserter` or the service-group `apiversion.*` properties.

6.8 Testing clients

Option 1 — `@RestClientTest` + `MockRestServiceServer` (fast, no network)

`@RestClientTest` is a **test slice** (a test that starts only a small part of the application): it starts only the listed components, Jackson and the `RestClient.Builder`/`RestTemplateBuilder` auto-configuration, and binds a `MockRestServiceServer` to the builder. Requests never leave the JVM; the mock checks each request against expectations and returns canned responses (fixed answers prepared in the test). Boot 4 dependency: `spring-boot-starter-restclient-test`.

```
@RestClientTest(components = CatalogClient.class, properties = "catalog.base-url=http://catalog.test")
class CatalogClientMockServerTest {

    @Autowired CatalogClient client;
    @Autowired MockRestServiceServer server;   // intercepts requests; no network at all

    @Test
    void foundNotFoundAndUnavailable() {
        server.expect(requestTo("http://catalog.test/catalog/books/1"))
              .andExpect(method(HttpMethod.GET))
              .andExpect(header("X-Client", "catalog-client"))
              .andRespond(withSuccess("""
                      {"id":1,"title":"Release It!","author":"Michael T. Nygard","pages":376}""",
                      MediaType.APPLICATION_JSON));
        server.expect(requestTo("http://catalog.test/catalog/books/2"))
              .andRespond(withStatus(HttpStatus.NOT_FOUND));
        server.expect(requestTo("http://catalog.test/catalog/books/3"))
              .andRespond(withStatus(HttpStatus.SERVICE_UNAVAILABLE));

        System.out.println("mock 200 -> " + client.findBook(1));
        System.out.println("mock 404 -> " + client.findBook(2));
        assertThatThrownBy(() -> client.findBook(3))
                .isInstanceOf(CatalogUnavailableException.class)
                .satisfies(e -> System.out.println("mock 503 -> " + e.getMessage()));
        server.verify();                        // every expected request really happened
    }
}
```

Output:

```
mock 200 -> Optional[BookDto[id=1, title=Release It!, author=Michael T. Nygard]]
mock 404 -> Optional.empty
mock 503 -> catalog answered 503 for /catalog/books/3
```

The unknown `pages` field was ignored. By default expectations must occur **in order**; `MockRestServiceServer.bindTo(builder).ignoreExpectOrder(true)` relaxes that. `withException(new SocketTimeoutException())` simulates I/O failures.

Option 2 — a real HTTP server

Mocks can't catch problems in the real HTTP layer (timeouts, headers, encoding, the version header). This chapter's other tests start the app on a real port with a local "remote" controller:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
abstract class LiveServerTest {

    static final int PORT = freePort();

    @DynamicPropertySource
    static void urls(DynamicPropertyRegistry registry) {
        String base = "http://localhost:" + PORT;
        registry.add("server.port", () -> PORT);
        registry.add("catalog.base-url", () -> base);
        registry.add("spring.http.serviceclient.catalog.base-url", () -> base);
    }
    ...
}
```

The port is picked **before** the context starts so the client base URLs can be set as properties (with `RANDOM_PORT` the port is only known after the clients were created). In real projects the "remote" side is usually a **WireMock** server (a standalone HTTP stub server, i.e. a fake server returning prepared responses; often run via Testcontainers, a library that starts Docker containers for tests) or a Spring Cloud Contract stub (a fake generated from an agreed API contract) — see Chapter 12.

For testing **your own** endpoints over HTTP, Boot 4 offers `RestTestClient` (`@AutoConfigureRestTestClient`), the `RestClient`-based successor of `TestRestTemplate` (which now needs `@AutoConfigureTestRestTemplate` and the `spring-boot-resttestclient` module).

Self-check

1. What does `server.verify()` check? → That every expected request was actually made.
2. Why also test against a real (stub) server? → Mocks skip the real HTTP layer: timeouts, encoding, headers, connection handling.

3. Which Boot 4 starter provides `@RestClientTest`? → `spring-boot-starter-restclient-test`.

6.9 `RestTemplate` and `WebClient` in brief

`RestTemplate` (legacy)

```
RestTemplate template = restTemplateBuilder.rootUri(baseUrl()).build();
BookDto viaTemplate = template.getForObject("/catalog/books/{id}", BookDto.class, 2);

RestClient wrapped = RestClient.create(template);     // migration step: reuse its config
BookDto viaWrapped = wrapped.get().uri("/catalog/books/{id}", 2).retrieve().body(BookDto.class);
```

```
RestTemplate  -> BookDto[id=2, title=Refactoring, author=Martin Fowler]
RestClient.create(template) -> BookDto[id=2, title=Refactoring, author=Martin Fowler]
```

Boot auto-configures a `RestTemplateBuilder` (not a `RestTemplate`). Migration map:

`RestTemplate`	`RestClient`
`getForObject(url, T.class, vars)`	`get().uri(url, vars).retrieve().body(T.class)`
`getForEntity(...)`	`get().uri(...).retrieve().toEntity(T.class)`
`postForEntity(url, body, T.class)`	`post().uri(url).body(body).retrieve().toEntity(T.class)`
`exchange(url, method, entity, T.class)`	`method(m).uri(url).headers(...).body(...).retrieve().toEntity(T.class)`
`setErrorHandler(...)`	`defaultStatusHandler(...)`
interceptors / request factory	reused as-is (`requestInterceptor`, `requestFactory`)

Steps: (1) wrap existing templates with `RestClient.create(restTemplate)` or `RestClient.builder(restTemplate)` so configuration is reused; (2) rewrite call sites; (3) move shared config to a `RestClientCustomizer` / builder; (4) remove the templates.

`WebClient` (reactive) — not run here

```
WebClient web = webClientBuilder.baseUrl("https://catalog.example.com").build();
Mono<BookDto> book = web.get().uri("/catalog/books/{id}", 1)
        .retrieve()
        .bodyToMono(BookDto.class)
        .timeout(Duration.ofSeconds(2));
```

Same shape as `RestClient`, but returns `Mono` / `Flux` and never blocks a thread. Use it in WebFlux apps or for streaming; don't call `.block()` inside a reactive pipeline. Full coverage in Chapter 13.

6.10 Common mistakes, real-world examples, interview

Common mistakes

Mistake	Consequence	Fix
New `RestClient` per request	loses connection pooling, slower, more sockets	build once, reuse
`RestClient.create()` in a Boot app	no Boot converters/timeouts/observability	inject `RestClient.Builder`
No timeouts	thread pile-up, cascading failure	`spring.http.clients.*-timeout` or per-group/per-client settings
String-concatenated URLs	broken encoding, injection (attacker-controlled text changing the meaning of the URL)	URI templates / `UriBuilder`
Using `exchange()` and expecting status handlers	errors silently treated as success	handle statuses yourself or use `retrieve()`
Retrying POSTs / 4xx	duplicate orders, wasted calls	retry only idempotent + transient
Leaking `HttpClientErrorException` into business code	HTTP details spread everywhere	translate at the client edge
Mock-only tests	real HTTP issues missed	add a test against a real/stub server

Real-world examples

- **Payment integration**: a `PaymentsApi` HTTP interface for the provider, read timeout ~5 s, retries only for GETs and for POSTs carrying an `Idempotency-Key`, errors mapped to `PaymentDeclined` vs `PaymentProviderUnavailable`.

- **Microservice calls** inside a company: one `@ImportHttpServices` group per downstream service (`inventory`, `pricing`, `shipping`) each with its own base URL and timeouts from config, tracing headers propagated automatically.
- **Third-party APIs with versions** (e.g. APIs that version by header or date): a default API version on the client, bumped deliberately after testing.

In the interview

Q: Which HTTP client would you use in a new Spring Boot 4 MVC service, and why? 30-second answer: "`RestClient` — or an `@HttpExchange` interface backed by it. It's the modern synchronous client: fluent, uses Boot's auto-configured converters, HTTP library, timeouts and observability. `RestTemplate` is being deprecated (formally in 7.1, removal in 8.0), and `WebClient` is for reactive stacks. With virtual threads, blocking calls are cheap enough that I don't need reactive just for concurrency."

- *How do HTTP interfaces get implemented?* → `HttpServiceProxyFactory` creates a JDK dynamic proxy; in Boot 4 `@ImportHttpServices` registers groups and `spring.http.serviceclient.<group>.*` configures each group's client.
- *How do you handle errors?* → `retrieve()` throws `HttpClientErrorException`/`HttpServerErrorException` (with nested per-status types) and `ResourceAccessException` for I/O/timeouts; I map them to domain exceptions with `onStatus`/`defaultStatusHandler` and read `ProblemDetail` bodies with `getResponseBodyAs`.
- *What timeouts do you set?* → Connect and read timeouts on every client (`spring.http.clients.connect-timeout/read-timeout` or per group), sized from downstream p99 and my own SLA (Service Level Agreement: the response time/availability promised to callers).

Q: How do you make remote calls resilient? → Timeouts, retries with exponential back-off + jitter only for idempotent/transient failures (`@Retryable` in Framework 7), circuit breaker + bulkhead (Resilience4j / `@ConcurrencyLimit`), fallbacks where acceptable, and idempotency keys for writes.

Q: How do you test a REST client? → `@RestClientTest` + `MockRestServiceServer` for fast request/response contract checks; plus an integration test against a real stub server (WireMock) or the provider's contract stubs for the HTTP layer.

6.11 Chapter TL;DR

- **Clients:** `RestClient` (sync, recommended) · HTTP interfaces `@HttpExchange` (declarative, on `RestClient` or `WebClient`) · `WebClient` (reactive) · `RestTemplate` (legacy; deprecated from 7.1, removed in 8.0).
- **Boot 4:** add `spring-boot-starter-restclient`; inject the **prototype** `RestClient.Builder`; build once, reuse (thread-safe).
- **Calls:** `get().uri(template, vars).retrieve().body(T)` / `toEntity(T)` / `toBodilessEntity()`; `ParameterizedTypeReference` for generics; `exchange()` for raw control (**status handlers not applied**).
- **Errors:** 4xx → `HttpClientErrorException.*`, 5xx → `HttpServerErrorException.*`, I/O/timeouts → `ResourceAccessException`; `getResponseBodyAs(ProblemDetail.class)`; translate with `onStatus`/`defaultStatusHandler`.
- **Timeouts:** `spring.http.clients.connect-timeout`/`read-timeout` (Boot 4 names), per group `spring.http.serviceclient.<g>.read-timeout`, or `ClientHttpRequestFactoryBuilder` + `HttpClientSettings`. Library order: Apache → Jetty → Reactor Netty → JDK → simple.
- **HTTP interfaces in Boot 4:** `@ImportHttpServices(group = "...", types = ...)` + `spring.http.serviceclient.<group>.base-url/default-header/apiversion.*`; customise with `RestClientHttpServiceGroupConfigurer`.
- **Versioning (Framework 7):** `ApiVersionInserter.useHeader(...)` + `defaultApiVersion(...)`, per-request `.apiVersion(...)`, `@GetExchange(version = ...)`.
- **Retries:** `@Retryable` for transient + idempotent calls only, with back-off, jitter and a total budget; add a circuit breaker.
- **Tests:** `@RestClientTest` + `MockRestServiceServer`; real/stub server tests for the HTTP layer; `RestTestClient` for your own endpoints.
- Next: Chapter 7 — Data access with JDBC and transactions.

Chapter 7 — Data Access with JDBC and Transactions

Almost every Spring application stores data in a relational database (a database that keeps data in tables of rows and columns and is queried with SQL, e.g. PostgreSQL). This chapter covers the plumbing under all of it. You will see how Spring Boot creates a **connection pool**, how to run SQL with `JdbcClient` and `JdbcTemplate`, and how Spring turns database errors into one family of exceptions. Most of the chapter is about **transactions** (a group of database changes that are saved together or not at all): what `@Transactional` really does, the seven propagation types (what happens when one transactional method calls another), rollback rules (when changes are undone), isolation (how much concurrent transactions see of each other), read-only and timeouts, and the pitfalls that make `@Transactional` silently do nothing.

This is not a SQL course. What a transaction is at the database level (ACID — Atomicity, Consistency, Isolation, Durability, the four guarantees of a transaction; isolation anomalies — wrong results caused by concurrent transactions; MVCC — multi-version concurrency control, where readers see a snapshot instead of waiting for writers; locks — marks that stop others from changing a row you are using) is taught in the SQL book: SQL ch13 — Transactions and concurrency. Here we look at the Spring side: who opens the transaction, on which connection, and when it commits (makes its changes permanent).

What you'll learn

- How `DataSource`, HikariCP, the JDBC driver (the database vendor's library that speaks its network protocol) and Spring's helpers fit together
- `JdbcClient` (the modern fluent API) and `JdbcTemplate` (the classic one): queries, parameters, generated keys (ids the database creates on insert), batches (many statements sent together)
- Schema (the table definitions)/data initialization with `schema.sql` / `data.sql`
- Spring's `DataAccessException` hierarchy (exception translation)
- `@Transactional`: every attribute, every propagation type, rollback rules, isolation, `readOnly`, `timeout`
- The pitfalls: self-invocation, `final`/`private` methods, the wrong layer, checked exceptions, catching an inner failure
- Programmatic transactions with `TransactionTemplate`
- `@TransactionalEventListener`: run code only after a commit

The code (Spring Boot 4.1.1, Spring Framework 7.0, Java 25):

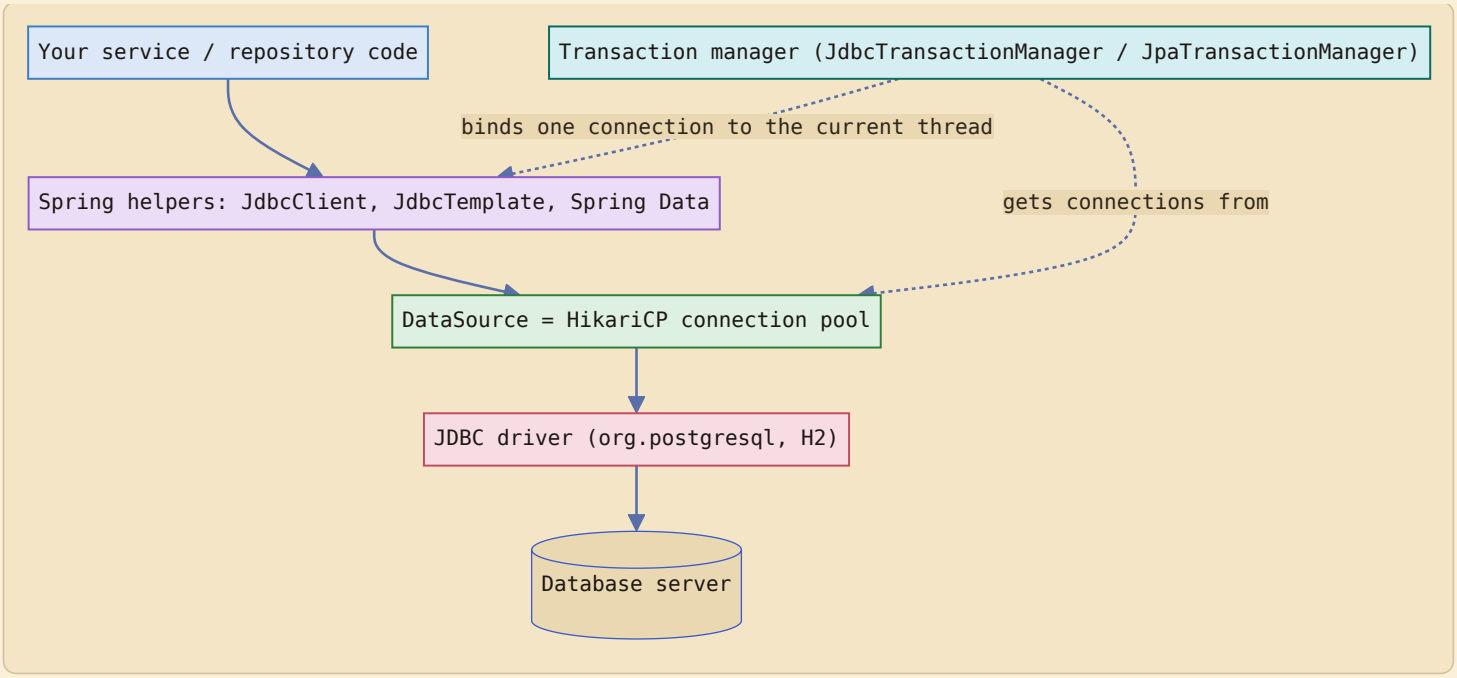
Project	Database	What it proves
`code/jdbc-basics`	H2 in memory	HikariCP settings, `JdbcClient`/`JdbcTemplate`, batch, exception translation, `@JdbcTest`
`code/tx-demos`	the book's private PostgreSQL 14 (port 5544)	propagation, rollback rules, pitfalls, isolation, read-only, timeout, events, `TransactionTemplate`

The transaction demos use real PostgreSQL. H2 (a small Java database that can run in memory) would hide some behaviour, such as how read-only transactions and serialization failures (the database refusing a transaction because a concurrent one changed the same data) act. The `tx-demos` test run creates a database called `spring_ch07_tx` before the tests and drops it afterwards (see 7.7). Every result below was printed by a test and checked by an assertion. Run everything with `bash tools_run_spring.sh spring '07-*'` from the repository root. Start PostgreSQL first with `bash tools_pg.sh start`.

7.1 The data-access stack

What it is

When your code runs `SELECT`, the call passes through several layers. Each layer has one job:



Layer	Job	Spring Boot gives you
JDBC (Java Database Connectivity)	The standard Java API for SQL: `Connection`, `PreparedStatement`, `ResultSet`	the driver from your dependencies
DataSource	A factory for connections. In practice it is a **pool** that keeps connections open and lends them out	`HikariDataSource`, set up from `spring.datasource.*`
Spring JDBC helpers	Take away the boilerplate (repetitive setup code): open/close, loops over `ResultSet`, turning `SQLException` into something useful	`JdbcTemplate`, `NamedParameterJdbcTemplate`, `JdbcClient` beans
Transaction manager	Starts, commits and rolls back transactions. Keeps the current connection on the current thread (a thread is one independent path of execution; each request usually runs on its own thread) so every helper call in one transaction uses the same connection	`JdbcTransactionManager` (JDBC) or `JpaTransactionManager` (JPA, ch08)

Plain JDBC (without Spring) is covered in the SQL book: SQL 14.11 — Using SQL from Java (JDBC). Compared with that code, Spring adds three things: a pool, much less boilerplate, and declarative transactions (you mark a method with an annotation instead of writing begin/commit code).

Choosing an API

Need	Use
Hand-written SQL, full control, reporting queries	`JdbcClient` (or `JdbcTemplate`) — this chapter
Simple aggregates (a root object plus the objects it owns, saved as one unit), no ORM (Object-Relational Mapper: a library that maps objects to tables and writes SQL for you), SQL you can predict	Spring Data JDBC — ch09
Rich domain model (classes that model the business) with relationships, change tracking (the library notices which objects you modified)	JPA / Hibernate + Spring Data JPA — ch08

You can mix them in one application. A `JdbcClient` query and a JPA repository call can even share one transaction when the `JpaTransactionManager` is in charge.

7.2 DataSource and HikariCP

What it is

A **connection pool** keeps a set of open database connections and lends one to each piece of work. Think of a car-rental desk: opening a new connection takes a TCP handshake (the network round trips that set up a connection), authentication and session setup, which is like building a car for each customer. The pool keeps a fleet of cars ready to hand out. **HikariCP** is the pool Spring Boot picks first whenever it is on the classpath (the set of jars the app runs with). `spring-boot-starter-jdbc` and `spring-boot-starter-data-jpa` both bring it in.

How Boot builds it

1. You add a starter plus a driver (`org.postgresql:postgresql`, `com.h2database:h2`).
2. Boot's `DataSourceAutoConfiguration` reads `spring.datasource.url`, `username` and `password`.
3. If you give **no URL** and an embedded database (one that runs inside your Java process: H2, HSQLDB, Derby) is on the classpath, Boot creates an in-memory database. Its name is unique per application context (Spring container instance) (`spring.datasource.generate-unique-name=true` by default).
4. Pool settings go under `spring.datasource.hikari.*`.

```
# jdbc-basics/src/main/resources/application.properties
spring.datasource.url=jdbc:h2:mem:shop;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.hikari.maximum-pool-size=5
spring.datasource.hikari.pool-name=shop-pool
spring.datasource.hikari.connection-timeout=2000
```

The test reads the settings back from the real bean:

```
HikariDataSource hikari = (HikariDataSource) dataSource;
System.out.println("DataSource class = " + hikari.getClass().getSimpleName()
        + ", pool = " + hikari.getPoolName()
        + ", maxPoolSize = " + hikari.getMaximumPoolSize()
        + ", connectionTimeout = " + hikari.getConnectionTimeout() + " ms");
```

Output:

```
DataSource class = HikariDataSource, pool = shop-pool, maxPoolSize = 5, connectionTimeout = 2000 ms
```

The settings that matter

Property (`spring.datasource.hikari.`)	HikariCP default	Meaning
`maximum-pool-size`	10	Most connections the pool will ever open
`minimum-idle`	same as max	Idle (open but unused) connections kept ready. Leaving it unset gives a fixed-size pool, which HikariCP recommends
`connection-timeout`	30 000 ms	How long a thread waits for a free connection before it gets an exception
`idle-timeout`	600 000 ms	Idle connections above `minimum-idle` are closed after this
`max-lifetime`	1 800 000 ms	Connections are retired after this. Keep it shorter than any database or firewall (network device that filters and may cut connections) idle limit
`keepalive-time`	120 000 ms	Pings idle connections so network gear does not drop them
`leak-detection-threshold`	0 (off)	Logs a stack trace (the chain of method calls at that moment) when a connection is held longer than this. Good for finding leaks (connections borrowed and never returned) in staging (a production-like test environment)

Boot 4.1 added `spring.datasource.connection-fetch=lazy`. It wraps the pool in Spring's `LazyConnectionDataSourceProxy`, so a transaction takes a physical connection only when the first SQL statement runs. A `@Transactional` method that returns early without touching the database never borrows a connection.

Pool sizing — the counter-intuitive part

A bigger pool is not faster. The database has a fixed number of CPU cores (units that can each run one task at a time) and disks. Hundreds of connections all running at once make it switch between them and fight over locks. HikariCP's own guidance starts from a small pool, roughly *cores × 2 + effective disks*, and then measures. Two rules follow from this:

- **Sum across instances.** 20 application pods (running copies of the app, in Kubernetes terms) × pool 10 = 200 connections, and the default PostgreSQL `max_connections` is 100.
- **Hold connections briefly.** A transaction that calls a slow HTTP API keeps its connection the whole time (see 7.13).

More on pooling in general: SQL 14.12 — Connection pooling.

Common mistakes

- Setting `maximum-pool-size=100` "to be safe", then running out of database connections as soon as you scale out (run more copies of the app).
- Leaving `connection-timeout` at 30 s in a user-facing API. Requests hang for 30 s when the pool is exhausted, when failing fast would be better.
- Forgetting the pool when you add virtual threads (cheap JVM-managed threads, so you can have thousands) (ch15). Thousands of virtual threads still share, say, 10 connections, and the pool becomes the limit.

Self-check

1. Why does Boot give each test context its own H2 database? → `generate-unique-name=true`, so contexts never see each other's tables.
2. What happens when all 5 connections are busy and a sixth thread asks? → It waits up to `connection-timeout` (2 s here), then gets a `SQLTransientConnectionException`, which Spring wraps as `CannotGetJdbcConnectionException`.
3. Why is `minimum-idle` best left unset? → The pool stays at a fixed size, so it never has to open connections during a traffic spike.

7.3 JdbcClient and JdbcTemplate

What it is

Both run SQL you write yourself. They open and close the connection, bind parameters, loop over rows, map each row to an object, and translate errors.

- **`JdbcTemplate`** (since Spring 1.x): the classic API with many overloaded methods (same name, different parameters), `?` placeholders (markers in the SQL that are filled with values safely), and a `RowMapper` (a function that turns one result row into an object) for mapping.
- **`NamedParameterJdbcTemplate`**: the same, with `:name` placeholders.
- **`JdbcClient`** (Spring Framework 6.1+): one fluent API that covers both styles. Boot auto-configures it from the `NamedParameterJdbcTemplate`. It is the recommended default for new code.

Querying with JdbcClient

```
@Repository
public class ProductRepository {

    private final JdbcClient jdbc;

    public ProductRepository(JdbcClient jdbc) { this.jdbc = jdbc; }

    public List<Product> findAll() {
        return jdbc.sql("SELECT id, sku, name, price, in_stock FROM product ORDER BY id")
                   .query(Product.class)          // columns -> record components by name
                   .list();
    }

    public Optional<Product> findBySku(String sku) {
        return jdbc.sql("SELECT id, sku, name, price, in_stock FROM product WHERE sku = :sku")
                   .param("sku", sku)             // named parameter
                   .query(Product.class)
                   .optional();                   // 0 rows -> empty, 2+ rows -> exception
    }

    public List<String> namesCheaperThan(BigDecimal max) {
        return jdbc.sql("SELECT name FROM product WHERE price < ? ORDER BY price")
                   .param(max)                     // positional parameter
                   .query(String.class)            // one column -> simple type
                   .list();
    }
    // ...
}
```

`Product` is a record: `record Product(Long id, String sku, String name, BigDecimal price, int inStock)`. `query(Product.class)` maps columns to constructor parameters by name. It understands `in_stock` → `inStock` because it ignores underscores and case. For a single column, `query(String.class)` (or `Long.class`, ...) returns plain values.

Output:

```
row: Product[id=1, sku=KB-01, name=Mechanical keyboard, price=89.00, inStock=12]
row: Product[id=2, sku=MS-01, name=Wireless mouse, price=25.50, inStock=40]
row: Product[id=3, sku=MN-27, name=27-inch monitor, price=249.99, inStock=3]
findBySku(MS-01) = Optional[Product[id=2, sku=MS-01, name=Wireless mouse, price=25.50, inStock=40]]
findBySku(NOPE)  = Optional.empty
cheaper than 100 = [Wireless mouse, Mechanical keyboard]
```

The terminal methods after `query(...)`:

Method	Returns	When
`.list()`	`List<T>`	any number of rows
`.optional()`	`Optional<T>`	0 or 1 row. More than one throws `IncorrectResultSizeDataAccessException`
`.single()`	`T`	exactly 1 row. Zero rows throws `EmptyResultDataAccessException`
`.set()` / `.stream()`	`Set<T>` / `Stream<T>`	a set, or streaming. Close the stream (try-with-resources)
`.query(rowMapper)` / `.query(rowCallbackHandler)`	custom	full control over mapping

Writing: generated keys and row counts

```
public long insert(String sku, String name, BigDecimal price, int stock) {
    KeyHolder keys = new GeneratedKeyHolder();
    jdbc.sql("INSERT INTO product (sku, name, price, in_stock) VALUES (:sku, :name, :price, :stock)")
        .param("sku", sku).param("name", name).param("price", price).param("stock", stock)
        .update(keys, "id");                  // ask the driver to return the generated "id"
    return keys.getKeyAs(Long.class);
}

public int changeStock(String sku, int delta) {
    return jdbc.sql("UPDATE product SET in_stock = in_stock + :d WHERE sku = :sku")
               .param("d", delta).param("sku", sku)
               .update();                     // number of rows changed
}
```

Output:

```
generated id = 6, rows updated = 1, now = Product[id=6, sku=HD-01, name=Headset, price=59.90, inStock=5]
```

Why 6 and not 4, when only three rows exist? Earlier tests inserted rows and were rolled back, but **identity and sequence values (the database's auto-increment id counters) are never rolled back**. The database hands out numbers without waiting to see whether the transaction commits, so gaps in ids are normal. Never use ids to count rows or to assume ordering without gaps.

The same with JdbcTemplate, plus a batch

```
private static final RowMapper<Product> MAPPER = (rs, rowNum) -> new Product(
        rs.getLong("id"), rs.getString("sku"), rs.getString("name"),
        rs.getBigDecimal("price"), rs.getInt("in_stock"));

public List<Product> inStockAtLeast(int min) {
    return jdbc.query("SELECT * FROM product WHERE in_stock >= ? ORDER BY sku", MAPPER, min);
}

/** One JDBC batch: one round trip per batch instead of one per row. */
public int[] restock(List<String> skus, int amount) {
    return jdbc.batchUpdate("UPDATE product SET in_stock = in_stock + ? WHERE sku = ?",
            skus.stream().map(s -> new Object[] {amount, s}).toList());
}
```

Output:

```
batch update counts = [1, 1, 0]
in stock >= 13: [KB-01, MN-27, MS-01]
```

The batch returns one count per parameter set. `NOPE` matched no row, so its count is 0. A batch sends all the statements in one network round trip (one request to the database and its reply). For 10 000 rows that is the difference between 10 000 round trips and a handful.

Trade-offs

	`JdbcTemplate`	`NamedParameterJdbcTemplate`	`JdbcClient`
Placeholders	`?`	`:name`	both
Style	overloaded methods	overloaded methods	fluent chain
Record/bean mapping	`DataClassRowMapper` by hand	same	automatic with `query(Type.class)`
Batch updates	`batchUpdate`	`batchUpdate`	not in the fluent API. Use `JdbcTemplate`
Stored procedures (functions stored and run inside the database)	`SimpleJdbcCall`	–	–

Common mistakes

- **String concatenation in SQL** (`"... WHERE sku = '" + sku + "'"`). This is SQL injection (an attacker putting SQL into an input so it runs as code). Always bind parameters. See SQL 14.9 — SQL injection.
- Using `.single()` when "no row" is a normal case. Use `.optional()`.
- `SELECT *` into a record. A column added later breaks nothing, but a renamed one fails at runtime. List the columns.
- Loading a million rows with `.list()`. Stream them, or page with keyset pagination (SQL 12.14).

In the interview

30-second answer: "`JdbcTemplate` and `JdbcClient` run hand-written SQL and handle connections, parameter binding, row mapping and exception translation. `JdbcClient` is the fluent API from Spring 6.1 and supports named and positional parameters and automatic mapping to records. Both take their connection from the `DataSource` pool. If a transaction is active, they use the connection bound to the current thread, so they take part in `@Transactional` automatically."

- *How does `JdbcTemplate` know about the current transaction?* → It gets its connection through `DataSourceUtils.getConnection`, which first checks `TransactionSynchronizationManager` for a connection already bound to this thread.
- *Is `JdbcTemplate` thread-safe?* → Yes. After configuration it holds no per-call state, so one shared bean is fine.

Self-check

1. Which API maps rows to a record without a `RowMapper`? → `JdbcClient` with `query(Product.class)`.
2. What does `update()` return? → The number of affected rows.
3. Why can a generated id "skip" numbers? → Identity/sequence values are not rolled back.

TL;DR

Use `JdbcClient` for new code: `sql(...)`, `param(...)`, `query(Type.class).list()/optional()/single()`, or `update()`. Use `JdbcTemplate.batchUpdate` for batches. Always bind parameters.

7.4 Schema and data initialization

Spring Boot can run SQL scripts at startup:

File	Runs	Purpose
`schema.sql` (and `schema-<platform>.sql`)	first	`CREATE TABLE ...`
`data.sql` (and `data-<platform>.sql`)	after `schema.sql`	seed rows

Property	Default	Meaning
`spring.sql.init.mode`	`embedded`	`embedded` = only for H2/HSQLDB/Derby, `always` = any database, `never` = off
`spring.sql.init.platform`	`all`	picks `schema-${platform}.sql`
`spring.sql.init.continue-on-error`	`false`	stop startup on a failing statement
`spring.jpa.defer-datasource-initialization`	`false`	with JPA: run the scripts *after* Hibernate has created the tables

`jdbc-basics` uses H2, so the scripts would run anyway. The project sets `mode=always` to make that visible. `tx-demos` points at PostgreSQL, which is not embedded, so there `mode=always` is required. Its `schema.sql` begins with `DROP TABLE IF EXISTS` so it can run again on every startup.

```
-- tx-demos/src/main/resources/schema.sql
CREATE TABLE account (
    owner   TEXT PRIMARY KEY,
    balance BIGINT NOT NULL CHECK (balance >= 0)
);
CREATE TABLE audit_log (id BIGSERIAL PRIMARY KEY, message TEXT NOT NULL);
```

When not to use it

In production the schema changes over time, and a script that only ever creates tables cannot handle that. Use a **migration tool** (Flyway or Liquibase: tools that keep track of which schema changes a database has already received) that applies numbered, versioned changes exactly once. See 8.16. `schema.sql` is fine for demos, tests and prototypes.

7.5 Exception translation: DataAccessException

The problem it solves

JDBC reports every failure as a checked (must be caught or declared) `SQLException` carrying a vendor error code (a number specific to that database product) and an SQLState (a standard five-character error code). To tell "duplicate key" from "deadlock" (two transactions each waiting for a lock the other holds, so neither can go on) you would have to check vendor codes such as PostgreSQL's `23505` or MySQL's `1062`. Spring translates them into one **unchecked** hierarchy that is the same for every database and every data-access technology (JDBC, JPA — Java's standard object/database mapping API, R2DBC — a non-blocking database API, ...).

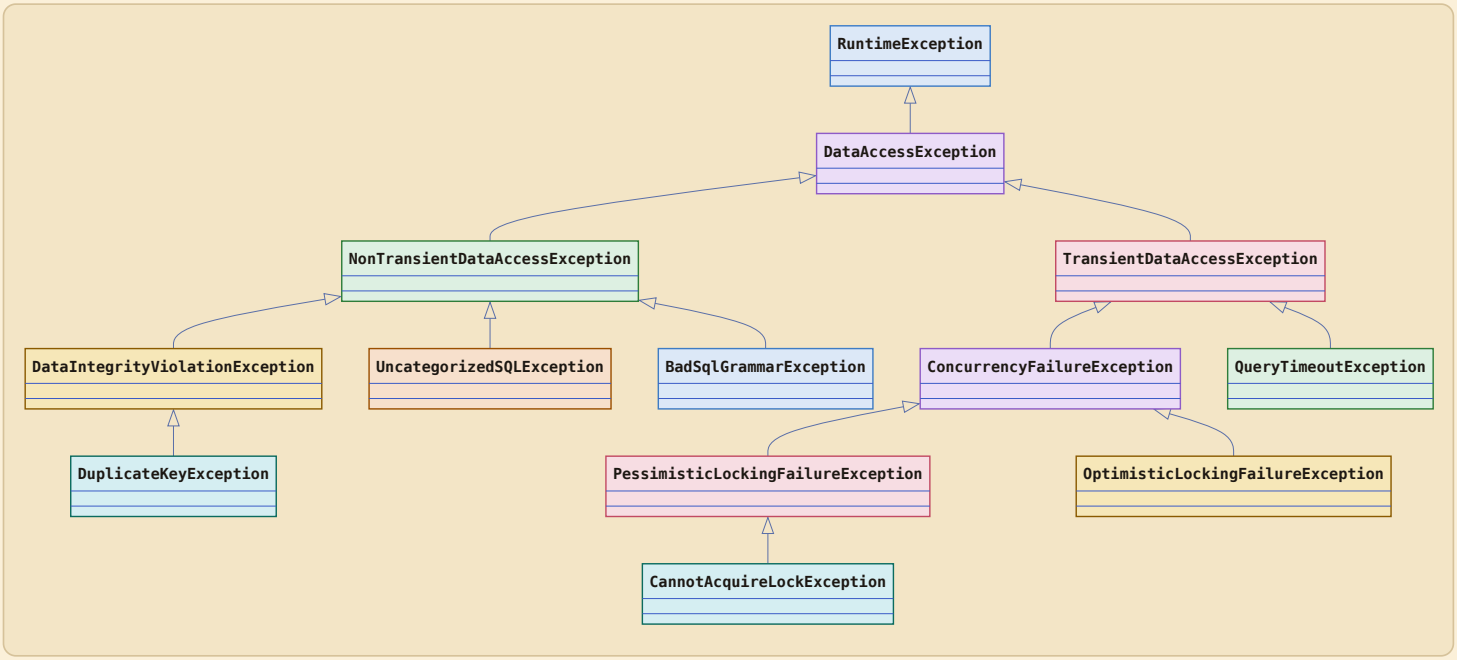
```
assertThatThrownBy(() -> repo.insert("KB-01", "Duplicate", BigDecimal.ONE, 1))
        .isInstanceOf(DuplicateKeyException.class)
        .isInstanceOf(DataAccessException.class);
assertThatThrownBy(() -> repo.insert("NEG-1", "Negative", new BigDecimal("-1"), 1))
        .isInstanceOf(DataIntegrityViolationException.class);
```

Output:

```
duplicate sku -> DuplicateKeyException
CHECK violated -> DataIntegrityViolationException
```

The part of the hierarchy you will meet

The diagram is simplified: a few intermediate classes are left out. For example, `BadSqlGrammarException` really extends `InvalidDataAccessResourceUsageException`.



Exception	Typical cause	Retry?
`DuplicateKeyException`	unique/primary key violated	no — fix the data
`DataIntegrityViolationException`	NOT NULL, CHECK, foreign key (a column that must point to an existing row in another table)	no
`BadSqlGrammarException`	typo in SQL, missing table	no — bug
`CannotAcquireLockException`, `PessimisticLockingFailureException`	lock timeout, deadlock, PostgreSQL serialization failure	**yes**, the whole transaction
`OptimisticLockingFailureException`	`@Version` conflict (someone else saved the row since you read it) (8.14)	yes, after reloading
`QueryTimeoutException`	statement cancelled by a timeout	maybe
`UncategorizedSQLException`	an error Spring has no specific class for	inspect `getCause()`

The **transient** branch means "the same operation might succeed if you try again". That is exactly the test to use when deciding whether to retry (ch04 covers `@Retryable`).

For JPA repositories the same translation is applied by a proxy around `@Repository` beans. In Boot 4 the switch is `spring.persistence.exceptiontranslation.enabled`. It was `spring.dao.exceptiontranslation.enabled` before Boot 4.

7.6 Testing JDBC code with @JdbcTest

`@JdbcTest` is a **test slice** (a test that starts only one layer of the app): it starts only the JDBC parts of the application (`DataSource`, `JdbcTemplate`, `JdbcClient`, the transaction manager, SQL init) and none of your services or controllers. Two defaults matter:

1. It replaces your `DataSource` with a fresh embedded database. `@AutoConfigureTestDatabase(replace = Replace.NONE)` keeps yours, which is how the Hikari settings above could be checked.
2. **Every test method runs in a transaction that is rolled back afterwards**, so tests do not see each other's changes.

```
@JdbcTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Import({ProductRepository.class, ProductTemplateDao.class})   // our beans are not in the slice
class JdbcClientTest {
    @Test
    void eachTestRunsInsideARolledBackTransaction() {
        System.out.println("test method in a transaction? "
                + TransactionSynchronizationManager.isActualTransactionActive());
        repo.changeStock("KB-01", -12);          // undone after the test
    }
}
```

Output:

```
test method in a transaction? true
```

In Boot 4 the slice annotations moved with the modules: `org.springframework.boot.jdbc.test.autoconfigure.JdbcTest`, from `spring-boot-starter-jdbc-test`. The rolled-back test transaction is convenient, but it can **hide transaction bugs**. Code that only works because the test opened a transaction will pass in the test and fail in production. That is why every demo in `tx-demos` is a non-transactional `@SpringBootTest`. More in ch12.

7.7 How a Spring transaction works

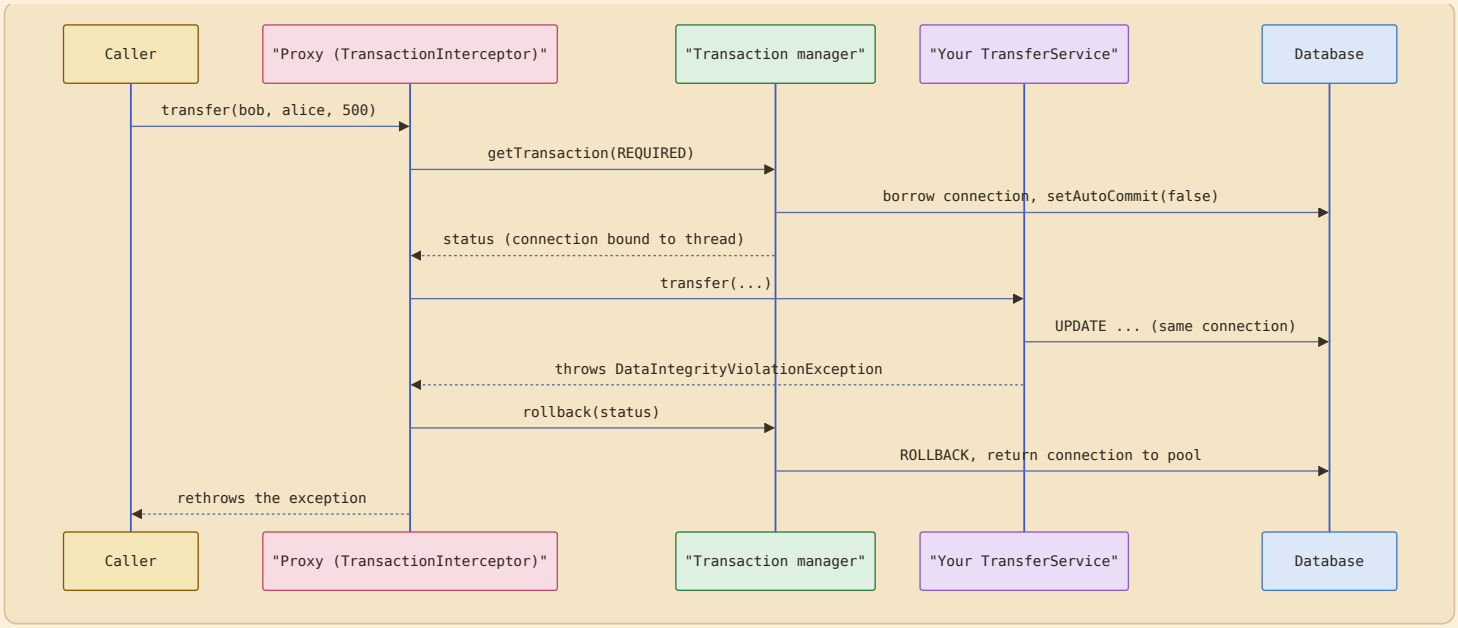
What it is

A **database transaction** groups statements so they all commit or all roll back (see SQL 13.1). On one JDBC connection that means `setAutoCommit(false)`, the statements, then `commit()` or `rollback()`. A **Spring transaction** does that work for you and adds three things:

1. **One connection per transaction, bound to the thread.** Every `JdbcClient`, `JdbcTemplate` or JPA call made inside the transaction, in any class, finds that connection through `TransactionSynchronizationManager`, which works like a `ThreadLocal` (a variable holding a separate value for each thread) map.
2. **Declarative boundaries.** `@Transactional` on a method means "wrap this method in a transaction".
3. **Rules** for what to do when a transactional method calls another one (propagation), when to roll back, which isolation level to use, and more.

How @Transactional works, step by step

Spring does not change your class. It puts a **proxy** (a generated stand-in object that intercepts calls and then forwards them) in front of the bean (AOP — see ch04). Other beans are injected with the proxy, not your object.



1. The caller calls the proxy.
2. The `TransactionInterceptor` reads the `@Transactional` attributes and asks the `PlatformTransactionManager` for a transaction.
3. The manager takes a connection from the pool, switches off auto-commit, applies isolation/read-only/timeout, and **binds the connection to the current thread**.
4. Your method runs. Every data-access call finds the bound connection.
5. If the method returns normally, the manager commits. If it throws, the rollback rules decide between commit and rollback (7.10).
6. The connection is unbound and returned to the pool.

Which transaction manager?

You use	Boot auto-configures	Notes
JDBC only (`spring-boot-starter-jdbc`)	`JdbcTransactionManager`	a `DataSourceTransactionManager` that also translates commit errors to `DataAccessException`
JPA (`spring-boot-starter-data-jpa`)	`JpaTransactionManager`	also manages the `EntityManager`. JDBC calls on the same `DataSource` join its transactions
Several resources (JMS — Java's messaging API — + DB)	none by default	a JTA (Java Transaction API) manager, or better, avoid distributed transactions (one transaction spanning several systems, which is slow and fragile) (see the outbox idea in ch14)
Reactive (R2DBC)	`R2dbcTransactionManager`	`ReactiveTransactionManager`, context-bound instead of thread-bound (ch13)

Seeing it

The demos print a small "where am I" string: whether a Spring transaction is active, the transaction's name (the method that started it), and the PostgreSQL **backend pid**. Every PostgreSQL connection is served by its own server process, so the same pid means the same connection.

```
public String where() {
    int pid = jdbc.sql("SELECT pg_backend_pid()").query(Integer.class).single();
    return "txActive=" + TransactionSynchronizationManager.isActualTransactionActive()
            + " name=" + TransactionSynchronizationManager.getCurrentTransactionName()
            + " pid=" + pid;
}
```

Test plumbing for the PostgreSQL demos

`tx-demos` needs its own database. A JUnit `LauncherSessionListener` (plain JUnit, not Spring, registered in `META-INF/services`) runs before the first test and after the last one. It connects to the admin database and creates or drops `spring_ch07_tx`:

```
public class DemoDatabase implements LauncherSessionListener {
    public void launcherSessionOpened(LauncherSession s) {
        run("DROP DATABASE IF EXISTS spring_ch07_tx WITH (FORCE)");
        run("CREATE DATABASE spring_ch07_tx");
    }
    public void launcherSessionClosed(LauncherSession s) {
        run("DROP DATABASE IF EXISTS spring_ch07_tx WITH (FORCE)"); // pool may still hold connections
    }
    // run(sql) = DriverManager.getConnection("jdbc:postgresql://127.0.0.1:5544/postgres", ...) + execute
}
```

The tests extend a base class that is **not** `@Transactional`. Commits and rollbacks really happen, and `@BeforeEach` resets the data to `alice = 100, bob = 50`.

7.8 @Transactional attributes

Attribute	Default	What it does
`propagation`	`REQUIRED`	what to do if a transaction already exists (7.9)
`isolation`	`DEFAULT` (= the database's default, READ COMMITTED on PostgreSQL)	7.11
`readOnly`	`false`	hint, and on some drivers a rule, that nothing is written (7.12)
`timeout` / `timeoutString`	none	seconds before statements are cancelled
`rollbackFor` / `rollbackForClassName`	–	also roll back for these (checked) exceptions
`noRollbackFor` / `noRollbackForClassName`	–	commit even though this exception was thrown
`transactionManager` (alias `value`)	the primary one	choose among several managers
`label`	–	free-form labels that a custom manager can read

Where to put it. On a class, it applies to every method the proxy can intercept. On a method, it overrides the class. The Spring team recommends annotating **concrete classes**, not interfaces. A common pattern is `@Transactional(readOnly = true)` on the class with `@Transactional` on the write methods. Spring Data's `SimpleJpaRepository` is built exactly that way.

7.9 Propagation

What it is

Propagation answers one question: *this method is transactional, and a transaction may already be running. What now?* Picture meetings. REQUIRED joins the meeting already in progress. REQUIRES_NEW steps out into a separate room and comes back. NESTED stays in the meeting but saves a checkpoint to return to.

Propagation	Transaction already running	No transaction running	Typical use
`REQUIRED` (default)	**join** it	start one	almost everything
`REQUIRES_NEW`	**suspend** it, start an independent one (new connection)	start one	audit logs that must survive a rollback, id generators
`NESTED`	create a **savepoint** (a marker inside a transaction you can roll back to without undoing everything) inside it	start one	"try this part, undo just this part on failure" (JDBC only)
`MANDATORY`	join it	**throw** `IllegalTransactionStateException`	methods that must never run on their own
`SUPPORTS`	join it	run **without**	read helpers that do not care
`NOT_SUPPORTED`	suspend it, run without	run without	slow non-transactional work inside a transactional flow
`NEVER`	**throw** `IllegalTransactionStateException`	run without	guard: "must not be called inside a transaction"

Demo 1 — one outer transaction calling each type

`OuterService.callEachPropagation()` is `@Transactional`. It calls methods on a **different** bean (`AuditService`, so the calls go through the proxy), and each method has a different propagation:

```
@Transactional
public List<String> callEachPropagation() {
    List<String> seen = new ArrayList<>();
    seen.add("outer        : " + db.where());
    seen.add("REQUIRED     : " + audit.required("required"));
    seen.add("REQUIRES_NEW : " + audit.requiresNew("requires-new"));
    seen.add("NESTED       : " + audit.nestedAndMaybeFail("nested", false));
    seen.add("MANDATORY    : " + audit.mandatory("mandatory"));
    seen.add("SUPPORTS     : " + audit.supports());
    seen.add("NOT_SUPPORTED: " + audit.notSupported());
    return seen;
}
```

The test raises the transaction manager's log level to DEBUG so you can read its decisions. Output (logger name and connection hashes shortened, a few "Initiating commit" and "Releasing connection" lines left out):

```
DEBUG JdbcTransactionManager - Creating new transaction with name [ch07.tx.OuterService.callEachPropagation]:
PROPAGATION_REQUIRED,ISOLATION_DEFAULT
DEBUG JdbcTransactionManager - Acquired Connection [HikariProxyConnection@... wrapping PgConnection@2b6262bc] for JDBC transaction
DEBUG JdbcTransactionManager - Switching JDBC Connection [...PgConnection@2b6262bc] to manual commit
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Suspending current transaction, creating new transaction with name [ch07.tx.AuditService.requiresNew]
DEBUG JdbcTransactionManager - Acquired Connection [...PgConnection@3e28dc96] for JDBC transaction
DEBUG JdbcTransactionManager - Committing JDBC transaction on Connection [...PgConnection@3e28dc96]
DEBUG JdbcTransactionManager - Resuming suspended transaction after completion of inner transaction
DEBUG JdbcTransactionManager - Creating nested transaction with name [ch07.tx.AuditService.nestedAndMaybeFail]
DEBUG JdbcTransactionManager - Releasing transaction savepoint
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Suspending current transaction
DEBUG JdbcTransactionManager - Resuming suspended transaction after completion of inner transaction
DEBUG JdbcTransactionManager - Committing JDBC transaction on Connection [...PgConnection@2b6262bc]
outer        : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
REQUIRED     : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
REQUIRES_NEW : txActive=true name=ch07.tx.AuditService.requiresNew pid=92827
NESTED       : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
MANDATORY    : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
SUPPORTS     : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
NOT_SUPPORTED: txActive=false name=ch07.tx.AuditService.notSupported pid=92827
```

Reading it (the pids and connection hashes change from run to run; what matters is which lines share a pid):

- **REQUIRED, MANDATORY, SUPPORTS** joined: same name, same pid 92825 ("Participating in existing transaction").
- **REQUIRES_NEW** took a **second connection** (pid 92827) and committed on its own before the outer transaction continued.
- **NESTED** stayed on the same connection. The log shows a savepoint being created and then released.
- **NOT_SUPPORTED** suspended the outer transaction and ran with no transaction (`txActive=false`). It still needed a connection, so it borrowed one from the pool, here the one REQUIRES_NEW had just returned.

A subtle cost: while REQUIRES_NEW runs, **one thread holds two connections**. If every request does that, a pool of 10 can deadlock with only 5 concurrent requests, each holding one connection and waiting for a second.

Demo 2 — without an outer transaction

```
SUPPORTS      : txActive=false name=ch07.tx.AuditService.supports pid=92825
MANDATORY     : No existing transaction found for transaction marked with propagation 'mandatory'
NEVER in tx   : Existing transaction found for transaction marked with propagation 'never'
```

SUPPORTS simply ran without a transaction. MANDATORY and NEVER threw `IllegalTransactionStateException`.

Demo 3 — REQUIRES_NEW survives the outer rollback

```
@Transactional
public void outerFailsAfterRequiresNew() {
    db.audit("outer work");
    audit.requiresNew("audit written in REQUIRES_NEW");
    throw new IllegalStateException("outer failed");
}
```

```
=== REQUIRES_NEW commits even though the outer transaction rolls back ===
audit_log = [audit written in REQUIRES_NEW]
```

"outer work" is gone and the audit row stayed. This is the standard way to record *"someone tried to do X and it failed"*.

Demo 4 — NESTED rolls back to the savepoint only

```
@Transactional
public void outerSurvivesNestedFailure() {
    db.audit("outer work");
    try {
        audit.nestedAndMaybeFail("nested work", true);
    } catch (IllegalStateException e) {
        db.audit("outer caught: " + e.getMessage());
    }
}
```

```
DEBUG JdbcTransactionManager - Creating nested transaction with name [ch07.tx.AuditService.nestedAndMaybeFail]
DEBUG JdbcTransactionManager - Rolling back transaction to savepoint
DEBUG JdbcTransactionManager - Committing JDBC transaction on Connection [...]
audit_log = [outer work, outer caught: inner NESTED failed]
```

"nested work" was undone with `ROLLBACK TO SAVEPOINT`, and the rest committed. NESTED is supported by the JDBC transaction managers. With JPA, `JpaTransactionManager` can create JDBC savepoints, but Hibernate's in-memory state (the persistence context) is not rolled back with them. Avoid NESTED with JPA.

Demo 5 — catching an inner REQUIRED failure does not save you

This surprises almost everyone:

```
@Transactional
public void outerCatchesRequiredFailure() {
    db.audit("outer work");
    try {
        audit.requiredAndFail("inner work");   // REQUIRED -> joins, then throws
    } catch (IllegalStateException e) {
        db.audit("outer caught: " + e.getMessage() + " and carries on");
    }
}
```

```
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Participating transaction failed - marking existing transaction as rollback-only
DEBUG JdbcTransactionManager - Global transaction is marked as rollback-only but transactional code requested commit
DEBUG JdbcTransactionManager - Rolling back JDBC transaction on Connection [...]
UnexpectedRollbackException: Transaction rolled back because it has been marked as rollback-only
audit_log = []
```

The inner method **shared** the outer transaction. When the exception left the inner proxy, that proxy could not roll back only its own part, because there is only one physical transaction. So it marked the whole transaction **rollback-only**. The outer method caught the exception and returned normally. At commit time Spring found the mark, rolled everything back, and threw `UnexpectedRollbackException` so the failure is not silent.

Fixes: let the exception propagate. Or make the inner call `REQUIRES_NEW`/`NESTED` if its failure really should be isolated. Or check for the error condition before calling instead of catching the exception.

Common mistakes

- Using REQUIRES_NEW everywhere "for safety". It doubles connection use and breaks atomicity (the all-or-nothing guarantee), because parts commit independently.
- Expecting propagation to apply to calls inside the same class. It does not (7.13).
- Expecting a transaction to follow work into another thread (`@Async`, `CompletableFuture`, parallel streams). Transactions are bound to one thread, so the new thread has none.

In the interview

30-second answer: "Propagation decides what a transactional method does when a transaction already exists. REQUIRED, the default, joins it or starts one. REQUIRES_NEW suspends the caller's transaction and runs an independent one on a new connection. NESTED uses a savepoint. MANDATORY and NEVER throw if a transaction is missing or present. SUPPORTS and NOT_SUPPORTED run with or without one."

- *Inner REQUIRED method throws, outer catches it. What happens?* → The shared transaction is marked rollback-only. The outer commit becomes a rollback and throws `UnexpectedRollbackException`.
- *REQUIRES_NEW vs NESTED?* → REQUIRES_NEW is a separate physical transaction that commits independently, even if the outer one rolls back later. NESTED is a savepoint in the same transaction, so if the outer transaction rolls back, the nested work is lost too.
- *Risk of REQUIRES_NEW?* → Two connections per thread, which can exhaust the pool. It can also deadlock if the inner transaction waits for a row lock the outer one holds.

Self-check

1. Which propagations never run inside the caller's transaction? → REQUIRES_NEW (new one), NOT_SUPPORTED (none), NEVER (throws instead).
2. Why did NOT_SUPPORTED show a different pid? → Without the transaction's bound connection, the query borrowed any free pool connection.
3. What does "rollback-only" mean? → A flag on the shared transaction saying that it may only roll back. Any later commit attempt rolls back instead.

TL;DR

REQUIRED joins, REQUIRES_NEW isolates on a second connection, NESTED uses a savepoint. A failure inside a *joined* transaction dooms the whole transaction, even if you catch the exception.

7.10 Rollback rules

What it is

When an exception leaves a `@Transactional` method, Spring must choose between rollback and commit. The default rule copies EJB (Enterprise JavaBeans, the older Java EE component model):

- **`RuntimeException` and `Error` → rollback.**
- **Checked exceptions (`Exception` subclasses that are not runtime exceptions) → commit.**

The idea was that checked exceptions are "business outcomes" the caller is expected to handle. In practice this causes bugs. (Checked vs unchecked is explained in the Java book: Java 8.2.)

The demo

```
public class FraudCheckException extends Exception { ... }  // CHECKED

@Transactional
public void transferThenFailChecked(String from, String to, long amount) throws FraudCheckException {
    transferSteps(from, to, amount);
    throw new FraudCheckException("fraud check failed (checked)");
}

@Transactional(rollbackFor = FraudCheckException.class)
public void transferThenFailCheckedWithRollbackFor(...) throws FraudCheckException { /* same */ }

@Transactional(noRollbackFor = IllegalArgumentException.class)
public void transferWithNoRollbackFor(...) { transferSteps(...); throw new IllegalArgumentException(...); }
```

Output (start: `alice=100 bob=50`, transfer 30 from alice to bob):

```
CHECK violation      -> rolled back, alice=100 bob=50
RuntimeException     -> rolled back, alice=100 bob=50
checked exception    -> COMMITTED (!), alice=70 bob=80
checked + rollbackFor -> rolled back, alice=100 bob=50
unchecked + noRollbackFor -> committed, alice=70 bob=80
```

The third line is the trap. The method **threw**, the caller saw an exception, and the money moved anyway.

Changing the default for the whole application (Framework 6.2+)

```
@Configuration
@EnableTransactionManagement(proxyTargetClass = true, rollbackOn = RollbackOn.ALL_EXCEPTIONS)
public class RollbackOnAllExceptionsConfig { }
```

With this configuration active (a separate test context with the profile `rollback-all`), the *same* method rolls back:

```
rollbackOn=ALL_EXCEPTIONS, checked exception -> rolled back, alice=100 bob=50
```

The Spring reference docs recommend `ALL_EXCEPTIONS` unless you really rely on "commit on checked exception". `rollbackFor`/`noRollbackFor` on a method still override the global default. When you declare your own `@EnableTransactionManagement`, Boot's version backs off. Keep `proxyTargetClass = true` so beans still get class-based proxies, as with Boot's default.

Rules summary

Thrown	Default	`rollbackFor = X`	`noRollbackFor = X`	`rollbackOn = ALL_EXCEPTIONS`
`RuntimeException` / `Error`	rollback	rollback	commit if it is an X	rollback
checked exception	**commit**	rollback if it is an X	commit	rollback
exception caught inside the method	nothing to decide — commits	same	same	same

Rules match **subclasses** too: `rollbackFor = Exception.class` covers every checked exception. The `...ClassName` variants match by name pattern, which is powerful but easy to over-match.

Common mistakes

- Catching an exception, logging it, and not rethrowing. The method ends normally and **commits**. If you must swallow it, call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`, or better, use `TransactionTemplate`.
- Wrapping a runtime exception into a checked one on the way out, which silently switches the outcome to commit.
- Assuming `@Transactional(rollbackFor = ...)` on an interface method is honoured by every proxy type. Annotate the class.

In the interview

> **30-second answer:** "By default Spring rolls back on unchecked exceptions and errors and commits on checked exceptions. You change it per method with `rollbackFor` / `noRollbackFor`, or since Spring 6.2 globally with `@EnableTransactionManagement(rollbackOn = ALL_EXCEPTIONS)`. A caught exception never triggers a rollback, because the proxy never sees it."

- *My method threw an `IOException` but the data was saved. Why?* → `IOException` is checked, so the default rule commits. Add `rollbackFor` or switch the global default.

Self-check

1. Does `@Transactional` roll back for `SQLException` thrown from your own JDBC code? → No, it is checked. (Spring's own helpers throw unchecked `DataAccessException`s, which do roll back.)
2. What happens if you catch the exception inside the method? → The proxy sees a normal return and commits.

TL;DR

Unchecked → rollback, checked → **commit** by default. Use `rollbackFor`, or `rollbackOn = ALL_EXCEPTIONS` (6.2+).

7.11 Isolation levels

What it is

Isolation controls what one transaction can see of other transactions that run at the same time. The anomalies (dirty read, non-repeatable read, phantom, lost update, write skew — each a different way concurrent transactions can see or produce wrong data) and how PostgreSQL implements each level with MVCC are taught in SQL 13.4 — Isolation and SQL 13.7 — Lost updates. In Spring you only choose the level:

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
```

Spring calls `Connection.setTransactionIsolation(...)` when it starts the transaction and resets it afterwards. `Isolation.DEFAULT` leaves the database default in place: READ COMMITTED on PostgreSQL, Oracle and SQL Server, REPEATABLE READ on MySQL InnoDB. Isolation only applies when a **new** transaction starts. A method that joins an existing transaction inherits the outer transaction's level.

Demo — two reads, and another user commits in between

"Another user" is simulated with a `REQUIRES_NEW` method on a second bean. It uses a second connection and commits immediately, which makes the demo run on one thread with a predictable result.

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public List<Long> readTwiceReadCommitted() {
    long first = db.balance("alice");
    other.deposit("alice", 50);          // REQUIRES_NEW: commits on another connection
    long second = db.balance("alice");
    return List.of(first, second);
}
// readTwiceRepeatableRead(): identical, with Isolation.REPEATABLE_READ
```

```
READ_COMMITTED  reads alice twice: [100, 150]  (non-repeatable read)
REPEATABLE_READ reads alice twice: [100, 100]  (same snapshot)
```

Demo — the lost update

Read a value into Java, change it, write it back. This is the most common concurrency bug in business code:

```java
@Transactional(isolation = Isolation.READ_COMMITTED)
public void addTenReadCommitted() {
    long read = db.balance("alice");     // 100
    other.deposit("alice", 50);          // another user: 150, committed
    db.setBalance("alice", read + 10);   // writes 110
}
```

```
READ_COMMITTED  read 100, other +50, write read+10 -> alice=110  (the +50 is LOST)
REPEATABLE_READ same steps -> CannotAcquireLockException: ERROR: could not serialize access due to concurrent update
alice=150 (the deposit survived; retry the whole transaction)
```

Under REPEATABLE READ, PostgreSQL notices that the row changed after this transaction's snapshot was taken and refuses the update (SQLState `40001`). Spring translates that into `CannotAcquireLockException`, a `ConcurrencyFailureException`, so it is in the "transient, retry" branch of 7.5. **The whole transaction must be retried from the start**, not just the failed statement.

Ways to prevent lost updates

Technique	How	Where
Do the math in SQL	`UPDATE account SET balance = balance + 10`	simplest — no read-modify-write in Java
Pessimistic lock	`SELECT ... FOR UPDATE`, then update	SQL 13.12, JPA `@Lock` in 8.14
Optimistic lock	version column, `UPDATE ... WHERE version = ?`	JPA `@Version` in 8.14
Stronger isolation	REPEATABLE READ / SERIALIZABLE + retry on `40001`	this section

Common mistakes

- Setting `isolation` on a method that is always called inside another transaction. It is ignored, because only the method that starts the transaction sets the level.
- Using SERIALIZABLE without a retry loop. Serialization failures are *expected* at that level.
- Relying on H2 to test isolation. Its behaviour differs from PostgreSQL's, which is why these demos use a real server.

Self-check

1. Why did REPEATABLE READ return `[100, 100]`? → PostgreSQL takes one snapshot at the first query and every later read uses it.
2. Which exception tells you to retry? → Anything in `TransientDataAccessException`, such as `CannotAcquireLockException` here.

7.12 readOnly and timeout

readOnly

`@Transactional(readOnly = true)` does different things depending on the stack:

Stack	Effect
JDBC (`JdbcTransactionManager`)	calls `Connection.setReadOnly(true)`. The PostgreSQL driver then starts the transaction as `READ ONLY`, so writes **fail**
JPA / Hibernate	the session becomes read-only and flush mode goes to manual (flushing = sending pending changes to the database). No snapshots for dirty checking (comparing objects with their loaded copies to find changes), no automatic flush (faster reads, 8.4)
Routing `DataSource`	can send read-only transactions to a replica (a read-only copy of the database kept in sync with the main one; see System Design Fundamentals) (a custom `AbstractRoutingDataSource` that checks `TransactionSynchronizationManager.isCurrentTransactionReadOnly()`)

On PostgreSQL:

```java
@Transactional(readOnly = true)
public void writeInReadOnly() { db.audit("should not be allowed"); }
```

```
readOnly=true + INSERT -> UncategorizedSQLException: ERROR: cannot execute INSERT in a read-only transaction
```

Other drivers may treat read-only only as a hint. Do not rely on it for security. Use database permissions for that (SQL 14.8).

timeout

`timeout` (seconds) is a deadline for the whole transaction. Spring's JDBC helpers turn the *remaining* time into a JDBC query timeout on each statement. When the deadline has already passed, the next statement fails before it runs.

```
@Transactional(timeout = 1)
public void slowQuery() { db.sleepSeconds(3); }   // SELECT pg_sleep(3)
```

```
timeout=1 + pg_sleep(3) -> QueryTimeoutException: ERROR: canceling statement due to user request
gave up after ~1000 ms
```

The driver asked PostgreSQL to cancel the statement after one second. Keep in mind: a timeout cancels **database statements**. Plain Java code, or a slow HTTP call made inside the transaction, is not interrupted. The deadline is only checked at the next statement.

7.13 Transaction pitfalls

All of these pitfalls come from one fact: **`@Transactional` works only when the call goes through the Spring proxy.** Proxies and AOP are covered in ch04. Here you see the effect on real data.

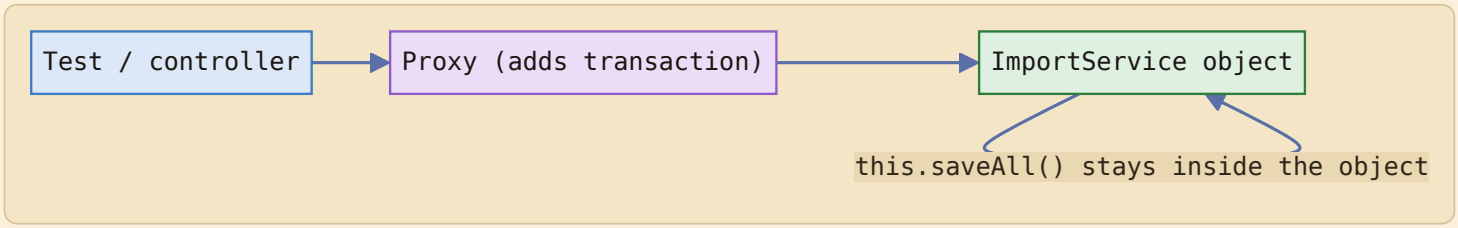
```
System.out.println("injected class = " + imports.getClass().getSimpleName()
        + ", CGLIB proxy? " + AopUtils.isCglibProxy(imports));
```

```
injected class = ImportService$$SpringCGLIB$$0, CGLIB proxy? true
```

Pitfall 1 — self-invocation

```
@Service
public class ImportService {
    public void importAll(List<String> lines) {   // NOT transactional
        saveAll(lines);                            // this.saveAll(): bypasses the proxy
    }

    @Transactional
    public void saveAll(List<String> lines) {
        for (String line : lines) {
            if (line.equals("BAD")) throw new IllegalArgumentException("bad line");
            db.audit(line);
        }
    }
}
```

```
via this.saveAll()  -> audit_log = [line-1, line-2]   (partial import!)
via proxy.saveAll() -> audit_log = []   (rolled back)
```



The `@Transactional` on `saveAll` was ignored because the call never left the object. Each `INSERT` ran in auto-commit mode.

Fixes, in order of preference:

1. Put the annotation on the method that is called from outside (`importAll`), or on both.
2. Move `saveAll` to another bean, so the call goes through that bean's proxy.
3. Use `TransactionTemplate` inside the method (7.14). No proxy is needed.
4. Inject the bean into itself (`@Lazy` self-reference), or use AspectJ weaving (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)`). Both work, and both are rarely worth the complexity.

Pitfall 2 — `final` and `private` methods

A CGLIB proxy is a generated **subclass** that overrides your methods. It cannot override:

- **`private` methods**, which are invisible to subclasses. `@Transactional` on them is silently ignored. They can only be called from inside the class anyway, which is self-invocation.
- **`final` methods**. The call runs directly on the *proxy instance*. The proxy object is created without running your constructor, so its fields are `null`:

```
@Transactional
public final String finalMethod() { return db.where(); }
```

```
final @Transactional method -> NullPointerException (runs on the proxy, whose 'db' field is null)
```

Why this happens, with a proof that does not involve transactions, is in 4.5.

Protected and package-private methods *are* transactional with class-based proxies since Spring Framework 6.0:

```
package-private @Transactional -> txActive=true name=ch07.tx.ImportService.packagePrivate pid=92825
```

With JDK interface proxies, only methods of the interface are intercepted. Spring Boot uses CGLIB class proxies by default (`spring.aop.proxy-target-class=true`).

Pitfall 3 — the transaction on the wrong layer

A transaction should cover one **use case**, "transfer money", not each individual SQL call. Here every DAO (Data Access Object: a class that wraps database calls) method is its own transaction, which is what you get by default with Spring Data repository methods:

```
@Repository
public class AccountDao {
    @Transactional public void add(String owner, long delta) { db.addToBalance(owner, delta); }
}

@Service
public class NoTxTransferService {           // no @Transactional on the use case
    public void transfer(String from, String to, long amount) {
        dao.add(to, amount);    // commits immediately
        dao.add(from, -amount); // fails (CHECK balance >= 0) -> credit above stays
    }
}
```

```
tx per DAO call     -> alice=600 bob=50   (500 created from nothing)
tx on service method -> alice=100 bob=50
```

Rule of thumb: `@Transactional` **goes on service methods (the use case)**. Not on controllers, which would hold a connection during serialization and HTTP work. And not only on repositories, whose transactions are too small.

Pitfall 4 — long transactions and remote calls

```
@Transactional
public void placeOrder(Order o) {
    orders.save(o);
    paymentClient.charge(o);   // 2 s HTTP call while holding a DB connection and row locks
}
```

The connection is held for the whole HTTP call. If the payment succeeds and the commit then fails, the customer has been charged and no order exists. Keep transactions short and database-only. Make remote calls before or after the transaction, or publish an event after commit (7.15), or use the transactional outbox (write the outgoing message to a table in the same transaction; a separate process sends it later) (ch14, event-driven patterns).

Pitfall 5 — other threads

A transaction is bound to the thread that started it. Work handed to `@Async` methods, executors, `CompletableFuture.supplyAsync` or parallel streams runs **outside** the transaction, on other connections, and cannot see uncommitted data.

Checklist: "why is my @Transactional not working?"

Symptom	Likely cause
Partial writes after an exception	self-invocation, `private`/`final` method, bean not managed by Spring (`new MyService()`)
Data committed despite an exception	checked exception without `rollbackFor`, or the exception was caught inside
`UnexpectedRollbackException`	an inner REQUIRED method failed and someone caught its exception
`LazyInitializationException`	JPA entity used after the transaction ended (8.6)
Works in tests, fails in production	the test itself was `@Transactional` and supplied the missing transaction
Pool exhausted under load	REQUIRES_NEW nesting, or slow remote calls inside transactions

In the interview

30-second answer: "`@Transactional` is implemented by a proxy, so it only applies to calls that come in through the proxy. Calls from one method to another in the same class skip it. So do `private` methods, and `final` methods break outright. Put

transactions on service methods that represent a use case, keep them short, and never make remote calls inside them."

- *How do you fix self-invocation?* → Move the method to another bean, annotate the entry method, or use `TransactionTemplate`.
- *Why not put `@Transactional` on the controller?* → It widens the transaction to include request parsing, serialization and remote calls, holding connections and locks longer. The service layer is the use-case boundary.

Self-check

1. Why did `importAll` leave two rows? → `this.saveAll()` bypassed the proxy, so each insert auto-committed.
2. Why did the `final` method throw `NullPointerException` rather than just running without a transaction? → It ran on the proxy object, whose injected fields are `null`.
3. Where should `@Transactional` go? → On the service method that represents the whole use case.

TL;DR

No proxy, no transaction. Watch for self-calls, `private`/`final` methods, transactions that are too small (DAO) or too big (controller, remote calls), and work moved to other threads.

7.14 Programmatic transactions: TransactionTemplate

What it is

`TransactionTemplate` starts a transaction around a lambda. It uses the same manager and the same rules, with **no proxy**, so self-invocation cannot affect it. Boot auto-configures one bean with default settings. For other settings, create your own from the `PlatformTransactionManager`.

```
@Service
public class ProgrammaticService {
    private final TransactionTemplate tx;           // auto-configured
    private final TransactionTemplate serializable;

    public ProgrammaticService(Db db, TransactionTemplate tx, PlatformTransactionManager txManager) {
        this.tx = tx;
        this.serializable = new TransactionTemplate(txManager);
        this.serializable.setIsolationLevel(TransactionDefinition.ISOLATION_SERIALIZABLE);
        this.serializable.setTimeout(5);
    }

    public long transferAndReturnNewBalance(String from, String to, long amount) {
        return tx.execute(status -> {               // only the lambda is transactional
            db.addToBalance(to, amount);
            db.addToBalance(from, -amount);
            return db.balance(to);
        });
    }

    public String tryThenUndo() {
        return tx.execute(status -> {
            db.audit("temporary");
            String seen = "inside: " + db.auditMessages();
            status.setRollbackOnly();               // roll back without throwing
            return seen;
        });
    }

    public String isolationOfCustomTemplate() {
        return serializable.execute(status -> db.showIsolation());   // SHOW transaction_isolation
    }
}
```

```
TransactionTemplate transfer -> bob now 70
setRollbackOnly: inside: [temporary], after: []
custom template isolation = serializable
```

When to use it

`@Transactional`	`TransactionTemplate`
whole method = one transaction	transaction around *part* of a method
clean business code	explicit and visible
proxy pitfalls apply	no proxy — works for self-calls
rollback via exceptions/rules	also `status.setRollbackOnly()`

Good fits for `TransactionTemplate`: batch jobs that commit every N items, loops where each item gets its own transaction, retry loops (the retry must be *outside* the transaction), and places where the proxy pitfalls bite. Even lower level, `PlatformTransactionManager.getTransaction/commit/rollback` exists, but the template handles the error cases for you.

7.15 @TransactionalEventListener

The problem it solves

"After an order is saved, send a confirmation e-mail." If the e-mail goes out inside the transaction and the transaction then rolls back, the customer gets an e-mail for an order that does not exist. You want **"only if the commit succeeded"**.

How it works

1. Inside the transaction, publish an application event (`ApplicationEventPublisher.publishEvent`).
2. A plain `@EventListener` runs **immediately**, in the same thread and the same transaction.
3. A `@TransactionalEventListener` is **registered with the transaction** and runs at a chosen phase:

Phase	Runs when	Typical use
`BEFORE_COMMIT`	just before commit (can still write, can still fail the commit)	last validations, flushing extra data
`AFTER_COMMIT` (default)	after a successful commit	e-mails, messages, cache eviction (removing stale entries from a cache)
`AFTER_ROLLBACK`	after a rollback	compensation (undoing earlier side effects), alerts
`AFTER_COMPLETION`	after commit **or** rollback	cleanup

```
@Transactional
public void place(String item, boolean failAfterPublish) {
    db.audit("order " + item);
    events.publishEvent(new OrderPlaced(item));
    if (failAfterPublish) throw new IllegalStateException("payment declined");
}

@Component
public class OrderListeners {
    @EventListener                        void immediately(OrderPlaced e)    { ... }
    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)    void beforeCommit(OrderPlaced e) { ... }
    @TransactionalEventListener           void afterCommit(OrderPlaced e)    { ... }
    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)   void afterRollback(OrderPlaced e) { ... }
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMPLETION) void afterCompletion(OrderPlaced e) { ... }
}
```

```
commit   -> [@EventListener book, BEFORE_COMMIT book, AFTER_COMMIT book, AFTER_COMPLETION book]
rollback -> [@EventListener lamp, AFTER_ROLLBACK lamp, AFTER_COMPLETION lamp], audit_log=[]
no tx    -> [@EventListener pen]
```

In the last line the event was published with no transaction active, so the transactional listeners were **skipped**. Set `@TransactionalEventListener(fallbackExecution = true)` if they should run in that case too.

Common mistakes

- **Writing to the database in an AFTER_COMMIT listener without a new transaction.** The original transaction has finished, but its resources are still bound during cleanup. Since Spring 6.1, a `@TransactionalEventListener` method may be `@Transactional` only with `REQUIRES_NEW` (or `NOT_SUPPORTED`, allowed since 6.1.3). Anything else fails at startup.
- **Assuming AFTER_COMMIT is reliable delivery.** If the process crashes right after the commit, the e-mail is never sent, because the event lived only in memory. For guaranteed delivery, write an "outbox" row in the same transaction and have a separate process send it (ch14).
- Doing slow work in the listener on the request thread. Combine it with `@Async` to move the work off the request thread.

In the interview

30-second answer: "`@TransactionalEventListener` defers an event listener to a transaction phase, by default after commit. So side effects like e-mails or messages only happen when the data is really saved. If no transaction is active the listener is skipped unless `fallbackExecution` is true. If the listener must write to the database it needs `REQUIRES_NEW`. For guaranteed delivery you still need an outbox, because the event is only in memory."

Self-check

1. Which listener ran on rollback? → `@EventListener` (immediately), then AFTER_ROLLBACK and AFTER_COMPLETION.
2. What happens to an AFTER_COMMIT listener when there is no transaction? → It is skipped, unless `fallbackExecution = true`.

7.16 Interview questions

1. **What does Spring add on top of JDBC?** → Connection pooling (HikariCP via Boot), boilerplate removal (`JdbcClient`/`JdbcTemplate`), exception translation to `DataAccessException`, and declarative transactions.
2. **`JdbcClient` vs `JdbcTemplate`?** → Same engine. `JdbcClient` (6.1+) is a fluent API with named/positional parameters and automatic record mapping. `JdbcTemplate` stays for batches and older code.
3. **How does `@Transactional` work internally?** → A proxy's `TransactionInterceptor` asks the `PlatformTransactionManager` to begin a transaction, which binds a connection to the thread. The method runs. Then the interceptor commits, or rolls back according to the rollback rules.
4. **Default propagation and isolation?** → REQUIRED, and the database default (READ COMMITTED on PostgreSQL).
5. **Checked exception thrown from `@Transactional` — commit or rollback?** → Commit, unless `rollbackFor` or `rollbackOn = ALL_EXCEPTIONS` (6.2+) says otherwise.
6. **Why is my transaction ignored when I call the method from the same class?** → Self-invocation bypasses the proxy.
7. **Explain `UnexpectedRollbackException`.** → A participating (joined) method failed, which marked the shared transaction rollback-only. The outer code caught the exception and tried to commit.
8. **REQUIRES_NEW vs NESTED?** → A separate physical transaction on a new connection vs a savepoint in the same transaction.
9. **What does `readOnly = true` do?** → With JDBC it makes the connection read-only (PostgreSQL rejects writes). With Hibernate it disables dirty checking and flushing. It can also route to replicas.
10. **How do you run code only after a commit?** → `@TransactionalEventListener` (AFTER_COMMIT) or `TransactionSynchronization.afterCommit`. For guaranteed delivery, an outbox.
11. **How do you handle a serialization failure?** → Catch `ConcurrencyFailureException`/`TransientDataAccessException` and retry the *whole* transaction, with the retry outside the transactional method.
12. **Does `@Transactional` work with `@Async`?** → The async method runs on another thread with no inherited transaction. It can have its own `@Transactional`, but it is a separate transaction.

Chapter TL;DR

- Boot builds a **HikariCP** pool from `spring.datasource.*`. Keep it small, keep connections short, and remember the total across instances.
- `JdbcClient` (6.1+) is the modern way to run SQL: `sql().param().query(Type.class).list()`, `update()`. Use `JdbcTemplate.batchUpdate` for batches. Always bind parameters.
- Spring translates `SQLException` into the unchecked `DataAccessException` hierarchy. The *transient* branch means "retry may work".
- `@Transactional` = proxy + transaction manager + a connection bound to the thread. Defaults: REQUIRED, database isolation, read-write, rollback on unchecked exceptions only.
- **Propagation**: REQUIRED joins, REQUIRES_NEW runs separately on a second connection, NESTED uses a savepoint, MANDATORY/NEVER guard, SUPPORTS/NOT_SUPPORTED are optional/none. A failure in a joined transaction dooms it (`UnexpectedRollbackException`).
- **Checked exceptions commit** by default. Use `rollbackFor` or `rollbackOn = ALL_EXCEPTIONS`.
- **Isolation** is the database's job (SQL ch13). Spring sets the level. Retry on serialization failures.
- **Pitfalls**: self-invocation, `private`/`final` methods, the wrong layer, remote calls inside transactions, other threads, `@Transactional` tests hiding bugs.
- `TransactionTemplate` for precise, proxy-free boundaries. `@TransactionalEventListener` for after-commit side effects.

Next: Chapter 8 — Spring Data JPA and Hibernate moves from SQL you write to entities and repositories.

Chapter 8 — Spring Data JPA and Hibernate

Chapter 7 wrote SQL by hand. This chapter lets an **ORM** (object-relational mapper) write it. You describe Java classes and their relationships, and Hibernate (the most widely used ORM for Java) generates the `SELECT`, `INSERT`, `UPDATE` and `DELETE` statements, tracks changes to your objects, and loads related objects on demand. Spring Data JPA adds **repositories**: interfaces whose methods become queries without any implementation code. (JPA, the Jakarta Persistence API, is the standard Java API that ORMs such as Hibernate implement.)

This is very productive, and it is also where many performance problems in Spring applications come from. So every important behaviour below is *measured*. The tests count the SQL statements Hibernate really sends and print them. You will see the N+1 problem (one query for a list, then one extra query per item — §8.7) happen, then disappear with three different fixes. You will also see a `LazyInitializationException` (the error when related data is loaded after the database session has closed), dirty checking (Hibernate spotting changed objects and writing them back on its own) writing an `UPDATE` without any `save()` call, and two users colliding on an optimistic lock (a version check that detects a conflicting save instead of blocking it).

What you'll learn

- How JPA, Hibernate and Spring Data JPA relate (and which versions Boot 4.1 ships)
- Entities (classes whose objects are stored as table rows), ids and id generation strategies
- The persistence context: entity states, the first-level cache, dirty checking, flushing (sending queued changes to the database)
- Relationships: `@ManyToOne`, `@OneToMany`, `@ManyToMany`, owning side, `mappedBy`, cascades, `orphanRemoval`
- Fetch types (when related data is loaded), `LazyInitializationException`, and Open Session in View
- The N+1 problem and its fixes: `JOIN FETCH`, `@EntityGraph`, batch fetching, projections
- Repositories: derived queries (queries generated from a method's name), `@Query` (JPQL — JPA's object-oriented query language — and native SQL), `@Modifying`, paging and sorting, projections (queries that return only selected fields)
- Auditing (recording who changed what and when), optimistic locking with `@Version`, pessimistic locking (locking the row in the database so others must wait)
- Schema migrations (versioned scripts that evolve the table structure) with Flyway

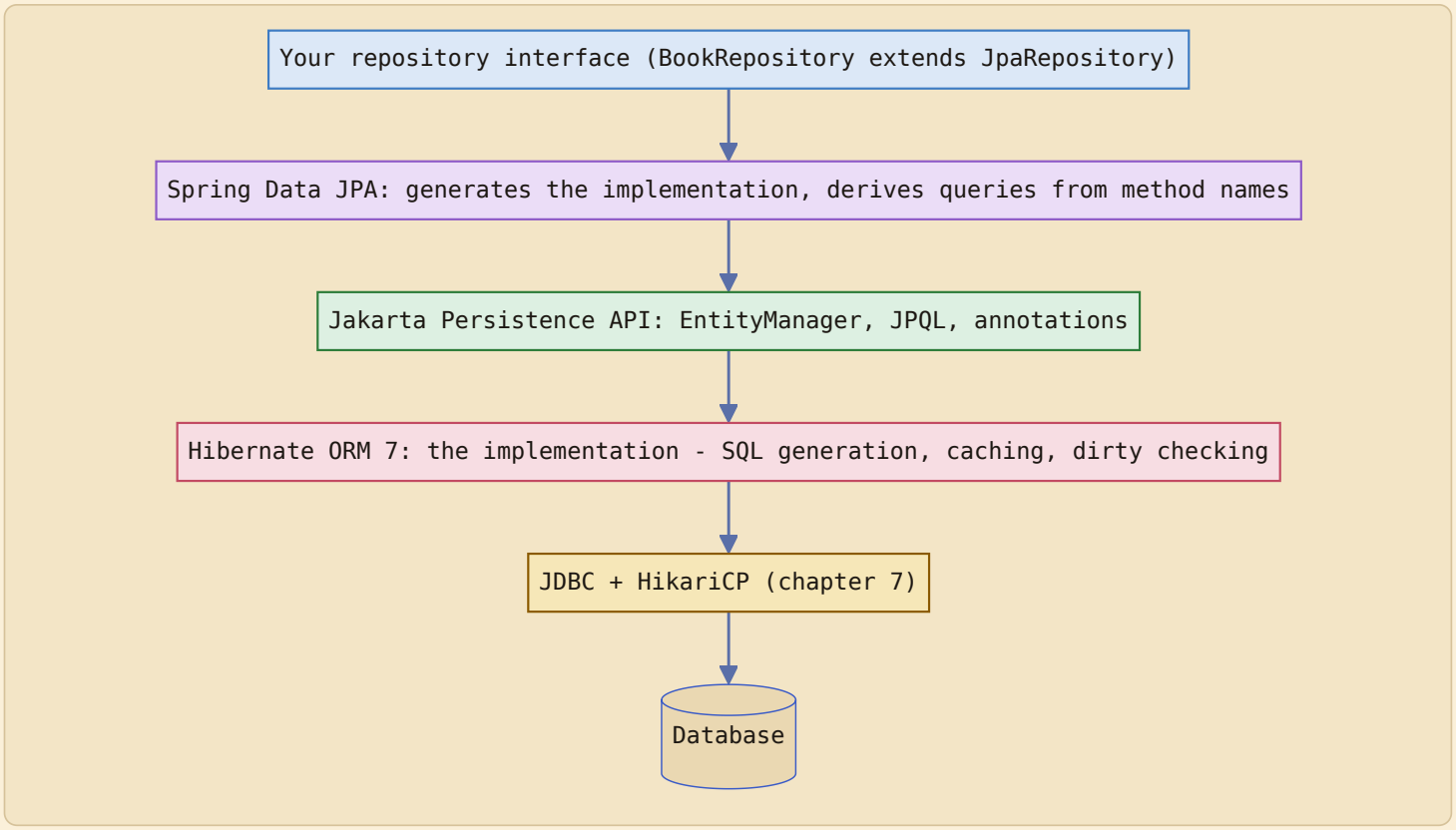
The code (Spring Boot 4.1.1 → Hibernate ORM 7.4, Spring Data 2026.0, H2 — a small Java database — in memory):

Project	What it proves
`code/jpa-demos`	persistence context, relationships, lazy loading, N+1 with statement counts, repositories, projections, auditing, locking, open-in-view
`code/flyway-migrations`	versioned migrations + `ddl-auto=validate`

Run everything with `bash tools_run_spring.sh spring '08-*'`. The sample data is a small library: 4 authors, 8 books, 3 tags, 6 reviews (`src/main/resources/data.sql`).

8.1 JPA, Hibernate and Spring Data — who does what

What it is



Layer	What it is	Version with Boot 4.1
Jakarta Persistence (JPA)	a *specification*: the `jakarta.persistence` annotations, `EntityManager`, the JPQL query language	3.2
Hibernate ORM	the most used *implementation* of JPA. It does the real work	7.4 (Boot 4.1 upgraded to 7.4; Hibernate 7.0 came out in 2025)
Spring Data JPA	a Spring layer on top of JPA: repositories, derived queries, paging, auditing	part of the Spring Data 2026.0 release train (a set of Spring Data modules released together under one version)
Spring ORM	the glue in Spring Framework: `JpaTransactionManager`, the shared `EntityManager` proxy, exception translation (turning vendor errors into Spring's `DataAccessException` family)	Framework 7.0

A few Hibernate 7 facts worth knowing: it implements **Jakarta Persistence 3.2** and **Jakarta Data 1.0**, it needs Java 17 or later, and it is the first Hibernate version licensed under **Apache License 2.0** (a permissive open-source licence; earlier versions were LGPL, a licence with more conditions on distribution). Spring Data 2025.1 (the Boot 4.0 generation) moved to JSpecify null annotations (standard markers for what may be `null`) and added **AOT repositories** (AOT = ahead-of-time), which generate repository code at build time for faster startup and native images (apps compiled to a stand-alone executable that needs no JVM). Spring Data 2026.0 (Boot 4.1) added **type-safe property paths**, for example `Sort.by(Book::getPrice)` instead of `Sort.by("price")`.

When not to use JPA

JPA is a good fit for a rich domain model (classes that model the business and its rules): many related entities, loaded, changed and saved as a unit of work (one business operation whose changes are saved together). It fits poorly for:

- reporting and analytics (big joins; aggregations, i.e. sums/counts over groups; window functions, i.e. SQL calculations over a sliding set of rows): use SQL through `JdbcClient` (7.3) or jOOQ (a library for writing type-safe SQL in Java)
- bulk data processing (millions of rows): use JDBC batches or database-side SQL
- teams who want to see every statement: Spring Data JDBC is simpler and has no lazy loading and no persistence context (ch09)

The general ORM vs raw SQL debate is covered in SQL 14.13.

8.2 Setting up

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>   <!-- Hibernate + Spring Data JPA + HikariCP -->
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>                                              <!-- @DataJpaTest -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
```

```
# jpa-demos/src/main/resources/application.properties
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.defer-datasource-initialization=true          # run data.sql after Hibernate created tables
spring.jpa.properties.hibernate.generate_statistics=true # count statements
spring.jpa.properties.hibernate.session_factory.statement_inspector=ch08.jpa.support.SqlRecorder
```

Anything under `spring.jpa.properties.*` is passed to Hibernate unchanged. That is how you set any Hibernate option Boot has no property for.

`ddl-auto`: who creates the tables?

Value	Hibernate at startup	Use for
`none`	nothing	production with a migration tool
`validate`	checks that tables and columns match the entities, and fails startup if not	production with Flyway/Liquibase (see 8.16)
`update`	adds missing tables and columns, never drops	quick prototypes only. Unreliable for real schema changes
`create`	drops and creates the schema	tests
`create-drop`	like `create`, and drops again at shutdown	tests, demos (this project)

Boot's default is `create-drop` for an embedded database when no migration tool is present, and `none` otherwise.

Seeing the SQL

Option	Shows
`spring.jpa.show-sql=true`	every statement on stdout (the console output), prefixed `Hibernate:` (`@DataJpaTest` turns this on by default)
`logging.level.org.hibernate.SQL=DEBUG`	the same through the logger (better)
`logging.level.org.hibernate.orm.jdbc.bind=TRACE`	the bound parameter values too
`hibernate.generate_statistics=true`	counters: statements, entity loads, collection fetches, flushes
a `StatementInspector`	your own code sees every SQL string, which is used here to print and count

The tests use a small helper built on Hibernate's `Statistics` and the `SqlRecorder` inspector:

```
final class SqlCount {
    private final Statistics stats;
    SqlCount(EntityManagerFactory emf) { stats = emf.unwrap(SessionFactory.class).getStatistics(); }
    void reset()      { stats.clear(); SqlRecorder.clear(); }
    long statements() { return stats.getPrepareStatementCount(); }   // JDBC statements prepared
    void print(String label) { /* label + count + each recorded SQL string */ }
}
```

In your own projects you can get the same effect in tests with the *datasource-proxy* library or Hibernate statistics, and fail a test when a request uses more statements than expected. That turns N+1 bugs into test failures.

8.3 Entities and ids

What it is

An **entity** is a class whose instances are rows of a table. The JPA rules: annotate it with `@Entity`, give it an `@Id`, give it a no-argument constructor (it may be `protected`), and do not make the class or its persistent methods `final`, because Hibernate subclasses entities to build lazy proxies (stand-in objects that load the real data on first use). **Records cannot be entities**, since they are final and immutable (cannot change after creation). Records work well as `@Embeddable` value types (small objects without their own id, stored as columns of the owning table) (Hibernate 6.2+) and as projections (8.12).

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;                // column "title" - names are mapped automatically
    private BigDecimal price;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id")
    private Author author;

    @ManyToMany
    @JoinTable(name = "book_tag",
               joinColumns = @JoinColumn(name = "book_id"),
               inverseJoinColumns = @JoinColumn(name = "tag_id"))
    private Set<Tag> tags = new HashSet<>();

    @Version private Long version;       // optimistic locking, 8.14
    @CreatedDate private Instant createdAt;       // auditing, 8.13
    @LastModifiedDate private Instant updatedAt;
    @CreatedBy private String createdBy;

    protected Book() {}                  // for JPA
    public Book(String title, BigDecimal price) { this.title = title; this.price = price; }
    // getters, a few setters ...
}
```

The chapter's tests switch `show-sql` off, but running one of them with `-Dlogging.level.org.hibernate.SQL=DEBUG` shows the table Hibernate generated at startup. Boot's default naming strategy (the rule that turns Java names into table/column names) turns `createdAt` into `created_at`:

```
create table book (price numeric(38,2), author_id bigint not null, created_at timestamp(6) with time zone,
  id bigint generated by default as identity, updated_at timestamp(6) with time zone, version bigint not null,
  created_by varchar(255), title varchar(255), primary key (id))
```

Useful column annotations: `@Column(nullable = false, length = 100, unique = true)`, `@Column(precision = 10, scale = 2)` (note the default `numeric(38,2)` above), `@Enumerated(EnumType.STRING)` (always use STRING, since ORDINAL — storing the enum's position number — breaks when someone reorders the enum), `@Transient` (not stored), and `@Embedded` for value objects such as `Address`.

Id generation strategies

Strategy	How the id is produced	Trade-off
`IDENTITY`	database auto-increment column (the database assigns the next number on insert)	simple, but Hibernate must `INSERT` **immediately** on `persist` to learn the id, which also rules out JDBC insert batching
`SEQUENCE`	database sequence (a database object that hands out increasing numbers). Hibernate fetches blocks of ids (`allocationSize`, default 50)	best for PostgreSQL/Oracle: inserts can wait until flush and be batched
`AUTO` (default)	Hibernate picks, which in Hibernate 6+ is usually a sequence per entity, `book_seq`	know what you get
`UUID` (JPA 3.1+)	`java.util.UUID` generated in Java	no round trip (no trip to the database and back), safe to generate anywhere. 16 bytes, and random UUIDs (universally unique identifiers) spread index inserts — an index is a lookup structure that speeds up queries, and random keys land all over it
assigned (no `@GeneratedValue`)	you set it	natural keys (ids that come from the real world, like an ISBN). `save()` must then use `merge`, see 8.8

This project uses IDENTITY so `data.sql` can insert rows without ids. You can see the immediate inserts in 8.5.

equals and hashCode

Entities live in `Set`s (which use `hashCode` to place items and `equals` to compare them) and are compared across persistence contexts, so these methods matter:

- The **default** (`Object` identity) is correct inside one persistence context, because Hibernate guarantees one instance per row there (8.4). This project relies on that.
- **Never** base `hashCode` on a generated id that is `null` before persist. The hash changes after saving, and the object gets lost inside a `HashSet`.
- Good options: a natural business key (ISBN), or `equals` on id with a **constant** `hashCode` (`getClass().hashCode()`).
- Never include lazy associations in `equals`, `hashCode` or `toString`. Doing so triggers loading, or a `LazyInitializationException`.

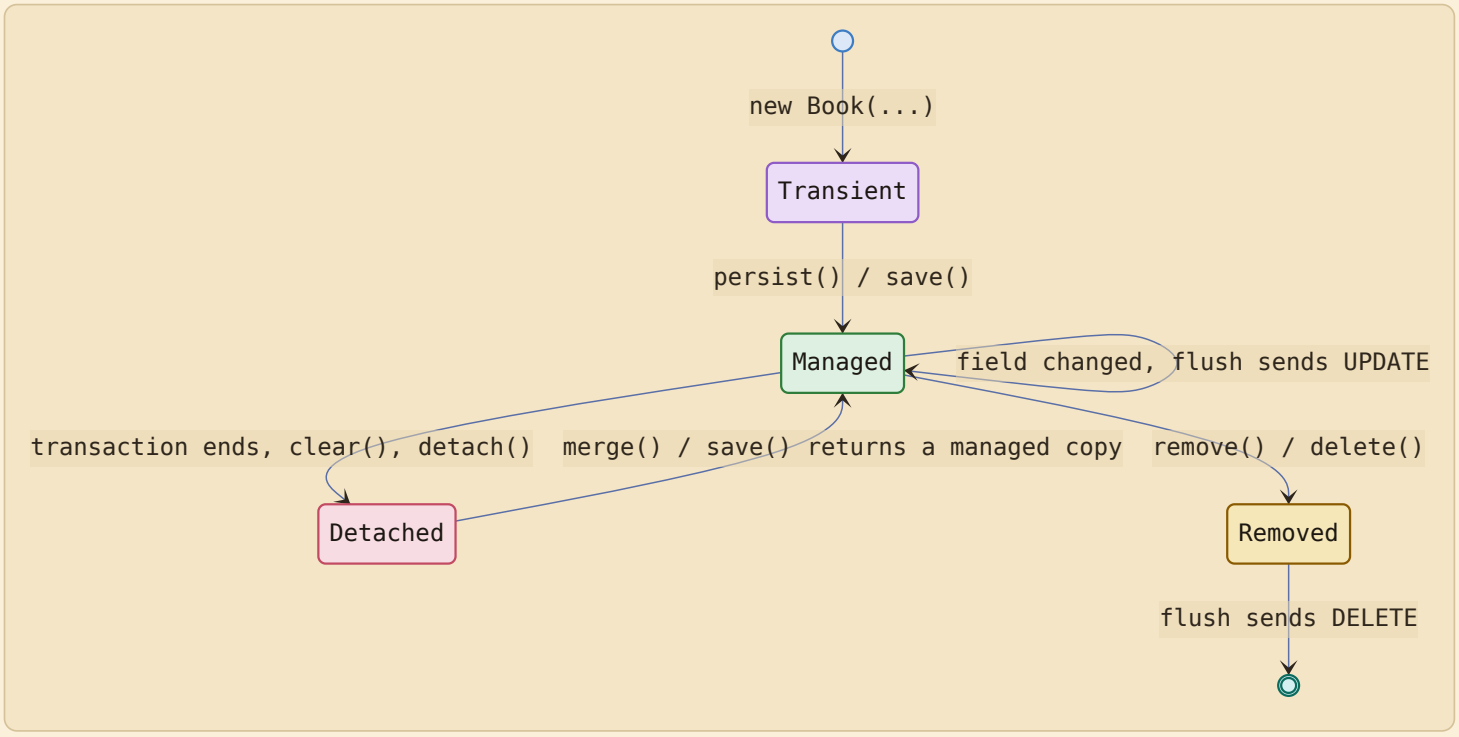
8.4 The persistence context and dirty checking

What it is

The **persistence context** is Hibernate's in-memory workspace for one unit of work. It is the `EntityManager` (Hibernate calls it `Session`). In Spring it usually lives exactly as long as the transaction. It does three jobs:

1. **Identity map / first-level cache**: each row is loaded at most once and always gives the *same* Java object.
2. **Change tracking (dirty checking)**: at load time it keeps a snapshot (a copy of the loaded values) of each entity. At **flush** time it compares every managed entity with its snapshot and writes an `UPDATE` for the ones that changed. You never call `update()`.
3. **Write-behind**: inserts, updates and deletes are queued and sent at flush. Flush happens before commit, before a query that might read the changed tables, or when you call `flush()`.

Entity states



State	Tracked?	Example
Transient	no — Hibernate does not know it	`new Book(...)`
Managed	yes — changes are flushed automatically	anything returned by `find`, a query, or `persist` inside the current transaction
Detached	no — it has an id but no persistence context	an entity returned by a repository call *after* its transaction ended
Removed	scheduled for `DELETE` at flush	after `remove()`

Demo — one row, one object, one SELECT

```
Book first = books.findById(1L).orElseThrow();
Book second = books.findById(1L).orElseThrow();
System.out.println("same Java object? " + (first == second));
```

```
same Java object? true
findById(1) twice -> 1 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version from book b1_0 where
b1_0.id=?
```

The second `findById` came from the persistence context without any SQL. (`@DataJpaTest` wraps each test in one transaction, and so in one persistence context.)

Demo — dirty checking: an UPDATE without save()

```
Book book = books.findById(1L).orElseThrow();
book.setPrice(new BigDecimal("14.99"));   // just change the Java object...
em.flush();                               // ...Hibernate compares with its snapshot and writes SQL
```

```
setPrice() + flush, no save() -> 2 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version from book b1_0 where
b1_0.id=?
    update book set author_id=?,created_at=?,created_by=?,price=?,title=?,updated_at=?,version=? where id=? and version=?
version is now 1
```

In real code the flush happens at commit. `LibraryService.changePrice` is exactly this, and it is tested in 8.13:

```
@Transactional
public void changePrice(Long bookId, BigDecimal newPrice) {
    books.findById(bookId).orElseThrow().setPrice(newPrice);   // no save()
}
```

Notice that the `UPDATE` lists **all** columns. By default Hibernate uses one prepared `UPDATE` (a statement compiled once and reused with different values) per entity type, which it can cache. `@DynamicUpdate` on the entity makes it send only the changed columns instead.

Demo — detached entities are not tracked

```
Book book = books.findById(1L).orElseThrow();
em.clear();                               // persistence context emptied: book is DETACHED
book.setPrice(new BigDecimal("1.00"));
em.flush();
```

```
change a detached entity + flush -> 1 SQL statement(s)
    select ... from book b1_0 where b1_0.id=?
```

No `UPDATE` was sent: the change is simply lost. To store changes to a detached entity you call `save()` (which calls `merge`). Merge loads the current row, copies your state onto the managed copy, and returns that copy. Use the return value.

readOnly transactions and JPA

`@Transactional(readOnly = true)` tells Spring to put the Hibernate session in read-only mode with manual flushing. Hibernate then keeps no snapshots and never runs dirty checking, which saves memory and CPU when you load many entities just to display them. Spring Data's `SimpleJpaRepository` is annotated `@Transactional(readOnly = true)` at class level and plain `@Transactional` on its write methods.

Common mistakes

- Calling `save()` on an entity you just loaded in the same transaction. It is harmless but unnecessary, because the entity is managed already.
- Ignoring the result of `save()`/`merge()` for a detached entity, then modifying the old detached object.
- Loading 100 000 entities in one transaction. The persistence context keeps all of them *and* their snapshots. Process in pages and call `clear()`, or use a read-only transaction, projections or JDBC.

In the interview

30-second answer: "The persistence context is the `EntityManager`'s first-level cache and unit of work. It returns the same object for the same row, keeps a snapshot of every managed entity, and at flush it writes `UPDATE`s for entities that changed. That is dirty checking, and it is why you don't need `save()` for loaded entities. Entities are transient, managed, detached or removed. Only managed ones are tracked."

- *When does Hibernate flush?* → Before commit, before a query that touches affected tables (AUTO flush mode), and on an explicit `flush()`.
- *`persist` vs `merge`?* → `persist` makes a transient object managed (the same instance). `merge` copies a detached object's state onto a managed copy and returns the copy.

Self-check

1. How many SELECTs does `findById(1)` twice cost in one transaction? → One.

2. Why was no UPDATE sent after `em.clear()`? → The entity was detached, so nothing tracked it.
3. What does `readOnly = true` save with Hibernate? → Snapshots and dirty checking, and flushes.

8.5 Relationships

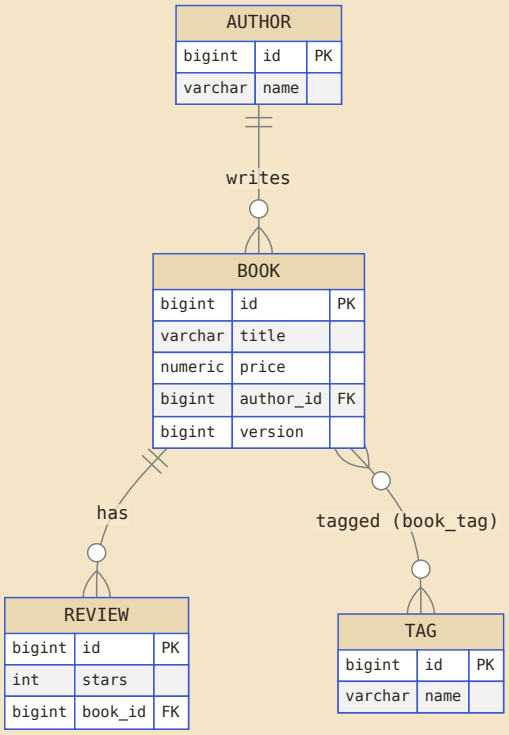
What it is

Annotation	Meaning	Default fetch	Example
`@ManyToOne`	many of these point to one of those	**EAGER**	`Book.author`
`@OneToMany`	one of these has a collection of those	LAZY	`Author.books`
`@OneToOne`	exactly one on each side	**EAGER**	`User.profile`
`@ManyToMany`	join table between the two	LAZY	`Book.tags`

The owning side and mappedBy

In the database a relationship is **one foreign-key column** (a column holding the id of a row in another table) or a join table (an extra table whose rows link two tables). In Java it may be navigable from both sides, but only one side may control that column. That side is the **owning side**:

- For `@ManyToOne` / `@OneToMany` the owner is the **many** side (`Book.author` → `book.author_id`).
- The other side says `mappedBy = "author"`, meaning "I am just a mirror of `Book.author`". This is the **inverse side**.
- JPA **only reads the owning side** when it writes SQL. Adding a book to `author.getBooks()` without setting `book.author` stores nothing.



That is why `Author` has helper methods that keep both sides in sync:

```
@Entity
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private String name;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book)    { books.add(book);    book.setAuthor(this); }
    public void removeBook(Book book) { books.remove(book); book.setAuthor(null); }
    // ...
}
```

Cascades

A **cascade** passes an operation from parent to children:

CascadeType	Passes on
`PERSIST`	saving a new parent also saves new children
`MERGE`	merging the parent merges the children
`REMOVE`	deleting the parent deletes the children
`REFRESH`, `DETACH`	reload / detach together
`ALL`	all of the above

Use cascades only from an **aggregate root** (the main object of a group that is always saved together, e.g. an order with its lines) to the parts it owns (author → books in this demo, order → order lines in real life). Never cascade `REMOVE` on `@ManyToMany` or towards shared entities. Deleting a book must not delete the "sci-fi" tag that other books use.

```
Author author = new Author("N. K. Jemisin");
author.addBook(new Book("The Fifth Season", new BigDecimal("11.00")));
author.addBook(new Book("The Obelisk Gate", new BigDecimal("11.00")));
authors.save(author);                       // persist cascades to both books
```

```
save(new author with 2 new books) -> 3 SQL statement(s)
    insert into author (name,id) values (?,default)
    insert into book (author_id,created_at,created_by,price,title,updated_at,version,id) values (?,?,?,?,?,?,?,default)
    insert into book (author_id,created_at,created_by,price,title,updated_at,version,id) values (?,?,?,?,?,?,?,default)
ids assigned: author 5, books [9, 10]
```

The inserts ran **during** `save()`, before any flush. That is the IDENTITY strategy at work (8.3).

orphanRemoval

`orphanRemoval = true` means *"a child removed from the collection should be deleted"*. `CascadeType.REMOVE` only deletes children when the *parent* is deleted.

```
Author pratchett = authors.findById(2L).orElseThrow();
Book smallGods = pratchett.getBooks().stream()
        .filter(b -> b.getTitle().equals("Small Gods")).findFirst().orElseThrow();
pratchett.removeBook(smallGods);            // no repository.delete() call
em.flush();
```

```
remove 'Small Gods' from author.books + flush -> 4 SQL statement(s)
    select a1_0.id,a1_0.name from author a1_0 where a1_0.id=?
    select b1_0.author_id,b1_0.id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version from book b1_0 where
b1_0.author_id=?
    delete from book_tag where book_id=?
    delete from book where id=? and version=?
```

Hibernate first removed the book's rows from the `book_tag` join table (Book owns that many-to-many), then deleted the book. The first version of this demo removed a book that had **reviews**, and the database rejected the delete with a foreign-key violation (`REVIEW FOREIGN KEY(BOOK_ID) REFERENCES BOOK(ID)`). Hibernate only cleans up relationships it knows it owns. `Review.book` points at the book from the other direction, so you must deal with those rows yourself.

Many-to-many advice

A plain `@ManyToMany` suits a pure link with no data of its own. As soon as the link needs data ("tagged by whom, when?"), turn the join table into an **entity** (`BookTag` with two `@ManyToOne`s). Use `Set`, not `List`, for many-to-many. Hibernate handles `List` without an order column (a "bag") inefficiently: removing one element deletes all the link rows and re-inserts the rest.

Common mistakes

- Setting only the inverse side (`author.getBooks().add(book)`), so nothing is saved.
- `cascade = ALL` everywhere, which deletes shared data.
- Bidirectional mappings (navigable from both sides) everywhere. Map the reverse side only when you really navigate it.
- `@OneToMany` without `mappedBy`. Hibernate then creates an extra join table.

In the interview

30-second answer: "The owning side is the one holding the foreign key, in a one-to-many that is the `@ManyToOne` side, and it is the only side JPA reads when writing. The other side uses `mappedBy`. Keep both sides in sync with helper methods. Cascade from aggregate roots only. `orphanRemoval` deletes children removed from the collection, while `CascadeType.REMOVE` deletes children when the parent is deleted."

Self-check

1. Which side owns `Author.books` ↔ `Book.author`? → `Book.author` (the `author_id` column).
2. Why were two DELETEs needed to remove one book? → The `book_tag` link rows go first, then the book row.
3. `orphanRemoval` vs `CascadeType.REMOVE`? → Removal from the collection vs deletion of the parent.

8.6 Fetch types and LazyInitializationException

What it is

- **LAZY**: the related data is loaded only when you first touch it. For a collection, Hibernate puts a special `PersistentBag` / `PersistentSet` in the field. For a single entity it puts a generated **proxy** subclass there.
- **EAGER**: the related data is loaded together with the owner, always.

Make everything LAZY and decide per use case what to load (see 8.7). The JPA defaults are EAGER for `@ManyToOne` and `@OneToOne`, so change them explicitly: `@ManyToOne(fetch = FetchType.LAZY)`. EAGER cannot be switched off for a single query, while LAZY can always be fetched eagerly when you need it.

LazyInitializationException

A lazy collection can only load itself while its persistence context is open. Outside a transaction, a repository call opens and closes its own short transaction. The entity it returns is already **detached**:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)   // no test transaction
class LazyLoadingTest {
    @Test
    void lazyCollectionOutsideTheTransaction() {
        Author a = authors.findById(2L).orElseThrow();   // transaction opens and closes inside findById
        assertThatThrownBy(() -> a.getBooks().size())
                .isInstanceOf(LazyInitializationException.class);
    }
}
```

```
LazyInitializationException: Cannot lazily initialize collection of role 'ch08.jpa.Author.books' with key '2' (no session)
```

"No session" means no open persistence context. The fixes:

```
@EntityGraph(attributePaths = "books")
Optional<Author> findWithBooksById(Long id);       // load the books in the same query

@Transactional(readOnly = true)                     // or: do the work inside a transaction
public AuthorSummary summary(Long authorId) {
    Author a = authors.findById(authorId).orElseThrow();
    return new AuthorSummary(a.getName(), a.getBooks().size());   // return a DTO, not the entity
}
```

```
fetched with @EntityGraph -> 3 books
inside @Transactional service -> AuthorSummary[name=Terry Pratchett, books=3]
```

Fix	Good?
fetch what the use case needs (JOIN FETCH / `@EntityGraph`)	✅ best
do the work in a `@Transactional` service and return a DTO	✅ best
a DTO projection query (8.12)	✅ for read-only views
`spring.jpa.open-in-view=true`	⚠️ hides the problem, see 8.15
switch the mapping to EAGER	❌ every query pays for it
`hibernate.enable_lazy_load_no_trans=true`	❌ each lazy load opens its own mini-transaction: N+1 with no consistency

---

8.7 The N+1 problem

What it is

You run **1** query to load N parents. Then, for each parent, touching a lazy association fires **1 more** query. With N = 1000 that is 1001 round trips for data that one query could return. It is the most common JPA performance bug, and a functional test (one that only checks the result is correct) cannot see it, because the result is correct, only slow.

Demo — watch it happen

```
List<Author> all = authors.findAll();
all.forEach(a -> System.out.println("  " + a.getName() + ": " + a.getBooks().size() + " books"));
```

```
  Ursula K. Le Guin: 2 books
  Terry Pratchett: 3 books
  Octavia E. Butler: 1 books
  Isaac Asimov: 2 books
findAll() then a.getBooks().size() -> 5 SQL statement(s)
    select a1_0.id,a1_0.name from author a1_0
    select b1_0.author_id,b1_0.id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version from book b1_0 where
b1_0.author_id=?
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id=?
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id=?
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id=?
```

`1 + 4 = 5` statements. The test asserts `statements() == 1 + all.size()`.

EAGER does not fix it — demo

`Review.book` was left at the JPA default (`@ManyToOne` = EAGER). We never touch `review.getBook()`:

```
var all = reviews.findAll();
```

```
reviews.findAll() with EAGER @ManyToOne Review.book (6 reviews) -> 6 SQL statement(s)
    select r1_0.id,r1_0.book_id,r1_0.stars from review r1_0
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
```

1 query for the reviews, then 1 per distinct book (5), **even though the code never used the books**. EAGER tells Hibernate that the association must be loaded. It does not say *how*. For JPQL and derived queries Hibernate runs the query as written, then loads each eager association separately.

Fix 1 — JOIN FETCH

```
@Query("select a from Author a join fetch a.books order by a.id")
List<Author> findAllWithBooks();
```

```
JOIN FETCH -> 1 SQL statement(s)
    select a1_0.id,b1_0.author_id,b1_0.id,...,a1_0.name from author a1_0 join book b1_0 on a1_0.id=b1_0.author_id order by a1_0.id
```

Hibernate 6+ removes the duplicate parent rows that a join produces on its own, so `distinct` is no longer needed. Note that it is an **inner** join (keeps only rows that have a match on both sides): authors with no books would disappear from the result. Use `left join fetch` (a left join keeps every row of the left table, even without a match) to keep them.

Fix 2 — @EntityGraph

```
@EntityGraph(attributePaths = "books")
@Query("select a from Author a order by a.id")
List<Author> findAllWithBooksGraph();
```

```
@EntityGraph(attributePaths = "books") -> 1 SQL statement(s)
    select a1_0.id,b1_0.author_id,b1_0.id,...,a1_0.name from author a1_0 left join book b1_0 on a1_0.id=b1_0.author_id order by a1_0.id
```

The effect is the same, but it is a **left** join and needs no JPQL. It also works on derived query methods (`findWithBooksById` above). Entity graphs can also be declared on the entity with `@NamedEntityGraph`.

Fix 3 — batch fetching

Same code as the N+1 demo, one property added:

```
@DataJpaTest(showSql = false, properties = "spring.jpa.properties.hibernate.default_batch_fetch_size=16")
```

```
findAll() + books with default_batch_fetch_size=16 -> 2 SQL statement(s)
    select a1_0.id,a1_0.name from author a1_0
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id in (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
```

When the first lazy collection is touched, Hibernate loads the collections of up to 16 authors in the persistence context at once. N+1 becomes `1 + ⌈N/16⌉`. It needs no query changes, which makes it a good **global safety net**. Per association, use `@BatchSize(size = 16)`.

Paging a JOIN FETCH (Hibernate 7.4)

Paging and fetching a collection in the same query was a classic trap. `LIMIT 2` on a joined result limits *rows*, not authors, so older Hibernate versions loaded **everything** and paged in memory, with the warning `HHH90003004: firstResult/maxResults specified with collection fetch; applying in memory`. Hibernate 7.4 pages the parents in a subquery instead. Our test, run on Boot 4.1.1:

```
@Query(value = "select a from Author a join fetch a.books",
       countQuery = "select count(a) from Author a")
Page<Author> pageWithBooks(Pageable pageable);

authors.pageWithBooks(PageRequest.of(0, 2, Sort.by("id")));
```

```
JOIN FETCH + Pageable(page 0, size 2) -> 2 SQL statement(s)
    select a1_0.id,b1_0.author_id,...,a1_0.name from (select a1_0.id,a1_0.name from author a1_0 where exists(select 1 from book b1_0 where
a1_0.id=b1_0.author_id) order by a1_0.id offset ? rows fetch first ? rows only) a1_0(id,name) join book b1_0 on a1_0.id=b1_0.author_id order
by a1_0.id
    select count(a1_0.id) from author a1_0
```

The inner subquery picks 2 authors with `offset/fetch first`, and the outer query joins their books. On older Hibernate versions, set `hibernate.query.fail_on_pagination_over_collection_fetch=true` so the in-memory case fails loudly. Then page the ids first and fetch the collections in a second query (or rely on batch fetching).

Choosing a fix

Fix	Queries	Good for	Watch out
`JOIN FETCH`	1	one known use case	inner join by default. Fetching **two** `List` collections at once → `MultipleBagFetchException` (use `Set`, or two queries)
`@EntityGraph`	1	the same, declared on a repository method	same cartesian-product (every row combined with every other row) risk with several collections
batch size	1 + N/size	global default, several collections	still more than one query
DTO projection	1	read-only screens and APIs	no entities, so no dirty checking (usually what you want)

Fetching two collections in one query multiplies rows: 10 books × 10 tags = 100 rows per author. For several collections, prefer batch fetching or separate queries.

Common mistakes

- Returning entities from a REST controller. Jackson walks every getter, triggering lazy loads (N+1) or `LazyInitializationException`, and exposes your schema. Return DTOs.
- Using `toString()`/Lombok `@Data` (Lombok is a library that generates getters, `equals`, `toString` and more) on entities with associations. Printing loads them.
- Testing with 3 rows. N+1 only hurts with real volumes, so assert statement counts in tests instead.

In the interview

30-second answer: "N+1 is one query for the parents plus one per parent when a lazy association is touched, or even untouched with EAGER to-one mappings. I find it by counting statements with Hibernate statistics or a datasource proxy (a wrapper around the `DataSource` that sees every SQL statement). I fix it per use case with `JOIN FETCH` or an `@EntityGraph`, globally reduce it with `default_batch_fetch_size`, or avoid entities with a DTO projection. EAGER is not a fix, it makes the problem permanent."

- *Why doesn't EAGER prevent N+1?* → EAGER says *whether* to load, not *how*. For queries Hibernate loads the associations with separate selects after the main query.
- *JOIN FETCH with pagination?* → Historically in-memory pagination (HHH90003004). Hibernate 7.4 pages parents in a subquery. Otherwise page ids first, then fetch.
- *`MultipleBagFetchException`?* → Two `List` collections fetched in one query. Use `Set`, or fetch them in separate queries.

Self-check

1. How many statements does `findAll()` + touching books cost for 4 authors without a fix? With batch size 16? → 5, and 2.
2. Which fix keeps authors that have no books? → `@EntityGraph` (left join), or `left join fetch`.
3. Why did `reviews.findAll()` run 6 statements? → EAGER `@ManyToOne` loaded each distinct book separately.

TL;DR

Mappings LAZY. Decide what to fetch per query with JOIN FETCH / `@EntityGraph`. `default_batch_fetch_size` as a safety net. DTO projections for read views. Count statements in tests.

8.8 Repositories

What it is

A **repository** is an interface. At startup Spring Data creates a proxy that implements it, using `SimpleJpaRepository` for the standard methods and generated query code for your own ones.

```
public interface BookRepository extends JpaRepository<Book, Long> { ... }
```

Interface	Adds
`Repository<T, ID>`	marker only. Declare exactly the methods you want
`CrudRepository` / `ListCrudRepository`	`save`, `findById`, `findAll`, `count`, `delete...` (the List variant returns `List` instead of `Iterable`)
`PagingAndSortingRepository` / `ListPagingAndSortingRepository`	`findAll(Sort)`, `findAll(Pageable)`
`JpaRepository`	all of the above + `flush`, `saveAndFlush`, `deleteAllInBatch`, `getReferenceById`, ...
`JpaSpecificationExecutor<T>`	dynamic `Specification` (criteria — conditions built in code) queries

Expose only what callers should use. A `Repository` with a handful of methods is often better than handing out `deleteAll()`.

How save() decides between INSERT and UPDATE

`save(entity)` asks "is this entity new?". If yes, it calls `persist` (the same instance becomes managed). If not, it calls `merge` (it returns a **different**, managed instance). An entity counts as new when its id is `null`, or when a wrapper-type `@Version` field is `null`. With **assigned ids** every entity has an id, so `save` runs `merge`, which first `SELECT`s the row. Implement `Persistable<ID>` to tell Spring Data yourself.

Transactions in repositories

`SimpleJpaRepository` methods are transactional: read methods `readOnly = true`, write methods read-write. Each call **outside** a service transaction is its own tiny transaction, which leads straight to the "wrong layer" bug of 7.13. Put `@Transactional` on the service method so that all repository calls join one transaction.

8.9 Derived queries

The method name *is* the query. Spring Data parses it at startup, so a typo fails startup, not a request:

```
List<Book> findByTitleContainingIgnoreCase(String part);
List<Book> findByPriceLessThanOrderByPriceAsc(BigDecimal max);
List<Book> findByAuthorName(String authorName);          // traverses book.author.name (a join)
long countByAuthorName(String authorName);
boolean existsByTitle(String title);
List<Book> findTop2ByOrderByPriceDesc();
List<Book> findByTagsName(String tag, Sort sort);        // many-to-many join + dynamic sort
```

```
containing 'go'      : [Small Gods, Going Postal]
price < 10 asc       : [I, Robot, Foundation, Guards! Guards!, A Wizard of Earthsea]
by author name       : [A Wizard of Earthsea, The Dispossessed]
count by author      : 3
exists 'Kindred'     : true
top 2 by price       : [Kindred, The Dispossessed]
tag sci-fi, title asc: [Foundation, I, Robot, Kindred, The Dispossessed]
tag fantasy, by price: [Guards! Guards!, A Wizard of Earthsea, Small Gods]
```

The last line used the Spring Data 2026.0 type-safe form `Sort.by(Book::getPrice)`. A renamed getter is then a compile error rather than a runtime one. The SQL Spring Data generated for some of them:

```
containing / count / exists -> 3 SQL statement(s)
    select b1_0.id,...,b1_0.version from book b1_0 where upper(b1_0.title) like upper(?) escape '\'
    select count(b1_0.id) from book b1_0 join author a1_0 on a1_0.id=b1_0.author_id where a1_0.name=?
    select b1_0.id from book b1_0 where b1_0.title=? fetch first ? rows only
```

`existsBy` fetches just one id. Note also that `upper(title) like` cannot use a normal index (a sorted lookup structure for a column) on `title` (SQL 12.13).

Keywords

Keyword	Example	SQL idea
`And`, `Or`	`findByTitleAndPrice`	`title = ? and price = ?`
`Is`, `Equals`, `Not`	`findByTitleNot`	`title <> ?`
`LessThan`, `GreaterThanEqual`, `Between`	`findByPriceBetween(a, b)`	`price between ? and ?`
`IsNull`, `IsNotNull`	`findByAuthorIsNull`	`author_id is null`
`Like`, `StartingWith`, `EndingWith`, `Containing`	`findByTitleStartingWith`	`like 'x%'`
`IgnoreCase`	`findByTitleIgnoreCase`	`upper(title) = upper(?)`
`In`, `NotIn`	`findByIdIn(List)`	`id in (...)`
`True`, `False`	`findByActiveTrue`	`active = true`
`OrderBy...Asc/Desc`	`findByAuthorNameOrderByTitleDesc`	`order by`
`First`, `Top`, `TopN`, `Distinct`	`findFirst3ByOrderByPriceAsc`	`fetch first 3 rows`
nested property	`findByAuthorName`	join to `author`
prefixes	`count...By`, `exists...By`, `delete...By`	count / exists / load-then-delete

Special parameters: `Sort`, `Pageable`, `Limit` (Spring Data 3.2+), `ScrollPosition` (keyset scrolling, 3.1+). Return types: `List`, `Optional`, `Stream` (close it), `Page`, `Slice`, `Window`, a single entity, or a projection.

When not to use derived queries: when the name grows past two or three conditions (`findByAuthorNameAndPriceLessThanAndTagsNameOrderBy...`). Switch to `@Query`.

8.10 @Query: JPQL, native SQL and @Modifying

JPQL

JPQL (Jakarta Persistence Query Language) looks like SQL but talks about **entities and fields**, not tables and columns: `b.author.name` instead of a join written by hand.

```
@Query("select b from Book b where b.author.name = :name and b.price <= :max order by b.price")
List<Book> byAuthorUpTo(@Param("name") String name, @Param("max") BigDecimal max);
```

```
JPQL  Pratchett <= 11 : [Guards! Guards!, Small Gods]
```

(`@Param` can be omitted when you compile with `-parameters`, which the Boot parent POM enables.)

Native SQL

```
@Query(value = "SELECT title FROM book WHERE price > ?1 ORDER BY title", nativeQuery = true)
List<String> titlesAbove(BigDecimal min);
```

```
native titles > 11    : [Going Postal, Kindred, The Dispossessed]
```

Use native queries for database features JPQL lacks (window functions; operators on `jsonb`, PostgreSQL's JSON column type; CTEs, i.e. `WITH` clauses that name a sub-query; `ON CONFLICT`, PostgreSQL's insert-or-update). The price is that the query is tied to one database and to table and column names.

Bulk updates with @Modifying

```
@Modifying(clearAutomatically = true) // bulk UPDATE bypasses the persistence context -> clear it
@Query("update Book b set b.price = b.price * :factor where b.author.id = :authorId")
int reprice(@Param("authorId") Long authorId, @Param("factor") BigDecimal factor);
```

```
@Modifying bulk update -> 1 SQL statement(s)
    update book b1_0 set price=(b1_0.price*cast(? as numeric(38,2))) where b1_0.author_id=?
bulk update rows      : 2 -> Asimov now [BookRow[title=Foundation, price=15.98], BookRow[title=I, Robot, price=13.98]]
```

A bulk statement goes straight to the database. Entities already loaded in the persistence context keep their **old** values, and `@Version` is **not** incremented. `clearAutomatically = true` empties the context afterwards, and `flushAutomatically = true` flushes pending changes before the statement runs. `@Modifying` methods need a read-write transaction.

Dynamic queries

For search screens with optional filters, write `Specification`s (`JpaSpecificationExecutor`), Query by Example (pass a sample object; its non-null fields become filters), or Querydsl (a library for type-safe queries in Java code), instead of dozens of derived methods. All three build the query at runtime from the filters that are present.

Common mistakes

- Building JPQL with string concatenation. That is injection again: always use parameters.
- Forgetting that JPQL uses **entity** names (`Book`, `b.author.name`), not table names.
- Bulk updates followed by reading "stale" entities from the same persistence context.

8.11 Pagination and sorting

```
Page<Book> findByPriceGreaterThan(BigDecimal min, Pageable pageable);   // also runs a COUNT query
Slice<Book> findByAuthorIdIn(List<Long> authorIds, Pageable pageable);  // no COUNT, fetches size+1 rows

Page<Book> page = books.findByPriceGreaterThan(new BigDecimal("8"),
        PageRequest.of(1, 2, Sort.by("price").descending()));     // page index 1 = the second page
Slice<Book> slice = books.findByAuthorIdIn(List.of(1L, 2L), PageRequest.of(0, 3, Sort.by("id")));
```

```
Page 1 of 3 (6 rows total): [Going Postal, Small Gods]
Slice 0: [A Wizard of Earthsea, The Dispossessed, Guards! Guards!], hasNext=true
Page + Slice -> 3 SQL statement(s)
    select b1_0.id,...,b1_0.version from book b1_0 where b1_0.price>? order by b1_0.price desc offset ? rows fetch first ? rows only
    select count(*) from book b1_0 where b1_0.price>?
    select b1_0.id,...,b1_0.version from book b1_0 where b1_0.author_id in (?,?) order by b1_0.id fetch first ? rows only
```

(Column lists shortened with `...` here and in a few other listings.) The `Slice` query asks for `size + 1` rows: if the extra row arrives, `hasNext()` is true.

Return type	Extra query	Knows	Use for
`Page<T>`	`COUNT(*)`	total elements and pages	UIs with page numbers
`Slice<T>`	none (fetches one extra row)	whether there is a next slice	"load more" / infinite scroll (more items load as the user scrolls down)
`Window<T>` + `ScrollPosition`	none	next position	keyset (cursor) scrolling over big tables
`List<T>` + `Limit`	none	nothing	"top N"

Pages are numbered from **0**. `Page` and `Slice` use `OFFSET`, which gets slower the deeper you page, because the database still reads and throws away the skipped rows. For large tables use keyset pagination (`Window` with `ScrollPosition.keyset()`). The reasoning is in SQL 12.14 — OFFSET vs keyset pagination. Always sort by something unique, such as `id` as a tie-breaker, or rows can repeat or vanish between pages.

In Spring MVC, a `Pageable` controller parameter is filled from `?page=0&size=20&sort=price,desc` (ch05).

8.12 Projections

What it is

A **projection** returns only the fields you need, as an interface or a record, not the whole entity. There is no persistence context overhead, no dirty checking and no lazy loading surprises, and the SELECT names fewer columns.

```
public interface BookSummary {          // interface projection (closed: only plain getters)
    String getTitle();
    BigDecimal getPrice();
}
public record BookRow(String title, BigDecimal price) {}   // class-based (DTO) projection

List<BookSummary> findByAuthorNameOrderByTitle(String authorName);      // interface
List<BookRow> findByPriceLessThan(BigDecimal max);                       // record
@Query("select new ch08.jpa.BookRow(b.title, b.price) from Book b where b.author.id = :id order by b.title")
List<BookRow> rowsForAuthor(@Param("id") Long authorId);                  // JPQL constructor expression
<T> List<T> findByAuthorId(Long authorId, Class<T> type);                // dynamic: caller picks the type
```

```
interface projection SQL -> 1 SQL statement(s)
    select b1_0.title,b1_0.price from book b1_0 join author a1_0 on a1_0.id=b1_0.author_id where a1_0.name=? order by b1_0.title
interface projection: Going Postal 11.50
interface projection: Guards! Guards! 8.99
interface projection: Small Gods 10.99
record projection     : [BookRow[title=Foundation, price=7.99], BookRow[title=I, Robot, price=6.99]]
constructor expression: [BookRow[title=A Wizard of Earthsea, price=9.99], BookRow[title=The Dispossessed, price=12.50]]
dynamic (BookSummary) : Kindred
```

Only `title` and `price` were selected, compared with eight columns for the entity.

Kind	How	Notes
closed interface	getters that match properties	optimized SELECT of just those columns. Nested interfaces for associations
open interface	`@Value("#{target.title + ' by ' + target.author.name}")`	loads the full entity, so no SELECT optimization
class / record (DTO)	constructor parameters match properties	derived queries select only those columns. With `@Query` use `select new ...`
dynamic	`<T> List<T> find...(..., Class<T> type)`	one method, several shapes
native query	interface projection with column aliases	`select title as title, ...`

Rule of thumb: load entities when you are going to *change* them, and use projections when you are going to *show* them.

8.13 Auditing

What it is

Auditing fills "who and when" columns automatically: `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`.

```
@SpringBootApplication
@EnableJpaAuditing                      // switch it on
public class JpaDemosApplication {
    @Bean
    AuditorAware<String> auditor() {     // "who is the current user?"
        return () -> Optional.of("system");   // real app: read it from Spring Security's context
    }
}

@Entity
@EntityListeners(AuditingEntityListener.class)   // on each audited entity
public class Book { @CreatedDate Instant createdAt; @LastModifiedDate Instant updatedAt; @CreatedBy String createdBy; ... }
```

This test runs without a test transaction, so each step commits:

```
Book b = books.findById(bookId).orElseThrow();
library.changePrice(bookId, new BigDecimal("10.50"));   // @Transactional, dirty checking, no save()
Book after = books.findById(bookId).orElseThrow();
```

```
created: by=system at=true version=0
after changePrice: price=10.50 version=1 updatedAt moved? true
```

The auditing listener runs on the JPA `@PrePersist` / `@PreUpdate` callbacks, so it only sees changes that go through the persistence context. Bulk `@Modifying` updates bypass it. For a full history of every change use Hibernate Envers (`@Audited`, revision tables — extra tables holding every past version of a row) or database-level change data capture (reading the database's own change log to stream every change elsewhere).

8.14 Locking: optimistic and pessimistic

Optimistic locking with @Version

The problem: Alice and Bob both open the edit page for the same book, which is version 0. Alice saves. Bob saves a few seconds later and silently overwrites Alice's change. This is the *lost update* from 7.11 and SQL 13.7, now spread over two HTTP requests, where no database lock can help.

How it works:

1. Add a `@Version` field (`Long` / `Integer`, or a timestamp).
2. Every `UPDATE` becomes `... SET ..., version = version + 1 WHERE id = ? AND version = ?`, using the version the entity was loaded with.

3. If another transaction changed the row in the meantime, **0 rows** match. Hibernate reports it, and Spring throws `ObjectOptimisticLockingFailureException` (an `OptimisticLockingFailureException`).

```
Book aliceCopy = books.findById(bookId).orElseThrow();   // two users load version 0
Book bobCopy = books.findById(bookId).orElseThrow();

aliceCopy.setPrice(new BigDecimal("8.00"));
books.save(aliceCopy);                                   // merge -> UPDATE ... WHERE id=? AND version=0

bobCopy.setPrice(new BigDecimal("12.00"));
books.save(bobCopy);                                     // still version 0 -> 0 rows -> exception
```

```
both loaded version 0 and 0
Alice saves -> 2 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version from book b1_0 where
b1_0.id=?
    update book set author_id=?,created_at=?,created_by=?,price=?,title=?,updated_at=?,version=? where id=? and version=?
Bob saves -> ObjectOptimisticLockingFailureException: Row was already updated or deleted by another transaction for entity [ch08.jpa.Book
with id '10']
price in DB = 8.00 (Alice's)
```

Alice's `save` is a `merge` of a detached entity, so it first `SELECT`s the current row, then updates with the version check. For Bob the merge found version 1 in the database, while his object said 0, so it failed.

Handling it: tell the user "someone else changed this, reload", which a REST API reports as `409 Conflict`. Or reload and retry automatically when the change is safe to reapply. In REST APIs, send the version to the client, for example as an `ETag` (an HTTP header identifying one version of a resource), and require it back in `If-Match` (a request header meaning "only apply if the current version is still this one") (ch05).

Pessimistic locking

Lock the row in the database for the rest of the transaction:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Book> findWithLockById(Long id);
```

```
@Lock(PESSIMISTIC_WRITE) -> 1 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version from book b1_0 where
b1_0.id=? for update
```

Other transactions that want the same row wait until this one commits. Set a lock timeout (the `jakarta.persistence.lock.timeout` query hint) so they do not wait forever. How `FOR UPDATE`, `NOWAIT` and `SKIP LOCKED` behave is covered in SQL 13.12.

	Optimistic (`@Version`)	Pessimistic (`@Lock`)
Conflict detected	at write time (0 rows updated)	prevented. Others wait
Cost	none until a conflict	locks held, possible deadlocks and waiting
Works across requests / user think time	✅	❌ (a lock lives only as long as its transaction)
Best when	conflicts are rare (most web apps)	conflicts are frequent and short (counters, stock, job queues — tables of pending work picked up by workers)

In the interview

30-second answer: "Optimistic locking adds a version column. Updates include `WHERE version = ?`, and if another transaction got there first, zero rows match and Spring throws `ObjectOptimisticLockingFailureException`, which I turn into a 409 or a retry. It holds no locks, so it works across user think time. Pessimistic locking uses `SELECT ... FOR UPDATE` through `@Lock(PESSIMISTIC_WRITE)` and makes others wait. I use it for short, high-contention updates."

8.15 Open Session in View

What it is

Open Session in View (OSIV) keeps the persistence context open for the **whole web request**, including controller code and JSON serialization, not just during the service transaction. Boot enables it by default for web applications (`spring.jpa.open-in-view=true`) and logs a warning at startup:

```
WARN  JpaBaseConfiguration$JpaWebConfiguration - spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed
during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
```

The demo controller touches a lazy collection outside any `@Transactional` method:

```
@GetMapping("/authors/{id}/summary")
String summary(@PathVariable Long id) {
    Author a = authors.findById(id).orElseThrow();
    return a.getName() + " has " + a.getBooks().size() + " books";   // lazy load in the controller
}
```

```
open-in-view=true  -> 200 Terry Pratchett has 3 books
open-in-view=false -> Cannot lazily initialize collection of role 'ch08.jpa.Author.books' with key '2' (no session)
```

Trade-offs

OSIV on	OSIV off
lazy loading "just works" anywhere in the request	lazy loading only inside transactions
a database connection may be held for the whole request, including slow serialization	connections held only during transactions
lazy loads during serialization run in auto-commit mode, one query each (hidden N+1)	forces you to fetch explicitly per use case
convenient for prototypes	predictable and recommended for APIs

Recommendation: set `spring.jpa.open-in-view=false` in new applications, fetch what each use case needs inside the service, and return DTOs. The failing test above is exactly what you want to see during development rather than in production under load.

8.16 Schema migrations with Flyway

What it is

In production you never let Hibernate change the schema. A **migration tool** applies small, numbered SQL scripts in order, **exactly once per database**, and records each one in a history table. The schema's history then lives in version control (e.g. Git, which tracks every change to your files), like code. **Flyway** (SQL files) and **Liquibase** (XML/YAML/SQL changelogs — files listing the changes) are the two Boot supports.

Boot 4 change: Flyway is only auto-configured when you use `spring-boot-starter-flyway`. `flyway-core` on its own is no longer enough, because of Boot 4's modular auto-configuration. For PostgreSQL also add `org.flywaydb:flyway-database-postgresql` (database support has been split into separate modules since Flyway 10).

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-flyway</artifactId>
</dependency>
```

```
src/main/resources/db/migration/
  V1__create_customer.sql        CREATE TABLE customer (id ... PRIMARY KEY, name VARCHAR(100) NOT NULL);
  V2__add_customer_email.sql     ALTER TABLE customer ADD COLUMN email VARCHAR(200); CREATE UNIQUE INDEX ...
  V3__seed_customers.sql         INSERT INTO customer (name, email) VALUES ('Asha', ...), ('Ben', ...);
```

```
spring.jpa.hibernate.ddl-auto=validate   # Hibernate only checks the entities against Flyway's schema
```

```
@DataJpaTest(showSql = false)            // the JPA slice includes Flyway
class MigrationTest {
    @Test
    void migrationsRanInOrderAndHibernateValidatedTheSchema() {
        for (MigrationInfo m : flyway.info().all())
            System.out.printf("V%s  %-25s %s%n", m.getVersion(), m.getDescription(), m.getState());
        // + print flyway_schema_history and the customers
    }
}
```

```
V1  create customer           SUCCESS
V2  add customer email        SUCCESS
V3  seed customers            SUCCESS
history: -1 | null | << Flyway Schema History table created >> | TABLE
history: 1 | 1 | create customer | SQL
history: 2 | 2 | add customer email | SQL
history: 3 | 3 | seed customers | SQL
customer: Asha <asha@example.com>
customer: Ben <ben@example.com>
```

(Flyway 12.4 also printed a warning that H2 2.4 is newer than the H2 version it was verified with, which is harmless here.)

How it works

1. At startup, before JPA initializes, Boot runs Flyway against the `DataSource`.
2. Flyway reads `flyway_schema_history`, finds the scripts not yet applied, runs them in version order (each in a transaction where the database allows), and records each with a **checksum** (a short fingerprint computed from the file's contents, so any edit is detected).
3. Hibernate starts with `ddl-auto=validate`. If an entity has a column the schema lacks, for example if V2 were missing, startup fails. That is a useful safety net.

Rules

Rule	Why
Never edit a migration that has been applied anywhere	Flyway's checksum check fails (`validate` error). Fix forward with a new version
Naming: `V<version>__<description>.sql` (two underscores), `R__<name>.sql` for repeatable scripts (views, functions)	how Flyway orders and classifies
Make changes backward-compatible (expand → migrate → contract: first add new columns, then move data and code, only later remove old ones)	during a rolling deploy (servers updated one at a time), old and new app versions share one schema
Big data changes in separate, batched migrations	one huge transaction locks tables
Test migrations against the real database type	H2 accepts SQL that PostgreSQL rejects, and the other way round. Tests with PostgreSQL via Testcontainers (a library that starts real databases in Docker for tests): ch12

8.17 Testing JPA code

`@DataJpaTest` (package `org.springframework.boot.data.jpa.test.autoconfigure` in Boot 4, from `spring-boot-starter-data-jpa-test`) starts only JPA: entities, repositories, `EntityManager`, `TestEntityManager`, an embedded database, and Flyway/Liquibase if present. Like `@JdbcTest`, it wraps each test in a rolled-back transaction and turns on `show-sql` (set `showSql = false` to silence it).

Remember what that transaction hides. Inside it, lazy loading always works and nothing is ever committed. That is why this chapter's `LazyLoadingTest`, `AuditingAndLockingTest` and `OpenInViewTest` use `@SpringBootTest` **without** a test transaction. Call `em.flush()` and `em.clear()` in `@DataJpaTest` tests when you want to see the SQL and read back from the database, not from the cache. More in ch12.

8.18 Performance checklist

Check	How
All `@ManyToOne` / `@OneToOne` LAZY	`fetch = FetchType.LAZY`
No N+1 on hot paths (code that runs very often)	assert statement counts in tests. Use JOIN FETCH / `@EntityGraph`
Safety net	`spring.jpa.properties.hibernate.default_batch_fetch_size=16` (or similar)
Read-only screens	projections / DTOs, `@Transactional(readOnly = true)`
Batched inserts	SEQUENCE ids + `hibernate.jdbc.batch_size=50` + `hibernate.order_inserts=true` (IDENTITY disables insert batching)
OSIV off	`spring.jpa.open-in-view=false`
No unbounded queries	paging (keyset for deep pages), `Limit`
Big batch jobs	chunk + `flush()` / `clear()`, or JDBC
Indexes for your query patterns	foreign keys are not indexed automatically in PostgreSQL. See SQL ch12

8.19 Interview questions

1. **JPA vs Hibernate vs Spring Data JPA?** → Specification vs implementation vs Spring's repository layer on top.
2. **What is the persistence context?** → The `EntityManager`'s first-level cache and unit of work. One instance per row, snapshots, dirty checking, write-behind at flush.
3. **Why don't I need `save()` after changing a loaded entity?** → Dirty checking at flush, inside a transaction.
4. **Entity states?** → Transient, managed, detached, removed.
5. **What is the owning side?** → The side with the foreign key (`@ManyToOne`). JPA only reads it. The other side uses `mappedBy`.

6. **`orphanRemoval` vs `CascadeType.REMOVE`?** → Removal from the collection deletes the child vs deleting the parent deletes the children.
7. **Default fetch types?** → To-one: EAGER. To-many: LAZY. Make everything LAZY.
8. **What causes `LazyInitializationException` and how do you fix it?** → Touching a lazy association after the persistence context closed. Fetch it in the query, work inside a transaction and return DTOs. Not EAGER, and preferably not OSIV.
9. **Explain N+1 and three fixes.** → 1 query + 1 per parent. JOIN FETCH, `@EntityGraph`, batch fetching (and DTO projections).
10. **JOIN FETCH vs `@EntityGraph`?** → The same idea. JOIN FETCH is written in JPQL (inner join unless `left`). An entity graph is declared on the method and uses a left join.
11. **How does `save()` choose between persist and merge?** → `isNew`: null id, or null wrapper version. Otherwise merge.
12. **Derived query vs `@Query` vs native?** → Short conditions vs readable JPQL vs database-specific SQL.
13. **`Page` vs `Slice`?** → `Page` runs an extra COUNT and knows totals. `Slice` only knows whether there is a next one.
14. **Interface vs DTO projections?** → Both select fewer columns. Records are plain values, interfaces are proxies. Open interface projections load the entity.
15. **How does optimistic locking work?** → A `@Version` column, `WHERE version = ?`, 0 rows → `ObjectOptimisticLockingFailureException`.
16. **What is OSIV and should it be on?** → The persistence context stays open for the whole web request. It is on by default with a warning. Turn it off for APIs.
17. **`ddl-auto` in production?** → `validate` or `none`, with Flyway/Liquibase owning the schema.
18. **What changed for Flyway in Boot 4?** → Use `spring-boot-starter-flyway`. `flyway-core` alone is not auto-configured.

Chapter TL;DR

- **JPA** is the spec, **Hibernate 7** the implementation, **Spring Data JPA** the repository layer. Boot 4.1 ships Hibernate 7.4 and Spring Data 2026.0.
- The **persistence context** gives one object per row, tracks changes, and flushes `UPDATE`s automatically (**dirty checking**). Detached entities are not tracked.
- **Relationships**: the owning side holds the foreign key. Use `mappedBy` on the other side and keep both in sync. Cascade from aggregate roots only. `orphanRemoval` deletes removed children.
- **LAZY everything.** `LazyInitializationException` means you touched it outside a transaction. Fetch per use case instead.
- **N+1** is 1 + N queries, measured here as 5 statements for 4 authors. Fix it with `JOIN FETCH` (1), `@EntityGraph` (1), batch size (2), or projections. EAGER makes it worse, not better.
- **Repositories**: derived queries, `@Query` (JPQL/native), `@Modifying` (clears nothing unless asked), `Page`/`Slice`/`Window`, projections for reads.
- **Auditing** fills created/modified columns. `@Version` turns lost updates into `ObjectOptimisticLockingFailureException`. `@Lock(PESSIMISTIC_WRITE)` gives `FOR UPDATE`.
- Turn **open-in-view off**, and let **Flyway** (`spring-boot-starter-flyway` in Boot 4) own the schema with `ddl-auto=validate`.

Next: Chapter 9 — Spring Data beyond JPA, and caching covers Spring Data JDBC, MongoDB, Redis and the cache abstraction.

Chapter 9 — Spring Data beyond JPA, and caching

Chapter 8 used JPA, the most common way to reach a database from Spring. But JPA is not the only option, and it is not always the best one. This chapter covers the other Spring Data modules you are most likely to meet: **Spring Data JDBC**, a simpler relational mapper built around *aggregates* (groups of objects saved and loaded as one unit, §9.2); **Spring Data MongoDB** for document databases (databases that store each record as a JSON-like document instead of table rows); **Spring Data Redis** for the in-memory key-value store (a store that maps keys to values and keeps them in RAM); and **Spring Data REST**, which turns repositories into a REST API. It then compares the two schema-migration tools (tools that apply versioned changes to table structure), **Flyway and Liquibase**.

The second half is about **caching**: remembering the result of an expensive call so the next identical call is fast. Spring's cache abstraction lets you add caching with one annotation, and it is also easy to get wrong. So every behaviour is measured. The tests count real method calls and cache hits and misses (whether the answer was already in the cache or not), move a fake clock to show expiry, and start eight threads (parallel paths of execution) at once to show a cache stampede (many requests recomputing the same missing entry at once) and its fix.

What you'll learn

- What the Spring Data modules share, and what each store adds
- Spring Data JDBC: aggregates, `@MappedCollection`, `AggregateReference`, `@Embedded`, the SQL it really runs, optimistic locking, queries, and the "new or existing?" trap
- Spring Data REST: repositories as hypermedia endpoints (HTTP endpoints whose responses include links to related resources)
- Flyway beyond the basics (Java and repeatable migrations, checksum validation), and Flyway vs Liquibase
- Spring Data MongoDB and Spring Data Redis: documents, repositories, templates, and Testcontainers (a library that starts real databases in Docker containers for tests) tests
- The cache abstraction: `@Cacheable`, `@CachePut`, `@CacheEvict`, `@Caching`, `@CacheConfig`, keys, `condition`/`unless`, TTL (time to live: how long an entry stays valid), `sync`
- Caffeine (a fast Java in-memory cache library) as the local cache, and Redis as a shared cache
- Cache pitfalls: self-invocation, key collisions, mutable cached objects, stampedes, stale data

The code (Spring Boot 4.1.1 → Spring Framework 7.0, Spring Data 2026.0, Flyway 12.4, Caffeine 3.2, H2 in memory):

Project	What it proves	Runs here?
`code/data-jdbc`	Spring Data JDBC aggregates with SQL printed, optimistic locking, queries, assigned-id trap; Flyway SQL + Java + repeatable migrations and checksum validation; Spring Data REST	yes
`code/caching`	`@Cacheable` / `@CachePut` / `@CacheEvict` with Caffeine, hit/miss counts, key generation, TTL with a fake clock, mutable-object pitfall, stampede with and without `sync`	yes
`code/nosql`	MongoDB and Redis repositories, `MongoTemplate` aggregation, `StringRedisTemplate`, Redis cache configuration, Testcontainers test	compiled only (needs Docker — the tool that runs containers — not run)

Run everything with `bash tools_run_spring.sh spring '09-*'`. The `nosql` project has a `.tools_run_skip` file, so the runner skips it.

9.1 The Spring Data family

What it is

Spring Data is an umbrella project with one module per kind of data store. Each module gives you the same programming model:

1. You write an interface that extends a repository interface, such as `CrudRepository<T, ID>`.
2. Spring creates the implementation at startup.
3. Method names like `findByStatus` become queries (**derived queries**), and `@Query` lets you write a query yourself.

What differs is the **mapping** (how an object becomes a row, a document or a hash) and the extra features each store has.

Module	Store	Boot starter	Maps objects to	Template class for hand-written work
Spring Data JPA	relational, via Hibernate	`spring-boot-starter-data-jpa`	rows, with a persistence context	`EntityManager`
Spring Data JDBC	relational, plain JDBC	`spring-boot-starter-data-jdbc`	rows, **aggregate by aggregate**	`JdbcAggregateTemplate`
Spring Data R2DBC	relational, reactive	`spring-boot-starter-data-r2dbc`	rows (no relationships)	`R2dbcEntityTemplate` (ch13)
Spring Data MongoDB	document database	`spring-boot-starter-data-mongodb`	BSON (binary JSON) documents	`MongoTemplate`
Spring Data Redis	in-memory key-value	`spring-boot-starter-data-redis`	Redis hashes (a key holding a small map of field → value)	`RedisTemplate`, `StringRedisTemplate`
Spring Data REST	(any of the above)	`spring-boot-starter-data-rest`	repositories → HTTP endpoints	—

Other modules exist (Elasticsearch, Cassandra, Neo4j, Couchbase, LDAP), and they follow the same pattern.

All Spring Data modules are released together as a **release train**. The train is named by year: Spring Boot 4.1.1 manages **Spring Data 2026.0**, which contains Spring Data JDBC 4.1, MongoDB 5.1 and Redis 4.1.

What is shared, and what is not

Shared by all modules (from `spring-data-commons`)	Store-specific
`CrudRepository`, `ListCrudRepository`, `PagingAndSortingRepository`	`JpaRepository`, `MongoRepository`, … with extra methods
Derived query parsing (`findByNameAndAgeGreaterThan`)	What a query *is*: SQL, a Mongo JSON filter, a Redis index lookup
`Pageable`, `Sort`, `Page`, `Slice` (8.11)	Transactions (full in JDBC/JPA; limited in Mongo; very different in Redis)
Auditing (`@CreatedDate`, `@LastModifiedBy`), `@Version`, `@Id`, `@Transient` from `org.springframework.data.annotation`	Mapping annotations: `@Table`/`@Column` (relational), `@Document`/`@Field` (Mongo), `@RedisHash` (Redis)
Projections, query by example, domain events (events an aggregate publishes when it changes)	Store features: aggregations, TTLs, geo queries (searches by location)

Because the model is shared, the repository chapter of ch08 (sections 8.8–8.12) applies here too. This chapter shows only what is different.

9.2 Spring Data JDBC and aggregates

What it is

Spring Data JDBC maps Java objects to relational tables **without JPA**. It has no Hibernate, no persistence context, no lazy loading, no dirty checking and no first-level cache. When you call `save`, it runs SQL right away. When you call `findById`, it runs SQL and gives you a plain object that is not connected to anything.

Its design comes from **Domain-Driven Design (DDD)**, which uses the word **aggregate** for a cluster of objects that are always loaded, changed and saved as one unit. One object, the **aggregate root**, is the only way in. Think of an order: an order has lines, and it makes no sense to change a line without its order. Rules like "you cannot add lines to a paid order" live in the root.

Spring Data JDBC builds on that idea:

- **One repository per aggregate root.** There is a `PurchaseOrderRepository` but no `OrderLineRepository`.
- **Everything the root references by object is part of the aggregate.** It is saved and deleted with the root.
- **References to other aggregates are ids, not objects.** An order holds the *id* of its customer, never a `Customer` object.

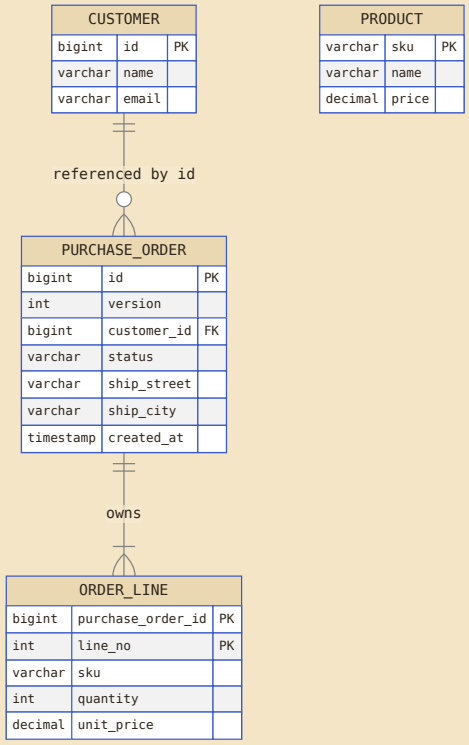
Why it matters

JPA is powerful but complex. Most of the surprises in ch08 (`LazyInitializationException`, N+1 queries, updates without a `save`, detached entities) come from the persistence context and lazy loading. Spring Data JDBC removes both. You get fewer features, and in exchange every call does exactly what it says.

	Spring Data JPA (Hibernate)	Spring Data JDBC
Loading	lazy or eager, proxies	everything in the aggregate, eagerly; nothing outside it
Change tracking	dirty checking: changes are flushed automatically	none: only `save()` writes
Caching	first-level cache per persistence context (+ optional second level, shared across sessions)	none
Relationships	any direction, many-to-many, bidirectional	one direction, root → children; other aggregates by id
Schema generation	`ddl-auto` can create tables	never; you write the DDL (Data Definition Language: `CREATE TABLE` and friends; use Flyway)
SQL you can predict	not always	yes
Updates of child collections	only the changed rows	**delete all children, insert them again**
Best for	rich object graphs (many objects linked to each other), existing JPA knowledge	clear aggregates, teams that want to see the SQL

The example aggregate

An online shop. `Customer` and `Product` are aggregates on their own. `PurchaseOrder` is an aggregate whose root holds a list of `OrderLine`s, an embedded shipping `Address`, and the *id* of its customer.



The schema comes from a Flyway migration (`V1__create_shop.sql`), because Spring Data JDBC never creates tables. The aggregate root:

```java
public class PurchaseOrder {

    @Id Long id;
    @Version Integer version;                                   // optimistic locking; null = new
    AggregateReference<Customer, Long> customerId;             // another aggregate: id only, never loaded
    Status status = Status.NEW;
    @Embedded.Nullable(prefix = "ship_") Address shipTo;      // value object in the same row
    @CreatedDate Instant createdAt;                             // filled by auditing

    // Names written in annotations are used exactly as given AND quoted. Upper case matches H2's
    // unquoted DDL (and Oracle/DB2); on PostgreSQL/MySQL write them in lower case.
    @MappedCollection(idColumn = "PURCHASE_ORDER_ID", keyColumn = "LINE_NO")
    List<OrderLine> lines = new ArrayList<>();                 // owned children (table order_line)

    PurchaseOrder() {}                                          // for Spring Data

    public PurchaseOrder(long customerId, Address shipTo) {
        this.customerId = AggregateReference.to(customerId);
        this.shipTo = shipTo;
    }

    /** Business rules live in the root: all changes to lines go through it. */
    public void addLine(String sku, int quantity, BigDecimal unitPrice) {
        if (status != Status.NEW) throw new IllegalStateException("order is " + status);
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
        lines.add(new OrderLine(sku, quantity, unitPrice));
    }
    // total(), getters ...
}

public record OrderLine(String sku, int quantity, BigDecimal unitPrice) {}   // no @Id: part of the order
public record Address(String street, String city) {}                          // value object
public record Customer(@Id Long id, String name, String email) {}             // its own aggregate
```

The mapping rules

Java	Table/column	Notes
class `PurchaseOrder`	table `purchase_order`	default naming: camelCase (`purchaseOrder`) → snake_case (`purchase_order`). Override with `@Table("...")`
field `createdAt`	column `created_at`	override with `@Column("...")`
`@Id Long id`	primary key	`null` means "new": the database generates the id (identity column — an auto-increment column)
`@Version Integer version`	`version`	optimistic locking, as in 8.14
`List<OrderLine>` + `@MappedCollection`	child table `order_line`	`idColumn` = foreign key to the root, `keyColumn` = the list index. `Set` needs no key column; `Map<K, V>` uses the map key
`AggregateReference<Customer, Long>`	`customer_id`	just the id. You cannot navigate to the customer. Load it with `CustomerRepository` if you need it
`@Embedded.Nullable(prefix = "ship_")`	`ship_street`, `ship_city` in the same row	`Nullable`: all columns `NULL` → field `null`. `@Embedded.Empty` gives an empty object instead
records	supported	Spring Data creates a new instance through the constructor when it must set a generated id

Spring Data JDBC quotes every identifier it generates. With H2 that means upper case: `"PURCHASE_ORDER"`. A name you write in an annotation is quoted **as you wrote it**, so `idColumn = "purchase_order_id"` became `"purchase_order_id"` and H2 answered `Column "line_no" not found`. That is why the example uses upper case. On PostgreSQL, unquoted names are folded (converted) to lower case, so there you write lower case.

How saving and loading work — measured

The tests capture the SQL that Spring's `JdbcTemplate` logs at `DEBUG` (helper `SqlLog.java`, which also removes the quotes to make it readable). `@DataJdbcTest` is the slice for Spring Data JDBC. It starts the repositories, JDBC, Flyway and an embedded database, and rolls back each test.

```java
@DataJdbcTest                                  // JDBC + Spring Data JDBC + Flyway, each test rolled back
class AggregateTests {
    @Autowired PurchaseOrderRepository orders;
    final SqlLog sql = new SqlLog();

    PurchaseOrder newOrder() {
        var order = new PurchaseOrder(1L, new Address("1 Main St", "Pune"));
        order.addLine("BOOK-1", 1, new BigDecimal("40.00"));
        order.addLine("PEN-7", 4, new BigDecimal("2.50"));
        return order;
    }

    @Test
    void savingTheRootSavesItsLinesAndLoadingReadsThemBack() {
        sql.clear();
        PurchaseOrder saved = orders.save(newOrder());
        sql.print("save(new order with 2 lines)");

        sql.clear();
        PurchaseOrder loaded = orders.findById(saved.getId()).orElseThrow();
        sql.print("findById");
        // print id, version, customer id, lines, total, address, createdAt ...
    }
}
```

Output:

```
-- save(new order with 2 lines)
   INSERT INTO PURCHASE_ORDER (CREATED_AT, CUSTOMER_ID, SHIP_CITY, SHIP_STREET, STATUS, VERSION) VALUES (?, ?, ?, ?, ?, ?)
   [batch] INSERT INTO ORDER_LINE (LINE_NO, PURCHASE_ORDER_ID, QUANTITY, SKU, UNIT_PRICE) VALUES (?, ?, ?, ?, ?)
-- findById
   SELECT ID, STATUS, SHIP_STREET, SHIP_CITY, VERSION, CREATED_AT, CUSTOMER_ID FROM PURCHASE_ORDER WHERE PURCHASE_ORDER.ID = ?
   SELECT SKU, QUANTITY, UNIT_PRICE, LINE_NO FROM ORDER_LINE WHERE ORDER_LINE.PURCHASE_ORDER_ID = ? ORDER BY LINE_NO
loaded: id=3 version=0 customer=1 lines=2 total=50.00 shipTo=Address[street=1 Main St, city=Pune] createdAt set=true
```

What happened, step by step:

1. `save` saw `id == null`, so the order is **new**. It inserted the root row and got the generated id back.
2. It inserted both lines in **one JDBC batch**, with `PURCHASE_ORDER_ID` set to the new id and `LINE_NO` set to the list index (0 and 1).
3. Auditing filled `createdAt`. `@EnableJdbcAuditing` sits on the application class, and the test slice picks it up from there. Adding `@EnableJdbcAuditing` a second time in a test configuration fails with a duplicate bean definition error.
4. `findById` ran **two** queries: one for the root, one for its lines. There is no join, no lazy proxy, and the customer was **not** loaded. The order only knows `customer=1`.
5. The version of a new row starts at 0.

Loading the root plus one query per collection is fine for one aggregate. When you load many aggregates, it becomes an N+1 pattern (one lines query per order). Spring Data JDBC has an opt-in **single query loading** mode (`JdbcMappingContext.setSingleQueryLoadingEnabled(true)`) that loads aggregates with one SQL statement using window functions. The reference docs call it experimental. It works only for `find…` methods of `CrudRepository`, for aggregates without nested collections, embedded values or `AggregateReference`s, and not on H2. So in practice, keep aggregates small and read lists of data with a `@Query` DTO (shown below).

Updating: all children are deleted and inserted again

```java
@Test
void updatingTheRootDeletesAndReinsertsAllLines() {
    PurchaseOrder order = orders.save(newOrder());
    order.addLine("PEN-7", 1, new BigDecimal("2.50"));     // change ONE thing: add a third line

    sql.clear();
    orders.save(order);
    sql.print("save(order with 1 added line)");
}
```

```
-- save(order with 1 added line)
   UPDATE PURCHASE_ORDER SET VERSION = ?, STATUS = ?, SHIP_STREET = ?, SHIP_CITY = ?, CREATED_AT = ?, CUSTOMER_ID = ? WHERE PURCHASE_ORDER.ID
= ? AND PURCHASE_ORDER.VERSION = ?
   DELETE FROM ORDER_LINE WHERE ORDER_LINE.PURCHASE_ORDER_ID = ?
   [batch] INSERT INTO ORDER_LINE (LINE_NO, PURCHASE_ORDER_ID, QUANTITY, SKU, UNIT_PRICE) VALUES (?, ?, ?, ?, ?)
```

Spring Data JDBC does not remember what it loaded, so it cannot know *which* line changed. It updates the root (checking the version), deletes **all** lines, and inserts all three again. This is simple and always correct, and it is the main cost of the approach:

- Fine for aggregates with a few children (an order with 1–50 lines).
- Bad for an aggregate with thousands of children. That is usually a sign the aggregate is too big: make the children a separate aggregate that references the parent by id.

- Child rows get new physical rows on every save, so do not point foreign keys from *other* tables at child rows.

Optimistic locking

The `UPDATE` above ends with `AND PURCHASE_ORDER.VERSION = ?`. If another transaction changed the row in between, no row matches:

```
Long id = orders.save(newOrder()).getId();
PurchaseOrder alice = orders.findById(id).orElseThrow();   // both read version 0
PurchaseOrder bob = orders.findById(id).orElseThrow();

alice.setStatus(Status.PAID);
orders.save(alice);                                        // version 0 -> 1

bob.setStatus(Status.SHIPPED);
assertThatThrownBy(() -> orders.save(bob))                 // still has version 0
        .isInstanceOf(OptimisticLockingFailureException.class);
```

```
alice saved, version now 1
bob: OptimisticLockingFailureException: Failed to update versioned entity with id '1' (version '0') in table ["PURCHASE_ORDER"]; Was the
entity updated or deleted concurrently?
```

It is the same idea as JPA's `@Version` (8.14). Because the whole aggregate is saved through the root, **one** version column protects the root and all its lines together. That is exactly what you want: two users cannot both add lines to the same order and silently overwrite each other.

Queries

```
public interface PurchaseOrderRepository extends ListCrudRepository<PurchaseOrder, Long> {

    List<PurchaseOrder> findByStatus(Status status);              // derived query

    List<PurchaseOrder> findByCustomerId(Long customerId);        // derived query on the reference column

    @Query("SELECT order_id, customer, total FROM order_totals ORDER BY order_id")
    List<OrderTotal> totals();                                    // hand-written SQL into a DTO

    @Modifying
    @Query("UPDATE purchase_order SET status = :status WHERE id = :id")
    boolean updateStatus(Long id, String status);                 // bulk-style update, bypasses the aggregate
}

public record OrderTotal(Long orderId, String customer, BigDecimal total) {}
```

```
findByCustomerId(2): 1 order(s)
shipTo of order without address: null
updateStatus -> true, findByStatus(PAID): 1
total: OrderTotal[orderId=4, customer=Asha, total=50.00]
total: OrderTotal[orderId=5, customer=Ben, total=80.00]
```

- **Derived queries** work as in JPA, but they may only use properties of the root row (and embedded values). You cannot write `findByLinesSku`, because that would need a join into the aggregate. Use `@Query` for that.
- `@Query` takes plain SQL, not JPQL. Parameters are named (`:id`) and bound by parameter name. The result can be the entity, a DTO record (columns mapped by name, `order_id` → `orderId`) or a simple value.
- `@Modifying` marks `UPDATE` / `DELETE` queries. It returns the row count as `int`, or `boolean` ("at least one row changed"). It bypasses the aggregate: no version check, no callbacks, no business rules. Use it for bulk or administrative changes only.
- The view (a saved query that can be read like a table) `order_totals` is a read model (a structure shaped just for queries) built by a repeatable migration (section 9.4). Reading reports with SQL and DTOs, and writing through aggregates, is a common split.

Pitfall: "new or existing?" with assigned ids

Spring Data must decide between `INSERT` and `UPDATE`. The rule (Spring Data JDBC reference):

- the entity is **new** if its id is `null` (or `0` for a primitive), **or** if it has a `@Version` property that is `null` (or `0` for a primitive);
- otherwise it is **existing**.

`Product` uses its SKU as the id, a value *we* assign. So a brand-new product already has a non-null id:

```
public record Product(@Id String sku, String name, BigDecimal price) {}

Product mug = new Product("MUG-3", "Coffee mug", new BigDecimal("9.00"));
products.save(mug);                                        // no exception...
template.insert(mug);                                      // JdbcAggregateTemplate: "this is new"
```

```
-- save(new Product("MUG-3", ...))
   UPDATE PRODUCT SET NAME = ?, PRICE = ? WHERE PRODUCT.SKU = ?
rows after save():   2   <- nothing was inserted!
rows after insert(): 3   Product[sku=MUG-3, name=Coffee mug, price=9.00]
```

`save` ran an `UPDATE` that matched zero rows, and **no exception was thrown**. The product was silently lost. This is one of the most common Spring Data JDBC bugs. The fixes:

Fix	How
Add a `@Version` field	a new object has version `null`/`0` → `INSERT`. Also gives you optimistic locking. Usually the best fix
Call `JdbcAggregateTemplate.insert(entity)` / `update(entity)`	you state the intent explicitly (shown above)
Implement `Persistable<ID>`	you write `isNew()` yourself, e.g. with a `@Transient boolean isNew` flag
Let the database generate ids	`null` id → `INSERT`. Keep the natural key as a unique column

The same `isNew` rule exists in Spring Data JPA (`save` then calls `merge`, which runs an extra `SELECT` rather than losing data).

Callbacks, events and auditing

Spring Data JDBC calls beans at fixed points of the lifecycle. **Callbacks** can change the entity: `BeforeConvertCallback` (set an id before insert), `BeforeSaveCallback`, `AfterSaveCallback`, `AfterConvertCallback` (after loading). **Events** such as `BeforeSaveEvent`, `AfterSaveEvent` and `AfterDeleteEvent` are published as application events (ch02) and are for observing only. Auditing (`@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`) is switched on with `@EnableJdbcAuditing`. For the "by" fields you also provide an `AuditorAware` bean.

Common mistakes

Mistake	Result	Fix
Expecting lazy loading or dirty checking	changes are lost unless you call `save`	load, change, **save** — explicitly
Mapping a child as an object reference to *another* aggregate	Spring Data treats it as part of your aggregate and deletes/inserts it on save	reference other aggregates with `AggregateReference` or a plain id
Huge child collections	every save deletes and re-inserts thousands of rows	smaller aggregates
Assigned ids without `@Version` or `Persistable`	`save` silently updates 0 rows	see the table above
Lower-case names in annotations on H2 (upper-case on PostgreSQL)	`column not found`	match the database's folding rules
Expecting `ddl-auto`	there is none; tables do not exist	Flyway/Liquibase
Relying on `@DataJdbcTest` rollback when testing concurrency	two "users" share one test transaction	`@SpringBootTest` without a test transaction, as in ch08

In the interview

30-second answer: "Spring Data JDBC is a simpler alternative to JPA. It maps aggregates, in the DDD sense: a root plus the entities it owns, loaded and saved as one unit. There is no persistence context, so no lazy loading, no dirty checking and no caching. `save` writes immediately, loading reads the whole aggregate, and references to other aggregates are just ids (`AggregateReference`). On update it rewrites the root and deletes and re-inserts all children, which is why aggregates must stay small. I pick it when I want predictable SQL and my model has clear aggregates."

- *"How does Spring Data decide between INSERT and UPDATE?"* → id `null`/`0` or `@Version` `null`/`0` → new. With assigned ids you need `@Version`, `Persistable`, or `JdbcAggregateTemplate.insert`, otherwise `save` issues an `UPDATE` that changes nothing.
- *"How do you model many-to-many?"* → As a separate small entity inside one aggregate that holds the *id* of the other aggregate (for example `Set<BookAuthor>` in `Book`, where `BookAuthor` has an `AggregateReference<Author, Long>`).
- *"Can you mix JPA and JDBC in one app?"* → Yes, on the same `DataSource`, but give them different entities and packages, and remember that JDBC writes are not seen by an open Hibernate persistence context.

Self-check

1. What SQL does `save` run for an existing order with 3 lines? → `UPDATE` of the root (with version check), `DELETE` of all its lines, then 3 `INSERT`s (one batch).
2. Why can `PurchaseOrder` not have a `Customer customer` field? → It would make the customer part of the order aggregate; references to other aggregates must be ids.
3. A `findById` loaded the order and its lines. How many queries? → Two: root, then one per collection.

4. Which annotation marks a list of child entities, and what are its two column settings? → `@MappedCollection`, `idColumn` (FK to the root) and `keyColumn` (list index or map key).

TL;DR

- Spring Data JDBC = repositories over plain JDBC, **one repository per aggregate root**, no persistence context.
- Saving an existing aggregate rewrites all of its children, so keep aggregates small. `@Version` on the root protects the whole aggregate.
- Other aggregates are referenced by id (`AggregateReference`). Reports: `@Query` SQL into DTO records.
- Assigned ids need `@Version` , `Persistable` or `insert()` , or `save` loses data silently.

9.3 Spring Data REST

What it is

Spring Data REST looks at your repositories and publishes each one as a REST resource, with no controller code. It uses **HAL** (Hypertext Application Language), a JSON format where every resource has a `_links` section pointing to related resources. That style is called **HATEOAS** (Hypermedia as the Engine of Application State; more in ch05).

The `data-jdbc` project adds `spring-boot-starter-data-rest` and one property:

```
spring.data.rest.base-path=/api
```

```
@RepositoryRestResource(path = "customers")
public interface CustomerRepository extends ListCrudRepository<Customer, Long> {
    List<Customer> findByName(String name);   // -> GET /api/customers/search/findByName?name=Ben
}

@RepositoryRestResource(exported = false)     // not every repository should become a public API
public interface ProductRepository extends ListCrudRepository<Product, String> {}
```

The test (`DataRestTests.java`) calls it with `MockMvcTester` :

```
GET /api/customers/2 -> 200
{
  "_links" : {
    "self" : {
      "href" : "http://localhost/api/customers/2"
    },
    "customer" : {
      "href" : "http://localhost/api/customers/2"
    }
  },
  "name" : "Ben",
  "email" : "ben@example.com"
}
GET /api/customers/search/findByName?name=Asha -> 200
GET /api/products -> 404 (exported = false)
```

For each exported repository you get `GET` (collection, with paging and sorting if the repository supports it), `GET /{id}` , `POST` , `PUT` , `PATCH` , `DELETE` , and a `/search` resource listing the query methods.

Trade-offs

Good for	Not good for
Internal admin tools, prototypes, simple CRUD services	public APIs with a designed contract
Clients that follow links (HAL browsers, hypermedia clients)	APIs with business operations ("pay order") rather than CRUD
	Hiding your table structure: the API *is* your domain model, so a field rename breaks clients

Common mistakes

- **Exposing every repository by accident.** Every public repository is exported by default. Use `exported = false` , or set `spring.data.rest.detection-strategy=annotated` so that only `@RepositoryRestResource` repositories are exported.
- **Forgetting security.** `DELETE /api/customers/1` works for anyone until you add Spring Security rules (ch10).
- **Business rules in the database layer.** Validation and side effects then have to go into `@RepositoryEventHandler` callbacks, which are easy to miss. When the API needs real behaviour, write controllers (ch05).

9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase

Section 8.16 introduced Flyway: numbered SQL scripts in `db/migration`, run once each, recorded in `flyway_schema_history`, and `spring-boot-starter-flyway` in Boot 4. Spring Data JDBC depends on migrations even more than JPA, because it has no `ddl-auto` at all. This section adds three Flyway features and compares Flyway with Liquibase.

Three kinds of Flyway migration

The `data-jdbc` project has all three:

```
src/main/resources/db/migration/
  V1__create_shop.sql            versioned SQL: tables
  V2__seed_customers.sql         versioned SQL: data
  R__order_totals_view.sql       REPEATABLE: CREATE OR REPLACE VIEW order_totals ...
src/main/java/db/migration/
  V3__normalise_emails.java      versioned JAVA migration
```

```java
package db.migration;

/** A Java-based migration: for data changes that are awkward in SQL. Found by the class name V3__... */
public class V3__normalise_emails extends BaseJavaMigration {
    @Override
    public void migrate(Context context) throws Exception {
        var conn = context.getConnection();              // same connection and transaction as Flyway
        try (Statement select = conn.createStatement();
             ResultSet rs = select.executeQuery("SELECT id, email FROM customer");
             PreparedStatement update = conn.prepareStatement("UPDATE customer SET email = ? WHERE id = ?")) {
            while (rs.next()) {
                update.setString(1, rs.getString("email").trim().toLowerCase());
                update.setLong(2, rs.getLong("id"));
                update.addBatch();
            }
            update.executeBatch();
        }
    }
}
```

Kind	Name	Runs	Use for
Versioned SQL	`V<version>__<description>.sql`	once, in version order	tables, columns, indexes, data fixes
Versioned Java	class `V<version>__<description>` extending `BaseJavaMigration`, in a scanned location (e.g. package `db.migration`), or a Spring bean implementing `JavaMigration` (Boot registers such beans)	once, in version order with the SQL ones	data changes needing logic, calling code, big batched updates
Repeatable	`R__<description>.sql`	after all versioned ones, **again whenever its checksum changes**	views, functions, stored procedures, grants (permission statements) — objects you can always re-create

The test prints what Flyway recorded:

```java
for (MigrationInfo m : flyway.info().all()) {
    System.out.printf("%-4s %-22s %-5s %s%n",
            m.getVersion() == null ? "R" : "V" + m.getVersion(), m.getDescription(), m.getType(), m.getState());
}
customers.findAll().forEach(c -> System.out.println("customer: " + c));
```

```
V1   create shop            SQL   SUCCESS
V2   seed customers         SQL   SUCCESS
V3   normalise emails       JDBC  SUCCESS
R    order totals view      SQL   SUCCESS
customer: Customer[id=1, name=Asha, email=asha@example.com]
customer: Customer[id=2, name=Ben, email=ben@example.com]
```

V2 inserted `ASHA@Example.com` and `ben@EXAMPLE.com`, and the Java migration V3 normalised them. The Java migration shows up with type `JDBC`.

Checksums: never edit an applied migration — proved

Flyway stores a **checksum** (a number computed from the file's content) for each applied migration. On startup it validates that the files still match. The second test points a new Flyway instance at the same database but at a folder `db/tampered` that holds copies of V1, V2 and the repeatable script, with V1 edited:

```
Flyway tampered = Flyway.configure()
        .dataSource(dataSource)
        .locations("classpath:db/tampered")                // same files, but V1 was edited
        .load();
ValidateResult result = tampered.validateWithResult();
```

```
validation successful: false
  1: Migration checksum mismatch for migration version 1
-> Applied to database : 1770847798
-> Resolved locally    : 415358952
Either revert the changes to the migration, or run repair to update the schema history.
```

Widening the `name` columns in V1 from `VARCHAR(100)` to `VARCHAR(150)` was enough: the file no longer matches the recorded checksum. In an application this validation runs at startup, and the app **refuses to start**. The right reaction is almost never `repair`. Undo the edit and add a new migration (`V4__widen_customer_name.sql`) instead.

Useful Flyway properties

Property	Default	Meaning
`spring.flyway.locations`	`classpath:db/migration`	where scripts are. Use `classpath:db/migration/{vendor}` for per-database folders
`spring.flyway.baseline-on-migrate`	`false`	adopting Flyway on an **existing** database: mark the current state as the baseline (the starting version) instead of failing on a non-empty schema
`spring.flyway.clean-disabled`	`true`	`clean` drops everything in the schema. Keep it disabled everywhere except throwaway databases
`spring.flyway.enabled`	`true`	switch off, e.g. when a separate pipeline (an automated build/deploy process) runs migrations
`spring.flyway.url` / `user` / `password`	the app's `DataSource`	migrate with a more powerful user than the app uses

Liquibase

Liquibase solves the same problem with a different model. Migrations are **changesets** inside **changelog** files (YAML, XML, JSON or SQL). Each changeset is identified by `id` + `author` + file name, and applied changesets are recorded in the `DATABASECHANGELOG` table (with a lock table `DATABASECHANGELOGLOCK`). Changesets can be written as database-independent *change types* (`createTable`, `addColumn`), which Liquibase turns into SQL for each database.

In Boot 4, add `spring-boot-starter-liquibase`. The default master changelog is `classpath:/db/changelog/db.changelog-master.yaml` (`spring.liquibase.change-log`). A snippet (not run):

```
# src/main/resources/db/changelog/db.changelog-master.yaml
databaseChangeLog:
  - changeSet:
      id: 1
      author: asha
      changes:
        - createTable:
            tableName: customer
            columns:
              - column: { name: id, type: BIGINT, autoIncrement: true, constraints: { primaryKey: true } }
              - column: { name: name, type: VARCHAR(100), constraints: { nullable: false } }
  - changeSet:
      id: 2
      author: asha
      changes:
        - addColumn:
            tableName: customer
            columns:
              - column: { name: email, type: VARCHAR(200) }
      rollback:
        - dropColumn: { tableName: customer, columnName: email }
```

Flyway vs Liquibase

	Flyway	Liquibase
Migration format	plain SQL files (+ Java)	changelogs in YAML/XML/JSON/SQL; database-independent change types
Ordering	version number in the file name	order of changesets in the changelog
Identity of a migration	version	`id` + `author` + file path
History table	`flyway_schema_history`	`DATABASECHANGELOG` (+ lock table)
Rollback / undo	undo migrations only in Redgate's paid editions; in practice you fix forward	`rollback` blocks, and automatic rollback for many change types, in the free edition
Conditional logic	little (placeholders, callbacks)	preconditions, contexts, labels (e.g. "only in test")
Multi-database products	per-vendor folders	one changelog can target several databases
Learning curve	very low: you already know SQL	higher
Boot 4 starter	`spring-boot-starter-flyway`	`spring-boot-starter-liquibase`
Version in Boot 4.1.1	Flyway 12.4 (Community edition is Apache 2.0)	Liquibase 5.0. **Since 5.0 the Community edition uses the Functional Source License (FSL)**, which turns into Apache 2.0 two years after each release. Free to use, but check it with your legal team

Rule of thumb: a team that writes SQL and targets one database picks **Flyway**. A product that must run on several databases, or a team that needs rollback scripts and conditional changesets, picks **Liquibase**. Do not use both in one application.

Migration rules that apply to both

- Fix forward: never edit or delete an applied migration.
- Make each change **backward-compatible** during rolling deploys (expand → migrate data → contract). See 8.16.
- Test migrations against the **real database type** (Testcontainers, ch12). The H2 in this chapter accepts SQL that PostgreSQL rejects, and the other way round.
- Long data migrations lock tables. Run them in batches, outside the startup path if needed.

In the interview

- *"Flyway or Liquibase?"* → "Flyway for SQL-first teams on one database: simple versioned SQL files and a checksum history. Liquibase when I need database-independent changelogs, rollbacks or conditional changes. Either way the tool owns the schema, and Hibernate's `ddl-auto` is `validate` or `none`."
- *"Someone edited V5 after it ran in production. What happens?"* → Startup fails with a checksum mismatch. Revert the edit and ship the change as V6. `repair` only rewrites the history table's checksums; it does not change the schema.
- *"What is a repeatable migration?"* → `R__` script re-applied whenever its content changes, after all versioned ones. For views and functions.

9.5 Spring Data MongoDB (not run — needs Docker)

What it is

MongoDB is a **document database**. It stores JSON-like documents (the binary format is called BSON) in **collections**, with no fixed schema. A document can hold nested objects and arrays, so data that is read together is stored together. What would be three joined tables in SQL (a review, its tags, its product reference) can be one document. The concepts are covered in SQL book ch16 and DDIA ch2. Here we look at how Spring talks to it.

Setup (Boot 4)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

```
spring.mongodb.uri=mongodb://localhost:27017/shop
spring.data.mongodb.auto-index-creation=true
```

Boot 4 change: Boot 4 split MongoDB support into a driver module (`spring-boot-mongodb`, properties `spring.mongodb.*`) and a Spring Data module (`spring-boot-data-mongodb`, properties `spring.data.mongodb.*` for repository and mapping settings). So the connection is now `spring.mongodb.uri`. The old `spring.data.mongodb.uri` (and `.host`, `.port`, `.database`, ...) is marked deprecated at **error** level in Boot 4.1's configuration metadata: it is no longer bound, and `spring-boot-properties-migrator` reports it.

Documents and repositories

```
/** One MongoDB document. Nested data (tags) is stored inside it - no join table. */
@Document("reviews")
public record Review(
        @Id String id,                 // Mongo's _id; null -> an ObjectId is generated on insert
        @Indexed String productSku,    // index is created only if spring.data.mongodb.auto-index-creation=true
        int stars,
        String text,
        List<String> tags,
        Instant createdAt) {}

public interface ReviewRepository extends MongoRepository<Review, String> {

    // derived queries become Mongo filters: { productSku: ?0 } sorted by createdAt desc
    List<Review> findByProductSkuOrderByCreatedAtDesc(String productSku);

    List<Review> findByStarsGreaterThanEqual(int stars);   // { stars: { $gte: ?0 } }

    @Query("{ 'tags': ?0 }")                               // a JSON filter written by hand
    List<Review> withTag(String tag);
}
```

A saved review looks like this in MongoDB (Spring Data adds a `_class` field with the Java type, used when reading back polymorphic data — documents of several subclasses in one collection):

```
{ "_id": ObjectId("..."), "productSku": "BOOK-1", "stars": 5, "text": "Great",
  "tags": ["clear"], "createdAt": ISODate("..."), "_class": "ch09.nosql.mongo.Review" }
```

`MongoTemplate` for aggregations and partial updates

Repositories cover simple reads and writes. For grouping, and for changing one field without rewriting the whole document, use `MongoTemplate`:

```
public record AverageRating(String id, double average, long count) {}   // id = the product sku

public List<AverageRating> averageByProduct() {
    var pipeline = newAggregation(
            group("productSku").avg("stars").as("average").count().as("count"),
            sort(Sort.Direction.DESC, "average"));
    return mongo.aggregate(pipeline, "reviews", AverageRating.class).getMappedResults();
}

/** Update one field in place ($push) without reading and rewriting the whole document. */
public long addTag(String reviewId, String tag) {
    return mongo.updateFirst(
            Query.query(Criteria.where("id").is(reviewId)),
            new Update().push("tags", tag),
            Review.class).getModifiedCount();
}
```

An **aggregation pipeline** is a list of stages that documents flow through (`$match`, `$group`, `$sort`, `$project`, `$lookup`), roughly the document-database version of `WHERE` / `GROUP BY` / `ORDER BY`.

Things that work differently from JPA

Topic	MongoDB with Spring Data
Relationships	embed data that belongs together. Reference other documents by id. `@DBRef` and `@DocumentReference` exist but cost extra queries, like lazy loading
Schema changes	no DDL. Old documents keep their old shape. Code must accept both, or you run a data migration (tools such as Mongock exist)
Transactions	single-document writes are always atomic. Multi-document transactions need a **replica set** (several MongoDB servers holding copies of the same data) and a `MongoTransactionManager` bean, which Boot does **not** create for you
Optimistic locking	`@Version` works (the version is part of the update filter)
Indexes	`@Indexed`, `@CompoundIndex`, created only if `auto-index-creation=true`. In production, many teams create indexes in a migration instead
Reactive	`spring-boot-starter-data-mongodb-reactive` + `ReactiveMongoRepository` (ch13)

When to use it (and when not)

- **Good fit:** data that is read as a whole and has a flexible shape (product catalogs with different attributes per category, content, event logs, user profiles).
- **Poor fit:** data with many relationships queried from many directions, or which needs multi-row transactions and strong constraints (accounting, inventory). A relational database with a `jsonb` column is often enough for "a bit of flexible

data".

9.6 Spring Data Redis (not run — needs Docker)

What it is

Redis is an **in-memory data structure server**. Keys map to strings, hashes, lists, sets, sorted sets, streams and more, and each command is atomic (runs completely, with no other command interleaving) on the server. Because data lives in memory, reads and writes take well under a millisecond. It is used as a cache (section 9.8), a session store, for rate limiting (capping how many requests a client may make per second), counters, leaderboards, distributed locks (a lock shared by many app instances) and simple queues.

`spring-boot-starter-data-redis` brings **Lettuce** as the default client library (Jedis is the alternative). Connection properties keep the `spring.data.redis.*` prefix in Boot 4:

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

Templates: the usual way to use Redis

```
@Component
public class PageViewCounter {
    private final StringRedisTemplate redis;

    public PageViewCounter(StringRedisTemplate redis) { this.redis = redis; }

    public long hit(String page) {
        return redis.opsForValue().increment("views:" + page);   // INCR is atomic on the Redis server
    }
}
```

`opsForValue()`, `opsForHash()`, `opsForList()`, `opsForSet()`, `opsForZSet()` match the Redis data types. `StringRedisTemplate` stores keys and values as plain strings. `RedisTemplate<K, V>` can use other **serializers** (components that turn objects into bytes and back; JDK serialization by default, JSON if you configure it). Choose them deliberately, because anything written with one serializer can only be read back with the same one.

Redis repositories

```
/** Stored as a Redis hash under key "carts:<id>"; removed by Redis after 1 hour (timeToLive, seconds). */
@RedisHash(value = "carts", timeToLive = 3600)
public record Cart(@Id String id, @Indexed String userId, List<CartItem> items) {
    public record CartItem(String sku, int quantity) {}
}

public interface CartRepository extends CrudRepository<Cart, String> {
    List<Cart> findByUserId(String userId);   // works only because userId is @Indexed (a secondary index set)
}
```

How it is stored: the cart becomes a Redis **hash** at key `carts:c1`, the key is added to a set `carts` (for `findAll`), and `@Indexed` adds it to a set `carts:userId:u42`. So derived queries work only on `@Indexed` properties (each such set acts as a secondary index: an extra lookup structure by a non-id field) and only for equality. There are no ranges and no sorting.

A trap: when Redis expires `carts:c1` because of the TTL, the index sets still contain its id. Spring Data cleans them only if you enable keyspace events (notifications Redis sends when keys change or expire) (`@EnableRedisRepositories(enableKeyspaceEvents = RedisKeyValueAdapter.EnableKeyspaceEvents.ON_STARTUP)`). Most teams use templates directly and use repositories only for simple cases.

9.7 Testing MongoDB and Redis with Testcontainers (not run)

Testcontainers starts real databases in Docker containers for the duration of a test. Boot's `@ServiceConnection` reads the container's host and port and configures the connection. You do not need `@DynamicPropertySource` or any properties. The full treatment of Testcontainers is in ch12. Here is the setup for this chapter's NoSQL code. It compiles, but it is not run here because Docker is not available:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-junit-jupiter</artifactId>   <!-- Testcontainers 2.x artifact names -->
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-mongodb</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>com.redis</groupId>
  <artifactId>testcontainers-redis</artifactId>
  <scope>test</scope>
</dependency>
```

```
@SpringBootTest
@Testcontainers                       // starts/stops the @Container fields (needs a Docker daemon)
class NoSqlTests {

    @Container @ServiceConnection     // Boot points spring.mongodb.* at this container
    static MongoDBContainer mongo = new MongoDBContainer("mongo:8.0");   // org.testcontainers.mongodb

    @Container @ServiceConnection     // ...and spring.data.redis.* at this one
    static RedisContainer redis = new RedisContainer("redis:8.2");       // com.redis.testcontainers

    @Autowired ReviewRepository reviews;
    @Autowired RatingService ratings;
    @Autowired CartRepository carts;
    @Autowired PageViewCounter views;

    @Test
    void mongoRepositoryAndAggregation() {
        reviews.deleteAll();
        reviews.save(new Review(null, "BOOK-1", 5, "Great", List.of("clear"), Instant.now()));
        reviews.save(new Review(null, "BOOK-1", 3, "Dense", List.of(), Instant.now()));
        reviews.save(new Review(null, "PEN-7", 4, "Smooth", List.of("clear"), Instant.now()));

        assertThat(reviews.findByProductSkuOrderByCreatedAtDesc("BOOK-1")).hasSize(2);
        assertThat(reviews.withTag("clear")).hasSize(2);
        assertThat(ratings.averageByProduct()).first()
                .isEqualTo(new RatingService.AverageRating("PEN-7", 4.0, 1));
    }

    @Test
    void redisRepositoryAndCounter() {
        carts.save(new Cart("c1", "u42", List.of(new Cart.CartItem("PEN-7", 3))));
        assertThat(carts.findByUserId("u42")).extracting(Cart::id).containsExactly("c1");
        assertThat(views.hit("home")).isEqualTo(1);
        assertThat(views.hit("home")).isEqualTo(2);
    }
}
```

Testcontainers 2.0 renamed its modules (`testcontainers-mongodb` instead of `mongodb`) and moved container classes into module packages (`org.testcontainers.mongodb.MongoDBContainer`). Boot 4.1.1 manages Testcontainers 2.0.

For narrower tests there are slices: `@DataMongoTest` (starter `spring-boot-starter-data-mongodb-test`) and `@DataRedisTest` (`spring-boot-starter-data-redis-test`). They start only the store's configuration and repositories. Combine them with the same `@Container` `@ServiceConnection` fields.

9.8 The caching abstraction

What it is

A **cache** keeps the results of expensive work (a slow query, a remote call, a calculation) so that the next request for the same thing is answered from memory (see also System Design Fundamentals). A **cache hit** means the answer was found there. A **cache miss** means the real work had to be done.

Spring's **cache abstraction** lets you cache the result of a *method* declaratively:

```
@Cacheable("products")
public Product find(String sku) { ... }      // runs only if "products" has no entry for sku
```

It is an *abstraction*: the annotations talk to a `CacheManager` interface, and a **provider** does the storing. That can be a local in-memory cache such as **Caffeine**, or a shared one such as **Redis**. Switching providers is a dependency and configuration change,

not a code change.

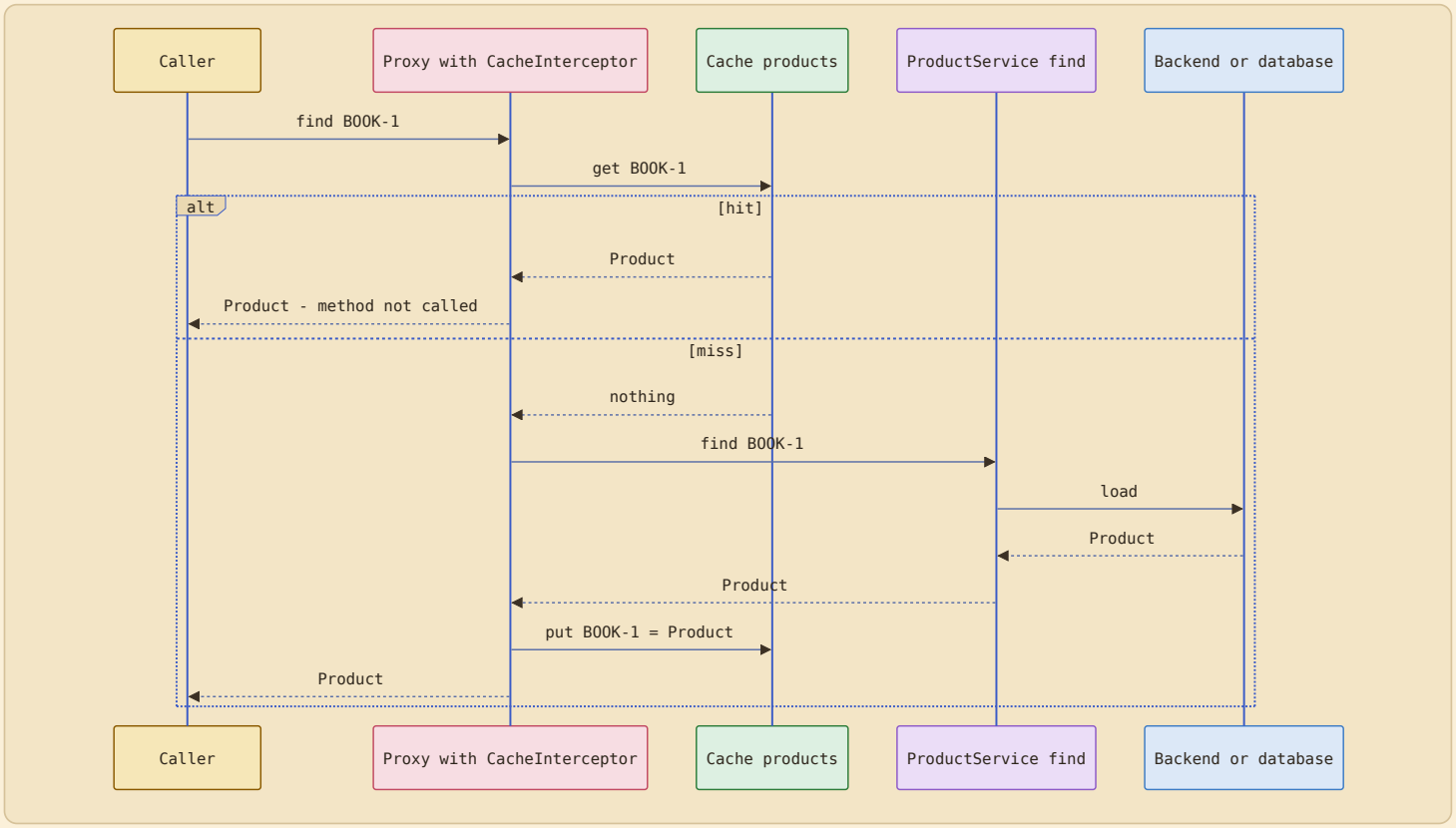
Why it matters

Reads usually far outnumber writes. A product page might be viewed 10,000 times between two price changes. Loading the product from the database every time wastes database capacity and adds latency (time per request). With a cache, one load serves thousands of requests. The price you pay is **staleness**: the cache may show old data until it is refreshed or evicted (removed from the cache). Every caching decision is about how stale is acceptable.

How it works

Caching is an AOP feature, built exactly like `@Transactional` (4.6).

1. `@EnableCaching` registers a `CacheInterceptor`. Every bean with cache annotations is wrapped in a **proxy**.
2. A call to `find("BOOK-1")` reaches the proxy first. The interceptor computes a **key** from the arguments (here `"BOOK-1"`).
3. It asks the `CacheManager` for the cache named `products`, and asks that cache for the key.
4. **Hit** → return the cached value. Your method does **not** run.
5. **Miss** → call your method, store the result under the key, return it.



This pattern, "look in the cache, and on a miss load and store", is called **cache-aside**. The system-design caching module compares it with read-through, write-through and write-behind.

Setup with Caffeine

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
<dependency>
  <groupId>com.github.ben-manes.caffeine</groupId>
  <artifactId>caffeine</artifactId>
</dependency>
```

```java
@SpringBootApplication
@EnableCaching                // without it every cache annotation is silently ignored
public class CachingApplication { ... }

@Configuration(proxyBeanMethods = false)
public class CacheConfig {

    /** The clock Caffeine uses for expiry. Production: Ticker.systemTicker(). Tests replace it. */
    @Bean
    Ticker cacheTicker() {
        return Ticker.systemTicker();
    }

    /** Boot's CaffeineCacheManager uses this builder for every cache it creates. */
    @Bean
    Caffeine<Object, Object> caffeine(Ticker cacheTicker) {
        return Caffeine.newBuilder()
                .maximumSize(10_000)                      // bound memory: evicts least-useful entries
                .expireAfterWrite(Duration.ofMinutes(10))   // TTL: entries are at most 10 minutes old
                .ticker(cacheTicker)
                .recordStats();                           // hit/miss counters (also used by Actuator metrics)
    }
}
```

```properties
# Only these caches exist (no caches created on the fly by a typo in a cache name)
spring.cache.cache-names=products,catalog,tags,reports-plain,reports-sync
```

How Boot chooses a provider: if you define no `CacheManager` bean yourself, Boot checks the providers in a fixed order (Generic, JCache, Hazelcast, Infinispan, Couchbase, **Redis**, **Caffeine**, Cache2k, **Simple**) and configures the first one it finds on the classpath. You can force one with `spring.cache.type`. **Simple** is a `ConcurrentHashMap` (Java's thread-safe hash map) with no expiry and no size limit, fine for demos but never for production.

For Caffeine, Boot applies the settings from, in order of precedence: `spring.cache.caffeine.spec` (e.g. `maximumSize=500,expireAfterWrite=10m`), a `CaffeineSpec` bean, or a `Caffeine` bean (used here, so that the test can swap the clock). **Caffeine** is the standard local cache for Java. Its eviction policy (W-TinyLFU) keeps the entries that are used most often and most recently, which gives better hit rates than a plain LRU (Least Recently Used: evict whatever was touched longest ago).

With `spring.cache.cache-names` set, the manager is **static**: only those caches exist, and using any other name fails with an error instead of quietly creating a new, unconfigured cache.

`@Cacheable` — hits and misses, measured

```java
@Service
@CacheConfig(cacheNames = "products")      // default cache name for the annotations in this class
public class ProductService {
    private final ProductBackend backend;   // stands in for a slow database; counts its calls

    @Cacheable(unless = "#result == null")          // do not cache "not found"
    public Product find(String sku) {
        return backend.load(sku);
    }
    // ...
}
```

```java
@Test
void cacheableRunsTheMethodOnlyOnAMiss() {
    CacheStats before = nativeCache("products").stats();          // Caffeine's own counters
    for (int i = 1; i <= 3; i++) {
        Product p = products.find("BOOK-1");
        System.out.println("call " + i + ": " + p.name() + "   backend loads so far = " + backend.loads.get());
    }
    CacheStats delta = nativeCache("products").stats().minus(before);
    // print cache manager class, hits, misses; assert loads = 1, hits = 2, misses = 1
}
```

```
call 1: Spring in Depth   backend loads so far = 1
call 2: Spring in Depth   backend loads so far = 1
call 3: Spring in Depth   backend loads so far = 1
cache manager: CaffeineCacheManager   hits=2 misses=1
```

Three calls, one real load. `nativeCache("products")` unwraps Spring's `CaffeineCache` to the underlying Caffeine cache, whose `recordStats()` counters prove what happened. In production, the same counters are published by Actuator as `cache.gets` metrics tagged `result=hit|miss` (ch15). For Caffeine they exist only when `recordStats()` is on. The `/actuator/caches` endpoint lists caches and can clear them.

`@CachePut` and `@CacheEvict` — keeping the cache correct

```java
/** ALWAYS runs the method, then writes the result into the cache (key = the saved product's sku). */
@CachePut(key = "#result.sku()")
public Product update(Product product) {
    return backend.store(product);
}

/** Runs the method, then removes that one key. */
@CacheEvict
public void delete(String sku) {
    backend.remove(sku);
}

/** Empties the whole "products" cache (e.g. after a bulk import). */
@CacheEvict(allEntries = true)
public void reloadAll() {}
```

```java
products.find("PEN-7");                                          // miss -> load 1
products.update(new Product("PEN-7", "Gel pen", new BigDecimal("3.00")));   // method runs + cache put
products.find("PEN-7");                                          // hit: the NEW price, no load
products.delete("PEN-7");                                        // method runs + key evicted
products.find("PEN-7");                                          // miss -> load 2 -> null
products.find("PEN-7");                                          // null was not cached -> load 3
products.find("BOOK-1");                                         // load 4
products.reloadAll();                                            // allEntries = true
```

```
after @CachePut:   3.00  loads=1
after @CacheEvict: null  loads=2
asked again:       loads=3  (null results are not cached)
after reloadAll(): entries in 'products' = 0
```

Annotation	Runs the method?	Effect on the cache	Typical use
`@Cacheable`	only on a miss	stores the result	reads
`@CachePut`	**always**	stores the result	updates, keeping the cache fresh
`@CacheEvict`	always	removes the key (or all entries with `allEntries = true`)	deletes; updates when you prefer "evict and reload"
`@Caching`	—	groups several of the above on one method (e.g. put in one cache, evict from another)	a method affecting several caches
`@CacheConfig`	—	class-level defaults: `cacheNames`, `keyGenerator`, `cacheManager`	less repetition

Details that matter:

- `@CacheEvict` runs **after** the method, and only if it returns normally. `beforeInvocation = true` evicts first, even if the method then throws.
- Do not put `@Cacheable` and `@CachePut` on the same method. One skips the method, the other always runs it; the Spring docs strongly discourage it.
- **Evict or put?** Evicting is safer: the next read loads fresh data from the database. `@CachePut` saves that read, but if two updates race, the cache can end up holding the older value.
- The key of `@CachePut` must match the key the readers use. Here readers use `sku`, so the put uses `#result.sku()`.

Keys: how they are generated, and how they collide

By default Spring uses `SimpleKeyGenerator`:

Parameters	Key
none	`SimpleKey.EMPTY`
exactly one	the parameter itself
two or more	`SimpleKey[p1, p2, …]` (uses each parameter's `equals`/`hashCode`)

The key does **not** include the method name. Two methods that share a cache and take the same argument therefore read each other's entries:

```java
@Service
public class CatalogService {

    @Cacheable("catalog")                              // key = sku  (default SimpleKeyGenerator)
    public String name(String sku) { return "name of " + sku; }

    @Cacheable("catalog")                              // SAME key = sku -> collides with name()!
    public String description(String sku) { return "description of " + sku; }

    @Cacheable(cacheNames = "catalog", key = "'desc:' + #sku")   // fix: make the key unique
    public String descriptionFixed(String sku) { return "description of " + sku; }

    @Cacheable(cacheNames = "catalog")                 // two parameters -> key = SimpleKey[sku, currency]
    public String price(String sku, String currency) { ... }

    @Cacheable(cacheNames = "catalog", key = "#sku", condition = "#currency == 'EUR'")
    public String priceEurOnly(String sku, String currency) { ... }   // condition: cache only EUR lookups
}
```

The test calls `name("A")`, `description("A")`, `descriptionFixed("A")`, `price("A", "EUR")`, `priceEurOnly("B", "USD")` and `priceEurOnly("C", "EUR")`, then prints the keys stored in the Caffeine cache:

```
name("A")             -> name of A
description("A")      -> name of A   <- WRONG: same cache, same key
descriptionFixed("A") -> description of A
key in 'catalog': A
key in 'catalog': C
key in 'catalog': SimpleKey [A, EUR]
key in 'catalog': desc:A
```

- `description("A")` returned the **name**. If the two methods returned different types, you would get a `ClassCastException` instead. Fix: one cache per method (the usual choice), a key prefix, or a custom `KeyGenerator` bean (referenced with `keyGenerator = "myKeyGen"`) that includes the method name.
- `B` is missing: `condition` was false for USD, so the method ran but nothing was cached.
- `key` is a **SpEL** expression (Spring Expression Language). `#sku` is a parameter by name (Boot compiles with `-parameters`, so names are available), `#root.methodName` is the method name, `#result` is the return value (in `unless` and `@CachePut` keys only).
- Parameters used in keys need proper `equals`/`hashCode`. Records have them. A mutable object used as a key that changes after being cached can never be found again.

`condition` vs `unless`

Attribute	Evaluated	Can use `#result`?	If it says "no"
`condition`	**before** the call	no	cache is bypassed completely: no lookup, no store
`unless`	**after** the call	yes	the result is not stored (the lookup still happened)

`@Cacheable(unless = "#result == null")` is the usual way to avoid caching "not found". Otherwise a lookup for a product that does not exist *yet* keeps returning `null` until the entry expires. For `Optional` return types, Spring caches the contained value (or `null`), and `#result` refers to that value, so write `unless = "#result == null"` there too.

Expiry (TTL) — tested with a fake clock

TTL (time to live) bounds staleness: after it passes, the entry is dropped and the next call reloads. Testing a 10-minute TTL by sleeping 10 minutes is not practical, so the test replaces Caffeine's `Ticker` (its clock) with one it can move:

```
class FakeTicker implements Ticker {
    private final AtomicLong nanos = new AtomicLong();
    @Override public long read() { return nanos.get(); }
    void advance(Duration d) { nanos.addAndGet(d.toNanos()); }
}

@SpringBootTest
class CachingTests {
    @TestConfiguration
    static class ClockConfig {
        @Bean @Primary FakeTicker fakeTicker() { return new FakeTicker(); }   // replaces the system ticker
    }

    @Test
    void entriesExpireAfterTheTtl() {
        products.find("BOOK-1");                                    // load 1, written at t=0
        ticker.advance(Duration.ofMinutes(9));
        products.find("BOOK-1");
        ticker.advance(Duration.ofMinutes(2));
        products.find("BOOK-1");
        assertThat(backend.loads).hasValue(2);
    }
}
```

```
t=9 min:  loads=1  (still fresh -> hit)
t=11 min: loads=2  (expired after 10 min -> miss -> reload)
```

Caffeine setting	Meaning	Use when
`expireAfterWrite(d)`	entry dies `d` after it was stored	data changes elsewhere; bounds staleness
`expireAfterAccess(d)`	entry dies `d` after its last read or write	keep hot entries, drop unused ones (staleness is **not** bounded)
`refreshAfterWrite(d)`	after `d`, the next read returns the old value and reloads in the background	needs a `CacheLoader` bean; avoids latency spikes
`maximumSize(n)` / `maximumWeight(w)`	size bound, evicts by W-TinyLFU	always: an unbounded cache is a memory leak

The Spring annotations have **no TTL attribute**. TTL is a provider setting, per cache manager or per cache.

Cache stampede and `sync = true` — measured

A **cache stampede** (also called *thundering herd* or *dog-piling*) happens when a popular entry is missing, because it expired or was never loaded, and many requests arrive at the same time. Each one misses, and each one runs the expensive computation. The database gets hit N times at the worst possible moment.

By default `@Cacheable` does nothing to prevent this. `sync = true` asks the provider to let **one** thread compute the value per key while the others wait for it.

```
@Cacheable("reports-plain")                       // every thread that misses computes the report
public String plain(String day) throws InterruptedException { ... }

@Cacheable(cacheNames = "reports-sync", sync = true)   // one thread computes, the others wait for it
public String synced(String day) throws InterruptedException { ... }
```

Both methods count their computations. The "expensive work" waits until all 8 callers are inside the method, or 300 ms at most, so the unsynchronised case really overlaps. The test submits 8 tasks at once to a thread pool (a fixed set of reusable worker threads):

```
8 concurrent callers, sync=false -> computed 8 times
8 concurrent callers, sync=true  -> computed 1 time, all got "report 2026-09-28"
```

With Caffeine, `sync = true` maps to Caffeine's atomic `get(key, loader)`: the first thread computes, and the others block on that key only. Other keys are not blocked.

Limits of `sync = true`: it cannot be combined with `unless`, it allows only one cache per annotation, it cannot be combined with other cache annotations on the same method, and it protects only **one JVM**. With 10 application instances you can still get 10 computations. Other stampede defences:

Technique	Idea
`sync = true` / `get(key, loader)`	one loader per key per instance
Random TTL jitter	entries written together do not all expire together
Refresh ahead (`refreshAfterWrite`)	reload *before* expiry, while still serving the old value
Distributed lock / single-flight (only one caller computes; the rest share its result) in Redis	one loader across all instances (costs complexity)
Pre-warming	load hot (frequently requested) keys at startup or on a schedule

More in the system-design caching module (section "Cache Stampede Prevention").

Pitfalls

1. Self-invocation: the cache is never consulted

A call from inside the same bean (`this.find(sku)`) does not pass through the proxy, so `@Cacheable` does nothing. The same applies to `private` methods (proxy mode only works on public methods) and to calls from a constructor or `@PostConstruct`. This was **proved** in 4.5: two internal calls ran the real method twice. Fix: move the cached method to another bean.

Related trap, hit while writing this chapter's test: reading a public **field** of the cached `ReportService` returned `null`, because the bean is a CGLIB proxy whose own fields are never initialised (also shown in 4.5). Access state on proxied beans through methods only.

2. Cached mutable objects are shared

A local cache like Caffeine stores a **reference** to your object. Every caller gets the *same instance*:

```
@Cacheable("tags")
public List<String> tags(String sku) {
    return new ArrayList<>(backend.tags(sku));
}

List<String> first = products.tags("BOOK-1");
first.add("hacked");                                  // caller changes "its" list...
List<String> second = products.tags("BOOK-1");        // ...another caller gets the same object
```

```
second caller sees: [new, sale, hacked]   same instance: true
```

One caller's change corrupted the cache for everyone, and nothing was written to the database. Cache **immutable** values: records, `List.copyOf`, unmodifiable collections. (A Redis cache stores a *serialized copy*, so this does not happen there. That is exactly why code that worked with Caffeine can behave differently after a switch to Redis, and the other way round.)

3. Caching inside a transaction

If `@CachePut` or `@CacheEvict` runs inside a `@Transactional` method that later rolls back, the cache already holds data that the database never committed. Options: wrap the manager in `TransactionAwareCacheManagerProxy` (puts and evicts are applied only after commit), use `RedisCacheManager.builder(...).transactionAware()`, or evict in an `@TransactionalEventListener(phase = AFTER_COMMIT)` (7.15).

4. Local caches in a cluster

With 5 instances, each has its own Caffeine cache. An update evicts the entry on the instance that handled it, and the other 4 keep serving the old value until their TTL expires. Either accept staleness up to the TTL (keep it short), use a shared cache (Redis), or broadcast evictions (e.g. Redis pub/sub — publish/subscribe messaging, where every subscriber gets each message). A common design is **two levels**: a small, short-TTL Caffeine cache in front of Redis.

5. Other mistakes

Mistake	Symptom	Fix
No `@EnableCaching`	annotations ignored, no error	add it
Caching everything	memory pressure, stale data everywhere, hard-to-reproduce bugs	cache only measured hot, expensive, rarely-changing reads
No size bound (or the `simple` provider in production)	memory grows until an `OutOfMemoryError` (the JVM has run out of heap memory)	`maximumSize`, a real provider
Caching per-user data under a key without the user	one user sees another's data	include the user or tenant in the key
Non-serializable values with Redis	`SerializationException` at runtime	JSON serializer, or `Serializable` types
Changing a cached class's shape with Redis	old entries fail to deserialize after deploy	versioned key prefix per release, or short TTL
Tests influenced by cached values from other tests	flaky tests (tests that pass or fail at random)	clear caches in `@BeforeEach` (as here), or disable caching with `spring.cache.type=none` / `@AutoConfigureCache`

Redis as the cache (snippet — not run)

With `spring-boot-starter-data-redis` on the classpath, Boot's detection order picks **Redis before Caffeine**. The annotations stay the same. Basic settings are properties:

```
spring.cache.cache-names=products,prices
spring.cache.redis.time-to-live=10m
spring.cache.redis.cache-null-values=false
# the prefix is followed by the cache name; bump v3 when cached classes change
spring.cache.redis.key-prefix=shop:v3:
```

For more control, define beans (`RedisCacheSetup.java`, compiled):

```
@Configuration(proxyBeanMethods = false)
public class RedisCacheSetup {

    /** Defaults for every Redis cache: JSON values instead of JDK serialization. */
    @Bean
    RedisCacheConfiguration redisCacheConfiguration() {
        var allowedTypes = BasicPolymorphicTypeValidator.builder()   // tools.jackson (Jackson 3)
                .allowIfSubType("ch09.")
                .allowIfSubType("java.")
                .build();
        return RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))
                .disableCachingNullValues()
                .serializeValuesWith(SerializationPair.fromSerializer(
                        GenericJacksonJsonRedisSerializer.builder().enableDefaultTyping(allowedTypes).build()));
    }

    /** Per-cache overrides on top of the defaults. */
    @Bean
    RedisCacheManagerBuilderCustomizer perCacheTtl(RedisCacheConfiguration defaults) {
        return builder -> builder
                .withCacheConfiguration("products", defaults.entryTtl(Duration.ofMinutes(30)))
                .withCacheConfiguration("prices", defaults.entryTtl(Duration.ofSeconds(30)));
    }
}
```

- Your own `RedisCacheConfiguration` bean **replaces** the one Boot would build from `spring.cache.redis.*`, so those properties no longer apply. Put everything in the bean.
- `RedisCacheManagerBuilderCustomizer` (package `org.springframework.boot.cache.autoconfigure` in Boot 4) adjusts the auto-configured manager, typically with per-cache TTLs.
- **Serialization:** the default is JDK serialization, which is binary, Java-only, and breaks when classes change. JSON is readable and survives small changes. Spring Data Redis 4 ships Jackson 3 serializers (`GenericJacksonJsonRedisSerializer`, `JacksonJsonRedisSerializer<T>`); the Jackson 2 ones are deprecated. The generic JSON serializer stores each value's class name, and the **type validator** restricts which classes may be created from it. Allowing any class is a known deserialization attack vector (an attacker crafts data that makes your app create and run dangerous classes). The builder's `enableUnsafeDefaultTyping()` is named that way for a reason.
- Every hit costs a network round trip plus deserialization (well under a millisecond on a local network, but not free). Cache larger, more expensive results in Redis, not tiny lookups.

Local vs shared cache

	Caffeine (local)	Redis (shared)
Speed	nanoseconds, no serialization	network hop + (de)serialization
Consistency across instances	each instance has its own copy	one copy for all
Survives restarts	no	yes (as far as Redis persistence is configured)
Stores	references (mutable-object trap)	serialized copies
Capacity	limited by the JVM heap (the memory area where Java objects live)	a separate server's memory
Operations cost	none	another system to run and monitor
Typical use	per-instance hot reference data, short TTLs	sessions, data shared between instances, expensive results

In the interview

30-second answer: "Spring's cache abstraction is proxy-based AOP. `@EnableCaching` adds an interceptor. `@Cacheable` checks the cache by a key made from the arguments and only runs the method on a miss. `@CachePut` always runs and stores the result, and `@CacheEvict` removes entries. The storage is a `CacheManager`: Caffeine for a local cache, Redis for a shared one. Boot auto-configures whichever is on the classpath. TTL and size are provider settings. The pitfalls are the proxy rules (no self-invocation, public methods only), key collisions between methods sharing a cache, mutable cached objects, stale data across instances, and stampedes, which `sync = true` mitigates within one JVM."

- *"How do you set a TTL on `@Cacheable`?"* → You don't. The annotation has no TTL. Configure it in the provider: `expireAfterWrite` in Caffeine, `spring.cache.redis.time-to-live` or `RedisCacheConfiguration.entryTtl` in Redis, per cache if needed.
- *"The cache returns stale data after an update. Why?"* → The update path does not evict or put the same key (key mismatch or a different cache name), the update happened on another instance with a local cache, or the eviction happened inside a transaction that rolled back.
- *"What is a cache stampede and how do you prevent it?"* → Many concurrent misses for the same hot key all hit the database. Use `sync = true` (one loader per JVM), TTL jitter, refresh-ahead, or a distributed lock for cluster-wide single-flight.
- *"`condition` vs `unless`?"* → `condition` is checked before the call and can bypass the cache entirely. `unless` is checked after the call, can see `#result`, and only vetoes storing.

Self-check

1. `@Cacheable` on a `private` method: what happens? → Nothing is cached; the proxy cannot intercept private methods.
2. Two `@Cacheable("x")` methods take one `Long id` each. What goes wrong? → Same key in the same cache, so each returns the other's cached value. Use separate caches or distinct keys.
3. How did the TTL test avoid sleeping? → It gave Caffeine a fake `Ticker` and advanced it by 9 and then 2 minutes.
4. Why did 8 threads compute the report 8 times without `sync`? → All 8 missed before any of them had stored the value; plain `@Cacheable` does not coordinate loaders.
5. Why can the mutable-list bug not happen with Redis? → Redis returns a freshly deserialized copy on every hit.

TL;DR

- `@EnableCaching` + `@Cacheable` (read), `@CachePut` (always run, store), `@CacheEvict` (remove). Proxy-based, so the AOP rules apply.
- Default key = the single argument, or `SimpleKey` of all arguments, **without the method name**.
- TTL, size and statistics are provider settings. Caffeine: `expireAfterWrite`, `maximumSize`, `recordStats`.
- Measured: 3 calls → 1 load (2 hits, 1 miss). A TTL of 10 min → reload at 11 min. 8 concurrent misses → 8 loads, or 1 with `sync = true`.
- Cache immutable values. Evict after commit. Local caches go stale across instances. Redis needs a deliberate serializer.

9.9 Choosing a data technology

Need	Choose
Relational data, rich object graph, team knows JPA	Spring Data JPA (ch08)
Relational data, clear aggregates, predictable SQL	**Spring Data JDBC**
Full SQL control, reports, bulk work	`JdbcClient` (ch07), or jOOQ
Non-blocking relational access	R2DBC (ch13)
Flexible, nested documents read as a whole	**MongoDB**
Sub-millisecond counters, sessions, rate limits, shared cache	**Redis**
Quick CRUD API over repositories, internal use	**Spring Data REST**
Faster repeated reads of rarely-changing data	**Caching** (Caffeine locally, Redis shared) — after measuring

---

9.10 Interview questions

Quick round; the section-level "In the interview" and "Self-check" blocks above cover `save` on existing aggregates, repeatable and edited migrations, default keys, TTL, `sync`, mutable values and transactions.

1. **What is an aggregate in Spring Data JDBC?** → A root entity plus the entities it owns, loaded and saved as one unit through one repository (9.2).
2. **Spring Data JDBC vs JPA?** → No persistence context, no lazy loading or dirty checking; eager aggregate loading; children rewritten on save; SQL is predictable.
3. **How do you reference another aggregate?** → By id: `AggregateReference<T, ID>` or a plain id field.
4. **Why did saving a `Product` with an assigned id insert nothing?** → Non-null id without `@Version` means "existing" → `UPDATE` of 0 rows. Use `@Version`, `Persistable` or `JdbcAggregateTemplate.insert`.
5. **What does Spring Data REST give you, and what is the risk?** → HAL CRUD endpoints for repositories without controllers. It exposes your domain model as the API and exports everything by default.
6. **Flyway vs Liquibase?** → SQL-file versioning vs changelogs with database-independent changes, rollbacks and preconditions (9.4).
7. **Where does the MongoDB connection go in Boot 4?** → `spring.mongodb.uri` (driver module). `spring.data.mongodb.*` holds Spring Data settings such as `auto-index-creation`.
8. **Does Boot configure MongoDB transactions?** → No. Declare a `MongoTransactionManager`, and run a replica set.
9. **How are Redis repository queries implemented?** → Through secondary-index sets for `@Indexed` properties; equality only.
10. **How do you test against real MongoDB/Redis?** → Testcontainers with `@Container @ServiceConnection` fields (9.7).
11. **How does `@Cacheable` work internally?** → Proxy + `CacheInterceptor` → key → `CacheManager` → `Cache` lookup; the method runs only on a miss.
12. **`@CachePut` vs `@Cacheable`?** → `@CachePut` always runs the method and stores the result; `@Cacheable` skips it on a hit.
13. **Why is my `@Cacheable` ignored?** → No `@EnableCaching`, self-invocation, non-public method, or the bean was created with `new` (ch04).
14. **Caffeine or Redis?** → Caffeine: fastest, per instance, stale across instances. Redis: shared, survives restarts, costs a network hop and serialization.

Chapter TL;DR

- **Spring Data** gives every store the same repository model. Only mapping and store features differ.
- **Spring Data JDBC** maps **aggregates** with no persistence context. Loading is eager, `save` writes immediately and rewrites child rows, other aggregates are referenced by id, and `@Version` on the root protects the whole aggregate. Assigned ids need `@Version` / `Persistable` / `insert()`.
- **Spring Data REST** turns repositories into HAL endpoints. Good for internal CRUD; restrict exports and secure them.
- **Flyway**: versioned SQL, Java and repeatable migrations with checksums (an edited migration stops startup). **Liquibase**: changelogs with rollbacks and preconditions; FSL-licensed since 5.0.
- **MongoDB**: documents, `MongoRepository`, `MongoTemplate` aggregations. Boot 4 uses `spring.mongodb.*` for the connection. **Redis**: templates for data structures, `@RedisHash` repositories for simple cases. Test both with Testcontainers + `@ServiceConnection`.
- **Caching**: `@Cacheable` / `@CachePut` / `@CacheEvict` over a `CacheManager`. Caffeine locally, Redis shared. TTL and size live in the provider. Watch for proxy rules, key collisions, mutable values, transactions, cluster staleness and stampedes (`sync = true` per JVM).

Next: Chapter 10 — Spring Security fundamentals protects the endpoints and data you have built so far.

Chapter 10 — Spring Security 1: Fundamentals

Every web application eventually has to answer two questions for every request: **who is calling?** and **are they allowed to do this?** Spring Security answers both with a chain of servlet filters (pieces of code the web server runs on every request before it reaches Spring MVC) that runs *before* your controllers. It also switches on protections you would otherwise forget — CSRF tokens (secret values that prove a form was sent by your own site, §10.8), security headers (response headers that turn on browser defences, §10.11), session-fixation defence (§10.10) — the moment you add the starter.

Spring Security has a reputation for being "magic". It is not. Once you can picture the filter chain, name the object that holds the current user, and tell a 401 (HTTP "I don't know who you are") from a 403 (HTTP "I know you, but no"), the rest is configuration. This chapter builds that picture with small projects whose tests prove each behaviour.

What you'll learn

1. What the starter does on its own (and why "everything is locked" is a feature).
2. The architecture: `DelegatingFilterProxy`, `FilterChainProxy`, `SecurityFilterChain`, the filters, `SecurityContextHolder`.
3. The lambda DSL (domain-specific language: a fluent, chained configuration API) in Spring Security 7 (what was removed and why), and several chains in one app.
4. Authentication: `AuthenticationManager`, `UserDetailsService`, form login vs HTTP Basic (username and password sent in a header on every request), 302 (HTTP redirect) vs 401.
5. Password storage: hashing, salt, work factor, bcrypt/Argon2 via `DelegatingPasswordEncoder`, upgrading old hashes on login.
6. Request authorization: matchers, **roles vs authorities**, **401 vs 403**, role hierarchy, `denyAll` as the safe default.
7. Method security: `@PreAuthorize`, `@PostAuthorize`, `@PostFilter` and the proxy rule behind them.
8. CSRF, CORS, sessions, logout and security headers — what Spring does and when to change it.
9. Testing all of it with `spring-security-test`.

All code lives in `code/` — three Spring Boot **4.1.1** projects (Spring Security **7.1.1**, Framework 7, Java 25):

Project	Shows
`security-defaults`	the starter with zero configuration
`security-basics`	filter chain, authentication, passwords, authorization, method security
`security-web-defenses`	two chains (API + browser app), CSRF on/off, CORS preflight, headers, sessions

Every output block is real `mvn -q -B -ntp verify` output with Spring's log lines trimmed to `...`. Hash values and timings change on every run.

> **Prerequisites.** Proxies (Chapter 4) explain how method security works. Spring MVC (Chapter 5) is where the controllers come from. The *browser* side of CORS (rules for letting one website's scripts call another site), CSRF and XSS (cross-site scripting: an attacker getting their JavaScript to run on your pages) is taught in the JavaScript handbook, Chapter 14. This chapter links there instead of repeating it. Tokens, JWT (JSON Web Token: a signed token carrying user details) and OAuth2 (the standard protocol for issuing access tokens) continue in Chapter 11.

10.1 Vocabulary and what the starter does by default

What it is

Spring Security is the Spring project for authentication, authorization and protection against common web attacks. Boot 4 adds it with `spring-boot-starter-security`. Seven terms carry the rest of the chapter:

Term	Plain meaning	Everyday analogy
Principal	the "who" — a user, a service, a device	the person at the door
Credentials	the proof — password, token, certificate	ID card
Authentication	checking the proof and deciding who the caller is	the guard checks the ID card
Authorization	deciding whether that caller may do *this* action	the guest list says "floors 1–3 only"
Authority (`GrantedAuthority`)	one permission string the caller holds, e.g. `report:read`	a stamp on the wristband
Role	an authority whose name starts with `ROLE_`, e.g. `ROLE_ADMIN`; a coarse group of permissions	"staff" wristband colour
Security context	the per-request holder of the current `Authentication`	the wristband you wear inside

In Spring terms, **authentication produces an `Authentication` object** (principal + authorities + "is authenticated" flag). **Authorization reads it.**

What happens with zero configuration

Add the starter, write one controller, and run the test in `security-defaults`:

```
@Test
void everythingIsLockedByDefault(CapturedOutput startupLog) throws Exception {
    // 1. Boot generated a user called "user" and logged its random password at startup
    String line = startupLog.getAll().lines()
            .filter(l -> l.contains("Using generated security password")).findFirst().orElseThrow();
    String password = line.substring(line.indexOf(':') + 1).trim();
    // 2. browser-style request -> redirect to the generated login page
    MvcTestResult browser = mvc.get().uri("/hello").header(HttpHeaders.ACCEPT, "text/html").exchange();
    assertThat(browser).hasStatus(302).hasRedirectedUrl("/login");
    // 3. API-style request -> 401 plus a Basic challenge
    MvcTestResult api = mvc.get().uri("/hello").header(HttpHeaders.ACCEPT, "application/json").exchange();
    assertThat(api).hasStatus(401);
    // 4. with the generated credentials it works
    MvcTestResult ok = mvc.get().uri("/hello").with(httpBasic("user", password)).exchange();
    assertThat(ok).hasStatusOk().hasBodyTextEqualTo("hello");
}
```

Output:

```
Using generated security password: 4030de5f-fb5d-4c46-bd81-7a0af09efd28

This generated password is for development use only. Your security configuration must be updated before running your application in
production.
...
startup log   : Using generated security password: <36-char UUID>
browser, anon : 302 -> /login
api, anon     : 401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
basic, user   : 200 body=hello
```

So, with no code at all, Boot and Spring Security:

1. Protect **every** URL, so any request must be authenticated.
2. Create one in-memory user named `user` with a random UUID (a random 128-bit identifier) password printed at startup. You can fix it with `spring.security.user.name`, `spring.security.user.password` and `spring.security.user.roles`, but only for experiments.
3. Enable **form login** (a generated `/login` page) and **HTTP Basic**.
4. Pick the response by client type. A browser asking for HTML gets a `302` redirect to `/login`. A client asking for JSON gets `401` with `WWW-Authenticate: Basic`. The `charset="UTF-8"` parameter is new in Spring Security 7.1.
5. Turn on CSRF protection, security headers, session-fixation protection and a logout endpoint.

As soon as you declare your own `SecurityFilterChain` bean, Boot's default chain backs off. As soon as you declare a `UserDetailsService` (or an `AuthenticationManager`/`AuthenticationProvider`), the generated user disappears.

Why "locked by default" matters

Secure by default means that forgetting something fails *closed*: a new endpoint nobody thought about is protected, not exposed. The price is the familiar "why does my new app return 401?" moment. The answer is always "because you have not told Spring who may call it yet".

Self-check

1. *You add the starter and your endpoint returns 401 in Postman (a desktop tool for sending API requests) but a login page in Chrome. Why?* → Chrome sends `Accept: text/html`, so Spring redirects it to the form login page. Postman's JSON-style request gets the HTTP Basic `401` challenge.
2. *Where does the password come from?* → Boot generates a random UUID at startup and logs it. Nothing is stored anywhere.
3. *What makes the generated user disappear?* → Defining your own `UserDetailsService`, `AuthenticationManager` or `AuthenticationProvider` bean (or an OAuth2 setup — Chapter 11).

TL;DR

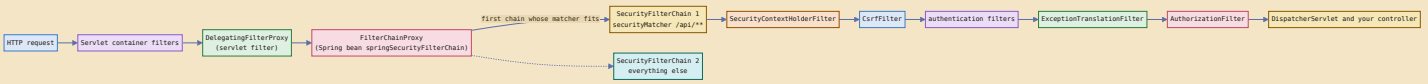
- Authentication = *who are you*, authorization = *may you*. The link between them is the `Authentication` object.
- The starter alone locks every URL, generates a dev-only user and enables form login, Basic, CSRF and headers.

10.2 Architecture: the filter chain

What it is

Spring Security is a **servlet filter** — code that sees every HTTP request before the `DispatcherServlet` does. It is not one filter doing everything. It is an ordered **list of small filters**, each with a single job. Think of an airport: one desk checks passports, the next checks boarding passes, the last one opens the gate. If any desk says no, you never reach the plane (your controller).

How it works



1. **`DelegatingFilterProxy`** is a plain servlet filter that Boot registers with the container. The container knows nothing about Spring beans, so this filter's only job is to hand the request to a Spring bean.
2. That bean is **`FilterChainProxy`**, named `springSecurityFilterChain`. It holds one or more **`SecurityFilterChain`**`s`. For each request it picks the **first** chain whose request matcher fits, and only that chain runs.
3. Each chain is an ordered list of **security filters**. The important ones:

Filter	Job
`SecurityContextHolderFilter`	loads the `SecurityContext` (e.g. from the HTTP session) into `SecurityContextHolder` for this request, and clears it afterwards
`HeaderWriterFilter`	adds security headers (§10.11)
`CorsFilter`	answers CORS preflights (§10.9), only if `cors()` is configured
`CsrfFilter`	rejects state-changing requests without a valid CSRF token (§10.8)
`LogoutFilter`	handles `POST /logout`
`UsernamePasswordAuthenticationFilter`	processes the login form `POST /login`
`BasicAuthenticationFilter`	reads `Authorization: Basic ...`
`BearerTokenAuthenticationFilter`	reads `Authorization: Bearer ...` (Chapter 11)
`AnonymousAuthenticationFilter`	if nobody authenticated, puts an *anonymous* `Authentication` in the context
`ExceptionTranslationFilter`	turns security exceptions from later filters into HTTP responses: 401/redirect or 403
`AuthorizationFilter`	the last gate: asks the `AuthorizationManager` whether this request is allowed

4. **`SecurityContextHolder`** stores the current `SecurityContext` in a `ThreadLocal` (a variable with a separate value for each thread; each request runs on its own thread) by default. Any code on the request thread — controller, service, logging — can read it with `SecurityContextHolder.getContext().getAuthentication()`.

The test in `security-basics` prints the real filter list of our chain:

```
@Autowired FilterChainProxy springSecurityFilterChain;

@Test
void printTheFilters() {
    List<SecurityFilterChain> chains = springSecurityFilterChain.getFilterChains();
    List<String> names = chains.getFirst().getFilters().stream().map(f -> f.getClass().getSimpleName()).toList();
    // ... print with numbers
    assertThat(names.getLast()).isEqualTo("AuthorizationFilter");
}
```

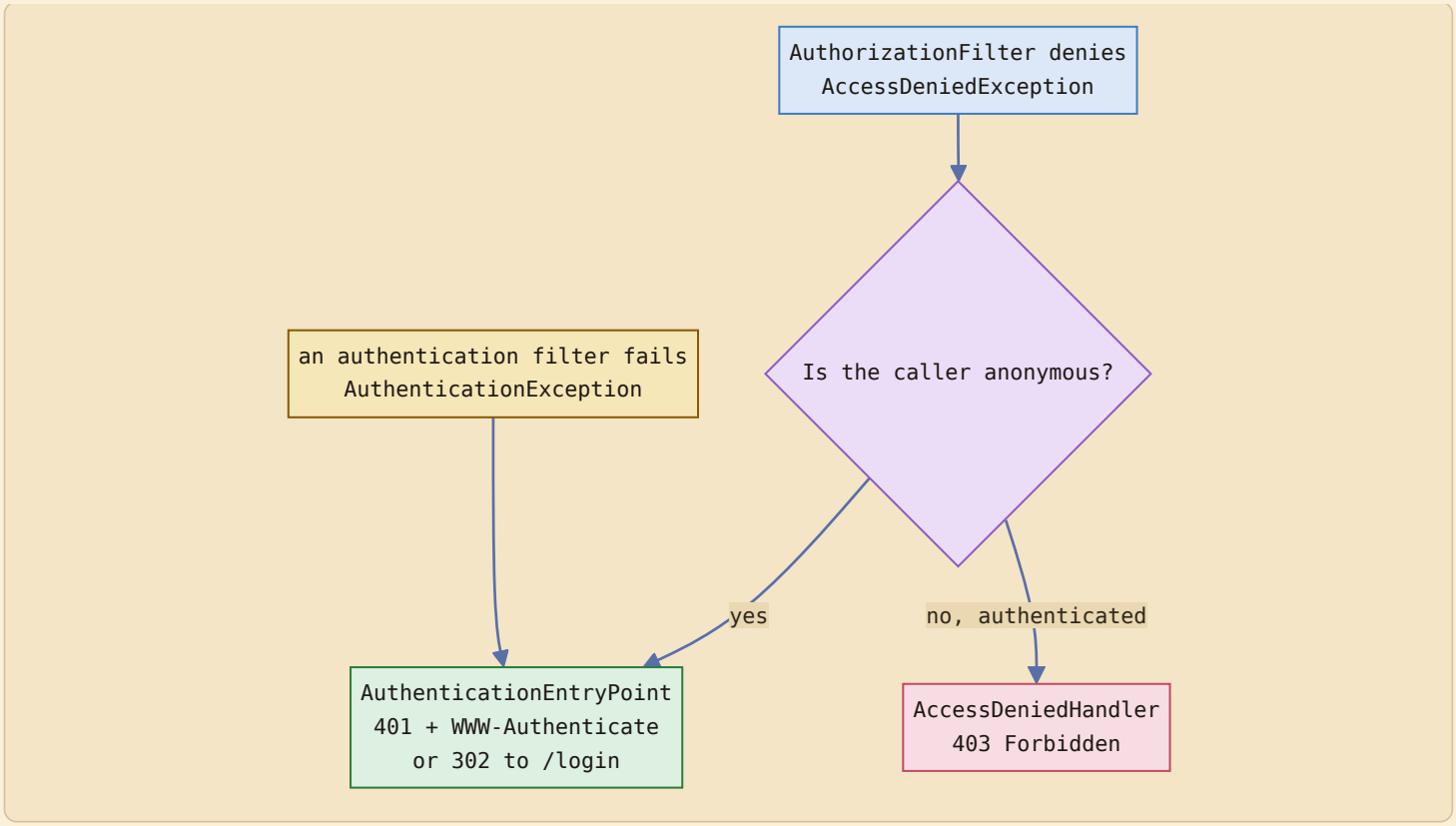
Output:

```
security filter chains: 1
 1 DisableEncodeUrlFilter
 2 WebAsyncManagerIntegrationFilter
 3 SecurityContextHolderFilter
 4 HeaderWriterFilter
 5 CsrfFilter
 6 LogoutFilter
 7 UsernamePasswordAuthenticationFilter
 8 DefaultResourcesFilter
 9 DefaultLoginPageGeneratingFilter
10 DefaultLogoutPageGeneratingFilter
11 BasicAuthenticationFilter
12 RequestCacheAwareFilter
13 SecurityContextHolderAwareRequestFilter
14 AnonymousAuthenticationFilter
15 ExceptionTranslationFilter
16 AuthorizationFilter
```

The list changes with your configuration. Removing `formLogin()` removes `UsernamePasswordAuthenticationFilter` and the generated login page, and adding `oauth2ResourceServer()` adds `BearerTokenAuthenticationFilter`. Printing it is the fastest way to understand what a chain really does. You can also set `logging.level.org.springframework.security=TRACE` and watch each filter run.

How a denied request becomes 401 or 403

`ExceptionTranslationFilter` sits just before `AuthorizationFilter` and catches what it throws:



- An `AuthenticationEntryPoint` "starts authentication". It is `401` with a challenge header for APIs, or a redirect to a login page for browsers. Before redirecting, the original request is saved in the **request cache**, so after login the user lands where they wanted to go.
- An `AccessDeniedHandler` answers `403` for callers who *are* known but lack permission.

Why it matters

- **Debugging.** "My endpoint returns 403 before my controller runs" is a filter problem, not an MVC problem. The filter list tells you which one.
- **Ordering.** Custom filters must go in the right place. Use `http.addFilterBefore(myFilter, BasicAuthenticationFilter.class)` and friends, never a guessed order number.
- **Threads.** The context lives in a `ThreadLocal`, so it does *not* follow work you hand to another thread. `@Async` tasks and executors need `DelegatingSecurityContextExecutor` (or the `DelegatingSecurityContext*` wrappers) to carry it over.

Common mistakes

- Registering your own security filter as a plain `@Component` `Filter`. Boot then *also* adds it to the servlet container, so it runs twice, once outside the security chain. Add it through `http.addFilter...` and disable the servlet registration with a `FilterRegistrationBean` whose `setEnabled(false)`.
- Expecting `SecurityContextHolder` to be filled inside a new thread, a `CompletableFuture` or a scheduled job.

Self-check

1. *Why does* `DelegatingFilterProxy` *exist?* → The servlet container creates filters itself and cannot see Spring beans. The proxy is a container filter that looks up the `springSecurityFilterChain` bean and delegates to it.
2. *Two chains both match* `/api/users`*. Which runs?* → Only the first one in order (`@Order`). Chains are never combined.
3. *Which filter decides 401 vs 403?* → `ExceptionTranslationFilter`: anonymous callers go to the entry point (401/redirect), authenticated ones to the access-denied handler (403).

TL;DR

Container → `DelegatingFilterProxy` → `FilterChainProxy` → the first matching `SecurityFilterChain` → ordered filters → `AuthorizationFilter` last. The current user lives in `SecurityContextHolder` (a `ThreadLocal`).

10.3 Configuring a chain: the lambda DSL in Spring Security 7

What it is

You configure a chain by declaring a `SecurityFilterChain` bean. You build it with the `HttpSecurity` **DSL** — a builder where each method configures one feature and takes a lambda (a `Customizer`). This is the whole configuration of `security-basics`:

```java
@Configuration
@EnableMethodSecurity // turns on @PreAuthorize / @PostAuthorize / @PreFilter / @PostFilter
class SecurityConfig {

    @Bean
    SecurityFilterChain appSecurity(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth                     // first match wins, top to bottom
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")        // checks authority "ROLE_ADMIN"
                .requestMatchers("/user/**").hasRole("USER")          // ADMIN also passes (hierarchy below)
                .requestMatchers(HttpMethod.GET, "/reports/**").hasAuthority("report:read")
                .requestMatchers("/me", "/orders/**").authenticated()
                .anyRequest().denyAll())                              // anything we forgot is closed
            .httpBasic(Customizer.withDefaults())                     // Authorization: Basic ... for APIs
            .formLogin(Customizer.withDefaults());                    // /login page for browsers
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

    @Bean
    static RoleHierarchy roleHierarchy() {
        return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER");
    }
}
```

- `Customizer.withDefaults()` means "turn this feature on with its default settings".
- `feature(f -> f.disable())` turns a feature off, e.g. `csrf(c -> c.disable())`.
- With Boot you do not need `@EnableWebSecurity`. Boot applies it for you. It is harmless to add.

What changed in Spring Security 7

Spring Security 7 finished several clean-ups that 6.x had deprecated. Old tutorials use the removed style, so it helps to recognise it:

Removed in 7	Use instead
`.and()` chaining (`http.csrf().disable().and().httpBasic()`)	one lambda per feature (above)
`authorizeRequests(...)`	`authorizeHttpRequests(...)` (built on `AuthorizationManager`)
`AntPathRequestMatcher`, `MvcRequestMatcher`	`PathPatternRequestMatcher` — the same `PathPattern` syntax Spring MVC uses. `requestMatchers("/x/**")` creates them for you
`WebSecurityConfigurerAdapter` (already gone in 6.0)	`SecurityFilterChain` beans
`AccessDecisionManager` / voters in the main jar	moved to the legacy `spring-security-access` module; prefer `AuthorizationManager`
OAuth2 *password* grant support	authorization code or client credentials (Chapter 11)

Other 7.0 additions worth knowing: built-in **multi-factor authentication** (MFA: requiring two or more kinds of proof, e.g. password plus a code; below and §10.4), `csrf(c -> c.spa())` for single-page apps (SPAs: browser apps that load once and then talk to the server only through JavaScript calls; §10.8), Password4j-based encoders (§10.5), and Spring Authorization Server (a server that issues OAuth2 tokens) now shipping *inside* Spring Security (Chapter 11).

Several chains in one application

Real apps often serve a JSON API *and* browser pages, and these need different rules. The API should be stateless (the server keeps no login state; every request carries its own credentials) with no CSRF. The browser pages should use a session cookie (a cookie holding the id of the user's server-side session) with CSRF on. Declare two chains and give each a `securityMatcher` (which requests this chain handles) and an `@Order`:

```java
@Bean @Order(1)
SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")                      // this chain only handles /api/**
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults())
        .cors(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable())                    // safe ONLY because nothing relies on cookies
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}

@Bean @Order(2)
SecurityFilterChain webChain(HttpSecurity http) throws Exception {   // no securityMatcher = everything else
    http.authorizeHttpRequests(auth -> auth.requestMatchers("/login").permitAll().anyRequest().authenticated())
        .formLogin(Customizer.withDefaults())
        .logout(Customizer.withDefaults());
    return http.build();
}
```

`securityMatcher` (which *requests* the chain handles) is different from `requestMatchers` inside `authorizeHttpRequests` (which *rules* apply within the chain). Mixing them up is a classic bug.

Common mistakes

- A chain **without** `securityMatcher` placed *before* a specific one. It matches everything, so the later chain never runs. Put catch-all chains last (highest `@Order` number).
- Copying `.and()` code from an old blog post. It no longer compiles in Spring Security 7.
- Declaring the `SecurityFilterChain` bean in a class that is not a `@Configuration`, or outside component scanning. Boot's default chain then stays active and nothing you wrote applies.

Self-check

1. *What does `Customizer.withDefaults()` do?* → It is a no-op lambda. It switches the feature on with default settings.
2. *Difference between `securityMatcher` and `requestMatchers`?* → `securityMatcher` selects which requests the whole chain handles. `requestMatchers` inside `authorizeHttpRequests` defines access rules for requests already in that chain.
3. *Your old code calls `authorizeRequests()`. What now?* → Rewrite it as `authorizeHttpRequests()`. The old API was removed in Spring Security 7.

TL;DR

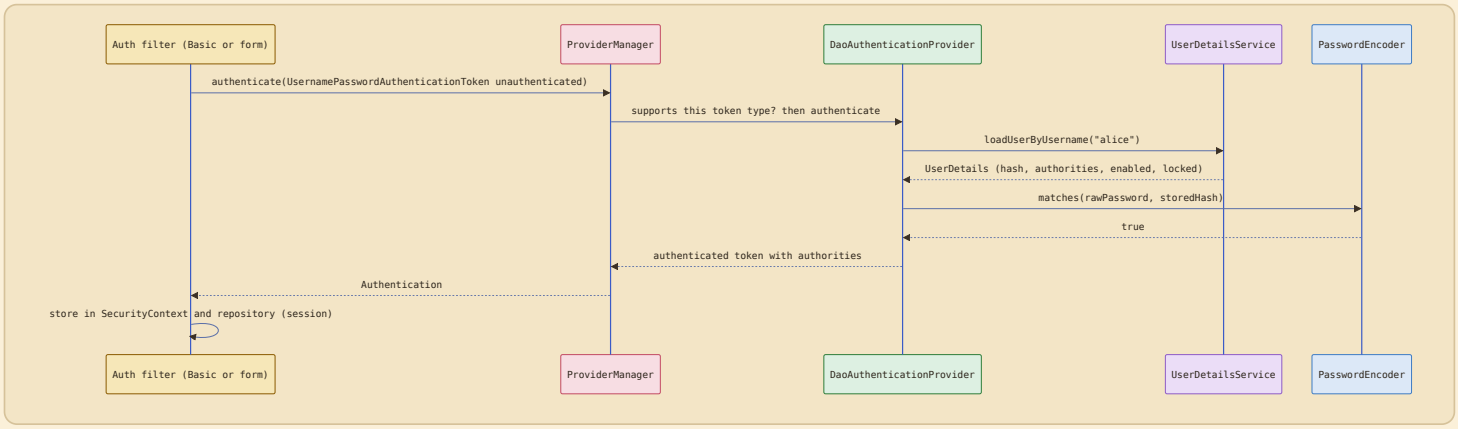
One `SecurityFilterChain` bean per chain, configured with one lambda per feature. Spring Security 7 removed `and()`, `authorizeRequests` and the Ant/Mvc matchers. Several chains = `securityMatcher` + `@Order`, specific first.

10.4 Authentication: users, providers, form login and Basic

What it is

Authentication turns *credentials* (username + password, a token, a certificate) into an `Authentication` that says who the caller is and what authorities they hold. Spring splits the work into small interfaces, so you can replace one piece without touching the rest.

How it works (username + password)



1. An **authentication filter** extracts credentials. `BasicAuthenticationFilter` reads the `Authorization` header. `UsernamePasswordAuthenticationFilter` reads the `POST /login` form. Each builds an *unauthenticated* token.
2. It calls the `AuthenticationManager`. The standard implementation, `ProviderManager`, asks each `AuthenticationProvider` in turn whether it `supports` that token type.

3. `DaoAuthenticationProvider` handles username/password. It loads the user through your `UserDetailsService`, checks the account flags (enabled, not locked, not expired), and verifies the password with the `PasswordEncoder`.
4. On success the filter stores the authenticated token in the `SecurityContext`. Form login also saves it in the HTTP session, so the next request is recognised via the session cookie. Basic auth sends credentials on *every* request instead.
5. On failure an `AuthenticationException` is thrown. A wrong password and an unknown user both surface as `BadCredentialsException`, so an attacker cannot tell which usernames exist. For an unknown user, the provider still runs a password check against a dummy hash, so both cases take about the same time.

`UserDetailsService`: your user store behind one method

`UserDetailsService` has a single method, `loadUserByUsername(String)`. Behind it can sit a database table, LDAP (a directory protocol companies use to store employee accounts), or anything else. `security-basics` implements it over a map (in a real app, a JPA repository — Chapter 8):

```
@Service
public class AccountUserDetailsService implements UserDetailsService, UserDetailsPasswordService {

    public record Account(String username, String passwordHash, List<String> authorities, boolean enabled) {}
    private final Map<String, Account> accounts = new ConcurrentHashMap<>();

    AccountUserDetailsService(PasswordEncoder encoder) {
        save(new Account("alice", encoder.encode("alice-pass"), List.of("ROLE_USER", "report:read"), true));
        save(new Account("bob", encoder.encode("bob-pass"), List.of("ROLE_ADMIN"), true));
        save(new Account("carol", encoder.encode("carol-pass"), List.of("ROLE_USER"), true));
        save(new Account("dave", "{noop}dave-pass", List.of("ROLE_USER"), true));    // legacy plain-text row
        save(new Account("erin", encoder.encode("erin-pass"), List.of("ROLE_USER"), false)); // disabled
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        Account a = accounts.get(username);
        if (a == null) throw new UsernameNotFoundException(username);
        return User.withUsername(a.username()).password(a.passwordHash())
                .authorities(a.authorities().toArray(String[]::new))
                .disabled(!a.enabled()).build();
    }

    @Override   // see 10.5: called when a stored hash should be upgraded
    public UserDetails updatePassword(UserDetails user, String newEncodedPassword) { /* save new hash */ }
}
```

Because this bean exists together with a `PasswordEncoder` bean, Spring Security builds the `DaoAuthenticationProvider` for you. No extra wiring is needed. Ready-made implementations include `InMemoryUserDetailsManager` (tests and demos), `JdbcUserDetailsManager` (Spring's own `users`/`authorities` table schema), and LDAP support.

Form login vs HTTP Basic

	Form login	HTTP Basic
Credentials sent	once, in `POST /login`	on **every** request, Base64 (a reversible text encoding) in `Authorization` (encoded, **not encrypted**, so HTTPS — HTTP over an encrypted TLS connection — only)
Remembered by	HTTP session cookie (`JSESSIONID`)	nothing, the client resends them
Unauthenticated response	`302` redirect to `/login`	`401` + `WWW-Authenticate: Basic realm=...`
Logout	`POST /logout` invalidates the session	impossible server-side (browser caches credentials)
Good for	server-rendered browser apps	scripts, service-to-service in trusted networks, quick demos

When both are enabled, Spring picks the entry point per request. HTML-accepting browsers get the redirect, and other clients (for example `Accept: application/json`) get the `401` challenge. The tests show every case:

```
MvcTestResult noCreds  = mvc.get().uri("/me").header("Accept", "application/json").exchange();
MvcTestResult wrong    = mvc.get().uri("/me").with(httpBasic("alice", "nope")).exchange();
MvcTestResult disabled = mvc.get().uri("/me").with(httpBasic("erin", "erin-pass")).exchange();
MvcTestResult ok       = mvc.get().uri("/me").with(httpBasic("alice", "alice-pass")).exchange();
// formLogin() builds POST /login with username, password AND a valid CSRF token
mockMvc.perform(formLogin().user("alice").password("alice-pass")).andExpect(authenticated().withUsername("alice"));
mockMvc.perform(formLogin().user("alice").password("wrong")).andExpect(unauthenticated());
```

Output:

```
no credentials (JSON)        401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
alice, wrong password        401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
erin, disabled account       401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
alice, right password        200 {"name":"alice","principalType":"User","authorities":["FACTOR_PASSWORD","ROLE_USER","report:read"]}
no credentials (HTML)        302 -> /login
form login, good password   302 -> /
form login, bad password    302 -> /login?error
```

Reading the current user

The `/me` endpoint shows three ways to reach the logged-in user:

```
@GetMapping("/me")
Me me(Authentication auth, @AuthenticationPrincipal UserDetails principal) {
    return new Me(auth.getName(), principal.getClass().getSimpleName(),
            auth.getAuthorities().stream().map(GrantedAuthority::getAuthority).sorted().toList());
}
```

- Declare an `Authentication` (or `Principal`) parameter and Spring MVC fills it in.
- `@AuthenticationPrincipal` gives you the principal object itself, here the `UserDetails` from our service.
- Anywhere else, use `SecurityContextHolder.getContext().getAuthentication()`, but prefer passing the user in explicitly. Code that reaches into a `ThreadLocal` is harder to test.

The `FACTOR_PASSWORD` authority: multi-factor authentication in Spring Security 7

Look at alice's authorities in the output: besides `ROLE_USER` and `report:read`, there is `FACTOR_PASSWORD`. Spring Security 7 adds a **factor authority** (`FactorGrantedAuthority`) for every authentication mechanism a user completes. Password login adds `FACTOR_PASSWORD`, one-time-token login (a single-use code, e.g. sent by e-mail) adds `FACTOR_OTT`, and passkeys (WebAuthn: password-less login with a key stored on the user's device) add `FACTOR_WEBAUTHN`. Authorization rules can then demand several factors, for example "`/admin/**` needs `ROLE_ADMIN` **and** both a password and a one-time token". You turn this on with `@EnableMultiFactorAuthentication(authorities = ...)` or selectively with `AuthorizationManagerFactories.multiFactor()`. If a factor is missing, Spring sends the user to the login step for that factor. The takeaway for now: **do not be surprised by `FACTOR_*` entries**, and filter them out if you copy authorities into a token (Chapter 11 does exactly that).

Common mistakes

- Returning `null` from `loadUserByUsername`. The contract is to throw `UsernameNotFoundException`.
- Telling users "unknown username" vs "wrong password". It lets attackers enumerate accounts (find out which usernames exist by trying many). Spring deliberately reports both the same way.
- Storing the `UserDetails` (with its password hash) in logs or sending it to the client. Map it to a DTO.
- Using HTTP Basic over plain HTTP. Base64 is readable by anyone on the path.

In the interview

30-second answer: "An authentication filter extracts credentials and hands an unauthenticated token to the `AuthenticationManager`. `ProviderManager` delegates to a provider that supports the token — for passwords, `DaoAuthenticationProvider`, which loads the user via `UserDetailsService` and verifies the hash with the `PasswordEncoder`. The authenticated result goes into the `SecurityContext`, and form login persists it in the session."

- *Why is `UsernameNotFoundException` turned into `BadCredentialsException`?* → To prevent username enumeration. The provider also spends a dummy password check so timing does not leak it either.
- *How would you add login by API key (a long secret string a client sends to identify itself)?* → Write a filter (or an `AuthenticationConverter`) that builds an `ApiKeyAuthenticationToken`, plus an `AuthenticationProvider` that validates it. Register the filter with `addFilterBefore(..., BasicAuthenticationFilter.class)`.

Self-check

1. *Which object checks the password?* → `DaoAuthenticationProvider`, using the `PasswordEncoder` on the hash that `UserDetailsService` returned.
2. *Why can't you "log out" of HTTP Basic?* → There is no server-side session. The browser caches and resends the credentials itself.
3. *What is `FACTOR_PASSWORD`?* → A Spring Security 7 factor authority recording that the user authenticated with a password. It is used for multi-factor rules.

TL;DR

Filter → `AuthenticationManager` (`ProviderManager`) → `AuthenticationProvider` (`DaoAuthenticationProvider`) → `UserDetailsService` + `PasswordEncoder` → `Authentication` in the `SecurityContext`. Browsers: form login + session. Scripts: Basic (HTTPS only). Tokens: Chapter 11.

10.5 Password storage: hashing with `DelegatingPasswordEncoder`

What it is

You must never store passwords, not even encrypted. Store a **password hash**: the output of a one-way function (easy to compute from the input, practically impossible to reverse) that is cheap to check once but expensive to compute billions of times. If the database leaks, attackers get hashes they must crack one guess at a time.

Three ideas make a hash suitable for passwords:

Idea	Meaning	Why
One-way	you can't compute the password from the hash	a leak does not reveal passwords directly
Salt	a random value stored with each hash and mixed in	equal passwords get different hashes, and precomputed "rainbow tables" are useless
Work factor (cost)	a tunable amount of CPU and/or memory per check	makes each guess slow. Raise it as hardware gets faster

General-purpose hashes like SHA-256 or MD5 (fast hash functions built for checksums and signatures) are designed to be *fast*, which is exactly wrong for passwords. Use an **adaptive password hash**:

Algorithm	Spring encoder	Notes
bcrypt	`BCryptPasswordEncoder` (default strength 10)	CPU-hard, battle-tested. Uses only the first **72 bytes** of the password
Argon2id	`Argon2PasswordEncoder` (needs Bouncy Castle `bcprov-jdk18on`)	memory-hard (resists GPU/ASIC — graphics cards and custom chips that try billions of guesses — cracking). Winner of the Password Hashing Competition; OWASP's (Open Worldwide Application Security Project, a respected web-security community) first choice
scrypt	`SCryptPasswordEncoder` (Bouncy Castle)	memory-hard
PBKDF2	`Pbkdf2PasswordEncoder`	choose it when FIPS-140 (a US government standard for approved cryptography) compliance is required

Spring Security 7.0 also added **Password4j-based** encoders (`Argon2Password4jPasswordEncoder`, `BcryptPassword4jPasswordEncoder`, `ScryptPassword4jPasswordEncoder`, `Pbkdf2Password4jPasswordEncoder`, `BalloonHashingPassword4jPasswordEncoder`) for teams that prefer that library.

How `DelegatingPasswordEncoder` works

Algorithms age: MD5 was once normal, and today bcrypt is being replaced by Argon2. `DelegatingPasswordEncoder` lets one database hold hashes from several generations. Each stored value carries its algorithm id as a prefix:

```
{bcrypt}$2a$10$oExJt1cCgjfvb4DzBrmNo.VLE45g8zG8GLlEik43AL6a3Xbk8q20a
{argon2@SpringSecurity_v5_8}$argon2id$v=19$m=16384,t=2,p=1$...
{noop}dave-pass                      <- legacy plain text (never do this for real)
```

1. **`encode(raw)`** always uses the *current default* id. `PasswordEncoderFactories.createDelegatingPasswordEncoder()` uses `bcrypt`.
2. **`matches(raw, stored)`** reads the `{id}` prefix and delegates to that algorithm's encoder. Old hashes keep working.
3. **`upgradeEncoding(stored)`** returns `true` when the stored hash is not in the current default format. `DaoAuthenticationProvider` calls it after every successful login. If it returns true *and* your `UserDetailsService` also implements **`UserDetailsPasswordService`**, the provider re-hashes the password it just received and calls `updatePassword(...)`. Users migrate silently, one login at a time.

`PasswordEncodingTest` (a plain unit test, no Spring context) shows all of this:

```java
PasswordEncoder enc = PasswordEncoderFactories.createDelegatingPasswordEncoder();
String h1 = enc.encode("s3cret");
String h2 = enc.encode("s3cret");
// ...
String argon = "{argon2@SpringSecurity_v5_8}" + Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8().encode("s3cret");
enc.matches("s3cret", argon);          // true: same encoder reads other algorithms
enc.upgradeEncoding("{noop}s3cret");   // true
```

Output:

```
encode #1 : {bcrypt}$2a$10$oExJt1cCgjfvb4DzBrmNo.VLE45g8zG8GLlEik43AL6a3Xbk8q20a
encode #2 : {bcrypt}$2a$10$oYZZ7zXC3LyJEl6nbQ9KbuQxmg1.oblxK/JPZ0/EufIiGHGFWX2ai
same input, different hash (random salt): true
matches(s3cret) = true, matches(S3cret) = false
argon2 row: {argon2@SpringSecurity_v5_8}$argon2id$v=19$m=16384,t=2,p=1$0...
matches(argon2 row) = true
upgradeEncoding({noop})  = true
upgradeEncoding(argon2)  = true
upgradeEncoding(bcrypt)  = false
```

Read a bcrypt value left to right: `$2a$` is the variant, `10` is the cost (2^10 rounds), then 22 characters of salt and 31 of hash. The salt is inside the string, so no separate column is needed. Note that `upgradeEncoding(argon2)` is `true` here only because this encoder's *default* is bcrypt. "Upgrade" just means "not the current default".

Choosing Argon2 as the default

To hash new passwords with Argon2 while still accepting everything already stored, build your own `DelegatingPasswordEncoder`:

```
Map<String, PasswordEncoder> known = Map.of(
        "argon2@SpringSecurity_v5_8", Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8(),
        "bcrypt", new BCryptPasswordEncoder(),
        "noop", NoOpPasswordEncoder.getInstance());          // only to READ legacy rows
var enc = new DelegatingPasswordEncoder("argon2@SpringSecurity_v5_8", known);
```

Output:

```
argon2 default: {argon2@SpringSecurity_v5_8}$argon2id$v=19$m=16384,t=2,p=1$Y...
old bcrypt row still matches: true, needs upgrade: true
```

`m=16384,t=2,p=1` means 16 MiB (mebibytes, 1 MiB = 1024 × 1024 bytes) of memory, 2 passes and 1 lane (parallel thread of work). Those are Spring's `v5_8` defaults. OWASP's Password Storage Cheat Sheet currently suggests somewhat more memory for Argon2id (for example 19 MiB with 2 passes). Use the `Argon2PasswordEncoder(saltLength, hashLength, parallelism, memory, iterations)` constructor if you follow it. If you change parameters under the same id, `upgradeEncoding` also notices weaker stored parameters. Always measure the cost on your production hardware.

The work factor in numbers

```
bcrypt strength 10 -> $2a$10$...  ~59 ms per check
bcrypt strength 12 -> $2a$12$...  ~239 ms per check
```

Each +1 of strength doubles the work (here about 4x for +2). A common target is **"as slow as your login latency budget allows"**, often several hundred milliseconds. Remember the cost hits *your* servers on every login. Slow hashing plus no rate limiting (capping how many attempts a client may make per second) makes the login endpoint an easy denial-of-service (overloading a service so real users cannot use it) target (see the rate-limiting note in Chapter 11).

bcrypt's 72-byte limit

```
73-byte password -> IllegalArgumentException: password cannot be more than 72 bytes
```

bcrypt only ever looked at the first 72 bytes, and current Spring Security versions refuse longer input instead of silently truncating it. If you allow long passphrases, validate the length in your sign-up form or choose Argon2.

Upgrading on login — proved

`dave` is stored as `{noop}dave-pass`. After one successful login his row holds a bcrypt hash:

```
System.out.println("before login: " + accounts.find("dave").passwordHash());
assertThat(mvc.get().uri("/me").with(httpBasic("dave", "dave-pass"))).hasStatusOk();
String after = accounts.find("dave").passwordHash();
assertThat(after).startsWith("{bcrypt}$2a$10$");
```

Output:

```
before login: {noop}dave-pass
after login : {bcrypt}$2a$10$6G6hiKY...
```

Common mistakes

- **Encrypting** passwords (AES, a standard two-way encryption algorithm) "so we can show them to support staff". Anyone with the key can decrypt them all. Hash them instead.
- A hand-made `sha256(password + "pepper")` (a pepper is a fixed secret mixed into every hash). It is fast, so it cracks quickly.
- Declaring `NoOpPasswordEncoder` as the bean "for now". It is deprecated for a reason.
- Storing hashes without the `{id}` prefix, then switching encoders later. Every old row breaks. If you have such rows, `DelegatingPasswordEncoder.setDefaultPasswordEncoderForMatches(...)` tells Spring how to read unprefixed values.

- Forgetting that the prefix makes the hash column longer. bcrypt is 60 characters plus `{bcrypt}`, and Argon2 values are longer still. Use `VARCHAR(255)` or more.

In the interview

> **30-second answer:** "Store only adaptive, salted hashes such as bcrypt or Argon2id, never encrypted or plain passwords. In Spring I use `DelegatingPasswordEncoder`: each hash is stored with an `{id}` prefix, new passwords use the current default, old formats still verify, and with `UserDetailsPasswordService` hashes are upgraded transparently on the next successful login."

- *Why a salt?* → Equal passwords get different hashes, and precomputed tables become useless. The salt is not secret.
- *bcrypt or Argon2?* → Both are fine. Argon2id is memory-hard, which makes GPU cracking more expensive, so it is OWASP's first choice. bcrypt is ubiquitous but has the 72-byte limit.
- *What is a pepper?* → A secret added to every hash and stored *outside* the database (in a vault — a secure service for storing secrets). It helps only if the database leaks without the app secrets. It is optional.

TL;DR

Hash, don't encrypt. Use an adaptive, salted, slow algorithm (Argon2id or bcrypt) through `DelegatingPasswordEncoder` (`{id}hash`). Implement `UserDetailsPasswordService` so old hashes upgrade on login. Tune the cost to your hardware.

10.6 Authorizing requests: roles, authorities, 401 vs 403

What it is

`authorizeHttpRequests` is a list of **rules**, each pairing a request matcher with a requirement. For each request, `AuthorizationFilter` walks the list **top to bottom** and applies the **first rule that matches**. The rules you write most often:

Rule	Passes when
`permitAll()`	always (still goes through the other filters, e.g. CSRF)
`denyAll()`	never
`authenticated()`	caller is not anonymous
`hasAuthority("report:read")`	caller holds exactly that authority string
`hasAnyAuthority("a", "b")`	holds at least one
`hasRole("ADMIN")`	holds authority `ROLE_ADMIN` (the prefix is added for you)
`hasAnyRole("ADMIN", "OPS")`	holds `ROLE_ADMIN` or `ROLE_OPS`
`access(authorizationManager)`	your own `AuthorizationManager` decides, e.g. IP ranges (blocks of network addresses) or time of day

Matchers can take HTTP methods (`requestMatchers(HttpMethod.GET, "/reports/**")`). They use the same `PathPattern` syntax as Spring MVC mappings: `*` matches one path segment and `**` matches any number of segments.

Roles vs authorities

Under the hood **there is only one concept: `GrantedAuthority`, a string**. A "role" is just the convention that the string starts with `ROLE_`.

	Role	Authority (permission)
Example	`ROLE_ADMIN`	`report:read`, `order:refund`
Granularity	coarse: *who the user is*	fine: *what the user may do*
Checked with	`hasRole("ADMIN")`	`hasAuthority("report:read")`
Typical source	user's group membership	role-to-permission mapping, OAuth2 scopes (permissions granted to a token, e.g. `read`; `SCOPE_...`, Chapter 11)

Many teams check **authorities** in code and assign them to users *through* roles. Then a role change doesn't mean a code change. The prefix trips people up in two ways, and Spring rejects both:

```
hasRole("ROLE_ADMIN") -> ROLE_ADMIN should not start with ROLE_ since ROLE_ is automatically prepended when using hasRole. Consider using hasAuthority instead.
User.roles("ROLE_ADMIN") -> ROLE_ADMIN cannot start with ROLE_ (it is automatically added)
```

`User.withUsername(...).roles("ADMIN")` stores `ROLE_ADMIN`. `.authorities("ROLE_ADMIN", "report:read")` stores the strings exactly as given. The two setters **replace** each other, so whichever you call last wins. Our users therefore use `authorities(...)` for mixed lists.

401 vs 403 — proved

Status	Name	Meaning	Client should
401	Unauthorized (really "unauthenticated")	"I don't know who you are"	authenticate, then retry
403	Forbidden	"I know who you are, and the answer is no"	not retry with the same identity

The matrix test calls five URLs as four callers:

```
assertThat(status("/admin/stats", null)).isEqualTo(401);      // who are you?
assertThat(status("/admin/stats", "carol")).isEqualTo(403);   // I know you, but no
assertThat(status("/reports/daily", "alice")).isEqualTo(200); // authority, not role
assertThat(status("/reports/daily", "bob")).isEqualTo(403);   // ADMIN role is not report:read
assertThat(status("/user/profile", "bob")).isEqualTo(200);    // role hierarchy ADMIN > USER
assertThat(status("/nowhere", "bob")).isEqualTo(403);         // denyAll catch-all
```

Output:

```
                          /public/hello  /user/profile  /admin/stats   /reports/daily /nowhere
anonymous                 200            401            401            401           401
alice (ROLE_USER, report:read) 200       200            403            200           403
carol (ROLE_USER)         200            200            403            403           403
bob (ROLE_ADMIN)          200            200            200            403           403
```

Read it row by row:

- **anonymous** gets 401 everywhere except the public URL, even for `/nowhere`. A denied *anonymous* caller is always asked to authenticate first (§10.2).
- **alice** reaches `/reports/daily` because she holds the `report:read` **authority**, although she is not an admin.
- **bob** is an admin but gets **403** on `/reports/daily`. A role does not imply permissions you never mapped to it.
- **bob** reaches `/user/profile` because of the **role hierarchy** bean `ROLE_ADMIN > ROLE_USER`. Spring treats him as also holding `ROLE_USER`, both in `authorizeHttpRequests` and in method security.
- `/nowhere` has no controller, yet it is still 403 for logged-in users, because `anyRequest().denyAll()` closes everything not listed. Security runs before Spring MVC looks for a handler.

Why `denyAll()` as the last rule

With `anyRequest().authenticated()` as the last rule, any endpoint someone adds next month is open to *every logged-in user*. With `denyAll()`, it stays closed until someone writes a rule, so mistakes fail closed. Pick consciously: `authenticated()` is common and fine for internal apps, and `denyAll()` is safer for anything customer-facing.

Common mistakes

- **Rule order.** `requestMatchers("/**").authenticated()` first makes every later rule dead code. Write rules from most specific to least specific.
- **Securing only the URL.** `/orders/{id}` is protected, but can alice read order 2, which belongs to carol? That question needs the *data* and belongs in method security or in the query ("insecure direct object reference": letting a user reach someone else's record just by changing an id in the URL; §10.7).
- Returning **404** for "exists but you may not see it" is sometimes a good idea, because it hides existence. Decide it deliberately. Spring gives 403 by default.
- Relying on `permitAll()` to *skip* security. The request still passes CSRF and header filters. To bypass the chain for static files, give them their own chain with `permitAll()`. Avoid `WebSecurityCustomizer.ignoring()`, which also drops the security headers.

In the interview

30-second answer: "401 means the server doesn't know who you are, so authenticate. 403 means it knows and refuses. In Spring, an `AccessDeniedException` for an anonymous user goes to the `AuthenticationEntryPoint` (401 or login redirect), and for an authenticated user to the `AccessDeniedHandler` (403). Roles are authorities with a `ROLE_` prefix. I check fine-grained authorities and map them to roles."

- *`hasRole("ADMIN")` vs `hasAuthority("ADMIN")`?* → The first checks `ROLE_ADMIN`, the second checks the literal string `ADMIN`.
- *How do you make admins inherit user permissions?* → A `RoleHierarchy` bean (`RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER")`).

Self-check

1. *Carol (USER) calls `/admin/stats`. Status?* → 403. She is authenticated but lacks `ROLE_ADMIN`.
2. *Same URL without credentials?* → 401 (or a login redirect for a browser).
3. *Why does bob (ADMIN) get 403 on `/reports/daily`?* → The rule requires the `report:read` authority, and bob only has `ROLE_ADMIN` (plus `ROLE_USER` via the hierarchy).

TL;DR

First matching rule wins, so write specific rules first and finish with `denyAll()` (or `authenticated()`). A role is an authority starting with `ROLE_`, and `hasRole` adds the prefix. 401 = unknown caller, 403 = known but not allowed.

10.7 Method security

What it is

URL rules protect *endpoints*. **Method security** protects *business methods*, wherever they are called from: a controller, a message listener, a scheduled job. It can also look at the method's **arguments and return value**, which URL rules cannot. `@EnableMethodSecurity` turns it on.

Annotation	When it runs	Typical use
`@PreAuthorize("expr")`	before the call	role/authority checks, argument checks (`#owner == authentication.name`)
`@PostAuthorize("expr")`	after the call, sees `returnObject`	"load it, then check it belongs to the caller"
`@PreFilter("expr")`	before, filters a collection argument	rarely needed
`@PostFilter("expr")`	after, removes elements of the returned collection (`filterObject`)	small lists only
`@Secured("ROLE_ADMIN")`	before (enable with `securedEnabled = true`)	legacy, no SpEL
`@RolesAllowed("ADMIN")`	before (enable with `jsr250Enabled = true`; JSR-250 is the Java standard that defined it)	Jakarta standard annotation

Expressions are **SpEL** (Spring Expression Language). `authentication`, `principal`, `hasRole(...)`, `hasAuthority(...)`, method parameters as `#name` and `returnObject` are available.

```
@Service
public class OrderService {

    @PreAuthorize("#owner == authentication.name or hasRole('ADMIN')")
    public List<Order> ordersOf(String owner) { ... }

    @PostAuthorize("returnObject.owner() == authentication.name")
    public Order find(long id) { ... }

    @PostFilter("filterObject.owner() == authentication.name or hasRole('ADMIN')")
    public List<Order> all() { return new ArrayList<>(orders); }   // must be mutable

    @PreAuthorize("hasAuthority('order:refund')")
    public String refund(long id) { ... }
}
```

How it works

Method security is **proxy-based AOP** (Chapter 4). Spring wraps `OrderService` in a proxy whose interceptor evaluates the expression and throws `AccessDeniedException` if it is false. The same proxy rules apply as for `@Transactional`:

- Only calls that go **through the proxy** are checked. A call from inside the same class (`this.refund(1)`) skips the check. This is **self-invocation**, proved for `@Cacheable` in Chapter 4.
- `private` methods, and in CGLIB proxies `final` methods, are never intercepted.

Proved

The tests call the service directly, with `@WithMockUser` supplying the security context:

```
@Test
@WithMockUser(username = "alice") // default role USER
void ownerCanReadOwnOrders() {
    orders.ordersOf("alice");                                  // allowed
    assertThatThrownBy(() -> orders.ordersOf("carol")).isInstanceOf(AccessDeniedException.class);
    assertThat(orders.all()).extracting(OrderService.Order::owner).containsOnly("alice");
    assertThatThrownBy(() -> orders.find(2)).isInstanceOf(AccessDeniedException.class);
    assertThatThrownBy(() -> orders.refund(1)).isInstanceOf(AccessDeniedException.class);
}
```

Output:

```
no SecurityContext, refund(1) -> AuthenticationCredentialsNotFoundException
alice ordersOf(alice)        -> [Order[id=1, owner=alice, cents=1999], Order[id=3, owner=alice, cents=250]]
alice ordersOf(carol)        -> AccessDeniedException
alice all() after @PostFilter -> [Order[id=1, owner=alice, cents=1999], Order[id=3, owner=alice, cents=250]]
alice find(2) (carol's)      -> AccessDeniedException (@PostAuthorize)
alice refund(1)              -> AccessDeniedException (needs order:refund)
bob(ADMIN) ordersOf(carol)   -> [Order[id=2, owner=carol, cents=500]]
support refund(1)            -> refunded 1
GET /orders/of/carol as alice -> HTTP 403
```

- Calling a secured method with **no** authentication at all (for example from a background thread) fails with `AuthenticationCredentialsNotFoundException`.
- Over HTTP, the `AccessDeniedException` thrown by the service travels back through the filters, and `ExceptionTranslationFilter` turns it into a **403**, just as for URL rules. If you add an `@ExceptionHandler(Exception.class)` in a `@ControllerAdvice`, make sure it does not swallow `AccessDeniedException` and return 500.

Trade-offs

	URL rules	Method security
Protects	HTTP endpoints	any bean method, any caller
Can see	path, method, headers	arguments, return value, domain objects
Cost	cheap, before MVC	a proxy per bean. `@PostAuthorize` runs the method (and its SQL) before denying
Best for	coarse gates (`/admin/**`)	ownership and business rules

Use both: URLs as the outer fence, methods for "may *this* user touch *this* record". For large collections, don't use `@PostFilter`. It loads everything and then throws most of it away. Filter in the query instead (`where owner = :currentUser`).

Common mistakes

- Forgetting `@EnableMethodSecurity`. The annotations are then silently ignored.
- Self-invocation, and annotating `private` methods.
- `@PostFilter` on an immutable `List.of(...)`. It must remove elements, so it needs a mutable collection.
- Writing complex SpEL strings. Move the logic into a bean and call it: `@PreAuthorize("@orderGuard.canRead(#id, authentication)")`.

Self-check

1. *Why can `@PreAuthorize` check `#owner` but a URL rule cannot?* → It runs inside the method proxy with access to the arguments. URL rules only see the HTTP request.
2. *A `@PreAuthorize` method is called from another method of the same class. Is it checked?* → No, the call does not pass through the proxy.
3. *What does `@PostAuthorize` cost?* → The method runs fully (including database access) before the check can deny.

TL;DR

`@EnableMethodSecurity` + `@PreAuthorize` for business rules and ownership checks. It is proxy-based, so self-invocation and private methods are not protected. A denial becomes 403 over HTTP.

10.8 CSRF protection

What it is

Cross-site request forgery (CSRF) tricks a logged-in user's browser into sending a state-changing request to your site from another site. The browser attaches the session cookie automatically. The attack and the browser side of the defence (tokens, `SameSite` cookies — a cookie flag that stops the browser sending it on requests from other sites) are explained in the JavaScript handbook: 14.11 Cross-site request forgery. Here we only cover what Spring does.

How Spring's CSRF protection works

1. `CsrfFilter` generates a random **CSRF token** per session and stores it (by default in the HTTP session: `HttpSessionCsrfTokenRepository`).
2. Server-rendered pages include it. Spring's login page, Thymeleaf forms and JSP tags add a hidden `_csrf` field automatically.
3. For every **unsafe** method (`POST`, `PUT`, `PATCH`, `DELETE`), the filter compares the token sent in the `_csrf` parameter or the `X-CSRF-TOKEN` / `X-XSRF-TOKEN` header with the stored one. A missing or wrong token gets **403**.
4. **Safe** methods (`GET`, `HEAD`, `OPTIONS`, `TRACE`) are not checked, so never change state in a `GET` handler.
5. Refinements since Spring Security 6: the token is loaded **lazily** (no session touch for requests that don't need it), and the value rendered into pages is **XOR-masked** (combined bit by bit with a fresh random value) on each request (protection against the BREACH compression attack, which recovers secrets from the size of compressed HTTPS responses). It looks different every time but validates the same.

Proved: on for the cookie app, off for the stateless API

```
// web chain (session cookie): CSRF on
mvc.post().uri("/web/notes").param("text", "hi").with(user("alice"))                              // no token
mvc.post().uri("/web/notes").param("text", "hi").with(user("alice")).with(csrf().useInvalidToken())
mvc.post().uri("/web/notes").param("text", "hi").with(user("alice")).with(csrf())                  // valid token
// api chain (Basic auth, STATELESS): CSRF disabled
mvc.post().uri("/api/notes").contentType(MediaType.TEXT_PLAIN).content("from curl").with(httpBasic("alice", "alice-pass"))
```

Output:

```
login page hidden field: <input name="_csrf" type="hidden" value="<token>" />
POST /web/notes, logged in, no CSRF token  -> 403
POST /web/notes, logged in, wrong token    -> 403
POST /web/notes, logged in, valid token    -> 200
GET  /web/home,  no token needed (safe)     -> 200
POST /api/notes, Basic auth, no CSRF token -> 201
```

When may you disable it?

CSRF needs **ambient credentials**, meaning credentials the *browser* attaches on its own: cookies, HTTP Basic cached by the browser, client certificates.

Your API authenticates with...	CSRF protection
session cookie (form login, OAuth2 login, BFF — Backend for Frontend, a server that holds tokens on behalf of a browser app)	**keep it on**
`Authorization: Bearer <token>` added by JavaScript or a server, never cookies	can be disabled for that chain
Basic auth from scripts / other services only	can be disabled
both cookies *and* tokens on the same endpoints	keep it on

Single-page apps: `csrf.spa()` (Spring Security 7)

A JavaScript SPA that uses a session cookie still needs CSRF protection, but it can't read a hidden form field. `http.csrf(c -> c.spa())` sets up the standard SPA arrangement in one call:

- the token is stored in a cookie named `XSRF-TOKEN` that JavaScript *can* read (`HttpOnly=false`),
- the app echoes it in the `X-XSRF-TOKEN` header (Angular, a front-end framework, and axios, a JavaScript HTTP library, do this automatically),
- a request handler accepts both the raw cookie value and the masked form value.

Before 7.0 this took several lines (`CookieCsrfTokenRepository.withHttpOnlyFalse()` plus a custom handler), and many tutorials still show that form.

Common mistakes

- `csrf.disable()` "because POST returned 403" in an app that logs users in with a session cookie. That re-opens the attack. Send the token instead.
- Using `GET` for actions (`GET /delete?id=5`). CSRF protection ignores `GET`, and link prefetchers (browsers or tools that load links in advance) can trigger it.
- Forgetting that `POST /logout` also needs the token (see §10.10).

TL;DR

CSRF protection is on by default for unsafe methods. Keep it wherever the browser sends credentials automatically. Disable it only for chains that authenticate with explicit headers. Use `csrf.spa()` for cookie-based SPAs.

10.9 CORS

What it is

CORS (Cross-Origin Resource Sharing) is how a *browser* lets JavaScript from origin A read responses from origin B (an *origin* is scheme + host + port, e.g. `https://app.example.com`). The server opts in with `Access-Control-Allow-*` headers. For "non-simple" requests (for example `POST` with JSON or an `Authorization` header), the browser first sends a **preflight** `OPTIONS` request to ask permission. The browser rules are covered in 14.2 The same-origin policy and CORS. CORS is **not** a protection for your server. It relaxes a browser restriction, and `curl` ignores it completely.

Why Spring Security must be involved

A preflight carries **no credentials**, because browsers never send cookies or `Authorization` on it. If the security chain ran first, it would answer the preflight with 401 and the browser would block the real request. `http.cors(...)` adds a `CorsFilter` *early* in the chain, so preflights are answered before authentication.

```java
@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration api = new CorsConfiguration();
    api.setAllowedOrigins(List.of("https://app.example.com"));
    api.setAllowedMethods(List.of("GET", "POST"));
    api.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    api.setMaxAge(3600L);                              // browser may cache the preflight 1 hour
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
}
// in the API chain:  .cors(Customizer.withDefaults())   -> uses this bean
```

Proved: a preflight, allowed and refused

```
mvc.options().uri("/api/notes")
        .header("Origin", origin)
        .header("Access-Control-Request-Method", "POST")
        .header("Access-Control-Request-Headers", "authorization,content-type")
```

Output:

```
preflight from https://app.example.com -> 200
  Access-Control-Allow-Origin: https://app.example.com
  Access-Control-Allow-Methods: GET,POST
  Access-Control-Allow-Headers: authorization, content-type
  Access-Control-Max-Age: 3600
preflight from https://evil.example   -> 403 body=Invalid CORS request
GET with Origin + Basic -> 200, Access-Control-Allow-Origin: https://app.example.com, Vary: [Origin, Access-Control-Request-Method, Access-Control-Request-Headers]
```

- The allowed preflight gets **200 without credentials**, so CORS runs before authentication.
- An unknown origin gets **403 "Invalid CORS request"**.
- The real request also gets `Access-Control-Allow-Origin`. `Vary: Origin` tells caches that the answer depends on the origin.

Trade-offs and choices

Option	Where	Use when
`CorsConfigurationSource` bean + `http.cors(...)`	security chain	the endpoints are secured (almost always)
`@CrossOrigin` on controllers / `WebMvcConfigurer.addCorsMappings`	Spring MVC (Chapter 5)	no Spring Security, or as the source that `http.cors()` falls back to when no `CorsConfigurationSource` bean exists
gateway / reverse proxy (a server in front of your apps that receives requests and forwards them; see System Design Fundamentals)	outside the app	many services behind one edge

The general CORS mistakes (`"*"` with credentials, CORS only in MVC) are listed in Chapter 5, section 5.10.

TL;DR

Configure a `CorsConfigurationSource` and call `http.cors(...)` so preflights are answered before authentication. List exact origins. CORS relaxes the browser's same-origin policy and protects nothing on the server.

10.10 Sessions, the security context and logout

What it is

After form login, the `Authentication` must survive to the next request. By default Spring saves the `SecurityContext` in the **HTTP session** (`HttpSessionSecurityContextRepository`), and the browser keeps the session id in the `JSESSIONID` cookie. `sessionManagement(...)` controls this:

SessionCreationPolicy	Meaning
`IF_REQUIRED` (default)	create a session when needed (e.g. at login)
`ALWAYS`	always create one
`NEVER`	never create one, but use an existing session
`STATELESS`	never create or use one for the security context, so every request must authenticate itself (Basic, Bearer token)

Session fixation, concurrency and logout — proved

Session fixation is an attack where an attacker plants a known session id in the victim's browser *before* login and reuses it after. The defence is simple: **change the session id at login**. Spring does this by default (`changeSessionId()`, via the Servlet `HttpServletRequest.changeSessionId()` method).

```
MockHttpSession session = new MockHttpSession();       // the anonymous visitor's session
String before = session.getId();
var login = mvc.post().uri("/login").session(session)
        .param("username", "alice").param("password", "alice-pass").with(csrf()).exchange();
MockHttpSession after = (MockHttpSession) login.getRequest().getSession(false);
// ... use the new session, then POST /logout with it
```

Output:

```
login  -> 302 /, session id changed: true (session fixation protection)
GET /web/home with that session -> 200
logout -> 302 /login?logout, session invalidated: true
API call -> session created? false, Set-Cookie: null
```

- The login changed the session id, and the old id is worthless.
- `POST /logout` (with CSRF token) invalidated the session, cleared the context and redirected to `/login?logout`. `GET /logout` only shows a confirmation page, because logout is a state change and needs CSRF protection.
- The `STATELESS` API chain never created a session or sent `Set-Cookie`.

Other session features:

- **`maximumSessions(1)`** limits concurrent logins per user (the second login expires the first, or `maxSessionsPreventsLogin(true)` rejects it). Register an `HttpSessionEventPublisher` bean so Spring hears about sessions that end by timeout.
- **Remember-me** (`rememberMe(...)`) keeps users logged in across browser restarts with a long-lived cookie. Prefer the persistent-token variant.
- **Session timeout**: `server.servlet.session.timeout=30m`. Harden the cookie with `server.servlet.session.cookie.secure=true`, `http-only=true` (the default) and `same-site=lax` or `strict`.
- Several app instances need a shared session store. **Spring Session** puts sessions in Redis or JDBC, so any node can serve any user.

Stateless vs session: which one?

	Server session + cookie	Stateless (token per request)
Logout / revoke	immediate (invalidate session)	hard until the token expires (Chapter 11)
Scaling	needs sticky sessions (the load balancer always sends a user to the same server) or Spring Session	any node can serve any request
CSRF	needed	not needed if the token is not in a cookie
Best for	browser apps, BFF	APIs called by other services, mobile apps

TL;DR

Form login stores the context in the HTTP session. Spring changes the session id at login (fixation defence). Logout is `POST /logout` with a CSRF token. `STATELESS` chains never create sessions.

10.11 Security headers

What it is

`HeaderWriterFilter` adds HTTP response headers that tell the browser to switch on its own defences. From the test:

```
response headers on GET /web/home:
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Content-Length: 4
  Content-Security-Policy: default-src 'self'; frame-ancestors 'none'
  Content-Type: text/plain;charset=UTF-8
  Expires: 0
  Pragma: no-cache
  Referrer-Policy: strict-origin-when-cross-origin
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  X-XSS-Protection: 0
over HTTPS also: Strict-Transport-Security: max-age=31536000 ; includeSubDomains
```

Header	Default?	What it tells the browser
`Cache-Control` / `Pragma` / `Expires`	yes	don't cache authenticated pages (shared computers, proxies). Your own `Cache-Control` wins
`X-Content-Type-Options: nosniff`	yes	trust `Content-Type`, don't guess (stops some script-injection tricks)
`X-Frame-Options: DENY`	yes	don't render this page inside a frame (**clickjacking**: hiding your page in an invisible frame so the user clicks something they did not mean to)
`Strict-Transport-Security` (HSTS — HTTP Strict Transport Security)	yes, **HTTPS responses only**	use HTTPS for this host for one year
`X-XSS-Protection: 0`	yes	*turn off* the old browser XSS filter, which itself caused vulnerabilities
`Content-Security-Policy`	**no**, you write it	which sources may load scripts, styles and frames. It is the strongest defence against **XSS**
`Referrer-Policy`	no (configured here)	how much of the URL to leak in `Referer`

We added CSP and Referrer-Policy in the web chain:

```
.headers(h -> h
    .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'; frame-ancestors 'none'"))
    .referrerPolicy(r -> r.policy(ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN)))
```

What XSS is and how to write a CSP are explained in 14.10 XSS and 14.12 Content Security Policy. Roll out a new CSP first with `.reportOnly()`, so violations are reported instead of breaking the page.

Common mistakes

- Disabling all headers (`headers(h -> h.disable())`) to fix one problem, such as an iframe. Change just that header: `frameOptions(f -> f.sameOrigin())`.
- Expecting HSTS on `http://localhost`. It is only sent over HTTPS. Behind a TLS-terminating proxy (one that decrypts HTTPS and forwards plain HTTP to the app), set `server.forward-headers-strategy=framework` so the app knows the request was HTTPS.

TL;DR

Five protective headers are on by default (HSTS only over HTTPS). Add a Content-Security-Policy yourself. Change individual headers instead of disabling them all.

10.12 Testing security with `spring-security-test`

What it is

`spring-boot-starter-security-test` brings in **spring-security-test**, a set of helpers that put an identity or a token into a test request. You can test rules without real passwords where that is convenient, or with real ones where the login itself is under test. Every test in this chapter uses them:

Helper	Kind	What it does
`@WithMockUser(username, roles, authorities)`	annotation	puts a `UsernamePasswordAuthenticationToken` in the context for the test method, with no `UserDetailsService` involved. Default user `user`, role `USER`
`@WithUserDetails("alice")`	annotation	loads the real user from *your* `UserDetailsService`
`@WithAnonymousUser`	annotation	explicit anonymous caller
`with(user("alice").roles("USER"))`	request post-processor	like `@WithMockUser`, per request
`with(httpBasic("alice", "alice-pass"))`	request post-processor	adds a real `Authorization: Basic` header, so the full login path runs
`with(csrf())` / `csrf().useInvalidToken()`	request post-processor	adds a valid (or broken) CSRF token
`formLogin().user(..).password(..)`	request builder	a real `POST /login` including CSRF
`logout()`	request builder	a real `POST /logout`
`jwt()`, `opaqueToken()`, `oauth2Login()`, `oidcLogin()`	request post-processors	token and OAuth2 identities (Chapter 11)
`authenticated().withUsername(..)`, `unauthenticated()`, `withAuthorities(..)`	result matchers	assert on the security context after the request. `withAuthorities(String...)` is new in 7.0

`MockMvcTester` (the AssertJ-based MockMvc used throughout this book) accepts these post-processors with `.with(...)`. With `@SpringBootTest` + `@AutoConfigureMockMvc`, the security filter chain is applied automatically. With the `@WebMvcTest` slice, add `@Import(SecurityConfig.class)` so the test runs with *your* rules and not Boot's defaults. See Chapter 12 for slices.

What to test

1. **Each rule's boundaries**: anonymous, wrong role, right role. The 401/403 matrix above is one test.
2. **The catch-all**: an unlisted URL is denied.
3. **CSRF**: unsafe requests fail without a token, and succeed with one.
4. **Method rules** directly on the service with `@WithMockUser`, including ownership cases.
5. **At least one real login** (`httpBasic`/`formLogin`) so the password encoder and user lookup are exercised.

Common mistakes

- Only testing the happy path (the case where everything goes right) with `@WithMockUser(roles = "ADMIN")`. It proves admins get in, not that others are kept out.
- Using `@WithMockUser` when the code needs your custom principal class. Use `@WithUserDetails`, or a custom `@WithSecurityContext` annotation.
- Forgetting `with(csrf())` and concluding "security blocks my POST" when the rule is actually fine.

TL;DR

`@WithMockUser`/`user()` for rules, `httpBasic()`/`formLogin()` for real logins, `csrf()` for unsafe methods, result matchers for "who is logged in now". Test the denials, not just the grants.

10.13 Common mistakes (whole chapter)

Mistake	Symptom	Fix
Own `SecurityFilterChain` not picked up	generated password still printed	bean inside a scanned `@Configuration`
Catch-all chain ordered first	specific chain ignored	`securityMatcher` + `@Order`, specific first
`hasRole("ROLE_X")`	startup `IllegalArgumentException`	`hasRole("X")` or `hasAuthority("ROLE_X")`
`roles()` then `authorities()` on `User`	roles vanish	one call with the full list
`csrf.disable()` with session login	CSRF hole	send the token, or `csrf.spa()`
CORS only in MVC	preflight 401	`http.cors(...)` + `CorsConfigurationSource`
Plain/SHA hashed passwords	leak = cracked passwords	`DelegatingPasswordEncoder` with bcrypt/Argon2
Method security not enabled / self-invocation	annotations ignored	`@EnableMethodSecurity`, call through another bean
`anyRequest().permitAll()` at the end	new endpoints public	`denyAll()` or `authenticated()`
Security filter declared as `@Component`	runs twice, once outside the chain	`addFilterBefore` + disabled `FilterRegistrationBean`
Context lost in `@Async`	`AuthenticationCredentialsNotFoundException`	`DelegatingSecurityContextExecutor`
Actuator exposed with the app rules	`/actuator/env` public	its own chain or rules (Chapter 15)

10.14 In the interview

"Walk me through what happens when a request hits a Spring Security app." "The container calls `DelegatingFilterProxy`, which hands off to the `FilterChainProxy` bean. It picks the first `SecurityFilterChain` whose matcher fits. The filters then run in order: the security context is loaded (e.g. from the session), headers are added, CSRF is checked for unsafe methods, authentication filters try to authenticate (form, Basic, bearer), an anonymous token is set if nobody did, and finally `AuthorizationFilter` checks the rules. A denial becomes 401/redirect for anonymous callers or 403 for known ones via `ExceptionTranslationFilter`. If everything passes, the request reaches the `DispatcherServlet`."

Follow-ups:

- *Where is the logged-in user stored during a request, and between requests?* → During: `SecurityContextHolder` (`ThreadLocal`). Between: the HTTP session (form login), or nowhere for stateless chains, where the client resends credentials or a token.
- *How do you store passwords?* → `DelegatingPasswordEncoder` with bcrypt or Argon2id, `{id}` prefix, upgrade on login via `UserDetailsPasswordService`.
- *When is it safe to disable CSRF?* → When the chain never authenticates with credentials the browser adds automatically (cookies, cached Basic), for example pure bearer-token APIs.
- *How do you protect "users can only see their own orders"?* → Method security with an ownership expression, or better, scope the query by the current user. URL rules alone can't see ownership.
- *What changed in Spring Security 7?* → Lambda DSL only, old matchers and `authorizeRequests` removed, built-in MFA, `csrf.spa()`, Authorization Server merged in (full list in §10.3).

Chapter 10 TL;DR

- **Architecture**: `DelegatingFilterProxy` → `FilterChainProxy` → first matching `SecurityFilterChain` → ordered filters → `AuthorizationFilter`. The user lives in `SecurityContextHolder`.
- **Configuration**: `SecurityFilterChain` beans, one lambda per feature. Spring Security 7 removed `and()`, `authorizeRequests` and the Ant/Mvc matchers. Use several chains with `securityMatcher` + `@Order`.
- **Authentication**: filter → `ProviderManager` → `DaoAuthenticationProvider` → `UserDetailsService` + `PasswordEncoder`. Browsers get a 302 to `/login`, APIs get a 401 challenge. Spring Security 7 adds `FACTOR_*` authorities for MFA.
- **Passwords**: adaptive salted hashes (bcrypt, Argon2id) via `DelegatingPasswordEncoder` (`{id}hash`), upgraded on login.
- **Authorization**: first matching rule wins. Roles are `ROLE_`-prefixed authorities. **401 = who are you, 403 = no**. End with `denyAll()`. Use method security for ownership.
- **Web defences**: CSRF on for cookie sessions (off only for header-token chains, `csrf.spa()` for SPAs). `http.cors()` so preflights pass. Session id changes at login. Default headers plus your own CSP.
- **Testing**: spring-security-test's `@WithMockUser`, `user()`, `httpBasic()`, `csrf()`, `formLogin()`, and test the denials.

Next: Chapter 11 — Tokens and OAuth2 replaces sessions and passwords with signed tokens and identity providers.

Chapter 11 — Spring Security 2: Tokens and OAuth2

Chapter 10 authenticated people with a password and remembered them with a session cookie. That works for one browser talking to one server. Modern systems have more parties: a mobile app, a **single-page app** (SPA: a web app that loads one page and then updates it with JavaScript instead of fetching new pages), twenty **microservices** (small, separately deployed services that together form one system), a partner's backend, and an **identity provider** (IdP: the system that stores user accounts and checks logins) such as Keycloak, Okta, Entra ID or Google that owns the user accounts. They need a way to prove identity **without sharing passwords and without shared sessions**. That way is **tokens**, and the standard for handing them out is **OAuth 2.0** with **OpenID Connect** on top.

This chapter explains the ideas from zero and then proves Spring's behaviour with tests. The signing keys are generated locally, so **no external identity provider is needed** to run anything.

What you'll learn

1. Bearer tokens and JWTs: structure, claims, signatures, keys (JWK/JWKS).
2. OAuth 2.0 and OpenID Connect: roles, grant types, access vs refresh vs ID tokens.
3. A **JWT resource server** in Spring Security 7: validation rules (signature, expiry, issuer, audience, `typ`), 401 `invalid_token` vs 403 `insufficient_scope`.
4. Minting tokens with `NimbusJwtEncoder` (new builder in Spring Security 7).
5. Mapping claims to authorities (`SCOPE_`, roles) and testing with the `jwt()` post-processor, including its biggest pitfall.
6. Opaque tokens and introspection, and multi-tenant resource servers.
7. **OAuth2 login** (the authorization-code flow with PKCE) as a sequence diagram, and the parts we can run.
8. Service-to-service calls (client credentials, `RestClient`, HTTP interfaces, token relay), refresh tokens, logout.
9. Where tokens should live in a browser (BFF), key and secret storage, Spring Authorization Server, rate limiting.
10. **Security fundamentals beyond Spring** (§11.18): encoding vs hashing vs encryption, symmetric vs asymmetric keys, signatures and certificates, the TLS 1.3 handshake, SSH keys, TOTP codes (RFC 6238, runnable), passkeys/WebAuthn, and SSO with SAML vs OIDC.

All code lives in `code/` — three Spring Boot **4.1.1** projects plus one plain-JDK project (Spring Security **7.1.1**, Java 25):

Project	Shows
`jwt-resource-server`	local RSA key, `/token` endpoint with `NimbusJwtEncoder`, a JWT-protected API, every validation failure, `jwt()` tests
`resource-server-variants`	opaque tokens with an introspector, multi-tenant JWTs (two issuers, two keys)
`oauth2-login-client`	OAuth2/OIDC login configuration. The redirect to the provider, `state`/`nonce`/PKCE, and `oidcLogin()` tests
`crypto-basics`	plain JDK: Base64 vs SHA-256 vs AES-GCM vs Ed25519 signatures, an X25519 key exchange, and TOTP checked against the RFC 6238 test vectors (§11.18)

Every output block is real `mvn -q -B -ntp verify` output (`mvn` is Maven, the Java build tool; `verify` compiles the code and runs all tests) (log lines trimmed as `...`). Timestamps and random values change on each run.

> **Prerequisites.** The filter chain, 401 vs 403, authorities and CSRF (cross-site request forgery: another site tricking your browser into sending a request that carries your cookies): Chapter 10. Browser storage, cookies and XSS (cross-site scripting: an attacker getting their JavaScript to run inside your page): JavaScript handbook Chapter 14. Gateways and BFFs (Backend for Frontend, §11.12) as architecture patterns: API Gateway, Backend for Frontend.

11.1 From sessions to tokens: bearer tokens and JWT

What it is

A **bearer token** is a string that grants access to whoever *bears* (holds) it, like a cinema ticket. The server does not ask who you are, only whether the ticket is valid. It travels in an HTTP **header** (a `Name: value` line sent before the request body):

```
GET /api/messages HTTP/1.1
Authorization: Bearer eyJraWQiOiJkZW1vLWtleS0xIiwidH...
```

A **JWT (JSON Web Token, RFC 7519)** is the most common token format. An **RFC** ("Request for Comments") is a numbered document that defines an internet standard. A JWT is three **Base64URL**-encoded parts joined by dots (Base64URL turns any bytes into letters, digits, `-` and `_` so they are safe in URLs and headers; it is reversible by anyone), `header.payload.signature`. Here is a token our `/token` **endpoint** (a URL path the server answers) issued, decoded:

```
token (3 base64url parts) : eyJraWQiOiJkZW1vLWtleS0xIiwidH... length=670
header  : {"kid":"demo-key-1","typ":"JWT","alg":"RS256"}
payload : {"sub":"bob","aud":"notes-api","scope":"messages.read messages.write","roles":["ADMIN"],"iss":"https://auth.demo.test","exp":
<epoch>,"iat":<epoch>,"jti":"<uuid>"}
```

Part	Contains	Notes
Header	`alg` (signature algorithm), `kid` (key id), `typ`	tells the receiver *how* to verify
Payload	**claims**: statements about the subject	readable by anyone, since it is only encoded, not encrypted
Signature	signature over `header.payload`	proves who issued it and that nothing changed

The registered claims you will meet everywhere:

Claim	Meaning	Checked by Spring by default?
`iss` (issuer)	who created the token	yes, if you configure an issuer
`sub` (subject)	who the token is about (user or client id)	becomes `Authentication#getName()`
`aud` (audience)	which API(s) the token is meant for	no, you add it (property `audiences` or a validator)
`exp` / `nbf` / `iat`	expires at / not before / issued at (epoch seconds: seconds since 1 Jan 1970 UTC)	`exp` and `nbf` yes, with 60 s clock skew (allowed gap between server clocks)
`jti`	unique token id	no, useful for revocation lists
`scope` / `scp`	granted OAuth scopes (space-separated string or list)	mapped to `SCOPE_x` authorities

Signing: symmetric vs asymmetric

	HMAC (`HS256` , a hash keyed with a shared secret)	**RSA / EC (`RS256` , `ES256` , `PS256` ; public-key algorithms, EC = elliptic curve)**
Keys	one shared secret signs *and* verifies	the **private** key signs, the **public** key verifies
Who can mint tokens	everybody who can verify	only the key owner
Fits	one service issuing tokens to itself	an identity provider and many APIs (the normal case)

Resource servers download the issuer's public keys as a **JWK Set** (JSON Web Key Set, JWKS) from a URL such as `https://idp.example.com/.well-known/jwks.json`. The `kid` in the token header says which key to use. That is how keys can **rotate** (be replaced by new keys from time to time, §11.13).

JWS vs JWE. A signed JWT is a **JWS** (JSON Web Signature). An *encrypted* JWT is a **JWE**, which is rare in APIs. Never put secrets or personal data you would not show the user into a normal JWT, because anyone holding it can decode the payload.

Why tokens

- **No server-side session.** Any instance can verify the signature locally, so APIs **scale horizontally** (handle more load by adding more copies of the service) without a shared session store (a database or cache that all copies would otherwise read login sessions from).
- **Many parties.** The identity provider issues and the APIs only verify. APIs never see passwords.
- **Delegation.** A token can carry *limited* rights (scopes), for example "read calendar" but not "delete account".

Trade-offs

- **Revocation is hard. Revocation** means cancelling a token before it expires. A self-contained JWT stays valid until `exp`, even after logout or a password change. Keep access tokens short-lived (minutes) and use refresh tokens (§11.11), or **opaque tokens** (random strings that mean nothing by themselves, §11.7).
- **Size.** Our small token is 670 bytes and is sent on every request. Big claim sets make headers heavy.
- **Theft = access.** A bearer token works for anyone. Always use HTTPS (HTTP encrypted with TLS, so nobody on the network can read the token; see System Design Fundamentals), short lifetimes and safe storage (§11.12).

Self-check

1. *Can a client read the claims of a JWT?* → Yes. The payload is only Base64URL-encoded. The signature protects integrity, not confidentiality.
2. *Why do APIs prefer RS256 over HS256?* → With RS256 the APIs hold only the public key, so a compromised API cannot mint tokens.
3. *What does `kid` do?* → It names the key that signed the token, so the verifier picks the right one from the JWK Set during key rotation.

TL;DR

A JWT is `header.payload.signature`. Claims are readable, and the signature proves origin and integrity. Use asymmetric keys, short lifetimes, and verify `exp`, `iss` and `aud`.

11.2 OAuth 2.0 and OpenID Connect in one picture

What it is

OAuth 2.0 (RFC 6749) is a framework for **delegated authorization**: letting an application act on a user's behalf *without receiving the user's password*. The classic analogy is a hotel key card. Reception (the authorization server) checks your passport once and gives you a card (an access token) that opens *your* room (scopes) until checkout (expiry). The doors (APIs) only check the card.

OpenID Connect (OIDC) is a thin layer on top of OAuth 2.0 that adds **authentication**: an **ID token** (always a JWT) telling the client *who logged in*, a standard `userinfo` endpoint, and discovery metadata at `/.well-known/openid-configuration`.

OAuth role	Who it is	Spring module
Resource owner	the user	—
Client	the app that wants access (web app, SPA backend, mobile app, service)	`spring-security-oauth2-client` (`oauth2Login`, `oauth2Client`)
Authorization server (identity provider, IdP)	issues tokens: Keycloak, Okta, Entra ID, Auth0, Google, Spring Authorization Server	`spring-security-oauth2-authorization-server` (§11.14)
Resource server	the API that accepts tokens	`spring-security-oauth2-resource-server` (`oauth2ResourceServer`)

The tokens

Token	For whom	Format	Lifetime
Access token	the resource server (API)	JWT or opaque	short: 5–60 minutes
Refresh token	the authorization server only, to get new access tokens	usually opaque	hours to weeks, and can be revoked
ID token (OIDC)	the client, to learn who logged in	always a JWT	short, used once at login

Never send an ID token to an API as an access token, and never use an access token to "log a user in" to the client. They have different audiences.

Grant types (how a client gets a token)

Grant	Used by	Status
Authorization code + PKCE	any app with a user: server web apps, SPAs (via BFF), mobile	the standard (§11.9)
Client credentials	service-to-service, no user involved	standard (§11.10)
Refresh token	a client renewing an access token	standard (§11.11)
Device code (RFC 8628)	TVs, CLIs without a browser	standard
Token exchange (RFC 8693)	a service swapping a token for one aimed at another API	growing
Implicit	old SPAs (token in the URL fragment)	**deprecated**, left out of OAuth 2.1
Resource owner password	app collects the user's password	**deprecated**. Spring Security 7 **removed** its client support

The **OAuth 2.1** draft consolidates these best practices: PKCE for every authorization-code client, no implicit and no password grant, exact redirect-URI matching.

Scopes vs roles

A **scope** (`messages.read`) limits what *the client* may do on the user's behalf. The user consented to it. A **role** is about what *the user* may do in your system. A token can carry both, and your API decides how to combine them. Spring maps scopes to `SCOPE_x` authorities by default (§11.5).

TL;DR

OAuth 2.0 = delegated authorization with access tokens. OIDC adds login with ID tokens. Use authorization code + PKCE for users and client credentials for services. Implicit and password grants are obsolete.

11.3 A JWT resource server in Spring

What it is

A **resource server** is an API that accepts access tokens. With `spring-boot-starter-security-oauth2-resource-server` (a **starter** is one dependency that pulls in a feature's libraries plus its auto-configuration; this is Boot 4's name; the old `spring-boot-starter-oauth2-resource-server` is deprecated), one **DSL** call (domain-specific language: here, the chain of fluent `http.xxx(...)` configuration methods) adds a `BearerTokenAuthenticationFilter` to the **filter chain** (the ordered list of security filters every request passes through, Chapter 10):

```
http.oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()));
```

With a real identity provider, one property is usually all you need:

```
# Spring reads https://idp.example.com/.well-known/openid-configuration (or oauth-authorization-server)
# to find the JWK Set URL, and validates that iss equals this value
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://idp.example.com/realms/demo
# optional extras
spring.security.oauth2.resourceserver.jwt.audiences=notes-api
spring.security.oauth2.resourceserver.jwt.jws-algorithms=RS256
# alternatives: jwk-set-uri=... (no discovery) or public-key-location=classpath:key.pub (static key)
```

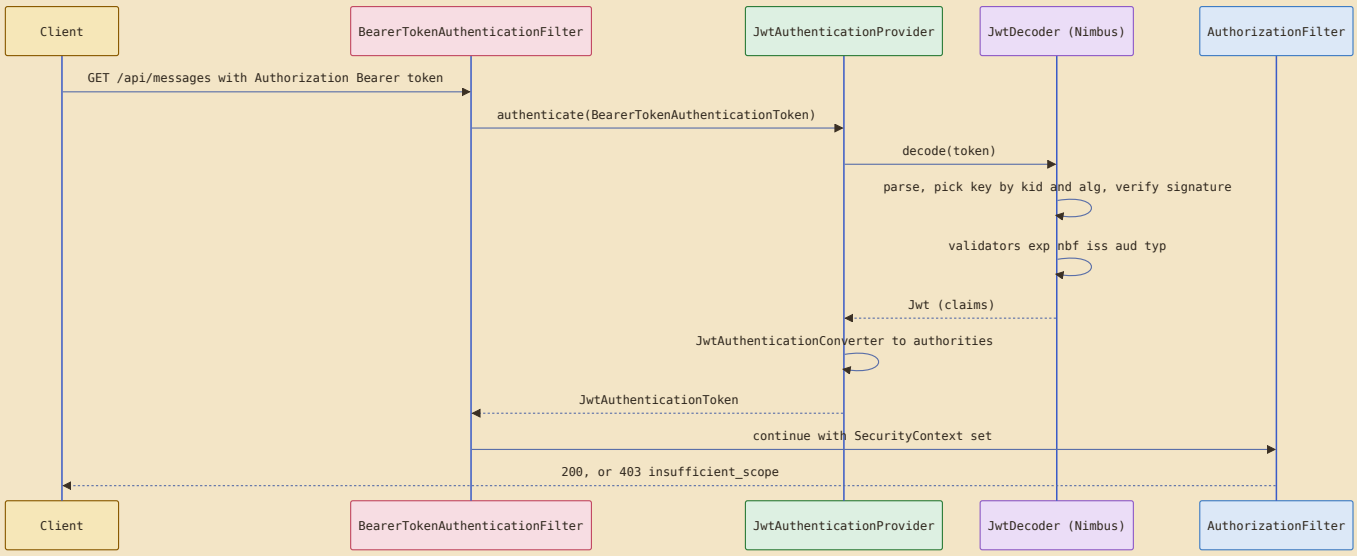
In our project there is no IdP, so we declare the `JwtDecoder` **bean** (an object that Spring creates, wires and manages for us) ourselves with a locally generated **key pair** (a private key that signs and a matching public key that verifies). A `JwtDecoder` bean always takes precedence over the properties.

```
@Bean
KeyPair signingKeys() throws Exception {                    // demo only: new keys at each start
    KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
    gen.initialize(2048);
    return gen.generateKeyPair();
}

@Bean
JwtDecoder jwtDecoder(KeyPair keys, @Value("${demo.jwt.issuer}") String issuer,
                      @Value("${demo.jwt.audience}") String audience) {
    NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey((RSAPublicKey) keys.getPublic()).build();
    OAuth2TokenValidator<Jwt> audienceCheck =
            new JwtClaimValidator<List<String>>("aud", aud -> aud != null && aud.contains(audience));
    decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
            JwtValidators.createDefaultWithIssuer(issuer), audienceCheck));
    return decoder;
}
```

The API chain:

```
@Bean @Order(2)
SecurityFilterChain api(HttpSecurity http, JwtAuthenticationConverter converter) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.GET, "/api/messages").hasAuthority("SCOPE_messages.read")
            .requestMatchers(HttpMethod.POST, "/api/messages").hasAuthority("SCOPE_messages.write")
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(jwt -> jwt.jwtAuthenticationConverter(converter)))
        .csrf(c -> c.disable())                // no cookies -> no CSRF risk
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}
```

How it works



1. `BearerTokenAuthenticationFilter` takes the token from `Authorization: Bearer ...`. Form parameters and query strings are off by default, because tokens in URLs leak into logs.
2. `JwtAuthenticationProvider` calls the **`JwtDecoder`**. Nimbus (the open-source JWT library Spring uses underneath) parses the token, rejects algorithms it wasn't configured for (the classic `alg: none` trick fails here), and **verifies the signature**.
3. **Validators** run: `JwtTimestampValidator` (`exp`/`nbf` with 60 s clock skew), the issuer validator if configured, your own (audience), and in Spring Security 7 a `JwtTypeValidator` that accepts `typ` = `JWT` or no `typ` at all. All `JwtValidators.createDefault...()` methods include it now.
4. A **`JwtAuthenticationConverter`** turns claims into authorities (§11.5) and builds a `JwtAuthenticationToken`. Controllers can take `@AuthenticationPrincipal Jwt jwt` (the **principal** is the identity of whoever is logged in).
5. Any failure → **401** with a `WWW-Authenticate: Bearer error="invalid_token"` header (RFC 6750). A valid token without the needed authority → **403** with `error="insufficient_scope"`.

Proved: status codes with real tokens

Nothing is mocked here. The test obtains real tokens from `/token` and calls the API:

```
no token                          401 WWW-Authenticate: Bearer resource_metadata="http://localhost/.well-known/oauth-protected-resource"
alice GET (messages.read)         200 ["welcome"]
alice POST (needs messages.write) 403 WWW-Authenticate: Bearer error="insufficient_scope", error_description="The request requires higher
privileges than provided by the access token.", error_uri="https://tools.ietf.org/html/rfc6750#section-3.1"
bob POST                          201 added by bob
bob GET admin (roles=[ADMIN])     200 audit log for bob
garbage token                     401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Malformed token", error_uri="https://tools.ietf.org/html/rfc6750#section-3.1", resource_metadata="http://localhost/.well-
known/oauth-protected-resource"
```

The `resource_metadata` parameter points to the **OAuth 2.0 Protected Resource Metadata** document (RFC 9728), which the resource server publishes on its own. A client that knows nothing about the API can discover which authorization server to ask:

```
GET /.well-known/oauth-protected-resource -> 200 {"resource":"http://localhost","bearer_methods_supported":
["header"],"tls_client_certificate_bound_access_tokens":true}
```

With a real issuer configured, the document also lists `authorization_servers`. Customise it with `oauth2ResourceServer(rs -> rs.protectedResourceMetadata(...))`. The Model Context Protocol's authorization spec (MCP: a standard way for AI assistants to call external tools and data sources), for example, relies on this discovery step.

Proved: every validation rule

The test mints tokens with deliberate defects:

```
String expiredLongAgo = mint(encoder, c -> c.issuedAt(now.minusSeconds(3600)).expiresAt(now.minusSeconds(600)));
String expiredJustNow = mint(encoder, c -> c.issuedAt(now.minusSeconds(600)).expiresAt(now.minusSeconds(30)));
String wrongIssuer    = mint(encoder, c -> c.issuer("https://evil.test"));
String wrongAudience  = mint(encoder, c -> c.audience(List.of("billing-api")));
String otherKey       = mint(attacker, c -> {});              // attacker's own RSA key pair
// valid token, payload swapped for one with more scopes, original signature kept
String tampered = parts[0] + "." + forgedPayload + "." + parts[2];
```

Output (the `error_uri` and `resource_metadata` parameters are trimmed as `...`):

```
valid                           200 ["welcome","hi"]
expired 10 min ago              401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Jwt expired at 2026-09-28T23:19:21Z", ...
expired 30 s ago (clock skew 60 s) 200 ["welcome","hi"]
wrong issuer                    401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: The iss claim is not valid", ...
wrong audience                  401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: The aud claim is not valid", ...
signed with another key         401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Signed JWT rejected: Invalid signature", ...
payload edited after signing    401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Signed JWT rejected: Invalid signature", ...
```

What to take away:

- A token that expired **30 s ago still passes**. `JwtTimestampValidator` allows **60 seconds of clock skew** between servers. Remember this when you test expiry.
- **Changing one character of the payload breaks the signature.** That is why claims can be trusted even though anyone can read them.
- **Audience matters.** A token issued for `billing-api` by the *same* trusted issuer is refused. Without an audience check, any token from your IdP, meant for any API, would be accepted by every API.

Common mistakes

- **Skipping the audience check.** Set `spring.security.oauth2.resourceserver.jwt.audiences` or add a validator.
- **Replacing** the validators with only your own. `decoder.setJwtValidator(audienceCheck)` alone drops the expiry check! Always combine with `JwtValidators.createDefault...()`, as above.
- Putting the resource server behind session-based security (`IF_REQUIRED`), so a session cookie quietly becomes a second way in. Use `STATELESS` (no server-side session is ever created; every request must bring its own token) and disable CSRF only on the token chain.
- Logging full tokens. Log `jti` or `sub` instead.

TL;DR

`oauth2ResourceServer(jwt)` + `issuer-uri` (or your own `JwtDecoder`). Spring verifies the signature, `exp`/`nbf` (60 s skew), `iss`, `typ`, and your `aud` rule. Bad or missing token → 401 `invalid_token`. Missing authority → 403 `insufficient_scope`.

11.4 Minting tokens with `NimbusJwtEncoder`

What it is

`JwtEncoder` is the counterpart of `JwtDecoder`: it signs claims into a JWT. Spring Security 7 added builder methods to `NimbusJwtEncoder`, so creating one from a key pair or secret is a one-liner (before, you had to build a Nimbus `JWKSource` by hand):

```
@Bean
JwtEncoder jwtEncoder(KeyPair keys) {
    return NimbusJwtEncoder.withKeyPair((RSAPublicKey) keys.getPublic(), (RSAPrivateKey) keys.getPrivate())
            .jwkPostProcessor(jwk -> jwk.keyID("demo-key-1"))   // becomes "kid" in the header
            .build();
}
// also: withKeyPair(ECPublicKey, ECPrivateKey) for ES256, withSecretKey(SecretKey) for HS256
```

Our `/token` endpoint is protected by HTTP Basic (the client sends username and password, Base64-encoded, in the `Authorization` header; its own chain, `@Order(1)`). It turns the logged-in user's authorities into claims:

```
@PostMapping("/token")
TokenResponse token(Authentication auth) {
    Instant now = Instant.now();
    Duration ttl = Duration.ofMinutes(15);
    List<String> authorities = auth.getAuthorities().stream().map(GrantedAuthority::getAuthority)
            .filter(a -> !a.startsWith("FACTOR_")).toList();          // drop Security 7 MFA factors
    String scope = String.join(" ", authorities.stream().filter(a -> !a.startsWith("ROLE_")).toList());
    List<String> roles = authorities.stream().filter(a -> a.startsWith("ROLE_")).map(a -> a.substring(5)).toList();
    JwtClaimsSet claims = JwtClaimsSet.builder()
            .issuer(issuer).audience(List.of(audience)).subject(auth.getName())
            .issuedAt(now).expiresAt(now.plus(ttl)).id(UUID.randomUUID().toString())
            .claim("scope", scope).claim("roles", roles)
            .build();
    JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256).type("JWT").build();
    String jwt = encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
    return new TokenResponse(jwt, "Bearer", ttl.toSeconds());
}
```

Note the `FACTOR_` filter. After password login, Spring Security 7 adds `FACTOR_PASSWORD` to the authorities (10.4). It describes *how* the user logged in and does not belong in a scope claim.

When to issue your own tokens — and when not

Situation	Self-issued JWT (`JwtEncoder`)	Real authorization server
tests, demos, this book	ideal: no infrastructure	overkill
one internal service issuing short tokens to its own clients	acceptable	better long-term
users from several apps, SSO (single sign-on), social login ("Sign in with Google"), MFA (multi-factor authentication), consent, refresh-token rotation, revocation	**no**, you would be re-writing an IdP	**yes**: Keycloak, a cloud IdP, or Spring Authorization Server (§11.14)

Writing your own login-and-token server means owning password storage, lockout, MFA, key rotation, refresh tokens and revocation forever. Use an existing authorization server unless the need is truly tiny.

TL;DR

`NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)` (Spring Security 7) + `JwtClaimsSet` + `JwsHeader` = a signed token in a few lines. Great for tests and demos. For real users, use an authorization server.

11.5 Mapping claims to authorities

What it is

The decoder produces claims. Authorization rules need **authorities**. By default `JwtAuthenticationConverter` uses a `JwtGrantedAuthoritiesConverter` that reads the `scope` (or `scp`) claim, splits it on spaces and prefixes each value with `SCOPE_`. So `scope: "messages.read messages.write"` becomes `SCOPE_messages.read`, `SCOPE_messages.write`. Nothing else in the token becomes an authority unless you say so.

IdPs put roles in different places: Keycloak uses `realm_access.roles`, Entra ID uses `roles`, Okta uses `groups`. Our token carries a `roles` claim, so the project combines two converters:

```
public class ScopeAndRoleAuthorities implements Converter<Jwt, Collection<GrantedAuthority>> {
    private final JwtGrantedAuthoritiesConverter scopes = new JwtGrantedAuthoritiesConverter(); // scope/scp -> SCOPE_
    private final JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();

    public ScopeAndRoleAuthorities() {
        roles.setAuthoritiesClaimName("roles");
        roles.setAuthorityPrefix("ROLE_");
    }

    @Override
    public Collection<GrantedAuthority> convert(Jwt jwt) {
        Collection<GrantedAuthority> all = new ArrayList<>(scopes.convert(jwt));
        all.addAll(roles.convert(jwt));
        return all;
    }
}

@Bean
JwtAuthenticationConverter jwtAuthenticationConverter(ScopeAndRoleAuthorities authorities) {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authorities);
    converter.setPrincipalClaimName("sub");                // Authentication#getName() = sub claim
    return converter;
}
```

This is why `bob GET admin (roles=[ADMIN])` returned 200: `roles: ["ADMIN"]` became `ROLE_ADMIN`, and `hasRole("ADMIN")` matched.

If one claim is enough, Boot can configure the default converter with properties and no code:

```
spring.security.oauth2.resourceserver.jwt.authorities-claim-name=roles
spring.security.oauth2.resourceserver.jwt.authority-prefix=ROLE_
spring.security.oauth2.resourceserver.jwt.principal-claim-name=preferred_username
```

Common mistakes

- Checking `hasRole("messages.read")` or `hasAuthority("messages.read")` for a scope. The authority is `SCOPE_messages.read`.
- Nested claims (`realm_access.roles`). `JwtGrantedAuthoritiesConverter` reads top-level claims only, so write a small converter that walks the map.
- Using `sub` for display. With many IdPs `sub` is an opaque id, so read `preferred_username` or `email` for humans, but keep keying data by `sub` (+ `iss`).

TL;DR

Default: `scope` → `SCOPE_x`. Anything else (roles, groups) needs a custom converter or the Boot `authorities-claim-name`/`authority-prefix` properties.

11.6 Testing token security

Two ways to test

Approach	How	Proves
Real tokens	mint with `JwtEncoder` (or call `/token`), send `Authorization: Bearer ...`	decoder, validators, converter *and* rules. Everything in §11.3 was tested this way
`jwt()` **post-processor** (a test helper that adjusts the request before it is sent)	`mvc.get().uri(..).with(jwt()...)`	only the **rules**. Decoding and validation are skipped, and a ready-made `JwtAuthenticationToken` is placed in the context

Use real tokens for a handful of tests around the decoder configuration (issuer, audience, expiry). Use `jwt()` for the many rule tests, since it is faster and needs no keys.

The `jwt()` pitfall — proved

```
var plain = mvc.get().uri("/api/messages").with(jwt());                  // sub=user, scope=read
var explicit = mvc.post().uri("/api/messages").contentType(MediaType.TEXT_PLAIN).content("x")
        .with(jwt().jwt(j -> j.subject("carol")).authorities(new SimpleGrantedAuthority("SCOPE_messages.write")));
var scopeClaim = mvc.get().uri("/api/messages").with(jwt().jwt(j -> j.claim("scope", "messages.read")));
// PITFALL: jwt() does not use your JwtAuthenticationConverter unless you pass it in
var rolesIgnored = mvc.get().uri("/api/admin/audit").with(jwt().jwt(j -> j.claim("roles", List.of("ADMIN"))));
var rolesMapped  = mvc.get().uri("/api/admin/audit")
        .with(jwt().jwt(j -> j.claim("roles", List.of("ADMIN"))).authorities(appConverter));
```

Output:

```
jwt() default, GET messages             -> 403
jwt() SCOPE_messages.write, POST        -> 201 added by carol
jwt() scope claim messages.read, GET    -> 200
jwt() roles=[ADMIN], default converter  -> 403
jwt() roles=[ADMIN], app converter      -> 200
```

1. `jwt()` alone gives subject `user` and scope `read`, so the authority is `SCOPE_read`, not `SCOPE_messages.read`. That's a 403.
2. `.authorities(...)` sets the authorities directly, whatever the claims say.
3. Claims are mapped with the **default** `JwtGrantedAuthoritiesConverter`, so `scope` works…
4. …but your custom `roles` mapping is **not applied**. The test gets 403 although the same token from a real IdP would get 200. Pass your converter with `.authorities(converter)`, or test that path with a real token.

The same helpers exist for the other token types: `opaqueToken()` (§11.7), `oauth2Login()` and `oidcLogin()` (§11.9), and `oauth2Client()` for an authorized client in tests. With `@WithMockUser` you can test URL rules too, but the principal is then not a `Jwt`, so controllers taking `@AuthenticationPrincipal Jwt` receive `null`.

TL;DR

Test the decoder with real, locally signed tokens. Test rules with `jwt()`, and hand it your authorities converter or you are testing a different mapping from production.

11.7 Opaque tokens and introspection

What it is

An **opaque token** is a random string with no readable content (`tok-alice-7f3a`). The resource server cannot verify it alone. It asks the authorization server's **introspection endpoint** (RFC 7662) "is this token active, and for whom?" on every request (with **caching**, i.e. remembering recent answers for a while, if you add it):

```
POST /oauth2/introspect   token=tok-alice-7f3a   (client authenticates with its own id/secret)
-> {"active": true, "sub": "alice", "scope": "notes.read", "exp": 1790000000, ...}
```

```
spring.security.oauth2.resourceserver.opaquetoken.introspection-uri=https://idp.example.com/oauth2/introspect
spring.security.oauth2.resourceserver.opaquetoken.client-id=notes-api
spring.security.oauth2.resourceserver.opaquetoken.client-secret=${INTROSPECTION_SECRET}
```

With those properties Boot creates an `OpaqueTokenIntrospector` that calls the endpoint (classic implementation: `SpringOpaqueTokenIntrospector`). Spring Security 7.1 adds `RestClientOpaqueTokenIntrospector`, built on `RestClient`. To stay offline,

our project supplies its own `OpaqueTokenIntrospector` that answers from a map, in the same shape the real endpoint would:

```
class FakeIntrospector implements OpaqueTokenIntrospector {
    private final Map<String, Map<String, Object>> activeTokens = Map.of(
            "tok-alice-7f3a", Map.of("active", true, "sub", "alice", "scope", "notes.read", "exp", Instant.now().plusSeconds(600)),
            "tok-bob-91c2", Map.of("active", true, "sub", "bob", "scope", "notes.read notes.write", "exp", Instant.now().plusSeconds(600)));

    @Override
    public OAuth2AuthenticatedPrincipal introspect(String token) {
        Map<String, Object> claims = activeTokens.get(token);
        if (claims == null) throw new BadOpaqueTokenException("Provided token isn't active"); // {"active":false}
        List<GrantedAuthority> authorities = Arrays.stream(((String) claims.get("scope")).split(" "))
                .map(s -> (GrantedAuthority) new SimpleGrantedAuthority("SCOPE_" + s)).toList();
        return new OAuth2IntrospectionAuthenticatedPrincipal((String) claims.get("sub"), claims, authorities);
    }
}

// chain for /opaque/**
http.securityMatcher("/opaque/**")
    .authorizeHttpRequests(a -> a
        .requestMatchers("/opaque/notes/write").hasAuthority("SCOPE_notes.write")
        .anyRequest().hasAuthority("SCOPE_notes.read"))
    .oauth2ResourceServer(rs -> rs.opaqueToken(o -> o.introspector(introspector)));
```

Output:

```
alice GET   -> 200 notes for alice (scope=notes.read)
alice write -> 403
bob write   -> 200 written by bob
inactive    -> 401 Bearer error="invalid_token", error_description="Provided token isn't active", ...
opaqueToken() -> 200 notes for carol (scope=notes.read)
```

The principal is an `OAuth2AuthenticatedPrincipal` (not a `Jwt`), and the last line shows the `opaqueToken()` test post-processor.

JWT vs opaque

	JWT (self-contained)	Opaque + introspection
Validation	local, just a signature check	network call to the IdP per request (or per cache period)
Revocation	only when it expires (or via a deny-list)	**immediate**: the IdP answers `active: false`
Latency (time per request) / IdP load	minimal	higher, and the IdP must be highly available (almost never down)
Privacy	claims readable by any holder	nothing readable
Typical use	APIs inside your platform	high-risk operations, tokens handed to third parties

A common hybrid is **opaque tokens outside, JWTs inside**: the **API gateway** (the single entry point that sits in front of all your services and routes requests to them) introspects the opaque token and forwards a short-lived JWT to internal services (the "phantom token" pattern).

TL;DR

Opaque tokens are meaningless strings that the API checks with the IdP (RFC 7662). You get instant revocation and privacy, and pay in latency and IdP load. `oauth2ResourceServer(rs -> rs.opaqueToken(...))`.

11.8 Multi-tenancy: several issuers in one API

What it is

A **multi-tenant** API serves several customers ("tenants"), each with its *own* identity provider or **realm** (Keycloak's word for a separate set of users, clients and keys), and so its own issuer and keys. The token's `iss` claim tells you which tenant it claims to come from. The API must pick **that tenant's keys** to verify it, and reject issuers it does not trust.

Spring's `JwtIssuerAuthenticationManagerResolver` reads `iss` *without trusting it yet*, looks up an `AuthenticationManager` (the Spring Security component that decides whether a credential is valid, Chapter 10) for that issuer, and lets that manager verify the signature with the issuer's keys:

```
Map<String, AuthenticationManager> managers = new ConcurrentHashMap<>();
JwtIssuerAuthenticationManagerResolver resolver = new JwtIssuerAuthenticationManagerResolver(issuer -> {
    if (!keys.isTrusted(issuer)) return null;         // unknown issuer -> 401
    return managers.computeIfAbsent(issuer, iss -> {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey(keys.publicKey(iss)).build();
        decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer(iss));
        return new ProviderManager(new JwtAuthenticationProvider(decoder));
    });
});
http.securityMatcher("/tenant/**")
    .authorizeHttpRequests(a -> a.anyRequest().authenticated())
    .oauth2ResourceServer(rs -> rs.authenticationManagerResolver(resolver));
```

Output:

```
tenant A token            -> 200 ann from https://idp.tenant-a.test
tenant B token            -> 200 ben from https://idp.tenant-b.test
B-signed, iss = tenant A  -> 401
untrusted issuer          -> 401
```

The third line is the important one. A token **signed by tenant B's key that claims to be from tenant A** is rejected, because the resolver verified it with tenant A's key. With real IdPs that publish discovery metadata, one line does the same job: `JwtIssuerAuthenticationManagerResolver.fromTrustedIssuers("https://idp.tenant-a.test", "https://idp.tenant-b.test")`. It discovers each issuer's JWK Set on first use.

Common mistakes

- Building a decoder for **any** `iss` value the token names ("dynamic issuer" without an allow-list). An attacker then runs their own IdP and names it. Always check against a trusted list.
- Forgetting tenant isolation *after* authentication. The token proves the user belongs to tenant A. Your queries must still filter by tenant.

TL;DR

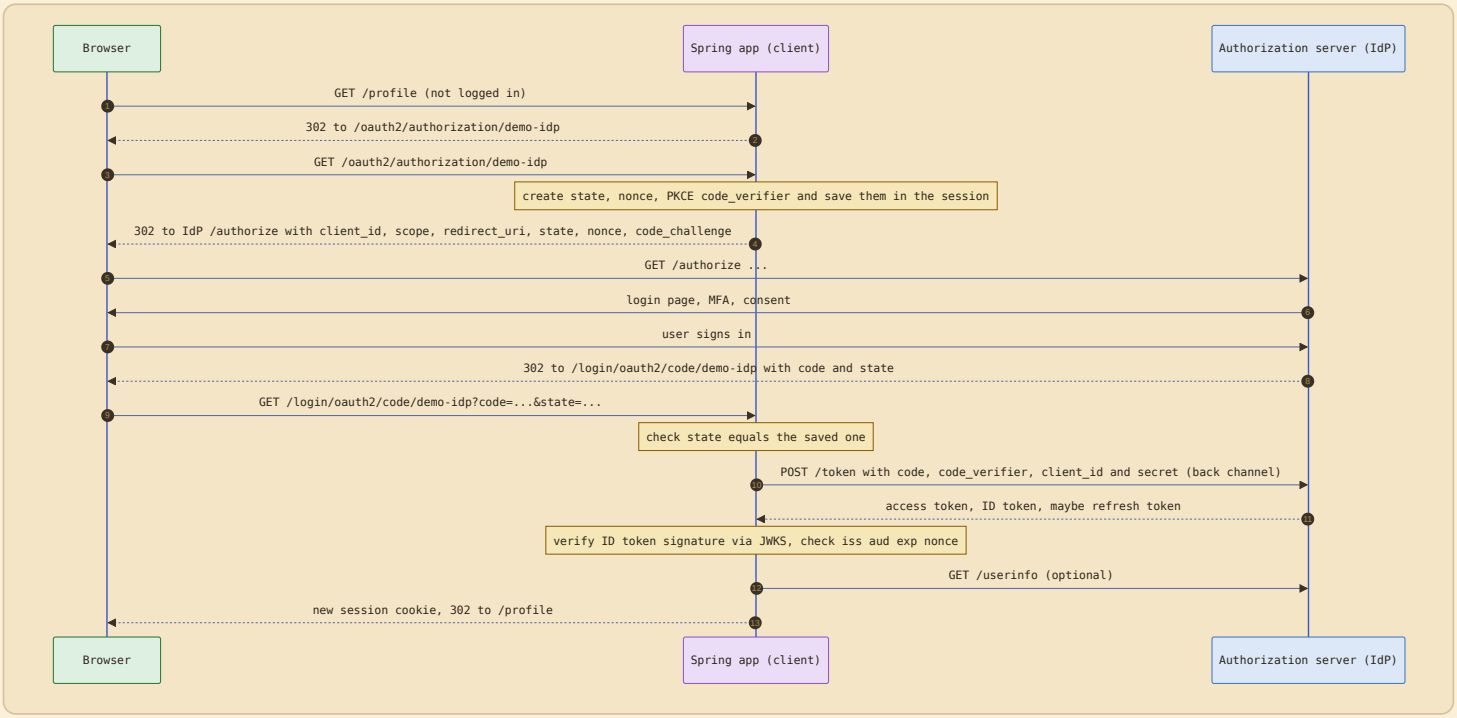
Several issuers → `JwtIssuerAuthenticationManagerResolver` with an allow-list. The `iss` claim chooses the keys, and the signature must match that issuer's key.

11.9 OAuth2 login: "Sign in with ..."

What it is

OAuth2 login lets users sign in to *your web app* with an account at an identity provider (Google, GitHub, Keycloak, your company's Entra ID). Your app is the **client**. It never sees the password. It receives an **authorization code** (a short-lived, single-use code that can be swapped for tokens), exchanges it for tokens *server-side*, and creates a normal session for the browser. This is the **authorization-code flow**, and with OIDC it also yields an ID token.

How it works



The protective parameters:

Parameter	Protects against	How
`state`	CSRF on the callback: an attacker making your browser finish *their* login	random value stored in the session and compared on return
`nonce` (OIDC; "number used once")	**replay** (an attacker re-sending a captured valid message) of an old ID token	random value that must come back inside the ID token
PKCE (`code_challenge` + `code_verifier`, RFC 7636)	a stolen authorization code	the app sends `SHA-256(verifier)` (a one-way hash, §11.18) first, and only the holder of the verifier can redeem the code
exact `redirect_uri`	codes sent to an attacker's site	the IdP only redirects to pre-registered URIs
code exchange on the **back channel**	tokens exposed to the browser	tokens travel server-to-server, and the browser only gets a session cookie

Configuration

With a real OIDC provider, Boot needs just the client credentials and the issuer:

```
spring.security.oauth2.client.registration.demo-idp.client-id=notes-web
spring.security.oauth2.client.registration.demo-idp.client-secret=${DEMO_IDP_SECRET}
spring.security.oauth2.client.registration.demo-idp.scope=openid,profile,email
spring.security.oauth2.client.provider.demo-idp.issuer-uri=https://idp.example.test
```

Boot has built-in provider defaults for `google`, `github`, `facebook` and `okta` (for those, the registration id is enough). Our project spells out every endpoint instead of `issuer-uri`, so it starts without contacting anyone:

```
spring.security.oauth2.client.registration.demo-idp.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.demo-idp.redirect-uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.provider.demo-idp.authorization-uri=https://idp.example.test/oauth2/authorize
spring.security.oauth2.client.provider.demo-idp.token-uri=https://idp.example.test/oauth2/token
spring.security.oauth2.client.provider.demo-idp.jwk-set-uri=https://idp.example.test/oauth2/jwks
spring.security.oauth2.client.provider.demo-idp.user-info-uri=https://idp.example.test/userinfo
spring.security.oauth2.client.provider.demo-idp.user-name-attribute=sub
```

```
http.authorizeHttpRequests(a -> a.requestMatchers("/", "/public/**").permitAll().anyRequest().authenticated())
    .oauth2Login(Customizer.withDefaults())
    .logout(Customizer.withDefaults());

@GetMapping("/profile")
String profile(@AuthenticationPrincipal OidcUser user) {
    return "hello " + user.getFullName() + " <" + user.getEmail() + ">, sub=" + user.getSubject()
            + ", issuer=" + user.getIssuer();
}
```

What we can run without an IdP

Steps 1–4 of the diagram (everything before the browser leaves our app; a **302** is an HTTP redirect telling the browser to go to another URL) and the callback's `state` check run for real. The user signing in at the IdP and the code exchange are not run, since there is no IdP. The `oidcLogin()` post-processor then simulates a finished login:

```
1. GET /profile (anonymous)       -> 302 /oauth2/authorization/demo-idp
2. GET /oauth2/authorization/demo-idp -> 302 to https://idp.example.test/oauth2/authorize
     client_id=notes-web
     code_challenge=<random>
     code_challenge_method=S256
     nonce=<random>
     redirect_uri=http://localhost/login/oauth2/code/demo-idp
     response_type=code
     scope=openid%20profile%20email
     state=<random>
   session attribute saved: true
3. callback with forged state      -> 302 /login?error
oidcLogin() GET /profile -> 200 hello Jane Doe <jane@example.com>, sub=248289761001, issuer=https://idp.example.test
```

- The authorization request contains `state`, `nonce` **and PKCE** (`code_challenge`, `S256`) even though this client has a secret. Spring Security 7 uses PKCE (Proof Key for Code Exchange) for **confidential clients** too (clients that run on a server and can keep a secret; browser and mobile apps are *public* clients that cannot). If an old IdP can't handle it, turn it off for that registration by building it with `.clientSettings(ClientRegistration.ClientSettings.builder().requireProofKey(false).build())`.
- A callback whose `state` was never issued to this browser session is refused (`/login?error`). This is the CSRF protection of the login flow.
- After a real login the principal is an `OidcUser` (ID-token claims + userinfo). The tokens are stored server-side as an `OAuth2AuthorizedClient`, and the browser only holds the session cookie. So OAuth2 login is **session-based**: keep CSRF protection on (Chapter 10).

OAuth2 vs OIDC login

With the `openid` scope Spring performs **OIDC** login: it validates the ID token and builds an `OidcUser`. Without it (for example GitHub, which is plain OAuth2) Spring calls the provider's user-info endpoint and builds an `OAuth2User` from the JSON. Mapping external users to *your* accounts (first login = sign-up, linking by verified e-mail, local roles) is your job. Do it in a custom `OidcUserService` / `OAuth2UserService`, or in a **listener** (a method Spring calls when a given event is published) for the login success event.

> **TL;DR**
>
> `oauth2Login()` + a client registration = "Sign in with X" via authorization code, `state`, `nonce` and PKCE. Tokens stay on the server, and the browser gets a session. Test with `oidcLogin()` / `oauth2Login()`.

11.10 Calling other services: client credentials and token relay

What it is

Your service often has to call *another* protected API. There are two cases:

Case	Whose token?	Grant
a nightly job calls the reports API	the **service's own** identity	**client credentials**: the service authenticates with its client id + secret (or a private-key JWT / mTLS, mutual TLS where both sides show a certificate) and gets an access token
a user's request fans out to **downstream** APIs (the services your service calls in turn)	the **user's** identity	**token relay**: forward the user's access token, or better, exchange it (RFC 8693) for one aimed at the downstream API

How it looks in Spring (not run: needs a token endpoint)

Configure a registration and let `OAuth2AuthorizedClientManager` fetch, cache and renew tokens:

```
spring.security.oauth2.client.registration.reports.provider=demo-idp
spring.security.oauth2.client.registration.reports.client-id=nightly-job
spring.security.oauth2.client.registration.reports.client-secret=${REPORTS_SECRET}
spring.security.oauth2.client.registration.reports.authorization-grant-type=client_credentials
spring.security.oauth2.client.registration.reports.scope=reports.read
```

```
// RestClient (Chapter 6) with an interceptor that adds "Authorization: Bearer <token>"
@Bean
RestClient reportsClient(RestClient.Builder builder, OAuth2AuthorizedClientManager manager) {
    return builder.baseUrl("https://reports.internal")
            .requestInterceptor(new OAuth2ClientHttpRequestInterceptor(manager))
            .build();
}

String csv = reportsClient.get().uri("/daily")
        .attributes(clientRegistrationId("reports"))   // which registration's token to use
        .retrieve().body(String.class);
```

With **HTTP interfaces** (Chapter 6; a Java interface whose annotated methods Spring turns into HTTP calls), Spring Security 7 lets you put `@ClientRegistrationId` on the whole interface. Register an `OAuth2RestClientHttpServiceGroupConfigurer` bean so the HTTP service clients get the interceptor:

```
@HttpExchange("/reports")
@ClientRegistrationId("reports")            // type-level support is new in Spring Security 7
interface ReportsApi {
    @GetExchange("/daily") String daily();
}

@Bean
OAuth2RestClientHttpServiceGroupConfigurer securityConfigurer(OAuth2AuthorizedClientManager manager) {
    return OAuth2RestClientHttpServiceGroupConfigurer.from(manager);
}
```

The **interceptor** (code that runs around every outgoing request and can change it, here by adding the `Authorization` header) asks the manager for a token each time. The manager returns the cached one while it is valid, and requests or refreshes one when needed. You never parse expiry yourself.

Token relay. In a resource server, the incoming token is available as `JwtAuthenticationToken#getToken().getTokenValue()`, and an interceptor can copy it to outgoing calls. In **Spring Cloud Gateway**, the `TokenRelay` filter does this for logged-in users (Chapter 16). Forwarding the same token to every service means every service must accept the same audience. **Token exchange** (supported by Spring Security's OAuth2 client since 6.3) gets a new, narrower token per downstream API instead.

TL;DR

Services call APIs with **client credentials**, via `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor` on `RestClient` (or `@ClientRegistrationId` on HTTP interfaces). User context flows by token relay or, better, token exchange.

11.11 Refresh tokens, lifetimes, revocation and logout

What it is

Short-lived access tokens limit damage, but users shouldn't have to log in every 10 minutes. A **refresh token** is a long-lived credential that the client sends **only to the authorization server** to get a new access token without user interaction.

Practice	Why
access token 5–15 min, refresh token hours/days	a stolen access token is soon useless
refresh-token rotation: every use returns a *new* refresh token and invalidates the old one	a stolen refresh token used twice is detected, and the IdP revokes the whole family
bind refresh tokens to the client (secret, DPoP, i.e. proof that the caller holds a private key, or mTLS)	a stolen token alone is not enough
never give refresh tokens to browser JavaScript	XSS could steal a long-lived credential (§11.12)

In Spring, the resource server never sees refresh tokens. The **client side** handles them: the `OAuth2AuthorizedClientManager` behind `oauth2Login` and the interceptor above refreshes an expired access token automatically when a refresh token is present.

Revocation and logout

- **Revoking** a token (RFC 7009) tells the IdP it's dead. Opaque tokens stop working at once. JWTs keep working until `exp`, unless your APIs check a deny-list of `jti` values (usually not worth the cost if lifetimes are short).
- **Logging out** of your app (`POST /logout`) ends *your* session but not the user's session at the IdP. The next "Sign in with ..." logs them straight back in. For single logout, configure **OIDC RP-initiated logout** (RP = relying party, the app that relies on the IdP for login) so Spring also redirects to the IdP's `end_session_endpoint`:

```
http.logout(l -> l.logoutSuccessHandler(new OidcClientInitiatedLogoutSuccessHandler(clientRegistrationRepository)));
```

Spring Security also supports **OIDC back-channel logout** (`http.oidcLogout(...)`), where the IdP calls your app to end sessions when the user logs out elsewhere.

TL;DR

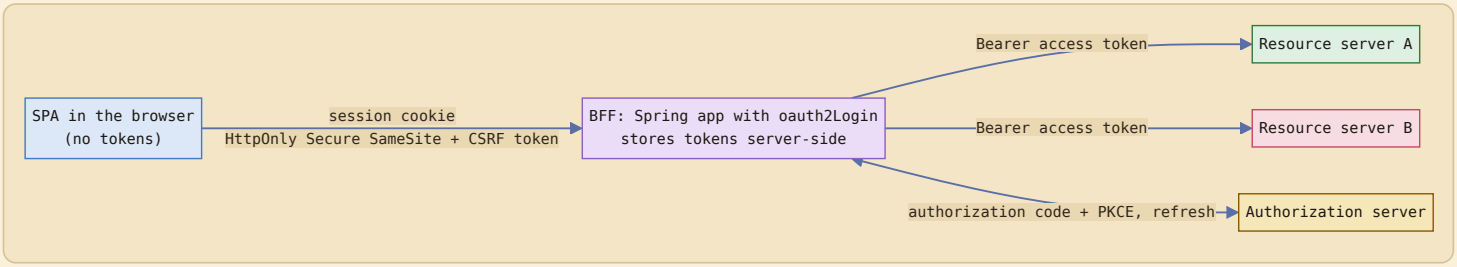
Short access tokens + rotating refresh tokens, handled by the client-side `OAuth2AuthorizedClientManager`. Revocation is instant only for opaque tokens. Use OIDC RP-initiated logout to end the IdP session too.

11.12 Where should a browser keep tokens? The BFF pattern

The problem it solves

A single-page app that holds tokens in JavaScript has to store them somewhere. `localStorage` and `sessionStorage` are readable by **any script on the page**, so one XSS bug leaks every user's tokens (see 14.3 Web Storage and 14.10 XSS). In-memory storage is lost on reload and is still reachable by injected script.

The recommended answer: Backend for Frontend (BFF)



1. The SPA talks only to its **BFF** (Backend for Frontend) on the same site, using an `HttpOnly` session cookie that JavaScript cannot read.
2. The BFF is a confidential OAuth2 client (`oauth2Login`, §11.9). It keeps access and refresh tokens server-side and refreshes them.
3. The BFF forwards API calls with the bearer token (token relay, §11.10). Spring Cloud Gateway with `TokenRelay` is a common BFF.

4. Because the SPA now uses cookies, **CSRF protection is required**. Use `csrf.spa()` (10.8).

The IETF (Internet Engineering Task Force, the body that publishes RFCs) draft *OAuth 2.0 for Browser-Based Applications* recommends this architecture as the most secure option. See also the BFF pattern notes.

Where tokens live	XSS steals tokens?	CSRF risk	Complexity
`localStorage`	**yes**	no	low
JS memory only	harder, still possible while the page runs	no	low, but lost on reload
BFF (`HttpOnly` cookie session)	no, the tokens never reach the browser	yes, so use CSRF tokens	an extra backend

> **TL;DR**
>
> Don't keep long-lived tokens in browser JavaScript. Put a BFF in front: the browser gets an `HttpOnly` session cookie (plus CSRF protection), and the tokens stay on the server.

11.13 Storing keys and secrets

Passwords are covered in 10.5: hash them with `DelegatingPasswordEncoder`. Token systems add more secrets:

Secret	Where it lives	Rules
JWT signing private key	the authorization server only: a vault (a secret store), KMS (key management service) or HSM (hardware security module)	never in Git, never in a resource server. Resource servers get *public* keys from the JWKS URL
Client secret (confidential clients)	environment variable (a named value set outside the program by the OS or platform) / secret manager, injected as `${...}` like `${DEMO_IDP_SECRET}` above	rotate it, and prefer `private_key_jwt` or mTLS client authentication for high-value clients
Refresh tokens stored by a BFF	server-side session or database, encrypted at rest (encrypted while stored on disk)	never sent to the browser
HMAC secret (if you use HS256)	secret manager	at least 256 bits of randomness. Everyone who can verify can also forge

Key rotation without **downtime** (time when the service is unavailable):

1. Publish the *new* public key in the JWKS next to the old one (a different `kid`).
2. Wait longer than the verifiers' JWKS cache time. `NimbusJwtDecoder` caches the JWK Set and re-fetches it when it meets an unknown `kid`.
3. Start signing with the new key.
4. Remove the old key after the longest token lifetime has passed.

Our demo generates a fresh key pair at every start. Every issued token becomes invalid on restart, and several instances would disagree. That's fine for tests and wrong for production.

11.14 Spring Authorization Server

If you need your own identity provider, **Spring Authorization Server** implements OAuth 2.1 and OpenID Connect 1.0 on top of Spring Security: authorization code + PKCE, client credentials, refresh tokens, device authorization, token exchange, introspection, revocation, JWKS and discovery endpoints, and dynamic client registration. Since **Spring Security 7.0 it ships as part of Spring Security** (module `spring-security-oauth2-authorization-server`). Boot 4 provides `spring-boot-starter-security-oauth2-authorization-server`, configured under `spring.security.oauth2.authorizationserver.*`. PKCE is required by default for authorization-code clients.

Choose it when you want the identity provider inside your Spring codebase, customised in Java. Choose Keycloak or a cloud IdP when you want an off-the-shelf product with admin UI, federation and user management included. Either way, *your APIs* look the same: resource servers configured with an `issuer-uri`. Running an authorization server needs a user store, persistent client and authorization storage (JDBC, Java's standard database API), and key management, so it is beyond this chapter's runnable examples.

11.15 Rate limiting and brute-force protection

Spring Security does **not** include **rate limiting** (capping how many requests a client may make per time window; see System Design Fundamentals). Yet login and token endpoints are exactly what attackers hammer: password guessing, **credential stuffing** (trying username/password pairs leaked from other sites), and making you pay for slow bcrypt/Argon2 (deliberately slow password-hashing algorithms) checks (10.5). Common layers:

Layer	Tool	Limits
edge	API gateway / WAF (web application firewall) / CDN (content delivery network), Spring Cloud Gateway `RequestRateLimiter` (Redis, an in-memory data store; token bucket algorithm)	requests per IP / client / API key
application	Bucket4j (token bucket, local or distributed via Redis/JCache) in a servlet filter (code that runs before every HTTP request reaches the app)	per user or per endpoint rules
login logic	count failures per account *and* per IP (listen to `AuthenticationFailureBadCredentialsEvent`), add delays, CAPTCHA (a human-or-bot challenge) or MFA after N failures	brute force on one account
identity provider	Keycloak/cloud IdPs have brute-force detection built in	when login happens at the IdP

Avoid hard **account lockout** after a few failures. It lets anyone lock out any user. Prefer progressive delays and step-up checks (asking for an extra factor only when something looks risky). Framework 7's `@ConcurrencyLimit` (Chapter 4) caps *concurrent* executions, not requests per second, so it is not a rate limiter. Algorithms (token bucket, sliding window) are explained in System Design Patterns — Rate Limiting.

11.16 Common mistakes (whole chapter)

Mistake	Consequence	Fix
no audience check	tokens for *other* APIs accepted	`jwt.audiences` property or `JwtClaimValidator("aud", ...)`
custom validator *replacing* defaults	expired tokens accepted	combine with `JwtValidators.createDefaultWithIssuer(...)`
`hasAuthority("messages.read")`	always 403	scopes are `SCOPE_messages.read`
`jwt()` tests without your converter	tests disagree with production	`.authorities(yourConverter)` or real tokens
long-lived JWT access tokens	stolen token valid for days	minutes + refresh tokens, or opaque tokens
tokens in `localStorage`	XSS = account takeover	BFF with `HttpOnly` cookie
accepting any `iss` in multi-tenant setups	attacker-run IdP accepted	allow-list of issuers
CSRF disabled on the BFF / OAuth2-login app	CSRF on a cookie session	keep CSRF on (`csrf.spa()`)
ID token sent to APIs	wrong audience, identity confusion	send the **access** token
rolling your own token server for real users	endless security work	use an authorization server
private key in `application.properties` / Git	anyone can mint tokens	vault/KMS, JWKS for public keys

11.17 In the interview

"How does a Spring resource server validate a JWT?" "`BearerTokenAuthenticationFilter` extracts the token from the `Authorization` header. `JwtAuthenticationProvider` calls the `JwtDecoder`, which Nimbus backs with the issuer's JWK Set found via `issuer-uri` discovery. It checks the algorithm, picks the key by `kid`, and verifies the signature. Then validators check `exp` and `nbf` with 60 seconds of skew, `iss`, `typ`, and the audience I configure. `JwtAuthenticationConverter` maps `scope` to `SCOPE_` authorities, plus any role claims I map. A failure returns 401 `invalid_token`, and a missing authority returns 403 `insufficient_scope`."

Follow-ups:

- *JWT or opaque token?* → JWT for local, fast validation inside the platform. Opaque when you need instant revocation or don't want claims exposed. The cost is an introspection call.
- *Why PKCE if the client has a secret?* → It binds the code to the party that started the flow, so a stolen code is useless. OAuth 2.1 requires it for all clients, and Spring Security 7 sends it by default.
- *How do you log a user out when tokens are stateless?* → Short access-token lifetimes, revoke the refresh token, end the local session, and use OIDC RP-initiated logout for the IdP session.
- *How do you store tokens in an SPA?* → Preferably not at all: a BFF keeps them server-side and gives the SPA an `HttpOnly` cookie with CSRF protection.
- *Service-to-service auth?* → Client credentials grant via `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor`, or mTLS inside a mesh. Relay or exchange the user's token when the user context matters.
- *What is `state` for?* → CSRF protection of the login callback. It ties the authorization response to the browser session that started the flow.

11.18 Security fundamentals beyond Spring

What it is

Everything in this chapter and the last one rests on a handful of cryptographic building blocks: hashes, symmetric and asymmetric keys, signatures, key exchange. Spring hides them behind `PasswordEncoder`, `JwtDecoder` and `https://` URLs, but interviews (and incident calls) expect you to know what they do. This section explains them from zero, shows each one with the plain JDK (Java Development Kit: the compiler plus standard library) in `code/crypto-basics`, and then covers the protocols built from them: TLS, SSH keys, one-time codes, passkeys and single sign-on.

Analogy: a **hash** is a fingerprint (identifies, cannot be turned back into the person); **symmetric encryption** is a padlock where sender and receiver hold copies of the same key; **asymmetric encryption** is a mailbox with a public slot anyone can post into and a private key only the owner has; a **signature** is a wax seal only one ring can make but everyone can check.

Encoding vs hashing vs encryption vs tokenisation

	Purpose	Key?	Reversible?	Example	Wrong use
Encoding	change representation so bytes survive a channel	no	yes, by anyone	Base64, URL encoding, UTF-8	"hiding" a password in Base64
Hashing	fixed-size fingerprint; detect change; store passwords	no (HMAC adds one)	**no**	SHA-256, BLAKE3; for passwords bcrypt/Argon2 (10.5)	fast SHA-256 for passwords (GPU-crackable: graphics cards can try billions of guesses per second)
Encryption	confidentiality: only key holders can read	yes	yes, with the key	AES-GCM, ChaCha20-Poly1305, RSA-OAEP	encrypting passwords (then someone can decrypt them)
Tokenisation	replace sensitive data with a random stand-in; the real value lives in a secured vault	vault lookup	only via the vault	card number → `tok_8f3a…` in payment systems	—

An **HMAC** (hash-based message authentication code) is a hash mixed with a secret key: only key holders can compute or check it. HS256 JWTs and TOTP codes (below) are HMACs.

Symmetric vs asymmetric encryption

	Symmetric	Asymmetric (public-key)
Keys	one shared secret key	a key pair: **public** (share freely) + **private** (never leaves the owner)
Speed	very fast (AES has CPU instructions)	100–1000× slower; small data only
Problem	how do both sides get the key safely?	who does this public key really belong to?
Algorithms	AES-GCM, ChaCha20-Poly1305	RSA, elliptic curves (ECDH/X25519 for key exchange, ECDSA/Ed25519 for signatures)

Real systems are **hybrid**: asymmetric crypto agrees or protects a random symmetric key, and the bulk data goes through AES (Advanced Encryption Standard, the standard fast symmetric cipher). **Authenticated encryption** modes (GCM, Poly1305) add a tag, so any tampering makes decryption fail rather than returning garbage — the demo proves it.

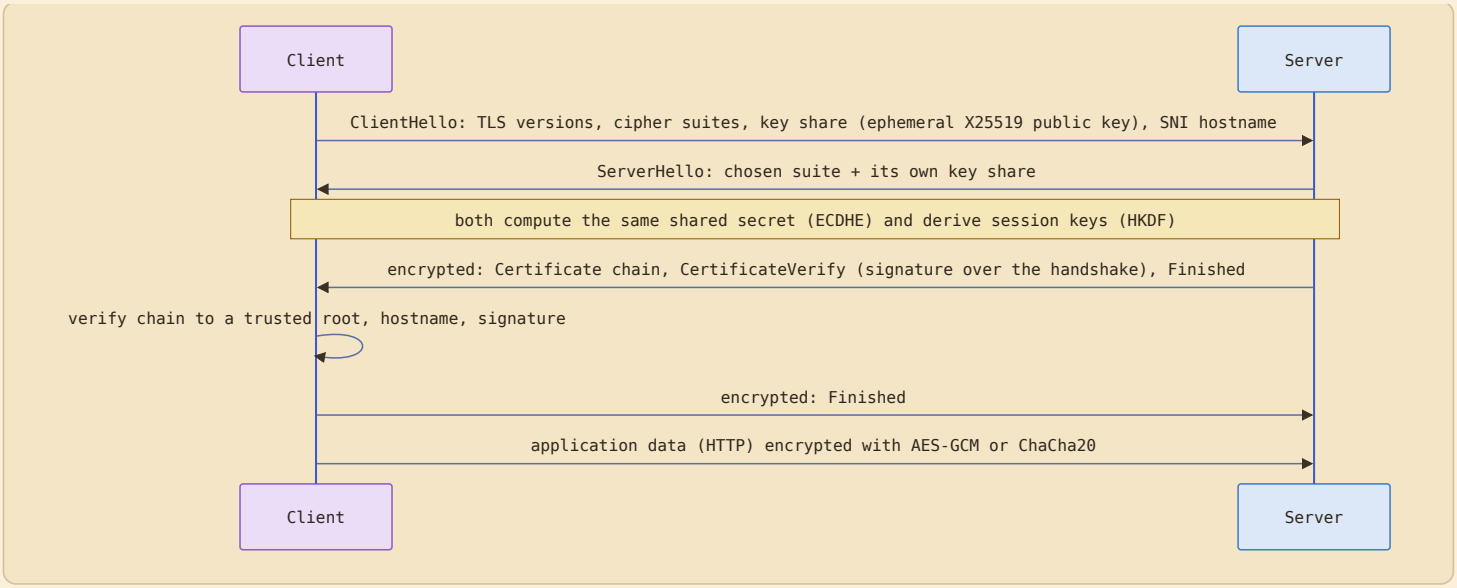
Digital signatures and certificates

A **digital signature** is computed with the private key over a hash of the message; anyone with the public key can verify that (1) the holder of that private key signed it and (2) not one bit changed. That is exactly how an RS256/ES256 JWT is trusted (§11.1).

A **certificate** (X.509) is a signed statement "this public key belongs to `api.example.com`", signed by a **certificate authority (CA)**, an organisation that browsers and operating systems trust to vouch for keys. Browsers and the JDK ship a list of trusted root CAs; a server sends its certificate plus intermediates, and the client checks the **chain** up to a trusted root, the name, and the validity dates.

The TLS 1.3 handshake

HTML book §1.4 says what TLS gives (confidentiality, integrity, server authentication). Here is how, in one round trip (RFC 8446):



Key ideas: the **key exchange** (ECDHE) gives both sides a secret nobody on the wire can compute; the **signature** in `CertificateVerify` proves the server owns the certificate's private key; the keys are **ephemeral**, so recording traffic today and stealing the server key later reveals nothing (**forward secrecy**). **mTLS** (mutual TLS) adds a client certificate so the server authenticates the client too — common between services (§11.10). In Spring Boot, TLS is configured with SSL bundles (`server.ssl.bundle`), usually terminated at a **load balancer** (a component that spreads incoming traffic over several servers) or **ingress** (the Kubernetes entry point for outside traffic) instead; see System Design Fundamentals.

SSH keys

SSH logs you into servers and Git hosts. `ssh-keygen -t ed25519` creates a key pair; you put the **public** key in the server's `~/.ssh/authorized_keys` (or your GitHub settings). On login the server sends a challenge, your client **signs** it with the private key, and the server verifies with the stored public key — no password crosses the wire. The first time you connect, SSH shows the **server's** host-key fingerprint; accepting it stores it in `known_hosts`, and a later mismatch warns of a possible **man-in-the-middle** (an attacker sitting between you and the server, reading or changing traffic). Protect the private key with a passphrase and `ssh-agent`; never copy it between machines.

TOTP: the six digits in an authenticator app

TOTP (time-based one-time password, RFC 6238) builds on **HOTP** (RFC 4226):

1. At enrolment the server generates a random secret and shows it as a QR code (`otpauth://totp/...?secret=BASE32...`); the app stores it. Both sides now share a secret.
2. Every 30 seconds both compute `counter = floor(unixTime / 30)`, then `HMAC-SHA1(secret, counter)`.
3. **Dynamic truncation**: the last 4 bits of the HMAC pick an offset; 31 bits from there, `mod 10^6`, give six digits.
4. The server accepts the current step and usually ±1 step for clock drift, and should remember used codes so one cannot be replayed.

`Totp.java` implements it in 30 lines; `TotpTest` checks every RFC 6238 Appendix B test vector (SHA-1, SHA-256 and SHA-512) and the RFC 4226 HOTP vectors.

```
byte[] h = mac.doFinal(ByteBuffer.allocate(8).putLong(counter).array()); // HMAC(secret, 8-byte counter)
int offset = h[h.length - 1] & 0x0f;                                     // dynamic truncation
int binary = ((h[offset] & 0x7f) << 24) | ((h[offset + 1] & 0xff) << 16)
           | ((h[offset + 2] & 0xff) << 8) | (h[offset + 3] & 0xff);
return String.format("%0" + digits + "d", binary % (int) Math.pow(10, digits));
```

TOTP is phishable (**phishing** = a fake look-alike site tricking you into typing secrets; a fake site can ask for the code and replay it within 30 s) and depends on a shared secret stored on the server. That is what passkeys fix.

Passkeys (WebAuthn / FIDO2)

A **passkey** is a key pair created by your device (phone, laptop, security key) **for one website**. Registration: the browser's WebAuthn API asks the device's **authenticator** to create a key pair scoped to the site's domain (the **relying party ID**); the site stores only the public key. Login: the site sends a random challenge; the authenticator checks you (fingerprint, face, PIN), **signs** the challenge together with the **origin** (scheme + domain + port, such as `https://shop.example.com`), and the site verifies with the stored public key.

Why it is stronger than passwords + TOTP: nothing secret is stored on the server (a database leak gives attackers only public keys), and the signature covers the origin, so a look-alike phishing domain gets a signature that is useless on the real site. Synced passkeys (iCloud Keychain, Google Password Manager) survive losing a phone. Spring Security supports them since 6.4 with `http.webAuthn(w -> w.rpName("Shop").rpId("shop.example.com").allowedOrigins("https://shop.example.com"))` (module `spring-security-webauthn` in 7.x), and records a `FACTOR_WEBAUTHN` authority for multi-factor rules (10.4).

Single sign-on: SAML vs OpenID Connect

SSO means logging in once at an identity provider (IdP) and being let into many applications (service providers / relying parties) without new passwords.

	SAML 2.0 (2005)	**OpenID Connect (2014, on OAuth 2.0)**
Token	signed **XML assertion** (XML: a tag-based text format)	signed **JWT** ID token (§11.2)
Transport	browser POSTs/redirects the assertion to the app	authorization-code flow + PKCE, back-channel token call
Fits	enterprise web apps, older corporate IdPs (AD FS)	web, SPA/BFF, mobile, APIs; also gives access tokens
Spring	`saml2Login()` (`spring-security-saml2-service-provider`)	`oauth2Login()` (§11.9)

Both rely on the same trick: the IdP **signs** a statement about the user, and the app verifies the signature with the IdP's published key or certificate. For new systems pick OIDC; add SAML when an enterprise customer's IdP only speaks SAML (Keycloak, Okta and Entra ID speak both and can bridge).

Seeing it: `code/crypto-basics`

Plain JDK 25, no Spring beans. `mvn -q -B -ntp verify` runs 13 JUnit (the standard Java test framework) tests (RFC vectors, AES-GCM tamper detection, signatures, X25519 agreement); then `java -cp target/classes handbook.security.CryptoBasicsDemo` prints:

```
encode  Base64      : cGF5IGJvYiAxMDA=  -> decodes back to: pay bob 100
hash    SHA-256     : aa843d8373c66e26...  ('pay bob 900' -> 70ef8a2487a6d483...)
encrypt AES-256-GCM : 27 bytes (11 plaintext + 16 auth tag) -> decrypts to: pay bob 100
encrypt tampered    : AEADBadTagException (integrity check failed)
sign    Ed25519     : 64-byte signature; verify original = true, verify 'pay bob 900' = false
ECDHE   X25519      : both sides derived the same 32-byte secret = true (e4582f7aff4d...)
ECDHE   record      : server decrypts client's record: GET /
TOTP    t=59          : counter=1         code=94287082
TOTP    t=1111111109  : counter=37037036  code=07081804
TOTP    t=1234567890  : counter=41152263  code=89005924
TOTP    t=2000000000  : counter=66666666  code=69279037
TOTP    6-digit app : 921300 valid 25 s later = true, valid 5 min later = false
```

The four `TOTP t=` codes are exactly the SHA-1 column of RFC 6238 Appendix B. Keys and the X25519 secret are random on every run.

Common mistakes

- Calling Base64 "encryption" → it is encoding; anyone can decode it.
- Hashing passwords with plain SHA-256 or MD5 → use a slow, salted password hash (bcrypt, Argon2, PBKDF2).
- Reusing an AES-GCM IV (initialisation vector: a per-message nonce) with the same key → breaks confidentiality and integrity; use a random 12-byte IV per message.
- Rolling your own crypto or protocol → use the JDK, TLS, and vetted libraries; your job is choosing parameters and managing keys.
- Disabling certificate checks ("trust all") to make a test pass → it removes the *authentication* half of TLS in production too.
- Comparing codes or MACs with `equals` → use a constant-time comparison (`MessageDigest.isEqual`) to avoid timing leaks.

In the interview

30-second answer — how HTTPS works: "TLS 1.3 does an ephemeral Diffie-Hellman key exchange (a method for two parties to agree on a shared secret over an open network) in the first round trip, so client and server derive the same session keys without sending them. The server proves its identity by signing the handshake with the private key matching its certificate, and the client checks the certificate chain up to a trusted CA and the hostname. After that, data is encrypted and integrity-protected with a fast symmetric cipher such as AES-GCM. Ephemeral keys give forward secrecy."

- *Hashing vs encryption?* → Hashing is one-way and keyless (fingerprints, password storage); encryption is reversible with a key (confidentiality).
- *Why both symmetric and asymmetric?* → Asymmetric solves key distribution and identity but is slow; symmetric is fast. TLS uses the first to set up the second.
- *How does an authenticator app know the right code offline?* → Shared secret + current time → HMAC → truncation (TOTP, RFC 6238).
- *Why are passkeys phishing-resistant?* → The authenticator signs a challenge bound to the site's origin, and the server holds only a public key.
- *SAML vs OIDC?* → Both are SSO with an IdP-signed statement; SAML uses XML assertions via browser POST, OIDC uses JWT ID tokens over OAuth 2.0 and suits mobile and APIs.

Self-check

1. Which primitive protects a JWT's claims from being changed, and which would hide them? → A signature (JWS) / encryption (JWE).

2. What gives TLS forward secrecy? → Ephemeral (EC)DHE key exchange; the long-term key only signs.
3. What does the server store for a passkey? → The user's public key and a credential ID.
4. Why does a TOTP server accept ±1 step? → To tolerate clock drift between phone and server.

> **TL;DR**

Encoding is reversible without a key, hashing is one-way, encryption needs a key, tokenisation swaps data for a vault reference. Symmetric = one fast shared key; asymmetric = public/private pair for key exchange and signatures; real systems combine them (TLS 1.3: ECDHE + certificate signature + AES-GCM). SSH and passkeys log in by signing a challenge; TOTP = HMAC(secret, time/30) truncated to six digits. SSO: SAML (XML) or OIDC (JWT) — prefer OIDC for new work.

Sources: RFC 6238 (TOTP), RFC 4226 (HOTP), RFC 8446 (TLS 1.3), RFC 4251/4252 (SSH), RFC 7748 (X25519), RFC 8032 (Ed25519), NIST SP 800-38D (GCM), W3C Web Authentication Level 2, OASIS SAML 2.0 Core, OpenID Connect Core 1.0, Spring Security reference (Passkeys, SAML 2.0 Login), JDK 25 `java.security` / `javax.crypto` API docs.

Chapter 11 TL;DR

- **Tokens**: a JWT is `header.payload.signature`. Claims are readable, and the signature is what makes them trustworthy. Prefer asymmetric keys (RS256/ES256) published as a JWKS.
- **OAuth 2.0/OIDC**: authorization code + PKCE for users, client credentials for services, refresh tokens for renewal. Implicit and password grants are gone (Spring Security 7 removed password-grant support).
- **Resource server**: `oauth2ResourceServer(jwt)` + `issuer-uri`. It validates signature, `exp`/`nbf` (60 s skew), `iss`, `typ`, and your `aud`. 401 `invalid_token` vs 403 `insufficient_scope`. It publishes RFC 9728 metadata.
- **Authorities**: `scope` → `SCOPE_x` by default. Map role claims with a converter or Boot properties.
- **Testing**: real tokens from a local `NimbusJwtEncoder` for decoder rules, `jwt()`/`opaqueToken()`/`oidcLogin()` for access rules. Pass your converter to `jwt()`.
- **Variants**: opaque tokens via introspection (instant revocation). Multi-tenant via `JwtIssuerAuthenticationManagerResolver` with an issuer allow-list.
- **Clients**: `oauth2Login` (state, nonce, PKCE, tokens server-side), `OAuth2AuthorizedClientManager` + `RestClient` interceptor or `@ClientRegistrationId` HTTP interfaces, token relay or exchange.
- **Browsers**: BFF + `HttpOnly` cookie + CSRF, and no tokens in `localStorage`.
- **Operations**: keys in a vault and rotated via `kid`, Spring Authorization Server (now inside Spring Security 7) if you must run an IdP, rate limiting at the edge.
- **Fundamentals** (§11.18): encoding ≠ hashing ≠ encryption; hybrid crypto (asymmetric key exchange + signatures, symmetric AES-GCM for data) is how TLS 1.3 works; SSH keys and passkeys sign a challenge; TOTP = HMAC(secret, time/30) truncated; SSO via SAML (XML) or OIDC (JWT).

Next: Chapter 12 — Testing Spring apps covers the test slices used here in depth.

Chapter 12 — Testing Spring Applications

A Spring application is plain Java objects plus a **container** (Spring's *application context*: the object that creates your **beans**, i.e. the objects Spring manages, and plugs them into each other) that wires them together, adds **proxies** (wrapper objects that add behaviour such as transactions around your real object) and talks to the outside world (HTTP, databases, other services). Good tests match that shape. Most of your logic can be tested **without Spring at all**. Some things only exist inside the container: URL mappings, JSON conversion, validation, SQL generated by Spring Data, **transactions** (a group of database changes that succeed or fail together), security filters. For those you start **just enough of the container** to run them. A few tests start **everything** and send real HTTP requests to prove the parts fit together.

This chapter shows how to do each of these with Spring Boot 4's test support, and how to keep the **suite** (the whole collection of tests in a project) fast as it grows. Every behaviour is proved by a test whose real output is shown.

What you'll learn

1. The test pyramid for a Spring app, with real timings for each layer.
2. The Boot 4 test starters (one per technology) and what `spring-boot-starter-test` contains (JUnit 6, AssertJ, Mockito, ...).
3. How the **Spring TestContext Framework** loads, injects and caches application contexts.
4. `@SpringBootTest` and its four web environments.
5. **Test slices**: `@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest` and the rest.
6. `MockMvcTester` (AssertJ-style MockMvc) and `RestTestClient` (new in Framework 7.0).
7. `@MockitoBean` / `@MockitoSpyBean`, which replaced Boot's `@MockBean` (removed in Boot 4).
8. **Context caching**: what makes a new context, measured hit/miss counts and timings, `@DirtiesContext`, context pausing (Framework 7.0).
9. `@DynamicPropertySource` for values only known at runtime (random ports, container URLs).
10. **Testcontainers +** `@ServiceConnection` for real databases in tests (shown, not run: needs Docker).
11. Test profiles and test properties.
12. **Types of API testing** (§12.18): smoke, functional, integration, contract, regression, load/stress/soak, security, fuzz and UI — the question each answers and the Spring or Java tool for it, with a runnable fuzz test.

All code is in `code/`. The projects use Spring Boot **4.1.1** (Framework 7.0, JUnit 6.0 — the framework that finds and runs Java tests, Mockito 5 — a library that creates fake objects, AssertJ 3 — a library of readable `assertThat(...)` checks) and Java 25:

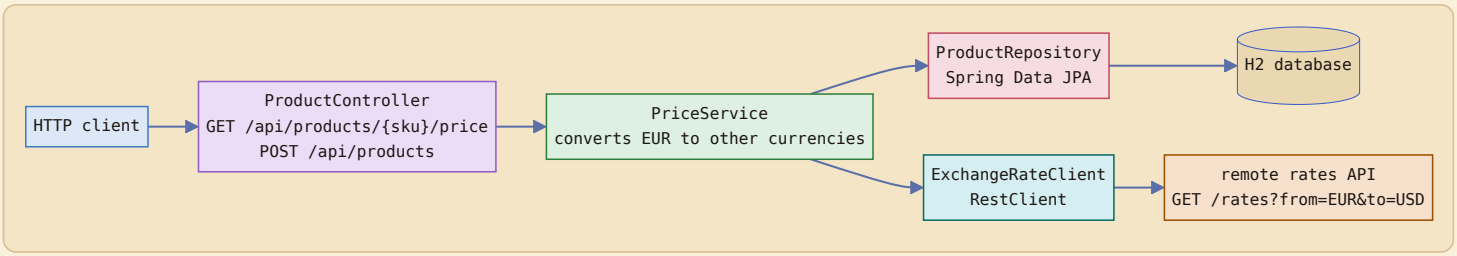
Project	Shows	Run here?
`testing-demo`	a small product-pricing API tested at every level: unit, slices, full context, real port, plus a seeded fuzz test	yes
`context-caching`	seven test classes that hit or miss the context cache, with timings	yes
`testcontainers-demo`	PostgreSQL (a popular open-source SQL database) in a container via `@ServiceConnection`	**needs Docker — not run** (compiled only)

Output blocks are real `mvn -q -B -ntp verify` output (Maven, the Java build tool, compiling the code and running every test). Spring's log lines are trimmed to `...`. **Timings are machine-dependent** (these came from a 14-core Apple M4 Pro laptop); only compare them with each other.

> **Prerequisites.** JUnit 5/6, Mockito and AssertJ basics are taught in the Java handbook, Chapter 18. This chapter assumes them and focuses on what Spring adds. The code under test uses Spring MVC (Chapter 5), `RestClient` (Chapter 6) and Spring Data JPA (Chapter 8). Security testing (`@WithMockUser`, `jwt()`) is in Chapter 10 and Chapter 11. Reactive testing (`StepVerifier`, `WebTestClient`) is in Chapter 13.

12.1 The application under test

`testing-demo` is a tiny catalogue service with the layers a real service has:



- `ProductController` maps HTTP to Java, validates the request body (`@Valid`) and turns `ProductNotFoundException` into a `404` Problem Detail (a standard JSON shape for API errors).
- `PriceService` holds the business rule: prices are stored in EUR; for another currency, fetch the rate and round to 2 decimals (`HALF_EVEN`, "banker's rounding": a 5 rounds towards the even neighbour). For EUR it must **not** call the remote API.

- `ProductRepository` is a Spring Data JPA interface (JPA = Java Persistence API, the standard for mapping Java objects to database tables) with a **derived query** (`findBySku`: Spring writes the SQL from the method name) and a **JPQL** query (`cheaperThan`: JPA's SQL-like language that talks about entity classes instead of tables).
- `ExchangeRateClient` calls a remote API with `RestClient`. Its base URL comes from the property `rates.base-url`.

The service, for reference:

```
@Service
public class PriceService {
    private final ProductRepository products;
    private final ExchangeRateClient rates;
    private final String home;

    public PriceService(ProductRepository products, ExchangeRateClient rates,
                        @Value("${catalog.home-currency}") String home) { ... }

    public PriceView priceOf(String sku, String currency) {
        Product p = products.findBySku(sku).orElseThrow(() -> new ProductNotFoundException(sku));
        BigDecimal amount = currency.equals(home)
                ? p.getPrice()                                  // no remote call needed
                : convert(p.getPrice(), rates.rate(home, currency));
        return new PriceView(p.getSku(), p.getName(), amount, currency);
    }
    ...
}
```

Every section below tests some part of this app, from the cheapest test to the most expensive.

12.2 The test pyramid in a Spring app

What it is

The **test pyramid** is a rule of thumb for how many tests of each kind to write. Write many small, fast tests at the bottom. Write fewer, slower tests that cover more of the system near the top. In a Spring app the layers map to how much of the container a test starts:

Layer	What starts	Spring annotation	Proves	Speed (this project)
Unit	nothing — `new` your class, mock (replace with a programmable fake) its collaborators	none (`@ExtendWith(MockitoExtension.class)`)	business logic, edge cases	~1 ms or less per test
Slice	one layer of the container (web, JPA, JSON, HTTP client)	`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest`, ...	mappings, validation, JSON shape, SQL, client parsing	context 0.03–0.14 s once warm (1.6 s for the very first, cold JVM — the Java Virtual Machine that runs the code, still loading classes)
Integration	the whole application context (mock servlet — a simulated Java web request/response layer — no port)	`@SpringBootTest` + `@AutoConfigureMockMvc`	the wiring: all real beans together	context ~0.7 s
Full stack / end-to-end	whole context + real server on a random port + real HTTP	`@SpringBootTest(webEnvironment = RANDOM_PORT)`	the app as a client sees it	context ~0.3–0.7 s, plus real network calls

The timings are the `Started <TestClass> in N seconds` lines Boot logs for each new context in this chapter's run:

```
... Started ProductRepositoryDataJpaTest in 1.574 seconds (process running for 2.205)   <- first context: cold JVM
... Started MockMvcFullContextTest in 0.726 seconds
... Started PriceViewJsonTest in 0.053 seconds
... Started ExchangeRateClientTest in 0.137 seconds
... Started ProductControllerWebMvcTest in 0.099 seconds
... Started PropertyPrecedenceTest in 0.034 seconds
... Started FullStackTest in 0.285 seconds
... Started RestTestClientMockMvcTest in 0.052 seconds
unit test: PriceView[sku=MUG-001, name=Mug, amount=10.85, currency=USD] in 0 ms
```

On a toy app the gap looks small. On a real service with 400 beans, Flyway migrations (versioned SQL scripts that update the database schema), Kafka (a message broker) and a security chain, a full context takes several seconds, and a suite of 300 test classes that each build one takes many minutes. The pyramid, together with context caching (§12.11), is what keeps a Spring build fast.

Why it matters

- **Fast feedback**: unit tests run in milliseconds, so you run them on every change.
- **Precise failures**: a failing unit or slice test points at one class. A failing end-to-end test only says "something is broken".
- **Confidence where it counts**: only an integration test catches a missing bean, a wrong property name, or two beans of the same type.

How to decide which layer

1. Is it pure logic (calculations, rules, mapping)? → **unit test**, no Spring.
2. Does it depend on something only Spring does (URL mapping, `@Valid`, JSON, a JPQL query, a `@Transactional` boundary)? → the **slice** for that layer.
3. Does it prove that beans work *together* (the real service with the real repository)? → `@SpringBootTest`, usually on the mock servlet environment.
4. Does it depend on a real server (filters in front of the servlet, HTTP compression, the port, the SSL setup — SSL/TLS is the encryption behind HTTPS)? → `RANDOM_PORT`.

Trade-offs

	Unit	Slice	`@SpringBootTest`	`RANDOM_PORT`
Speed	fastest	fast	slower	slowest
Realism	lowest (mocks everywhere)	real for one layer	real wiring	real HTTP
Failure points at	one class	one layer	"somewhere in the app"	"somewhere in the system"
Typical share of a suite	most	many	some	few

Common mistakes

- **Only** `@SpringBootTest`. Every test starts the whole app, and each small config difference forces a new context. This is the most common cause of 20-minute builds.
- **Mocking everything, even in integration tests**. If the repository, the client and the service are all mocks, the test proves nothing the unit test didn't.
- **Testing the framework**. You don't need a test showing that `JpaRepository.save` saves. Test *your* queries, mappings and rules.

Self-check

1. Which layer would you use to test that `POST /api/products` returns `400` for a bad SKU? → A `@WebMvcTest` slice: validation and error mapping are web-layer behaviour.
2. Which layer proves `cheaperThan` sorts by price? → `@DataJpaTest`: it runs the real JPQL on a real (embedded) database (one that runs inside the test's own JVM process, such as H2).
3. Why not test everything end-to-end? → Slow, and failures don't tell you where the bug is.

TL;DR

Most tests should be plain unit tests. Use a slice when the behaviour lives in Spring. Use a few `@SpringBootTest`s to prove the wiring. Use even fewer `RANDOM_PORT` tests.

12.3 Test dependencies in Boot 4

What changed

Boot 4 split the old monolithic (all-in-one) **auto-configuration** (Boot's beans that set things up automatically based on what is on the classpath) into **modules**, one per technology (Chapter 3). Test support was split the same way. Each technology now has a test starter named `spring-boot-starter-<technology>-test`. It pulls in the main starter, `spring-boot-starter-test`, and that technology's test module (slice annotation, test clients).

`testing-demo/pom.xml`:

```
<!-- Boot 4: one test starter per technology -->
<!-- @WebMvcTest, MockMvcTester, RestTestClient, JUnit 6, AssertJ, Mockito -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<!-- @DataJpaTest, TestEntityManager -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
<!-- @RestClientTest, MockRestServiceServer -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient-test</artifactId>
  <scope>test</scope>
</dependency>
```

Test starter	Adds (besides `spring-boot-starter-test`)
`spring-boot-starter-webmvc-test`	`@WebMvcTest`, `@AutoConfigureMockMvc`, `MockMvcTester`, `RestTestClient` / `TestRestTemplate` support
`spring-boot-starter-webflux-test`	`@WebFluxTest`, `WebTestClient` support, `reactor-test`
`spring-boot-starter-data-jpa-test`	`@DataJpaTest`, `TestEntityManager`, `@AutoConfigureTestDatabase`
`spring-boot-starter-jdbc-test`	`@JdbcTest`
`spring-boot-starter-data-r2dbc-test`	`@DataR2dbcTest`, `reactor-test`
`spring-boot-starter-restclient-test`	`@RestClientTest`, `MockRestServiceServer` wiring
`spring-boot-starter-jackson-test`	Jackson 3 + `spring-boot-starter-test` (`@JsonTest` itself lives in `spring-boot-test-autoconfigure`); pulled in by the webmvc/webflux/restclient test starters
`spring-boot-starter-security-test`	`spring-security-test` (`@WithMockUser`, `jwt()`, ...)
`spring-boot-starter-kafka-test`	`spring-kafka-test` (`@EmbeddedKafka`)

`spring-boot-starter-test` itself contains, in Boot 4.1.1: **JUnit Jupiter 6.0** (JUnit 6 needs Java 17+), **AssertJ**, **Mockito** (+ `mockito-junit-jupiter`), **Hamcrest** (an older matcher library), **JSONassert** (compares JSON documents), **JsonPath** (picks values out of JSON with paths like `$.amount`), **Awaitility** (wait for asynchronous results), XMLUnit, `spring-test` and `spring-boot-test`.

The slice annotations also **moved packages** with their modules. Your IDE (the code editor, such as IntelliJ IDEA) finds them, but old imports break when upgrading:

Annotation	Boot 3.x package	Boot 4 package
`@WebMvcTest`, `@AutoConfigureMockMvc`	`org.springframework.boot.test.autoconfigure.web.servlet`	`org.springframework.boot.webmvc.test.autoconfigure`
`@DataJpaTest`	`...boot.test.autoconfigure.orm.jpa`	`org.springframework.boot.data.jpa.test.autoconfigure`
`TestEntityManager`	`...boot.test.autoconfigure.orm.jpa`	`org.springframework.boot.jpa.test.autoconfigure`
`@RestClientTest`	`...boot.test.autoconfigure.web.client`	`org.springframework.boot.restclient.test.autoconfigure`
`@WebFluxTest`	`...boot.test.autoconfigure.web.reactive`	`org.springframework.boot.webflux.test.autoconfigure`
`@JsonTest`	`org.springframework.boot.test.autoconfigure.json`	unchanged
`@SpringBootTest`, `@TestConfiguration`	`org.springframework.boot.test.context`	unchanged

Common mistakes

- Adding only `spring-boot-starter-test` and wondering why `@WebMvcTest` doesn't resolve. In Boot 4 it lives in the **webmvc** test module.
- Keeping Boot 3 imports after an upgrade. Fix the imports as shown in the table above; the annotations work the same.

12.4 Unit tests: no Spring at all

What it is

A **unit test** creates the class with `new`, hands it **test doubles** (objects that stand in for real collaborators, see Java ch18), calls a method and checks the result. No application context, no annotations scanned, no database.

This works because the service uses **constructor injection** (Chapter 2: the class receives its dependencies as constructor parameters). The dependencies are ordinary constructor parameters, so the test passes them in itself.

```
/** Layer 1 of the pyramid: a plain unit test. No Spring, no context, milliseconds. */
@ExtendWith(MockitoExtension.class)
class PriceServiceTest {

    @Mock ProductRepository products;
    @Mock ExchangeRateClient rates;
    PriceService service;

    @BeforeEach
    void setUp() {                      // constructor injection = no framework needed
        service = new PriceService(products, rates, "EUR");
    }

    @Test
    void convertsWithTheRemoteRate() {
        long t0 = System.nanoTime();
        when(products.findBySku("MUG-001"))
                .thenReturn(Optional.of(new Product("MUG-001", "Mug", new BigDecimal("10.00"))));
        when(rates.rate("EUR", "USD")).thenReturn(new BigDecimal("1.0850"));

        PriceView v = service.priceOf("MUG-001", "USD");

        System.out.println("unit test: " + v + " in " + (System.nanoTime() - t0) / 1_000_000 + " ms");
        assertThat(v.amount()).isEqualByComparingTo("10.85");   // 10.00 * 1.085 = 10.85
    }

    @Test
    void homeCurrencySkipsTheRemoteCall() {
        when(products.findBySku("MUG-001"))
                .thenReturn(Optional.of(new Product("MUG-001", "Mug", new BigDecimal("10.00"))));
        assertThat(service.priceOf("MUG-001", "EUR").amount()).isEqualByComparingTo("10.00");
        verify(rates, never()).rate(anyString(), anyString());
    }

    @Test
    void unknownSkuThrows() { ... assertThatThrownBy(...).hasMessage("No product with SKU NOPE"); }
}
```

Output:

```
unit test: PriceView[sku=MUG-001, name=Mug, amount=10.85, currency=USD] in 0 ms
```

What each part does:

- `@ExtendWith(MockitoExtension.class)` — Mockito's own JUnit extension creates the `@Mock` fields. This is not Spring.
- `isEqualByComparingTo("10.85")` — compares `BigDecimal`s by value. `isEqualTo` would fail on `10.85` vs `10.850` because `BigDecimal.equals` also compares the scale (the number of digits after the decimal point).
- `verify(rates, never())` — proves the rule "no remote call for the home currency". Only a unit test can check this so cheaply.

When not to use it

When the behaviour you care about is **produced by Spring**: a `@Transactional` rollback, a `@PreAuthorize` check, the JSON field names, a derived query. A unit test would bypass the proxy or the framework and prove nothing. Use a slice instead.

Common mistakes

- **Field injection** (`@Autowired private Repo repo;`) makes unit tests awkward: you need reflection (Java code that inspects and changes classes and private fields at runtime) or a Spring context just to set the field. Prefer constructors.
- **Starting Spring "because it's easier"**. `@SpringBootTest` to test a calculation costs seconds and hides design problems.

TL;DR

Constructor injection + Mockito + AssertJ = Spring-free unit tests that run in milliseconds. Most of your tests should look like this.

12.5 How the Spring TestContext Framework works

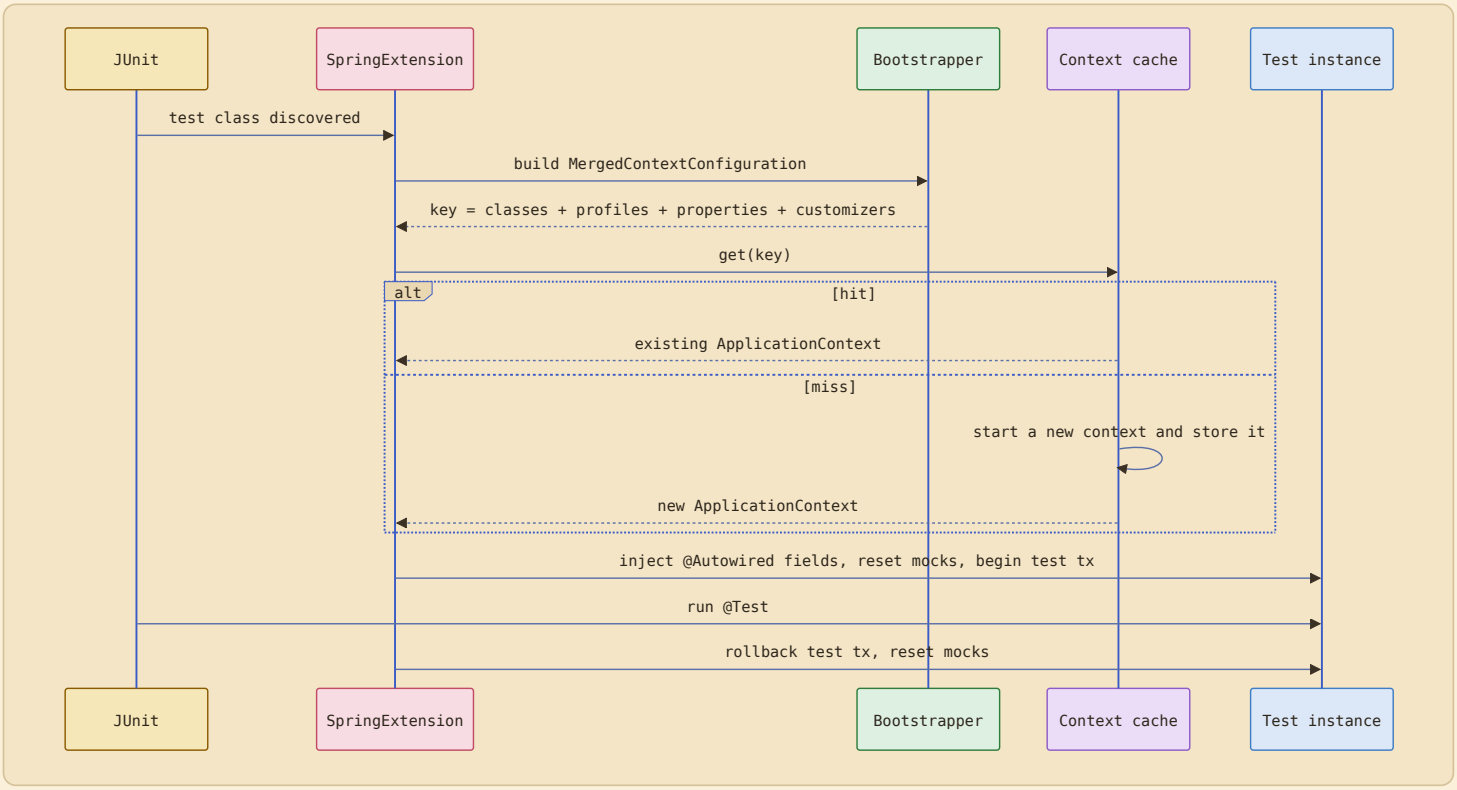
What it is

The **Spring TestContext Framework** (in `spring-test`) is the engine behind every Spring test annotation. Boot's `@SpringBootTest` and the slices are layers on top of it. Knowing its four moving parts explains most "why is my test slow / why is this bean missing" questions:

Part	What it is
`SpringExtension`	the JUnit Jupiter extension that connects JUnit's lifecycle to Spring. Every Boot test annotation is meta-annotated (itself annotated) with `@ExtendWith(SpringExtension.class)`, so you never add it yourself.
`TestContext`	per test class: holds the test class, current instance and method, and gives access to the `ApplicationContext`.
`TestExecutionListener`s	plug-ins called before/after each class and method: dependency injection into test fields, `@Transactional` test transactions, `@DirtiesContext`, `@MockitoBean` resets, `@Sql` scripts...
Context cache	a static (one copy per class, not per object), JVM-wide map from a *context configuration key* to a started `ApplicationContext`.

How it works

1. JUnit discovers the test class. `SpringExtension` asks a **bootstrapper** (Boot's `SpringBootTestContextBootstrapper`, or a slice's own) to build a `MergedContextConfiguration`: the configuration classes, active **profiles** (named sets of configuration, such as `test` or `prod`, switched on per run), test properties, context customizers (such as `@MockitoBean` definitions and `@DynamicPropertySource` methods), and so on.
2. For `@SpringBootTest`, when no classes are given, the bootstrapper searches **up the package tree** from the test's package for a class annotated with `@SpringBootConfiguration` (your `@SpringBootApplication`). This is why tests must live in the same package as the main class or below it.
3. The merged configuration is the **cache key**. If the cache has a context for it → reuse (**hit**). If not → create and start one (**miss**) and put it in the cache.
4. Listeners inject `@Autowired` fields into the test instance, start a transaction if the test is `@Transactional`, and so on.
5. After the test the listeners roll back the test transaction, reset `@MockitoBean` mocks, and close the context if it is marked dirty.
6. The context stays in the cache for later test classes with the same key, until the JVM exits (or it is evicted).



Self-check

1. Where does `@SpringBootTest` find the configuration if you give none? → It searches the test's package and its parents for `@SpringBootConfiguration`.
2. What decides whether two test classes share a context? → Whether their `MergedContextConfiguration`s are equal (the cache key).
3. Who injects `@Autowired` fields in a test? → A `TestExecutionListener` (`DependencyInjectionTestExecutionListener`), not the context itself — the test instance is not a bean.

12.6 `@SpringBootTest`: the whole application

What it is

`@SpringBootTest` starts your **entire application context** the way `SpringApplication.run` would: **component scanning** (finding classes annotated `@Component`, `@Service`, ... in your packages and turning them into beans), all auto-configuration, `application.properties`. It is the right tool for proving that beans work together.

The four web environments

webEnvironment	What you get	Use for
`MOCK` (default)	a web context with a **mock servlet environment** — no server, no port. Add `@AutoConfigureMockMvc` to get `MockMvcTester`.	most integration tests of a web app
`RANDOM_PORT`	a **real embedded server** (Tomcat/Netty, web servers started inside your app) on a free port; inject it with `@LocalServerPort`	end-to-end HTTP tests
`DEFINED_PORT`	a real server on `server.port` (default 8080)	rare: breaks parallel builds when the port is taken
`NONE`	no web environment at all	batch jobs (programs that process data in bulk and exit), messaging consumers, clients

> **Boot 4 change:** `@SpringBootTest` no longer auto-configures test clients by itself. Add `@AutoConfigureMockMvc` for MockMvc, `@AutoConfigureRestTestClient` for `RestTestClient`, `@AutoConfigureTestRestTemplate` for `TestRestTemplate`, `@AutoConfigureWebTestClient` for `WebTestClient` (per the Boot 4.0 migration guide).

Example: real beans, one fake

```
/** Whole context, mock servlet environment (no port): every real bean except the remote client. */
@SpringBootTest(properties = "catalog.home-currency=EUR")
@AutoConfigureMockMvc
class MockMvcFullContextTest {

    @TestConfiguration            // extra beans for this test only; not picked up by component scanning
    static class SeedData {
        @Bean ApplicationRunner seed(ProductRepository repo) {
            return args -> repo.save(new Product("LMP-001", "Lamp", new BigDecimal("40.00")));
        }
    }

    @Autowired MockMvcTester mvc;
    @Autowired ApplicationContext ctx;
    @MockitoBean ExchangeRateClient rates;    // the only fake: no network in this test

    @Test
    void realServiceAndDatabaseMockedRemote() throws Exception {
        given(rates.rate("EUR", "GBP")).willReturn(new BigDecimal("0.85"));

        MvcTestResult r = mvc.get().uri("/api/products/LMP-001/price?currency=GBP").exchange();

        System.out.println("@SpringBootTest beans: " + ctx.getBeanDefinitionCount());
        System.out.println("GET GBP -> " + r.getResponse().getContentAsString());
        assertThat(r).hasStatusOk().bodyJson().extractingPath("$.amount").isEqualTo(34.0);
    }
}
```

Output:

```
@SpringBootTest beans: 253
GET GBP -> {"sku":"LMP-001","name":"Lamp","amount":34.00,"currency":"GBP"}
```

What each part does:

- `properties = "..."` — adds test-only property values with high precedence (§12.15).
- `@TestConfiguration` — a `@Configuration` meant for tests. As a static nested class it is **added to** the application's configuration; it does not replace it. (A nested plain `@Configuration` would *replace* the auto-detected `@SpringBootApplication`, which is rarely what you want.) Top-level `@TestConfiguration` classes are skipped by component scanning and must be pulled in with `@Import`.
- The controller, service, repository, JPA and H2 are all real. Only the class that would call the internet is replaced.

When not to use it

- To test one controller's validation or one query: a slice is 5–10× faster and fails more precisely.
- With a different `properties=` / mock set on every class: each variation is a new context (§12.11).

Common mistakes

- **Expecting a `@Transactional` test to roll back what the server did in a `RANDOM_PORT` test.** The HTTP request runs on a server **thread** (an independent path of execution inside the JVM) in its own transaction, so the test's transaction can't roll it back. Clean up explicitly, as `FullStackTest` does with `repo.deleteAll()` in `@BeforeEach`.
- **Putting `@SpringBootTest` on a test in a package outside the main class's package** → "Unable to find a @SpringBootConfiguration".

Self-check

1. What's the default `webEnvironment`? → `MOCK`: no server, no port.
2. How do you add one extra bean for a single test class? → A static nested `@TestConfiguration` (or a top-level one + `@Import`).
3. How do you get `MockMvcTester` in a `@SpringBootTest`? → Add `@AutoConfigureMockMvc`.

12.7 Test slices

What it is

A **test slice** starts only the part of the application context needed for one layer. The slice annotation (for example `@WebMvcTest`) does three things:

1. It **disables full auto-configuration** and imports only the auto-configurations listed for that slice (for `@WebMvcTest`: MVC, Jackson — the JSON library that converts Java objects to JSON and back — validation, security filters, ...).
2. It adds a **type-exclude filter** to component scanning, so only matching beans of *your* code are registered (for `@WebMvcTest`: `@Controller`, `@ControllerAdvice`, `@JacksonComponent`, `Converter`, `Filter`, `WebMvcConfigurer`, `HandlerInterceptor`, ...). `@Service`, `@Repository` and `@Component` classes are **not** registered.
3. It adds test helpers (`MockMvcTester`, `TestEntityManager`, `MockRestServiceServer`, ...).

> **Analogy:** a slice is a stage rehearsal for one scene. Only the actors in that scene are on stage. Everyone else is replaced by a stand-in (`@MockitoBean`).

The slices you will meet

Annotation	Starts	Your beans registered	Extras	Transactional?
`@WebMvcTest`	Spring MVC, Jackson, validation, Security	controllers, advice, filters, converters, `WebMvcConfigurer`	`MockMvcTester`, `MockMvc`	no
`@WebFluxTest`	WebFlux, codecs, validation	controllers, `WebFluxConfigurer`, ...	`WebTestClient` (ch13)	no
`@DataJpaTest`	JPA, Hibernate (the JPA implementation), Spring Data JPA, embedded DB, Flyway/Liquibase	entities (classes mapped to tables), repositories	`TestEntityManager`, SQL logging on	**yes, rolled back**
`@JdbcTest`	`DataSource` (the factory for database connections), `JdbcTemplate`, `JdbcClient`, embedded DB	none	—	yes, rolled back
`@DataR2dbcTest`	R2DBC (non-blocking database access), Spring Data R2DBC	repositories	—	**no** (ch13)
`@JsonTest`	Jackson (or Gson / JSON-B)	`@JacksonComponent`, Jackson modules	`JacksonTester`, `BasicJsonTester`	no
`@RestClientTest`	`RestClient.Builder`, `RestTemplateBuilder`, Jackson	the classes you name	`MockRestServiceServer`	no
`@DataMongoTest`, `@DataRedisTest`, `@DataJdbcTest`, ...	that store	repositories	—	depends

`@WebMvcTest`

```
/** Web slice: only the MVC layer is real; the service is a Mockito mock. */
@WebMvcTest(ProductController.class)
class ProductControllerWebMvcTest {

    @Autowired MockMvcTester mvc;
    @Autowired ApplicationContext ctx;
    @MockitoBean PriceService service;       // replaces (or adds) the bean in the test context

    @Test
    void sliceContainsOnlyWebBeans() {
        System.out.println("@WebMvcTest beans: " + ctx.getBeanDefinitionCount()
                + ", ProductController present: " + (ctx.getBeanNamesForType(ProductController.class).length == 1)
                + ", ProductRepository present: " + (ctx.getBeanNamesForType(ProductRepository.class).length == 1));
        assertThat(ctx.getBeanNamesForType(ProductRepository.class)).isEmpty();
    }
    ...
}
```

Output:

```
@WebMvcTest beans: 127, ProductController present: true, ProductRepository present: false
```

127 beans instead of 253, and no JPA, no database, no `RestClient`. `@WebMvcTest(ProductController.class)` narrows it further to **one** controller. Without the argument, every controller in the app is registered, and each one's dependencies must then be mocked.

The web tests themselves are in §12.8.

`@DataJpaTest`

```
/** JPA slice: entities, repositories, an embedded DB; every test runs in a rolled-back transaction. */
@DataJpaTest
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class ProductRepositoryDataJpaTest {

    @Autowired ProductRepository repo;
    @Autowired TestEntityManager em;
    @Autowired DataSource dataSource;

    @Test @Order(1)
    void derivedAndJpqlQueries() throws Exception {
        System.out.println("DB = " + dataSource.getConnection().getMetaData().getURL()
                + ", test transaction active = " + TestTransaction.isActive());
        em.persist(new Product("MUG-001", "Mug", new BigDecimal("10.00")));
        em.persist(new Product("PEN-001", "Pen", new BigDecimal("2.50")));
        em.persist(new Product("LMP-001", "Lamp", new BigDecimal("40.00")));
        em.flush();   // send the INSERTs now
        em.clear();   // forget cached entities so the queries really hit the database

        assertThat(repo.findBySku("PEN-001")).map(Product::getName).contains("Pen");
        assertThat(repo.cheaperThan(new BigDecimal("15"))).extracting(Product::getSku)
                .containsExactly("PEN-001", "MUG-001");
        System.out.println("test 1 saw " + repo.count() + " rows");
    }

    @Test @Order(2)
    void previousTestWasRolledBack() {
        System.out.println("test 2 starts with " + repo.count() + " rows");
        assertThat(repo.count()).isZero();
    }
}
```

Output (`@DataJpaTest` turns on `spring.jpa.show-sql`, so Hibernate prints its SQL):

```
DB = jdbc:h2:mem:fa1ebe8d-78e8-4d82-a212-1a1b655ce2ab, test transaction active = true
Hibernate: select next value for product_seq
Hibernate: select next value for product_seq
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: select p1_0.id,p1_0.name,p1_0.price,p1_0.sku from product p1_0 where p1_0.sku=?
Hibernate: select p1_0.id,p1_0.name,p1_0.price,p1_0.sku from product p1_0 where p1_0.price<=? order by p1_0.price
Hibernate: select count(*) from product p1_0
test 1 saw 3 rows
Hibernate: select count(*) from product p1_0
test 2 starts with 0 rows
```

What it shows:

- **Embedded database**: `@DataJpaTest` includes `@AutoConfigureTestDatabase`. Its default mode `Replace.NON_TEST` (since Boot 3.4) swaps your `DataSource` for a fresh embedded one (a random `jdbc:h2:mem:` name). It leaves the `DataSource` alone when it is already a

test database, such as a Testcontainers `@ServiceConnection` (§12.14). Use `replace = NONE` to always keep the configured database.

- **Rolled-back transaction**: every test method runs in a transaction that is rolled back at the end, so test 2 sees 0 rows. That keeps tests independent without cleanup code.
- `TestEntityManager`: a test helper around the JPA `EntityManager` (`persist`, `persistAndFlush`, `find`, `clear`) for setting up data without going through the repository you are testing.
- `flush()` + `clear()`: without them the INSERTs would still sit in the **persistence context** (JPA's in-memory set of loaded and changed entities, also called the **first-level cache**) and `findBySku` might be answered from the first-level cache (ch08). Clearing forces real SQL.

> The rolled-back test transaction is convenient but can **hide bugs**. **Lazy loading** (fetching related rows only when first accessed) always works inside it, and nothing is ever **committed** (made permanent in the database). So constraint checks deferred to commit, `@TransactionalEventListener(AFTER_COMMIT)` and code that relies on its own transaction never run. Chapter 7 and Chapter 8 use non-transactional `@SpringBootTest`s for exactly this reason. Opt out per class with `@Transactional(propagation = Propagation.NOT_SUPPORTED)`.

`@JsonTest`

Checks the **wire format** (exactly how an object looks once **serialized**, i.e. turned into JSON text for sending): field names, number and date formats, and that nothing internal leaks.

```
@JsonTest
class PriceViewJsonTest {

    @Autowired JacksonTester<PriceView> json;

    @Test
    void serialize() throws Exception {
        JsonContent<PriceView> out = json.write(new PriceView("MUG-001", "Mug", new BigDecimal("10.85"), "USD"));
        System.out.println("serialized: " + out.getJson());
        assertThat(out).extractingJsonPathStringValue("$.currency").isEqualTo("USD");
        assertThat(out).extractingJsonPathNumberValue("$.amount").isEqualTo(10.85);
        assertThat(out).doesNotHaveJsonPath("$.id");
    }

    @Test
    void deserialize() throws Exception {
        PriceView v = json.parseObject("""
                {"sku":"PEN-001","name":"Pen","amount":2.5,"currency":"EUR"}""");
        System.out.println("parsed: " + v);
        assertThat(v.amount()).isEqualByComparingTo("2.50");
    }
}
```

Output:

```
serialized: {"sku":"MUG-001","name":"Mug","amount":10.85,"currency":"USD"}
parsed: PriceView[sku=PEN-001, name=Pen, amount=2.5, currency=EUR]
```

The slice builds the same Jackson 3 `JsonMapper` your app uses, including `spring.jackson.*` properties and any `@JacksonComponent` classes. A plain `new JsonMapper()` in a unit test would miss those customisations.

`@RestClientTest`

Tests an HTTP **client** class without a network. The auto-configured `RestClient.Builder` is connected to a `MockRestServiceServer`, which checks what the client sent and plays back a canned (pre-recorded) answer.

```java
@RestClientTest(ExchangeRateClient.class)
@TestPropertySource(properties = "rates.base-url=https://rates.test")
class ExchangeRateClientTest {

    @Autowired ExchangeRateClient client;
    @Autowired MockRestServiceServer server;

    @Test
    void parsesTheRate() {
        server.expect(requestTo("https://rates.test/rates?from=EUR&to=USD"))
              .andExpect(method(HttpMethod.GET))
              .andRespond(withSuccess("""
                      {"from":"EUR","to":"USD","rate":1.0850}""", MediaType.APPLICATION_JSON));

        var rate = client.rate("EUR", "USD");
        System.out.println("rate EUR->USD = " + rate);
        assertThat(rate).isEqualByComparingTo("1.085");
        server.verify();                  // every expected request really happened
    }

    @Test
    void serverErrorBecomesAnException() {
        server.expect(requestTo("https://rates.test/rates?from=EUR&to=JPY"))
              .andRespond(withServerError());

        assertThatThrownBy(() -> client.rate("EUR", "JPY"))
                .isInstanceOf(HttpServerErrorException.InternalServerError.class);
        System.out.println("500 from rates API -> HttpServerErrorException.InternalServerError");
    }
}
```

Output:

```
rate EUR->USD = 1.0850
500 from rates API -> HttpServerErrorException.InternalServerError
```

`MockRestServiceServer` works only for clients built from the auto-configured builder. More client-testing options (a real local server, WireMock — a configurable fake HTTP server) are in Chapter 6.

Common mistakes

- **"No qualifying bean of type PriceService" in a `@WebMvcTest`.** Services are not part of the slice. Add `@MockitoBean PriceService`, or `@Import` the real class if it is cheap.
- **Extra `@ComponentScan` or `@EnableJpaRepositories` on the main class.** The slices rely on Boot's type-exclude filters. A hand-written `@ComponentScan` on `@SpringBootApplication` bypasses them, and suddenly your `@WebMvcTest` loads repositories. Put such configuration in a separate `@Configuration` class.
- **Configuration beans the slice doesn't know about.** A `@Configuration` class in your main package *is* scanned by every slice. If it creates a heavy bean (a Kafka client), every slice pays for it. Guard it with a profile or a condition, or move it out of the scanned package.
- **Assuming `@DataJpaTest` = production database behaviour.** H2 accepts SQL that PostgreSQL rejects and vice versa. Test database-specific SQL against the real engine (§12.14).

When not to use it

Don't write a slice for every class. If a controller only forwards to a service, one web-slice test per endpoint (happy path — the normal, everything-valid case — + error mapping) is enough.

In the interview

Q: What is a test slice and why use one? 30-second answer: "A slice like `@WebMvcTest` or `@DataJpaTest` starts only one layer of the application context: its auto-configurations plus the matching beans from my code. It's much faster than `@SpringBootTest` and tests exactly what Spring adds to that layer: mappings, validation and JSON for the web, real SQL for JPA. Everything outside the slice I mock with `@MockitoBean`."

- *Does `@WebMvcTest` load `@Service` beans?* → No; only web components. Mock or `@Import` services.
- *Are `@DataJpaTest` tests transactional?* → Yes, each test rolls back; and the `DataSource` is replaced by an embedded one unless it's already a test database or you set `replace = NONE`.
- *How would you test a `RestClient`-based client?* → `@RestClientTest` + `MockRestServiceServer`, or a real local stub server for timeouts and connection-level behaviour.

Self-check

1. Why is `ProductRepository` absent in `@WebMvcTest`? → The slice's type-exclude filter only registers web components; JPA auto-configuration isn't imported.
2. Why call `em.flush(); em.clear();` before querying? → To send the INSERTs and empty the first-level cache so the query really hits the database.
3. What does `server.verify()` check? → That every expected request was actually made.

TL;DR

Slices = partial contexts: `@WebMvcTest` (web), `@DataJpaTest` (JPA, rolled back, embedded DB), `@JsonTest` (Jackson), `@RestClientTest` (clients + `MockRestServiceServer`). Mock what's outside the slice.

12.8 `MockMvcTester`: AssertJ-style MockMvc

What it is

MockMvc sends requests straight into the `DispatcherServlet` (Spring MVC's single front controller that receives every request and routes it to your controller) without a real server or network. The request goes through filters, mapping, argument resolution (turning request parts into method parameters), validation, the controller, exception handlers and message converters (which turn objects into JSON and back), exactly as in production.

`MockMvcTester` (Framework 6.2, auto-configured since Boot 3.4) wraps MockMvc in **AssertJ** assertions. You build a request, call `exchange()`, and assert on the result with `assertThat(...)`. It replaces the older `perform(...).andExpect(...)` style.

	Classic `MockMvc`	`MockMvcTester`
Request	`mvc.perform(get("/x"))`	`mvc.get().uri("/x").exchange()`
Assertions	`andExpect(status().isOk())`, `jsonPath(...).value(...)` (Hamcrest)	`assertThat(result).hasStatusOk()`, `.bodyJson().extractingPath("$.x")` (AssertJ)
Checked exceptions (ones Java forces you to catch or declare)	`perform` throws `Exception`	none
Unhandled exceptions	thrown out of `perform`	captured in the result: `assertThat(result).hasFailed()`
Both available?	yes — Boot auto-configures both with `@WebMvcTest` / `@AutoConfigureMockMvc`	

Example

```
@Test
void returnsPriceAsJson() throws Exception {
    given(service.priceOf("MUG-001", "USD"))
            .willReturn(new PriceView("MUG-001", "Mug", new BigDecimal("10.85"), "USD"));

    MvcTestResult r = mvc.get().uri("/api/products/{sku}/price?currency=USD", "MUG-001").exchange();

    System.out.println("GET price -> " + r.getResponse().getStatus() + " " + r.getResponse().getContentAsString());
    assertThat(r).hasStatusOk()
            .hasContentType(MediaType.APPLICATION_JSON)
            .bodyJson().extractingPath("$.amount").isEqualTo(10.85);
}

@Test
void unknownSkuIsProblemDetail404() throws Exception {
    given(service.priceOf("NOPE-000", "EUR")).willThrow(new ProductNotFoundException("NOPE-000"));

    MvcTestResult r = mvc.get().uri("/api/products/NOPE-000/price").exchange();

    assertThat(r).hasStatus(404)
            .hasContentType(MediaType.APPLICATION_PROBLEM_JSON)
            .bodyJson().extractingPath("$.detail").isEqualTo("No product with SKU NOPE-000");
}

@Test
void invalidBodyIs400AndServiceIsNeverCalled() {
    MvcTestResult r = mvc.post().uri("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                    {"sku":"mug1","name":"","price":0}""")
            .exchange();

    assertThat(r).hasStatus(400);
    verifyNoInteractions(service);
}

@Test
void validBodyIs201WithLocation() {
    given(service.create(any())).willReturn(new Product("MUG-002", "Big mug", new BigDecimal("12.50")));
    MvcTestResult r = mvc.post().uri("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                    {"sku":"MUG-002","name":"Big mug","price":12.50}""")
            .exchange();
    assertThat(r).hasStatus(201).hasHeader("Location", "/api/products/MUG-002/price");
}
```

Output:

```
GET unknown -> 404 {"detail":"No product with SKU NOPE-000","instance":"/api/products/NOPE-000/price","status":404,"title":"Not Found"}
GET price -> 200 {"sku":"MUG-001","name":"Mug","amount":10.85,"currency":"USD"}
POST valid -> 201 Location=/api/products/MUG-002/price
... WARN ... DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for
argument [0] ... with 3 errors: ...]
POST invalid -> 400
```

What the tests prove:

- The URL template, the `currency` default and the JSON field names (`amount`, not `price`).
- `ProductNotFoundException` → **`application/problem+json`** with status, title, detail and instance (RFC 9457, ch05).
- Invalid input is rejected **before** the controller runs: `verifyNoInteractions(service)`. The WARN line is Spring logging the three constraint violations (bad pattern, blank name, price below 0.01).
- A `201 Created` with a `Location` header for the new resource.

Common mistakes

- **Asserting on `getContentAsString()` with `equals`.** Key order and spacing make string comparisons brittle. Use `bodyJson()` paths, or `bodyJson().isLenientlyEqualTo("{...}")`.
- **Expecting servlet-container behaviour.** MockMvc has no real Tomcat. Multipart size limits, HTTP/2, compression and error pages rendered by the container are not exercised. Use `RANDOM_PORT` for those.
- **Forgetting security.** With Spring Security on the **classpath** (the set of JARs and folders the JVM loads classes from), `@WebMvcTest` applies Boot's default security, not your `SecurityFilterChain`, unless you `@Import` your config (ch10).

Self-check

1. What runs in a MockMvc request? → The real `DispatcherServlet` pipeline (filters, mapping, conversion, validation, advice) without a network or server.
2. What does `bodyJson().extractingPath("$.amount")` use under the hood? → JsonPath on the response body, with AssertJ assertions on the extracted value.

3. How do you assert an unhandled exception with `MockMvcTester`? → `assertThat(result).hasFailed().failure().isInstanceOf(...)`.

12.9 `RestTestClient` (Framework 7.0)

What it is

`RestTestClient` is a new test client in **Spring Framework 7.0**. It has the **fluent** (method calls chained one after another) `WebTestClient`-style API (`get().uri(...).exchange().expectStatus()...`), but for **servlet** (Spring MVC) applications and without needing WebFlux. One API can target different back ends:

Binding	Request goes to	Typical use
`RestTestClient.bindTo(mockMvc)`	an existing `MockMvc` (no server)	slice / mock-environment tests
`bindToController(controller)`	a standalone MockMvc for one controller	pure controller tests without Spring Boot
`bindToRouterFunction(...)`	functional MVC routes	`RouterFunction` tests
`bindToApplicationContext(ctx)`	MockMvc built from a `WebApplicationContext`	custom setups
`bindToServer()`	a **real HTTP server**	`RANDOM_PORT` end-to-end tests

So the same test code can run against MockMvc or against a real server.

Boot 4 status (verified): Boot 4.0 added auto-configuration for it. Annotate the test class with **`@AutoConfigureRestTestClient`** (module `spring-boot-resttestclient`, included in `spring-boot-starter-webmvc-test`). In a mock environment the client is bound to the `MockMvc`. In a `RANDOM_PORT`/`DEFINED_PORT` test it points at the running server, with the base URL already set. Without the annotation there is no bean. Removing it from the test below fails with:

```
No qualifying bean of type 'org.springframework.test.web.servlet.client.RestTestClient' available
```

Example against MockMvc

```
/** RestTestClient (Framework 7.0) bound to MockMvc: a fluent HTTP-client API, no real server. */
@WebMvcTest(ProductController.class)
@AutoConfigureRestTestClient
class RestTestClientMockMvcTest {

    @Autowired RestTestClient client;
    @MockitoBean PriceService service;

    @Test
    void fluentExpectations() {
        given(service.priceOf("MUG-001", "EUR"))
                .willReturn(new PriceView("MUG-001", "Mug", new BigDecimal("10.00"), "EUR"));

        PriceView body = client.get().uri("/api/products/MUG-001/price")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType("application/json")
                .expectBody(PriceView.class)              // JSON -> record, with the app's JsonMapper
                .returnResult().getResponseBody();

        System.out.println("RestTestClient (MockMvc) body = " + body);
        assertThat(body.amount()).isEqualByComparingTo("10.00");
    }

    @Test
    void assertjStyle() {
        given(service.priceOf("NOPE-000", "EUR")).willThrow(new ProductNotFoundException("NOPE-000"));
        RestTestClient.ResponseSpec spec = client.get().uri("/api/products/NOPE-000/price").exchange();

        assertThat(RestTestClientResponse.from(spec))       // AssertJ adapter (Framework 7.0)
                .hasStatus(404)
                .bodyJson().extractingPath("$.title").isEqualTo("Not Found");
        System.out.println("RestTestClient (AssertJ) 404 title = Not Found");
    }
}
```

Output:

```
RestTestClient (AssertJ) 404 title = Not Found
RestTestClient (MockMvc) body = PriceView[sku=MUG-001, name=Mug, amount=10.00, currency=EUR]
```

The real-server version is in §12.13.

Which test client?

Client	Stack	Server needed	Status in Boot 4
`MockMvcTester`	MVC	no	recommended for MVC slice/mock tests
`RestTestClient`	MVC (any HTTP server with `bindToServer`)	optional	**new**, recommended for fluent HTTP-style tests
`WebTestClient`	WebFlux (can also bind to MockMvc)	optional	recommended for WebFlux (ch13)
`TestRestTemplate`	any	yes	still available (`@AutoConfigureTestRestTemplate`, moved to `org.springframework.boot.resttestclient`); `RestTemplate` itself is heading for deprecation (ch06)

Self-check

1. What does `RestTestClient` add over `MockMvcTester`? → A client-style API that can target MockMvc *or* a real server with the same test code, and body decoding to types (`expectBody(PriceView.class)`).
2. What annotation do you need in Boot 4 to inject it? → `@AutoConfigureRestTestClient`.

12.10 `@MockitoBean` and `@MockitoSpyBean`

What it is

- `@MockitoBean` (Framework 6.2, in `org.springframework.test.context.bean.override.mockito`) puts a **Mockito mock into the application context**. It replaces the bean of that type (or name), or adds one if none exists. Every bean that depends on it, such as the controller, receives the mock.
- `@MockitoSpyBean` wraps the **real existing bean** in a Mockito **spy**. Real methods run, but you can `verify` calls or **stub** single methods (tell them what to return).

Compare with Mockito's own `@Mock`: `@Mock` creates a mock that only your test holds. `@MockitoBean` creates a mock that **the container injects** into other beans.

> **Boot 4:** Boot's `@MockBean` and `@SpyBean` (deprecated in 3.4) are **removed**. Replace them with `@MockitoBean` / `@MockitoSpyBean`. One difference: the new annotations work on test-class fields (including superclasses and `@Nested` classes), **not inside `@Configuration` classes**. To share mocks, declare them on a common base class, or at class level with `@MockitoBean(types = {A.class, B.class})`.

How it works

1. While building the `MergedContextConfiguration`, Spring finds bean-override fields and adds them to the **cache key** (so each distinct set of mocks is a different context, see §12.11).
2. When the context starts, the bean definition for that type is replaced by one that returns the Mockito mock (or, for a spy, the real bean is wrapped after creation).
3. The mock is injected into the test field.
4. **After each test method the mock is reset** (default `MockReset.AFTER`), so stubbing from one test doesn't leak into the next.

Example: a spy on the real client

In `FullStackTest` the `ExchangeRateClient` is real and makes real HTTP calls. The spy lets the test count them:

```
@MockitoSpyBean ExchangeRateClient rates; // the REAL bean, wrapped so we can verify calls
...
verify(rates, times(1)).rate("EUR", "USD");   // EUR request did not call the API
```

The mocks in the `@WebMvcTest`s above (`@MockitoBean PriceService service`) are the more common case.

Related: `@TestBean`

`@TestBean` (Framework 6.2) overrides a bean with an instance returned by a **static factory method** (a `static` method whose job is to create and return an object) in the test class. Use it when a hand-written fake is clearer than a Mockito mock:

```
@TestBean ExchangeRateClient rates;                    // looks for static method rates()
static ExchangeRateClient rates() { return new FixedRateClient(new BigDecimal("1.10")); }
```

Common mistakes

- **Mocking in a way that changes the context for every class.** Class A mocks `PriceService`, class B mocks `ExchangeRateClient`, class C mocks both: that's three contexts. Standardise the mock set in a base class.

- **Mocking what you're trying to test.** A `@SpringBootTest` with the repository, the client *and* the service mocked is a slow unit test.
- `@MockitoSpyBean` **on a bean that doesn't exist** → the context fails to start. A spy needs a real bean to wrap.
- **Final classes/methods.** Mockito 5 uses the inline mock maker by default, so finals can be mocked. On current JDKs it attaches itself as an **agent** (a helper that can rewrite classes while the JVM runs) at runtime and prints *"Mockito is currently self-attaching..."*. The Mockito docs recommend adding it as a `-javaagent` in the Surefire (Maven's unit-test runner plugin) configuration before the JDK disallows dynamic agent loading by default.

In the interview

Q: `@Mock` **vs** `@MockitoBean`**?** → `@Mock` is plain Mockito: a mock only the test holds, no Spring. `@MockitoBean` registers the mock *in the application context*, so Spring injects it into other beans. It also becomes part of the context cache key.

- *What replaced* `@MockBean` *in Boot 4?* → `@MockitoBean` from Spring Framework (6.2+); `@MockBean` was removed.
- *When would you use a spy bean?* → To keep real behaviour but verify interactions or stub one method, e.g. count calls to a real client.

TL;DR

`@MockitoBean` = mock inside the context. `@MockitoSpyBean` = spy around the real bean. Both are reset after each test and both change the cache key.

12.11 Test context caching

What it is

Starting an application context is the most expensive part of a Spring test. The TestContext Framework therefore keeps started contexts in a **static cache** that lives for the whole test JVM. The second test class with an *identical* configuration gets the already-running context in about 0 ms.

> **Analogy:** a restaurant kitchen that keeps the oven hot between orders. Any order that needs the same oven setting is cooked at once. A different setting means heating up another oven.

What is in the cache key

A new context is created when **any** of these differ between two test classes:

- configuration classes / locations (`@SpringBootTest(classes=...)`, `@ContextConfiguration`, `@Import` on the test, nested `@TestConfiguration`)
- active profiles (`@ActiveProfiles`)
- test properties (`@TestPropertySource`, `@SpringBootTest(properties=...)`) and property source files
- **context customizers**: `@MockitoBean` / `@MockitoSpyBean` / `@TestBean` definitions, `@DynamicPropertySource` methods, `@AutoConfigureMockMvc` and other `@AutoConfigure...` settings, `webEnvironment`
- the slice type (`@WebMvcTest(A.class)` and `@WebMvcTest(B.class)` are different)
- context initializers, the context loader, the parent context (`@ContextHierarchy`)

`@DirtiesContext` doesn't change the key. It **closes and evicts** the context after the class or method, so the next user must build it again.

The measured demo

`context-caching` has one base class and seven test classes, run in name order (`junit.jupiter.testclass.order.default=...ClassOrderer$ClassName` in `junit-platform.properties`). To make startup cost visible, the app has a bean that sleeps **400 ms** in its constructor, standing in for real startup work such as warming a cache or loading a model:

```
@Component
public static class SlowStartup {
    public static final AtomicInteger CREATED = new AtomicInteger();
    public SlowStartup() throws InterruptedException {
        Thread.sleep(400);
        CREATED.incrementAndGet();          // = number of contexts built so far
    }
}
```

```
/** One shared configuration: every subclass that adds nothing reuses the same cached context. */
@SpringBootTest
@AutoConfigureMockMvc
abstract class BaseIntegrationTest { }

class C1_GreetingApiTest   extends BaseIntegrationTest { ... }  // first user -> MISS
class C2_SameConfigTest    extends BaseIntegrationTest { ... }  // identical config -> HIT
class C3_WithMockTest      extends BaseIntegrationTest {        // a @MockitoBean changes the key -> MISS
    @MockitoBean CachingApplication.GreetingService service; ... }
@TestPropertySource(properties = "feature.discounts=true")    // different properties -> MISS
class C4_OtherPropertyTest extends BaseIntegrationTest { ... }
@DirtiesContext                                               // HIT, then closed and evicted
class C5_DirtiesContextTest extends BaseIntegrationTest { ... }
class C6_AfterDirtiesTest  extends BaseIntegrationTest { ... }  // base config again, evicted -> MISS
class C7_BackToBaseTest    extends BaseIntegrationTest { ... }  // HIT on the rebuilt context
```

A small `TestExecutionListener` measures, for each class, the time from `beforeTestClass` until the context is available, and reads the cache statistics. It is registered for all tests in `src/test/resources/META-INF/spring.factories`:

```
public class CacheReportListener implements TestExecutionListener, Ordered {

    /** The JVM-wide context cache is reachable through this protected accessor. */
    static final class Stats extends DefaultCacheAwareContextLoaderDelegate {
        static ContextCache cache() { return new Stats().getContextCache(); }
    }

    @Override public int getOrder() { return Ordered.LOWEST_PRECEDENCE; }
    @Override public void beforeTestClass(TestContext tc) { start = System.nanoTime(); }

    @Override
    public void prepareTestInstance(TestContext tc) {
        ...
        tc.getApplicationContext();                         // loads it, or takes it from the cache
        long ms = (System.nanoTime() - start) / 1_000_000;
        ContextCache c = Stats.cache();
        System.out.printf("%-26s context in %4d ms | cache size=%d hits=%d misses=%d | contexts built=%d%n", ...);
    }
}
```

Output:

```
C1_GreetingApiTest         context in  892 ms | cache size=1 hits=3 misses=1 | contexts built=1
C2_SameConfigTest          context in    0 ms | cache size=1 hits=18 misses=1 | contexts built=1
C3_WithMockTest            context in  575 ms | cache size=2 hits=32 misses=2 | contexts built=2
C4_OtherPropertyTest       context in  503 ms | cache size=3 hits=45 misses=3 | contexts built=3
C5_DirtiesContextTest      context in    0 ms | cache size=3 hits=60 misses=3 | contexts built=3
C6_AfterDirtiesTest        context in  467 ms | cache size=3 hits=73 misses=4 | contexts built=4
C7_BackToBaseTest          context in    0 ms | cache size=3 hits=88 misses=4 | contexts built=4
```

How to read it:

- **misses** = contexts actually built (4); **contexts built** (the counter in `SlowStartup`) agrees.
- **hits** count every *lookup* of the cache, not test classes. Spring's own listeners call `getApplicationContext()` many times per test, so the number climbs quickly. Look at how it changes, not at the absolute value.
- Every miss costs ~0.4 s of simulated startup plus ~0.1 s of real Boot work; the first one also pays for JVM/class loading (892 ms). Every hit costs **0 ms**.
- A mock (C3) and one property (C4) each forced a whole new context. `@DirtiesContext` (C5) threw away a good context, so C6 had to rebuild it.
- Seven classes, four builds. With one shared configuration and no `@DirtiesContext`, it would be **one** build.

To see Spring's own statistics, set the logger `org.springframework.test.context.cache` to `DEBUG`. It logs lines like `Spring test ApplicationContext cache statistics: [DefaultContextCache@... size = 1, maxSize = 32, ... hitCount = ..., missCount = ...]`.

Limits and Framework 7 behaviour

Setting	Default	How to change
Maximum cached contexts	**32**, least-recently-used evicted (and closed)	JVM system property or `spring.properties`: `spring.test.context.cache.maxSize=16`
Context pausing (new in Framework 7.0)	`ON_CONTEXT_SWITCH`: when the next test needs a *different* context, the inactive one is **paused**, meaning its `Lifecycle` / `SmartLifecycle` beans (beans with start/stop methods: schedulers, message listener containers) are stopped, and restarted when it is used again	`spring.test.context.cache.pause=always` or `never`

Pausing fixes an old problem: a cached context with a `@Scheduled` job or a Kafka/JMS (Java Message Service, the standard Java messaging API) listener kept consuming messages while *another* test class was running, stealing its messages or skewing its results.

Tips for a fast suite

1. Put the common annotations (`@SpringBootTest`, `@AutoConfigureMockMvc`, `@ActiveProfiles("test")`, the shared `@MockitoBean`s, container config) on **one abstract base class** and extend it.
2. Prefer **slices** for layer tests; they're small, so their extra contexts are cheap.
3. Avoid `@DirtiesContext`. Reset state instead (clean tables, `reset()` fakes). If you must, use it on the few tests that truly break the context.
4. Watch for **one-off** `properties=` values and one-off mocks. Each one is a new context that stays in memory until eviction.
5. Surefire runs all classes in one JVM by default (`forkCount=1`, `reuseForks=true`). That's what makes the cache useful. Settings that fork (start a separate) JVM per class throw the cache away.

Common mistakes

- **Too many contexts in memory.** Up to 32 full contexts, each with its thread pools and connection pools (sets of reusable threads and open database connections) and embedded servers, can exhaust memory, database connections or ports. Symptoms are `Too many connections` errors or out-of-memory errors late in the build.
- **Static state across contexts.** Two contexts in the same JVM share static fields, and the same H2 database name if the URL is fixed.

In the interview

Q: Why is our Spring test suite slow and how do you fix it? "Almost always too many application contexts. The TestContext Framework caches contexts by their configuration. Every different set of `@MockitoBean`s, properties, profiles or `@Import`s, and every `@DirtiesContext`, builds a new one. I'd enable DEBUG logging for `org.springframework.test.context.cache` to count misses, move shared configuration to a base class, replace one-off `@SpringBootTest`s with slices or unit tests, and remove `@DirtiesContext` where cleanup is enough."

- *What's the default cache size?* → 32, LRU eviction; `spring.test.context.cache.maxSize`.
- *What is context pausing?* → Framework 7.0: inactive cached contexts get their lifecycle beans stopped when the tests switch to another context, and restarted on reuse.

Self-check

1. Two classes differ only in `@MockitoBean`. Same context? → No, bean overrides are part of the key.
2. What does `@DirtiesContext` do to the cache? → Closes the context and removes it; the next class with that key rebuilds it.
3. Why does `hits` grow by ~15 per class in the demo? → It counts every cache lookup, and several listeners look up the context for each test.

TL;DR

Same configuration → same (cached) context. Mocks, properties, profiles, imports and `@DirtiesContext` cost a new one. Aim for a handful of configurations per suite.

12.12 `@DynamicPropertySource`: properties known only at runtime

The problem it solves

Some configuration only exists **after** the test starts: a stub server's random port, a container's mapped port, a temporary directory. You cannot write it in `application-test.properties`.

What it is

A **static** method annotated with `@DynamicPropertySource` (Framework 5.2.5) that receives a `DynamicPropertyRegistry`. Each property is registered with a `Supplier` (a Java function that takes nothing and returns a value), which is called lazily (only when needed) when the property is first read, so it can return a value that is only known once something has started. These properties have the **highest precedence**, above `application.properties`, `@TestPropertySource` and `@SpringBootTest(properties)`.

Framework 6.2 added `DynamicPropertyRegistrar`: a *bean* that registers properties. Use it when the value comes from another bean in the context (e.g. a stub server defined as a bean) rather than from a static field.

Example: a fake exchange-rate API on a random port

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
@ActiveProfiles("test")
class FullStackTest {

    // A tiny fake "rates" API started before the Spring context (JDK built-in HTTP server).
    static final HttpServer RATES = startRatesServer();

    static HttpServer startRatesServer() {
        try {
            HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0); // port 0 = any free port
            s.createContext("/rates", ex -> {
                byte[] body = """
                        {"from":"EUR","to":"USD","rate":1.10}""".getBytes(StandardCharsets.UTF_8);
                ex.getResponseHeaders().add("Content-Type", "application/json");
                ex.sendResponseHeaders(200, body.length);
                ex.getResponseBody().write(body);
                ex.close();
            });
            s.start();
            return s;
        } catch (IOException e) { throw new IllegalStateException(e); }
    }

    @DynamicPropertySource
    static void ratesUrl(DynamicPropertyRegistry registry) {
        // evaluated lazily, after the fake server has picked its port
        registry.add("rates.base-url", () -> "http://127.0.0.1:" + RATES.getAddress().getPort());
    }

    @AfterAll
    static void stop() { RATES.stop(0); }
    ...
}
```

The `ExchangeRateClient` bean is created with `rates.base-url` pointing at the fake server, so the test exercises the real `RestClient`, real JSON parsing and real **sockets** (the operating system's network connections). The server is the JDK's built-in `com.sun.net.httpserver.HttpServer`, so no extra dependency is needed.

Common mistakes

- **Non-static method** → the annotation is ignored or rejected; it must be `static` because it runs before any test instance exists.
- **Passing the value, not a supplier** (`registry.add("x", port)` computed too early). Pass a lambda (a short inline function such as `() -> port`).
- **Every class with its own `@DynamicPropertySource`** → each is a different cache key. Put shared containers/stubs and their properties in a base class.

Self-check

1. Why a `Supplier` and not a value? → The value may not exist yet when the method runs; the supplier is read when the property is first needed.
2. What wins: `@TestPropertySource` or `@DynamicPropertySource`? → `@DynamicPropertySource` (highest precedence).

12.13 Full-stack tests on a real port

`FullStackTest` (continued) runs the **whole app on a real Tomcat port**, with a real H2 database and a real HTTP call to the fake rates API, using `RestTestClient` bound to the server:

```
    @LocalServerPort int port;
    @Autowired RestTestClient client;        // bound to the running server in RANDOM_PORT tests
    @Autowired ProductRepository repo;
    @MockitoSpyBean ExchangeRateClient rates; // the REAL bean, wrapped so we can verify calls

    @BeforeEach
    void data() {
        repo.deleteAll();
        repo.save(new Product("MUG-001", "Mug", new BigDecimal("10.00")));
    }

    @Test
    void endToEnd() {
        System.out.println("app on port " + port + ", rates API on port " + RATES.getAddress().getPort());

        PriceView usd = client.get().uri("/api/products/MUG-001/price?currency=USD")
                .exchange().expectStatus().isOk()
                .expectBody(PriceView.class).returnResult().getResponseBody();
        PriceView eur = client.get().uri("/api/products/MUG-001/price")
                .exchange().expectStatus().isOk()
                .expectBody(PriceView.class).returnResult().getResponseBody();

        System.out.println("USD -> " + usd + "\nEUR -> " + eur);
        assertThat(usd.amount()).isEqualByComparingTo("11.00");
        verify(rates, times(1)).rate("EUR", "USD");   // EUR request did not call the API
    }

    @Test
    void createThenRead() {
        client.post().uri("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .body("""
                        {"sku":"PEN-001","name":"Pen","price":2.50}""")
                .exchange()
                .expectStatus().isCreated()
                .expectHeader().location("/api/products/PEN-001/price");

        assertThat(repo.findBySku("PEN-001")).isPresent();   // committed for real, no test rollback
        System.out.println("rows after POST = " + repo.count());
    }
```

Output:

```
app on port 51807, rates API on port 51806
USD -> PriceView[sku=MUG-001, name=Mug, amount=11.00, currency=USD]
EUR -> PriceView[sku=MUG-001, name=Mug, amount=10.00, currency=EUR]
rows after POST = 2
```

Notes:

- `11.00` = 10.00 × 1.10 from the fake server: the full chain controller → service → repository → H2, and service → `RestClient` → HTTP → JSON, worked.
- The spy confirms the business rule end-to-end: exactly **one** remote call, for USD only.
- `rows after POST = 2` (MUG-001 from `@BeforeEach` + the new PEN-001): data is **committed**. Hence `repo.deleteAll()` before each test instead of relying on rollback.
- Ports are random on every run, so parallel builds on one CI (continuous integration: the server that builds and tests every push) machine don't clash.

12.14 Testcontainers and `@ServiceConnection`

(needs Docker — not run.) Docker is not running on the machine that built this book, so `testcontainers-demo` contains a `.tools_run_skip` marker. Its code was **compiled** (`mvn test-compile`) against Boot 4.1.1 and Testcontainers 2.0.5 but not executed.

What it is

Testcontainers is a Java library that starts real services (PostgreSQL, Kafka, Redis, ...) in throw-away **Docker containers** (Docker runs a program with everything it needs in an isolated, lightweight box called a container; the template it starts from is an **image**) from a test and removes them afterwards. `@ServiceConnection` (Boot 3.1) tells Boot to read the connection details (host, mapped port, user, password) straight from the container. You don't copy them into properties yourself.

Why it matters

H2 in PostgreSQL mode is still H2. `ILIKE`, `jsonb`, `ON CONFLICT`, partial indexes, `SKIP LOCKED`, exact numeric and collation (the rules for sorting and comparing text) behaviour, and Flyway migrations written for PostgreSQL can only be proven against PostgreSQL. A container gives every developer and CI run the same real database version, with no shared test server.

How it works

1. `@Testcontainers` (JUnit extension) finds `@Container` fields and starts them. **Static** fields start once per class, instance fields once per test.
2. The container picks a random free host port (a port on your real machine) mapped to 5432, PostgreSQL's port inside the container.
3. `@ServiceConnection` registers a `JdbcConnectionDetails` bean (for PostgreSQL) built from the running container. Boot's `DataSource` auto-configuration uses those details *instead of* `spring.datasource.*`.
4. After the tests the container is stopped. Testcontainers' "Ryuk" helper container also removes leftovers if the JVM crashes.

Code (not run)

`pom.xml` — note the **Testcontainers 2.x artifact names** (`testcontainers-<module>`; 1.x used plain `postgresql`, `junit-jupiter`):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-postgresql</artifactId>
  <scope>test</scope>
</dependency>
```

A repository method that uses PostgreSQL-only SQL:

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    // PostgreSQL-only SQL (ILIKE): H2 would not prove this works in production
    @Query(value = "select * from customer where email ilike concat('%', :part, '%')", nativeQuery = true)
    List<Customer> search(String part);
}
```

The test:

```
/** (needs Docker - not run in this book) A real PostgreSQL in a throw-away container. */
@DataJpaTest     // default Replace.NON_TEST: a @ServiceConnection database is a "test" one, so it is kept
@Testcontainers
class CustomerRepositoryPostgresTest {

    @Container                // started before the tests, stopped after (static = once per class)
    @ServiceConnection        // Boot reads host/port/user/password from the container
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");

    @Autowired CustomerRepository repo;

    @Test
    void ilikeWorksOnRealPostgres() {
        repo.save(new Customer("Ada@Example.com"));
        assertThat(repo.search("example")).extracting(Customer::getEmail).containsExactly("Ada@Example.com");
    }
}
```

In Testcontainers 2 the class is `org.testcontainers.postgresql.PostgreSQLContainer` and is no longer generic. The 1.x class `org.testcontainers.containers.PostgreSQLContainer<?>` still ships but is deprecated.

Containers as beans, reused by tests and by local development

```
@TestConfiguration(proxyBeanMethods = false)
public class ContainersConfig {
    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer("postgres:17-alpine");
    }
}
```

- Tests: `@SpringBootTest @Import(ContainersConfig.class)`. The container is a bean, so it is started with the context and **shared through the context cache** by every test class with the same configuration.
- Local development: a `main` in `src/test/java` starts the real app plus the containers:

```
public class TestTcApplication {
    public static void main(String[] args) {
        SpringApplication.from(TcApplication::main).with(ContainersConfig.class).run(args);
    }
}
```

Run it from the IDE or with `mvn spring-boot:test-run` (Boot 3.1+). New developers get a working database without installing one.

`@ServiceConnection` **vs** `@DynamicPropertySource`

	`@ServiceConnection`	`@DynamicPropertySource`
Code	one annotation	`registry.add("spring.datasource.url", postgres::getJdbcUrl)` + user + password
Supported services	those Boot knows, e.g. JDBC and R2DBC databases, MongoDB, Redis, Kafka, RabbitMQ, Elasticsearch, Cassandra, Neo4j (full list in the Boot reference, "Testcontainers")	anything
Use when	the container is a supported type	custom containers (WireMock, your own image), custom properties

Trade-offs

- **Pro:** production-like behaviour, the same version everywhere, no shared test database.
- **Con:** needs Docker on every machine and CI agent. The first run pulls images, and each container start adds seconds. Mitigate with static/shared containers, the context cache, and CI image caching.

Common mistakes

- **One container per test class** without sharing → dozens of PostgreSQL starts. Share through a base class or `@Import(ContainersConfig.class)`.
- `image:latest` → a new major version breaks the build one morning. Pin the version you run in production.
- **Mixing** `@ServiceConnection` **with** `spring.datasource.url` **in** `application-test.properties` → the connection details win, and the property silently does nothing. Remove it to avoid confusion.

In the interview

Q: How do you test against a real database in Spring Boot? "Testcontainers with `@ServiceConnection`. A static `PostgreSQLContainer` field or a `@TestConfiguration` bean, and Boot takes the JDBC details from the container, so no URL properties are needed. I keep it in one shared configuration so the context cache reuses the same container, and I pin the image to the production version. For services Boot doesn't support, I use `@DynamicPropertySource`."

12.15 Profiles and properties in tests

The tools

Mechanism	Scope	Example
`src/test/resources/application-test.properties` + `@ActiveProfiles("test")`	classes that activate the profile	`FullStackTest` in this chapter
`@SpringBootTest(properties = "k=v")`	one test class	`MockMvcFullContextTest`
`@TestPropertySource(properties = "k=v")` or `(locations = "classpath:x.properties")`	one test class (inherited by subclasses)	`ExchangeRateClientTest`, `C4_OtherPropertyTest`
`@DynamicPropertySource`	one test class, runtime values	`FullStackTest`

Who wins? Measured

`PropertyPrecedenceTest` sets the same keys in several places. It uses the tiny `@JsonTest` slice, because slice annotations also have a `properties` attribute:

```java
@JsonTest(properties = {"demo.a=from-properties-attribute", "demo.b=from-properties-attribute", "demo.d=from-properties-attribute"})
@TestPropertySource(properties = {"demo.a=from-@TestPropertySource", "demo.c=from-@TestPropertySource", "demo.d=from-@TestPropertySource"})
@ActiveProfiles("test")
class PropertyPrecedenceTest {

    @DynamicPropertySource
    static void dynamic(DynamicPropertyRegistry r) {
        r.add("demo.a", () -> "from-@DynamicPropertySource");
        r.add("demo.b", () -> "from-@DynamicPropertySource");
        r.add("demo.c", () -> "from-@DynamicPropertySource");
    }

    @Value("${demo.a}") String a;
    ...
    @Value("${catalog.home-currency}") String fromProfileFile;   // from application-test.properties
}
```

Output:

```
all three set demo.a   -> from-@DynamicPropertySource
attribute vs dynamic   -> from-@DynamicPropertySource
@TestPropertySource vs dynamic -> from-@DynamicPropertySource
attribute vs @TestPropertySource -> from-@TestPropertySource
application-test.properties    -> catalog.home-currency=EUR
```

So, highest first: **`@DynamicPropertySource` → `@TestPropertySource` → the `properties` attribute of `@SpringBootTest`/slices → command-line arguments → system properties → environment variables → `application-{profile}.properties` → `application.properties`**. This matches the Spring Framework reference ("dynamic properties have higher precedence than `@TestPropertySource`"). The ordered list in the Boot reference shows `@TestPropertySource` above `@DynamicPropertySource`, but that is not what Boot 4.1.1 does. Chapter 3 has the full non-test list.

Common mistakes

- **A `src/test/resources/application.properties`** with the same name as the main one *shadows* the main file completely (the test classpath comes first), so every property in the main file disappears in tests. Use `application-test.properties` + `@ActiveProfiles("test")` to *add to* the main file.
- **Activating a different profile in every class** → a new context each time.
- **Production secrets in test profiles.** Tests should use fakes, containers or local stubs.

Self-check

1. Why prefer `application-test.properties` over a test `application.properties`? → The profile file is layered on top of the main file; a same-named file replaces it.
2. Which is applied last and wins: `@TestPropertySource` or `application-test.properties`? → `@TestPropertySource`.

12.16 Common mistakes — checklist

Symptom	Likely cause	Fix
"Unable to find a @SpringBootConfiguration"	test package is not under the main class's package	move the test, or `@SpringBootTest(classes = App.class)`
`No qualifying bean of type ...Service` in `@WebMvcTest`	services aren't in the slice	`@MockitoBean` or `@Import`
`No qualifying bean of type ...MockMvc/RestTestClient/TestRestTemplate` in `@SpringBootTest`	Boot 4 needs explicit opt-in	`@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureTestRestTemplate`
`@MockBean` doesn't compile after upgrading to Boot 4	removed	`@MockitoBean` / `@MockitoSpyBean`
Slice loads repositories/services	custom `@ComponentScan` on the main class	move scanning config to a separate `@Configuration`
Test passes, production fails with `LazyInitializationException`	test-managed transaction kept the session open	non-transactional integration test (ch08)
Data from one test visible in the next	real-port / non-transactional test didn't clean up	`@BeforeEach` cleanup, `@Sql` scripts
Suite gets slower every sprint (a fixed work period, often two weeks)	many unique context configurations, `@DirtiesContext`	base class, slices, cache DEBUG log
Random failures only on CI	fixed ports, shared databases, time or order dependence	`RANDOM_PORT`, containers or embedded DBs, fixed `Clock` bean, no static state

12.17 In the interview

Q: How do you test a Spring Boot REST API? "Three levels. Unit tests for the service logic with Mockito, no Spring. A `@WebMvcTest` per controller with `MockMvcTester` and `@MockitoBean` services to check mappings, validation, status codes and Problem Details. A few `@SpringBootTest`s: mock environment with `@AutoConfigureMockMvc` for wiring, and one or two `RANDOM_PORT` tests with `RestTestClient` for the real HTTP path. Repositories get `@DataJpaTest`, against PostgreSQL via Testcontainers when the SQL is database-specific."

- *`@SpringBootTest` vs `@WebMvcTest`?* → Whole context vs web slice only; the slice is faster and needs mocks for non-web beans.
- *What is `MockMvcTester`?* → Framework 6.2's AssertJ-based MockMvc API: `mvc.get().uri(..).exchange()`, `assertThat(result).hasStatusOk().bodyJson()...`.
- *What's new for testing in Spring Boot 4 / Framework 7?* → Per-technology test starters and moved packages; `@MockBean` removed in favour of `@MockitoBean`; explicit `@AutoConfigure...` for test clients; `RestTestClient`; context pausing; JUnit 6.

Q: How does Spring cache test contexts and what breaks the cache? → See §12.11: the key is the merged configuration; mocks, properties, profiles, imports, dynamic properties and `@DirtiesContext` cause new contexts; default max 32 (LRU).

Q: How do you test code that calls another service? → Unit: mock the client. Client class: `@RestClientTest` + `MockRestServiceServer`, or a local stub server via `@DynamicPropertySource`. End-to-end: stub server or WireMock container; consumer-driven contract tests (Pact; Spring Cloud Contract has been archived) to keep stubs honest (§16.10).

Q: Why might a `@Transactional` test give false confidence? → It rolls back everything, so commit-time behaviour (deferred constraints, `AFTER_COMMIT` listeners, lazy loading outside the session) never runs; and in `RANDOM_PORT` tests it doesn't even cover the server's transaction.

12.18 Types of API testing — and the Spring tool for each

What it is

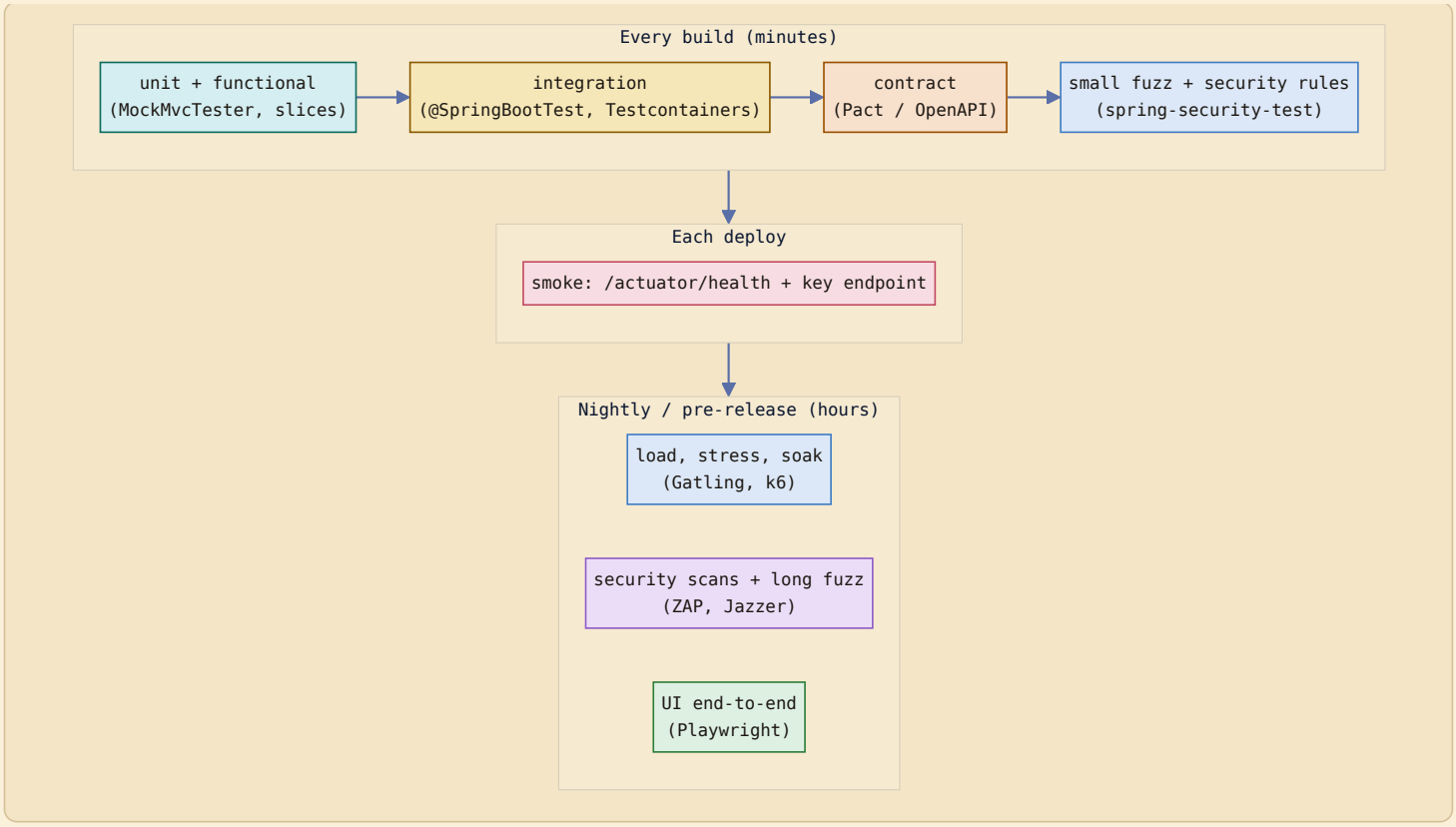
§12.2 sorted tests by **how much of the container they start**. Teams, QA (quality assurance, i.e. testing) engineers and interviewers also sort them by **what question they answer**: "does it start?", "does it follow the spec?", "does it survive 5,000 users?", "can an attacker break it?". These names — smoke, functional, integration, contract, regression, load, stress, security, fuzz, UI — describe the *goal* of a test, not its size. Many are built from the same Spring tools you have seen in this chapter; a few need tools outside Spring.

The problem it solves

A suite of 400 green `@WebMvcTest`s can still ship an API that falls over at 300 requests per second, breaks the mobile app because a field was renamed, or leaks another user's order to anyone who changes an ID in the URL. Naming the test types is a checklist for *which risks you have not tested yet*.

How it works — the matrix

Type	Question it answers	When it runs	Spring / Java tool	In this book
Smoke	Does the app start and answer at all?	every build; right after every deploy (release to a server)	one `@SpringBootTest` that loads the context; after deploy, a script hitting `/actuator/health` (Spring Boot Actuator's health-check URL) and one key endpoint	§12.6, ch15 health
Functional	Does each endpoint behave as the spec says (status, body, errors, validation)?	every build	`@WebMvcTest` + `MockMvcTester`, `@MockitoBean` for services	§12.7, §12.8
Integration	Do the real parts work together: DB, SQL, transactions, HTTP clients, messaging?	every build	`@SpringBootTest`, `@DataJpaTest`, `@RestClientTest`, Testcontainers + `@ServiceConnection`	§12.6, §12.7, §12.14
Contract	Do provider and consumers still agree on the API's shape?	provider and consumer builds	consumer-driven contracts (Pact), OpenAPI (a standard machine-readable description of an HTTP API) schema validation in tests	§16.10
Regression	Did this change break something that used to work?	every build	not a separate tool: the whole automated suite, plus one new test for every bug fixed	whole chapter
Load	Does it meet its latency/throughput (time per request / requests per second) goal (SLO, service level objective) at expected peak?	before releases, nightly	Gatling (Java/Scala DSL), k6, JMeter (load-generating tools) against a production-like environment; watch Micrometer (Spring's metrics library) metrics	ch15
Stress / spike / soak	Where does it break and how; does it recover from a sudden burst; does it leak over 8 hours?	before big launches	same tools, different load shape	—
Security	Are unauthenticated users rejected, authorisation rules enforced, inputs sanitised?	every build (+ periodic scans)	`spring-security-test` (`@WithMockUser`, `jwt()`), OWASP ZAP (an automated web security scanner) scans, dependency scanning for known-vulnerable libraries (OWASP Dependency-Check, Dependabot)	10.12, 11.6
Fuzz	Does random, malformed or hostile input ever crash it (5xx = server-error status codes, hang, leak)?	every build (small), nightly (large)	a seeded random loop over `MockMvcTester` (below), property-based testing (jqwik; generates many inputs and checks a rule holds for all), coverage-guided fuzzing (Jazzer; mutates inputs to reach new code paths), OpenAPI-driven fuzzers (Schemathesis)	below
UI / end-to-end	Does a real user journey work through the browser?	before release	Playwright or Selenium (tools that drive a real browser) against a `RANDOM_PORT` app or staging (a production-like pre-release environment)	§12.13



Two terms that are often mixed up: **load** testing checks behaviour *at* the expected peak (e.g. 2,000 requests/s with **p99** under 200 ms — p99 is the latency that 99% of requests beat; see System Design Fundamentals), **stress** testing pushes *beyond* it to find the breaking point and check the failure is graceful (429s — "Too Many Requests" responses — and **back-pressure**, i.e. pushing back on callers when overloaded, not an out-of-memory crash), and **soak** testing holds normal load for hours to reveal leaks and slow degradation. A load test on a laptop measures the laptop — run it against an environment sized like production.

Seeing it: a fuzz test — `ProductApiFuzzTest`

The test sends 500 requests to the web slice: `POST /api/products` bodies built from valid and hostile values (empty strings, `null`, wrong types, `1e400`, SQL-injection (input crafted to change the meaning of a database query) and `<script>` strings, a 5,000-character SKU, truncated JSON, 200 levels of nesting) and `GET` requests with random path segments. It asserts one property: **the API never answers 5xx**. The `Random` **seed** (the starting number that determines the whole random sequence) is fixed, so any failure can be replayed exactly. It reuses the same configuration as `ProductControllerWebMvcTest`, so it costs no extra context start.

```
Random rnd = new Random(12);                                  // simplified excerpt: POST branch only
for (int i = 0; i < 500; i++) {
    String body = rnd.nextInt(6) == 0 ? pick(rnd, JUNK)
            : "{\"sku\":%s,\"name\":%s,\"price\":%s}".formatted(pick(rnd, SKUS), pick(rnd, NAMES), pick(rnd, PRICES));
    int status = mvc.post().uri("/api/products").contentType(MediaType.APPLICATION_JSON).content(body)
            .exchange().getResponse().getStatus();
    assertThat(status).as("request %d", i).isLessThan(500);   // 4xx is fine; 5xx is a bug
}
```

```
fuzz: 500 requests, status histogram {201=5, 400=395, 404=100}
```

Bean Validation (`@Valid` plus constraint annotations such as `@NotBlank`), Jackson's parse errors and the Problem Details handler turned every bad input into a 400 or 404. If an input had produced a 500, the fix would be a validation rule or an `@ExceptionHandler`, plus a normal functional test for that exact input (a new **regression** test).

When not to use it

- Don't build a separate "smoke suite" that duplicates functional tests; a context-load test plus a post-deploy health check is enough.
- Don't run load or stress tests on every commit; they need a stable, production-like environment and minutes to hours.
- Don't write end-to-end UI tests for rules a functional test can prove; keep UI tests to a handful of critical journeys.

Common mistakes

- **Only happy-path functional tests** → add negative cases: 400, 401, 403, 404, 409, and "user A reads user B's order" (broken object-level authorisation, OWASP API Security Top 10 #1; OWASP is a non-profit that publishes widely used security checklists).
- **Load testing without a goal** → state the target first (requests/s, p95/p99 latency, error rate), then test against it.

- **Unseeded random tests** → a failure you can't reproduce; always log or fix the seed.
- **Mocking the other service in every test and never checking the real contract** → green builds, broken integration; add contract tests.

In the interview

> **30-second answer:** "I map test types to risks. Functional tests with `@WebMvcTest` and `MockMvcTester` check each endpoint's contract, including error cases. Integration tests with `@SpringBootTest` and Testcontainers check the real database and wiring. Contract tests with Pact keep services compatible. Security tests use `spring-security-test` for 401/403 rules plus a ZAP scan. A smoke check runs after every deploy. Load, stress and soak tests with Gatling or k6 run before releases against a production-like environment, and I fuzz inputs to make sure nothing returns a 5xx."

- *Regression test?* → Any automated test that guards behaviour that once worked; the rule is one new test per fixed bug.
- *What does a fuzz test assert if you don't know the expected output?* → Properties that must always hold: no 5xx, no hang, no stack trace in the body, response matches the schema.

Self-check

1. Load vs stress test? → Load checks the SLO at expected peak; stress goes past it to find the breaking point and failure mode.
2. Which test type catches a renamed JSON field before the mobile app breaks? → A contract test.
3. Why fix the random seed in a fuzz test? → So a failing input can be replayed and turned into a regression test.

TL;DR

Smoke = does it start; functional = does each endpoint follow the spec (`@WebMvcTest`); integration = real parts together (`@SpringBootTest`, Testcontainers); contract = services agree (Pact); regression = the suite guarding old behaviour; load/stress/soak = performance at, beyond and over time (Gatling, k6); security and fuzz = hostile users and inputs (`spring-security-test`, ZAP, seeded random inputs); UI = a few critical browser journeys (Playwright).

Sources: Spring Framework reference (MockMvc, TestContext), Spring Boot reference (testing, Actuator health), OWASP API Security Top 10 (2023), OWASP ZAP docs, Gatling and k6 documentation, Pact documentation, jqwik and Jazzer user guides.

12.19 Chapter TL;DR

- **Pyramid**: many Spring-free unit tests (ms) → slices for layer behaviour (~0.1 s context) → a few `@SpringBootTest`s (~0.7 s context) → very few real-port tests.
- **Boot 4**: test starters per technology (`spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test`, ...); slice annotations moved packages; JUnit 6, AssertJ, Mockito inside `spring-boot-starter-test`.
- **TestContext Framework**: `SpringExtension` + listeners + a JVM-wide **context cache** keyed by the merged configuration.
- `@SpringBootTest`: `MOCK` (default), `RANDOM_PORT`, `DEFINED_PORT`, `NONE`; add `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` explicitly in Boot 4; `@TestConfiguration` adds test beans.
- **Slices**: `@WebMvcTest` (127 vs 253 beans here), `@DataJpaTest` (embedded DB, rolled back, SQL shown), `@JsonTest`, `@RestClientTest` + `MockRestServiceServer`.
- `MockMvcTester`: AssertJ assertions on MockMvc results; `RestTestClient` (Framework 7.0): one fluent client for MockMvc or a real server.
- `@MockitoBean` / `@MockitoSpyBean` replace the removed `@MockBean` / `@SpyBean`; reset after each test; part of the cache key.
- **Caching demo**: 7 classes, 4 context builds; hits cost 0 ms, misses ~0.5 s. Mocks, properties and `@DirtiesContext` cause misses. Max 32 contexts; Framework 7 **pauses** inactive ones.
- `@DynamicPropertySource`: lazy suppliers for runtime values (random ports), highest precedence.
- **Testcontainers +** `@ServiceConnection`: real PostgreSQL with zero URL config (needs Docker — not run here); TC 2 artifacts `testcontainers-postgresql`; containers as beans also power `mvn spring-boot:test-run`.
- **Test types by goal** (§12.18): smoke, functional, integration, contract, regression, load/stress/soak, security, fuzz, UI — map each risk to a type; fuzz with a fixed seed and assert "never 5xx".
- Next: Chapter 13 — Reactive programming and WebFlux.

Chapter 13 — Reactive Programming and Spring WebFlux

Most Spring applications spend their time **waiting**: for the database, for another HTTP service, for a **message broker** (a server such as Kafka or RabbitMQ that stores messages between the programs that send and receive them). In the classic model (Spring MVC on Tomcat, the default Java web server) each request holds one **thread** (an independent path of execution that the CPU runs; each has its own stack of method calls) from start to finish, including all that waiting. **Reactive programming** takes the opposite approach: no thread ever waits. Work is described as a pipeline of steps, and a small number of threads run each step when its data arrives. **Spring WebFlux** is Spring's web framework built on that model, using **Project Reactor**.

Since Java 21 there is a second way to get the same **scalability** (the ability to handle more load by adding resources): **virtual threads** (very lightweight threads managed by the JVM instead of the operating system), which make waiting cheap instead of avoiding it. This chapter teaches the reactive model properly, then **measures** blocking MVC on virtual threads against WebFlux, so you can choose between them with numbers rather than opinions.

What you'll learn

1. Why blocking I/O limits a thread-per-request server, and how an event loop avoids it.
2. The **Reactive Streams** contract: `Publisher`, `Subscriber`, `Subscription`, demand (`request(n)`).
3. **Project Reactor**: `Mono` and `Flux`, laziness, cold vs hot, the core operators.
4. Testing reactive code with `StepVerifier` and **virtual time**.
5. Error handling: fallbacks, retry with backoff, timeouts.
6. **Backpressure** in practice: demand, `limitRate`, drop/latest/buffer strategies.
7. **Schedulers**: `publishOn` vs `subscribeOn`, and how to wrap blocking calls.
8. **WebFlux**: annotated controllers, functional routes, Server-Sent Events, tested with `WebTestClient`.
9. `WebClient` against a real local server: timeouts, retries, concurrent calls.
10. **R2DBC**: reactive SQL with Spring Data R2DBC on H2, and reactive transactions.
11. **Virtual threads vs WebFlux**: a measured comparison for an I/O-bound endpoint, and when to choose which.

All code is in `code/`. Spring Boot **4.1.1** (Framework 7.0, Reactor 3.8 from the Reactor 2025.0 release train — a set of library versions released together, Reactor Netty — a non-blocking network server built on the Netty library, R2DBC H2 1.1), Java 25:

Project	Shows
`reactor-basics`	Reactor only (no Spring context): creation, operators, errors, virtual time, backpressure, schedulers, sinks, context — all as `StepVerifier` tests
`webflux-app`	a book API: annotated controller, functional routes, SSE, `WebClient`, R2DBC repository + `DatabaseClient` + reactive transaction
`load-mvc`, `load-webflux`	the same I/O-bound endpoint (one that spends its time waiting for input/output rather than computing) under load: MVC on platform threads (ordinary OS threads), MVC on virtual threads, WebFlux

Output blocks are real `mvn -q -B -ntp verify` output, with Spring log lines trimmed to `...`. **All timings are machine-dependent** (14-core Apple M4 Pro, load generator — the program that fires test requests — in the same JVM as the server).

> **Prerequisites and links.** If you know JavaScript, the mental model is the same as Node's: see the JavaScript handbook, Chapter 12 — Asynchronous JavaScript, especially "The event loop" and "Promises". A `Mono` is roughly a lazy, cancellable Promise (JavaScript's object for a value that will arrive later), and a `Flux` a lazy async stream. Virtual threads and `CompletableFuture` (Java's equivalent of a Promise) are in the Java handbook, Chapter 15. Spring MVC is Chapter 5, the blocking `RestClient` is Chapter 6, and general Spring testing is Chapter 12.

13.1 Why reactive? The cost of waiting

The problem it solves

In Spring MVC, a request is served by **one thread from start to finish** (*thread-per-request*). Suppose the handler calls a database that answers in 100 ms. For those 100 ms the thread does nothing but wait, yet it is not available to anybody else. Tomcat's worker pool (the fixed set of threads that handle requests) has **200 threads** by default. So at most 200 requests can be *in progress* at once, and with 100 ms of waiting each, the server tops out around **200 / 0.1 s = 2,000 requests per second**, however idle the CPU is. More platform threads cost memory (each has a **stack**, the memory that holds its chain of method calls, typically reserving up to 1 MB) and scheduling overhead (the OS's work of switching between threads), so "just add threads" stops working at a few thousand.

What reactive does instead

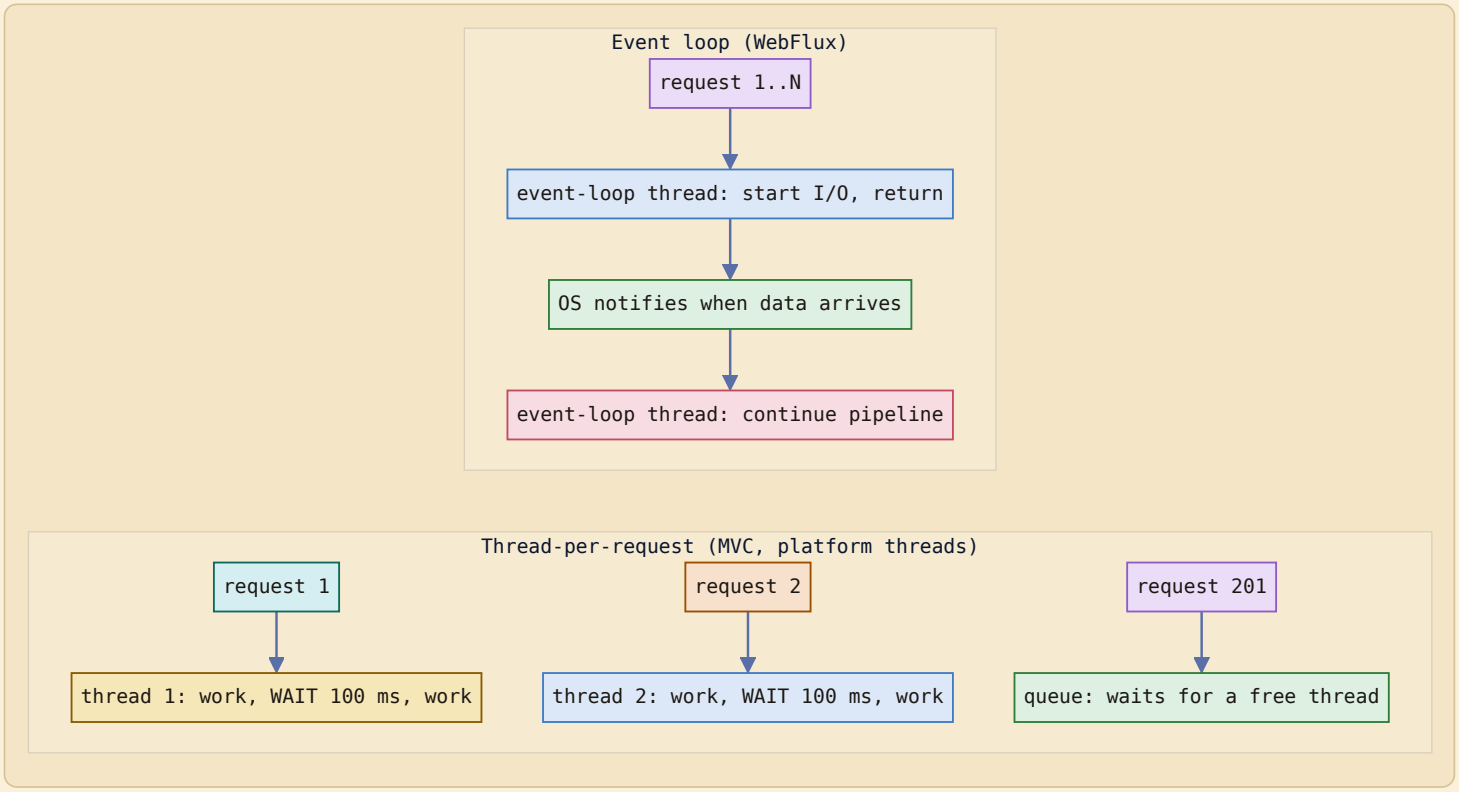
I/O (input/output) means reading or writing outside the program: network, disk, database. **Blocking** I/O makes the calling thread stop until the data is there; reactive (non-blocking) I/O never parks a thread on a **socket** (an open network connection):

1. The handler starts the database or HTTP call and registers **what to do when the answer arrives**.

2. The thread immediately returns to the **event loop** and picks up other work.
3. When the operating system reports that the answer has arrived (via epoll/kqueue, the Linux and macOS mechanisms that tell a program which sockets have data ready), an event-loop thread runs the next step of the pipeline.

A handful of event-loop threads (by default one per CPU core in Reactor Netty) can then keep thousands of requests in progress. **Analogy:** a waiter who takes an order, hands it to the kitchen, and serves other tables until the bell rings, instead of standing at the kitchen door until the dish is ready.

This is exactly how Node.js works. Everything in the JavaScript event-loop chapter (never block the loop, **callbacks** — functions you hand over to be called later — run when I/O completes, one slow **synchronous** function — one that does all its work before returning — freezes everyone) applies to WebFlux's event loop too.



The price

- Code becomes a **pipeline of operators** instead of plain statements. Stack traces, debugging and `ThreadLocal`-based libraries (a `ThreadLocal` is a variable with a separate value per thread; MDC logging — the Mapped Diagnostic Context that adds fields like a request id to every log line, security context, transactions) need reactive-aware replacements.
- **Everything** on the request path must be non-blocking: web server, HTTP client, database driver (the library that speaks a database's network protocol). One blocking call on an event loop stalls every request that loop serves (§13.15 measures this).
- Since Java 21, **virtual threads** remove the thread-count limit for ordinary blocking code, which removes the main *performance* reason to go reactive for many apps.

Self-check

1. Why does a 200-thread Tomcat cap out near 2,000 req/s for a 100 ms I/O call? → Each in-flight (started but not finished) request holds a thread while it waits; 200 threads × 10 requests/s each.
2. What does an event-loop thread do while a database call is in progress? → Other work. The continuation runs when the OS signals that data has arrived.

13.2 Reactive Streams: the contract

What it is

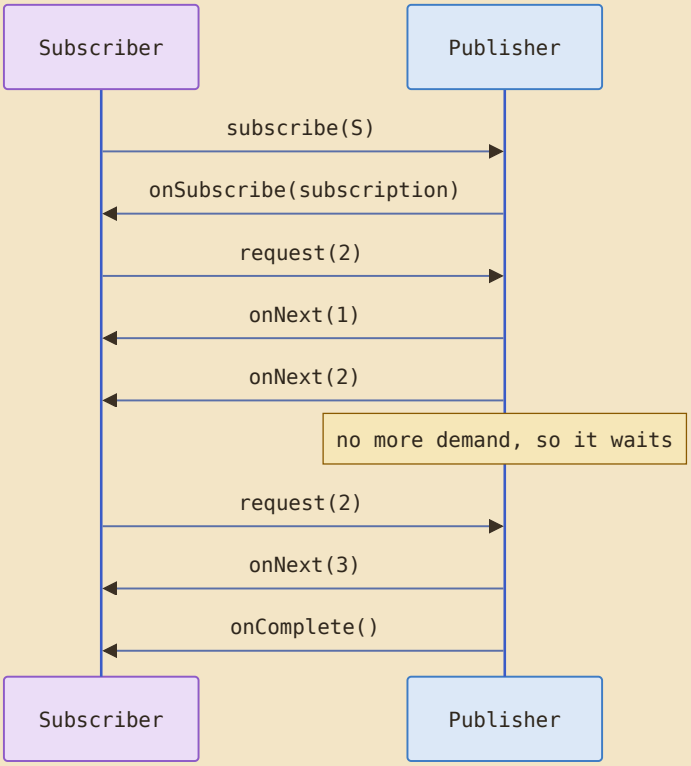
Reactive Streams (version 1.0, 2015) is a small specification for **asynchronous stream processing with non-blocking backpressure** (*asynchronous* = the caller does not wait; the result is delivered later). It is four interfaces, copied into the JDK in Java 9 as `java.util.concurrent.Flow`:

Interface	Role	Methods
`Publisher<T>`	produces items	`subscribe(Subscriber)`
`Subscriber<T>`	consumes items	`onSubscribe(Subscription)`, `onNext(T)`, `onError(Throwable)`, `onComplete()`
`Subscription`	the link between the two	`request(long n)`, `cancel()`
`Processor<T,R>`	both a subscriber and a publisher	—

Backpressure means the **consumer controls the flow rate**: the publisher may send at most as many items as the subscriber has requested. A slow consumer is never flooded.

How it works

1. `publisher.subscribe(subscriber)` — nothing flows yet.
2. The publisher calls `subscriber.onSubscribe(subscription)`.
3. The subscriber calls `subscription.request(n)`, meaning "I can take n more".
4. The publisher sends up to n `onNext` signals, then waits for more demand.
5. The stream ends with exactly one `onComplete` **or** `onError`, or the subscriber calls `cancel()`.



You rarely implement these interfaces yourself. Libraries do: **Project Reactor** (Spring's choice), RxJava, Mutiny, Akka Streams, and drivers such as R2DBC. Because they share the contract they interoperate.

Self-check

1. What does `request(n)` mean? → The subscriber can accept n more items; the publisher must not send more than the outstanding demand.
2. How many terminal signals (signals that end the stream) can a stream send? → At most one: `onComplete` or `onError` (none if cancelled or infinite).

13.3 Project Reactor: `Mono` and `Flux`

What it is

Project Reactor implements Reactive Streams with two publisher types and hundreds of operators:

Type	Emits	Like	Examples
`Mono<T>`	0 or 1 item, then complete (or an error)	a lazy `CompletableFuture<Optional<T>>` / a lazy Promise	find by id, save, an HTTP response
`Flux<T>`	0..N items (possibly infinite), then complete (or an error)	a lazy async `Stream<T>` / async iterator	query results, SSE events, Kafka records

An **operator** (`map`, `filter`, `flatMap`, ...) returns a *new* publisher that wraps the previous one. Chaining operators **assembles a pipeline**. Nothing runs until something **subscribes**.

Creating publishers — `CreationAndLazinessTest`

```
StepVerifier.create(Mono.just("one")).expectNext("one").verifyComplete();
StepVerifier.create(Mono.empty()).verifyComplete();                        // 0 items, then complete
StepVerifier.create(Flux.range(1, 3)).expectNext(1, 2, 3).verifyComplete();
StepVerifier.create(Flux.fromIterable(List.of("a", "b"))).expectNext("a", "b").verifyComplete();
StepVerifier.create(Flux.error(new IllegalStateException("boom")))
        .expectErrorMessage("boom").verify();
```

Other factories you'll meet: `Mono.fromCallable(() -> ...)` (wrap a computation), `Mono.fromFuture(cf)`, `Flux.interval(Duration)` (a tick every period, on the parallel scheduler — a **scheduler** decides which threads run the work, §13.8), `Flux.create(sink -> ...)` (bridge callback APIs), `Mono.delay(Duration)`.

Nothing happens until you subscribe

```
@Test
void nothingHappensUntilYouSubscribe() {
    AtomicInteger calls = new AtomicInteger();
    Flux<Integer> pipeline = Flux.range(1, 3)
            .map(i -> { calls.incrementAndGet(); return i * 10; });

    System.out.println("after assembling the pipeline, map ran " + calls.get() + " times");

    List<Integer> out = pipeline.collectList().block();   // block() subscribes (fine in a test)
    System.out.println("after subscribing: " + out + ", map ran " + calls.get() + " times");

    pipeline.blockLast();                                 // cold: a 2nd subscriber re-runs everything
    System.out.println("after a second subscriber, map ran " + calls.get() + " times");
}

@Test
void justIsEagerDeferIsLazy() {
    AtomicInteger dbHits = new AtomicInteger();
    Mono<Integer> eager = Mono.just(dbHits.incrementAndGet());                // runs NOW, once
    Mono<Integer> lazy  = Mono.defer(() -> Mono.just(dbHits.incrementAndGet())); // runs per subscribe
    System.out.println("hits after building both monos: " + dbHits.get());
    eager.block(); eager.block(); lazy.block(); lazy.block();
    System.out.println("hits after 2x eager + 2x lazy subscribe: " + dbHits.get());
}
```

Output:

```
just/empty/range/fromIterable/error verified
after assembling the pipeline, map ran 0 times
after subscribing: [10, 20, 30], map ran 3 times
after a second subscriber, map ran 6 times
hits after building both monos: 1
hits after 2x eager + 2x lazy subscribe: 3
```

Lessons:

- **Assembly vs subscription.** Building the chain does nothing. Subscribing runs it. In WebFlux **the framework subscribes** to the `Mono` / `Flux` your controller returns. If you build a pipeline and neither return it nor subscribe to it, it never runs (a classic bug: a `repo.save(x)` whose result is ignored saves nothing).
- **Cold publishers** replay the whole pipeline for every subscriber: two subscribers = two database queries.
- **`Mono.just(expr)` evaluates `expr` immediately**, at assembly time, even if nobody ever subscribes. Use `Mono.defer(() -> ...)` or `Mono.fromCallable(...)` for work that should happen per subscription (and only when subscribed).

`block()`

`block()` subscribes and **waits** for the result. It is fine in tests, in `main` methods and at the boundary of a non-reactive app. Inside a reactive pipeline or on an event-loop thread it is a bug: Reactor throws `IllegalStateException("block()/blockFirst()/blockLast() are blocking, which is not supported in thread reactor-http-nio-...")`.

Self-check

1. A controller method calls `repo.save(book);` and returns `Mono.just(book)`. What's wrong? → The `save` Mono is never subscribed, so nothing is saved. Return `repo.save(book)` instead.

2. `Mono.just(expensive())` vs `Mono.fromCallable(this::expensive)`? → The first runs `expensive()` immediately, once; the second runs it on each subscription.

13.4 Operators

The ones you need daily

Goal	Operator	Note
transform each item (sync)	`map`	must not block or return a publisher
keep some items	`filter`, `take(n)`, `skip(n)`, `distinct`	
async call per item	**`flatMap`**	runs inner publishers **concurrently**, output **interleaved**
async call per item, in order, one at a time	**`concatMap`**	sequential; slower
async per item, concurrent, output in source order	**`flatMapSequential`**	buffers early results
combine by position	`zip`, `zipWith`	waits for one from each
one after another	`concat`, `concatWith`	
as they come	`merge`, `mergeWith`	
fold to one value	`reduce`, `count`, `collectList`, `collectMap`	returns `Mono`
fallback if empty	`switchIfEmpty`, `defaultIfEmpty`	common for 404s
group	`buffer(n)`, `window(n)`, `groupBy`	
side effects (logging, metrics)	`doOnNext`, `doOnError`, `doOnSubscribe`, `doFinally`	don't put business logic here
ignore values, wait for completion	`then()`, `then(mono)`	"do A, then B"

`flatMap` vs `concatMap` vs `flatMapSequential` — `OperatorsTest`

Three users; fetching the orders of user N takes (4 − N) seconds per order, so user 3 answers first:

```
/** Simulated remote call: user N answers after (4 - N) seconds, so later users answer first. */
static Flux<String> ordersOf(int user) {
    return Flux.just("u" + user + "-a", "u" + user + "-b")
            .delayElements(Duration.ofSeconds(4 - user));
}

@Test
void flatMapInterleavesConcatMapKeepsOrder() {
    // withVirtualTime: the delays run on a fake clock, so this 12-"second" test takes milliseconds
    StepVerifier.withVirtualTime(() -> Flux.range(1, 3).flatMap(OperatorsTest::ordersOf).collectList())
            .thenAwait(Duration.ofHours(1))
            .consumeNextWith(l -> System.out.println("flatMap           : " + l))
            .verifyComplete();
    // ... same with concatMap and flatMapSequential
}

@Test
void howLongDoTheyTake() {
    // concatMap: user 1 (3 s + 3 s), then user 2 (2 s + 2 s), then user 3 (1 s + 1 s) = 12 s
    var concat = StepVerifier.withVirtualTime(() -> Flux.range(1, 3).concatMap(OperatorsTest::ordersOf))
            .expectSubscription()
            .thenAwait(Duration.ofSeconds(11)).expectNextCount(5)
            .expectNoEvent(Duration.ofMillis(999))
            .thenAwait(Duration.ofMillis(1)).expectNext("u3-b").verifyComplete();
    // flatMap: all three run at the same time, the slowest (user 1) needs 6 s
    var flat = StepVerifier.withVirtualTime(() -> Flux.range(1, 3).flatMap(OperatorsTest::ordersOf))
            .expectSubscription()
            .thenAwait(Duration.ofSeconds(5)).expectNextCount(5)
            .thenAwait(Duration.ofSeconds(1)).expectNext("u1-b").verifyComplete();
    System.out.println("concatMap needs 12 virtual s, flatMap 6 virtual s; real time: "
            + concat.toMillis() + " ms and " + flat.toMillis() + " ms");
}
```

Output:

```
flatMap           : [u3-a, u2-a, u3-b, u1-a, u2-b, u1-b]
concatMap         : [u1-a, u1-b, u2-a, u2-b, u3-a, u3-b]
flatMapSequential : [u1-a, u1-b, u2-a, u2-b, u3-a, u3-b]
concatMap needs 12 virtual s, flatMap 6 virtual s; real time: 0 ms and 0 ms
```

- `flatMap` subscribes to all inner publishers at once (up to a `concurrency` limit, default 256) and emits results **as they arrive**: fastest, order lost.
- `concatMap` waits for each inner publisher to finish before starting the next: order kept, total time = sum (12 s).
- `flatMapSequential` runs concurrently like `flatMap` (6 s) but re-orders the output to match the source.
- Limit fan-out (one item triggering many parallel calls) against a fragile service with `flatMap(fn, 8)`: at most 8 calls in flight.

Combining

```
Flux<String> names = Flux.just("Ada", "Linus", "Grace");
Flux<Integer> years = Flux.just(1815, 1969, 1906);
StepVerifier.create(Flux.zip(names, years, (n, y) -> n + "@" + y))
        .expectNext("Ada@1815", "Linus@1969", "Grace@1906").verifyComplete();

StepVerifier.create(Flux.concat(Flux.just(1, 2), Flux.just(3))).expectNext(1, 2, 3).verifyComplete();

Mono<String> fromCache = Mono.empty();
StepVerifier.create(fromCache.switchIfEmpty(Mono.just("from DB")))      // fallback when empty
        .expectNext("from DB").verifyComplete();

StepVerifier.create(Flux.range(1, 7).buffer(3))
        .expectNext(List.of(1, 2, 3), List.of(4, 5, 6), List.of(7)).verifyComplete();
```

`Mono.zip(a, b, c)` is the reactive "call three services in parallel and combine", the equivalent of `Promise.all` in JavaScript or `CompletableFuture.allOf` in Java.

Common mistakes

- **`map` with an async call**: `flux.map(id -> repo.findById(id))` produces a `Flux<Mono<Book>>` whose inner Monos are never subscribed. Use `flatMap`.
- **`flatMap` when order matters**, such as applying events in sequence. Use `concatMap`.
- **Unbounded `flatMap` against a rate-limited API.** Pass a concurrency limit.
- **Business logic in `doOnNext`**. It's for side effects. An exception thrown there becomes an error signal, and it's easy to forget the call isn't part of the data flow.

Self-check

1. You need to call a service for each of 1,000 ids, at most 10 at a time, order irrelevant. → `Flux.fromIterable(ids).flatMap(this::call, 10)`.
2. Why did `flatMap` finish in 6 s and `concatMap` in 12 s? → `flatMap` ran the three inner calls concurrently (time of the slowest); `concatMap` ran them one after another (sum).

13.5 Testing with `StepVerifier` and virtual time

What it is

`StepVerifier` (in `reactor-test`) subscribes to a publisher and checks, step by step, the signals it emits: `expectNext(...)`, `expectNextCount(n)`, `assertNext(v -> ...)`, `expectError(Type)`, `expectComplete()`, `thenRequest(n)`, `thenCancel()`. A script ends with `verify()` (or `verifyComplete()` / `verifyError()`), which **runs** the subscription and **blocks** until the script is done. It returns the time it took.

Virtual time replaces Reactor's time-based schedulers with a `VirtualTimeScheduler` whose clock only moves when the test says so (`thenAwait(Duration)`), or jumps when it's idle. A test of a one-hour interval or a retry with **exponential backoff** (waiting longer after each failure: 1 s, 2 s, 4 s, ...) runs in milliseconds and is **deterministic** (gives the same result every run).

How to use virtual time

1. Build the publisher **inside the supplier**: `StepVerifier.withVirtualTime(() -> Flux.interval(...))`. The virtual scheduler must be installed *before* the operators capture a scheduler. A publisher created outside the lambda keeps using real time.
2. Use `expectSubscription()` first, then `expectNoEvent(d)` to assert that nothing happened for a period (it also advances the clock).
3. Use `thenAwait(d)` to move the clock forward, then `expectNext(...)`.

The trap: a test that waits forever

If the script expects a signal that only comes after virtual time passes, but never calls `thenAwait`, `verify()` waits **forever**. This chapter's first attempt at `howLongDoTheyTake` did exactly that and hung the build until an external alarm killed it. Two defences:

```
# src/test/resources/junit-platform.properties
# safety net: a reactive test that waits for a signal that never comes fails instead of hanging
junit.jupiter.execution.timeout.default=30 s
```

and `verify(Duration.ofSeconds(5))` on real-time scripts (used throughout `webflux-app`).

Self-check

1. Why must the publisher be created inside `withVirtualTime(() -> ...)`? → So its time-based operators pick up the virtual scheduler.
2. What does `StepVerifier.create(flux, 0)` do? → Subscribes with initial demand 0, so you control demand with `thenRequest(n)` (§13.7).

13.6 Error handling

How errors flow

An exception inside an operator becomes an `onError` **signal**. It travels down the pipeline, skipping `map`/`filter`, until an error operator handles it or it reaches the subscriber. **An error terminates the sequence**: items after it are never processed. This is like a rejected Promise, not like a `try/catch` inside a loop.

```
@Test
void anErrorEndsTheStream() {
    StepVerifier.create(Flux.just(1, 2, 0, 4).map(i -> 12 / i))
            .expectNext(12, 6)
            .expectError(ArithmeticException.class)   // 4 is never processed
            .verify();
}

@Test
void fallbacks() {
    Flux<Integer> risky = Flux.just(1, 2, 0, 4).map(i -> 12 / i);
    StepVerifier.create(risky.onErrorReturn(-1)).expectNext(12, 6, -1).verifyComplete();
    StepVerifier.create(risky.onErrorResume(e -> Flux.just(100, 200))).expectNext(12, 6, 100, 200).verifyComplete();
    StepVerifier.create(risky.onErrorMap(e -> new IllegalArgumentException("bad input", e)))
            .expectNext(12, 6).expectErrorMessage("bad input").verify();
}
```

Operator	Effect	Imperative equivalent
`onErrorReturn(v)`	emit a fallback value, then complete	`catch → return v`
`onErrorResume(e -> publisher)`	switch to another publisher (cache, default, another service)	`catch → return other()`
`onErrorMap(e -> new X(e))`	translate the exception	`catch → throw new X(e)`
`doOnError(e -> log)`	side effect, error continues	`catch → log; throw`
`retry(n)`	**re-subscribe** to the source up to n times	loop around the call
`retryWhen(Retry.backoff(...))`	retry with delays, filters, limits	retry library
`timeout(d[, fallback])`	error (or fallback) if no signal within d	`future.get(timeout)`
`onErrorContinue`	drop the failing item and continue (only with supporting operators)	avoid — surprising semantics

Retry with exponential backoff, verified on virtual time

```
@Test
void retryWithExponentialBackoffOnVirtualTime() {
    List<Long> attemptTimes = new ArrayList<>();
    StepVerifier.withVirtualTime(() -> {
                VirtualTimeScheduler clock = VirtualTimeScheduler.get();
                return Mono.defer(() -> {
                            attemptTimes.add(clock.now(TimeUnit.MILLISECONDS));
                            return Mono.<String>error(new IllegalStateException("down"));
                        })
                        .retryWhen(Retry.backoff(3, Duration.ofSeconds(1)).jitter(0)); // 1 s, 2 s, 4 s
            })
            .thenAwait(Duration.ofMinutes(1))
            .expectErrorMatches(e -> e.getMessage().startsWith("Retries exhausted: 3/3"))
            .verify();
    System.out.println("attempts at virtual ms: " + attemptTimes);
}

@Test
void timeoutWithFallback() {
    StepVerifier.withVirtualTime(() -> Mono.delay(Duration.ofSeconds(5)).map(x -> "slow answer")
                    .timeout(Duration.ofSeconds(2), Mono.just("fallback")))
            .expectSubscription()
            .thenAwait(Duration.ofSeconds(2))
            .expectNext("fallback")
            .verifyComplete();
    ...
}
```

Output:

```
attempts at virtual ms: [0, 1000, 3000, 7000]
retry(5): succeeded after 3 attempts
timeout(2s) on a 5 s call -> fallback / TimeoutException
onErrorReturn / onErrorResume / onErrorMap verified
```

The first attempt runs at 0. The retries wait 1 s, 2 s and 4 s, so they run at 1,000, 3,000 and 7,000 ms. After 3 retries the error becomes `Retries exhausted: 3/3`. Real systems keep the default **jitter** (random ±50 % here) so that many clients don't retry in lockstep; `jitter(0)` is only for the deterministic test. Retry and circuit-breaker (a guard that stops calling a failing service for a while) design in general: microservices notes — retry.

Retry re-subscribes. That only repeats the work if the source is lazy. `Mono.defer(...)`, `fromCallable` and `WebClient` calls are lazy. A `Mono.just(valueComputedEarlier)` would just replay the same value.

Common mistakes

- **Retrying non-idempotent calls** (an *idempotent* call has the same effect whether it runs once or many times; a `POST` that charges a card is not) without an idempotency key (a unique id the server uses to spot and ignore duplicates; see System Design Fundamentals).
- **Retrying on 4xx** (client-error status codes, which will fail again). Filter to transient errors (temporary ones, such as a 503, that may succeed on a later try): `Retry.backoff(...).filter(e -> e instanceof ServiceUnavailable)`.
- **`try/catch` around a reactive chain.** It only catches assembly-time errors, not errors that happen later when the pipeline runs.

13.7 Backpressure in practice

What it is

Backpressure = the consumer tells the producer how much it can handle, through `request(n)`. In Reactor most operators pass demand **upstream** (towards the source; *downstream* is towards the subscriber) automatically. A subscriber like `block()` or WebFlux's HTTP writer requests in batches.

Demand you can see — `BackpressureTest`

```
@Test
void subscriberControlsTheRate() {
    List<Long> requests = new CopyOnWriteArrayList<>();
    List<Integer> received = new ArrayList<>();

    Flux.range(1, 10)
        .doOnRequest(requests::add)                 // what the subscriber asked for
        .subscribe(new BaseSubscriber<Integer>() {
            @Override protected void hookOnSubscribe(Subscription s) { request(2); }  // start with 2
            @Override protected void hookOnNext(Integer v) {
                received.add(v);
                if (received.size() % 2 == 0) request(2);                           // then 2 at a time
            }
        });
    System.out.println("requests = " + requests + "\nreceived = " + received);
}

@Test
void limitRateReplenishesInBatches() {
    List<Long> upstreamRequests = new CopyOnWriteArrayList<>();
    Flux.range(1, 100)
        .doOnRequest(upstreamRequests::add)
        .limitRate(10)                              // never ask upstream for more than 10
        .blockLast();                               // downstream asks for "unbounded"
    System.out.println("limitRate(10) upstream requests = " + upstreamRequests);
}

@Test
void stepVerifierCanRequestManually() {
    StepVerifier.create(Flux.range(1, 5), 0)          // initial demand 0
            .expectSubscription()
            .expectNoEvent(Duration.ofMillis(50))     // nothing arrives without demand
            .thenRequest(2).expectNext(1, 2)
            .thenRequest(3).expectNext(3, 4, 5)
            .verifyComplete();
}
```

Output:

```
limitRate(10) upstream requests = [10, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8]
demand 0 -> nothing; request(2) -> 1,2; request(3) -> 3,4,5
requests = [2, 2, 2, 2, 2, 2]
received = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

- The source sent exactly as many items as requested, 2 at a time.
- `limitRate(10)` turns a downstream "give me everything" (`Long.MAX_VALUE`) into bounded requests. It first asks for 10, then **re-requests 8 when 75 % have been consumed** (the default "low tide"), so the next batch is already on its way before the buffer runs dry. Use it to protect a source that fetches in pages (a database cursor — a pointer that walks through query results a batch at a time, a paged API).
- With demand 0, a well-behaved publisher sends nothing.

When the source can't slow down: overflow strategies

Some sources **ignore demand**: UI events, sensor readings, a price ticker, a hot message stream. For them you choose what happens to the excess. The test's `ticker()` pushes 1..20 at once through `Flux.create(..., OverflowStrategy.IGNORE)`, while the consumer asks for only 5:

```
static Flux<Integer> ticker() {
    return Flux.create(sink -> {
        for (int i = 1; i <= 20; i++) sink.next(i);
        sink.complete();
    }, FluxSink.OverflowStrategy.IGNORE);   // IGNORE = no built-in protection, the operator below decides
}

StepVerifier.create(ticker().onBackpressureDrop(dropped::add), 5).expectNext(1, 2, 3, 4, 5).verifyComplete();

SlowConsumer latest = new SlowConsumer();             // requests 5 on subscribe
ticker().onBackpressureLatest().subscribe(latest);    // 20 items pushed during this call
latest.request(5);                                    // ask for more afterwards

SlowConsumer buffered = new SlowConsumer();
ticker().onBackpressureBuffer(10).subscribe(buffered); // 6..15 fit in the buffer, 16 overflows
buffered.request(50);
```

Output:

```
DROP   : kept 1..5, dropped 15 items (6..20)
LATEST : [1, 2, 3, 4, 5, 20] complete
BUFFER(10): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15] error: The receiver is overrun by more signals than expected (bounded queue...)
```

Strategy	What happens to excess	Use for
`onBackpressureDrop`	discarded (optional callback to count/log them)	metrics samples, mouse moves
`onBackpressureLatest`	keep only the newest waiting item	"current price", UI state
`onBackpressureBuffer(n)`	queue up to n, then **error** (or a chosen overflow action)	short bursts that must not be lost
`onBackpressureBuffer()` (unbounded)	queue everything	risky: memory grows until `OutOfMemoryError`
`onBackpressureError`	fail immediately	when overflow is a bug

`Flux.create` accepts the same choices directly (`OverflowStrategy.DROP/LATEST/BUFFER/ERROR`).

Backpressure across the network

Inside one JVM, demand is exact. Across HTTP it becomes **TCP flow control** (TCP, the protocol under HTTP, stops the sender when the receiver's buffer is full; see System Design Fundamentals). When a WebFlux server writes a `Flux` to a slow client, the socket's send buffer fills, Netty stops requesting, and the demand stops at the source (for example the R2DBC cursor). Between services, HTTP/1.1 has no message-level backpressure. Protocols like RSocket or gRPC (binary service-to-service protocols with built-in flow control), or a message broker in between, provide it.

Self-check

1. What does `limitRate(10)` change? → Upstream never receives a request larger than 10; replenishes in batches of 8 (75 %).
2. A price feed pushes faster than your UI can render. Which strategy? → `onBackpressureLatest`: only the newest price matters.
3. Why is an unbounded buffer dangerous? → A consumer that stays slower than the producer makes the buffer grow until memory runs out.

13.8 Schedulers and threads

What it is

Reactor is **concurrency-agnostic** (it does not pick threads for you): by default, each signal runs on whatever thread emitted it (the subscribing thread, a Netty event loop, a timer thread). **Schedulers** move work onto specific thread pools:

Scheduler	Threads	For
`Schedulers.parallel()`	one per CPU core	CPU-bound work (limited by computation, not waiting), timers (`interval`, `delayElements`)
`Schedulers.boundedElastic()`	grows on demand up to 10 × cores (default), idle threads reclaimed	**wrapping blocking calls** (JDBC, legacy SDKs — older vendor client libraries, file I/O)
`Schedulers.single()`	one reusable thread	low-latency one-off tasks
`Schedulers.immediate()`	the current thread	tests, defaults
Reactor Netty event loops (`reactor-http-nio-N`)	one per core (min 4)	network I/O — **never block these**

- **`publishOn(s)`** switches the thread for everything **after** it in the chain.
- **`subscribeOn(s)`** chooses the thread on which the **subscription happens**, which moves the **source** (and everything up to the first `publishOn`). Its position in the chain doesn't matter.

```java
@Test
void publishOnSwitchesDownstreamSubscribeOnMovesTheSource() {
    List<String> log = new CopyOnWriteArrayList<>();
    Flux.just(1)
        .doOnNext(i -> log.add("source    on " + thread()))
        .publishOn(Schedulers.parallel())               // everything BELOW runs on parallel
        .doOnNext(i -> log.add("after pub on " + thread()))
        .subscribeOn(Schedulers.boundedElastic())       // the subscription (and source) runs here
        .blockLast();
    log.forEach(System.out::println);
}

@Test
void wrapABlockingCall() {
    // A blocking API (JDBC, legacy SDK) must not run on the few event-loop/parallel threads.
    Mono<String> legacy = Mono.fromCallable(() -> {
                Thread.sleep(50);                       // pretend: blocking I/O
                return "legacy result on " + thread();
            })
            .subscribeOn(Schedulers.boundedElastic());   // dedicated, growable pool for blocking work
    System.out.println(legacy.block());
}
```

Output (thread numbers replaced by `N`):

```
legacy result on boundedElastic-N
source    on boundedElastic-N
after pub on parallel-N
```

The `Mono.fromCallable(blocking).subscribeOn(Schedulers.boundedElastic())` pattern is how you call unavoidable blocking code from a reactive app. It keeps the event loops free, at the cost of a thread per concurrent call, exactly the cost reactive was meant to avoid. If most calls are blocking, a reactive stack buys you nothing.

`BlockHound`, a Java agent (a plug-in that instruments classes as the JVM loads them) from the Reactor team, detects blocking calls on non-blocking threads during tests.

13.9 Hot vs cold, sinks and context

Cold vs hot

- **Cold** (the default): each subscriber gets its own run from the start. Examples: a database query, a `WebClient` call, `Flux.range`.
- **Hot**: data is produced whether or not anyone listens, and late subscribers **miss** what was emitted earlier. Examples: stock ticks, mouse events, a chat room.

Sinks (`Sinks.many()`, `Sinks.one()`) are the safe way to push values into a `Flux` from **imperative** code (ordinary step-by-step statements), for example to broadcast events to all SSE clients:

```java
Sinks.Many<String> prices = Sinks.many().multicast().directBestEffort();
prices.asFlux().subscribe(early::add);
prices.tryEmitNext("p1");
prices.asFlux().subscribe(late::add);   // joins after p1 was emitted
prices.tryEmitNext("p2");
prices.tryEmitComplete();
```

Output:

```
early subscriber: [p1, p2], late subscriber: [p2]
```

Other sinks: `Sinks.many().replay().limit(n)` (late subscribers get the last n), `unicast()` (one subscriber, buffers). Turn a cold publisher hot with `share()`, `publish().autoConnect()`, or `cache()` (replay to later subscribers).

Context: the reactive replacement for `ThreadLocal`

A pipeline may hop between threads, so `ThreadLocal` values (request id in the logging MDC, the security context, the current transaction) get lost. Reactor's `Context` is an immutable (cannot be changed after creation) key/value map attached to the **subscription**. You write it at the bottom with `contextWrite` and read it anywhere above with `deferContextual`:

```java
Mono<String> greeting = Mono.deferContextual(ctx -> Mono.just("request " + ctx.get("requestId")));
StepVerifier.create(greeting.contextWrite(c -> c.put("requestId", "r-42")))
        .expectNext("request r-42").verifyComplete();
```

Output:

```
context value read inside the pipeline: r-42
```

Spring uses it for you: reactive Spring Security stores the `SecurityContext` there, and reactive transactions keep the connection there. For logging MDC and **tracing** (following one request across threads and services, Chapter 15), Boot can bridge `ThreadLocal`s and the Reactor `Context` through Micrometer's context-propagation library (`spring.reactor.context-propagation=auto`).

13.10 Spring WebFlux: annotated controllers

What it is

Spring WebFlux (Framework 5.0+) is the reactive counterpart of Spring MVC. It runs on **Reactor Netty** by default (Tomcat and Jetty also work, through the Servlet non-blocking I/O API). The request flow mirrors MVC: `HttpHandler` → `WebFilter`s → `DispatcherHandler` → `HandlerMapping` → `HandlerAdapter` → your handler → `HttpMessageWriter` (Jackson 3 **codecs**: encoders/decoders between bytes and Java objects). Every step is non-blocking.

	Spring MVC	Spring WebFlux
I/O model	blocking (one thread per request) or virtual threads	non-blocking event loop
Server	Tomcat / Jetty (Servlet)	Reactor Netty (default), Tomcat, Jetty
Return types	`T`, `ResponseEntity<T>` (also accepts `Mono`/`Flux`)	`Mono<T>`, `Flux<T>`, `T`
Data access	JDBC, JPA	R2DBC, reactive MongoDB/Redis/Cassandra
HTTP client	`RestClient`	`WebClient`
Test client	`MockMvcTester`, `RestTestClient` (ch12)	`WebTestClient`
Starter (Boot 4)	`spring-boot-starter-webmvc`	`spring-boot-starter-webflux`

If **both** starters are on the classpath, Boot starts **MVC**. Force WebFlux with `spring.main.web-application-type=reactive`.

`webflux-app/pom.xml` (main parts):

```
<!-- Spring WebFlux + Reactor Netty server + Jackson 3 -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-webflux</artifactId></dependency>
<!-- Boot 4: the auto-configured WebClient.Builder lives in its own module/starter -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-webclient</artifactId></dependency>
<!-- Spring Data R2DBC + the reactive H2 driver -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-r2dbc</artifactId></dependency>
<dependency><groupId>io.r2dbc</groupId><artifactId>r2dbc-h2</artifactId><scope>runtime</scope></dependency>
<!-- tests: @WebFluxTest, WebTestClient, reactor-test / @DataR2dbcTest -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-webflux-test</artifactId><scope>test</scope>
</dependency>
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-r2dbc-test</artifactId><scope>test</scope>
</dependency>
```

> **Boot 4 gotcha (found while building this chapter):** `spring-boot-starter-webflux` no longer brings the `WebClient.Builder` auto-configuration. Without `spring-boot-starter-webclient` the app fails with *"No qualifying bean of type 'org.springframework.web.reactive.function.client.WebClient$Builder' available"*.

The controller

Same annotations as MVC (Chapter 5); only the return types change:

```java
@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookRepository repo;                // reactive repository (§13.14)

    public BookController(BookRepository repo) { this.repo = repo; }

    @GetMapping
    public Flux<Book> all(@RequestParam(required = false) String q) {
        return q == null ? repo.findAll() : repo.search(q);
    }

    @GetMapping("/{id}")
    public Mono<Book> one(@PathVariable long id) {
        return repo.findById(id)
                .switchIfEmpty(Mono.error(new ResponseStatusException(HttpStatus.NOT_FOUND, "no book " + id)));
    }

    @PostMapping
    public Mono<ResponseEntity<Book>> create(@RequestBody Mono<Book> body) {
        return body.map(b -> b.withId(null))                    // never trust a client-sent id
                .flatMap(repo::save)
                .map(saved -> ResponseEntity.created(URI.create("/api/books/" + saved.id())).body(saved));
    }
    ...
}
```

- `Flux<Book>` is written as a JSON array (or streamed as NDJSON — newline-delimited JSON, one object per line — / SSE when the client asks for `application/x-ndjson` / `text/event-stream`).
- An empty `Mono` would produce `200` with an empty body, so `switchIfEmpty(Mono.error(...))` turns "not found" into a 404.
- `@RequestBody Mono<Book>` makes even body parsing non-blocking. A plain `@RequestBody Book` also works.
- **You return the pipeline, the framework subscribes.** Nothing in this class calls `subscribe()` or `block()`.

Testing it — `@WebFluxTest` + `WebTestClient`

```java
/** WebFlux slice: controller + codecs + error handling; the repository is a mock returning Mono/Flux. */
@WebFluxTest(BookController.class)
class BookControllerWebFluxTest {

    @Autowired WebTestClient client;
    @MockitoBean BookRepository repo;

    static final Book DDD = new Book(1L, "Domain-Driven Design", "Eric Evans", "9780321125217");

    @Test
    void listReturnsJsonArray() {
        given(repo.findAll()).willReturn(Flux.just(DDD));
        client.get().uri("/api/books").exchange()
              .expectStatus().isOk()
              .expectHeader().contentType(MediaType.APPLICATION_JSON)
              .expectBodyList(Book.class).hasSize(1).contains(DDD);
    }

    @Test
    void missingBookIs404() {
        given(repo.findById(99L)).willReturn(Mono.empty());
        client.get().uri("/api/books/99").exchange()
              .expectStatus().isNotFound()
              .expectBody()
              .consumeWith(r -> System.out.println("GET /api/books/99 -> 404 " + new String(r.getResponseBody())))
              .jsonPath("$.status").isEqualTo(404);
    }

    @Test
    void createReturns201() {
        given(repo.save(new Book(null, "Refactoring", "Martin Fowler", "9780134757599")))
                .willReturn(Mono.just(new Book(7L, "Refactoring", "Martin Fowler", "9780134757599")));
        client.post().uri("/api/books").contentType(MediaType.APPLICATION_JSON)
              .bodyValue("""
                      {"id":123,"title":"Refactoring","author":"Martin Fowler","isbn":"9780134757599"}""")
              .exchange()
              .expectStatus().isCreated()
              .expectHeader().location("/api/books/7")
              .expectBody().jsonPath("$.id").isEqualTo(7);
    }
}
```

Output:

```
POST /api/books (client id 123 ignored) -> 201 Location: /api/books/7
GET /api/books/99 -> 404 {"timestamp":"2026-09-29T00:24:18.499Z","path":"/api/books/99","status":404,"error":"Not
Found","requestId":"25c24062"}
GET /api/books -> 200 [Domain-Driven Design]
```

- `@WebFluxTest` is the WebFlux slice (ch12 §12.7). Unlike `@SpringBootTest`, it provides a `WebTestClient` without an extra annotation.
- The mock returns `Mono`/`Flux`: mocks of reactive repositories must never return `null` (that causes a `NullPointerException` inside the framework).
- The 404 body is Boot's default reactive error format. For RFC 9457 Problem Details set `spring.webflux.problemdetails.enabled=true` or add a `@RestControllerAdvice` returning `ProblemDetail`, exactly as in MVC.

13.11 Functional endpoints

What it is

WebFlux (and MVC, via `RouterFunctions` in `spring-webmvc`) offers a second programming model: routes are **code**, a `RouterFunction<ServerResponse>` bean, and handlers are plain functions `ServerRequest → Mono<ServerResponse>`. No annotations, no reflection, so routing is explicit and easy to unit-test.

```
@Configuration(proxyBeanMethods = false)
public class BookRoutes {

    @Bean
    RouterFunction<ServerResponse> bookRouter(BookHandler h) {
        return route()
                .path("/fn/books", b -> b
                        .GET("/{isbn}", h::byIsbn)
                        .GET("", h::byAuthor))                    // /fn/books?author=...
                .filter((req, next) -> {                          // a filter for this router only
                    req.exchange().getResponse().getHeaders().add("X-Router", "functional");
                    return next.handle(req);
                })
                .build();
    }

    @Component
    static class BookHandler {
        private final BookRepository repo;
        BookHandler(BookRepository repo) { this.repo = repo; }

        Mono<ServerResponse> byIsbn(ServerRequest req) {
            return repo.findByIsbn(req.pathVariable("isbn"))
                    .flatMap(b -> ServerResponse.ok().contentType(MediaType.APPLICATION_JSON).bodyValue(b))
                    .switchIfEmpty(ServerResponse.notFound().build());
        }

        Mono<ServerResponse> byAuthor(ServerRequest req) {
            return req.queryParam("author")
                    .map(a -> ServerResponse.ok().body(repo.findByAuthor(a), Book.class))
                    .orElseGet(() -> ServerResponse.badRequest().bodyValue("author is required"));
        }
    }
}
```

A detail found while testing: the first version of the filter changed the headers of the returned `ServerResponse` (`res.headers().add(...)`). That failed with `UnsupportedOperationException` and a 500, because a built `ServerResponse` is immutable. Add headers through the exchange (as above) or build a new response.

Testing without Spring

Routes are plain objects, so `WebTestClient` can bind straight to them. No context, no server:

```
class FunctionalRoutesTest {
    BookRepository repo = mock(BookRepository.class);
    WebTestClient client = WebTestClient
            .bindToRouterFunction(new BookRoutes().bookRouter(new BookRoutes.BookHandler(repo)))
            .build();

    @Test
    void byIsbnFoundAndNotFound() {
        given(repo.findByIsbn("111")).willReturn(Mono.just(new Book(1L, "T", "A", "111")));
        given(repo.findByIsbn("222")).willReturn(Mono.empty());

        client.get().uri("/fn/books/111").exchange()
              .expectStatus().isOk()
              .expectHeader().valueEquals("X-Router", "functional")
              .expectBody().jsonPath("$.title").isEqualTo("T");
        client.get().uri("/fn/books/222").exchange().expectStatus().isNotFound();
    }
    ...
}
```

Output:

```
GET /fn/books/111 -> 200 (X-Router: functional), /fn/books/222 -> 404
GET /fn/books?author=Evans -> 200 [1 book], /fn/books -> 400 author is required
```

Annotated vs functional

	Annotated controllers	Functional endpoints
Routing	annotations, discovered by scanning	explicit code, composable
Argument binding / validation	automatic (`@PathVariable` , `@Valid`)	manual (`req.pathVariable` , `req.bodyToMono` , call a `Validator`)
Familiar to most Spring devs	yes	less
Good for	most CRUD (create, read, update, delete) APIs	small services, dynamic routes, tight control, GraalVM-friendly code (GraalVM compiles Java ahead of time into a native executable)

Both can live in one app (as here).

13.12 Full-stack WebFlux tests and Server-Sent Events

Server-Sent Events

Server-Sent Events (SSE) is a simple standard for a server to **push** a stream of text events over one long-lived HTTP response (`Content-Type: text/event-stream`). Browsers consume it with `EventSource` and reconnect automatically. It's one-way (server → client). For two-way messaging use WebSockets (a protocol that upgrades an HTTP connection into a long-lived, two-way channel).

In WebFlux an SSE endpoint is just a `Flux` with the right media type:

```
/** Server-Sent Events: one "tick" per interval, pushed over a single open HTTP response. */
@GetMapping(path = "/ticks", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> ticks(@RequestParam(defaultValue = "1000") long everyMs) {
    return Flux.interval(Duration.ofMillis(everyMs)).map(i -> "tick " + i);
}
```

Each element is written as `data:tick 0` plus a blank line. For event names, ids or retry hints, return `Flux<ServerSentEvent<T>>` . When the client disconnects, the subscription is **cancelled** and `Flux.interval` stops: no leaked timers. (Spring MVC offers the same with `SseEmitter` , or by returning a `Flux` from an MVC controller, but each open stream then holds a servlet async request.)

The full-stack test

```
/** Real Netty server on a random port, real R2DBC/H2 database seeded by data.sql. */
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureWebTestClient
class FullStackWebFluxTest {

    @Autowired WebTestClient client;

    @Test
    void searchHitsTheDatabase() {
        client.get().uri("/api/books?q=reactive").exchange()
              .expectStatus().isOk()
              .expectBody()
              .consumeWith(r -> System.out.println("GET /api/books?q=reactive -> " + new String(r.getResponseBody())))
              .jsonPath("$.length()").isEqualTo(2);
    }

    @Test
    void serverSentEvents() {
        Flux<String> events = client.get().uri("/api/books/ticks?everyMs=100")
                .accept(MediaType.TEXT_EVENT_STREAM)
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentTypeCompatibleWith(MediaType.TEXT_EVENT_STREAM)
                .returnResult(String.class)
                .getResponseBody();                      // a live Flux of the streamed events

        StepVerifier.create(events)
                .expectNext("tick 0", "tick 1", "tick 2")
                .thenCancel()                            // closes the connection; the server stops its interval
                .verify(Duration.ofSeconds(5));
    }
}
```

Output:

```
GET /api/books?q=reactive -> [{"id":2,"title":"Hands-On Reactive Programming in Spring 5","author":"Oleh Dokuka","isbn":"9781787284951"},
{"id":1,"title":"Reactive Spring","author":"Josh Long","isbn":"9781732910409"}]
SSE received: tick 0, tick 1, tick 2 (then cancelled)
```

- In Boot 4, `WebTestClient` in a `@SpringBootTest` needs **`@AutoConfigureWebTestClient`** (from `spring-boot-starter-webflux-test`). It is then bound to the random-port server.
- `returnResult(String.class).getResponseBody()` gives the **live stream**. Combining `WebTestClient` with `StepVerifier` is the standard way to test infinite or streaming endpoints: take a few items, then cancel.

13.13 `WebClient`: calling other services without blocking

What it is

`WebClient` is the non-blocking, reactive HTTP client (Framework 5.0+). Boot auto-configures a `WebClient.Builder` with its codecs and observability (metrics and tracing for each call, Chapter 15) (from `spring-boot-starter-webclient` in Boot 4). Its API mirrors `RestClient` (Chapter 6) but returns `Mono`/`Flux`.

```java
@Component
public class InventoryClient {
    public record Stock(String isbn, int copies) { }

    private final WebClient http;

    public InventoryClient(WebClient.Builder builder, @Value("${inventory.base-url}") String baseUrl) {
        this.http = builder.baseUrl(baseUrl).build();
    }

    public Mono<Stock> stock(String isbn) {
        return http.get().uri("/stock/{isbn}", isbn)
                .retrieve()
                .onStatus(HttpStatusCode::is5xxServerError,
                        r -> Mono.error(new InventoryUnavailable("inventory said " + r.statusCode().value())))
                .bodyToMono(Stock.class)
                .timeout(Duration.ofMillis(500))                                  // per attempt
                .retryWhen(Retry.backoff(2, Duration.ofMillis(100))
                        .filter(e -> e instanceof InventoryUnavailable));          // only retry 5xx
    }

    /** Several calls at once: total time is roughly the slowest call, not the sum. */
    public Mono<Integer> totalCopies(List<String> isbns) {
        return Flux.fromIterable(isbns)
                .flatMap(this::stock)
                .map(Stock::copies)
                .reduce(0, Integer::sum);
    }
}
```

- `retrieve()` + `bodyToMono` is the simple path. 4xx/5xx responses become `WebClientResponseException` unless you map them with `onStatus`.
- The `timeout` sits **before** `retryWhen`, so it limits **each attempt**. Put it after `retryWhen` to limit the whole operation.
- Only `InventoryUnavailable` (5xx) is retried. A 404 is not.

Tested against a real local server

Instead of a mock, the test starts a real **Reactor Netty HTTP server** on a random port, scripted to be fast, slow, flaky or down. `@DynamicPropertySource` points the client at it:

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)
class InventoryClientTest {

    static final AtomicInteger flakyCalls = new AtomicInteger();
    static final AtomicInteger downCalls = new AtomicInteger();

    static final DisposableServer INVENTORY = HttpServer.create()
            .host("127.0.0.1").port(0)
            .route(r -> r.get("/stock/{isbn}", (req, res) -> {
                String isbn = req.param("isbn");
                String json = "{\"isbn\":\"" + isbn + "\",\"copies\":3}";
                return switch (isbn) {
                    case "slow" -> res.header("Content-Type", "application/json")
                            .sendString(Mono.just(json).delayElement(Duration.ofSeconds(2)));
                    case "flaky" -> flakyCalls.incrementAndGet() <= 2
                            ? res.status(503).send()
                            : res.header("Content-Type", "application/json").sendString(Mono.just(json));
                    case "down" -> { downCalls.incrementAndGet(); yield res.status(500).send(); }
                    default -> res.header("Content-Type", "application/json")      // 300 ms "network + work"
                            .sendString(Mono.just(json).delayElement(Duration.ofMillis(300)));
                };
            }))
            .bindNow();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("inventory.base-url", () -> "http://127.0.0.1:" + INVENTORY.port());
    }

    @Autowired InventoryClient client;

    @Test
    void callsRunConcurrently() {
        long t0 = System.nanoTime();
        Integer total = client.totalCopies(List.of("a", "b", "c", "d", "e")).block();
        long ms = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("5 calls of 300 ms each -> total copies " + total + " in " + ms + " ms (machine-dependent)");
    }

    @Test
    void timeoutFires() {
        StepVerifier.create(client.stock("slow")).expectError(TimeoutException.class).verify(Duration.ofSeconds(5));
    }

    @Test
    void retriesTransient5xx() {
        StepVerifier.create(client.stock("flaky")).expectNextCount(1).verifyComplete();
        StepVerifier.create(client.stock("down"))
                .expectErrorMatches(e -> e.getMessage().startsWith("Retries exhausted: 2/2"))
                .verify(Duration.ofSeconds(5));
        assertThat(downCalls).hasValue(3);   // 1 try + 2 retries
    }
}
```

Output:

```
5 calls of 300 ms each -> total copies 15 in 322 ms (machine-dependent)
slow (2 s) -> TimeoutException after the 500 ms timeout
flaky: 503, 503, 200 -> success after 3 calls; down: gave up after 3 calls
```

Five 300 ms calls finished in **322 ms**, not 1,500 ms, because `flatMap` ran them concurrently and no thread waited on any of them. (With `RestClient` you'd get the same wall time by running the calls on virtual threads. §13.15 compares the two approaches.)

Common mistakes

- **`.block()` on a `WebClient` call inside a WebFlux handler** → `IllegalStateException` (blocking on a non-blocking thread). Return the `Mono`.
- **A new `WebClient.create()` per call** → no shared **connection pool** (a set of open connections that are reused instead of opened per call), no Boot customisation. Build once from the injected builder.
- **Ignoring the body of an error response** → the connection may not be released. Let `retrieve()` handle it, or consume/release the body in `exchangeToMono`.
- **No timeout.** A hung remote service keeps the request open forever. Add `timeout(...)` and/or set connect/response timeouts on the underlying `HttpClient`.

13.14 R2DBC: reactive SQL

What it is

JDBC is **blocking by design**: `executeQuery()` holds the calling thread until rows arrive. **R2DBC** (Reactive Relational Database Connectivity, spec 1.0 in 2022) is a non-blocking SPI (service provider interface: a standard API that each database vendor implements in its driver) for SQL databases. Drivers exist for PostgreSQL, MySQL/MariaDB, SQL Server, Oracle and H2. **Spring Data R2DBC** adds repositories and `R2dbcEntityTemplate` on top. Spring's `DatabaseClient` offers fluent SQL.

It is **not an ORM** (object-relational mapper, such as Hibernate, which maps object graphs to tables automatically): no lazy loading, no relationships or cascades (operations that spread to related rows), no dirty checking (detecting changed fields to update automatically), no first-level cache. An entity maps to one table. Joins are written by hand. That is deliberate: lazy loading would mean hidden blocking I/O. Compare Chapter 8.

Configuration

```
spring.r2dbc.url=r2dbc:h2:mem:///books;DB_CLOSE_DELAY=-1
# run schema.sql / data.sql through R2DBC at startup
spring.sql.init.mode=always
```

For PostgreSQL: `spring.r2dbc.url=r2dbc:postgresql://host:5432/db` with the `org.postgresql:r2dbc-postgresql` driver. Boot adds a connection pool (`r2dbc-pool`) automatically. Schema migrations (versioned scripts that create and change tables): Flyway and Liquibase need JDBC, so apps usually keep a JDBC driver *only* for migrations.

Entity and repository

```
/**
 * A row of table BOOK. id == null means "not saved yet" (INSERT), otherwise UPDATE.
 * No explicit name: Spring Data R2DBC quotes identifiers, so @Table("book") would look for a
 * lower-case "book" table, while H2 stored the unquoted name as BOOK.
 */
@Table
public record Book(@Id Long id, String title, String author, String isbn) {
    public Book withId(Long newId) { return new Book(newId, title, author, isbn); }
}

public interface BookRepository extends ReactiveCrudRepository<Book, Long> {

    Flux<Book> findByAuthor(String author);            // derived query, like Spring Data JPA

    Mono<Book> findByIsbn(String isbn);

    @Query("SELECT * FROM book WHERE LOWER(title) LIKE LOWER(CONCAT('%', :word, '%')) ORDER BY title")
    Flux<Book> search(String word);
}
```

The Javadoc comment records a real failure from this chapter. With `@Table("book")` every generated statement failed with `Table "book" not found (candidates are: "BOOK")`. Spring Data R2DBC **quotes** identifiers, which makes them case-sensitive. The unquoted `CREATE TABLE book` in `schema.sql` produced `BOOK` in H2 (PostgreSQL would fold it to lower case instead). Either omit the name, as here, so Spring derives it in the dialect's (the specific database's SQL flavour) default case, or match the case exactly.

Tests — `@DataR2dbcTest`, `DatabaseClient`, a reactive transaction

```java
/** R2DBC slice: repositories + DatabaseClient on H2. Reactive tests are NOT rolled back automatically. */
@DataR2dbcTest
class BookRepositoryR2dbcTest {

    @Autowired BookRepository repo;
    @Autowired DatabaseClient db;
    @Autowired ReactiveTransactionManager txManager;

    @Test
    void derivedAndCustomQueries() {
        StepVerifier.create(repo.findByAuthor("Brian Goetz").map(Book::title))
                .expectNext("Java Concurrency in Practice").verifyComplete();
        StepVerifier.create(repo.search("SPRING").map(Book::title).collectList())
                .assertNext(l -> System.out.println("search('SPRING') -> " + l))
                .verifyComplete();
    }

    @Test
    void saveAssignsId() {
        StepVerifier.create(repo.save(new Book(null, "Release It!", "Michael Nygard", "9781680502398"))
                        .flatMap(saved -> repo.findById(saved.id())))
                .assertNext(b -> System.out.println("saved and re-read: " + b))
                .verifyComplete();
        repo.deleteAll(repo.findByIsbn("9781680502398")).block();   // clean up ourselves
    }

    @Test
    void databaseClientAndReactiveTransaction() {
        TransactionalOperator tx = TransactionalOperator.create(txManager);

        Mono<Long> insertThenFail = db.sql("INSERT INTO book(title, author, isbn) VALUES ('Tmp', 'X', 'tmp-1')")
                .fetch().rowsUpdated()
                .then(Mono.<Long>error(new IllegalStateException("fail after insert")))
                .as(tx::transactional);                      // the whole pipeline is one transaction

        StepVerifier.create(insertThenFail).expectErrorMessage("fail after insert").verify();

        StepVerifier.create(db.sql("SELECT COUNT(*) AS n FROM book WHERE isbn = 'tmp-1'")
                        .map(row -> row.get("n", Long.class)).one())
                .assertNext(n -> System.out.println("rows with isbn tmp-1 after rollback: " + n))
                .verifyComplete();
    }
}
```

Output:

```
saved and re-read: Book[id=7, title=Release It!, author=Michael Nygard, isbn=9781680502398]
rows with isbn tmp-1 after rollback: 0
search('SPRING') -> [Hands-On Reactive Programming in Spring 5, Reactive Spring]
```

What it shows:

- **Derived queries and `@Query`** work as in Spring Data JPA, but return `Flux`/`Mono`. `save` on a record with `id == null` INSERTs and returns the row with its generated id. (The id is 7 because the in-memory database is shared with the other test contexts in the same JVM, and `data.sql` ran several times.)
- **Reactive transactions**: `TransactionalOperator` (or `@Transactional` on a method returning `Mono`/`Flux`, backed by `R2dbcTransactionManager`) binds the connection to the **subscription's Context**, not to a thread (Chapter 7 covers the thread-bound version). The error after the INSERT rolled it back: 0 rows.
- **`@DataR2dbcTest` is not transactional.** Unlike `@DataJpaTest`, it is not meta-annotated with `@Transactional` (checked in Boot 4.1.1), because the TestContext's test-managed transactions are thread-bound. Clean up in the test, as `saveAssignsId` does.

When not to use R2DBC

When you need JPA features (relationships, lazy loading, caching), when your team knows JPA well, or when you're on virtual threads anyway. The ecosystem is smaller and some drivers lag behind their JDBC versions.

13.15 Virtual threads vs WebFlux: a measured comparison

The question

Java 21 made **virtual threads** final (Java ch15): threads managed by the JVM that cost a few hundred bytes to a few KB each instead of a platform thread's MB-sized stack. When a virtual thread blocks on I/O, the JVM **unmounts** (detaches) it from its carrier (platform) thread, and the carrier runs another virtual thread. Blocking code gets event-loop-like scalability *without changing the code*. Spring Boot (3.2+) switches Tomcat's request handling, `@Async`, `@Scheduled` and more to virtual threads with **one property**:

```
spring.threads.virtual.enabled=true
```

So: for an I/O-bound endpoint, how do **MVC on platform threads**, **MVC on virtual threads** and **WebFlux** compare?

The setup

Two tiny projects with the same endpoint: wait 100 ms (simulated I/O), then answer.

```
// load-mvc: blocking. The thread WAITS 100 ms, like a JDBC query or a blocking HTTP call.
@GetMapping("/io")
String io(@RequestParam(defaultValue = "100") long ms) throws InterruptedException {
    Thread.sleep(ms);
    return "ok on " + (Thread.currentThread().isVirtual() ? "virtual" : "platform") + " thread";
}

// load-webflux: non-blocking. A timer completes the response after 100 ms; no thread waits.
@GetMapping("/io")
Mono<String> io(@RequestParam(defaultValue = "100") long ms) {
    return Mono.delay(Duration.ofMillis(ms)).map(x -> "ok (non-blocking)");
}

// load-webflux: ANTI-PATTERN. Blocking inside a reactive handler parks an event-loop thread.
@GetMapping("/io-blocking")
Mono<String> ioBlocking(@RequestParam(defaultValue = "100") long ms) throws InterruptedException {
    Thread.sleep(ms);
    return Mono.just("ok (blocked " + ... + ")");
}
```

- `load-mvc` runs twice: `spring.threads.virtual.enabled=false` (Tomcat's default 200 platform worker threads) and `=true`. Each is a separate `@SpringBootTest(webEnvironment = RANDOM_PORT)` class, marked `@DirtiesContext` so its server is shut down before the next one is measured.
- The load generator (`LoadGenerator.java`, in both projects) is a **closed loop**: `concurrency` clients, each a virtual thread using the JDK `HttpClient` (HTTP/1.1), send requests back-to-back until the total is reached. It reports wall time (real elapsed time), throughput (requests completed per second), median (p50) and 99th-percentile (p99: 99% of requests were faster) latency (time per request), and the JVM's **peak platform thread count** (`ThreadMXBean`, which doesn't count virtual threads).
- Each scenario first runs a warm-up (JIT compilation — the JVM compiling hot code to fast machine code while running, connection setup) that is not reported.
- **Caveats:** client and server share one JVM and one laptop; `Thread.sleep`/`Mono.delay` stand in for real I/O; there's no database pool or downstream service to saturate. The numbers show the *shape* of the difference, not what your production system will do.

Results (machine-dependent)

Real output from the three runs (14-core Apple M4 Pro, JDK 26 running Java 25 bytecode — the compiled `.class` format the JVM executes):

```
MVC, spring.threads.virtual.enabled=false -> "ok on platform thread"
| mode                         | conc. |   reqs | wall ms |  req/s | p50 ms | p99 ms | peak platform threads | errors |
| MVC Platform threads         |   100 |   2000 |    2157 |    927 |    105 |    119 |                   231 |      0 |
| MVC Platform threads         |  1000 |  10000 |    5238 |   1909 |    517 |    535 |                   231 |      0 |
MVC, spring.threads.virtual.enabled=true -> "ok on virtual thread"
| mode                         | conc. |   reqs | wall ms |  req/s | p50 ms | p99 ms | peak platform threads | errors |
| MVC Virtual threads          |   100 |   2000 |    2131 |    938 |    106 |    111 |                    33 |      0 |
| MVC Virtual threads          |  1000 |  10000 |    1259 |   7942 |    108 |    346 |                    33 |      0 |
CPU cores = 14 (Reactor Netty event-loop threads default to this number)
| mode                         | conc. |   reqs | wall ms |  req/s | p50 ms | p99 ms | peak platform threads | errors |
| WebFlux non-blocking         |   100 |   2000 |    2185 |    915 |    107 |    122 |                    56 |      0 |
| WebFlux non-blocking         |  1000 |  10000 |    1241 |   8058 |    110 |    263 |                    56 |      0 |
| WebFlux + Thread.sleep (bad) |   100 |    500 |    5107 |     97 |    724 |   2479 |                    56 |      0 |
```

Reading the numbers

1. **At 100 concurrent clients, all three are the same** (~930 req/s, p50 ≈ 105 ms). The ceiling is the clients themselves: 100 clients × 10 requests/s. Below the thread limit, platform threads are perfectly fine.
2. **At 1,000 concurrent clients, platform threads hit the wall.** 200 workers × 10 req/s = 2,000 req/s is the theoretical maximum, and 1,909 was measured. The extra requests **queue**: the median latency jumps from ~105 ms to **517 ms** (≈ 5 queue rounds of 100 ms).
3. **Virtual threads remove the wall without touching the code.** 7,942 req/s at a 108 ms median, about 4× the platform-thread throughput, using **33** platform threads in the whole JVM instead of 231. Each request got its own virtual thread, and while it slept its carrier thread served others.
4. **WebFlux matches virtual threads** (8,058 req/s, p50 110 ms). The difference is noise on this setup. Both are now limited by CPU and the load generator, not by threads. (The p99 values of 263–346 ms at 1,000 clients come from CPU contention between the client and server in one JVM, plus GC pauses (moments when the garbage collector, which frees unused memory, briefly stops the program). Treat tail latencies (the slowest few percent of requests) here with care.)
5. **Blocking on an event loop is a disaster.** The same `Thread.sleep(100)` inside a WebFlux handler dropped throughput to **97 req/s** with a p99 of **2.5 s**. Each of the 14 event-loop threads can serve only one sleeping request at a time: at best 14 × 10 = 140

req/s. Connections aren't spread perfectly evenly over the loops, so the real figure is lower. And every *other* request on a blocked loop waits too, even requests that wouldn't block.

Which one should you choose?

Situation	Choose	Why
Typical CRUD/REST service on JDBC/JPA, blocking clients	**MVC + virtual threads**	same scalability for I/O-bound work, plain imperative code, all blocking libraries work, easy debugging
Streaming: SSE, WebSockets, long-lived connections, NDJSON, backpressure end-to-end	**WebFlux**	a stream is a `Flux`; backpressure and cancellation are built in
Gateways/proxies that mostly forward bytes (Spring Cloud Gateway's reactive variant)	**WebFlux**	minimal per-connection cost, non-blocking all the way
Heavy fan-out/composition: call 10 services, combine, time out, retry, race	either; **Reactor** operators are very expressive, structured concurrency (a Java API that treats a group of parallel subtasks as one unit) is the (preview, i.e. not yet final) imperative alternative	
CPU-bound work (image processing, crypto)	neither helps	you need more cores; keep it off event loops
Team unfamiliar with reactive, blocking dependencies you can't replace	**MVC + virtual threads**	a reactive stack with blocking calls inside is the worst of both

Virtual-thread caveats

- **Pinning**: before Java 24, a virtual thread blocking inside a `synchronized` block **pinned** its carrier thread and couldn't unmount. JEP 491 (a JEP is a numbered JDK Enhancement Proposal; Java 24) removed this for `synchronized`. Native calls (calls into C code) and some class-initialisation paths still pin. See Java ch15 §15.5.
- **Resources are still finite.** 10,000 virtual threads still share a JDBC pool of, say, 10 connections; the pool becomes the queue (Chapter 7). Rate-limit calls to downstream services with semaphores (counters that allow only N holders at once) or bulkheads (separate, capped resource pools per dependency, so one slow service cannot use them all).
- `ThreadLocal` **caches** (big per-thread buffers) multiply with millions of threads. Prefer scoped values (Java's newer, immutable, per-call alternative to `ThreadLocal`) or pass data explicitly.

In the interview

Q: WebFlux or virtual threads? "For I/O-bound request/response services I'd default to Spring MVC with `spring.threads.virtual.enabled=true`. In my measurements it gives the same throughput as WebFlux for a 100 ms I/O endpoint: about 8,000 req/s at 1,000 concurrent clients vs about 1,900 on 200 platform threads. And the code stays simple and debuggable. I'd pick WebFlux for streaming (SSE, WebSockets), gateways, or where end-to-end backpressure and rich async composition matter, and only if the whole call path is non-blocking. A single blocking call on an event loop cut WebFlux to ~100 req/s in the same test."

- *Do virtual threads make code faster?* → No; they make *waiting* cheap, so more requests can wait at once. CPU-bound work gets no faster.
- *What is pinning?* → A virtual thread that can't unmount from its carrier while blocked (before Java 24 inside `synchronized`; still for native frames), so it holds a platform thread.

13.16 Common mistakes — checklist

Mistake	Symptom	Fix
Blocking call (JDBC, `RestClient`, `Thread.sleep`, `block()`) on an event loop	throughput collapses, timeouts under load; `block()` throws `IllegalStateException`	non-blocking alternative, or `fromCallable(..).subscribeOn(boundedElastic())`; BlockHound in tests
Pipeline built but not returned/subscribed	nothing happens (no save, no call)	return the `Mono`/`Flux`; chain with `then`/`flatMap`
`map` with an async function	`Flux<Mono<T>>`, inner calls never run	`flatMap` / `concatMap`
`Mono.just(expensiveCall())`	runs eagerly, even when not subscribed; retry doesn't repeat it	`Mono.fromCallable` / `Mono.defer`
Relying on `ThreadLocal` (MDC, security, tx)	values missing after a thread hop	Reactor `Context`, context-propagation
Unbounded `flatMap` / `onBackpressureBuffer()`	overwhelmed downstream, memory growth	concurrency argument, bounded buffers, `limitRate`
`@Table("book")` with Spring Data R2DBC on H2	`Table "book" not found (candidates are: "BOOK")`	omit the name or match the case
Missing `spring-boot-starter-webclient` in Boot 4	no `WebClient.Builder` bean	add the starter
Virtual-time test without `thenAwait`	test hangs	advance the clock; JUnit default timeout
Mixing MVC and WebFlux starters	Boot silently starts MVC	pick one, or set `spring.main.web-application-type`

13.17 In the interview

Q: What is reactive programming and what problem does it solve? "It's programming with asynchronous streams of data and a subscriber-controlled flow rate (backpressure). In a server it means no thread waits for I/O: a handful of event-loop threads handle thousands of concurrent requests because work resumes only when data arrives. Spring's implementation is Project Reactor, with `Mono` for 0..1 and `Flux` for 0..N items. Pipelines are lazy and cold until someone subscribes; in WebFlux the framework subscribes."

- *`Mono` vs `Flux`?* → 0..1 vs 0..N items, both ending with complete or error.
- *`flatMap` vs `concatMap`?* → Concurrent and interleaved vs sequential and ordered.
- *`publishOn` vs `subscribeOn`?* → `publishOn` moves the operators after it to another scheduler; `subscribeOn` moves the subscription (the source) regardless of its position.
- *How do you call a blocking library from WebFlux?* → `Mono.fromCallable(...).subscribeOn(Schedulers.boundedElastic())`, and question whether WebFlux is the right choice.
- *What is backpressure? Give an example.* → The subscriber's `request(n)` limits what the publisher may send; `limitRate`, `onBackpressureDrop/Latest/Buffer` handle sources that can't slow down.
- *How do you test reactive code?* → `StepVerifier` (with virtual time for delays and retries), `WebTestClient` for endpoints, `@WebFluxTest` / `@DataR2dbcTest` slices.
- *Can you use JPA with WebFlux?* → Not without blocking (wrap every call on `boundedElastic`). Use R2DBC, or use MVC with virtual threads instead.
- *How do transactions work in reactive code?* → `ReactiveTransactionManager` (`R2dbcTransactionManager`); `@Transactional` on methods returning `Mono`/`Flux` or `TransactionalOperator`; the transaction lives in the Reactor `Context`, not a `ThreadLocal`.

13.18 Chapter TL;DR

- **Why**: thread-per-request wastes threads on waiting; a 200-thread Tomcat caps near 2,000 req/s for 100 ms I/O (measured 1,909). Event loops never wait. Same idea as Node's event loop.
- **Reactive Streams**: `Publisher`/`Subscriber`/`Subscription`/`Processor`; demand via `request(n)`; one terminal signal. JDK copy: `java.util.concurrent.Flow`.
- **Reactor**: `Mono` (0..1), `Flux` (0..N); lazy and cold until subscribed; `just` is eager, `defer`/`fromCallable` are lazy; don't `block()` in reactive code.
- **Operators**: `map`, `filter`, `flatMap` (concurrent, 6 s) vs `concatMap` (ordered, 12 s) vs `flatMapSequential`; `zip`, `concat`, `merge`, `switchIfEmpty`, `buffer`, `reduce`.
- **Errors** end the stream; `onErrorReturn/Resume/Map`, `retryWhen(Retry.backoff(3, 1s))` → attempts at 0/1/3/7 s, `timeout`.
- **Testing**: `StepVerifier`, virtual time (`withVirtualTime(() -> ...)` + `thenAwait`), JUnit default timeout against hangs.
- **Backpressure**: `request(n)`, `limitRate(10)` → requests 10 then 8s; overflow: DROP / LATEST / BUFFER(n) → error.

- **Schedulers**: `publishOn` (downstream) vs `subscribeOn` (source); `boundedElastic` for unavoidable blocking calls; never block event loops.
- **WebFlux**: annotated controllers returning `Mono`/`Flux`, functional `RouterFunction`s, SSE with `text/event-stream`; tests with `@WebFluxTest` + `WebTestClient` and `@SpringBootTest(RANDOM_PORT)` + `@AutoConfigureWebTestClient`. Boot 4: add `spring-boot-starter-webclient` for `WebClient`.
- **WebClient**: `retrieve`, `onStatus`, per-attempt `timeout`, filtered `retryWhen`; 5 × 300 ms calls in 322 ms with `flatMap`.
- **R2DBC**: non-blocking SQL, not an ORM; `ReactiveCrudRepository`, `DatabaseClient`, `TransactionalOperator`; `@DataR2dbcTest` doesn't roll back; mind identifier quoting.
- **Measured**: at 1,000 concurrent clients, MVC platform threads 1,909 req/s (p50 517 ms), MVC **virtual threads 7,942 req/s**, **WebFlux 8,058 req/s**, WebFlux with a blocking call **97 req/s**. Default to MVC + virtual threads for I/O-bound CRUD; choose WebFlux for streaming, gateways and end-to-end non-blocking pipelines.
- Next: Chapter 14 — Messaging, async and scheduling.

Chapter 14 — Messaging, Async and Scheduling

So far every piece of work in this book happened **inside the request**: the controller calls a service, the service calls the database, and the HTTP response waits for all of it. Real systems also need work that happens **later**, **elsewhere** or **on a timer**:

- "After sign-up, send a welcome e-mail, but don't make the user wait for the SMTP server." (SMTP is the protocol for sending e-mail.)
- "Every night at 02:00, expire unpaid orders."
- "When an order is placed, tell the shipping, billing and analytics services. They run in other processes and may be down right now." (A **process** is one running program with its own memory, such as one JVM.)

Spring gives you one tool for each layer: **application events** (in-process notifications), `@Async` (run a method on another **thread**, i.e. a separate path of execution), `@Scheduled` (run a method on a timer), and **messaging** with Kafka or RabbitMQ (two popular **message brokers**, servers that store messages between programs; between processes, **durable** = kept safely on disk so they survive a crash). See System Design Fundamentals for queues and brokers in general. The chapter ends with the **transactional outbox**, the pattern that joins a database **transaction** (a group of changes that commit together or not at all) and a message broker without losing messages.

What you'll learn

- Synchronous vs `@Async` event listeners, and every `@TransactionalEventListener` phase, proved with tests
- `@Async`: how it works, return types, Boot's auto-configured executor, named executors, `TaskDecorator`, virtual threads, and what happens to exceptions
- `@Scheduled`: `fixedRate` vs `fixedDelay` vs `cron`, the one-thread default, overlap, and running jobs in a cluster
- Messaging basics: queues vs logs, delivery guarantees, ordering, idempotence
- Kafka with Spring for Apache Kafka: producer, `@KafkaListener`, keys and partitions, consumer groups, JSON, `DefaultErrorHandler` with a dead-letter topic, blocking and non-blocking retries, idempotent and transactional producers, and testing with `@EmbeddedKafka`
- RabbitMQ concepts and configuration (not run: it needs a broker)
- The transactional outbox with a poller, an outage, a duplicate and an idempotent consumer

The code (Spring Boot 4.1.1, Spring Framework 7.0, Spring for Apache Kafka 4.1.1, Kafka clients 4.2.1, Java 25):

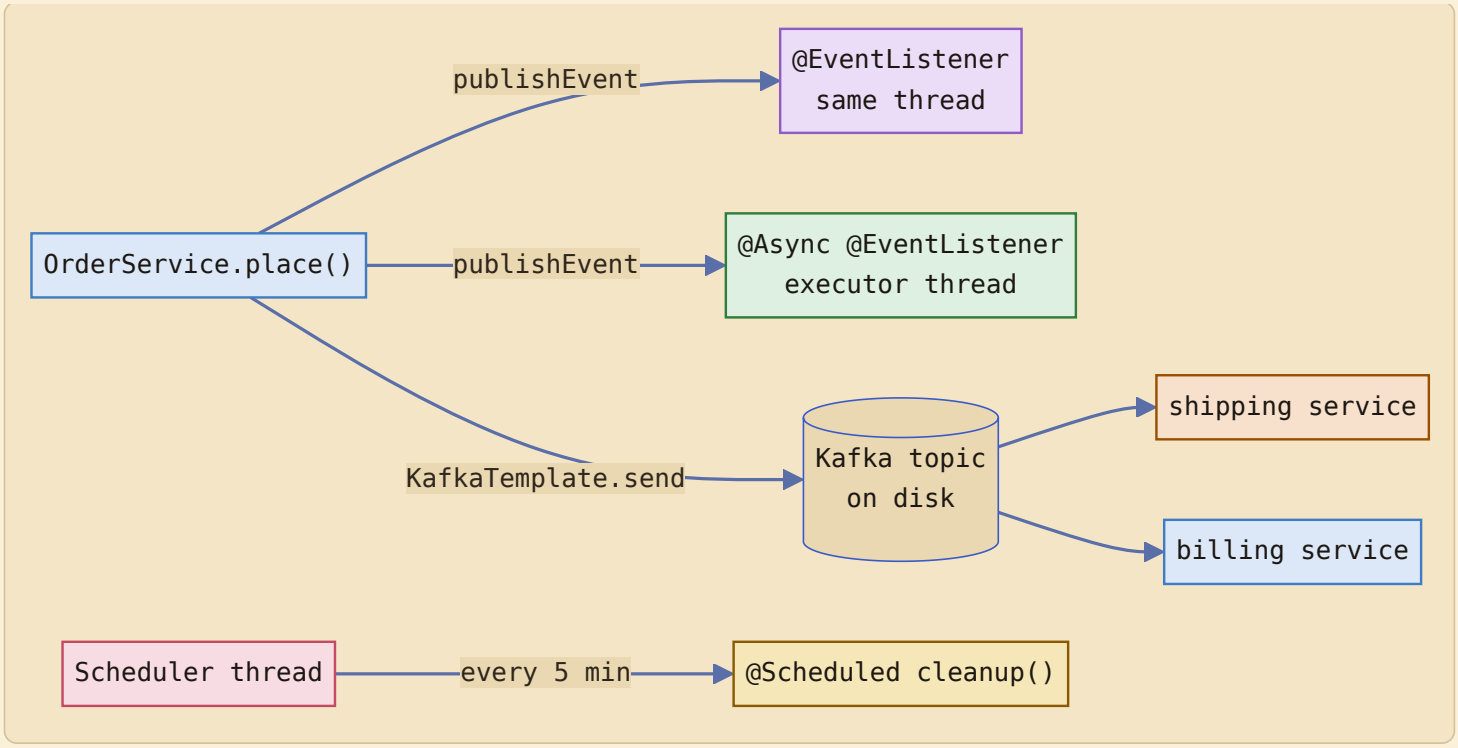
Project	Needs	What it proves
`code/events-async`	H2 in memory (a small Java database that lives inside the JVM)	sync vs async listeners, transaction phases, `@Async` executors and thread names, MDC propagation, virtual threads, async exceptions
`code/scheduling`	nothing	`fixedRate` vs `fixedDelay` vs `cron` timing, turning jobs off in tests, overlap on platform vs virtual threads
`code/kafka-orders`	nothing: an embedded Kafka broker runs inside the test JVM	keys → partitions, consumer groups, JSON on the wire, retries + DLT, `@RetryableTopic`, idempotence, transactions
`code/outbox`	H2 in memory	atomic order + outbox row, polling publisher, broker outage, duplicate delivery, idempotent consumer
`code/rabbit-orders`	a RabbitMQ broker (Docker)	exchange/queue/binding/DLX configuration; **compiles, not run**

Every output below was printed by a test and checked by an assertion. Run everything with `bash tools_run_spring.sh spring '14-*'` from the repository root (the top folder of the project's Git **repository**). No Docker (a tool that runs software in isolated containers) is needed: Kafka runs as `@EmbeddedKafka`, a real broker started inside the test JVM.

14.1 The map: four ways to do work "somewhere else"

What it is

Tool	Where the work runs	Survives a crash?	Typical use
Application event (`publishEvent` + `@EventListener`)	same JVM, same thread by default	no, it lives in memory	decoupling modules (letting them work together without referring to each other directly) inside one app
`@Async` method	same JVM, another thread	no	"don't make the caller wait": e-mails, reports, fan-out calls
`@Scheduled` method	same JVM, a scheduler thread	the schedule restarts with the app, a run in progress is lost	housekeeping, polling (checking repeatedly whether there is new work), periodic reports
Message broker (Kafka, RabbitMQ)	another process, maybe another service	yes: the broker stores the message on disk	integration between services, buffering load, event streaming (a continuous flow of "something happened" records)



How to choose

1. **Only this app cares, and losing the work in a crash is acceptable?** Use an application event. Add `@Async` if the listener is slow.
2. **Must happen at a time or interval?** Use `@Scheduled`. If the app runs on several instances, add a lock (14.7).
3. **Another service must react, or the work must not be lost?** Use a broker. If the message must match a database change exactly, use the **transactional outbox** (14.17).

The patterns behind this map (message queue, publish-subscribe, event-driven architecture) are explained without code in System-design patterns, Module 2 — Moving data and in the event-driven architecture notes. This chapter is about the Spring side.

14.2 Application events: synchronous vs `@Async` listeners

What it is

Chapter 2 introduced the container's built-in **publish/subscribe**: any bean publishes an object with `ApplicationEventPublisher.publishEvent(...)`, and every `@EventListener` method whose parameter type matches receives it (2.12). The key fact from that section: **listeners run synchronously, on the publisher's thread, before `publishEvent` returns.**

Put `@Async` on a listener and it changes character. The publisher hands the event to an **executor** (an object that owns a set of worker threads and a queue of tasks, §14.4) and continues at once. Think of the difference between handing a letter to a colleague and waiting while they read it, versus dropping it in their in-tray.

How it works

1. `@EnableAsync` on a configuration class installs a post-processor (a hook that can modify or wrap each bean after Spring creates it). It wraps every bean that has `@Async` methods in a **proxy** (ch04; a stand-in object that intercepts calls before passing them to the real one).
2. `publishEvent` finds the matching listeners and calls them in order.
3. For a plain listener, the call runs the method right away.
4. For an `@Async` listener, the call goes through the proxy. The proxy submits the method to an executor and returns immediately.

```java
@SpringBootApplication
@EnableAsync   // turns on the proxy that makes @Async methods run on an executor
public class AsyncApplication { ... }

public record UserSignedUp(String email) {}

@Service
public class SignupService {
    // constructor injection of ApplicationEventPublisher events, Trace trace ...
    public void signUp(String email) {
        trace.add("publisher: publishing on " + Trace.thread());
        events.publishEvent(new UserSignedUp(email));
        trace.add("publisher: publishEvent returned on " + Trace.thread());
    }
}

@Component
public class SignupListeners {
    @EventListener                      // synchronous: runs inside publishEvent, same thread
    void createProfile(UserSignedUp e) {
        trace.add("  sync listener: profile for " + e.email() + " on " + Trace.thread());
    }

    @Async                              // asynchronous: handed to the executor, publisher does not wait
    @EventListener
    void sendWelcomeMail(UserSignedUp e) throws InterruptedException {
        Thread.sleep(200);              // pretend the SMTP call is slow
        trace.add("  async listener: welcome mail to " + e.email() + " on " + Trace.thread());
    }
}
```

`Trace` is a small thread-safe (safe to use from several threads at once) list (a `CopyOnWriteArrayList`) that the tests print. The test calls `signUp`, prints the trace at once, then waits with **Awaitility**, a test library that polls a condition until it is true or a timeout expires:

```java
signups.signUp("ana@example.com");
trace.lines().forEach(System.out::println);
assertThat(trace.lines()).hasSize(3);                    // the async listener has not finished yet
await().atMost(Duration.ofSeconds(2)).until(() -> trace.lines().size() == 4);
```

Output:

```
--- right after signUp() returned ---
publisher: publishing on main
  sync listener: profile for ana@example.com on main
publisher: publishEvent returned on main
--- 200 ms later ---
  async listener: welcome mail to ana@example.com on task-1
```

The synchronous listener ran on `main` (the test thread) between the two publisher lines. The async listener ran 200 ms later on `task-1`, a thread of Boot's auto-configured executor (14.4).

Trade-offs

	Sync listener	`@Async` listener
Publisher waits?	yes	no
Exception reaches the publisher?	yes, and it can roll back the publisher's transaction	no: it goes to the `AsyncUncaughtExceptionHandler` (14.6)
Runs in the publisher's transaction?	yes	no: a new thread has no transaction
Return value published as a new event?	yes	no
`@Order` between listeners	respected	only the hand-off order is ordered; the executions race (run at the same time, finishing in an unpredictable order)

Common mistakes

- **Forgetting `@EnableAsync`.** Boot auto-configures an executor but does **not** enable `@Async`. Without the annotation, `@Async` is silently ignored and the listener runs synchronously.
- **Expecting ThreadLocal state on the async thread** (a `ThreadLocal` holds a separate value per thread): the current transaction, the Spring Security context, the logging MDC (per-thread values such as a request id that appear in every log line, explained in §14.4). None of it moves across threads unless you copy it (14.4 shows a `TaskDecorator`).
- **Treating async events as reliable.** They live in memory. If the JVM stops before the listener runs, the work is gone.

TL;DR

`@EventListener` = same thread, publisher waits. `@Async @EventListener` = executor thread, publisher continues, exceptions and transactions do not cross over. Remember `@EnableAsync`.

14.3 `@TransactionalEventListener`: events tied to a commit

What it adds to 7.15

7.15 explains the four phases (`BEFORE_COMMIT`, `AFTER_COMMIT` (the default), `AFTER_ROLLBACK`, `AFTER_COMPLETION`) and how Spring registers the listener with the transaction as a **synchronization** (a callback the transaction manager calls around commit and rollback). Here we add two things: what each phase can *see* in the database, and the combination most production code uses, `@Async` + `AFTER_COMMIT`.

```
@Transactional
public void place(String item, boolean paymentFails) {
    jdbc.sql("INSERT INTO orders(item) VALUES (?)").param(item).update();
    events.publishEvent(new OrderPlaced(item));
    trace.add("service: event published inside the transaction");
    if (paymentFails) throw new IllegalStateException("payment declined");
}

@Component
public class OrderListeners {
    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
    void beforeCommit(OrderPlaced e) { /* counts the order row: visible, same transaction */ }

    @TransactionalEventListener   // phase = AFTER_COMMIT is the default
    void afterCommit(OrderPlaced e) { ... }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)
    void afterRollback(OrderPlaced e) { /* counts the order row again: gone */ }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMPLETION)
    void afterCompletion(OrderPlaced e) { ... }

    /** The common production combo: only after commit, off the request thread, in its own transaction. */
    @Async
    @TransactionalEventListener
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    void emailAfterCommit(OrderPlaced e) {
        jdbc.sql("INSERT INTO email_log(message) VALUES (?)").param("confirmation for " + e.item()).update();
        trace.add("async AFTER_COMMIT email for " + e.item() + " on " + Trace.thread());
    }
}
```

Output (`EventsTest`):

```
--- commit ---
service: event published inside the transaction
BEFORE_COMMIT    book (row visible in tx: 1)
AFTER_COMMIT     book on main
AFTER_COMPLETION book
async AFTER_COMMIT email for book on task-2
--- rollback ---
service: event published inside the transaction
AFTER_ROLLBACK   lamp (rows now: 0)
AFTER_COMPLETION lamp
```

What the test proved:

- On **commit**, `BEFORE_COMMIT` still saw the uncommitted row (it runs inside the transaction). Then `AFTER_COMMIT` and `AFTER_COMPLETION` ran on `main`, and the async e-mail ran on `task-2` and wrote to `email_log` in its own transaction.
- On **rollback**, no after-commit listener ran and **no e-mail was written**. The test waited 300 ms and asserted `email_log` had zero rows for `lamp`. `AFTER_ROLLBACK` saw that the order row was gone.

Why `REQUIRES_NEW` on the async listener

Since Framework 6.1, a `@TransactionalEventListener` that is also `@Transactional` must use `REQUIRES_NEW` (always start a fresh, separate transaction) or `NOT_SUPPORTED` (run with no transaction), or the context fails to start (details in 7.15). With `@Async` the method runs on another thread anyway, so `REQUIRES_NEW` just states the intent: the e-mail log gets its own transaction.

When not to use it

When the side effect **must** happen. `AFTER_COMMIT` + `@Async` is still an in-memory hand-off. If the JVM dies between the commit and the e-mail, the e-mail is lost and nothing will retry it. For "must not be lost", write an outbox row in the same transaction (14.17).

In the interview

> **30-second answer:** see 7.15. Add: "With `@Async` the side effect also leaves the request thread and gets its own transaction (`REQUIRES_NEW`). It is still an in-memory hand-off, so critical messages go through an outbox."

- *What if no transaction is active?* → Transactional listeners are skipped, unless `fallbackExecution = true`.
- *Can a `BEFORE_COMMIT` listener veto (block) the commit?* → Yes. An exception there propagates and the transaction rolls back.

Self-check

1. Which listeners ran for the rolled-back `lamp` order? → `AFTER_ROLLBACK` and `AFTER_COMPLETION` only.
2. Why could `BEFORE_COMMIT` count the new row? → It runs inside the same, still open, transaction.
3. Why is `AFTER_COMMIT` + `@Async` not "guaranteed delivery"? → The event exists only in memory. A crash after the commit loses it.

14.4 `@Async` and executors

What it is

`@Async` on a public (or at least non-private, non-final) method of a Spring bean means "run this on another thread and return to the caller at once". An **executor** is the object that owns the worker threads and a queue of waiting tasks, the same `Executor` idea as in Java ch14 §14.11. Spring decides **which** executor to use, wraps the call, and adapts the return value.

How it works

1. The caller calls `reports.build("daily")`. `reports` is really the proxy.
2. The proxy's `AsyncExecutionInterceptor` picks an executor:
   - the one named in `@Async("name")`, if given;
   - else the one returned by an `AsyncConfigurer`;
   - else a unique `TaskExecutor` bean, or the bean named `taskExecutor`. With Boot this is the auto-configured `applicationTaskExecutor`.
3. It submits the real method call as a task and returns right away:
   - `void` → returns nothing. An exception goes to the `AsyncUncaughtExceptionHandler`.
   - `CompletableFuture<T>` (or `Future<T>`) → returns a **future** (a placeholder object for a result that will exist later) that completes when the task finishes.
4. Inside the method you return `CompletableFuture.completedFuture(value)`. Spring unwraps it and completes the caller's future.

Boot's auto-configured executor

If you define **no** `Executor` bean, Boot creates one called `applicationTaskExecutor`:

Setting	Default	Property
type	`ThreadPoolTaskExecutor`; `SimpleAsyncTaskExecutor` if virtual threads are on	`spring.threads.virtual.enabled`
core threads	8	`spring.task.execution.pool.core-size`
queue capacity	unbounded	`spring.task.execution.pool.queue-capacity`
max threads	unlimited, but only used when the queue is full, so with an unbounded queue there are never more than core threads	`spring.task.execution.pool.max-size`
thread names	`task-1`, `task-2`, ...	`spring.task.execution.thread-name-prefix`
wait for tasks on shutdown	no	`spring.task.execution.shutdown.await-termination(-period)`

(This chapter's example sets `core-size=4`.)

> **Watch out:** if you declare *any* `Executor` bean (for example a pool for reports), Boot's `applicationTaskExecutor` backs off. Unless you do something about it, `@Async`, Spring MVC async requests and other integrations then fall back to other defaults. Two fixes: declare the extra pool with `@Bean(defaultCandidate = false)`, which the Boot docs show for exactly this case, or set `spring.task.execution.mode=force` (since Boot 3.5) so the auto-configured executor is created anyway.

A dedicated pool, a TaskDecorator, and the self-call trap

```
@Configuration(proxyBeanMethods = false)
public class AsyncConfig {

    /** A second, dedicated pool for slow report jobs. defaultCandidate = false keeps it out of
     *  by-type injection, so Boot still auto-configures the normal applicationTaskExecutor. */
    @Bean(defaultCandidate = false)
    @Qualifier("reportExecutor")
    ThreadPoolTaskExecutor reportExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setThreadNamePrefix("report-");
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(2);
        executor.setQueueCapacity(10);          // bounded: an 11th waiting job is rejected
        return executor;
    }

    /** Boot applies a TaskDecorator bean to the executor it auto-configures: copy the MDC across threads. */
    @Bean
    TaskDecorator mdcCopyingDecorator() {
        return runnable -> {
            Map<String, String> callerMdc = MDC.getCopyOfContextMap();   // runs on the caller thread
            return () -> {                                               // runs on the worker thread
                if (callerMdc != null) MDC.setContextMap(callerMdc);
                try { runnable.run(); } finally { MDC.clear(); }
            };
        };
    }
}

@Service
public class ReportService {
    @Async   // default executor: Boot's applicationTaskExecutor
    public CompletableFuture<String> build(String name) {
        return CompletableFuture.completedFuture(name + " built on " + Trace.thread()
                + " requestId=" + MDC.get("requestId"));
    }

    @Async("reportExecutor")   // pick a specific executor bean by name / qualifier
    public CompletableFuture<String> buildBig(String name) { /* same body */ }

    /** Self-invocation: the call below does not go through the proxy, so it is NOT asynchronous. */
    public String buildViaThis(String name) {
        return this.build(name).join();
    }
}
```

The **MDC** (Mapped Diagnostic Context) is SLF4J's (the standard Java logging facade) per-thread map of values, such as a request id, that every log line can include. It is a `ThreadLocal`, so a new thread starts with an empty one. A **`TaskDecorator`** wraps each submitted task. It reads the caller's values while still on the caller thread and restores them on the worker thread.

The test puts `requestId=r-42` into the MDC and calls the three methods:

Output (`AsyncMethodsTest`):

```
auto-configured executor: ThreadPoolTaskExecutor
daily built on task-1 requestId=r-42
yearly built on report-1 requestId=null
self-call built on main requestId=r-42
```

- `daily` ran on Boot's pool (`task-1`), and the decorator carried `r-42` across.
- `yearly` ran on the dedicated pool (`report-1`). It has no decorator because we built it by hand, so the MDC was empty. If you want it, call `executor.setTaskDecorator(...)` yourself.
- `self-call` ran on `main`. `this.build(...)` skips the proxy, so `@Async` did nothing. This is the same self-invocation rule as for `@Transactional` (4.5).

Boot 4.1 also has `spring.task.execution.propagate-context=true`. It registers Spring's `ContextPropagatingTaskDecorator`, which copies the thread-locals that are registered with Micrometer's context-propagation library (tracing, for example) instead of a hand-written decorator.

Sizing a pool

Workload	Pool shape
CPU-bound (image resizing, PDF rendering)	about as many threads as cores. More threads only add context switching (the CPU's cost of swapping one thread out for another)
I/O-bound (HTTP calls, SMTP, database)	many threads, or virtual threads (14.5)
Must not pile up in memory	a **bounded** queue plus a rejection policy. The default `AbortPolicy` throws `TaskRejectedException` to the caller; `CallerRunsPolicy` runs the task on the caller's thread, which slows the producer down
Isolate slow jobs from fast ones	separate pools (a **bulkhead**, see microservices notes)

Common mistakes

- **Self-invocation, `private` or `final` methods**: no proxy, so no async. Move the method to another bean.
- **An unbounded queue in front of a slow dependency.** Tasks accumulate until the heap (the JVM's memory area for objects) is full. Set `queue-capacity`.
- **Declaring your own `Executor` bean and wondering why MVC async or `@Async` changed pools.** Boot's executor backed off. Use `defaultCandidate = false` or `mode=force`.
- **Calling `.join()` right after the async call.** The caller waits anyway, and you only added a thread hop. Start several futures first, then join them all.
- **Losing work on shutdown.** Tasks still queued when the context closes are dropped unless `shutdown.await-termination=true`.

Real-world examples

- An online shop sends the confirmation e-mail and pushes the order to an analytics service through `@Async`, so checkout responds in 80 ms instead of 600 ms.
- A dashboard backend fires three `@Async` calls to three internal services, then combines them with `CompletableFuture.allOf(...)`. The page waits for the slowest one instead of the sum of all three (Java 15.2 — CompletableFuture).

TL;DR

`@EnableAsync` + `@Async` = proxy + executor. Boot auto-configures `applicationTaskExecutor` (8 threads, unbounded queue, `task-` names). Choose a pool with `@Async("name")`. Declare extra pools with `defaultCandidate = false`. Copy ThreadLocals such as the MDC with a `TaskDecorator`. Never self-invoke.

14.5 `@Async` on virtual threads

What it is

A **virtual thread** (final in Java 21) is a thread managed by the JVM rather than the operating system. It costs a few hundred bytes instead of about a megabyte of stack, so you can have hundreds of thousands. When it blocks on I/O (waits for network or disk), the JVM parks it and reuses the underlying carrier thread (the real OS thread it was running on). Java's side of the story is in Java 15.4 — Virtual threads.

With `spring.threads.virtual.enabled=true` (Boot 3.2+, needs Java 21+), Boot switches its executors to virtual threads. The same property also moves Tomcat request threads, `@Scheduled` and the Kafka and RabbitMQ listener containers to virtual threads.

```
@SpringBootTest(properties = "spring.threads.virtual.enabled=true")
class VirtualThreadsTest {
    @Autowired ReportService reports;
    @Autowired AsyncTaskExecutor applicationTaskExecutor;

    @Test
    void asyncMethodsRunOnVirtualThreads() {
        System.out.println("auto-configured executor: " + applicationTaskExecutor.getClass().getSimpleName());
        List<CompletableFuture<String>> futures = IntStream.rangeClosed(1, 3)
                .mapToObj(i -> reports.build("report-" + i)).toList();
        futures.forEach(f -> System.out.println(f.join()));
        assertThat(futures).allSatisfy(f -> assertThat(f.join()).contains("on virtual task-"));
    }
}
```

Output:

```
auto-configured executor: SimpleAsyncTaskExecutor
report-1 built on virtual task-1 requestId=null
report-2 built on virtual task-2 requestId=null
report-3 built on virtual task-3 requestId=null
```

`Trace.thread()` prints `virtual` when `Thread.isVirtual()` is true. The executor is now a `SimpleAsyncTaskExecutor`: **no pool and no queue**, one new virtual thread per task. Pool settings such as `core-size` are ignored.

Trade-offs

	Platform-thread pool	Virtual threads
Concurrency limit	the pool size, a natural throttle	none by default: 10,000 submitted tasks means 10,000 concurrent tasks
Cost per blocked task	a whole OS thread	tiny
Best for	CPU-bound work, or when you want a hard cap	I/O-bound work: HTTP calls, JDBC, SMTP
Limit concurrency with	pool size + queue	`spring.task.execution.simple.concurrency-limit`, or Framework 7's `@ConcurrencyLimit` (4.7)

Common mistakes

- **Flooding a downstream system.** Without a pool there is no throttle. If each task opens a database connection, the connection pool (the fixed set of reusable database connections; 10 by default in HikariCP, Boot's default pool) becomes the real limit and tasks wait there instead. Set a concurrency limit on purpose.
- **Expecting CPU-bound work to get faster.** Virtual threads help with waiting, not with computing.
- **Pinning.** Before JDK 24, a virtual thread blocked inside `synchronized` pinned its carrier thread. JEP 491 (JDK 24) removed that for `synchronized`. Native calls can still pin.
- **An app whose only work is scheduled jobs exits at startup.** Virtual threads are daemon threads (background threads that do not keep the JVM alive), so the JVM may see "no non-daemon threads" and stop. Set `spring.main.keep-alive=true`.

TL;DR

`spring.threads.virtual.enabled=true` → `@Async` runs on a new virtual thread per call (`SimpleAsyncTaskExecutor`), with no pool and no queue. Great for I/O. Add a concurrency limit yourself.

14.6 Exceptions in async methods

The problem it solves

In a normal call an exception travels up the caller's stack. With `@Async` the caller has already returned, so **where does the exception go?** The answer depends on the return type.

Return type	Where the exception goes	How to see it
`CompletableFuture<T>` / `Future<T>`	stored in the future, which completes exceptionally	`get()` throws `ExecutionException`, `join()` throws `CompletionException`, or use `exceptionally` / `handle` / `whenComplete`
`void`	nowhere near the caller	the `AsyncUncaughtExceptionHandler`. The default one only **logs** the error

```
@Async
public CompletableFuture<String> buildBroken(String name) {
    throw new IllegalStateException("no data for " + name);   // becomes an exceptionally completed future
}

@Async
public void fireAndForget(String name) {
    throw new IllegalStateException("cannot export " + name);  // void: goes to AsyncUncaughtExceptionHandler
}

/** Only overrides the exception handler, so @Async keeps using Boot's executor.
 *  Keep AsyncConfigurer in its own class with no @Bean methods: Boot 4 wraps AsyncConfigurer beans. */
@Configuration(proxyBeanMethods = false)
public class AsyncErrorConfig implements AsyncConfigurer {
    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
                trace.add("uncaught handler: " + method.getName() + " threw " + ex.getMessage() + " on " + Trace.thread());
    }
}
```

The test:

```
CompletableFuture<String> future = reports.buildBroken("march");
assertThatThrownBy(future::get).isInstanceOf(ExecutionException.class)
        .hasCauseInstanceOf(IllegalStateException.class);
String viaHandle = future.handle((ok, ex) -> "recovered from: " + ex.getMessage()).join();

reports.fireAndForget("april");                  // returns at once, no exception for the caller
await().atMost(Duration.ofSeconds(2)).until(() -> !trace.lines().isEmpty());
```

Output:

```
CompletableFuture: recovered from: java.lang.IllegalStateException: no data for march
void method:       uncaught handler: fireAndForget threw cannot export april on task-4
```

Notice that the message inside `handle` starts with `java.lang.IllegalStateException:`. The exception passed to `handle` was a `CompletionException` wrapping the original, and its message includes the cause. Use `ex.getCause()` when you need the original.

> **Found while building this chapter:** with `getAsyncUncaughtExceptionHandler()` **and** `@Bean` methods in one `AsyncConfigurer` class, Boot 4 failed at startup (`Illegal factory instance for factory method 'mdcCopyingDecorator'`), because it wraps user `AsyncConfigurer` beans. Hence two classes.

Common mistakes

- `void` **+ no handler = silent failure in production.** The default handler logs at ERROR level and moves on. Nobody retries. Return a `CompletableFuture`, or install a handler that records a metric and alerts.
- **Catching the exception inside the async method and returning** `null`**.** The caller can no longer tell success from failure.
- **Expecting** `@Transactional` **rollback in the caller** because an async method failed. It runs in another thread and another transaction (or none).

Self-check

1. `@Async void` throws. Does the caller see it? → No. It goes to the `AsyncUncaughtExceptionHandler`.
2. `@Async CompletableFuture` throws. What does `get()` throw? → `ExecutionException` with the original as its cause.
3. Why is the handler in its own class? → Boot 4 wraps `AsyncConfigurer` beans, which breaks `@Bean` methods declared in the same class.

14.7 Scheduling with `@Scheduled`

What it is

`@Scheduled` makes the container call a method on a timer: every N milliseconds, N milliseconds after the previous run finished, or at calendar times given by a **cron expression** (a compact text format, borrowed from the Unix `cron` tool, that describes repeating times). Like `@Async`, it needs a switch: `@EnableScheduling`. Boot then auto-configures a `ThreadPoolTaskScheduler` with **one thread** named `scheduling-1`.

How it works

1. `@EnableScheduling` registers a post-processor that finds `@Scheduled` methods (no parameters, return value ignored).
2. For each one it registers a trigger with the `TaskScheduler`.
3. The scheduler thread wakes up at the trigger time and runs the method.

Attribute	Meaning	Example
`fixedRate = 300`	start every 300 ms, measured **start to start**	heartbeat (a regular "I'm alive" signal), metrics push
`fixedDelay = 300`	wait 300 ms **after the previous run ends**	polling a table or an API: never overlaps, never floods
`initialDelay = 50`	wait before the first run	let the app warm up
`cron = "0 0 2 * * *"`	calendar-based, 6 fields	nightly jobs
`zone = "Europe/Berlin"`	time zone for `cron` (default: the server's zone)	daylight-saving-safe jobs (clocks jump an hour in spring and autumn)
`timeUnit = TimeUnit.SECONDS`	unit for rate/delay (default ms)	`fixedDelay = 30, timeUnit = SECONDS`
`fixedDelayString = "${outbox.poller.delay-ms}"`	value from configuration	tunable per environment

Spring's cron has **six** fields, one more than Unix cron, because it starts with **seconds**:

Field	Values	`0 30 2 * * MON-FRI` means
second	0–59	at second 0
minute	0–59	minute 30
hour	0–23	02:xx
day of month	1–31, `L` = last day	any
month	1–12 or JAN–DEC	any
day of week	0–7 (0 and 7 = Sunday) or MON–SUN	Monday to Friday

Macros exist too: `@hourly`, `@daily` (also `@midnight`), `@weekly`, `@monthly`, `@yearly` (also `@annually`). The special value `"-"` disables a cron trigger, which is handy with a property: `cron = "${jobs.cleanup.cron:-}"`.

Proving fixedRate vs fixedDelay vs cron

Both jobs pretend to work for 200 ms and have a 300 ms period. The test records every start time and prints the gaps:

```
@Scheduled(fixedRate = 300)          // measured from START to START
void fixedRateJob() { rateRuns.add(now()); work(200); }

@Scheduled(fixedDelay = 300, initialDelay = 50)   // measured from END to next START
void fixedDelayJob() { delayRuns.add(now()); work(200); }

@Scheduled(cron = "*/1 * * * * *", zone = "UTC")  // every second, on the second
void cronJob() { cronRuns.add(LocalTime.now().withNano(0) + " on " + Thread.currentThread().getName()); }
```

```
# default is ONE scheduler thread shared by every @Scheduled method; give each job its own
spring.task.scheduling.pool.size=3
```

```
await().atMost(Duration.ofSeconds(5))
       .until(() -> jobs.rateRuns.size() >= 6 && jobs.delayRuns.size() >= 4 && jobs.cronRuns.size() >= 2);
assertThat(median(rateGaps)).isBetween(270L, 360L);    // about 300 = the period
assertThat(median(delayGaps)).isBetween(470L, 600L);   // about 500 = 200 work + 300 delay
```

Output (`ScheduledJobsTest`; the millisecond gaps and clock times vary slightly from run to run):

```
fixedRate  = 300 ms, job takes 200 ms -> gaps between starts [300, 300, 300, 300] on scheduling-1
fixedDelay = 300 ms, job takes 200 ms -> gaps between starts [500, 510, 510] on scheduling-2
cron */1 * * * * *                    -> [19:48:15 on scheduling-1, 19:48:16 on scheduling-3]
```

`fixedRate` started every 300 ms whatever the job took. `fixedDelay` started every 500 ms, which is 200 ms of work plus the 300 ms pause. The cron job fired exactly on whole seconds, on whichever of the three scheduler threads was free. (The zone only affects how the cron expression is evaluated. The printed time is the local clock.)

The one-thread default, and overlap

With the default `pool.size=1`, **all** `@Scheduled` methods share one thread. A 10-minute report job delays every other job, including the one-second heartbeat. That is why the example raises the pool size.

What if a fixed-rate job is **slower than its period**? The answer depends on the scheduler. `SlowJob` does 250 ms of work every 100 ms and records how many runs were active at the same time:

```
@Scheduled(fixedRate = 100)
void tick() throws InterruptedException {
    int now = running.incrementAndGet();
    maxRunningAtOnce.accumulateAndGet(now, Math::max);
    ...
    try { Thread.sleep(250); } finally { running.decrementAndGet(); }
}
```

Output (`OverlapTest`, 1.2 seconds each; run counts vary by one or two between runs):

```
ThreadPoolTaskScheduler: runs=5 maxRunningAtOnce=1 (scheduling-2)
SimpleAsyncTaskScheduler: runs=13 maxRunningAtOnce=3 (virtual thread scheduling-22)
```

- **Platform threads** (`ThreadPoolTaskScheduler`): runs of the same method never overlap. Late runs are started as soon as the previous one ends, so only 5 runs happened.
- **Virtual threads** (`SimpleAsyncTaskScheduler`, Framework 6.1+): every run gets its own new thread, and the scheduler does not wait for the previous run to finish. Three runs overlapped. The Framework docs note that this scheduler only tracks the hand-off, and that `fixedDelay` tasks are forced onto a single scheduler thread to keep their meaning.

If a job must never overlap itself, use `fixedDelay`, or guard it (an `AtomicBoolean`, or a lock).

Turning jobs off in tests

Background jobs make tests **flaky** (sometimes passing, sometimes failing, with no code change): they touch data at random moments. Put `@EnableScheduling` on its own configuration class behind a property (`@ConditionalOnBooleanProperty` is new in Boot 3.5):

```
@Configuration(proxyBeanMethods = false)
@EnableScheduling
@ConditionalOnBooleanProperty(name = "app.jobs.enabled", matchIfMissing = true)
public class SchedulingConfig {}
```

```
app.jobs.enabled=false -> runs after 700 ms: 0
```

Then test the job's **method** directly, as the outbox test in 14.17 does with `poller.publishBatch()`, and keep one small test with scheduling on and Awaitility.

Running in a cluster

`@Scheduled` knows nothing about other instances (running copies of the same app). With three pods (a **pod** is Kubernetes' unit for one running app instance; Kubernetes is the common platform for running containers), the 02:00 job runs **three times**. Options:

Option	How
ShedLock (a library)	`@SchedulerLock(name = "cleanup")` takes a lock row in a shared database table. Only one instance runs the job
Leader election (the instances agree that exactly one of them is "leader" and does the job)	Spring Integration `LockRegistryLeaderInitiator`, Kubernetes leader election
Move the job out	a Kubernetes `CronJob`, or Quartz (a Java job-scheduling library) in clustered JDBC mode for persistent, missed-fire-aware schedules
Make the job idempotent and safe to run concurrently	`UPDATE ... WHERE status = 'PENDING'` claims rows, or `SELECT ... FOR UPDATE SKIP LOCKED` (14.17)

Common mistakes

- **Forgetting `@EnableScheduling`.** Nothing runs, and nothing complains.
- **A long job on the single default thread**, which blocks every other job.
- **Five-field Unix cron** such as `"0 2 * * *"`. Spring needs six fields and rejects the expression at startup.
- **`fixedRate` for polling a slow resource** on virtual threads. Runs pile up. Use `fixedDelay`.
- **Relying on server time zones.** Set `zone`, because 02:30 happens twice or not at all on daylight-saving days.
- **Ignoring clusters.** N instances means N runs.

In the interview

30-second answer: "`@Scheduled` needs `@EnableScheduling`. `fixedRate` is start-to-start, `fixedDelay` is end-to-start, and cron has six fields starting with seconds. Boot's default scheduler has one thread, so I raise `spring.task.scheduling.pool.size` or jobs block each other. On platform threads a slow fixed-rate job doesn't overlap itself, but on virtual threads it can. In a cluster every instance runs the job, so I use ShedLock or make the job idempotent."

- *How do you test a scheduled job?* → Call the method directly with scheduling switched off. Add one integration test with scheduling on, using Awaitility with a timeout, never `Thread.sleep` plus an exact count.
- *fixedRate or fixedDelay for polling an outbox?* → `fixedDelay`: the next poll starts only after the previous one finished.

TL;DR

`@EnableScheduling` + `@Scheduled(fixedRate | fixedDelay | cron)`. Six-field cron with seconds. One scheduler thread by default. Overlap depends on the scheduler. One run per instance, so lock or be idempotent in a cluster.

14.8 Messaging concepts you need first

What it is

A **message broker** is a server that accepts messages from **producers**, stores them, and hands them to **consumers**. Producer and consumer never talk directly, so either can be slow, down, or deployed at a different time. Think of a post office. You drop the parcel off and leave. The recipient collects it when they are home.

Two broker designs dominate:

	Queue broker (RabbitMQ, ActiveMQ, Amazon SQS)	**Log broker (Apache Kafka, Redpanda, Amazon Kinesis)**
Storage	a queue per consumer group of interest. A message is **deleted** once acknowledged ("acked": the consumer confirms it has handled it)	an append-only **log**. Messages stay for a retention period (e.g. 7 days) whether read or not
Reading	the broker pushes messages and tracks which are acknowledged	each consumer group tracks its own **offset** (position) and pulls
Replay old messages	no, they are gone after the ack	yes: reset the offset
Routing	rich: exchanges, routing keys, patterns, headers (explained in §14.18)	by topic and partition only (§14.9)
Ordering	per queue, weakened by multiple consumers and redelivery	strict **per partition**
Typical use	task distribution, request/reply, complex routing	event streaming, event sourcing (storing every change as an event and rebuilding state from them), feeding many independent consumers, high throughput

Delivery guarantees

Guarantee	**Meaning**	**How you get it**
At-most-once	a message may be lost, never duplicated	ack or commit **before** processing
At-least-once	never lost, may be **duplicated**	ack or commit **after** processing. Spring's default for both Kafka and RabbitMQ
Exactly-once	processed once	only inside a closed system, such as Kafka transactions for read-process-write within Kafka. Across a database and a broker you build it yourself: at-least-once plus an **idempotent consumer**

An **idempotent** operation gives the same result when repeated: "set status to PAID" is idempotent, "add 10 € to the balance" is not. Because at-least-once delivery is the normal case, **every consumer should tolerate duplicates**. 14.17 shows the usual technique: remember processed message ids.

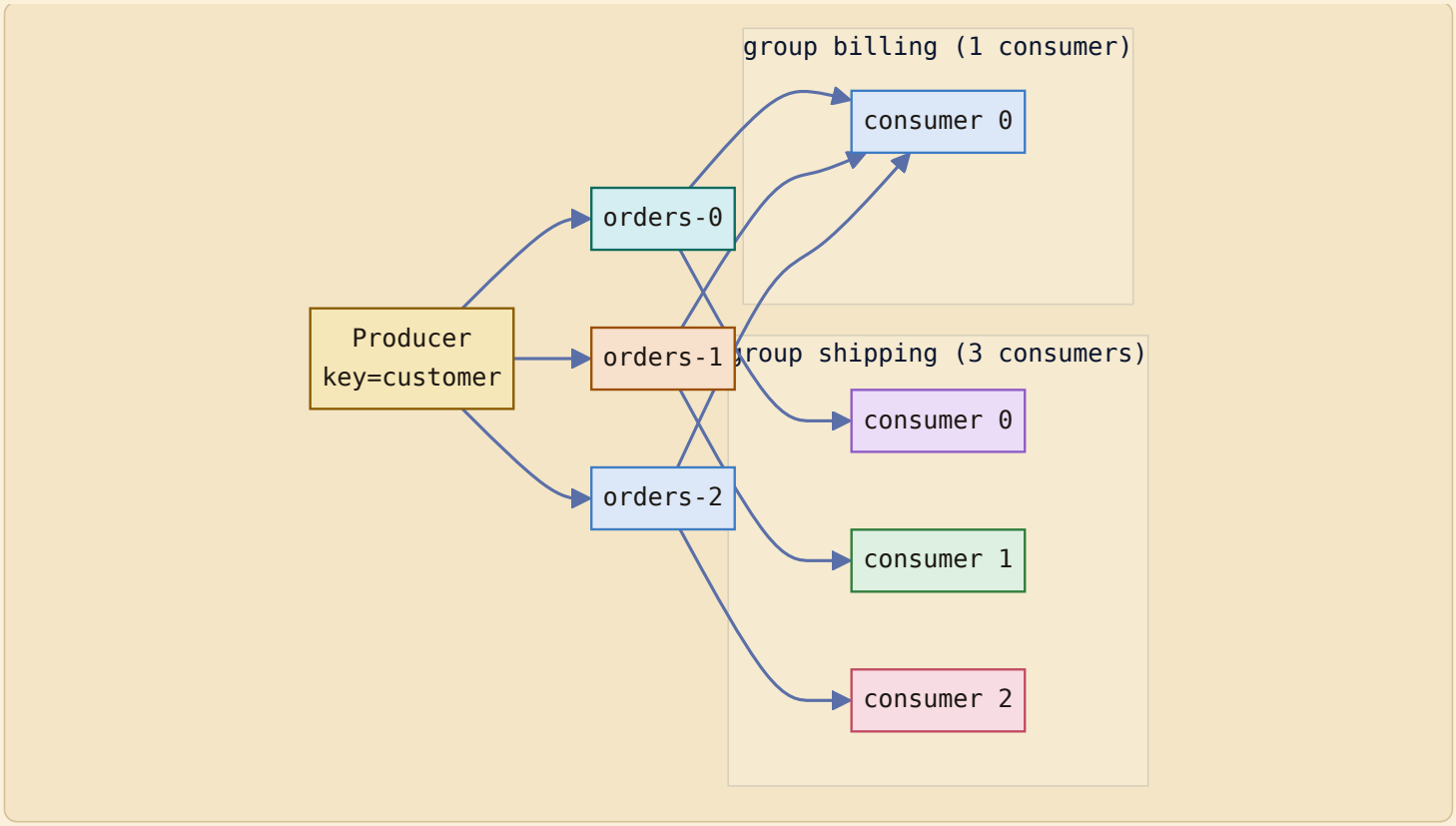
The theory, including why exactly-once is hard, is in DDIA (the book *Designing Data-Intensive Applications*) ch11 — Stream processing.

14.9 Kafka in five minutes

What it is

Apache Kafka is a distributed (spread over several servers), replicated (each piece stored on several servers) **commit log** (an append-only list of records, read in order). Its vocabulary:

Term	**Meaning**
Broker	one Kafka server. A **cluster** has several. Since Kafka 4.0 the cluster manages itself with **KRaft** (Kafka Raft; Raft is an algorithm that lets servers agree on shared state); ZooKeeper (the separate coordination service Kafka used before) is gone
Topic	a named stream of records, e.g. `orders`
Partition	a topic is split into N partitions. Each is an ordered, append-only log stored on a broker, which is the unit of parallelism
Offset	a record's position inside its partition: 0, 1, 2, ...
Record	key (optional) + value + headers + timestamp
Key	decides the partition: the default partitioner hashes the key (turns it into a number with a fixed function, here murmur2) modulo (remainder after dividing by) the partition count. **Same key → same partition → ordered**. No key → records are spread across partitions
Replication factor	copies of each partition on different brokers. One is the **leader**, the others are followers
Consumer group	consumers sharing a `group.id`. Each partition is read by **exactly one** member of the group, and each group gets **every** record
Committed offset	per group and partition: "we've processed up to here". After a restart the group continues from it
Rebalance	reassigning partitions when members join or leave



Two rules follow from the picture:

1. **Scale consumers up to the partition count.** A fourth consumer in `shipping` would sit idle. Choose partitions for the parallelism you will need.
2. **Ordering is per partition.** Records with the same key are processed in order. There is no order across partitions.

Spring for Apache Kafka ("spring-kafka") wraps the Kafka Java client with `KafkaTemplate` (sending), `@KafkaListener` (receiving), error handlers, retry topics, transactions and test support. Boot 4.1.1 manages **`spring-kafka 4.1.1`** and **`kafka-clients 4.2.1`** (checked in the Boot 4.1.1 dependency BOM — the "bill of materials" that pins compatible library versions — and the spring-kafka 4.1 reference docs).

14.10 Setting up Spring Kafka (with JSON)

Dependencies

```
<!-- spring-kafka 4.1.x + kafka-clients 4.2.x + Jackson 3, versions managed by Boot 4.1.1 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
<!-- the Kafka starter does not bring Jackson; JacksonJsonSerializer needs it (Jackson 3, tools.jackson.*) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jackson</artifactId>
</dependency>
<!-- tests: spring-boot-starter-test + spring-kafka-test (EmbeddedKafka) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka-test</artifactId>
  <scope>test</scope>
</dependency>
```

Boot 4's starters are modular (ch01), so Kafka has its own `spring-boot-starter-kafka` and `spring-boot-starter-kafka-test`. In Boot 3 you added `org.springframework.kafka:spring-kafka` directly. Our first run failed with `ClassNotFoundException: tools.jackson.core.type.TypeReference` because the Kafka starter does not include Jackson. Add `spring-boot-starter-jackson` (or a web starter, which brings it).

Configuration

```
# in tests @EmbeddedKafka sets spring.embedded.kafka.brokers; otherwise a local broker
spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers:localhost:9092}

# ---- producer: String keys, JSON values (Jackson 3) ----
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JacksonJsonSerializer
spring.kafka.producer.acks=all

# ---- consumer: JSON values, wrapped so that a bad record cannot crash the consumer ----
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.properties[spring.deserializer.value.delegate.class]=org.springframework.kafka.support.serializer.JacksonJsonDeserializer
spring.kafka.consumer.properties[spring.json.trusted.packages]=ch14.kafka
```

What each part does:

Setting	Why
`JacksonJsonSerializer` / `JacksonJsonDeserializer`	JSON via **Jackson 3**. The older `JsonSerializer`/`JsonDeserializer` (Jackson 2) are **deprecated for removal since spring-kafka 4.0**; the jar marks them `@Deprecated(since="4.0", forRemoval=true)`
`ErrorHandlingDeserializer`	wraps the real deserializer. If a record's bytes are not valid JSON (a **poison pill**), the failure is handed to the error handler instead of failing inside `poll()` again and again forever
`spring.json.trusted.packages`	the deserializer creates the class named in the record's `__TypeId__` header only if it is in a trusted package. Never use `*` for data from untrusted producers
`auto-offset-reset=earliest`	a **new** group with no committed offset starts at the beginning. The Kafka default `latest` would skip everything sent before the consumer joined
`acks=all`	the leader waits for all in-sync replicas (copies that are fully up to date) before confirming. Already the client default since Kafka 3.0; written out here for clarity

The event is a plain record:

```
/** The message payload. Serialized to JSON by JacksonJsonSerializer. */
public record OrderEvent(String orderId, String customer, long cents) {}
```

Topics are declared as beans. Boot's `KafkaAdmin` creates them at startup if they don't exist:

```
@Bean NewTopic orders()      { return TopicBuilder.name("orders").partitions(3).replicas(1).build(); }
@Bean NewTopic payments()    { return TopicBuilder.name("payments").partitions(1).replicas(1).build(); }
@Bean NewTopic paymentsDlt() { return TopicBuilder.name("payments-dlt").partitions(1).replicas(1).build(); }
```

What goes over the wire

A test reads one record back with a plain `StringDeserializer` to show the raw bytes:

Output (`ProducerGuaranteesTest.jsonOnTheWire`):

```
value bytes as text: {"orderId":"o-json","customer":"zoe","cents":4200}
__TypeId__ header  : ch14.kafka.OrderEvent
```

The **type header** tells the consumer which class to build. It couples producer and consumer to the same Java class name. Across services, either give the consumer a fixed target type (`spring.json.value.default.type` plus `spring.json.use.type.headers=false`), map type ids to local classes (`spring.json.type.mapping`), or use a schema format such as Avro or Protobuf (compact binary formats with an explicit schema) with a schema registry (a server that stores the schemas and checks that new versions stay compatible).

14.11 Producing: `KafkaTemplate`, keys and partitions

How it works

1. `kafkaTemplate.send(topic, key, value)` serializes (turns into bytes) the key and value, picks a partition (hash of the key), and puts the record into an in-memory batch.
2. A background I/O thread sends batches to the partition leaders. Batching is where Kafka's throughput comes from (`linger.ms`, `batch.size`).
3. `send` returns a `CompletableFuture<SendResult<K, V>>`. It completes when the broker acknowledges, and its `RecordMetadata` holds the topic, partition and offset. It completes exceptionally if the send failed after the client's own retries.

```java
@Service
public class OrderProducer {
    private final KafkaTemplate<String, Object> kafka;
    // constructor ...

    /** Key = customer: all events of one customer go to the same partition, so they stay in order. */
    public CompletableFuture<SendResult<String, Object>> publish(OrderEvent event) {
        return kafka.send("orders", event.customer(), event);
    }
}
```

The test sends six orders for three customers and prints where each one landed:

```java
RecordMetadata meta = producer.publish(e).join().getRecordMetadata();
System.out.printf("%s key=%-5s -> %s-%d@%d%n", e.orderId(), e.customer(), meta.topic(), meta.partition(), meta.offset());
```

Output (`ProducerConsumerTest`):

```
--- producer: send(topic, key, value).get() -> RecordMetadata ---
o-1 key=alice -> orders-0@0
o-2 key=dave  -> orders-1@0
o-3 key=carol -> orders-2@0
o-4 key=alice -> orders-0@1
o-5 key=dave  -> orders-1@1
o-6 key=carol -> orders-2@1
```

`alice` always lands on partition 0, `dave` on 1, `carol` on 2. The hash is deterministic. In an early version of the test, the customers `alice`, `bob` and `carol` produced partitions 0, 0 and 2: two keys shared partition 0 and partition 1 got nothing. **Hashing does not promise an even spread**, and a "hot" key (one huge customer) overloads one partition.

Common mistakes

- **Fire-and-forget without looking at the future.** A failed send is only logged. Attach `whenComplete`, or `join()` where the caller must know.
- **No key when order matters.** Events for one order can then land on different partitions and be processed out of order.
- **Changing the partition count later.** `hash(key) % N` changes, so a key moves to another partition and ordering across the change is lost.
- **Huge messages.** The default maximum is about 1 MB. Store big payloads elsewhere and send a reference.

14.12 Consuming: `@KafkaListener`, concurrency and consumer groups

How it works

1. Boot's auto-configured `ConcurrentKafkaListenerContainerFactory` creates a **listener container** for each `@KafkaListener`.
2. `concurrency = "3"` creates three consumers (three threads) in the same group. Kafka assigns each one a share of the partitions.
3. Each thread loops: `poll()` a batch of records, call your method once per record, then **commit the offsets**. The default ack mode is `BATCH`: commit after the whole polled batch was processed.
4. If your method throws, the container's **error handler** decides what happens (14.13).

```java
/** Group "shipping", 3 consumer threads: the 3 partitions are shared out between them. */
@KafkaListener(id = "shipping", topics = "orders", groupId = "shipping", concurrency = "3")
void ship(ConsumerRecord<String, OrderEvent> record) {
    received.add("shipping", record.value().orderId() + " key=" + record.key()
            + " p" + record.partition() + " on " + Thread.currentThread().getName());
}

/** Group "billing", 1 thread: a different group gets its OWN copy of every message. */
@KafkaListener(id = "billing", topics = "orders", groupId = "billing")
void bill(@Payload OrderEvent event,
          @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
          @Header(KafkaHeaders.OFFSET) long offset) {
    received.add("billing", event.orderId() + " p" + partition + "@" + offset
            + " on " + Thread.currentThread().getName());
}
```

A listener method can take the whole `ConsumerRecord`, or just the converted payload plus `@Header` values, or `Acknowledgment` for manual commits, or a `List<...>` for batch listeners.

Before sending, the test waits until the group has **settled** (each of the three consumers owns one partition). Without the wait, the first consumer to join briefly owned all three partitions and processed everything, as happened in this test's first run.

```java
var shipping = (ConcurrentMessageListenerContainer<?, ?>) registry.getListenerContainer("shipping");
await().atMost(Duration.ofSeconds(30)).until(() -> shipping.getContainers().stream()
        .allMatch(c -> c.getAssignedPartitions() != null && c.getAssignedPartitions().size() == 1));
```

Output:

```
assignment: shipping-0 -> [orders-0]
assignment: shipping-1 -> [orders-1]
assignment: shipping-2 -> [orders-2]
...
--- group 'shipping' (concurrency 3) ---
o-1 key=alice p0 on shipping-0-C-1
o-2 key=dave p1 on shipping-1-C-1
o-3 key=carol p2 on shipping-2-C-1
o-4 key=alice p0 on shipping-0-C-1
...
--- group 'billing' (concurrency 1) ---
o-1 p0@0 on billing-0-C-1
o-2 p1@0 on billing-0-C-1
... (o-3 to o-6, all on billing-0-C-1)
```

- In `shipping`, each partition had its own thread (`shipping-N-C-1` = listener id, consumer index, "C" for consumer thread). All of `alice`'s orders went to one thread, in order.
- `billing` is a **different group**, so it got **all six** records again, on its single thread. That is publish-subscribe: every group gets every record. Within one group it behaves like a queue: each record goes to one member.

Commit modes

spring.kafka.listener.ack-mode	Commits
`BATCH` (default)	after all records from one `poll()` were processed
`RECORD`	after each record. Safer and slower
`TIME` / `COUNT` / `COUNT_TIME`	after a time or a number of records
`MANUAL` / `MANUAL_IMMEDIATE`	when your code calls `acknowledgment.acknowledge()`

spring-kafka turns Kafka's own auto-commit off (`enable.auto.commit=false`), so offsets are committed by the container **after** processing, which gives at-least-once. After a crash, the uncommitted records of the last batch are delivered again. The consumer must be idempotent.

Common mistakes

- **More consumers than partitions.** The extra ones sit idle.
- **Slow processing.** If one `poll()` batch takes longer than `max.poll.interval.ms` (5 minutes by default), Kafka assumes the consumer died, rebalances, and another member reprocesses the batch. Reduce `max.poll.records` or speed up the work.
- **Blocking the listener thread on remote calls without timeouts.** One hanging HTTP call stops a whole partition.
- **Same `groupId` for two different purposes.** They split the records instead of each getting all of them.

14.13 Testing with `@EmbeddedKafka`

What it is

`spring-kafka-test` can start a **real Kafka broker inside the test JVM**, in KRaft mode on a random port. No Docker, no installation. The annotation publishes the broker address as the property `spring.embedded.kafka.brokers`, which our `application.properties` already reads.

```
@SpringBootTest
@EmbeddedKafka(partitions = 1, brokerProperties = {
        "transaction.state.log.replication.factor=1", "transaction.state.log.min.isr=1"})
abstract class KafkaTestBase {}

class ProducerConsumerTest extends KafkaTestBase { ... }
class ErrorHandlingTest extends KafkaTestBase { ... }
class ProducerGuaranteesTest extends KafkaTestBase { ... }
```

- All three test classes inherit the **same** annotations, so Spring's test-context cache reuses one context and one broker (see ch12 on context caching).
- `partitions = 1` applies to topics listed in `@EmbeddedKafka(topics = ...)` and to auto-created topics. Our `NewTopic` beans set their own counts.
- A single broker cannot store the transaction log with the default replication factor of 3. The two broker properties make Kafka transactions (14.16) work.
- Asynchronous consumption is asserted with **Awaitility** (`await().atMost(...).until(...)`), never with `Thread.sleep` and hope.

Test style	Speed	Realism	Needs
Unit test of the listener method (call it directly)	ms	logic only	nothing
`@EmbeddedKafka`	seconds	real protocol, real client, one broker	spring-kafka-test
Testcontainers Kafka + `@ServiceConnection`	slower	the real broker image you run in production	Docker (ch12)

Timing (varies by machine): the whole `kafka-orders` suite (6 tests, one broker) takes about 12 seconds of test time (surefire reported 5.2 s + 4.9 s + 2.2 s), mostly broker start-up and back-off waits.

14.14 Error handling: `DefaultErrorHandler` and the dead-letter topic

The problem it solves

A listener throws. What now? If the container simply re-reads the same record, a record that can **never** succeed (bad data, a poison pill) blocks its partition forever. If it skips the record, data is lost. The standard answer:

1. **Retry** a few times with a back-off, for errors that may go away (timeouts, a service restarting).
2. **Don't retry** errors that cannot go away (validation failures, deserialization errors).
3. After the last attempt, **publish the record to a dead-letter topic (DLT)** with headers saying what went wrong, **commit** its offset, and move on. Someone inspects the DLT later, fixes the cause, and replays the records.

How it works in spring-kafka

- `DefaultErrorHandler` is the container's error handler. On failure it **seeks** (moves the read position of) the partition back to the failed record, so the next `poll()` returns it again. That is a **blocking** retry: the partition waits.
- Its `BackOff` decides how often and how long. The default is `FixedBackOff(0, 9)`: 9 retries, no delay, 10 attempts in total.
- When retries are exhausted, it calls a `ConsumerRecordRecoverer`. `DeadLetterPublishingRecoverer` sends the record to the DLT.
- Some exceptions are **fatal by default** and skip retries: `DeserializationException`, `MessageConversionException`, `ConversionException`, `MethodArgumentResolutionException`, `NoSuchMethodException`, `ClassCastException`. Add your own with `addNotRetryableExceptions(...)`.
- Boot applies a `CommonErrorHandler` bean to its auto-configured container factory. Declaring the bean is enough.

> **Version note:** since **spring-kafka 3.3** (so in every 4.x) the default DLT name is `<topic>-dlt`. Up to 3.2 it was `<topic>.DLT`, and most articles online still show that. Our first run proved it: the log said `Destination resolver returned non-existent partition payments-dlt-0`, because we had created `payments.DLT`.

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> jsonTemplate, ProducerFactory<String, Object> pf) {
    // A record that could not be deserialized has only raw bytes: publish those bytes unchanged.
    KafkaTemplate<String, Object> bytesTemplate =
            new KafkaTemplate<>(pf, Map.of(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, ByteArraySerializer.class));
    Map<Class<?>, KafkaOperations<?, ?>> templates = new LinkedHashMap<>();
    templates.put(byte[].class, bytesTemplate);
    templates.put(Object.class, jsonTemplate);

    var recoverer = new DeadLetterPublishingRecoverer(templates);   // default target: same topic + "-dlt", same partition
    var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(100, 2));
    handler.addNotRetryableExceptions(IllegalArgumentException.class);   // bad data will never get better
    return handler;
}
```

Why two templates? A normal failed record is an `OrderEvent` and is re-published as JSON. A **poison pill** was never deserialized, so the recoverer only has its raw `byte[]`. The map picks the template by value type, and the `LinkedHashMap` keeps `byte[]` ahead of the `Object` catch-all.

The listener simulates three kinds of payment:

```
@KafkaListener(id = "payments", topics = "payments", groupId = "payments")
void charge(OrderEvent e) {
    int attempt = attempts.computeIfAbsent(e.orderId(), k -> new AtomicInteger()).incrementAndGet();
    received.add("payments", e.orderId() + " attempt " + attempt);
    if (e.cents() < 0) throw new IllegalArgumentException("negative amount " + e.cents());
    if (e.orderId().equals("p-down") || (e.orderId().equals("p-flaky") && attempt < 3))
        throw new TransientPaymentException("provider timeout");
    received.add("payments", e.orderId() + " CHARGED");
}

/** Reads the dead-letter topic as plain text (overrides the JSON deserializer for this listener only). */
@KafkaListener(id = "payments-dlt", topics = "payments-dlt", groupId = "dlt-inspector",
        properties = "value.deserializer=org.apache.kafka.common.serialization.StringDeserializer")
void inspectDeadLetter(String value,
                       @Header(KafkaHeaders.DLT_EXCEPTION_CAUSE_FQCN) String causeClass,
                       @Header(KafkaHeaders.DLT_ORIGINAL_TOPIC) String originalTopic) { ... }
```

The test sends `p-ok`, `p-flaky` (fails twice, then works), `p-down` (always fails), `p-bad` (negative amount) and a raw string `this is not JSON`:

Output (`ErrorHandlingTest.blockingRetriesThenDeadLetterTopic`):

```
--- payments listener ---
p-ok attempt 1
p-ok CHARGED
p-flaky attempt 1
...
p-flaky attempt 3
p-flaky CHARGED
p-down attempt 1
...
p-down attempt 3
p-bad attempt 1
--- payments-dlt ---
{"orderId":"p-down","customer":"carol","cents":300} <- payments, cause TransientPaymentException
{"orderId":"p-bad","customer":"dave","cents":-5} <- payments, cause IllegalArgumentException
this is not JSON <- payments, cause DeserializationException
```

Record	What happened
`p-flaky`	1 attempt + 2 retries, succeeded on the third. No DLT
`p-down`	3 attempts, then DLT with the cause class in a header
`p-bad`	`IllegalArgumentException` is not retryable: 1 attempt, then DLT
`this is not JSON`	the listener was **never called**. `ErrorHandlingDeserializer` turned the failure into a fatal `DeserializationException`, and the original bytes went to the DLT unchanged

The listener exception is wrapped in `ListenerExecutionFailedException`, but the classifier looks through the **cause chain**, which is why `IllegalArgumentException` was recognized.

DLT records carry headers such as `kafka_dlt-original-topic`, `-original-partition`, `-original-offset`, `-exception-fqcn`, `-exception-cause-fqcn`, `-exception-message` and `-exception-stacktrace`. The constants are in `KafkaHeaders.DLT_*`.

Trade-offs of blocking retries

Pro	Con
Order within the partition is kept	the partition **stops** while one record is retried. A long back-off stalls everything behind it
Simple, no extra topics	the total back-off must stay well under `max.poll.interval.ms`, or the consumer is kicked out of the group

Common mistakes

- **Relying on the default back-off** (10 instant attempts). It hammers a struggling dependency. Set a real back-off, e.g. `ExponentialBackOff`.
- **No `ErrorHandlingDeserializer`.** A poison pill fails inside `poll()` before your code runs, so the container cannot hand it to the error handler.
- **A DLT with fewer partitions than the source topic.** The recoverer targets the **same partition number**. Create the DLT with at least as many partitions, or give it a custom destination resolver.
- **Nobody watches the DLT.** Add an alert on its size, and a documented replay procedure.

14.15 Non-blocking retries with `@RetryableTopic`

What it is

Instead of seeking back and blocking the partition, a failed record is **re-published to a retry topic** and its offset is committed. The main topic keeps flowing. A separate consumer reads the retry topic after the back-off delay. After the last attempt the record goes to the DLT and a `@DltHandler` method runs.

```
@RetryableTopic(attempts = "3", backOff = @BackOff(delay = 200))
@KafkaListener(id = "invoices", topics = "invoices", groupId = "invoices")
void createInvoice(OrderEvent e, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    received.add("invoices", e.orderId() + " from " + topic);
    if (e.orderId().startsWith("inv-bad")) throw new IllegalStateException("tax service unavailable");
}

@DltHandler
void giveUp(OrderEvent e, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    received.add("invoices", e.orderId() + " DLT handler, from " + topic);
}
```

Output (`ErrorHandlingTest.nonBlockingRetryTopics`, `inv-bad` sent first, then `inv-ok`):

```
--- @RetryableTopic ---
inv-bad from invoices
inv-ok from invoices
inv-bad from invoices-retry
inv-bad from invoices-retry
inv-bad DLT handler, from invoices-dlt
```

`inv-ok` was processed **before** the retries of `inv-bad`. The failing record no longer blocked the partition. The framework created `invoices-retry` and `invoices-dlt` itself. With a fixed delay, **one** retry topic is reused for all attempts (the annotation default since spring-kafka 3.2; the programmatic `RetryTopicConfigurationBuilder` got the same default in 4.1). In spring-kafka 4.0 the back-off attribute changed from Spring Retry's `backoff = @Backoff` to `backOff = @BackOff` (`org.springframework.kafka.annotation.BackOff`).

Blocking vs non-blocking

	`DefaultErrorHandler` (blocking)	`@RetryableTopic` (non-blocking)
Partition during retries	stopped	keeps flowing
Ordering	kept	**lost** for the retried record: later records overtake it
Extra topics	only the DLT	retry topic(s) + DLT
Long delays (minutes)	dangerous (`max.poll.interval.ms`)	fine
Choose when	order matters (account balance updates)	independent records, slow-recovering dependencies

When not to use it

When the order of events for one key matters. If `OrderCancelled` overtakes a retried `OrderPlaced`, the consumer sees a cancel for an order it doesn't know.

14.16 Producer guarantees: idempotence and transactions

Idempotent producer

The problem: a producer sends a batch, the broker writes it, and the acknowledgement is lost. The producer retries and the broker writes the batch **twice**. The **idempotent producer** solves this. The broker gives the producer an id, the producer numbers every batch per partition, and the broker drops sequence numbers it has already written. Since Kafka 3.0 it is **on by default**, together with `acks=all`.

```
var effective = new ProducerConfig(bootProducerFactory.getConfigurationProperties());
System.out.println("enable.idempotence = " + effective.getBoolean(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG));
System.out.println("acks               = " + effective.getString(ProducerConfig.ACKS_CONFIG));
System.out.println("max.in.flight      = " + effective.getInt(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION));
```

Output:

```
enable.idempotence = true
acks               = -1
max.in.flight      = 5
```

`acks=-1` is `all`. Idempotence requires `max.in.flight ≤ 5` and keeps ordering per partition even when retries happen. It removes duplicates caused by **producer retries**. It does not stop your application from calling `send` twice for the same business event.

Transactional producer

A **Kafka transaction** makes several sends (to several topics or partitions, plus consumer offsets) **atomic** (all happen or none do). Consumers with `isolation.level=read_committed` see all of them or none.

```
var pf = new DefaultKafkaProducerFactory<String, String>(props);
pf.setTransactionIdPrefix("ledger-tx-");        // turns on transactions (and requires idempotence)
var tx = new KafkaTemplate<>(pf);

tx.executeInTransaction(t -> {                  // committed: both records become visible together
    t.send("ledger", "acc-1", "debit 50");
    t.send("ledger", "acc-2", "credit 50");
    return null;
});
tx.executeInTransaction(t -> {                  // aborted: the record stays invisible
    t.send("ledger", "acc-1", "debit 999");
    t.flush();                                  // the record really reaches the broker...
    throw new IllegalStateException("crash before commit");   // ...but the transaction is aborted
});
```

Output (`ProducerGuaranteesTest.transactionalProducerAndReadCommitted`):

```
isolation.level=read_committed   -> [debit 50, credit 50]
isolation.level=read_uncommitted -> [debit 50, credit 50, debit 999]
```

The aborted record **is in the log**, as `read_uncommitted` shows. The broker writes an abort marker, and `read_committed` consumers skip the record. Without the `flush()` the record never left the client, and both lists were equal in our first run, which proves nothing.

In a Boot app you set `spring.kafka.producer.transaction-id-prefix=tx-`. Boot then makes the auto-configured `KafkaTemplate` transactional and creates a `KafkaTransactionManager`. After that, every `send` must run inside a transaction: `executeInTransaction`, `@Transactional("kafkaTransactionManager")`, or a listener container that starts one. Consumers must set `spring.kafka.consumer.isolation-level=read_committed`, because the Kafka default is `read_uncommitted`.

What Kafka transactions do not give you

Exactly-once holds for the loop "consume from Kafka → process → produce to Kafka, and commit the input offsets in the same transaction". It does **not** cover a database write plus a Kafka send. There is no two-phase commit (a protocol where a coordinator first asks every system "can you commit?" and then tells all of them to commit) between them. The old `ChainedKafkaTransactionManager` (deprecated since spring-kafka 2.7) only committed one after the other, which is best effort, not atomic. For database + broker, use the transactional outbox.

14.17 The transactional outbox

The problem it solves: the dual write

```
@Transactional
public void placeOrder(...) {
    orderRepository.save(order);                // 1. database
    kafkaTemplate.send("orders", event);        // 2. broker
}                                               // 3. commit
```

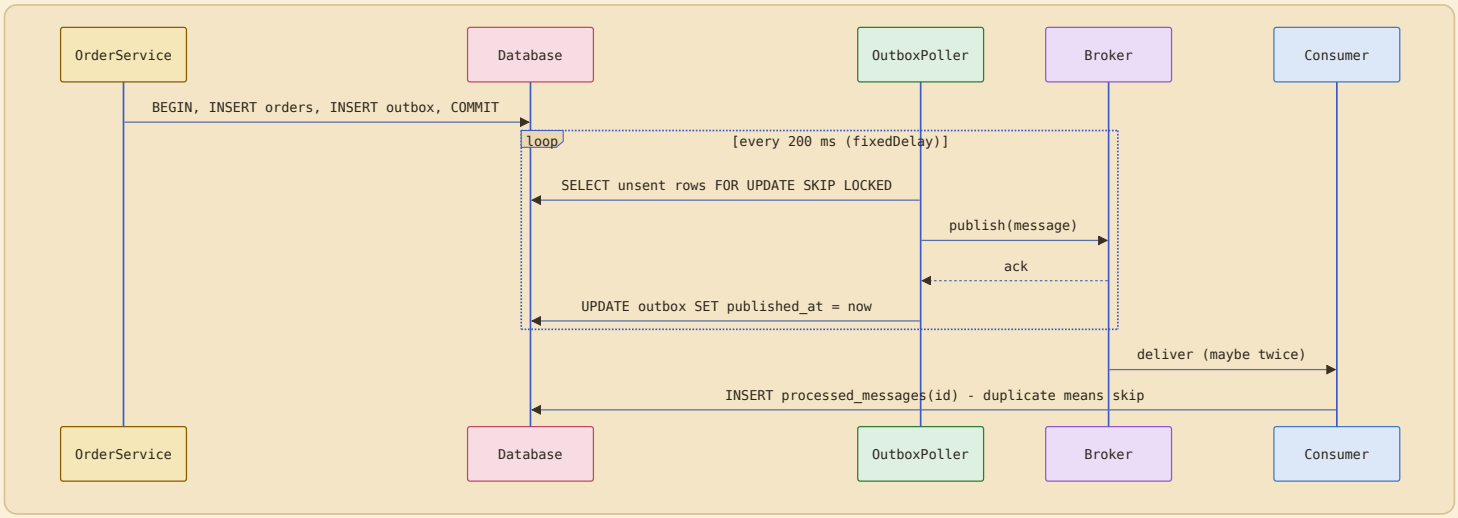
Every ordering of these steps fails somewhere:

Failure	Result
send succeeds, then the commit fails	consumers hear about an order that **does not exist**
commit succeeds, then the app crashes before the send	the order exists and **nobody is told**
send after the commit, broker down	same: lost event, unless you build retries and persistence... which is an outbox

This is the **dual-write problem**. The database and the broker cannot share one transaction.

How it works

1. In the **same local database transaction** as the business change, insert a row into an `outbox` table describing the event.
2. Commit. Now the order and the event exist together, or neither does.
3. A separate **message relay** reads unsent outbox rows in order, publishes them to the broker, and marks them sent. It is either a **poller** (shown here) or **change data capture** (CDC: streaming every committed change out of the database): Debezium reads the database's write-ahead log (the file where the database records each change before applying it) and publishes the outbox rows.
4. Consumers get **at-least-once** delivery, so they must be **idempotent**. Each message carries a unique id they can de-duplicate on.



The pattern is taught without Spring in the Saga notes — Part 13D, implementing the saga log, the event-driven architecture notes, and System-design patterns, Module 10 — the outbox pattern. Here it runs.

The schema

```
-- the outbox: messages waiting to be sent, written in the SAME transaction as the order
CREATE TABLE IF NOT EXISTS outbox (
    id            IDENTITY PRIMARY KEY,                -- also gives the publishing order
    message_id    UUID         NOT NULL UNIQUE,        -- consumers de-duplicate on this
    aggregate_id  VARCHAR(50)  NOT NULL,               -- becomes the Kafka key
    event_type    VARCHAR(50)  NOT NULL,
    payload       VARCHAR(2000) NOT NULL,              -- JSON
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP NOT NULL,
    published_at  TIMESTAMP,                           -- NULL = not sent yet
    attempts      INT          DEFAULT 0 NOT NULL
);
CREATE INDEX IF NOT EXISTS outbox_unpublished ON outbox (published_at, id);
-- consumer side: ids of messages already handled (idempotent consumer)
CREATE TABLE IF NOT EXISTS processed_messages (message_id UUID PRIMARY KEY);
```

Writing the order and the event together

```
/** ONE local transaction: the order row and the outbox row commit together or not at all. */
@Transactional
public long placeOrder(String item, long cents, boolean failBeforeCommit) {
    var keys = new GeneratedKeyHolder();
    jdbc.sql("INSERT INTO orders(item, cents) VALUES (?, ?)").params(item, cents).update(keys, "id");
    long orderId = keys.getKey().longValue();

    String payload = json.writeValueAsString(new OrderPlaced(orderId, item, cents));   // Jackson 3 JsonMapper
    jdbc.sql("""
            INSERT INTO outbox(message_id, aggregate_id, event_type, payload)
            VALUES (?, ?, 'OrderPlaced', ?)""")
        .params(UUID.randomUUID(), "order-" + orderId, payload)
        .update();

    if (failBeforeCommit) throw new IllegalStateException("crash before commit");
    return orderId;       // no broker call here: nothing can be half-done
}
```

The poller (message relay)

```
@Scheduled(fixedDelayString = "${outbox.poller.delay-ms}")
public void poll() { publishBatch(); }

@Transactional
public int publishBatch() {
    // FOR UPDATE SKIP LOCKED: several app instances can poll at once without sending the same row twice
    List<OutboxRow> rows = jdbc.sql("""
            SELECT id, message_id, aggregate_id, event_type, payload
            FROM outbox
            WHERE published_at IS NULL
            ORDER BY id
            FETCH FIRST 10 ROWS ONLY
            FOR UPDATE SKIP LOCKED""")
        .query(OutboxRow.class).list();

    int sent = 0;
    for (OutboxRow row : rows) {
        try {
            publisher.publish(new Message(row.messageId(), row.aggregateId(), row.eventType(), row.payload()));
        } catch (Exception e) {
            jdbc.sql("UPDATE outbox SET attempts = attempts + 1 WHERE id = ?").param(row.id()).update();
            log.warn("outbox row {} not published ({}), will retry", row.id(), e.getMessage());
            break;                                  // keep order: do not send later rows before this one
        }
        jdbc.sql("UPDATE outbox SET published_at = CURRENT_TIMESTAMP, attempts = attempts + 1 WHERE id = ?")
            .param(row.id()).update();
        sent++;
    }
    return sent;
}
```

Design choices:

- `fixedDelay`, not `fixedRate`: a slow round never overlaps the next (14.7).
- `FOR UPDATE SKIP LOCKED` (PostgreSQL 9.5+, MySQL 8.0+, and H2 2.x, which runs this demo): with several app instances, each poller locks different rows. The SQL book explains row locks in SQL ch13.
- **Stop at the first failure** so the messages stay in order.
- `MessagePublisher` is an interface. The demo uses `InMemoryBroker`, which can simulate an outage (`DOWN`) or a lost acknowledgement (`ACK_LOST`: the message is stored, then a timeout is thrown). With Kafka the implementation is one line, and

the `.get()` with a timeout is what makes "published" mean "acknowledged":

```
kafkaTemplate.send("orders", m.key(), m.payload()).get(10, TimeUnit.SECONDS);
```

Proving it

The tests switch the scheduler off (`outbox.poller.enabled=false`) and call `publishBatch()` themselves, so every step is deterministic. One extra test leaves the scheduler on.

Output (`OutboxTest`, `OutboxSchedulerTest`):

```
after rollback: orders=0 outbox=0
after commit:   orders=1 outbox=1 unpublished=1
outbox payload: {"orderId":5,"item":"book","cents":1999}
poller sent 1 -> broker has 1, unpublished=0

WARN  c.o.OutboxPoller - outbox row 2 not published (broker unreachable), will retry
broker DOWN -> sent 0, unpublished=2, attempts of first row=1
broker UP   -> sent 2, unpublished=0
broker order: [order-2, order-3]

WARN  c.o.OutboxPoller - outbox row 1 not published (no ack from broker), will retry
broker holds 2 copies of message eab1f5f0...
consumer handled: [true, false] -> stock_reservations=1

published by the scheduled poller within ~200 ms: OrderPlaced {"orderId":1,"item":"desk","cents":25000}
```

(The id numbers depend on test order, because identity columns (auto-incrementing id columns) keep counting after `DELETE`.)

Scenario	What the test proved
Crash before commit	**no** order and **no** outbox row: nothing to publish, nothing wrong published
Commit	one order, one unpublished outbox row, published on the next poll
Broker down	nothing lost: rows stay unpublished with `attempts=1`, and go out in order once the broker is back
Ack lost	the broker got the message but the poller didn't know, so it sent it **again**. The consumer's `processed_messages` insert failed with `DuplicateKeyException` on the second copy, so the stock was reserved **once**

The idempotent consumer:

```
@Transactional
public boolean handle(Message m) {
    try {
        jdbc.sql("INSERT INTO processed_messages(message_id) VALUES (?)").param(m.messageId()).update();
    } catch (DuplicateKeyException alreadySeen) {
        return false;                              // duplicate: skip silently
    }
    jdbc.sql("INSERT INTO stock_reservations(order_id) VALUES (?)").param(m.key()).update();
    return true;
}
```

The id insert and the business change are in **one** transaction. If the business change fails, the id is rolled back too, and a redelivery is processed normally.

Trade-offs

Poller (this demo)	CDC (Debezium outbox event router)
plain SQL + `@Scheduled`, easy to understand	no polling load, lower latency
adds query load; latency = poll interval	extra infrastructure: Kafka Connect (Kafka's framework for moving data in and out), Debezium, access to the DB log
works on any database	needs DB support (Postgres logical decoding, MySQL binlog)

Housekeeping: delete or archive published rows (e.g. a nightly job that removes rows with `published_at` older than 7 days), or the table grows forever.

Spring Modulith offers a ready-made variant for modular monoliths (one deployable application split internally into well-separated modules). Its *event publication registry* stores application events in a database table before commit and re-delivers the ones whose listeners did not complete, and its *event externalization* forwards selected events to Kafka or AMQP (Advanced Message Queuing Protocol, the protocol RabbitMQ speaks).

When not to use it

- The message is not tied to a database change (e.g. a metrics tick). Send it directly.
- Losing an occasional event is acceptable (cache warm-up hints). `@TransactionalEventListener(AFTER_COMMIT)` + send is simpler.

Common mistakes

- **Calling the broker inside the business transaction** "just to be sure". That is the dual write again, and it also holds a database connection during a network call.
- **Non-idempotent consumers.** The outbox guarantees at-least-once, not exactly-once.
- **Publishing rows out of order** by continuing after a failure, or by running several pollers without `SKIP LOCKED` and without a key-based order.
- **Letting the outbox grow forever.**

In the interview

> **30-second answer:** "You can't atomically write to a database and publish to Kafka. That's the dual-write problem. The outbox pattern writes the event into an outbox table in the same local transaction as the business change. A relay (a poller with `SELECT ... FOR UPDATE SKIP LOCKED`, or Debezium CDC) publishes the rows and marks them sent. Delivery is at-least-once, because the relay can crash between publishing and marking, so consumers de-duplicate by message id in the same transaction as their own change."

- *Why not a Kafka transaction?* → It only covers Kafka. The database commit is outside it.
- *How do you keep ordering?* → Publish in id order, stop at the first failure, and use the aggregate id as the Kafka key so each aggregate's events share a partition (an **aggregate** is a business object that changes as one unit, such as an order with its lines).
- *Poller or CDC?* → Start with a poller. Switch to CDC when polling load or latency matters, or when Kafka Connect is already there.

Self-check

1. In the "ack lost" test, why did the broker hold two copies? → The message was stored, but the poller saw a timeout and didn't mark the row, so the next poll sent it again.
2. What makes the consumer safe against that? → Inserting the message id into `processed_messages` in the same transaction as the business change. A duplicate id fails and the message is skipped.
3. Why does the poller `break` on the first failure? → So later messages are not published before an earlier one, which would break ordering.

14.18 RabbitMQ with Spring AMQP (configuration, not run)

What it is

RabbitMQ is a queue broker that speaks AMQP 0-9-1. Producers never write to a queue directly. They publish to an **exchange**, which routes the message to zero or more **queues** according to **bindings**. Consumers read from queues and **acknowledge** each message.

Exchange type	Routes by	Example
direct	exact routing key	`payment.failed` → `payments.alerts` queue
topic	pattern: `*` = one word, `#` = zero or more words	`order.placed.*` matches `order.placed.eu`
fanout	nothing: copies to every bound queue	broadcast "cache invalidated"
headers	message header values	rarely used

Other terms: **prefetch** is how many unacknowledged messages one consumer may hold. A **dead-letter exchange (DLX)** is where rejected or expired messages go. **Quorum queues** (RabbitMQ 3.8+) are replicated queues for durability. **Competing consumers**: several consumers on one queue share its messages.

The project `code/rabbit-orders` **compiles** (`mvn compile` passed) but is **not run**, because it needs a broker (for example `docker run -p 5672:5672 rabbitmq:4-management`). Its folder contains `.tools_run_skip`.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>   <!-- spring-amqp / spring-rabbit 4.1.x -->
</dependency>
```

```java
/** Boot's RabbitAdmin declares these exchanges, queues and bindings on the broker at startup. */
@Configuration(proxyBeanMethods = false)
public class RabbitConfig {
    @Bean TopicExchange ordersExchange() { return new TopicExchange("orders"); }
    @Bean DirectExchange deadLetterExchange() { return new DirectExchange("orders.dlx"); }

    @Bean Queue shippingQueue() {
        return QueueBuilder.durable("orders.shipping")
                .quorum()                                          // replicated queue type (RabbitMQ 3.8+)
                .deadLetterExchange("orders.dlx")                  // rejected messages go here...
                .deadLetterRoutingKey("orders.shipping.dlq")       // ...with this routing key
                .build();
    }
    @Bean Queue shippingDlq() { return QueueBuilder.durable("orders.shipping.dlq").build(); }

    /** Topic exchange: routing key "order.placed.eu" matches pattern "order.placed.*". */
    @Bean Binding shippingBinding() {
        return BindingBuilder.bind(shippingQueue()).to(ordersExchange()).with("order.placed.*");
    }
    @Bean Binding dlqBinding() {
        return BindingBuilder.bind(shippingDlq()).to(deadLetterExchange()).with("orders.shipping.dlq");
    }

    /** Boot sets a MessageConverter bean on RabbitTemplate and on the listener container factory. */
    @Bean MessageConverter jsonConverter() { return new JacksonJsonMessageConverter(); }
}

// producer: exchange, routing key, body (converted to JSON)
rabbit.convertAndSend("orders", "order.placed." + region, event);

// consumer: competing consumers on one queue
@RabbitListener(queues = "orders.shipping")
void ship(OrderEvent e) {
    if (e.cents() < 0) throw new IllegalArgumentException("bad amount");   // 1 attempt + 2 retries, then dead-lettered
    ...
}
```

```properties
spring.rabbitmq.host=localhost
spring.rabbitmq.listener.simple.prefetch=10
spring.rabbitmq.listener.simple.concurrency=2
# retry in the listener (blocking), then reject -> the queue's dead-letter exchange
spring.rabbitmq.listener.simple.retry.enabled=true
# retries AFTER the first attempt (Boot 4 replaced max-attempts with max-retries)
spring.rabbitmq.listener.simple.retry.max-retries=2
spring.rabbitmq.listener.simple.retry.initial-interval=500ms
spring.rabbitmq.listener.simple.default-requeue-rejected=false
# producer side: broker confirms + returns for unroutable messages
spring.rabbitmq.publisher-confirm-type=correlated
spring.rabbitmq.publisher-returns=true
spring.rabbitmq.template.mandatory=true
```

(Property names checked against Boot 4.1.1's `spring-boot-amqp` configuration metadata. Boot 4 replaced `retry.max-attempts`, which counted the first attempt, with `retry.max-retries`, which does not.)

How acknowledgement works with Spring AMQP's default `AUTO` mode: the container acks the message when your method returns normally. If it throws, the message is rejected, and it is **requeued** unless `default-requeue-rejected=false` or the exception is `AmqpRejectAndDontRequeueException`. Requeueing a message that always fails creates an endless loop, which is why the DLX setup matters. **Publisher confirms** make the broker acknowledge each publish, the RabbitMQ counterpart of Kafka's `acks`. **Returns** with `mandatory=true` report messages that no queue was bound to receive.

Testing without Docker: Spring AMQP has no embedded broker. Use `@RabbitListenerTest` / `RabbitListenerTestHarness` from `spring-rabbit-test` to spy on listeners, test the listener methods as plain methods, or run the real broker with Testcontainers and `@ServiceConnection` (ch12).

Kafka or RabbitMQ?

Question	Leans Kafka	Leans RabbitMQ
Several independent consumers read the same stream, maybe later	yes: each group has its own offset, and records are retained	each consumer needs its own queue, and messages are gone after the ack
Replay history, rebuild a read model (a copy of data shaped for queries)	yes	no
Complex routing by key patterns or headers	limited	yes: exchanges
Per-message priorities, TTL (time to live: an expiry time), delayed delivery	no	yes (TTL, priorities; delays via a plugin)
Throughput	very high (batching, sequential disk I/O)	high, lower per node
Ordering	per partition, scales with partitions	per queue, weakened by competing consumers
Operational model	a cluster plus partitions to plan	simpler for small setups

Spring also supports **JMS** (`JmsTemplate`, `@JmsListener`; ActiveMQ Artemis, IBM MQ), **Apache Pulsar** (Spring for Apache Pulsar) and a binder abstraction (one API with plug-in "binders" for each broker), **Spring Cloud Stream** (ch16).

14.19 Common mistakes (whole chapter)

Mistake	Symptom	Fix
No `@EnableAsync` / `@EnableScheduling`	`@Async` runs synchronously, `@Scheduled` never runs	add the annotation, on its own config class if you want to switch it with a property
Self-invocation of `@Async`	work runs on the caller's thread	call through another bean (4.5)
`@Async void` without a handler	failures only logged	return `CompletableFuture`, or add an `AsyncUncaughtExceptionHandler`
Declaring an `Executor` bean	Boot's `applicationTaskExecutor` disappears	`@Bean(defaultCandidate = false)` or `spring.task.execution.mode=force`
One default scheduler thread	jobs delay each other	`spring.task.scheduling.pool.size`
`@Scheduled` on N instances	job runs N times	ShedLock / leader election / idempotent job
Sending to Kafka inside a DB transaction	ghost events or lost events	transactional outbox
No key on ordered events	events processed out of order	key = aggregate id
Default `FixedBackOff(0, 9)`	retry storm against a failing service	real back-off + DLT
No `ErrorHandlingDeserializer`	poison pill blocks the partition	wrap the value deserializer
Looking for `orders.DLT` on spring-kafka 3.3+	DLT "empty"	the default name is now `orders-dlt`
Non-idempotent consumers	double charges after a rebalance or redelivery	de-duplicate by message id
`Thread.sleep` in async tests	flaky tests	Awaitility with a timeout

14.20 In the interview

The per-section interview blocks cover transactional listeners (14.3), `fixedRate` vs `fixedDelay` and clustered jobs (14.7), blocking vs retry-topic retries and poison pills (14.14–14.15) and the outbox (14.17). The rest in one round:

1. **How does `@Async` work?** → `@EnableAsync` adds a proxy. Calls through the proxy are submitted to an executor, and the proxy returns `null`/a future at once. Self-invocation bypasses it.
2. **Which executor does `@Async` use in Boot?** → The one named in `@Async("…")`, else an `AsyncConfigurer`'s, else the `taskExecutor`/`applicationTaskExecutor` that Boot auto-configures: a `ThreadPoolTaskExecutor` with 8 core threads and an unbounded queue, or a virtual-thread `SimpleAsyncTaskExecutor` if `spring.threads.virtual.enabled=true`.
3. **What happens to an exception in an `@Async` method?** → For a future return type it is stored in the future. For `void` it goes to the `AsyncUncaughtExceptionHandler`, which only logs by default.
4. **How do you pass the MDC or security context to async threads?** → A `TaskDecorator` (Boot applies a `TaskDecorator` bean to its executor), `spring.task.execution.propagate-context=true`, or `DelegatingSecurityContextAsyncTaskExecutor` for security.

5. **Kafka partition vs consumer group?** → Partitions split a topic for parallelism and ordering. A group splits the partitions among its members. Different groups each get every record.
6. **How do you keep events for one order in sequence?** → Use the order id as the key, so they share one partition, and use blocking retries, not retry topics.
7. **What does `DefaultErrorHandler` do?** → It seeks back and retries per its `BackOff` (default 10 attempts, no delay). Then it calls the recoverer, e.g. `DeadLetterPublishingRecoverer` → `<topic>-dlt` (since spring-kafka 3.3). Fatal exceptions such as `DeserializationException` skip retries.
8. **Is Kafka exactly-once?** → The idempotent producer removes duplicates caused by producer retries. Transactions plus `read_committed` give exactly-once for Kafka-to-Kafka processing. Anything involving a database needs an outbox and idempotent consumers.
9. **How do you test Kafka code without Docker?** → `@EmbeddedKafka` from spring-kafka-test (an in-JVM KRaft broker), `spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers}`, and Awaitility for assertions.
10. **Kafka vs RabbitMQ in one sentence?** → Kafka is a replayable, partitioned log for streams and many independent readers. RabbitMQ is a routing queue broker for task distribution with per-message features.

Chapter TL;DR

- **Events:** `@EventListener` is synchronous. `@Async @EventListener` is not, and it gets no transaction and no exception propagation. `@TransactionalEventListener` runs at `BEFORE_COMMIT` / `AFTER_COMMIT` / `AFTER_ROLLBACK` / `AFTER_COMPLETION`. `@Async` + `AFTER_COMMIT` + `REQUIRES_NEW` is the usual combination, and it is still not durable.
- **`@Async`:** needs `@EnableAsync`. Proxy + executor. Boot's `applicationTaskExecutor` (8 threads, `task-` names, unbounded queue). Pick pools with `@Async("name")` and `defaultCandidate = false`. Copy the MDC with a `TaskDecorator`. `void` errors go to `AsyncUncaughtExceptionHandler`, future errors go into the future.
- **Virtual threads:** `spring.threads.virtual.enabled=true` → one virtual thread per task, no pool, no queue. Limit concurrency yourself.
- **`@Scheduled`:** needs `@EnableScheduling`. `fixedRate` (start-to-start) vs `fixedDelay` (end-to-start) vs six-field `cron`. One scheduler thread by default. Overlap depends on the scheduler. One run per instance in a cluster.
- **Kafka:** Boot 4.1.1 → spring-kafka 4.1.1, kafka-clients 4.2.1, `spring-boot-starter-kafka` (+ `spring-boot-starter-jackson` for JSON). Key → partition → order. Group = one reader per partition, every group gets everything. `JacksonJsonSerializer` (Jackson 3) + `ErrorHandlingDeserializer`. `DefaultErrorHandler` + `DeadLetterPublishingRecoverer` → `<topic>-dlt`. `@RetryableTopic` for non-blocking retries. Idempotence is on by default. Transactions + `read_committed` for atomic multi-sends. Test with `@EmbeddedKafka`.
- **RabbitMQ:** exchanges → bindings → queues, acks, prefetch, DLX, publisher confirms. `spring-boot-starter-amqp`, `@RabbitListener`, `JacksonJsonMessageConverter`.
- **Outbox:** write the event row in the same transaction, relay it with a `fixedDelay` poller (`FOR UPDATE SKIP LOCKED`) or CDC, and expect duplicates: consumers de-duplicate by message id.
- Next: Chapter 15 — Observability and production.

Chapter 15 — Observability and Production

Everything so far has been about making the application *work*. This chapter is about keeping it working once real users depend on it: knowing whether it is healthy, how fast it is, which request failed and why, shutting it down without dropping requests, and starting it quickly in **containers** (isolated, lightweight packages that hold an app plus everything it needs to run, started by Docker or Kubernetes). Spring Boot's answer is a set of "production-ready features": **Actuator** (management endpoints), **Micrometer** (metrics and the Observation API), **Micrometer Tracing + OpenTelemetry** (distributed tracing: following one request across several services), **structured logging**, graceful shutdown (finishing in-flight requests before stopping), virtual threads, and faster startup with AOT (ahead-of-time) caches and native images (§15.11).

What you'll learn

1. The three pillars of observability — logs, metrics, traces — and how Spring connects them.
2. Actuator: endpoints, exposure, access control and securing them.
3. Health: indicators, groups, Kubernetes liveness/readiness probes (on by default in Boot 4), writing a custom `HealthIndicator`.
4. `/actuator/info`: build, git, Java and environment information.
5. Micrometer metrics: `Counter`, `Timer`, `Gauge`, tags, cardinality, the Prometheus scrape format.
6. The Observation API and `@Observed`: one instrumentation, both a metric and a span.
7. Distributed tracing with the OpenTelemetry bridge: trace/span ids in logs, propagation across HTTP calls, exporters, sampling.
8. Structured JSON logging (ECS, Logstash, GELF).
9. Graceful shutdown and what happens in Kubernetes.
10. Virtual threads with one property.
11. Faster startup: extracted jars, CDS, the JDK AOT cache (measured), Spring AOT and GraalVM native images; container images with buildpacks or a Dockerfile.
12. A production configuration checklist.

The code lives in two Spring Boot 4.1.1 / Framework 7.0 projects on Java 25:

- `code/observability-demo/` — Actuator, health, metrics, Prometheus, tracing, structured logs (8 tests).
- `code/production-runtime/` — graceful shutdown and virtual threads (4 tests), plus `code/aot-cache-demo.sh` for the startup comparison.

Outputs are real `mvn -q verify` output (Maven building the projects and running their tests); Spring's log noise is trimmed as `...`.

> **Prerequisites.** Configuration properties and profiles: Chapter 3. Security filter chains: Chapter 10. `RestClient`: Chapter 6. `@ConcurrencyLimit`: 4.7. The system-design view of health checks and tracing is in Operating in Production.

15.1 Observability in one page

What it is

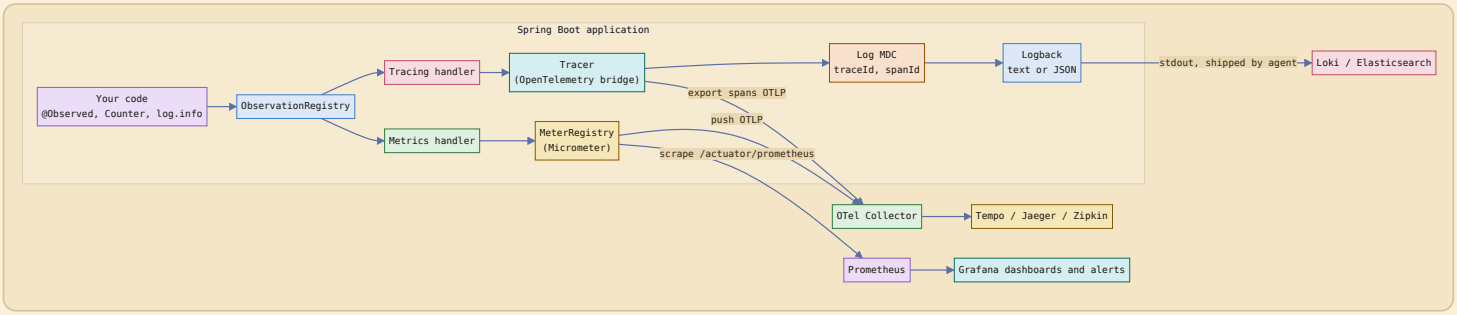
Observability is how well you can understand what a running system is doing from the outside, using the data it emits — without attaching a debugger (a tool that pauses a program so you can inspect it step by step) or adding new code. The data comes in three kinds, often called the **three pillars**:

Pillar	What it is	Answers	Spring tool
Logs	timestamped text or JSON events	"what exactly happened in this request?"	SLF4J + Logback (Java's standard logging API and Boot's default implementation of it), structured logging
Metrics	numbers aggregated over time (counts, durations, gauges)	"how many, how fast, how full — right now and as a trend?"	Micrometer `MeterRegistry`
Traces	the path of one request through many services, as a tree of timed **spans** (one span = one timed step of work, §15.7)	"where did this slow request spend its time?"	Micrometer Tracing + OpenTelemetry

Analogy: a hospital patient. The **metrics** are the monitor showing heart rate and blood pressure over time; the **logs** are the nurse's notes; the **trace** is the record of which departments the patient visited, in what order, and how long each took. You need all three: the monitor tells you *something* is wrong, the visit record tells you *where*, the notes tell you *what*.

Monitoring is the older, narrower idea: watching known metrics and alerting (sending a notification to a person) on thresholds (limits such as "error rate above 1%"). Observability adds the ability to ask new questions (e.g. "which customer's requests are slow?") from rich, correlated data.

How Spring wires it together



- **Micrometer** is a vendor-neutral **facade** (a single API in front of many interchangeable implementations) for metrics ("SLF4J for metrics"): you write to a `MeterRegistry`, and a registry implementation (Prometheus — the popular open-source metrics database, OTLP, Datadog, ...) formats and ships the numbers.
- **Micrometer Observation** is one API that produces *both* a metric and a span for a piece of work.
- **Micrometer Tracing** is a facade over a tracer. Boot 4 supports two: **OpenTelemetry** (`spring-boot-starter-opentelemetry`) and **Brave** with Zipkin (`spring-boot-starter-zipkin`).
- The tracer puts the current `traceId` and `spanId` into the logging **MDC** (Mapped Diagnostic Context, a per-thread map that log patterns can print), so every log line is linked to its trace.

Real-world examples

- Grafana's "LGTM" stack (Loki for logs, Grafana, Tempo for traces, Mimir/Prometheus for metrics) is a common open-source destination for exactly these three outputs.
- The OpenTelemetry Collector is widely deployed as a **sidecar** (a helper container that runs next to the app container in the same pod) or agent that receives OTLP (the OpenTelemetry protocol) and forwards to any **backend** (the storage and UI system where the data ends up), so apps don't hard-code the vendor.

Self-check

1. Name the three pillars and a question each answers. → Logs (what happened), metrics (how many/how fast over time), traces (where time went across services).
2. What is Micrometer? → A vendor-neutral metrics facade; you record to `MeterRegistry`, a registry implementation exports to the chosen backend.
3. How are logs linked to traces? → The tracer puts `traceId`/`spanId` in the MDC and the log pattern or JSON encoder prints them.

15.2 Spring Boot Actuator

What it is

Actuator adds **management endpoints** to your application: small built-in HTTP (or JMX — Java Management Extensions, the JVM's older built-in management protocol) APIs for operators (the people and tools that run the app in production) — health, metrics, info, loggers, environment, thread dumps (a snapshot of what every thread is doing) and more. Add `spring-boot-starter-actuator` and they appear under `/actuator`.

Analogy: the dashboard and service port of a car. Drivers (users) never see it; mechanics (operators, Kubernetes — the platform that runs and restarts containers, Prometheus) plug into it.

The endpoints

ID	Shows / does	Notes
`health`	UP/DOWN status, components, probes	the only endpoint exposed over HTTP by default
`info`	build, git, Java, custom `info.*` properties	contributors switched on by properties
`metrics`	list of meters; `/metrics/{name}` for one	JSON, for humans
`prometheus`	all meters in Prometheus text format	needs `micrometer-registry-prometheus`
`loggers`	view and **change** log levels at runtime	`POST /actuator/loggers/com.acme` `{"configuredLevel":"DEBUG"}`
`env`, `configprops`	resolved properties, `@ConfigurationProperties` beans	values masked by default (`show-values=never`)
`beans`, `conditions`, `mappings`	beans, auto-configuration report, request mappings	great for debugging (Chapter 3)
`threaddump`, `heapdump`	JVM thread dump, heap dump file (a copy of every object in memory)	heap dumps can contain secrets — protect them
`httpexchanges`	recent requests/responses	needs an `HttpExchangeRepository` bean
`scheduledtasks`, `caches`, `flyway`, `liquibase`, `quartz`, `sessions`	feature-specific	appear when the feature is present
`startup`	startup steps and their duration	needs `BufferingApplicationStartup`
`shutdown`	shuts the app down	access `none` by default

Exposure vs access

Two separate switches decide whether an endpoint can be called:

1. **Access** (since Boot 3.4, replacing the older `enabled` flags): is the endpoint allowed at all, and may it change things? Values `none`, `read-only`, `unrestricted`.
2. **Exposure**: is it reachable over **HTTP** or **JMX**? By default only `health` is exposed on both.

```
# expose more endpoints over HTTP ("*" = all; quote it in YAML)
management.endpoints.web.exposure.include=health,info,metrics,prometheus,env,loggers
management.endpoints.web.exposure.exclude=heapdump

# access control
management.endpoint.loggers.access=read-only        # can read levels, cannot change them
management.endpoints.access.default=none            # opt-in style: everything off ...
management.endpoint.health.access=read-only         # ... except what you list
management.endpoints.access.max-permitted=read-only # hard ceiling for the whole app

# a separate port (and optionally address) for all management endpoints
management.server.port=8081
management.endpoints.web.base-path=/manage          # default /actuator
```

A separate **management port** is common in Kubernetes: the public **ingress** (Kubernetes' entry point for traffic from outside the cluster) only routes to 8080, while probes and Prometheus reach 8081 inside the cluster.

Securing actuator endpoints

If Spring Security is on the classpath (the set of libraries the app loads), you decide who can call what with `EndpointRequest` matchers. In Boot 4 the class moved to `org.springframework.boot.security.autoconfigure.actuate.web.servlet.EndpointRequest` (it was `org.springframework.boot.actuate.autoconfigure.security.servlet` in Boot 3).

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
            // probes, info and the Prometheus scrape stay open (normally reachable only inside the cluster)
            .requestMatchers(EndpointRequest.to("health", "info", "prometheus")).permitAll()
            // every other actuator endpoint needs the OPS role
            .requestMatchers(EndpointRequest.toAnyEndpoint()).hasRole("OPS")
            .anyRequest().permitAll())
        .httpBasic(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable());   // demo API only; see chapter 10 for CSRF
    return http.build();
}
```

`EndpointRequest` matches by endpoint **ID**, so it keeps working if you change the base path or the management port. The test (`ActuatorEndpointsTests`) proves both sides:

```
anonymous /actuator/env -> 401, ops /actuator/loggers/ch15.obs -> {"effectiveLevel":"INFO"}
```

Common mistakes

- `management.endpoints.web.exposure.include=*` on a public port without security: `env`, `heapdump` and `threaddump` can leak secrets and internals. Expose the minimum; protect the rest.
- Thinking exposure is enough — an endpoint with access `none` stays unavailable even when exposed.
- Forgetting that `health` details (`show-details`) can reveal hostnames, disk paths and versions (you will see a disk path in the output below).

Self-check

1. Which endpoints are exposed over HTTP by default? → Only `health`.
2. Difference between exposure and access? → Access decides if the endpoint exists and whether it may write; exposure decides whether HTTP/JMX can reach it.
3. Why a separate management port? → Keep operational endpoints off the public ingress while probes and scrapers reach them internally.

15.3 Health, health groups and Kubernetes probes

What it is

The `health` endpoint aggregates **health indicators** — small checks such as "can I reach the database?" or "is there disk space?" — into one status. Status values are `UP`, `DOWN`, `OUT_OF_SERVICE` and `UNKNOWN`; the overall status is the "worst" one (default order: `DOWN`, `OUT_OF_SERVICE`, `UP`, `UNKNOWN`). `DOWN` and `OUT_OF_SERVICE` are returned with HTTP **503**, others with 200.

Auto-configured indicators appear when their technology is present: `ping`, `diskSpace`, `db` (a `DataSource`), `redis`, `mongo`, `rabbit`, `mail`, `ssl` (certificate validity), `livenessState`, `readinessState`, and more.

Why it matters

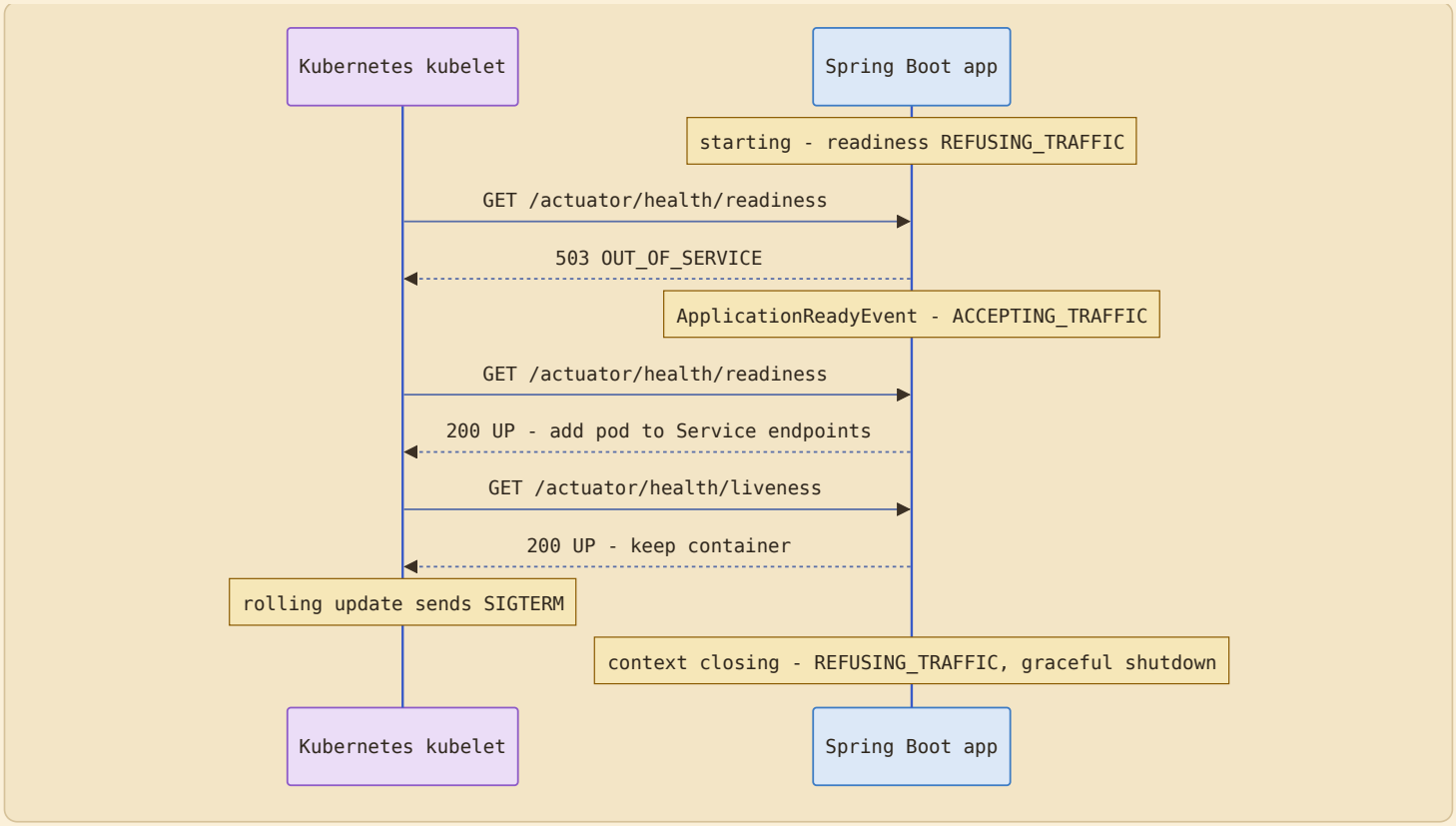
Load balancers (components that spread requests over instances) and **orchestrators** (platforms such as Kubernetes that start, stop and move containers) make decisions from these answers: take an instance out of rotation (stop sending it requests), restart a container, stop a **rolling deployment** (replacing instances with the new version a few at a time). See System Design Fundamentals. A wrong health design can **cause** outages (periods when the service is unavailable): if every instance reports DOWN because one optional dependency is slow, the platform removes all of them.

Liveness vs readiness

Kubernetes asks two different questions:

Probe	Question	On failure Kubernetes...	Should include
Liveness	"Is this process broken beyond repair?"	**restarts** the container	only internal state (deadlock — threads stuck waiting for each other forever, corrupted state) — never external systems
Readiness	"Can this instance serve traffic right now?"	**stops routing** traffic to it (no restart)	warming up, shutting down, and — carefully — dependencies it cannot work without
Startup (K8s only)	"Has it finished starting?"	holds off the other probes	usually point it at the liveness endpoint with a larger threshold

Spring Boot tracks two **availability states** for you: `LivenessState` (`CORRECT` / `BROKEN`) and `ReadinessState` (`ACCEPTING_TRAFFIC` / `REFUSING_TRAFFIC`). Boot marks the app ready only after startup (including `CommandLineRunner`s) has finished, and switches readiness to `REFUSING_TRAFFIC` when the application context starts closing. **In Boot 4 the probe groups are enabled by default** — `/actuator/health/liveness` and `/actuator/health/readiness` exist without any configuration (in Boot 2.3–3.x they were enabled automatically only on Kubernetes, or with `management.endpoint.health.probes.enabled=true`).



A custom `HealthIndicator`

In Boot 4 the health API moved to its own module: `org.springframework.boot.health.contributor.HealthIndicator` and `Health` (Boot 3: `org.springframework.boot.actuate.health`).

```
// Bean name "inventoryHealthIndicator" -> health component name "inventory"
@Component
public class InventoryHealthIndicator implements HealthIndicator {

    private final InventoryStatus status;   // pretend connection state, flipped by the tests

    public InventoryHealthIndicator(InventoryStatus status) {
        this.status = status;
    }

    @Override
    public Health health() {
        if (status.reachable()) {
            return Health.up().withDetail("latencyMs", 12).build();
        }
        return Health.down().withDetail("error", "inventory service not reachable").build();
    }
}
```

Keep `health()` **fast and cheap** — it runs on every probe (typically every 5–10 s per instance). For slow checks, cache the last result or compute it in the background. For reactive apps implement `ReactiveHealthIndicator`.

Health groups

A **group** is a named subset of indicators with its own settings, served at `/actuator/health/{group}`. Liveness and readiness are just two built-in groups.

```
management.endpoint.health.show-details=when-authorized     # never | when-authorized | always
management.endpoint.health.roles=OPS
# a custom group, e.g. for a deployment smoke check
management.endpoint.health.group.deploy.include=ping,inventory
management.endpoint.health.group.deploy.show-details=always
# readiness also fails when inventory is down (a deliberate choice - see Trade-offs)
management.endpoint.health.group.readiness.include=readinessState,inventory
```

Other group options: `exclude`, `status.order`, `status.http-mapping.*`, `additional-path` (serve the group on the main port, e.g. `server:/healthz`). `management.endpoint.health.probes.add-additional-paths=true` also serves `/livez` and `/readyz` on the main server port.

Proof

The tests call the endpoints anonymously and as the `ops` user, then make "inventory" unreachable:

```
anonymous /actuator/health -> {"groups":["deploy","liveness","readiness"],"status":"UP"}
ops       /actuator/health -> {"components":{"diskSpace":{"details":
{"total":494384795648,"free":63519252480,"threshold":10485760,"path":".../observability-demo/.","exists":true},"status":"UP"},"inventory":
{"details":{"latencyMs":12},"status":"UP"},"livenessState":{"status":"UP"},"ping":{"status":"UP"},"readinessState":{"status":"UP"},"ssl":
{"details":{"expiringChains":[],"invalidChains":[],"validChains":[]},"status":"UP"}},"groups":
["deploy","liveness","readiness"],"status":"UP"}
inventory reachable:
  liveness  -> {"status":"UP"}
  readiness -> {"status":"UP"}
inventory DOWN:
  liveness  -> {"status":"UP"}
  readiness -> {"status":"DOWN"} (HTTP 503)
  deploy    -> {"components":{"inventory":{"details":{"error":"inventory service not reachable"},"status":"DOWN"},"ping":
{"status":"UP"}},"status":"DOWN"} (HTTP 503)
```

Things to notice:

- Anonymous callers get only the status and the group names — details need the `OPS` role (`when-authorized`).
- The `liveness` group exists though we never configured it: probes are on by default in Boot 4.
- Liveness stays UP while a dependency is down — restarting our container would not fix the inventory service.

The application can also change its own state, for example to drain traffic before maintenance:

```
AvailabilityChangeEvent.publish(context, ReadinessState.REFUSING_TRAFFIC);
```

```
after REFUSING_TRAFFIC: readiness -> {"status":"OUT_OF_SERVICE"} (HTTP 503)
```

Kubernetes configuration (not run here):

```
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 8081 }
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8081 }
  periodSeconds: 5
  failureThreshold: 3
startupProbe:
  httpGet: { path: /actuator/health/liveness, port: 8081 }
  periodSeconds: 5
  failureThreshold: 30        # up to 150 s to start before liveness takes over
```

Trade-offs

Putting a dependency in **readiness** protects users from an instance that cannot serve — but if *all* instances share that dependency, all become unready at once and the service disappears entirely, even for requests that don't need it. Include a dependency in readiness only if the instance is truly useless without it; otherwise stay ready and **degrade** (keep working with reduced features: fallbacks, cached data, circuit breakers — guards that stop calling a failing service for a while, Chapter 16).

Common mistakes

- Database check in **liveness** → a DB blip restarts every pod (a **pod** is Kubernetes' unit for one running instance), making recovery slower (a "restart storm").
- Slow indicators (calling a remote API synchronously on every probe) → probe timeouts → false failures.
- Same thresholds for liveness and readiness → a slow start gets the container killed; use a startup probe.
- `show-details=always` on a public port → leaks infrastructure details.

In the interview

Q: What's the difference between liveness and readiness, and how does Spring Boot support them? 30-second answer: Liveness asks "should the container be restarted?", readiness asks "should it receive traffic?". Boot tracks `LivenessState` and `ReadinessState`, exposes them as health groups at `/actuator/health/liveness` and `/readiness` (on by default in Boot 4), marks readiness only after startup completes and refuses traffic during shutdown. External dependencies never belong in liveness.

- *Follow-up: how do you add a custom check?* → A `HealthIndicator` bean returning `Health.up()/down().withDetail(...)`, then include it in a group if needed.
- *Follow-up: how do you take one instance out of rotation without killing it?* → Publish `AvailabilityChangeEvent` with `ReadinessState.REFUSING_TRAFFIC`.

Self-check

1. Which HTTP status does a DOWN health group return? → 503.
2. Where does a custom indicator's name come from? → The bean name minus the `HealthIndicator` suffix.
3. Why not include the database in liveness? → A restart doesn't fix the database, and restarting all pods at once worsens the outage.

TL;DR

`HealthIndicator` beans → `/actuator/health`; groups (`management.endpoint.health.group.<name>.include`) → `/actuator/health/<name>`; liveness/readiness groups are on by default in Boot 4; keep liveness internal-only, be careful with dependencies in readiness.

15.4 The info endpoint

`/actuator/info` shows descriptive facts about the running build — invaluable when you ask "which version is actually deployed?". It is assembled from **info contributors**:

Contributor	Content	Enabled by
`build`	group, artifact, version, build time	present when `META-INF/build-info.properties` exists (Maven plugin goal `build-info`)
`git`	branch, commit id and time	present when `git.properties` exists (`git-commit-id-maven-plugin`)
`env`	every `info.*` property	`management.info.env.enabled=true`
`java`	JVM and runtime vendor/version	`management.info.java.enabled=true`
`os`, `process`	OS name/version, PID (process id), CPUs, memory	`management.info.os.enabled`, `management.info.process.enabled`
custom	anything	an `InfoContributor` bean

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <execution><goals><goal>build-info</goal></goals></execution>
  </executions>
</plugin>
```

```
management.info.env.enabled=true
management.info.java.enabled=true
info.app.description=Orders demo for chapter 15
```

```
/actuator/info -> {"app":{"description":"Orders demo for chapter 15"},"build":{"artifact":"ch15-observability-demo","name":"ch15-observability-demo","time":"2026-09-28T23:50:25.503Z","version":"1.0.0","group":"handbook.spring"},"java":{"jvm":{"name":"OpenJDK 64-Bit Server VM","vendor":"Homebrew","version":"25.0.2"},"runtime":{"name":"OpenJDK Runtime Environment","version":"25.0.2"},"vendor":{"name":"Homebrew","vers...
```

Never put secrets in `info.*` — the endpoint is usually public.

15.5 Metrics with Micrometer

What it is

A **metric** is a number measured repeatedly over time. Micrometer records metrics in **meters** registered in a `MeterRegistry`. Each meter has a **name** and **tags** (key/value dimensions, e.g. `status=200`) — every unique combination of name + tag values is its own **time series**.

Meter	Measures	Example	Prometheus output
`Counter`	a count that only increases	orders placed, errors	`name_total`
`Timer`	count + total time + max of short events	request latency	`name_seconds_count`, `_sum`, `_max` (+ buckets if histogram on: counts of values per range, e.g. "under 100 ms")
`Gauge`	a current value sampled when read	queue size, pool usage	`name`
`DistributionSummary`	distribution of non-time values	payload size	`name_count`, `_sum`, `_max`
`LongTaskTimer`	tasks *currently* running and for how long	a batch job in progress	`name_active_count`, `_duration_sum`

Analogy: a car. The odometer is a **counter**, the fuel needle a **gauge**, the lap-time stopwatch a **timer**.

What Boot gives you for free

With Actuator on the classpath, Boot registers JVM memory/GC/threads/classes, CPU, uptime, Tomcat sessions, HikariCP pool, logback events, `http.server.requests` (every MVC/WebFlux request, tagged `method`, `uri` template, `status`, `outcome`, `exception`), `http.client.requests` (for `RestClient`/`WebClient` built from Boot's builders), cache, Kafka and more.

Custom meters

From `OrderService.java`:

```
public OrderService(MeterRegistry registry) {
    this.registry = registry;
    // A Timer records how many calls and how long they took
    this.placeTimer = Timer.builder("orders.place")
            .description("time to place an order")
            .register(registry);
    // A Gauge samples a current value on demand (here: the map size)
    registry.gauge("orders.stored", orders, Map::size);
}

@Observed(name = "order.lookup", contextualName = "order-lookup")   // see 15.6
public Optional<String> find(long id) {
    log.info("looking up order {}", id);
    Optional<String> item = Optional.ofNullable(orders.get(id));
    // A Counter only goes up; the tag splits it into one time series per result
    Counter.builder("orders.lookups")
            .tag("result", item.isPresent() ? "found" : "missing")
            .register(registry)             // returns the existing counter if already registered
            .increment();
    return item;
}

public long place(String item) {
    return placeTimer.record(() -> { /* ... store it ... */ });
}
```

Common tags for every meter come from properties:

```
management.metrics.tags.region=eu-west-1
# optional: publish histogram buckets so Prometheus can compute p95/p99 across instances
management.metrics.distribution.percentiles-histogram.http.server.requests=true
```

Reading metrics: `/actuator/metrics` and `/actuator/prometheus`

The test makes two successful lookups, one miss and one order, then reads both endpoints:

```
/actuator/metrics/orders.lookups -> {"availableTags":[{"tag":"result","values":["missing","found"]},{"tag":"region","values":["eu-west-
1"]}],"measurements":[{"statistic":"COUNT","value":3.0}],"name":"orders.lookups"}
/actuator/prometheus (selected lines):
  http_server_requests_seconds_count{error="none",exception="none",method="GET",outcome="CLIENT_ERROR",region="eu-west-
1",status="404",uri="/orders/{id}"} 1
  http_server_requests_seconds_count{error="none",exception="none",method="GET",outcome="SUCCESS",region="eu-west-
1",status="200",uri="/orders/{id}"} 2
  order_lookup_seconds_count{class="ch15.obs.OrderService",error="none",method="find",region="eu-west-1"} 3
  orders_lookups_total{region="eu-west-1",result="found"} 2.0
  orders_lookups_total{region="eu-west-1",result="missing"} 1.0
  orders_place_seconds_count{region="eu-west-1"} 1
  orders_stored{region="eu-west-1"} 3.0
```

What to notice:

- `/actuator/metrics/{name}` aggregates across tags (COUNT 3.0); drill down with `?tag=result:found`.
- **Naming conventions** are translated per backend: Micrometer names use dots (`orders.lookups`); Prometheus gets `orders_lookups_total` (counter suffix), timers become `_seconds_*`.
- The `uri` tag is the **template** `/orders/{id}`, not `/orders/1` — otherwise every id would create a new time series.
- Every meter carries the common `region` tag.
- `@Observed` produced the timer `order_lookup_seconds` without any `MeterRegistry` code.

Push vs pull. Prometheus **pulls** (scrapes) `/actuator/prometheus` every 15–60 s. Other backends are **pushed** to, e.g. OTLP with `management.otlp.metrics.export.url=http://collector:4318/v1/metrics`. In `@SpringBootTest`, Boot switches metric exporters off (only an in-memory registry stays) unless the class has `@AutoConfigureMetrics` — that's why the Prometheus test uses it.

Cardinality — the one rule that matters

Cardinality = the number of distinct tag-value combinations. Each one is a separate time series stored by the monitoring system. Tags must have a **small, bounded** set of values (HTTP method, status, region, result). Never tag with user ids, order ids, emails, full URLs or timestamps — a million users means a million series, which can crash Prometheus or explode a SaaS (software-as-a-service, i.e. a hosted, pay-per-use monitoring product) bill. Put such values in **traces or logs** instead.

Common mistakes

- High-cardinality tags (see above).
- Creating a new `Timer.builder(...).register(...)` in a hot loop is fine (it returns the cached meter), but building meters with *dynamic names* is not — use tags.
- Averages for latency. Use percentiles (p95, p99: the time that 95% / 99% of requests stay under; see System Design Fundamentals) from histograms; averages hide the slow tail that users feel.
- Gauges on objects that get garbage-collected (freed automatically by the JVM once nothing uses them): `registry.gauge(name, obj, fn)` holds a **weak** reference (one that does not stop the object being freed); if nothing else holds `obj`, the gauge reports `NaN`.

Self-check

1. Counter vs gauge? → Counter only increases (rate over time); gauge is a current value that goes up and down.
2. Why does `http.server.requests` tag `uri="/orders/{id}"`? → To keep cardinality bounded — the template, not the concrete path.
3. How does Prometheus get the data? → It scrapes `/actuator/prometheus` (pull).

15.6 The Observation API and `@Observed`

What it is

An **Observation** (Micrometer 1.10+, Boot 3+) wraps a unit of work — "look up an order" — once, and **handlers** decide what to produce from it: a `Timer` (metrics handler), a span (tracing handler), a log line, or anything else. Spring's own instrumentation (MVC, WebFlux, `RestClient`, `WebClient`, JDBC via extensions, Spring Security, Kafka, `@Scheduled`...) is written this way, which is why one dependency change gives you both metrics and traces.

```
Observation.createNotStarted("order.lookup", observationRegistry)
        .lowCardinalityKeyValue("channel", "web")     // -> metric tag AND span attribute
        .highCardinalityKeyValue("orderId", "42")     // -> span attribute only
        .observe(() -> service.find(42));
```

- **Low-cardinality key values** become metric tags *and* span attributes.
- **High-cardinality key values** go only to spans (safe for ids).

`@Observed(name = "order.lookup", contextualName = "order-lookup")` on a bean method does the same through an aspect (AOP code that runs around annotated methods, Chapter 4). It needs the AspectJ starter (`spring-boot-starter-aspectj` in Boot 4) and, since Boot 3.2, the property:

```
management.observations.annotations.enabled=true   # also activates @Timed, @Counted, @NewSpan
```

The metric is named after `name` (`order.lookup` → `order_lookup_seconds_*` above, tagged `class` and `method`); the span is named after `contextualName`:

```
spans for GET /orders/2 -> [http get /orders/{id}, order-lookup]
```

Tuning which observations happen

```
# switch off whole families by name prefix
management.observations.enable.spring.security=false
# common key values added to every observation
management.observations.key-values.region=eu-west-1
```

Spring Security creates several observations per request (filter chain before/after, authentication, authorization). In the demo's first run they made up most of the exported spans; disabling the `spring.security` prefix keeps the traces readable. For finer control register an `ObservationPredicate` bean.

Context propagation. Observations live in thread-local state. For `@Async` methods on Boot's executor set `spring.task.execution.propagate-context=true` (or register a `ContextPropagatingTaskDecorator` on your own executor); for Reactor use `spring.reactor.context-propagation=auto`.

Self-check

1. Low vs high cardinality key values? → Low go to metrics and spans; high only to spans.
2. What must you add for `@Observed` to work? → AspectJ starter + `management.observations.annotations.enabled=true`.

15.7 Distributed tracing

What it is

A **trace** follows one request across every service it touches. It is a tree of **spans**; each span is one timed operation (an incoming HTTP request, an outgoing call, a DB query) with a **span id**, the **trace id** shared by the whole tree, a **parent span id**,

start/end time and attributes.

Analogy: a parcel tracking number. Every depot that handles the parcel scans the same number (trace id) and adds its own entry (span) with arrival and departure times.

How it works

1. A request arrives without trace headers → the server observation starts a **root span** with a new 128-bit trace id.
2. The trace context is stored in the current thread and copied into the log MDC (`traceId`, `spanId`).
3. An outgoing call through an instrumented client (Boot's `RestClient.Builder`, `WebClient.Builder`, `RestTemplateBuilder`, Kafka templates...) creates a **client span** and injects the context into headers — by default the **W3C Trace Context** header (a web standard from the W3C, the body behind HTML and CSS) `traceparent: 00-<traceId>-<spanId>-01`.
4. The downstream service reads the header and continues the same trace with a child span.
5. Finished spans go to a **span processor** → **exporter** (OTLP to a collector, Zipkin, ...). Boot batches them (`BatchSpanProcessor`).
6. The backend (Tempo, Jaeger, Zipkin) stitches spans by trace id into a timeline.

Sampling decides which traces are recorded: `management.tracing.sampling.probability` (default **0.1** = 10%). The decision is made at the root and travels in the header, so a trace is either complete or absent.

Setup in Boot 4

```
<!-- Micrometer Tracing + OpenTelemetry bridge + OTLP exporters -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-opentelemetry</artifactId>
</dependency>
<!-- tests: spring-boot-starter-opentelemetry-test (provides @AutoConfigureTracing) -->
```

```
management.tracing.sampling.probability=1.0     # 100% in dev/test; lower in high-traffic prod
# send spans to an OpenTelemetry Collector (not used in the demo - no collector running)
# management.opentelemetry.tracing.export.otlp.endpoint=http://otel-collector:4318/v1/traces
```

The Brave/Zipkin alternative is `spring-boot-starter-zipkin` with `management.tracing.export.zipkin.endpoint`. Boot 3 users: the dependency used to be `micrometer-tracing-bridge-otel` + `opentelemetry-exporter-otlp`; the one starter replaces both.

A "logging exporter" instead of a collector

To see spans without running Jaeger, the demo registers its own `SpanExporter` bean; Boot adds every `SpanExporter` bean to its batch processor:

```
@Component
public class PrintingSpanExporter implements SpanExporter {
    private final List<SpanData> finished = new CopyOnWriteArrayList<>();

    @Override
    public CompletableResultCode export(Collection<SpanData> spans) {
        finished.addAll(spans);                      // a real exporter would send them over OTLP
        return CompletableResultCode.ofSuccess();
    }
    // ... flush(), shutdown(), finished(), clear()
}
```

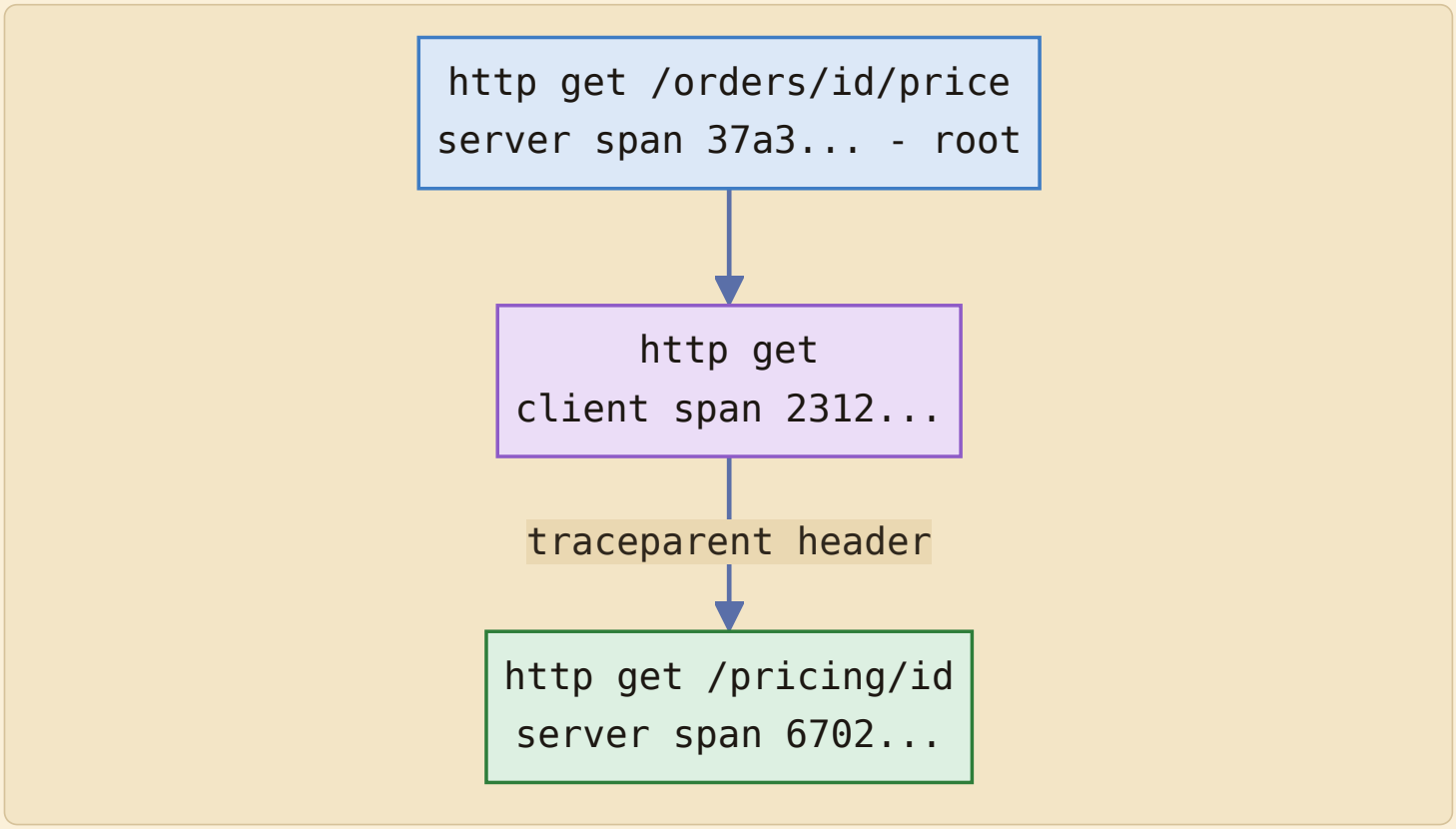
The controller calls a second endpoint over HTTP with Boot's `RestClient.Builder`, standing in for a downstream service:

```
@GetMapping("/orders/{id}/price")
public String price(@PathVariable long id) {
    log.info("asking pricing service for order {}", id);
    String base = "http://localhost:" + env.getProperty("local.server.port");
    return restClientBuilder.build().get()
            .uri(base + "/pricing/{id}", id)
            .retrieve().body(String.class);
}

@GetMapping("/pricing/{id}")
public String pricing(@PathVariable long id) {
    log.info("pricing order {}", id);
    return "price for " + id + " = 49.99";
}
```

`TracingTests` (with `@AutoConfigureTracing`, because tests disable span export by default) calls `/orders/1/price`, flushes the batch processor and prints the log lines and spans:

```
response: price for 1 = 49.99
log lines (trace id - span id in brackets):
  [orders] [o-auto-1-exec-8] [89fb1d6934d0cf54e0910695a8df7859-37a3311dfdcc61c3] ch15.obs.OrderController               : asking pricing
service for order 1
  [orders] [o-auto-1-exec-9] [89fb1d6934d0cf54e0910695a8df7859-6702c6550d5a9bfd] ch15.obs.OrderController               : pricing order 1
finished spans:
  http get /orders/{id}/price  trace=89fb1d6934d0cf54e0910695a8df7859 span=37a3311dfdcc61c3 parent=0000000000000000
  http get                     trace=89fb1d6934d0cf54e0910695a8df7859 span=2312285359b3b845 parent=37a3311dfdcc61c3
  http get /pricing/{id}       trace=89fb1d6934d0cf54e0910695a8df7859 span=6702c6550d5a9bfd parent=2312285359b3b845
```

Read it as a tree:



- Two different Tomcat threads (`exec-8`, `exec-9`) logged with the **same trace id** — that is propagation through the `traceparent` header.
- Each log line's span id matches the span that was active (`37a3...` for the outer request, `6702...` for the downstream one).
- Boot's default console pattern prints `[application-name] [thread] [traceId-spanId]`; change it with `logging.pattern.correlation`.
- Across *real* services the picture is identical; only the exporter sends the spans to a shared backend.

Baggage is extra key/value context that travels with the trace (e.g. a tenant id): `management.tracing.baggage.remote-fields=tenant` sends it in headers, `management.tracing.baggage.correlation.fields=tenant` also puts it in the MDC.

Trade-offs

Choice	Pro	Con
Sampling 100%	every request traceable	CPU, network and storage cost at scale
Head sampling (e.g. 10%, Boot default)	cheap, simple	the rare failing request is often not sampled
Tail sampling in the OTel Collector	keep all errors/slow traces, drop the rest	collector must buffer whole traces
OTel **Java agent** (a jar attached at JVM start that instruments classes automatically) instead of Micrometer	zero code, instruments many libraries	a second instrumentation layer; version coupling with the JVM agent

Common mistakes

- Building `RestClient.builder()` / `new RestTemplate()` yourself → no instrumentation, the trace breaks at that hop. Inject Boot's builder.
- Losing context on your own thread pools or `CompletableFuture` → wrap executors with context propagation.
- Using trace ids as metric tags → cardinality explosion.
- Sampling 1.0 in a busy production system without a plan for storage.

In the interview

Q: How does distributed tracing work in Spring Boot? 30-second answer: Micrometer Observation instruments incoming and outgoing calls; Micrometer Tracing with the OpenTelemetry (or Brave) bridge turns observations into spans. A trace id is created at the edge, propagated in the W3C `traceparent` header by instrumented clients, and put in the log MDC. Spans are batched and exported via OTLP to a collector and viewed in Tempo/Jaeger/Zipkin. Sampling defaults to 10%.

- *Follow-up: why is my trace broken between two services?* → The client isn't instrumented (created with `new`), a proxy strips headers, the propagation format differs (W3C vs B3, Zipkin's older header format), or async code lost the context.
- *Follow-up: logs vs traces?* → Traces show timing and causality across services; logs give detail within one; the shared trace id links them.

Self-check

1. What header carries the context by default? → W3C `traceparent`.
2. Default sampling probability? → 0.1.
3. What is a parent span id of `0000000000000000`? → A root span (no parent).

TL;DR

`spring-boot-starter-opentelemetry` + `management.tracing.sampling.probability`; inject Boot's client builders; trace/span ids appear in logs automatically; export via `management.opentelemetry.tracing.export.otlp.endpoint`.

15.8 Structured logging

What it is

Structured logging writes each event as a machine-readable record (JSON) instead of a formatted line. Log platforms (Elasticsearch/Kibana, Loki, Graylog, Datadog, Cloud Logging) can then filter on fields — `log.level:ERROR AND service.name:orders AND traceId:...` — instead of parsing text with regexes (regular expressions: text-matching patterns). Built into Boot since **3.4**.

```
logging.structured.format.console=ecs        # ecs | logstash | gelf
logging.structured.format.file=logstash      # file output can use a different format
logging.structured.ecs.service.version=1.0.0
logging.structured.ecs.service.environment=test
```

Format	Standard	Typical destination
`ecs`	Elastic Common Schema	Elasticsearch / Elastic Agent, also fine for Loki
`logstash`	Logstash JSON	Logstash, Fluentd/Fluent Bit
`gelf`	Graylog Extended Log Format	Graylog

`StructuredLoggingTests` starts the app with `ecs` and captures the log line of one request:

```
ECS log line:
{"@timestamp":"2026-09-28T23:50:30.319867Z","log":{"level":"INFO","logger":"ch15.obs.OrderService"},"process":{"pid":17381,"thread":
{"name":"http-nio-auto-2-exec-1"}},"service":{"name":"orders","version":"1.0.0","environment":"test","node":{}},"message":"looking up order
1","traceId":"636cb6141b8e904e29a6f7824c3eb713","spanId":"8ceaaf80e43135b1","ecs":{"version":"8.11"}}
```

`service.name` comes from `spring.application.name`; all MDC entries — here `traceId` and `spanId` from the tracer — become fields automatically, as do key/values from SLF4J's fluent API (`log.atInfo().addKeyValue("orderId", id).log("placed")`).

Customise the JSON without code:

```
logging.structured.json.exclude=process.pid
logging.structured.json.rename.process.thread.name=thread
logging.structured.json.add.team=checkout
logging.structured.json.stacktrace.root=first          # root cause first
logging.structured.json.stacktrace.max-throwable-depth=20
```

For full control implement `StructuredLogFormatter<ILoggingEvent>` and name the class in `logging.structured.format.console`.

When not to use it: local development in a terminal — JSON is hard for humans to read. A common setup is plain text by default and JSON in the `prod` profile (`application-prod.properties`), or JSON to a file and text to the console.

Common mistakes: logging secrets or personal data (JSON makes them *easier* to search — mask them); multi-line stack traces in text logs that the shipper (the agent that collects log output and sends it to the log platform) splits into many events (JSON keeps them in one field); logging at DEBUG in production for hot paths (cost and noise). The demo also shows a practical catch: the logging system is global to the JVM, so the structured-logging test runs last (`junit-platform.properties` orders the classes).

15.9 Graceful shutdown

What it is

When an instance is stopped (a deployment, scale-down, node drain — moving everything off a machine before maintenance), requests that are **in flight** should finish instead of failing. **Graceful shutdown** stops accepting new connections, waits for active requests up to a timeout, then stops. It is the **default since Boot 3.4** for Tomcat, Jetty and Reactor Netty (`server.shutdown=graceful`); `server.shutdown=immediate` turns it off.

```
spring.lifecycle.timeout-per-shutdown-phase=10s   # default 30s
```

How it works

1. The JVM receives `SIGTERM` — the operating system's polite "please stop" signal, sent by Kubernetes and `docker stop` (Ctrl+C sends the similar `SIGINT`); unlike `SIGKILL` it lets the JVM run its shutdown hooks → Spring's shutdown hook closes the application context.
2. Readiness switches to `REFUSING_TRAFFIC`.
3. The web server's lifecycle bean stops first: it stops accepting new connections at the network level and waits for active requests.
4. When they finish (or the timeout expires), the rest of the context closes: `@PreDestroy`, connection pools, executors.

Proof

`GracefulShutdownTests` starts the app, begins a request to `/slow?ms=1000`, closes the context while it runs (as `SIGTERM` would), and tries a new request 200 ms later — once with each mode:

```
ConfigurableApplicationContext ctx = SpringApplication.run(RuntimeApplication.class,
        "--server.port=0", "--server.shutdown=" + shutdownMode, "--spring.main.banner-mode=off");
// ... start a 1-second request, wait until it is inside the controller ...
Thread closer = Thread.ofPlatform().start(ctx::close);        // like SIGTERM from Kubernetes
Thread.sleep(200);
// ... try a new request, then wait for both ...
```

```
server.shutdown=immediate in-flight -> FAILED: IOException | new request during shutdown -> rejected (ConnectException) | close took < 0.7 s (did not wait)
server.shutdown=graceful  in-flight -> HTTP 200 'done after 1000 ms' | new request during shutdown -> rejected (ConnectException) | close took ~1 s (waited)
  log: Commencing graceful shutdown. Waiting for active requests to complete
  log: Graceful shutdown complete
```

In Kubernetes

There is a race: Kubernetes sends `SIGTERM` and removes the pod from the Service endpoints **at the same time**, and load balancers may still send new requests for a few seconds. Since the server refuses new connections immediately, add a short delay before shutdown starts:

```
spec:
  terminationGracePeriodSeconds: 45          # must exceed preStop + Spring timeout
  containers:
    - name: orders
      lifecycle:
        preStop:
          sleep: { seconds: 10 }             # K8s 1.30+; older: exec ["sh","-c","sleep 10"]
```

Common mistakes

- `timeout-per-shutdown-phase` longer than Kubernetes' `terminationGracePeriodSeconds` (default 30 s) → the pod is `SIGKILL`ed mid-drain.
- Expecting graceful shutdown to finish **message consumers** or `@Async` jobs — configure those separately (listener container stop, `spring.task.execution.shutdown.await-termination=true`).
- Running from an IDE that kills the process instead of sending SIGTERM (then shutdown looks immediate).

Self-check

1. Is graceful shutdown on by default? → Yes, since Boot 3.4.
2. What happens to new requests during the grace period? → They are refused at the network layer (connection refused), not answered with 503.
3. Why a `preStop` sleep? → To let endpoint removal propagate before the server stops accepting connections.

15.10 Virtual threads

Virtual threads (final in Java 21, Java book 15.4) are cheap threads managed by the JVM; a blocked virtual thread releases its carrier OS thread. For blocking servlet apps this raises the number of concurrent requests you can hold without a huge thread pool. In Boot (3.2+) one property switches them on:

```
spring.threads.virtual.enabled=true
```

It makes Tomcat/Jetty run each request on a virtual thread and switches Boot's auto-configured `@Async` executor, `@Scheduled` scheduler and several messaging listener containers to virtual threads (the `@Async` side is shown in 14.5). `VirtualThreadsTests` calls an endpoint that reports `Thread.currentThread()`:

```
spring.threads.virtual.enabled=true  -> virtual thread: VirtualThread[#100,tomcat-handler-0]/runnable@ForkJoinPool-1-worker-1
default                              -> platform thread: Thread[#108,http-nio-auto-4-exec-1,5,main]
```

Caveats.

- Your **bottleneck** (the one slowest part that limits the whole system) **moves**: 10,000 concurrent requests now reach the database pool (10 connections by default) or a downstream API. Limit concurrency explicitly — `@ConcurrencyLimit` (4.7), pool sizes, bulkheads.
- Pinning: before JDK 24, blocking inside `synchronized` pinned the carrier thread; JDK 24 (JEP 491) removed that for `synchronized`. Native calls can still pin.
- Virtual threads are daemon threads: an app whose only work is `@Scheduled` tasks may exit; set `spring.main.keep-alive=true`.
- CPU-bound work gains nothing. Virtual threads vs reactive: see Chapter 13.

15.11 Faster startup: extracted jars, CDS, AOT cache, native images

The problem it solves

A typical Boot app takes 1–5 s to start on the JVM (class loading, reflection, bean creation, JIT warm-up). That matters for autoscaling (adding and removing instances automatically with load), serverless (scale-to-zero: no instance runs until a request arrives) and fast rollbacks (returning to the previous version). Options, from least to most invasive:

Technique	What it does	Startup gain	Cost
Extracted jar	run from unpacked `lib/` instead of nested jars	small	a build step
CDS (Class Data Sharing; application classes since JDK 10, dynamic archives since JDK 13)	archive of parsed classes from a training run (a short run of the app whose only purpose is to record what it loads)	~20–40%	training run, same JDK/classpath
JDK AOT cache (JEP 483 in JDK 24, simpler flags JEP 514 and method profiles JEP 515 in JDK 25)	caches classes *loaded and linked* (plus profiles) from a training run	often ~2×	training run; cache tied to the JDK build and exact jars
Spring AOT on the JVM	generates bean-definition code at build time (`spring-boot:process-aot`), run with `-Dspring.aot.enabled=true`	modest	profiles/conditions fixed at build time
GraalVM native image	compiles the whole app ahead of time to a native executable	~50–100× (tens of ms)	long builds, closed-world limits, reflection hints, lower peak throughput (maximum requests per second) without PGO (profile-guided optimisation)

Measured: AOT cache on JDK 25

`aot-cache-demo.sh` builds `production-runtime`, extracts it and times start-until-context-refreshed (`-Dspring.context.exit=onRefresh` exits right after the context refresh, a Framework 6.1+ feature made for training runs):

```
java -Djarmode=tools -jar target/app.jar extract --destination target/app   # app jar + lib/
cd target/app
# training run: record and write the cache in one step (JDK 25)
java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar app.jar
# production run
java -XX:AOTCache=app.aot -jar app.jar
```

Run it as `bash aot-cache-demo.sh <JDK-25-home>` (the script otherwise uses `$JAVA_HOME`). Rows 2 and 3 are the median of 5 runs. Output on this machine (Apple silicon laptop, Homebrew OpenJDK 25.0.2; **timings are machine-dependent** — measure your own):

```
java: openjdk version "25.0.2" 2026-01-20
1. fat jar (java -jar)               : 1152 ms (mean of 3)
2. extracted jar                     : 934 ms
   AOT cache file                    :  56M
3. extracted jar + -XX:AOTCache      : 467 ms
```

The AOT cache cut JVM-to-context-ready time by more than half for this small web app, at the price of a 56 MB file and a training step in the build. For CDS on JDK 13–24 the equivalent flags are `-XX:ArchiveClassesAtExit=app.jsa` (training) and `-XX:SharedArchiveFile=app.jsa`. The cache is only valid for the same JDK build and the same jars; if they differ, the JVM ignores it (with a warning) and starts normally.

GraalVM native image (not run — `native-image` is not installed here)

```
# needs GraalVM / Liberica NIK 25; spring-boot-starter-parent provides the "native" profile
mvn -Pnative native:compile          # -> target/<artifactId>, a native executable
mvn -Pnative spring-boot:build-image # same, inside a container image (needs Docker)
```

Plus the `org.graalvm.buildtools:native-maven-plugin` in the POM (`pom.xml`, Maven's project file). The build runs Spring AOT, then GraalVM analyses everything reachable from `main` (**closed world**): classes not seen at build time cannot be loaded later; reflection, resources, proxies and serialization need **hints** (Spring generates most; add your own with `RuntimeHintsRegistrar` or `@RegisterReflectionForBinding`). Profiles and `@ConditionalOnProperty` are evaluated at build time. Test the native build in CI (`mvn -PnativeTest test`) because some libraries fail only there.

Choose native for CLIs (command-line tools), functions (small serverless handlers), scale-to-zero services and memory-tight deployments. **Stay on the JVM** (with the AOT cache) for long-running, throughput-heavy services, where the JIT's peak performance and easy debugging win.

Container images (not run — Docker is not running here)

Cloud Native **Buildpacks** turn the jar into an **OCI image** (the standard container-image format that Docker and Kubernetes run) with no Dockerfile:

```
mvn spring-boot:build-image -Dspring-boot.build-image.imageName=acme/orders:1.0
# JVM options for the buildpack, e.g. enable the AOT cache or CDS training at build time:
#   <env><BP_JVM_AOTCACHE_ENABLED>true</BP_JVM_AOTCACHE_ENABLED></env>   (or BP_JVM_CDS_ENABLED)
```

Or a layered Dockerfile (the recipe file Docker builds an image from; each instruction creates a cached layer) — dependencies change rarely, so they go in their own layer and are cached between builds:

```
FROM eclipse-temurin:25-jre AS build
WORKDIR /build
COPY target/orders.jar app.jar
RUN java -Djarmode=tools -jar app.jar extract --layers --destination out

FROM eclipse-temurin:25-jre
WORKDIR /app
COPY --from=build /build/out/dependencies/ ./
COPY --from=build /build/out/spring-boot-loader/ ./
COPY --from=build /build/out/snapshot-dependencies/ ./
COPY --from=build /build/out/application/ ./
RUN java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar app.jar
USER 1000
ENTRYPOINT ["java", "-XX:AOTCache=app.aot", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

`-XX:MaxRAMPercentage` sizes the heap from the container's memory limit; running as a non-root user (not the all-powerful administrator account) is a basic hardening (reducing attack surface) step.

Self-check

1. What does `-Dspring.context.exit=onRefresh` do? → Starts the app until the context is refreshed, then exits — used for AOT cache / CDS training runs.
2. AOT cache vs native image? → AOT cache keeps the JVM and JIT, speeds startup by reusing loaded/linked classes; native image compiles to a standalone executable with much faster startup but closed-world constraints.
3. Why extract the jar first? → The AOT cache/CDS need a plain classpath of jars; nested jars in the fat jar (the single jar Boot builds, with all dependency jars packed inside) are not cache-friendly.

15.12 Production configuration checklist

Area	Setting / practice
Actuator exposure	expose only what you use (`health,info,prometheus`); protect the rest with `EndpointRequest`; consider `management.server.port`
Health	probes (on by default in Boot 4) wired to Kubernetes; custom indicators fast; no dependencies in liveness
Metrics	Prometheus or OTLP registry; common tags (`application`, `region`); histograms for latency SLOs (service level objectives: targets such as "p99 under 300 ms"); bounded tag values
Tracing	`spring-boot-starter-opentelemetry`; sampling rate chosen consciously; OTLP endpoint to a collector
Logging	structured JSON in prod profile; levels INFO/WARN; no secrets or personal data; change levels at runtime via `/actuator/loggers`
Shutdown	graceful (default) + `timeout-per-shutdown-phase` < K8s grace period; `preStop` delay
Threads	`spring.threads.virtual.enabled=true` for blocking I/O apps, with explicit concurrency limits
Startup	extracted jar + AOT cache (JDK 25) or native image where startup matters
Config & secrets	externalised config per environment (Chapter 3); secrets from a vault or K8s secrets, never in the jar
Resources	container memory limit + `MaxRAMPercentage`; connection pool sized to the database
Errors	`spring.web.error.include-stacktrace=never` (default; Boot 4 name — the old `server.error.*` keys are no longer bound since 4.0.0) and Problem Details (Chapter 5)

15.13 Common mistakes (whole chapter)

Symptom	Cause	Fix
`/actuator/prometheus` returns 404	registry dependency missing or endpoint not exposed	add `micrometer-registry-prometheus`, include `prometheus` in exposure
Prometheus endpoint missing only in tests	Boot disables metric export in tests	`@AutoConfigureMetrics`
No trace ids in logs	no tracer on the classpath, or a custom log pattern without `%X{traceId}`	add `spring-boot-starter-opentelemetry`; keep Boot's correlation pattern
Trace breaks between services	client created with `new` / `builder()`	inject Boot's `RestClient.Builder` / `WebClient.Builder`
`@Observed` does nothing	property or AspectJ starter missing, or self-invocation	`management.observations.annotations.enabled=true`, `spring-boot-starter-aspectj`, call through the proxy
Prometheus memory explodes	high-cardinality tags	remove ids from tags
Pods restart during DB outage	DB in liveness	keep liveness internal
502s (Bad Gateway: the proxy got no valid answer from the app) during deployments	no `preStop` delay, grace timeout too short	`preStop` sleep, align timeouts
Secrets visible in `/actuator/env`	`show-values=always`	leave the default `never`, secure the endpoint

---

15.14 In the interview

Q: What is Spring Boot Actuator and how do you secure it? Management endpoints (health, info, metrics, prometheus, loggers, env, ...). Only `health` is exposed by default; expose more with `management.endpoints.web.exposure.include`, restrict with `management.endpoint.<id>.access`, and secure with a `SecurityFilterChain` using `EndpointRequest.to(...)` / `toAnyEndpoint()`, often on a separate management port.

- *Follow-up: how would you change a log level in production without redeploying?* → `POST /actuator/loggers/<logger>` with `{"configuredLevel":"DEBUG"}` (protected endpoint).

Q: Counter, Gauge, Timer — when to use which? → Counter for events that accumulate (rate via PromQL, Prometheus's query language, `rate()`), Gauge for current values (queue depth), Timer for durations (count/sum/max/histogram → latency percentiles). Keep tags low-cardinality.

Q: What does the Observation API add over plain Micrometer metrics? → Instrument once, get metrics *and* traces (and custom handlers); Spring's own instrumentation uses it; `@Observed` for methods.

Q: How does a trace id get from service A to B? → The instrumented HTTP client injects `traceparent`; B's server observation extracts it and creates a child span; both log the same trace id via MDC.

Q: How do you do zero-downtime deployments (releasing a new version without any moment of unavailability) with Spring Boot on Kubernetes? → Readiness probe (Boot marks ready after startup), graceful shutdown (default), `preStop` delay, grace period longer than the drain timeout, rolling update with `maxUnavailable: 0`. See blue-green and canary.

Q: How would you speed up startup? → Lazy init (create beans only when first used) where safe, fewer auto-configurations, extracted jar + CDS/AOT cache (JDK 25: `-XX:AOTCacheOutput` training, `-XX:AOTCache`), Spring AOT, or GraalVM native image — trading build complexity and closed-world limits for startup.

Q: Virtual threads in Spring Boot? → `spring.threads.virtual.enabled=true` (Java 21+, Boot 3.2+); great for blocking I/O; move concurrency limits to explicit places (pools, `@ConcurrencyLimit`).

Chapter TL;DR

- Observability = **logs + metrics + traces**, linked by the trace id. Spring: Logback, **Micrometer**, **Micrometer Tracing + OpenTelemetry**, all fed by the **Observation API**.
- **Actuator**: only `health` exposed by default; control with `exposure.include` and `access`; secure with `EndpointRequest` (Boot 4 package `org.springframework.boot.security.autoconfigure.actuate.web.servlet`).
- **Health**: `HealthIndicator` (Boot 4 package `org.springframework.boot.health.contributor`), groups, **liveness/readiness on by default in Boot 4**; DOWN → 503; no external deps in liveness.
- **Metrics**: Counter/Timer/Gauge with **low-cardinality** tags; `/actuator/prometheus` for scraping; `@AutoConfigureMetrics` in tests.
- **Observation / `@Observed`**: one instrumentation → timer + span; needs `management.observations.annotations.enabled=true` + AspectJ.
- **Tracing**: `spring-boot-starter-opentelemetry`; W3C `traceparent` propagation via Boot's client builders; `[traceId-spanId]` in logs; sampling default 0.1; export over OTLP.
- **Structured logging** (Boot 3.4+): `logging.structured.format.console=ecs|logstash|gelf`; MDC fields included.
- **Graceful shutdown** is the default (Boot 3.4+); tune `spring.lifecycle.timeout-per-shutdown-phase`; add a `preStop` delay on Kubernetes.
- **Virtual threads**: `spring.threads.virtual.enabled=true`.
- **Startup**: extract the jar, train an **AOT cache** (JDK 25; 934 → 467 ms here), or build a **GraalVM native image**; package with buildpacks or a layered Dockerfile.
- Next: Chapter 16 — Spring Cloud and Microservices.

Chapter 16 — Spring Cloud and Microservices

A **microservice architecture** splits one application into small services that are deployed independently and talk over the network. That buys independent releases and scaling, and costs you every problem of **distributed systems** (systems whose parts run on different machines and can only talk over an unreliable network; see System Design Fundamentals): where is the other service, what if it is slow, how do clients find a single entry point, where does configuration live, how do you follow one request through ten services. **Spring Cloud** is a family of projects that gives Spring Boot applications ready-made answers to those problems.

This chapter is about the *Spring implementation*. The patterns themselves — why a gateway (one front door for all clients), how a circuit breaker (a switch that stops calls to a failing service) thinks, sagas (multi-service workflows made of local steps plus undo steps), CQRS (separate models for writing and for reading data) — are explained in the Microservices Patterns notes and the System Design Patterns modules; each section links to the right one instead of re-teaching it.

What you'll learn

1. Which Spring project solves which microservice problem, and whether you need Spring Cloud at all on Kubernetes.
2. The Spring Cloud **release train** that matches Boot 4.1 and how to import it.
3. **Config Server** and the config client: layered configuration, refresh at runtime (run).
4. **Service discovery**: Eureka, Consul, Kubernetes (explained, not run) and the "simple" static discovery (run).
5. **Client-side load balancing** with Spring Cloud LoadBalancer (run).
6. **Declarative clients**: OpenFeign vs Spring's HTTP interface clients (run).
7. **API gateway** with Spring Cloud Gateway (server Web MVC flavour): routes, predicates, filters, circuit-breaker fallback (run).
8. **Resilience**: Resilience4j circuit breaker, retry and rate limiter state transitions (run) vs Framework 7's `@Retryable` / `@ConcurrencyLimit`.
9. Distributed tracing across services, contract testing, messaging between services.

The code: four Spring Boot 4.1.1 projects on Java 25 in `code/` — `config-demo`, `service-clients`, `gateway-demo`, `resilience-demo` (7 tests, all run). Downstream services (the services a service calls) in the tests are tiny local HTTP servers on random ports, so nothing external is needed. Outputs are real `mvn -q verify` output (log noise trimmed as `...`).

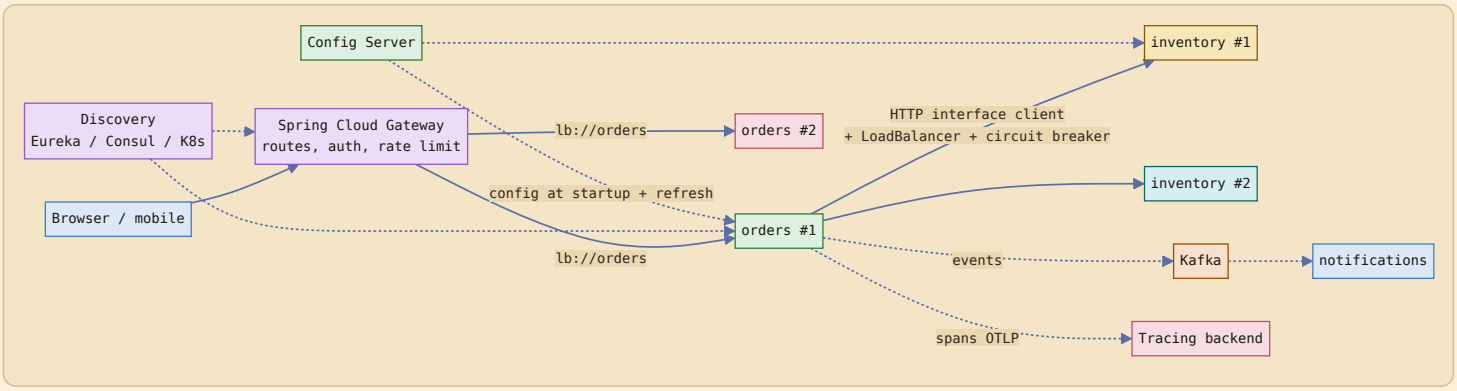
> **Prerequisites.** HTTP clients and HTTP interfaces: Chapter 6. Proxies and Framework 7 `@Retryable`: Chapter 4. Configuration and `spring.config.import`: Chapter 3. Tracing and health probes: Chapter 15. Messaging: Chapter 14.

16.1 The map: microservice problems and Spring answers

Why it matters

Before adding any dependency, know which problem it solves — and whether your platform already solves it. Many teams on Kubernetes (the standard platform for running containers across machines) still add Eureka and a Config Server out of habit, doubling the moving parts.

Problem	Pattern notes	Spring answer	On Kubernetes, often instead
One entry point for clients, cross-cutting edge concerns (things every request needs at the front door, such as auth and rate limits)	API Gateway, BFF	**Spring Cloud Gateway**	Ingress / Gateway API controller (Kubernetes' built-in HTTP entry points), service mesh (a network layer of sidecar proxies that handles routing, retries and encryption between services)
Find instances whose addresses change	Service Discovery	Eureka, Consul, Zookeeper, Spring Cloud Kubernetes	Kubernetes Services + DNS (the system that turns names into IP addresses)
Spread calls over instances	Load balancing	**Spring Cloud LoadBalancer** (client side)	kube-proxy (the Kubernetes component that forwards a Service's traffic to its pods) / mesh (server side)
Configuration per environment, changed without rebuild	Config externalization	**Spring Cloud Config**, Vault, Consul KV	ConfigMaps / Secrets (Kubernetes objects that hold configuration and secret values) (`spring.config.import=configtree:`)
Failing or slow dependencies	Circuit Breaker, Retry, Bulkhead	**Resilience4j** (+ Spring Cloud CircuitBreaker), Framework 7 `@Retryable` / `@ConcurrencyLimit`	mesh retries/outlier detection (automatically ejecting instances that keep failing) (coarser)
Following a request across services	Distributed Tracing	Micrometer Tracing + OpenTelemetry (15.7)	same, plus mesh telemetry
Declarative HTTP calls	—	HTTP interface clients (Framework), OpenFeign (maintenance)	—
Async communication, events	Event-Driven, Saga, CQRS	Spring Kafka / AMQP (Chapter 14), Spring Cloud Stream	—
Cross-cutting helpers next to each service	Sidecar	—	sidecar containers, service mesh



When not to use microservices

Distributed systems are harder to build, test, debug and operate. A small team, an unclear domain (the business area the software models) or no need for independent scaling are strong reasons to start with a **well-modularised monolith** (one deployable application with clean internal module boundaries) (Spring Modulith helps enforce module boundaries) and extract services later with the Strangler Fig approach. Also see DDIA (the book *Designing Data-Intensive Applications*) for the consistency problems you inherit.

16.2 Release trains and versions

What it is

Spring Cloud is ~15 separate projects (Commons, Config, Gateway, OpenFeign, Netflix, Consul, Kubernetes, Circuit Breaker, Stream, Function, Bus, Vault, Task, Zookeeper...), each with its own version. A **release train** is a BOM (bill of materials) that pins a set of versions known to work together and with one Spring Boot generation. Trains are named `YYYY.MINOR.PATCH` (since 2020) with a codename (a nickname such as "Oakwood").

Train	Codename	Spring Boot	Component versions
2025.1.x	**Oakwood**	**4.0.x, 4.1.x** (4.1 from 2025.1.2)	5.0.x
2025.0.x	Northfields	3.5.x	4.3.x — open-source support ended June 2026
2024.0.x	Moorgate	3.4.x	4.2.x

(Source: the compatibility table on spring.io/projects/spring-cloud; 2025.1.3, August 2026, is the latest service release at the time of writing and states it is based on Boot 4.0.x and compatible with 4.1.x. A `2026.0.0-M1` milestone (an early preview build) exists on Maven Central, the public repository Maven downloads libraries from.)

```
<properties>
  <spring-cloud.version>2025.1.3</spring-cloud.version>
</properties>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
<!-- then Spring Cloud starters without <version> -->
```

Things that changed recently and trip people up:

- **Gateway artifacts were renamed** (2025.0): `spring-cloud-starter-gateway-server-webflux` (reactive) and `spring-cloud-starter-gateway-server-webmvc` (servlet); properties moved to `spring.cloud.gateway.server.webflux.*` / `...server.webmvc.*`.
- **Spring Cloud Sleuth is gone** (last in 2021.0 for Boot 2.x); tracing is Micrometer Tracing in Boot itself.
- **Netflix Hystrix, Ribbon, Zuul** were removed years ago; use Resilience4j, Spring Cloud LoadBalancer and Spring Cloud Gateway.
- **OpenFeign** is feature-complete (maintenance mode) — the Spring team recommends HTTP interface clients.
- **Spring Cloud Contract**: the 2025.1.0–2025.1.2 BOMs manage it (5.0.x), but the 2025.1.3 BOM no longer lists it, and its GitHub repository has been moved to the `spring-attic` organisation and archived (read-only; last commit July 2026). Existing users can pin `spring-cloud-contract-dependencies` 5.0.3 explicitly; for new projects consider consumer-driven contracts with Pact (16.10).
- Resilience4j is **not** managed by Spring Boot; the Spring Cloud CircuitBreaker BOM pins Resilience4j 2.3.0, while Resilience4j's own Boot 4 module is `resilience4j-spring-boot4` 2.4.0 (used standalone in 16.8).

Common mistakes

- Mixing trains and Boot generations (e.g. 2025.0 with Boot 4) → `NoSuchMethodError` / `ClassNotFoundException` at startup. Spring Cloud checks compatibility on startup and fails fast (stops immediately instead of misbehaving later) with a clear message (`spring.cloud.compatibility-verifier.enabled`).
- Hard-coding component versions instead of importing the BOM.

16.3 Centralized configuration: Spring Cloud Config

What it is

A **Config Server** is a small Spring Boot app that serves configuration files over HTTP from a backend — usually a **Git repository** (a folder whose full change history is tracked by the Git version-control tool), also a local folder (`native` profile), Vault (HashiCorp's secret store), JDBC, S3 (Amazon's file storage) and others. Every service is a **config client**: at startup it imports its properties from the server, layered by *application name*, *profile* and *label* (Git branch/tag).

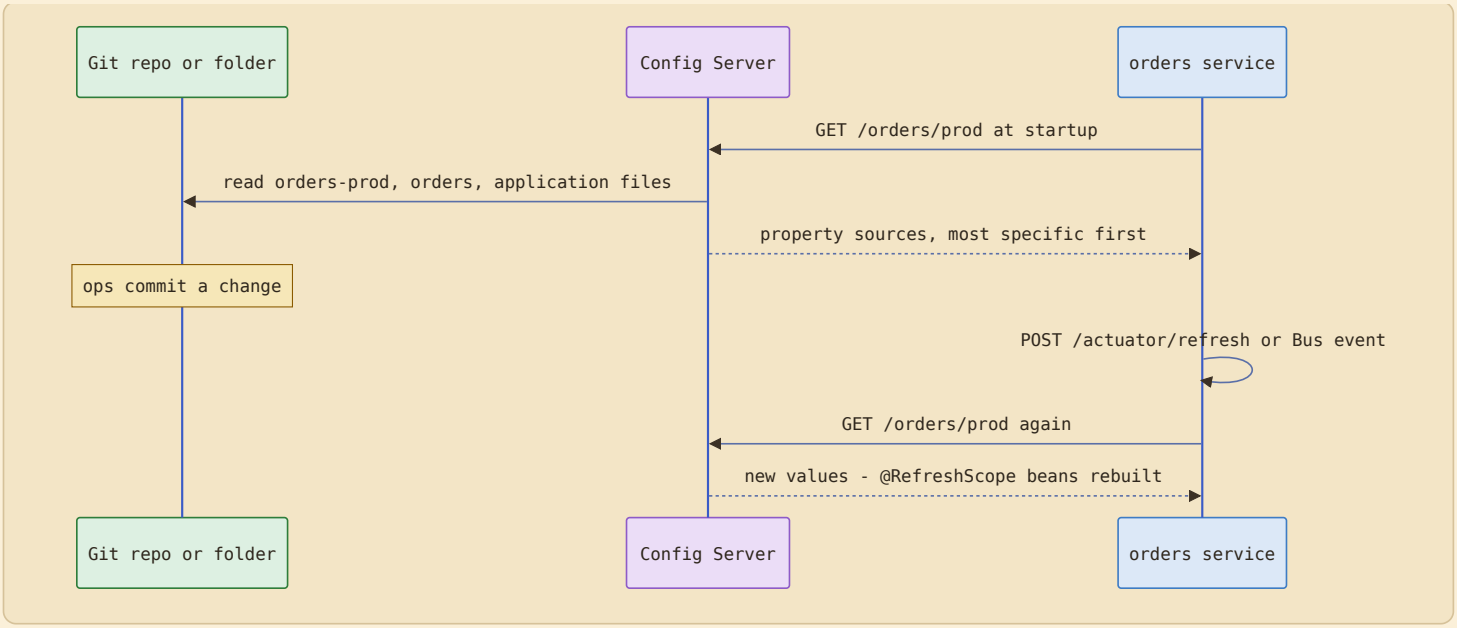
Analogy: a company handbook kept in one shared binder. Each department (service) reads the general chapter plus its own chapter; when the binder is updated, departments can re-read it without re-hiring staff.

Why it matters

With 30 services and 4 environments you have 120 configuration sets. A Config Server gives one place to review, version (Git history = audit log, a record of who changed what and when), and change them — and to refresh running services without a rebuild. See Config externalization notes.

How it works

1. The server exposes `GET /{application}/{profile}[/{label}]` (plus file-style URLs like `/{application}-{profile}.properties`).
2. For `orders` + profile `prod` it returns an ordered list of property sources — most specific first: `orders-prod.properties` → `orders.properties` → `application-prod.properties` → `application.properties`.
3. The client adds `spring.config.import=configserver:http://config:8888` (Boot's config-data API; the old `bootstrap.yml` mechanism is legacy). Use `optional:configserver:...` if the service may start without it.
4. Remote properties are added to the `Environment` and **override the service's own** `application.properties`. Whether local system properties or environment variables (named values set outside the program by the OS or platform) may override remote values is controlled from the repository side (`spring.cloud.config.allow-override`, `override-none`, `override-system-properties`).
5. On refresh the client refetches, rebinds `@ConfigurationProperties` beans and re-creates `@RefreshScope` beans.



The demo

`ConfigServerAndClientTests` writes three files into a temporary folder (standing in for the Git repo), starts a Config Server with the `native` backend on a random port, then starts an `orders` client with profile `prod`:

```
// server
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { ... }
//   --spring.profiles.active=native
//   --spring.cloud.config.server.native.search-locations=file:<repo>/

// client bean
@Component
@RefreshScope   // re-created with fresh values after a refresh
public class OrderSettings {
    public OrderSettings(@Value("${orders.discount-percent}") int discountPercent,
                         @Value("${orders.greeting}") String greeting,
                         @Value("${shared.timeout}") String timeout) { ... }
}
//   --spring.application.name=orders --spring.profiles.active=prod
//   --spring.config.import=configserver:http://localhost:<port>
```

File in the repo	Content
`application.properties` (all services)	`shared.timeout=5s`, `orders.greeting=hello from application.properties`
`orders.properties`	`orders.greeting=hello from orders.properties`, `orders.discount-percent=10`
`orders-prod.properties`	`orders.discount-percent=15`

Then it edits `orders-prod.properties` to 20 and calls `ContextRefresher.refresh()` — what `POST /actuator/refresh` does:

```
GET /orders/prod -> property sources, highest priority first:
   orders-prod.properties
   orders.properties
   application.properties
client sees:    discount=15%, greeting='hello from orders.properties', timeout=5s
refresh changed [orders.discount-percent]
after refresh:  discount=20%, greeting='hello from orders.properties', timeout=5s
```

- `discount=15%` comes from the profile file, `greeting` from the service file (overriding the shared one), `timeout` from the shared file.
- `refresh()` returns exactly the keys that changed.

One demo-only detail: because server and client share one classpath here, the client passes `spring.cloud.config.enabled=true` — the server jar switches the client off by default (its `ConfigServerBootstrapApplicationListener`) so a Config Server does not try to fetch config from itself. In real deployments they are separate apps.

Refreshing many instances, secrets and alternatives

- **Spring Cloud Bus** (`spring-cloud-starter-bus-kafka` / `-amqp`) broadcasts one refresh to all instances (`POST /actuator/busrefresh`); the Config Server's `spring-cloud-config-monitor` can trigger it from a Git webhook (an HTTP call the Git host makes to your server when something is pushed).

- Values like `{cipher}AQB...` are decrypted by the server (`encrypt.key` or a keystore, a protected file that holds keys); better, keep secrets in **Vault** (`spring-cloud-starter-vault-config`) or Kubernetes Secrets.
- On Kubernetes, mounting ConfigMaps/Secrets as files and importing them with `spring.config.import=configtree:/etc/config/` needs no extra server (3.8); Spring Cloud Kubernetes can also read ConfigMaps via the API and reload.

Trade-offs and when not to use it

Pro	Con
One reviewed, versioned source; environment diffs visible	Another critical service: if it is down, new instances can't start (use `optional:`, `fail-fast` + retry, run ≥2 replicas, i.e. copies)
Runtime refresh without redeploy	Runtime refresh is also runtime *risk* — a bad value hits all instances at once
Works on any platform	On Kubernetes, ConfigMaps may be enough

Common mistakes

- Expecting every bean to see new values after refresh — only `@ConfigurationProperties` beans (rebound) and `@RefreshScope` beans (re-created) do. A `@Value` field in a normal singleton (a bean created once and shared, Spring's default) keeps the old value.
- Putting secrets in plain text in the Git repo.
- Forgetting `spring.application.name` — the client then asks for `application`, not `orders`.

Self-check

1. What does the server return for `/orders/prod`? → Property sources ordered most specific first: `orders-prod`, `orders`, (`application-prod`), `application`.
2. How does a Boot 4 client connect? → `spring.config.import=configserver:<url>` (optionally `optional:`).
3. Which beans pick up refreshed values? → `@ConfigurationProperties` beans and `@RefreshScope` beans.

16.4 Service discovery: Eureka, Consul, Kubernetes

What it is

In a dynamic environment instances come and go and their IP addresses (the numeric network addresses of machines) change. **Service discovery** keeps a **registry** of live instances per service name; clients ask "where is `inventory`?" instead of hard-coding addresses. Details and client-side vs server-side discovery: Service Discovery notes.

Spring Cloud Commons defines the abstraction — `DiscoveryClient` (`getInstances("inventory")` → list of `ServiceInstance` with host, port, metadata) — and each backend implements it:

Backend	Starter	How instances register	Health	Notes (not run here)
Netflix Eureka	`spring-cloud-starter-netflix-eureka-server` / `-client`	client registers itself and sends heartbeats ("I'm still alive" messages, every 30 s by default)	heartbeat lease expiry (dropped if heartbeats stop)	AP design (in CAP terms: stays **A**vailable during a network **P**artition rather than **C**onsistent): may serve stale (out-of-date) entries (self-preservation mode)
HashiCorp Consul	`spring-cloud-starter-consul-discovery`	client registers via the local Consul agent	Consul calls the app's health URL	also KV store for config (`spring-cloud-starter-consul-config`)
Kubernetes	`spring-cloud-starter-kubernetes-client-all` (or `-fabric8`)	Kubernetes itself (Endpoints of a Service)	readiness probes	needs RBAC (role-based access control) permission to read endpoints
Simple (in Commons)	none	static list in properties	none	tests, local dev, fixed topologies — used in 16.5
Zookeeper	`spring-cloud-starter-zookeeper-discovery`	ephemeral nodes (entries that vanish when their owner disconnects)	session	CP design (stays consistent, may refuse requests during a partition)

Eureka in a nutshell (not run):

```
@SpringBootApplication
@EnableEurekaServer              // the registry: a dashboard at http://localhost:8761
public class RegistryApplication { ... }
```

```
# each service (client) - with spring-cloud-starter-netflix-eureka-client on the classpath
spring.application.name: inventory                  # the service id
eureka:
  client:
    service-url:
      defaultZone: http://registry:8761/eureka/
  instance:
    prefer-ip-address: true
```

On Kubernetes, do you need it?

Usually **no**. A Kubernetes `Service` named `inventory` already gives a stable DNS name (`http://inventory:8080`) and kube-proxy spreads connections over ready pods (server-side discovery and load balancing). Add Spring Cloud Kubernetes only if you need client-side features: instance metadata, per-request load balancing over pods (Kubernetes' running app instances), or discovery across namespaces (separate named areas inside one cluster). The table in 16.1 shows the platform equivalents.

Self-check

1. What does `DiscoveryClient.getInstances("inventory")` return? → The currently known `ServiceInstance`s (host, port, secure flag, metadata) for that service id.
2. Why is Eureka called AP? → During network partitions it keeps serving possibly stale registrations rather than refusing — availability over consistency.
3. Why might you skip Eureka on Kubernetes? → Services + DNS + readiness probes already provide registration, health and load balancing.

16.5 Client-side load balancing with Spring Cloud LoadBalancer

What it is

Client-side load balancing (load balancing = spreading requests over several instances; see System Design Fundamentals): the *caller* picks which instance of a service to call, from the list discovery provides — no load-balancer box in between. Spring Cloud LoadBalancer (the replacement for Netflix Ribbon) resolves a service id in a URL (`http://inventory/...` or `lb://inventory`) to a concrete instance per request.

How it works

1. A client marked for load balancing (`@LoadBalanced RestClient.Builder`, a Feign client with only a `name`, an HTTP service group without a `base-url`, the gateway's `lb://` URIs) makes a request to `http://inventory/stock/1`.
2. An interceptor asks the `ReactorLoadBalancer` for service `inventory`.
3. A `ServiceInstanceListSupplier` gets instances from the `DiscoveryClient` (cached), optionally filtered (health-check, zone, hints).
4. The algorithm picks one — **round robin** by default (take the instances in turn: A, B, A, B; start position randomised), or `RandomLoadBalancer`, weighted, "same instance preference", subset.
5. The URL is rewritten to `http://host:port/stock/1` and sent.

The demo: static instances, two clients

`service-clients` uses the **simple** discovery client — instances listed in properties — so nothing else must run. The test starts two tiny HTTP servers, "A" and "B", and points the properties at them:

```
spring.application.name=orders
# "Simple" discovery: a fixed list of instances per service id (instead of Eureka/Consul/Kubernetes).
spring.cloud.discovery.client.simple.instances.inventory[0].uri=http://localhost:9001
spring.cloud.discovery.client.simple.instances.inventory[1].uri=http://localhost:9002
# No spring.http.serviceclient.inventory.base-url: Spring Cloud LoadBalancer then uses
# the group name "inventory" as the service id and picks an instance per request.
```

Output (both clients shown in 16.6):

```
HTTP interface 4 calls served by [A, B, A, B]
OpenFeign      4 calls served by [A, B, A, B]  5th call -> Stock[sku=SKU-9, quantity=7, servedBy=A]
```

Round robin alternates between the instances. The starting instance is random, so another run may print `[B, A, B, A]` — the test asserts the alternation, not the order. The startup log also warns:

```
WARN ... LoadBalancerCaffeineWarnLogger : Spring Cloud LoadBalancer is currently working with the default cache. While this cache
implementation is useful for development and tests, it's recommended to use Caffeine cache in pr...
```

— in production add `com.github.ben-manes.caffeine:caffeine` (Caffeine, a fast in-memory cache library) so the instance list cache is bounded and refreshed properly (`spring.cloud.loadbalancer.cache.ttl` — TTL = time to live, how long an entry is kept; default 35 s).

Useful settings:

```
# instance-list supplier chain: default | health-check | zone-preference | weighted | same-instance-preference | request-based-sticky-session | subset
spring.cloud.loadbalancer.configurations=health-check
# per-service overrides live under spring.cloud.loadbalancer.clients.<serviceId>.*
```

For a plain `RestClient`, mark the builder:

```
@Bean @LoadBalanced
RestClient.Builder loadBalancedBuilder() { return RestClient.builder(); }
// restClientBuilder.baseUrl("http://inventory").build()
```

Trade-offs

Client-side (Spring Cloud LoadBalancer)	Server-side (K8s Service, NGINX, cloud LB, mesh)
per-request choice, custom algorithms, zone/metadata aware	nothing in the app; works for every language
each client needs the library and a discovery source	extra hop (one more network step between caller and service; except kube-proxy/mesh sidecars), fewer app-level decisions

---

16.6 Declarative clients: OpenFeign vs HTTP interface clients

What it is

Both let you declare a Java interface and get a generated proxy (an object Spring creates at runtime that implements the interface) that makes the HTTP calls (6.6 teaches HTTP interfaces in depth).

```
// Spring Cloud OpenFeign: name = service id -> resolved by the LoadBalancer (no url given)
@FeignClient(name = "inventory")
public interface InventoryFeignClient {
    @GetMapping("/stock/{sku}")                    // Spring MVC annotations
    Stock stock(@PathVariable("sku") String sku);
}

// Spring Framework HTTP service interface (no extra library)
public interface InventoryHttpClient {
    @GetExchange("/stock/{sku}")
    Stock stock(@PathVariable String sku);
}

@SpringBootApplication
@EnableFeignClients                                                  // scan for @FeignClient
@ImportHttpServices(group = "inventory", types = InventoryHttpClient.class) // group name = service id
public class ClientsApplication { ... }
```

Since Spring Cloud 2025.1 (Commons 5.0), LoadBalancer plugs into Boot 4's **HTTP service client groups**: a group without `spring.http.serviceclient.<group>.base-url` gets `http://<groupName>` load-balanced, and a `base-url` with the `lb://` scheme is also supported. That's why the `inventory` group above needed no URL. The test (`LoadBalancedClientsTests`) calls each client four times — output in 16.5.

Comparison

	OpenFeign	HTTP interface client
Where it lives	Spring Cloud OpenFeign (separate project)	Spring Framework 6+ core; Boot 4 groups via `@ImportHttpServices`
Status	**feature-complete / maintenance** — only fixes	actively developed; the Spring team's recommendation
Annotations	Spring MVC's `@GetMapping`, ... on the interface	`@GetExchange`, `@PostExchange`, ...
Underlying client	Feign's own client (or Apache HC5 / OkHttp)	`RestClient` or `WebClient` (reactive return types possible)
Config	`spring.cloud.openfeign.client.config.<name>.*` (timeouts, loggers, interceptors)	`spring.http.serviceclient.<group>.*`, `RestClientHttpServiceGroupConfigurer`
Load balancing	automatic when only `name` is set	automatic for groups without base URL (Spring Cloud 2025.1+)
Circuit breaker	`spring.cloud.openfeign.circuitbreaker.enabled=true` + fallbacks	Spring Cloud 2025.1+: per-group breaker with an `@HttpServiceFallback` class (switch `spring.cloud.circuitbreaker.http-services.enabled`, on by default; not run here), or wrap the call in a service method with Resilience4j
Tracing	instrumented	instrumented (via the observed `RestClient`)

Recommendation: new code → HTTP interface clients; existing Feign code keeps working, migrate when convenient (the annotations map almost one-to-one).

16.7 API gateway: Spring Cloud Gateway

What it is

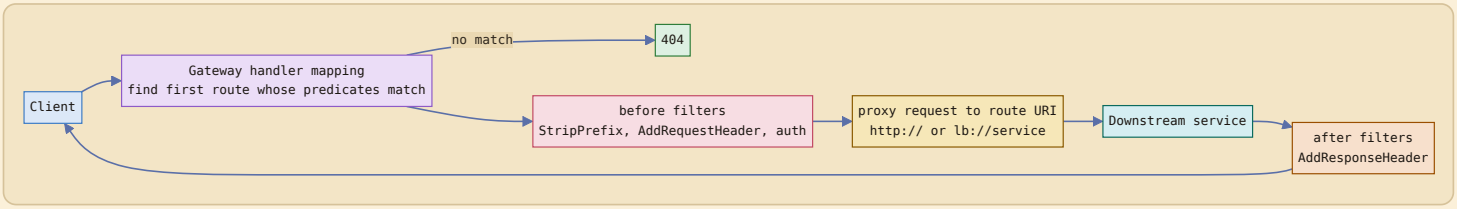
An **API gateway** is the single front door for external clients: it **routes** each request to the right service and applies **edge concerns** once — authentication/token relay, rate limiting, CORS (Cross-Origin Resource Sharing: the headers that tell browsers which other websites may call the API), header rewriting, retries/circuit breaking, request size limits. Pattern details and trade-offs: API Gateway notes, BFF, API & Edge module.

Spring Cloud Gateway comes in two flavours with the same concepts:

	Server Web MVC	Server WebFlux
Starter	`spring-cloud-starter-gateway-server-webmvc`	`spring-cloud-starter-gateway-server-webflux`
Runtime	Servlet (Tomcat/Jetty), blocking; pairs well with virtual threads	Netty, reactive
Routes in code	`RouterFunction` DSL (`GatewayRouterFunctions.route`)	`RouteLocatorBuilder` DSL
Properties	`spring.cloud.gateway.server.webmvc.routes`	`spring.cloud.gateway.server.webflux.routes`
Choose when	team knows MVC, uses blocking libraries	very high connection counts, streaming, already reactive

How it works

A **route** = id + destination URI + **predicates** (when does it match? `Path`, `Method`, `Header`, `Host`, `Query`, `Cookie`, `After`, `Weight`, `Version`...) + **filters** (what to change? `StripPrefix`, `AddRequestHeader`, `RewritePath`, `CircuitBreaker`, `Retry`, `RequestRateLimiter`, `TokenRelay`, ...).



The demo

`gateway-demo` defines one route in YAML and one in Java.

```yaml
spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            # Route 1, in YAML: /api/orders/** -> orders service
            - id: orders
              uri: ${services.orders.url}             # with discovery: lb://orders
              predicates:
                - Path=/api/orders/**
                - Method=GET,POST
              filters:
                - StripPrefix=1                         # /api/orders/7 -> /orders/7
                - AddRequestHeader=X-Gateway, ch16-gateway
                - AddResponseHeader=X-Served-Via, gateway
```

```java
/** Route 2, in the Java DSL: /api/inventory/** -> inventory service, protected by a circuit breaker. */
@Bean
RouterFunction<ServerResponse> inventoryRoute(@Value("${services.inventory.url}") String inventoryUrl) {
    return route("inventory")
            .GET("/api/inventory/**", http())
            .before(uri(inventoryUrl))
            .filter(circuitBreaker("inventoryCB", URI.create("forward:/fallback/inventory")))
            .build();
}

@GetMapping("/fallback/inventory")
String inventoryFallback() {
    return "{\"items\":[],\"note\":\"inventory temporarily unavailable\"}";
}
```

(Static imports: `GatewayRouterFunctions.route`, `HandlerFunctions.http`, `BeforeFilterFunctions.uri`, `CircuitBreakerFilterFunctions.circuitBreaker`. Pass the target with `before(uri(...))`: the older `http(String)` / `http(URI)` overloads, deprecated in Gateway 4.x, no longer exist in Gateway 5.0 — only `http()` remains.) The `CircuitBreaker` filter uses Spring Cloud CircuitBreaker, so the project adds `spring-cloud-starter-circuitbreaker-resilience4j`.

`GatewayRoutingTests` runs an "orders service" that echoes what it received, and points the inventory route at a port where nothing listens:

```
GET /api/orders/7 -> orders-service saw GET /orders/7?expand=items X-Gateway=ch16-gateway | response X-Served-Via=gateway
DELETE /api/orders/7 and GET /api/unknown -> 404 (no route matches)
GET /api/inventory/items (service down) -> {"items":[],"note":"inventory temporarily unavailable"}
```

- `StripPrefix=1` removed `/api`; the query string was kept; the request and response headers were added.
- `DELETE` fails the `Method` predicate and nothing matches `/api/unknown` → 404.
- The connection failure to inventory went through the circuit breaker to the fallback — the client got a degraded but valid answer.

Other common gateway jobs (not run)

- **Token relay** (`TokenRelay` filter with `spring-boot-starter-oauth2-client`): the gateway logs the user in and forwards the access token to services — see 11.10 and the BFF discussion in 11.12.
- **Rate limiting**: WebFlux flavour has `RequestRateLimiter` backed by Redis; the MVC flavour offers a Bucket4j-based filter. See Rate Limiting.
- **Weighted routes** (`Weight=group, 90`) for canary releases (sending a small share of traffic to a new version first).

Common mistakes

- Putting business logic or aggregation of many services in the gateway → it becomes a "distributed monolith" bottleneck. Keep it thin; use a BFF for client-specific aggregation.
- Using Spring Cloud Gateway *and* an Ingress/cloud gateway that do the same things — pick one layer for each concern.
- Retrying non-idempotent `POST`s at the gateway (an *idempotent* call can safely run twice with the same effect; a `POST` that creates an order usually cannot).

16.8 Resilience: circuit breaker, retry, rate limiter

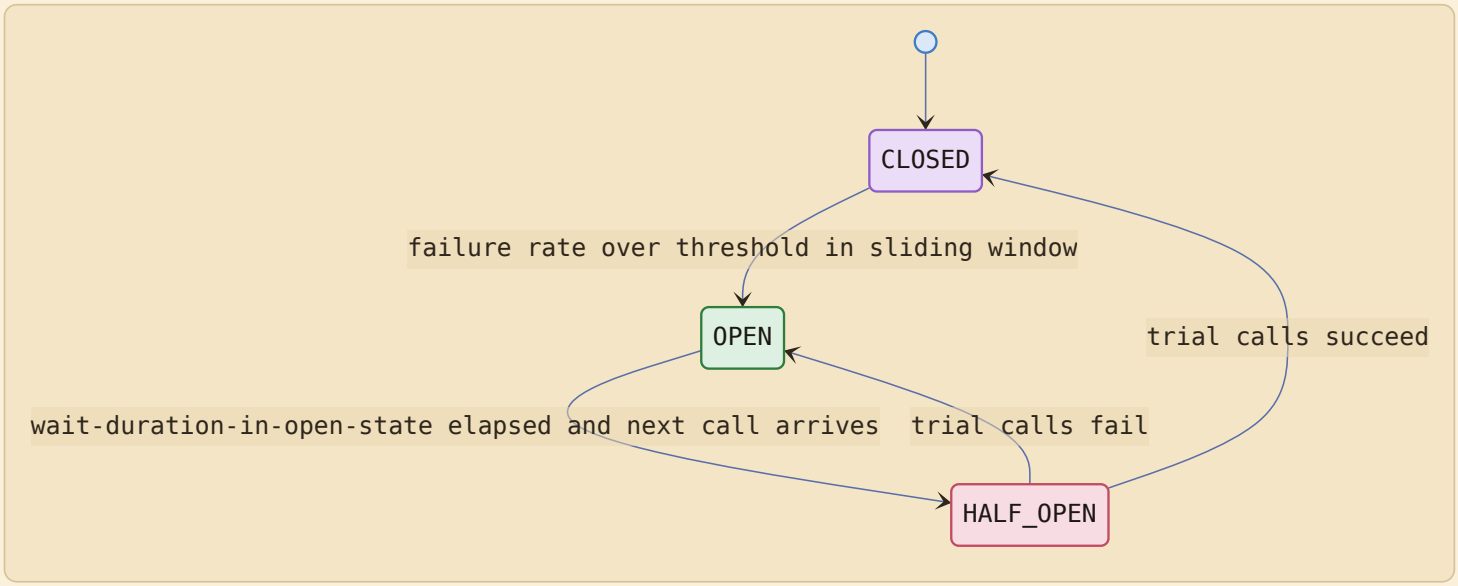
What it is

Remote calls fail and, worse, become **slow**. Without protection, callers pile up waiting threads until they fail too — a **cascading failure**. The main tools (Reliability module):

Tool	Idea	Notes
Timeout	never wait forever	the first line of defence; set on every client (Chapter 6)
Retry	try again after a transient (temporary) failure, with back-off (growing waits) + jitter (a random extra wait so clients don't retry in lockstep)	only idempotent operations — Retry notes
Circuit breaker	after many failures, stop calling for a while and fail fast / fall back	Circuit Breaker notes
Bulkhead	cap concurrent calls to one dependency	Bulkhead notes
Rate limiter	cap calls per time period	protects a dependency or enforces a quota (an agreed maximum usage)
Fallback	a degraded answer (cache, default, empty list)	Graceful degradation

Resilience4j is the standard Java library for all of these (Hystrix's successor). Use it directly (`resilience4j-spring-boot4`: annotations + properties + Micrometer metrics + health) or through **Spring Cloud CircuitBreaker**, an abstraction (`CircuitBreakerFactory.create("id").run(supplier, fallback)`) with a Resilience4j implementation, used e.g. by the gateway filter above.

Circuit breaker states



- **CLOSED** — calls pass; outcomes are recorded in a **sliding window** (last N calls, or last N seconds).
- **OPEN** — calls are rejected immediately with `CallNotPermittedException` (→ fallback) without touching the dependency, giving it time to recover.
- **HALF_OPEN** — a limited number of trial calls decide whether to close again.

The demo

`resilience-demo` wraps a fake remote `RemoteInventory` (the tests switch it up or down):

```
@CircuitBreaker(name = "inventory", fallbackMethod = "cachedStock")
public String stockWithBreaker(String sku) {
    return "live stock " + remote.stock(sku);
}

// fallback: same parameters + the exception as the last parameter
private String cachedStock(String sku, Throwable t) {
    return "cached stock 5 (" + t.getClass().getSimpleName() + ")";
}

@Retry(name = "inventory-retry")
public String stockWithRetry(String sku) { return "live stock " + remote.stock(sku); }

@RateLimiter(name = "search")
public String search(String text) { return "results for " + text; }
```

```yaml
resilience4j:
  circuitbreaker:
    instances:
      inventory:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 4                # judge the last 4 calls
        minimum-number-of-calls: 4            # ... but only once 4 calls were recorded
        failure-rate-threshold: 50            # open when >= 50 % of them failed
        wait-duration-in-open-state: 300ms    # stay OPEN this long, then allow trial calls
        permitted-number-of-calls-in-half-open-state: 2
        record-exceptions: ch16.resilience.RemoteCallException
  retry:
    instances:
      inventory-retry:
        max-attempts: 3                       # total attempts, including the first call
        wait-duration: 20ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2     # 20 ms, then 40 ms
        retry-exceptions: ch16.resilience.RemoteCallException
  ratelimiter:
    instances:
      search:
        limit-for-period: 3                   # 3 permits ...
        limit-refresh-period: 10s             # ... per 10-second cycle
        timeout-duration: 0                   # don't wait for a permit: reject immediately
```

`CircuitBreakerTests` subscribes to the breaker's state-transition events and walks it through all states:

```
1) dependency goes down, 4 calls:
   call 1: cached stock 5 (RemoteCallException)  state=CLOSED
   call 2: cached stock 5 (RemoteCallException)  state=CLOSED
   call 3: cached stock 5 (RemoteCallException)  state=CLOSED
   call 4: cached stock 5 (RemoteCallException)  state=OPEN
2) while OPEN the remote is not even called:
   call 5: cached stock 5 (CallNotPermittedException)  remote calls +0
3) wait 300 ms, dependency is back; 2 trial calls in HALF_OPEN:
   call 6: live stock 7  state=HALF_OPEN
   call 7: live stock 7  state=CLOSED
transitions: [CLOSED -> OPEN, OPEN -> HALF_OPEN, HALF_OPEN -> CLOSED]
```

- Calls 1–3 fail but the breaker stays CLOSED: the window needs `minimum-number-of-calls` (4) before judging.
- After call 4 the failure rate is 100% ≥ 50% → OPEN.
- Call 5 is rejected without calling the remote (`remote calls +0`) — this is what stops the cascade.
- After 300 ms the next call moves the breaker to HALF_OPEN — by default (`automatic-transition-from-open-to-half-open-enabled: false`) no timer does this; the first call after the wait triggers it. Two successful trials close it.
- Without these settings Resilience4j's defaults apply: `COUNT_BASED` window of 100 calls, `minimum-number-of-calls` 100, `failure-rate-threshold` 50, `wait-duration-in-open-state` 60 s, 10 permitted calls in HALF_OPEN — sensible for production traffic, far too slow for a demo. Retry defaults to 3 attempts 500 ms apart.

`RetryAndRateLimiterTests`:

```
Framework 7 @Retryable, remote fails twice -> live stock 7 after 3 attempts
RateLimiter (3 per 10 s): call1=OK call2=OK call3=OK call4=REJECTED call5=REJECTED
Resilience4j @Retry, remote fails twice:
   retry #1 after inventory unavailable, waiting 20 ms
   retry #2 after inventory unavailable, waiting 40 ms
   -> live stock 7 after 3 attempts
   retry #1 after inventory unavailable, waiting 20 ms
   retry #2 after inventory unavailable, waiting 40 ms
   remote down: gave up after 3 attempts, caller gets RemoteCallException
```

- Resilience4j's `max-attempts: 3` counts the **first call**; Framework 7's `maxRetries = 2` counts only **retries** — both mean 3 attempts.
- Exponential back-off doubled the wait (20 → 40 ms). Add jitter in production (`randomized-wait-factor` / `enable-randomized-wait`).
- The rate limiter rejects with `RequestNotPermitted` once the 3 permits of the current period are used.

The Framework 7 version used in the comparison (Chapter 4.7 covers it fully):

```java
@Retryable(includes = RemoteCallException.class, maxRetries = 2, delay = 20, multiplier = 2)
@ConcurrencyLimit(limit = 10, policy = ConcurrencyLimit.ThrottlePolicy.REJECT)   // a simple bulkhead
public String stock(String sku) { return "live stock " + remote.stock(sku); }
// needs @EnableResilientMethods on a configuration class
```

Resilience4j vs Framework 7 core

Need	Framework 7 core	Resilience4j
Retry with back-off, jitter	`@Retryable`, `RetryTemplate`	`@Retry`, `Retry`
Bulkhead (concurrency cap)	`@ConcurrencyLimit` (BLOCK or REJECT)	`@Bulkhead` (semaphore — a counter of free slots — or thread pool)
Circuit breaker	—	`@CircuitBreaker`
Rate limiter	—	`@RateLimiter`
Time limiter	— (use client timeouts)	`@TimeLimiter` (async/reactive)
Fallback methods	— (catch at the caller)	`fallbackMethod`
Config in properties, per instance	attributes (placeholders allowed)	yes, rich
Metrics, health, events	`MethodRetryEvent`	Micrometer meters, health indicators, event publishers
Extra dependency	none	yes

Rule of thumb: plain retries and concurrency caps → Framework 7; anything involving circuit breaking, rate limiting or fallbacks → Resilience4j.

Order when stacking annotations (Resilience4j defaults): `Retry( CircuitBreaker( RateLimiter( TimeLimiter( Bulkhead( call ) ) ) ) )` — the retry is outermost, so each retry attempt is counted by the breaker, and an OPEN breaker's rejection can be retried unless you exclude `CallNotPermittedException`. Change the order with `resilience4j.*.<type>-aspect-order` properties.

Common mistakes

- Retrying everything, including 4xx errors and non-idempotent calls.
- Retries at gateway **and** client **and** service → retry storms (3 × 3 × 3 = 27 calls).
- A circuit breaker without timeouts: slow calls never "fail", so the breaker never opens (or configure `slow-call-duration-threshold` / `slow-call-rate-threshold`).
- One breaker for all endpoints of a service — one bad endpoint opens the circuit for the healthy ones.
- Self-invocation — all these annotations are proxy-based (Chapter 4).
- Fallbacks that hide outages forever — alert on breaker state (`resilience4j_circuitbreaker_state` metric).

Self-check

1. What happens to calls while a breaker is OPEN? → Rejected immediately with `CallNotPermittedException` (fallback if defined), without calling the dependency.
2. Why did the breaker stay CLOSED after 3 failures? → `minimum-number-of-calls` was 4.
3. Resilience4j `max-attempts: 3` vs Framework `maxRetries = 3`? → 3 total attempts vs 4 total attempts.
4. Which is the outermost Resilience4j aspect by default? → Retry.

TL;DR

Timeouts first; retry only transient, idempotent failures with back-off + jitter; circuit breaker to fail fast and give the dependency room; bulkhead/rate limiter to cap load; fallbacks for degraded answers. Framework 7 covers retry + concurrency limits; Resilience4j covers everything.

16.9 Distributed tracing across services

Little is needed beyond Chapter 15.7: every service adds `spring-boot-starter-opentelemetry` and exports to the same collector; the gateway and clients built from Boot's `RestClient.Builder` (including HTTP interface groups) and OpenFeign propagate the W3C `traceparent` header. For messaging, switch observation on explicitly (e.g. `spring.kafka.template.observation-enabled=true` and `spring.kafka.listener.observation-enabled=true`, same idea for RabbitMQ) so the context travels in message headers. Things specific to microservices:

- Use the **same propagation format** everywhere (W3C by default; set `management.tracing.propagation.type=b3` only for legacy Zipkin/B3 services, or produce both).
- Set `spring.application.name` in every service — it becomes the service name in traces and logs.
- Let the **edge decide sampling** (the gateway) — downstream services follow the sampled flag.
- Tag spans with business ids as **high-cardinality** values (order id), never as metric tags.

16.10 Contract testing

What it is

Service A (consumer) and service B (provider) are tested separately — how do you know they still agree on the API? End-to-end tests of all services are slow and flaky. **Contract testing** checks each side against a shared **contract** (request → expected response):

1. Consumer-driven: the consumer writes what it needs.
2. The provider's build **verifies** it really responds that way.
3. The consumer's tests run against a **stub** (a fake server that returns canned answers) generated from the same contract.

Tool	How	Status
Pact (pact-jvm)	consumer tests produce a pact file; a Pact Broker (a server that stores contracts and results) shares it; provider verifies; `can-i-deploy` gate	active, polyglot (works across programming languages)
Spring Cloud Contract	contracts in YAML/Groovy/Java in the provider repo; the plugin generates provider tests and a stubs jar; consumers use `@AutoConfigureStubRunner`	JVM/Spring-centric; not in the 2025.1.3 BOM, repository archived in `spring-attic` (see 16.2)
OpenAPI-based checks	validate both sides against the OpenAPI document	good for provider-driven public APIs

Spring Cloud Contract, for recognition (not run):

```
# provider: src/test/resources/contracts/stock/shouldReturnStock.yml
request:
  method: GET
  url: /stock/SKU-1
response:
  status: 200
  headers: { Content-Type: application/json }
  body: { sku: "SKU-1", quantity: 7 }
```

```
// consumer test: runs against a WireMock stub built from the provider's contracts
@SpringBootTest
@AutoConfigureStubRunner(ids = "com.acme:inventory:+:stubs:8090", stubsMode = StubRunnerProperties.StubsMode.LOCAL)
class InventoryClientContractTest { /* call http://localhost:8090/stock/SKU-1 */ }
```

When not to use it: a single team owning both sides that deploys them together, or public APIs where you can't know consumers (use versioning + OpenAPI instead).

16.11 Messaging between services

Synchronous HTTP chains couple availability: if any service in the chain is down, the request fails. Events decouple them in time (Event-Driven notes). Spring options:

- **Spring for Apache Kafka / Spring AMQP** — templates and `@KafkaListener`/`@RabbitListener` (Chapter 14, including the transactional outbox).
- **Spring Cloud Stream** — a binder abstraction: you write `java.util.function` beans (`Supplier`, `Function`, `Consumer`) and bind them to Kafka/RabbitMQ destinations by configuration (`spring.cloud.function.definition=enrichOrder`, `spring.cloud.stream.bindings.enrichOrder-in-0.destination=orders`). Good for many similar pipelines; it hides broker-specific features you may need.
- Cross-service consistency without distributed transactions (one transaction spanning several databases): Saga (choreography — each service reacts to others' events — or orchestration — one coordinator tells each service what to do), idempotent consumers, and CQRS read models.

16.12 Common mistakes (whole chapter)

Symptom	Cause	Fix
App fails at startup with a compatibility error	Spring Cloud train doesn't match Boot	2025.1.x for Boot 4.0/4.1
`No spring.config.import property has been defined`	config client on classpath without import	add `spring.config.import=configserver:...` (or `optional:`)
Config change not visible after refresh	plain `@Value` in a singleton	`@RefreshScope` or `@ConfigurationProperties`
`UnknownHostException: inventory`	client not load-balanced	`@LoadBalanced` builder, Feign `name`, or HTTP group without base URL
LoadBalancer "default cache" warning	Caffeine missing	add Caffeine
Gateway returns 404	predicates don't match (method, path, prefix)	check the route definition; DEBUG logging for `org.springframework.cloud.gateway`
Breaker never opens	no timeouts, slow calls not recorded, window too large	timeouts + slow-call thresholds + sensible window
Retry storm during an outage	retries at several layers	retry in one layer, with jitter; breaker around retries
Everything down when one service is down	synchronous chains, no fallbacks	timeouts, breakers, fallbacks, async events

---

16.13 In the interview

Q: Which Spring Cloud version do you use with Spring Boot 4.1? The 2025.1.x "Oakwood" train (Boot 4.0 and 4.1, the latter from 2025.1.2), imported as the `spring-cloud-dependencies` BOM; its components are 5.0.x.

Q: How do microservices find each other in Spring? Through the `DiscoveryClient` abstraction backed by Eureka, Consul, Zookeeper or Kubernetes; Spring Cloud LoadBalancer resolves service ids (`http://inventory`, `lb://inventory`) to instances per request, round robin by default. On Kubernetes plain Services + DNS usually suffice.

- *Follow-up: client-side vs server-side load balancing?* → Client picks the instance (smarter, per-request, needs a library) vs a proxy picks it (language-agnostic, extra hop).

Q: Explain the circuit breaker states and how you configure them in Resilience4j. CLOSED records outcomes in a sliding window; when the failure (or slow-call) rate exceeds the threshold after `minimum-number-of-calls`, it goes OPEN and rejects calls fast; after `wait-duration-in-open-state` it goes HALF_OPEN and lets `permitted-number-of-calls-in-half-open-state` trial calls decide between CLOSED and OPEN. Configure per instance under `resilience4j.circuitbreaker.instances.<name>`; use `@CircuitBreaker(name, fallbackMethod)`.

- *Follow-up: how is this different from retry?* → Retry repeats a single call hoping for success; the breaker stops calling at all when the dependency is clearly unhealthy. Combine them, with the breaker protecting against retry storms.
- *Follow-up: does Spring Framework 7 have a circuit breaker?* → No — it added `@Retryable` and `@ConcurrencyLimit` only.

Q: What does an API gateway do, and which Spring Cloud Gateway flavour would you pick? Single entry point: routing by predicates, filters for auth/token relay, rate limiting, header rewriting, resilience. Web MVC flavour for servlet teams and virtual threads; WebFlux flavour for reactive stacks and very high concurrency. Keep business logic out.

Q: How do you manage configuration across 30 services? Externalised config: Spring Cloud Config (Git-backed, layered by app/profile/label, refresh via actuator or Bus) or platform-native ConfigMaps/Secrets with `configtree:`; secrets in Vault; config changes reviewed like code.

Q: OpenFeign or HTTP interfaces? → HTTP interfaces for new code (core Spring, actively developed, Boot 4 groups, load-balanced by Spring Cloud); OpenFeign is in maintenance mode.

Q: How do you test that services stay compatible? → Consumer-driven contract tests (Pact, or Spring Cloud Contract in existing projects) plus a small number of end-to-end smoke tests.

Chapter TL;DR

- Spring Cloud = ready-made answers to distributed-system problems; check what your platform (Kubernetes) already gives you before adding them. Patterns: microservices-patterns notes.
- **Boot 4.1 → Spring Cloud 2025.1.x (Oakwood)**, BOM `spring-cloud-dependencies` (2025.1.3 used here); components 5.0.x; OpenFeign in maintenance; Spring Cloud Contract no longer in the 2025.1.3 BOM.
- **Config Server**: `@EnableConfigServer`, Git or `native` backend; clients use `spring.config.import=configserver:`; precedence `app-profile` > `app` > `application`; refresh re-creates `@RefreshScope` beans (proved: 15% → 20%).
- **Discovery**: `DiscoveryClient` over Eureka/Consul/Kubernetes/simple; **LoadBalancer** resolves service ids per request, round robin by default (proved: B, A, B, A); add Caffeine.

- **Declarative clients**: prefer HTTP interfaces (`@ImportHttpServices`, load-balanced when the group has no base URL); OpenFeign still works.
- **Gateway**: routes = predicates + filters; server-webmvc or server-webflux; `StripPrefix`, headers, `CircuitBreaker` with fallback (proved).
- **Resilience4j**: breaker CLOSED → OPEN → HALF_OPEN → CLOSED (proved), retry with back-off, rate limiter (proved); Framework 7 `@Retryable`/`@ConcurrencyLimit` for simple cases.
- Tracing across services = Chapter 15 setup in every service; contracts with Pact/Spring Cloud Contract; events with Kafka/Stream.
- Next: Chapter 17 — Spring AI and what's new.

Chapter 17 — Spring AI and What's New in Boot 4 / Framework 7

This chapter has two halves.

The first half is **Spring AI**: the Spring project for calling large language models (LLMs) from a Spring application. It gives you one API for many AI providers (the companies or tools that host the models: OpenAI, Anthropic, Google, Amazon Bedrock, Ollama, ...). It also covers the things a real AI feature needs besides "send text, get text": prompt templates (the **prompt** is the text you send to the model), conversation memory, turning answers into Java objects, letting the model call your code (tools), searching your own documents (RAG, retrieval-augmented generation, §17.9), and the Model Context Protocol (MCP: a standard way to offer tools to AI apps, §17.10).

The second half is a **migration map**: what changed from Spring Boot 3.x to 4.x, from Spring Framework 6 to 7, and from Spring Security 6 to 7. Each change points to the chapter of this book that teaches it.

This chapter does **not** re-teach how LLMs work. For tokens (the word pieces a model reads and writes, and is billed by), context windows (the maximum amount of text one call can hold), sampling (how the model picks each next token), prompting, embeddings (lists of numbers that capture a text's meaning, §17.9), RAG design and agents (programs where a model repeatedly decides which tool to call next), read the AI book: LLMs, prompt engineering, building LLM apps, embeddings, vector databases and RAG, evaluation and agents. Here we focus on the **Spring way** of doing those things.

What you'll learn

1. What Spring AI is, which version fits Boot 4.1, and its building blocks (§17.1).
2. How to set up a project for a real provider (OpenAI, Anthropic) and how to run everything **offline** with a fake model (§17.2–17.3).
3. `ChatClient`, messages and prompt templates (§17.3–17.4).
4. Advisors: logging, a custom redaction advisor, and chat memory (§17.5–17.6).
5. Structured output into Java records (§17.7).
6. Tool calling: the model asks, Spring runs your method (§17.8).
7. RAG with `SimpleVectorStore`, an embedding model and `QuestionAnswerAdvisor` (§17.9).
8. MCP servers and clients (§17.10), and production concerns (§17.11).
9. The Boot 3 → 4 migration table (§17.13), Framework 6 → 7 (§17.14), Security 6 → 7 (§17.15) and Boot 4.1 additions (§17.16).

The code. `code/spring-ai-offline/` is a Spring Boot 4.1.1 project (Framework 7.0, Java 25) using **Spring AI 2.0.1**. It needs **no API key** (the secret string a provider gives you to authenticate and bill your calls) **and no network**: the `ChatModel` bean is a small fake that returns canned (pre-written) answers (`FakeChatModel`), and the `EmbeddingModel` bean is a word-hashing fake (`FakeEmbeddingModel`). Everything *above* the model — `ChatClient`, advisors, memory, tool calling, structured output, the vector store — is the real Spring AI code. All output shown is real `mvn -q -B -ntp verify` output (14 tests, all passing). Snippets that need a real provider are marked **(needs an API key — not run)**.

17.1 What Spring AI is

What it is

Spring AI is a Spring project that connects Spring applications to AI models. It does for AI providers what Spring Data does for databases: it gives you **portable interfaces** (`ChatModel`, `EmbeddingModel`, `VectorStore`, ...), **Boot starters** that auto-configure a provider from properties, and a **fluent client** (`ChatClient`) for everyday use.

Analogy: `RestClient` (Chapter 6) hides the HTTP library under one API. `ChatClient` hides the AI provider under one API. Switching from OpenAI to Anthropic is mostly a change of starter and properties, not of code.

Why it matters

Calling an LLM's HTTP API directly is easy for a demo. A production feature also needs:

- prompt templates kept outside the code;
- conversation memory (the model itself remembers nothing between calls);
- answers converted into typed Java objects;
- tool calling (the model asks your application to run a function);
- retrieval of your own documents (RAG);
- logging, metrics, tracing, retries, and tests that do not call a paid API.

Spring AI provides each of these as a Spring building block, so they plug into things you already know: beans, properties, Micrometer observations (Chapter 15) and Boot tests (Chapter 12).

Versions (verified September 2026)

Spring AI	Released	Works with	Notes
1.0	May 2025	Boot 3.4 / 3.5	first GA (general availability: the first production-ready release)
1.1	Nov 2025	Boot 3.4 / 3.5	MCP annotations, more providers
2.0	**Jun 2026**	**Boot 4.0 / 4.1, Framework 7**	Jackson 3, JSpecify null-safety (annotations that mark what may be `null`), tool loop moved into advisors, immutable (unchangeable once built) options, MCP Java SDK 2.0. **Current: 2.0.1** (used here)

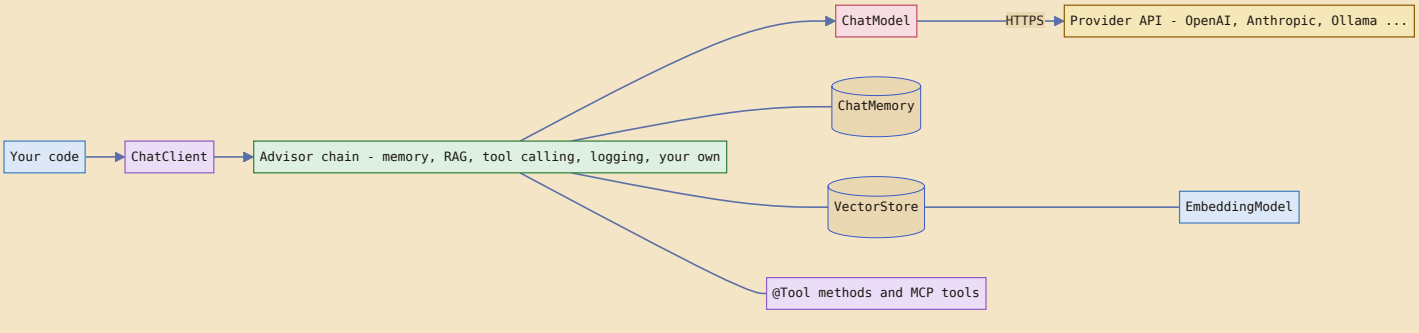
Spring AI 2.0 **requires** Boot 4. A Boot 3.5 application stays on Spring AI 1.1.x until it migrates. Note that 2.0.0's starters were built against Boot 4.1 dependency versions (spring-ai issue #6465), so Boot 4.1.x is the smoothest choice.

Spring AI is a separate project with its own version number. Boot does not manage its version, so you import the **Spring AI BOM** (bill of materials — a POM that pins the versions of a family of artifacts):

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>2.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

The building blocks

Building block	What it does	Real implementations
`ChatModel`	low-level: sends a `Prompt` (list of messages + options), returns a `ChatResponse`	`OpenAiChatModel`, `AnthropicChatModel`, `OllamaChatModel`, ... (our `FakeChatModel`)
`ChatClient`	the fluent, everyday API on top of a `ChatModel`, with advisors	one implementation, built from a `ChatClient.Builder`
Advisor	an interceptor around every `ChatClient` call (like a servlet filter)	logging, memory, RAG, tool calling, safeguard, validation
`ChatMemory`	stores past messages per conversation	`MessageWindowChatMemory` over in-memory / JDBC / other repositories
Structured output	converts the text answer into a Java type	`BeanOutputConverter`, `.entity(...)`
`ToolCallback` / `@Tool`	a Java method the model may ask to run	method tools, function tools, MCP tools
`EmbeddingModel`	turns text into a vector (a fixed-length list) of numbers	OpenAI, Ollama, ONNX transformers, ... (our `FakeEmbeddingModel`)
`VectorStore`	stores document vectors and finds similar ones	`SimpleVectorStore` (in memory), PgVector (a PostgreSQL extension for vectors), Redis, Elasticsearch, Qdrant, ...
MCP	standard protocol to expose / consume tools across processes	MCP server and client starters



Self-check

1. Which Spring AI version goes with Boot 4.1? → 2.0.x (2.0.1 at the time of writing).
2. Why do you import a BOM? → Spring AI is versioned separately from Boot; the BOM keeps all `spring-ai-*` artifacts on the same version.

3. What is the difference between `ChatModel` and `ChatClient`? → `ChatModel` is the provider-level interface (one prompt in, one response out). `ChatClient` is the fluent API on top, with advisors, templates, tools and output conversion.

17.2 Setting up a project

With a real provider (needs an API key — not run)

1. Add the BOM (above) and **one model starter**:

```
<!-- OpenAI -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
<!-- or Anthropic (Claude) -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-anthropic</artifactId>
</dependency>
```

2. Configure it. Keep the key in an environment variable or a secret store, **never** in Git (Chapter 3 explains `${...}` placeholders):

```
# OpenAI (needs an API key — not run)
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.model=gpt-5-mini
spring.ai.openai.chat.temperature=0.2
spring.ai.openai.embedding.model=text-embedding-3-small
```

```
# Anthropic (needs an API key — not run)
spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}
spring.ai.anthropic.chat.model=claude-sonnet-5-5
# a larger model for harder tasks: claude-opus-5-5
spring.ai.anthropic.chat.max-tokens=4096
```

3. Inject `ChatClient.Builder` and use it. (In the properties above, `temperature` controls randomness — lower is more predictable — and `max-tokens` caps the length of the answer.) The code in the rest of this chapter works unchanged with either provider.

Facts worth knowing about Spring AI 2.0 configuration (from the 2.0 upgrade notes):

Change in 2.0	Before (1.x)	Now (2.0)
option properties are **flat**	`spring.ai.openai.chat.options.model=...`	`spring.ai.openai.chat.model=...`
Anthropic module	Spring's own `AnthropicApi` client	the official `anthropic-java` SDK (software development kit: the vendor's own client library); default `max-tokens` raised from 500 to 4096
OpenAI modules	three variants (`openai`, `openai-sdk`, `azure-openai`)	one `spring-ai-openai` (Azure/Microsoft Foundry through its properties)
options objects	mutable, `copy()`	immutable, `mutate()`; `ChatClient.options(...)` takes a **builder**
several model starters on the classpath	—	pick one with `spring.ai.model.chat=openai` / `anthropic` / ...

Default models change over time (Spring AI 2.0's OpenAI default is `gpt-5-mini`, Anthropic's is `claude-haiku-4-5`). Always set the model explicitly so an upgrade does not silently change cost and behaviour.

Offline, as in this chapter

Our `pom.xml` has the BOM plus these artifacts and **no model starter**:

Artifact	Why
`spring-boot-starter`	a plain (non-web) Boot app
`spring-ai-autoconfigure-model-chat-client`	auto-configures a `ChatClient.Builder` bean from whatever `ChatModel` bean exists
`spring-ai-autoconfigure-model-tool`	the tool-calling manager and its properties (an *optional* dependency of the previous one, so you add it yourself — without it Boot logs `TypeNotPresentException ... ToolCallingAutoConfiguration`)
`spring-ai-autoconfigure-model-chat-memory`	a `ChatMemory` bean (window of the last 20 messages, stored in memory)
`spring-ai-vector-store`, `spring-ai-vector-store-advisor`	`SimpleVectorStore` and `QuestionAnswerAdvisor`

A model starter would pull these in for you. Then we declare our own `ChatModel` and `EmbeddingModel` beans (`AiConfig`):

```
@Configuration
public class AiConfig {

    /** With a real starter on the classpath, Boot would create OpenAiChatModel / AnthropicChatModel here. */
    @Bean
    FakeChatModel chatModel() { return new FakeChatModel(); }

    @Bean
    EmbeddingModel embeddingModel() { return new FakeEmbeddingModel(); }
    // ... vectorStore and supportClient beans, shown later
}
```

Why a fake model?

A **fake** (a test double with simple working logic, see Chapter 12) lets you test everything around the model without cost, network, rate limits or random answers. Real LLM output is **non-deterministic**: the same prompt can give different words each time. Tests that assert exact LLM text are flaky. The usual strategy:

What you test	How
your prompts, advisors, tools, parsing, error handling	fake or stub `ChatModel` — fast, deterministic (same result every run), runs in CI (continuous integration: the server that builds and tests every change)
"is the answer good?"	a small **evaluation** set run against the real model, scored by rules or by another model (Spring AI has `RelevancyEvaluator` and `FactCheckingEvaluator`); see AI 8.5
the provider integration	a few tagged integration tests with a real key, run on demand

---

17.3 `ChatModel` and `ChatClient`

`ChatModel`: the provider contract

`ChatModel` has one essential method: `ChatResponse call(Prompt prompt)`. A `Prompt` is a list of **messages** plus **options** (model, temperature, tools, ...). A `ChatResponse` holds one or more `Generation`s, each with an `AssistantMessage`.

Message type	Class	Who writes it
system	`SystemMessage`	you: the rules and persona (the role the model should play: "You are a support assistant...")
user	`UserMessage`	the end user's question (often via a template)
assistant	`AssistantMessage`	the model's reply; may contain **tool calls** instead of text
tool	`ToolResponseMessage`	your application: the result of a tool the model asked for

Our fake implements exactly this interface (`FakeChatModel`). It records each prompt so tests can show what reached "the model", then applies a few rules:

```java
public class FakeChatModel implements ChatModel {

    private final List<Prompt> received = new CopyOnWriteArrayList<>();

    @Override
    public ChatResponse call(Prompt prompt) {
        received.add(prompt);                                   // remember what the model was sent
        return new ChatResponse(List.of(new Generation(reply(prompt))));
    }

    /** Tool-calling-capable default options, like a real provider's options class. */
    @Override
    public ChatOptions getOptions() {
        return ToolCallingChatOptions.builder().build();
    }

    private AssistantMessage reply(Prompt prompt) {
        Message last = prompt.getInstructions().getLast();
        if (last instanceof ToolResponseMessage toolResult) { /* Rule 1: explain the tool result */ }
        String user = prompt.getUserMessage().getText();
        // Rule 2: tools offered + an order id in the question -> return an AssistantMessage with a ToolCall
        // Rule 3: "JSON Schema" in the prompt -> return JSON in a ```json fence
        // Rule 4: RAG context present -> answer from its first line
        // Rule 5: "what is my name" -> look for "my name is X" in earlier messages
        return new AssistantMessage("(fake model) You asked: " + user);
    }
    // ...
}
```

`getOptions()` matters: `ChatClient` copies tool definitions into the prompt's options only when the model's options type supports tools (`ToolCallingChatOptions`). Every real provider's options class does.

`ChatClient`: the everyday API

Boot auto-configures a **prototype-style** (a new instance per injection, instead of one shared instance) `ChatClient.Builder` bean: each injection point gets its own builder, already wired to the `ChatModel` bean and to observability. You customise it and `build()` a `ChatClient`, which is thread-safe (safe to share between threads) and meant to be reused (make it a bean).

```java
@SpringBootTest
class ChatClientTests {

    @Autowired ChatClient.Builder builder;    // auto-configured by Spring AI from the ChatModel bean
    @Autowired FakeChatModel model;

    @Test
    void simplestCall() {
        ChatClient chat = builder.build();
        String answer = chat.prompt()          // start a request
                .user("What is Spring AI?")    // the user message
                .call()                        // blocking call (use .stream() for a Flux)
                .content();                    // just the text of the reply
        System.out.println("answer: " + answer);
        assertThat(answer).isEqualTo("(fake model) You asked: What is Spring AI?");
    }
}
```

Output:

```
answer: (fake model) You asked: What is Spring AI?
```

What `call()` can return:

Method	Returns	Use when
`.content()`	`String`	you only need the text
`.entity(Type.class)`	your Java type	structured output (§17.7)
`.chatResponse()`	`ChatResponse`	you need metadata: token usage, finish reason (why the model stopped, e.g. done or hit the length limit), model id
`.chatClientResponse()`	response + advisor context	you need what advisors stored, e.g. the retrieved documents
`.stream().content()`	`Flux<String>`	show tokens as they arrive (chat UIs, SSE — Chapter 13)

Common mistakes

- Creating a new `ChatClient` for every request instead of one bean per use case.
- Injecting `ChatModel` and calling it directly everywhere, losing advisors, memory and observability. Use `ChatClient` unless you are writing infrastructure.

- Assuming the model remembers the previous call. It does not (§17.6).

17.4 Prompts and prompt templates

What it is

A **prompt template** is text with placeholders, filled at run time — the same idea as a SQL query with bind parameters (`?` placeholders whose values are supplied separately). Spring AI's `PromptTemplate` uses `{name}` placeholders by default (rendered by the StringTemplate engine; the delimiters can be changed with `StTemplateRenderer` if your text contains braces, e.g. JSON examples).

How it works

1. Write the template, inline or in a file such as `prompts/explain.st`: `Explain {topic} to a {audience} in at most {words} words.`
2. Supply values with `render(Map)` / `create(Map)`, or with `.param(...)` on `ChatClient`.
3. A **missing** value is an error, not an empty string — so a typo fails fast.

```
PromptTemplate template = new PromptTemplate(
        "Explain {topic} to a {audience} in at most {words} words.");
String text = template.render(Map.of("topic", "dependency injection", "audience", "beginner", "words", 50));

PromptTemplate fromFile = new PromptTemplate(new ClassPathResource("prompts/explain.st"));
Prompt prompt = fromFile.create(Map.of("topic", "virtual threads", "audience", "Java developer", "words", 80));

new PromptTemplate("Hello {name}, your plan is {plan}.").render(Map.of("name", "Ana"));   // throws
```

Output (`PromptTemplateTests`):

```
rendered: Explain dependency injection to a beginner in at most 50 words.
prompt messages: 1, user text: Explain virtual threads to a Java developer in at most 80 words.
missing variable -> IllegalStateException: Not all variables were replaced in the template. Missing variable names are: [plan].
```

With `ChatClient`, templates and a system message look like this:

```
@Value("classpath:prompts/explain.st") Resource explainTemplate;

ChatClient chat = builder.defaultSystem("You are a concise Java tutor.").build();
chat.prompt()
    .user(u -> u.text(explainTemplate)
                .param("topic", "records")
                .param("audience", "beginner")
                .param("words", 40))
    .call()
    .content();
```

Output — the messages the model actually received:

```
messages the model received:
  SYSTEM: You are a concise Java tutor.
  USER: Explain records to a beginner in at most 40 words.
```

`defaultSystem(...)`, `defaultUser(...)`, `defaultAdvisors(...)`, `defaultTools(...)` and `defaultOptions(...)` on the builder apply to every request of that client. The per-request `.system(...)`, `.user(...)`, `.advisors(...)`, `.tools(...)` and `.options(...)` add to or override them.

Common mistakes

- Concatenating user input into the **system** message. User text belongs in the user message. Mixing them makes **prompt injection** (text crafted to override your instructions to the model) easier (see AI 9.4).
- Templates buried in Java strings. Put long prompts in `src/main/resources/prompts/*.st` files so they can be reviewed and versioned like code.
- Putting JSON examples with `{...}` in a template without changing the delimiters — the renderer treats the braces as placeholders.

Self-check

1. What happens if a template variable is missing? → `IllegalStateException` listing the missing names.
2. Where should long prompts live? → In classpath resource files, loaded as `Resource`.

17.5 Advisors

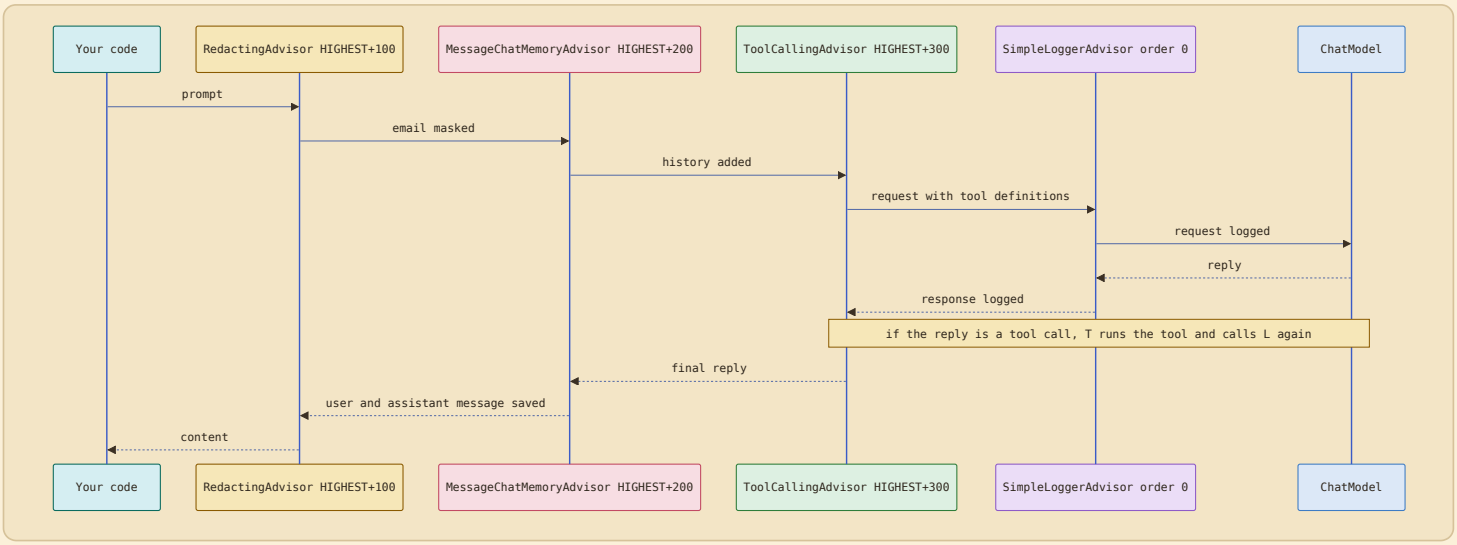
What it is

An **advisor** intercepts every `ChatClient` call on the way **to** the model and on the way **back**. It can change the prompt (add memory, add retrieved documents, redact — hide — data), change the response, or short-circuit the call (answer itself without calling the model). Advisors are to `ChatClient` what servlet filters are to Spring MVC (5.11) and what `@Around` advice is to AOP (Chapter 4).

How it works

1. `ChatClient` builds a `ChatClientRequest` (the `Prompt` + a shared **context** map).
2. Advisors are sorted by `getOrder()` — **lower runs first** on the way in and **last** on the way out.
3. Each advisor calls `chain.nextCall(request)` to pass control on. The last link calls the `ChatModel`.
4. The response travels back through the advisors in reverse order.

`HIGHEST` in the diagram is `Ordered.HIGHEST_PRECEDENCE` (the smallest `int`). The default orders are real Spring AI 2.0 values: memory advisors use `HIGHEST_PRECEDENCE + 200`, the auto-registered `ToolCallingAdvisor` `HIGHEST_PRECEDENCE + 300`, and `SimpleLoggerAdvisor` `0`. So the logger sits *inside* the tool loop and logs every round trip to the model, while memory sits outside it and saves only the final exchange.



Built-in advisors (Spring AI 2.0)

Advisor	Purpose	Section
`SimpleLoggerAdvisor`	logs request and response at DEBUG	below
`MessageChatMemoryAdvisor`	adds past messages of the conversation to the prompt, saves the new exchange	§17.6
`VectorStoreChatMemoryAdvisor`	long-term memory: retrieves *relevant* past messages from a vector store	§17.6
`QuestionAnswerAdvisor`	simple RAG: similarity search (finding the stored texts closest in meaning) + context pasted into the user message	§17.9
`RetrievalAugmentationAdvisor`	modular RAG: query rewriting, expansion, retrieval, joining, augmentation	§17.9
`ToolCallingAdvisor`	runs the tool-call loop; **auto-registered** by `ChatClient` (new in 2.0; replaced the loop inside each `ChatModel`)	§17.8
`SafeGuardAdvisor`	blocks requests containing configured sensitive words	—
`StructuredOutputValidationAdvisor`	validates JSON output against the schema and asks the model to retry if it does not match (new in 2.0)	§17.7

A custom advisor: redacting e-mail addresses

Many companies must not send personal data to an external AI provider. An advisor is the right place to enforce that once, for every call (`RedactingAdvisor`):

```java
public class RedactingAdvisor implements CallAdvisor {

    private static final String EMAIL = "[\\w.+-]+@[\\w-]+\\.[\\w.]+";

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request, CallAdvisorChain chain) {
        Prompt redacted = request.prompt().augmentUserMessage(
                user -> user.mutate().text(user.getText().replaceAll(EMAIL, "[email]")).build());
        return chain.nextCall(request.mutate().prompt(redacted).build());   // pass it on down the chain
    }

    @Override
    public String getName() { return "RedactingAdvisor"; }

    /** Run BEFORE the memory advisor (HIGHEST_PRECEDENCE + 200) so the stored history is redacted too. */
    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE + 100; }
}
```

- `CallAdvisor` handles blocking calls; implement `StreamAdvisor` too if you use `.stream()`. For simple before/after logic, `BaseAdvisor` gives you `before(...)` and `after(...)` hooks.
- Requests and prompts are **immutable**: you create a modified copy (`mutate()`) and pass it on.

The support client wires three advisors as defaults:

```java
@Bean
ChatClient supportClient(ChatClient.Builder builder, ChatMemory chatMemory) {
    return builder
            .defaultSystem("You are a polite support assistant for Acme Shop. Answer briefly.")
            .defaultAdvisors(
                    MessageChatMemoryAdvisor.builder(chatMemory).build(),
                    new RedactingAdvisor(),
                    SimpleLoggerAdvisor.builder()         // logs at DEBUG; here: one short line each way
                            .requestToString(req -> req.prompt().getUserMessage().getText())
                            .responseToString(res -> res.getResult().getOutput().getText())
                            .build())
            .build();
}
```

`SimpleLoggerAdvisor` logs at **DEBUG**, so enable `logging.level.org.springframework.ai.chat.client.advisor=DEBUG` (`application.properties`). By default it logs the whole request and response objects; the two functions above shorten them. The test sends `alice@example.com` and a question about shipping:

```
model saw:      Please send the invoice to [email]
memory stored:  Please send the invoice to [email]
LOG SimpleLoggerAdvisor    : request: Do you ship to Canada?
LOG SimpleLoggerAdvisor    : response: (fake model) You asked: Do you ship to Canada?
```

Because the redactor runs *before* the memory advisor, neither the model nor the stored history ever sees the address. With the order reversed, memory would store the raw address and send it again in every later prompt.

Common mistakes

- Wrong order. Memory must run before RAG and logging if you want them to see the history. Redaction must run before anything that stores or sends text.
- Forgetting to call `chain.nextCall(...)` — the model is never called.
- Logging full prompts in production. Prompts may contain personal data and are large. Spring AI's observations also keep prompt/completion content **out** of logs and traces by default (`spring.ai.chat.client.observations.log-prompt=false`).

In the interview

Q: What is an advisor in Spring AI? → An interceptor around every `ChatClient` call, ordered by `getOrder()`, that can modify the request (add memory, retrieved documents, redaction) or the response. Memory, RAG, logging and — since 2.0 — the tool-calling loop are all advisors.

- *Follow-up: how is it like something else in Spring?* → Like a servlet filter chain or AOP `@Around` advice: each link calls the next.
- *Follow-up: where do you enforce a "no personal data to the provider" rule?* → In an early advisor, so it applies to every call and to stored memory.

17.6 Chat memory

What it is

LLM APIs are **stateless** (the server keeps nothing between calls): each request must carry the whole conversation. "Memory" means your application stores past messages and **re-sends** the relevant ones. Spring AI splits this into:

Piece	Role	Default
`ChatMemoryRepository`	stores messages per conversation id	`InMemoryChatMemoryRepository`; JDBC (`spring-ai-starter-model-chat-memory-repository-jdbc`), Cassandra, Neo4j, Redis, ...
`ChatMemory`	decides **which** messages to keep	`MessageWindowChatMemory`: the last **20** messages (system messages are kept)
memory advisor	loads history into the prompt, saves the new exchange	`MessageChatMemoryAdvisor` (history as messages) or `VectorStoreChatMemoryAdvisor` (relevant old messages from a vector store)

Why a window? The context window of a model is limited and every token costs money (AI 6.9). Long conversations need trimming or summarising.

How it works

1. Every call passes a **conversation id** (a user id, a chat session id).
2. The advisor loads that conversation's messages and puts them before the new user message.
3. After the reply, it saves the user message and the assistant reply.

In Spring AI **2.0 the conversation id is required at call time** (the old default id and the builder's `.conversationId(...)` were removed):

```
private String ask(String conversationId, String text) {
    return support.prompt()
            .user(text)
            .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call()
            .content();
}

ask("alice-42", "Hi, my name is Alice.");
ask("alice-42", "What is my name?");
ask("bob-7",    "What is my name?");
```

Output (`AdvisorTests`):

```
alice> (fake model) You asked: Hi, my name is Alice.
alice> Your name is Alice.
bob>   I don't know your name.
second call for alice-42 sent these messages to the model:
  SYSTEM: You are a polite support assistant for Acme Shop. Answer briefly.
  USER: Hi, my name is Alice.
  ASSISTANT: (fake model) You asked: Hi, my name is Alice.
  USER: What is my name?
```

The fake model has no memory of its own. It "knows" Alice's name only because the advisor re-sent the first exchange. Bob's conversation is separate. The test also asserts that the memory holds 4 messages for `alice-42` and 2 for `bob-7`.

Common mistakes

- Using one fixed conversation id for everybody — users see each other's history. Derive the id from the authenticated user and session (Chapter 10).
- In-memory repository in a cluster: each instance has different memory and it is lost on restart. Use the JDBC (or another shared) repository.
- Unbounded history: cost and latency grow with every turn. Keep a window; summarise older turns if needed.

Self-check

1. Does the model remember earlier calls? → No; the memory advisor re-sends stored messages.
2. What changed for conversation ids in 2.0? → They must be passed per call with `ChatMemory.CONVERSATION_ID`.
3. Which memory for three app instances behind a load balancer? → A shared repository such as JDBC.

17.7 Structured output

What it is

Structured output means getting the answer as data — a record, a list, a map — instead of free text. Spring AI does it with a **converter** that (1) appends format instructions with a **JSON Schema** (a standard JSON document that describes which fields and types a JSON value must have) of your type to the prompt, and (2) parses the reply into your type with Jackson 3. See AI 6.4 for why schemas help.

```java
public record MovieRecommendation(String title, int year, List<String> genres, double score) {}

MovieRecommendation movie = builder.build().prompt()
        .user("Recommend one science-fiction movie about survival.")
        .call()
        .entity(MovieRecommendation.class);          // schema added, reply parsed
```

Output (`StructuredOutputTests`):

```
user message sent to the model (first 3 lines):
  | Recommend one science-fiction movie about survival.
  | Your response should be in JSON format.
  | Do not include any explanations, only provide a RFC8259 compliant JSON response following this format without deviation.
  | ... (31 lines in total, including a JSON Schema)
parsed record: MovieRecommendation[title=The Martian, year=2015, genres=[sci-fi, survival], score=8.0]
```

Notice two things. The converter added about 30 lines to the prompt (tokens you pay for). And the fake replied with JSON **inside a Markdown code fence** (Markdown is the lightweight text-formatting syntax; a fence is the ``` wrapper around code) (```` ```json ````), which models often do; the converter strips the fence before parsing.

The converter can also be used on its own:

```java
var converter = new BeanOutputConverter<>(MovieRecommendation.class);
MovieRecommendation m = converter.convert("""
        {"title":"Arrival","year":2016,"genres":["sci-fi","drama"],"score":7.9}""");
```

```
converted: MovieRecommendation[title=Arrival, year=2016, genres=[sci-fi, drama], score=7.9]
schema mentions the fields: true, true
```

Options

Option	How	Notes
a class / record	`.entity(MovieRecommendation.class)`	most common
generic types	`.entity(new ParameterizedTypeReference<List<MovieRecommendation>>() {})`	lists, maps
map / list converters	`MapOutputConverter`, `ListOutputConverter`	loose structures
native structured output	`.advisors(AdvisorParams.ENABLE_NATIVE_STRUCTURED_OUTPUT)`	the provider itself constrains the output to the schema (only for providers and models that support it, such as OpenAI; check your provider's Spring AI page) — more reliable than instructions
validation + retry	`StructuredOutputValidationAdvisor` (2.0)	checks the JSON against the schema; asks the model again on mismatch

Common mistakes

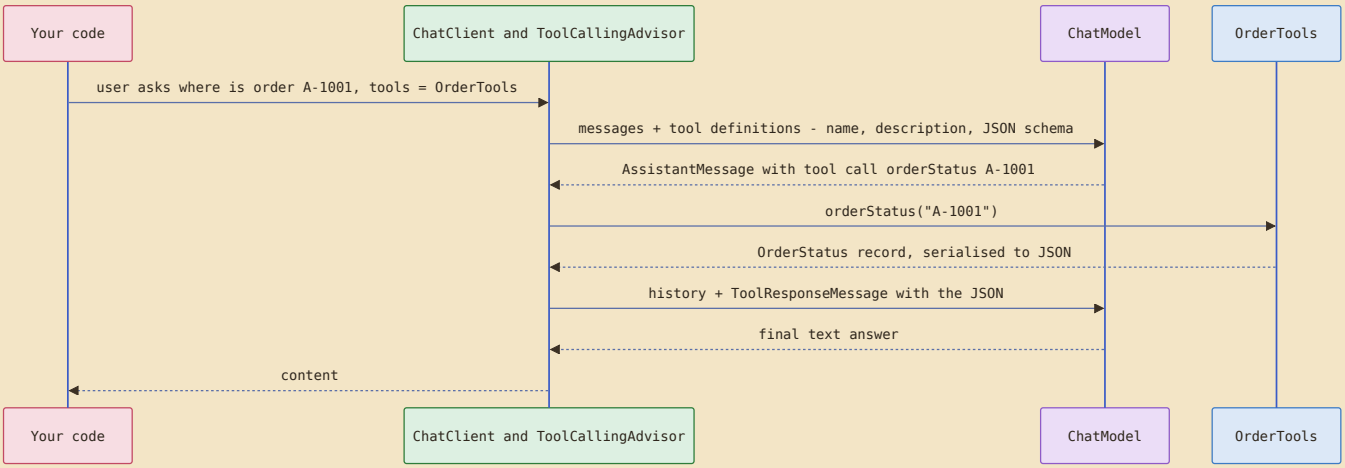
- Trusting the parsed object blindly. The model can still return wrong *values* with the right *shape*. Validate business rules (Bean Validation, 5.6).
- Huge nested types. The schema grows and costs tokens; keep output types small.
- Forgetting that parsing can fail. Catch the exception, retry once, or use the validation advisor.

17.8 Tool calling

What it is

Tool calling (function calling) lets the model ask your application to run a function — look up an order, check stock, call an API — and then answer with the result. The model **never runs code itself**: it only returns a request "call `orderStatus` with `{"orderId":"A-1001"}`". Your application decides whether to run it. See AI 6.5 and AI 9.3 for the concept.

How it works



1. You mark methods with `@Tool` and pass the object with `.tools(...)` (or `.defaultTools(...)` on the builder).
2. Spring AI generates a **tool definition** from the method: name, description, and a JSON Schema of the parameters.
3. The model replies with a **tool call** instead of text.
4. The `ToolCallingAdvisor` (auto-registered since 2.0) runs the method, converts the result to JSON, adds a `ToolResponseMessage`, and calls the model **again**.
5. The loop repeats until the model answers with text.

```
public class OrderTools {

    public record OrderStatus(String orderId, String status, String eta) {}

    @Tool(name = "orderStatus", description = "Look up the delivery status of an order by its id, e.g. A-1001")
    public OrderStatus orderStatus(@ToolParam(description = "the order id") String orderId) {
        return new OrderStatus(orderId, "SHIPPED", "2026-10-02");   // a real app would call the order service
    }
}

String answer = builder.build().prompt()
        .user("Where is my order A-1001?")
        .tools(new OrderTools())               // expose the @Tool methods for this request
        .call()
        .content();
```

Output (`ToolCallingTests`) — the test inspects the two prompts the model received:

```
model calls: 2
1) tools offered to the model: [orderStatus]
2) model asked for: orderStatus {"orderId":"A-1001"}
3) Spring ran the method, sent back: {"orderId":"A-1001","status":"SHIPPED","eta":"2026-10-02"}
4) final answer: Here is what I found via orderStatus: {"orderId":"A-1001","status":"SHIPPED","eta":"2026-10-02"}
no tools -> (fake model) You asked: Where is my order A-1001?
```

One user question cost **two model calls**. Every tool round trip adds latency and tokens.

Details worth knowing (Spring AI 2.0)

Topic	Fact
description	the model decides *when* to call a tool from its **description** — write it like documentation for a new colleague
`returnDirect = true`	return the tool result straight to the caller without a second model call
`ToolContext`	pass hidden data (tenant id, user id) to tools with `.toolContext(Map)`; it is **not** sent to the model
limits	at most **40 calls per tool** and **150 in total** per request by default (`spring.ai.tools.limits.*`), to stop runaway (never-ending) loops
resolution	only tools attached to the request or client can run; resolving arbitrary beans by name is **off** by default in 2.0
errors	by default a tool exception is sent back to the model as an error message (`spring.ai.tools.throw-exception-on-error=false`) so it can recover
programmatic tools	`FunctionToolCallback` / `MethodToolCallback` build tools without annotations

Security: tools are an attack surface

Tools are an **attack surface** (a place where an attacker can try to get in). The model's tool arguments are **untrusted input**. A prompt-injection text in a retrieved document or user message can make the model call tools with harmful arguments. Rules:

- Check authorization **inside** the tool, for the real user (take the user from `ToolContext` or the security context, never from model-supplied arguments).
- Give tools the least power possible: read-only where you can, and a human confirmation step for anything irreversible (AI 9.4).
- Validate arguments like any request body.

In the interview

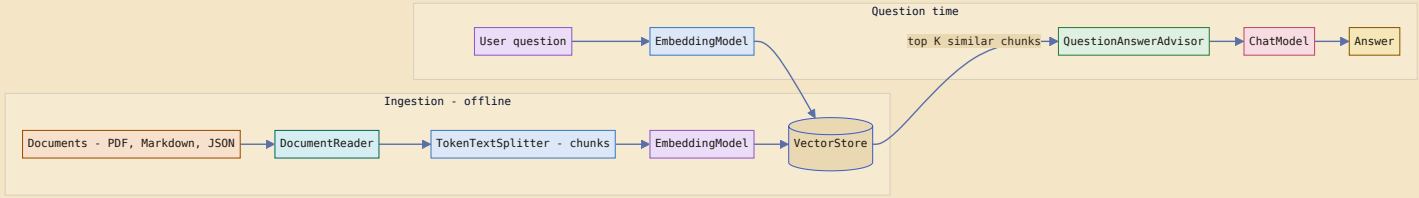
Q: Explain tool calling in Spring AI. → You annotate methods with `@Tool`; Spring AI sends their JSON-schema definitions with the prompt. When the model returns a tool call, the `ToolCallingAdvisor` executes the method, sends the result back as a tool message, and calls the model again until it produces a final answer. The model only *requests* calls; the application executes them.

- *Follow-up: how do you stop an infinite loop?* → Tool-call limits (40 per tool, 150 total by default in 2.0) plus timeouts.
- *Follow-up: how do you pass the current user to a tool safely?* → `ToolContext` or the security context — never trust a user id argument chosen by the model.

17.9 RAG: retrieval-augmented generation

What it is

RAG answers questions from **your** documents: find the most relevant chunks, paste them into the prompt, and tell the model to answer only from them. It reduces made-up answers and lets you use private or fresh data without retraining (teaching the model new data, which is slow and expensive). The design questions (chunking — splitting documents into pieces, hybrid search — combining keyword and meaning search, reranking — re-sorting results with a stronger model, access control, evaluation) are covered in AI Chapter 7; here is the Spring AI wiring.



Piece	Spring AI type	Examples
read	`DocumentReader`	`TextReader`, `JsonReader`, `MarkdownDocumentReader`, `PagePdfDocumentReader`, `TikaDocumentReader`
split	`TokenTextSplitter`	chunks of about 800 tokens by default
embed	`EmbeddingModel`	OpenAI `text-embedding-3-small`, Ollama, ONNX transformers (local)
store and search	`VectorStore`	`SimpleVectorStore` (in memory), PgVector, Redis, Elasticsearch, Qdrant, Milvus, ...
augment	advisor	`QuestionAnswerAdvisor` (simple), `RetrievalAugmentationAdvisor` (modular)

The embedding model and the vector store

An **embedding** is a list of numbers (a vector) representing a text, so that texts with similar meaning have vectors pointing in similar directions (AI 7.2). Similarity is usually the **cosine** of the angle between vectors: 1 = same direction, 0 = unrelated.

Our `FakeEmbeddingModel` is deliberately simple: it hashes each word (turns it into a number with a fixed function; stop words — very common words such as "the" — are skipped, and words are cut to 5 letters) into one of 256 slots and normalises the vector (scales it to length 1). Texts sharing words become similar. A real model captures *meaning* ("refund" is close to "money back"), which word hashing cannot.

```
@Bean
VectorStore vectorStore(EmbeddingModel embeddingModel) {
    SimpleVectorStore store = SimpleVectorStore.builder(embeddingModel).build();
    store.add(List.of(
            new Document("Refunds are issued to the original payment method within 5 business days.",
                    Map.of("source", "refund-policy.md")),
            new Document("Standard shipping takes 3 to 7 days. Express shipping arrives the next day.",
                    Map.of("source", "shipping.md")),
            new Document("You can change your password under Settings, then Security.",
                    Map.of("source", "account.md"))));
    return store;
}
```

`add(...)` embeds each document and stores the vector with its text and **metadata** (source, tenant, date — used for filtering and citations).

```
List<Document> hits = vectorStore.similaritySearch(
        SearchRequest.builder().query("How long do refunds take?").topK(2).build());

List<Document> filtered = vectorStore.similaritySearch(SearchRequest.builder()
        .query("How long do refunds take?").topK(2).similarityThreshold(0.1).build());
```

Output (`RagTests`):

```
score 0.192  refund-policy.md   Refunds are issued to the original payment method within 5 business days.
score 0.000  shipping.md        Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
with similarityThreshold(0.1): 1 document(s)
```

`topK(2)` always returns two documents, even an irrelevant one (score 0.000). A **similarity threshold** drops weak matches. Without it, RAG pastes noise into the prompt. Tune K and the threshold on real questions.

`QuestionAnswerAdvisor`

```
ChatClient chat = builder
        .defaultAdvisors(QuestionAnswerAdvisor.builder(vectorStore)
                .searchRequest(SearchRequest.builder().topK(1).similarityThreshold(0.2).build())
                .build())
        .build();

String answer = chat.prompt().user("How fast is express shipping?").call().content();
```

Output — the user message after the advisor rewrote it:

```
augmented user message:
  | How fast is express shipping?
  |
  | Context information is below, surrounded by ---------------------
  |
  | ---------------------
  | Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
  | ---------------------
  |
  | ...
answer: According to our docs: Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
```

The rest of the default template (cut above) tells the model to answer from the context and to say so when the answer is not there. You can replace the template with `.promptTemplate(...)`, and restrict the search per request with a **filter expression** on metadata, e.g. `a.param(QuestionAnswerAdvisor.FILTER_EXPRESSION, "tenant == 'acme'")` — this is how you enforce document-level permissions (AI 7.9). The retrieved documents are also stored in the response context under `QuestionAnswerAdvisor.RETRIEVED_DOCUMENTS`, which you can use to show citations.

For more control, `RetrievalAugmentationAdvisor` (module `spring-ai-rag`) builds a pipeline from parts: `RewriteQueryTransformer`/`CompressionQueryTransformer` (improve the query), `MultiQueryExpander`, `VectorStoreDocumentRetriever`, a `DocumentJoiner`, post-processors (e.g. reranking) and `ContextualQueryAugmenter`.

A production vector store (needs a database — not run)

`SimpleVectorStore` keeps everything in memory (it can `save`/`load` a JSON file) — fine for tests and demos, not for real data. With PostgreSQL + the `pgvector` extension:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>
```

```
# (needs PostgreSQL with pgvector — not run)
spring.datasource.url=jdbc:postgresql://localhost:5432/docs
spring.ai.vectorstore.pgvector.initialize-schema=true
spring.ai.vectorstore.pgvector.index-type=hnsw
spring.ai.vectorstore.pgvector.distance-type=cosine-distance
spring.ai.vectorstore.pgvector.dimensions=1536
```

`dimensions` must match the embedding model (1536 for `text-embedding-3-small`). Changing the embedding model means **re-embedding every document**. `hnsw` (Hierarchical Navigable Small World) is an approximate nearest-neighbour index (AI 7.3); `initialize-schema=true` is convenient in development, but in production create the table with Flyway (8.16).

Common mistakes

- No similarity threshold — irrelevant chunks confuse the model.
- Chunks too large (costly, diluted) or too small (no context). Start around a few hundred tokens with some overlap.
- Ignoring permissions: every user can retrieve every document. Store tenant/role in metadata and filter.

- Mixing embeddings from two different models in one store — the vectors are not comparable.

Self-check

1. What does `QuestionAnswerAdvisor` add to the prompt? → The top matching documents, wrapped in a template that says to answer from them.
2. Why a similarity threshold? → `topK` always returns K documents; the threshold drops weak matches.
3. What must match between the embedding model and a PgVector table? → The number of dimensions (and the model itself).

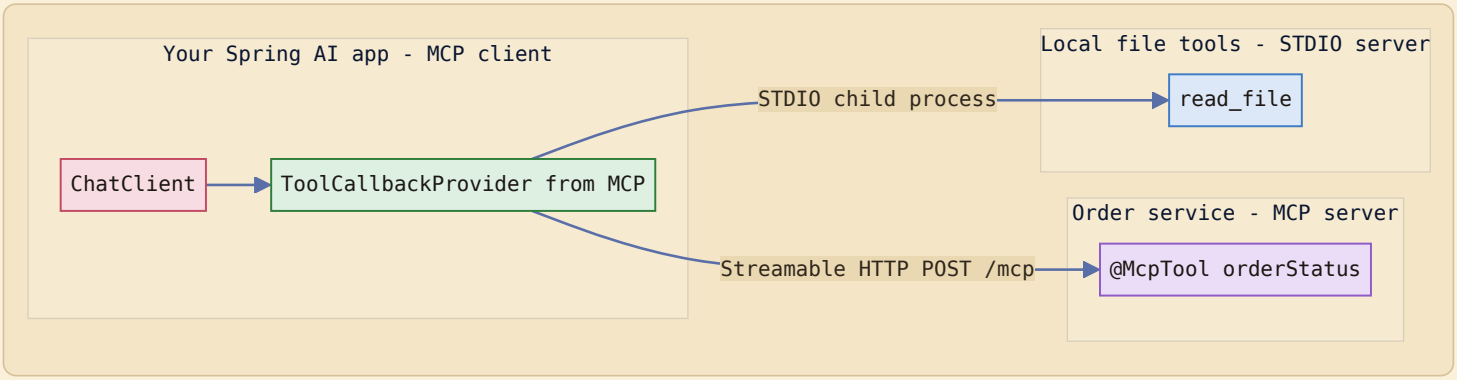
17.10 MCP: the Model Context Protocol

What it is

MCP is an open protocol (started by Anthropic in November 2024, now widely adopted) that standardises how an AI application discovers and calls **tools**, reads **resources** and uses **prompts** exposed by another process. Think "USB for AI tools": write a tool server once, and any MCP-capable host (Claude Desktop, IDE — code editor — assistants, your own Spring AI app) can use it. Concepts and security are in AI 6.10 and AI 9.8.

Role	What it does	Spring AI starter
MCP server	exposes tools/resources/prompts	`spring-ai-starter-mcp-server` (STDIO), `spring-ai-starter-mcp-server-webmvc`, `spring-ai-starter-mcp-server-webflux`
MCP client	connects to servers, turns their tools into `ToolCallback`s	`spring-ai-starter-mcp-client`, `spring-ai-starter-mcp-client-webflux`

Transports: **STDIO** (standard input/output: the host starts the server as a child process and they talk through its input and output streams — local tools) and **Streamable HTTP** (a remote server; the default for Spring AI 2.0's web servers, endpoint `POST /mcp`). The older SSE transport is deprecated since 2.0.



An MCP server (snippet — not run)

```
// spring-ai-starter-mcp-server-webmvc on the classpath
import org.springframework.ai.mcp.annotation.McpTool;
import org.springframework.ai.mcp.annotation.McpToolParam;
import org.springframework.stereotype.Component;

@Component
public class OrderMcpTools {

    @McpTool(name = "orderStatus", description = "Delivery status of an order by id")
    public OrderStatus orderStatus(@McpToolParam(description = "order id", required = true) String orderId) {
        return orderService.status(orderId);
    }
}
```

```
spring.ai.mcp.server.name=order-tools
# already the default in 2.0 (endpoint /mcp); shown for clarity
spring.ai.mcp.server.protocol=STREAMABLE
```

In Spring AI 2.0 the MCP annotations live in `org.springframework.ai.mcp.annotation` (they came from the `org.springaicommunity.mcp` community project in 1.1). Beans with `@McpTool`, `@McpResource`, `@McpPrompt` methods are discovered automatically.

An MCP client (snippet — needs a running server, not run)

```
spring.ai.mcp.client.streamable-http.connections.orders.url=http://localhost:8081
# a local STDIO server, started by the client:
spring.ai.mcp.client.stdio.connections.files.command=npx
spring.ai.mcp.client.stdio.connections.files.args=-y,@modelcontextprotocol/server-filesystem,/data/docs
```

```java
@Bean
ChatClient agent(ChatClient.Builder builder, ToolCallbackProvider mcpTools) {
    return builder.defaultToolCallbacks(mcpTools).build();   // MCP tools behave like @Tool methods
}
```

The client auto-configuration exposes the remote tools as a `ToolCallbackProvider` bean (`spring.ai.mcp.client.toolcallback.enabled=true` by default), so the tool-calling loop from §17.8 works unchanged.

Trade-offs

	In-process `@Tool`	MCP tool
latency	method call	network or process hop
reuse	this app only	any MCP host, any language
deployment	ships with the app	separate service, separate lifecycle
security	your normal code review	you now trust a remote server's tool descriptions and results — authenticate it (OAuth 2 for HTTP servers), allow-list (explicitly permit only named) tools, treat results as untrusted input

Use `@Tool` for logic that belongs to your application. Use MCP to share tools across teams or AI hosts, or to consume existing MCP servers.

17.11 Production concerns

Concern	What Spring AI gives you	Also see
observability	Micrometer observations: `spring.ai.chat.client`, `spring.ai.advisor`, `gen_ai.client.operation` (model call, with `gen_ai.client.token.usage` metrics), `spring.ai.tool`, `db.vector.client.operation`. Prompt/completion content is **not** recorded unless you enable `log-prompt` / `log-completion`	Chapter 15
cost	token usage in `chatResponse().getMetadata().getUsage()`	AI 6.6
retries and timeouts	provider SDK retries (e.g. `spring.ai.openai.chat.max-retries`, default 3) and `spring.ai.retry.*` for providers using Spring's retry; set HTTP timeouts	AI 6.7, 4.7
streaming	`.stream().content()` returns `Flux<String>`; send it to browsers with SSE	Chapter 13
blocking calls	model calls take seconds; on Spring MVC turn on virtual threads (`spring.threads.virtual.enabled=true`) so waiting threads are cheap	Java 15.4
safety	redaction advisor, `SafeGuardAdvisor`, output validation, tool permissions	AI Chapter 12
testing	fake `ChatModel` for logic, evaluators for quality	§17.2

Common mistakes (Spring AI, whole part)

- Calling the model inside a database transaction (Chapter 7): the transaction and its connection stay open for seconds.
- No timeout or limit on tool loops and output tokens.
- Letting the model's text reach SQL, shell commands or HTML without validation or escaping (neutralising special characters so text cannot act as code).
- Hard-coding a model id that the provider later retires — keep it in configuration.
- Tests that call the real API on every build.

In the interview

Q: How would you add an AI assistant to an existing Spring Boot service? → Add the Spring AI BOM and a model starter, configure the key and model in properties, build a `ChatClient` bean with a system prompt and advisors (memory with a per-user conversation id, RAG over our documents with metadata filters, logging/redaction), expose business functions as `@Tool` methods that check authorization themselves, and return typed results with `.entity(...)`. Test logic with a fake `ChatModel`, measure quality with an evaluation set, watch token usage and latency through Micrometer.

- *Follow-up: why Spring AI rather than the provider's SDK?* → Portability across providers, and the Spring integration: auto-configuration, observability, advisors, vector stores, testing. For one provider-specific feature you can still reach the native options.
- *Follow-up: how do you keep answers grounded (based only on real sources, not invented)?* → RAG with a similarity threshold, a template that says "answer only from the context", citations from retrieved-document metadata, and evaluation of faithfulness.

Spring AI TL;DR

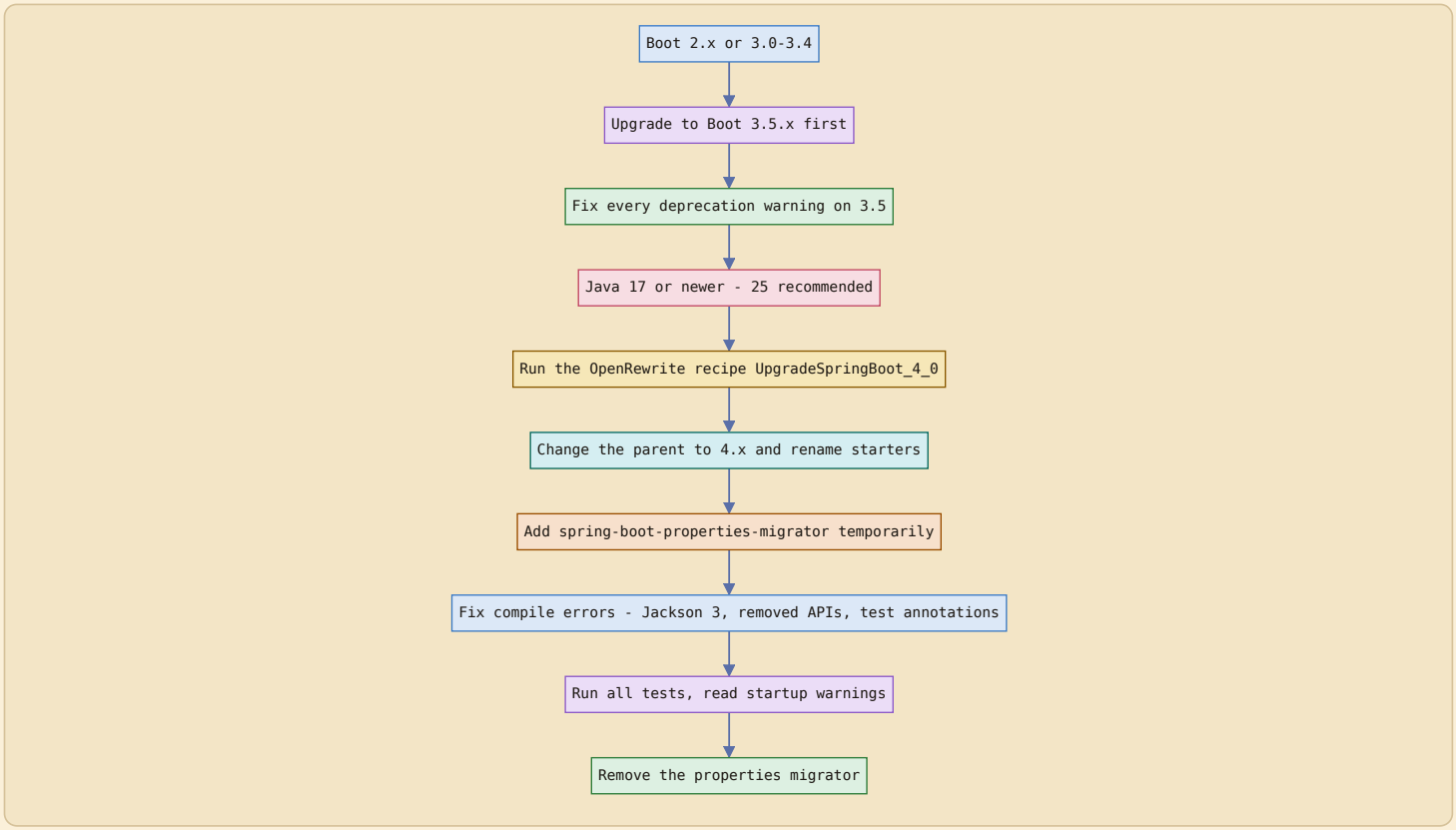
- Spring AI **2.0.x** for Boot 4 (1.1.x for Boot 3.5). Import `spring-ai-bom`, add one model starter, set `spring.ai.<provider>.api-key` and `...chat.model`.
- `ChatClient` = fluent API over `ChatModel`; `.content()`, `.entity(...)`, `.stream()`.
- **Advisors** wrap every call (ordered, lower first): logging, memory, RAG, tool calling, your own (redaction).
- **Memory** re-sends history; pass `ChatMemory.CONVERSATION_ID` per call; window of 20 by default.
- **Structured output**: schema instructions + Jackson parsing; native mode and validation advisor for reliability.
- **Tools**: `@Tool` methods; the model requests, `ToolCallingAdvisor` executes and loops (limits 40/150). Authorize inside tools.
- **RAG**: reader → splitter → `EmbeddingModel` → `VectorStore`; `QuestionAnswerAdvisor` with threshold and metadata filters.
- **MCP**: share tools across processes; `@McpTool` servers, client tools arrive as a `ToolCallbackProvider`.
- Test with a **fake model**; evaluate quality separately.

17.12 What's new: how to read the rest of this chapter

The Spring generation that arrived in **November 2025** (Framework 7.0 on 13 Nov, Boot 4.0 on 20 Nov, Security 7.0) is the biggest change since Boot 3 moved everything from `javax.*` to `jakarta.*` (Java's enterprise APIs — servlets, JPA, validation — were handed to the Eclipse Foundation as **Jakarta EE** and had to change package names). This book targets **Boot 4.1.1**, whose dependency BOM gives you Framework 7.0.9, Security 7.1.1, Hibernate ORM 7.4, Spring Data 2026.0, Jackson 3.1.5, Tomcat 11.0.24 and JUnit 6.0.3 (1.4).

The tables below are a **map**, not a second textbook: one line per change, with a pointer to the chapter that explains and demonstrates it. They were checked against the Boot 4.0 migration guide, the Boot 4.0 and 4.1 release notes, the Framework 7.0 release notes and the Spring Security 7 reference.

A safe upgrade path (Boot 3.x → 4.x)



1. **Go to 3.5 first.** The migration guide assumes 3.5; code deprecated (marked as "will be removed, stop using") in 3.x is removed in 4.0.
2. **Automate the boring part** with OpenRewrite (a tool that applies automated, rule-based source-code changes called *recipes*; not run here):

   ```
   mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
     -Drewrite.recipeArtifactCoordinates=org.openrewrite.recipe:rewrite-spring:RELEASE \
     -Drewrite.activeRecipes=org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0
   ```

   It updates versions, renames starters and properties, and applies the Framework 7 / Security 7 sub-recipes. Review its diff (the list of line-by-line changes) like any other change.
3. `spring-boot-properties-migrator` (runtime dependency, temporarily) reports renamed/removed properties at startup and maps the old names for you while you fix them.

4. If you are stuck, the **"classic" starters** (`spring-boot-starter-classic`, `spring-boot-starter-test-classic`) bring back the old all-in-one auto-configuration jar layout to get you compiling, then move to the modular starters.

17.13 Spring Boot 3.x → 4.x migration table

Baselines

Item	Boot 3.x	Boot 4.x	Where in this book
Java	17+	17+ (25 recommended; this book uses 25)	1.4
Spring Framework	6.x	7.0	§17.14
Jakarta EE	10 (Servlet 6.0 — the Java web request/response API)	**11** (Servlet 6.1, JPA 3.2, Bean Validation 3.1)	1.4
Embedded servers	Tomcat 10.1, Jetty 12, Undertow	Tomcat 11, Jetty 12.1; **Undertow removed** (no Servlet 6.1 support)	Chapter 1
Hibernate ORM	6.x	7.x (7.4 in Boot 4.1)	Chapter 8
JUnit	5	**6** (JUnit 4 support in the TestContext framework deprecated)	Chapter 12
Kotlin / GraalVM	1.7+ / 22.3+	Kotlin 2.2+ / GraalVM 25+	Chapter 15
Gradle	7.x / 8.x	8.14+ or 9	1.6

Modularization and starters

Change	What to do	Where
`spring-boot-autoconfigure` split into **one module per technology** (`spring-boot-webmvc`, `spring-boot-jdbc`, `spring-boot-jackson`, ...) with its own starter and **test starter**	use the technology starters; imports of auto-configuration classes (for `exclude`) changed packages	3.14
`spring-boot-starter-web` → **`spring-boot-starter-webmvc`**	rename (the old name remains as a deprecated alias)	1.7
`spring-boot-starter-aop` → **`spring-boot-starter-aspectj`**	rename	4.2
`spring-boot-starter-oauth2-client` / `-resource-server` / `-authorization-server` → `spring-boot-starter-security-oauth2-...`	rename	Chapter 11
`spring-boot-starter-web-services` → `spring-boot-starter-webservices`	rename	—
test support per module: `spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test`, ...	add the test starter for each technology you test	Chapter 12
Flyway / Liquibase auto-configured only with **`spring-boot-starter-flyway`** / `-liquibase` (the library alone is no longer enough)	add the starter	8.16
`RestClient` / `RestTemplate` builders come from **`spring-boot-starter-restclient`**	add it for client code	Chapter 6
`@EntityScan` moved to `org.springframework.boot.persistence.autoconfigure`	fix the import	Chapter 8
`hibernate-jpamodelgen` → `hibernate-processor`	rename the annotation processor (a compiler plug-in that generates code from annotations)	Chapter 8
Spring Batch runs **without a database** by default	use `spring-boot-starter-batch-jdbc` for JDBC job metadata	—

Jackson 3

Change	What to do	Where
Jackson 3 is the default; packages and group ids moved from `com.fasterxml.jackson` to **`tools.jackson`** (annotations such as `@JsonProperty` keep `com.fasterxml.jackson.annotation`)	fix imports; `ObjectMapper` usage mostly becomes `JsonMapper`	5.5
`Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`; `@JsonMixin` → `@JacksonMixin`	rename	5.5
JSON-specific parser/generator features moved from `spring.jackson.parser.*` / `generator.*` to **`spring.jackson.json.read.*`** / **`spring.jackson.json.write.*`** (`spring.jackson.read.*` / `write.*` remain for format-neutral stream features); `spring.jackson.use-jackson2-defaults=true` restores Jackson 2 behaviour	update properties	5.5
Jackson 2 still possible through the **deprecated** `spring-boot-jackson2` module (`spring.jackson2.*`); Framework plans to disable its auto-detection in 7.1 and remove it in 7.2	migrate now	5.5
`HttpMessageConverters` bean type deprecated	use `ServerHttpMessageConvertersCustomizer` / `ClientHttpMessageConvertersCustomizer`	Chapter 5

Null-safety, testing, and other code changes

Change	What to do	Where
Boot and Framework annotated with **JSpecify** (`@NullMarked`, `@Nullable`); Spring's own `org.springframework.lang.Nullable` deprecated	use `org.jspecify.annotations` in your code; Kotlin and null checkers may flag new warnings	2.13
`@MockBean` / `@SpyBean` **removed**	`@MockitoBean` / `@MockitoSpyBean`	Chapter 12
`@SpringBootTest` no longer sets up MockMvc, `WebTestClient` or `TestRestTemplate` on its own	add `@AutoConfigureMockMvc`, `@AutoConfigureRestTestClient`, `@AutoConfigureTestRestTemplate`	Chapter 12
`TestRestTemplate` moved to `org.springframework.boot.resttestclient` (module `spring-boot-resttestclient`)	fix imports, or switch to the new **`RestTestClient`**	1.12
`EnvironmentPostProcessor` moved to `org.springframework.boot`; `BootstrapRegistry` to `org.springframework.boot.bootstrap`	update imports and `spring.factories`	Chapter 3
Spring Retry no longer version-managed	declare its version, or move to Framework 7 `@Retryable`	4.7
Spring Authorization Server moved into Spring Security 7 (`spring-authorization-server.version` property gone)	use `spring-security.version`	11.14

Property renames and changed defaults (the most common)

Before (3.x)	After (4.x)
`spring.jackson.parser.*`, `spring.jackson.generator.*`	`spring.jackson.json.read.*`, `spring.jackson.json.write.*`
`spring.data.mongodb.host` / `.port` / `.uri` / `.database` / ...	`spring.mongodb.host` / `.port` / `.uri` / `.database` / ...
`management.health.mongo.enabled`	`management.health.mongodb.enabled`
`server.error.*` (`include-stacktrace`, `include-message`, `path`, `whitelabel.enabled`, ...)	`spring.web.error.*` — the old keys are no longer bound since 4.0.0
`spring.session.redis.*`	`spring.session.data.redis.*` (Spring Session MongoDB left Boot, see below)
`spring.dao.exceptiontranslation.enabled`	`spring.persistence.exceptiontranslation.enabled`
`spring.kafka.retry.topic.backoff.random`	`spring.kafka.retry.topic.backoff.jitter`
DevTools LiveReload on by default	`spring.devtools.livereload.enabled` defaults to **`false`** (and LiveReload is deprecated in 4.1)
Kubernetes liveness/readiness probes only on Kubernetes	health **probes enabled by default** (`management.endpoint.health.probes.enabled`) — Chapter 15
`server.forward-headers-strategy` applied to war deployments (apps packaged as `.war` files and installed into a separate servlet server)	no longer applies to wars — register a `ForwardedHeaderFilter` yourself

Removed features

Removed in Boot 4.0	Replacement
Undertow	Tomcat or Jetty
executable "fully executable" jar launch scripts	run with `java -jar`, systemd, containers
Maven `CLASSIC` loader implementation	the default loader
optional dependencies packed into the uber jar (the single "fat" jar containing the app and all its libraries)	`<includeOptional>true</includeOptional>` if you really need them
Spock integration (Groovy 5 incompatibility)	**back in Boot 4.1** with Spock 2.4
Spring Session Hazelcast / MongoDB (moved to those vendors)	the vendors' own modules
Pulsar reactive support	Spring Pulsar imperative API

New in Boot 4.0

Feature	Where
API versioning auto-configuration (`spring.mvc.apiversion.*`, `spring.webflux.apiversion.*`)	5.9
HTTP service clients auto-configured: `@ImportHttpServices` + `spring.http.serviceclient.<group>.*`	6.6
`spring.http.clients.*` for the auto-configured HTTP client library and timeouts	6.4
`RestTestClient` in tests	1.12, Chapter 12
`spring-boot-starter-opentelemetry` (OTLP metrics and traces)	Chapter 15
`JmsClient` auto-configuration	Chapter 14
several `TaskDecorator` beans combined automatically	Chapter 14
`logging.console.enabled=false` to turn console logging off	Chapter 15
Kotlin serialization starter	—

Self-check

1. You upgrade and your app no longer runs Flyway migrations. Why? → Boot 4 auto-configures Flyway only with `spring-boot-starter-flyway`.
2. `@MockBean` no longer compiles. Replacement? → `@MockitoBean` (and `@MockitoSpyBean` for `@SpyBean`).
3. Why does `import com.fasterxml.jackson.databind.ObjectMapper` fail? → Boot 4 uses Jackson 3 in `tools.jackson.*`; only the annotations kept the old package.
4. Your `@SpringBootTest` can no longer inject `MockMvc`. Fix? → Add `@AutoConfigureMockMvc`.

17.14 Spring Framework 6 → 7

Area	What changed in 7.0	Where
baselines	Java 17 (25 recommended), Jakarta EE 11, Servlet 6.1, JPA 3.2, Bean Validation 3.1, Kotlin 2.2, GraalVM 25, Netty 4.2, JUnit 6	1.4
null-safety	whole codebase annotated with **JSpecify**, including generics, arrays and varargs; JSR 305-style Spring annotations deprecated	2.13
programmatic beans	new `BeanRegistrar` contract for registering beans with code (loops, conditions)	2.14
resilience	`RetryTemplate` / `RetryPolicy` in `spring-core` (`org.springframework.core.retry`), `@Retryable` and `@ConcurrencyLimit` in `spring-context`, enabled with **`@EnableResilientMethods`**; `@Retryable` also adapts to reactive return types	4.7
proxies	CGLIB (the library that creates proxies by generating subclasses) default applied consistently to all proxy processors (including `@Async`); **`@Proxyable`** overrides per bean	4.4
API versioning	first-class in Spring MVC and WebFlux: resolve the version from header, query parameter, path segment or media type; deprecation hints; client-side support in `RestClient`, `WebClient`, HTTP interfaces; testing support	5.9, 6.7
HTTP interface clients	group-based configuration with `@ImportHttpServices` and `HttpServiceGroupConfigurer`; `InputStream` / `OutputStream` support	6.6
`RestTemplate`	deprecated in the reference docs in 7.0, `@Deprecated` planned for 7.1, removal in 8.0	6.1
Jackson 3	default JSON library across MVC, WebFlux, messaging; Jackson 2 support deprecated	5.5
message converters	new `HttpMessageConverters` builder API (`configureMessageConverters(ServerBuilder)`)	Chapter 5
path matching	`PathMatcher` / `AntPathMatcher` for MVC mappings deprecated in favour of `PathPattern`, which now supports `/**` at the start of a pattern; suffix-pattern and trailing-slash matching options removed	5.3
`HttpHeaders`	no longer a `MultiValueMap`; some map methods removed	Chapter 5
data access	JPA 3.2: inject `EntityManagerFactory` / `EntityManager` with `@Autowired`; Hibernate 7 support moved to `orm.jpa.hibernate`; `JdbcClient` gains fetch size, max rows, query timeout	Chapter 7, Chapter 8
messaging	new **`JmsClient`** (like `JdbcClient` / `RestClient` for JMS)	Chapter 14
testing	**`RestTestClient`** (non-reactive `WebTestClient`), test application contexts **paused** when not in use, `@MockitoBean` on non-singleton beans, better `@Nested` injection	Chapter 12
SpEL (Spring Expression Language, the `#{...}` expressions)	`Optional` support with `?.` and Elvis (`?:`, "use this default if null"); since 7.0.8 a default limit of 10,000 operations per evaluation	—
Java 24+	bytecode metadata read with the JDK **Class-File API** (JEP 484) — transparent	Java Ch. 17
removed	`javax.annotation` / `javax.inject` support (use `jakarta.*`), `spring-jcl` (Apache Commons Logging 1.3 instead), `ListenableFuture` (use `CompletableFuture`), OkHttp3 support, `Theme` support, `webjars-locator-core` (use `webjars-locator-lite`), Undertow-specific classes	2.15
deprecated	JUnit 4 support (`SpringRunner`, ...), `<mvc:*>` XML namespace, document/feed views (PDF, XLS, RSS), BeanShell	—

In the interview

Q: What's new in Spring Framework 7 / Boot 4? → Jakarta EE 11 and Java 17 baseline (25 recommended); JSpecify null-safety; built-in API versioning; HTTP service client groups with Boot auto-configuration; core resilience (`@Retryable`, `@ConcurrencyLimit`, `RetryTemplate`); `BeanRegistrar`; Jackson 3; `RestTestClient`; and in Boot, modular auto-configuration with a starter and test starter per technology.

- *Follow-up: what breaks first when you upgrade?* → Jackson imports (`tools.jackson`), renamed starters (web → webmvc, aop → aspectj), Flyway needing its starter, removed `@MockBean`, `javax.*` annotations, and `.and()`-style security configuration.
- *Follow-up: do you still need Spring Retry?* → Usually not: Framework 7's `@Retryable` covers retries with backoff and jitter. For circuit breakers use Resilience4j (Chapter 16).

17.15 Spring Security 6 → 7

Change	Details	Where
lambda DSL only	`.and()` chaining removed; one lambda per feature	10.3
`authorizeRequests()` removed	use `authorizeHttpRequests()` (built on `AuthorizationManager`)	10.3
`AntPathRequestMatcher`, `MvcRequestMatcher` removed	**`PathPatternRequestMatcher`** (same syntax as Spring MVC)	10.3
`AccessDecisionManager` / voters	moved out of the main jar to the legacy `spring-security-access` module; use `AuthorizationManager`	10.6
multi-factor authentication	every login mechanism adds a factor authority (`FACTOR_PASSWORD`, `FACTOR_OTT`, `FACTOR_WEBAUTHN`); `@EnableMultiFactorAuthentication` or `AuthorizationManagerFactories.multiFactor()` require several factors	10.4
CSRF for single-page apps	`csrf(c -> c.spa())`	10.8
password encoders	Password4j-based encoders (Argon2, bcrypt, scrypt, PBKDF2, balloon hashing)	10.5
Spring Authorization Server	now shipped inside Spring Security 7	11.14
OAuth2 password grant	client support removed	11.2
PKCE	used by default for confidential clients too	11.9
JWT	`NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)` builders; `JwtTypeValidator` checks the `typ` header	11.3, 11.4
HTTP interfaces + OAuth2	`@ClientRegistrationId` on a whole HTTP interface	11.10
7.1 (in Boot 4.1)	`RestClientOpaqueTokenIntrospector`; `charset` parameter in `WWW-Authenticate: Basic`	11.7, 10.1

17.16 What Boot 4.1 adds (June 2026)

Change in 4.1	Notes	Where
everything **deprecated in 4.0 removed**	upgrade 4.0 → 4.1 with zero deprecation warnings first	—
`layertools` jar mode removed	use the `tools` jar mode for layered images	Chapter 15
gRPC (Google's binary remote-procedure-call protocol) support (server and client, Netty or Servlet over HTTP/2)	new	—
Spock support back (Spock 2.4, Groovy 5)		Chapter 12
`spring.datasource.connection-fetch=lazy`	wraps the pool in `LazyConnectionDataSourceProxy`: no connection taken until the first statement	7.2
`@ConfigurationProperties` records bind `Optional` parameters as `Optional.empty()` instead of `null`		3.6
`spring.security.oauth2.resourceserver.jwt.authorities-claim-expressions`	map authorities from JWT claims with properties	11.5
observability: context propagation into `@Async` methods, `management.opentelemetry.*` properties, Kafka/RabbitMQ observation conventions		Chapter 15
HTTP clients: cookie handling (`spring.http.clients.cookie-handling`), `InetAddressFilter` against SSRF (server-side request forgery: tricking a server into calling internal addresses)		Chapter 6
`@RedisListener` auto-configuration		Chapter 9
dependency upgrades: Hibernate 7.4, Spring Data 2026.0, Micrometer 1.17		Chapter 8
deprecated: DevTools LiveReload, Derby support		—

Common migration mistakes

- Jumping from Boot 2.7 straight to 4.x. Go 2.7 → 3.5 (the `javax` → `jakarta` step) → 4.x.

- Ignoring startup warnings after the upgrade. Removed auto-configuration usually shows up as "bean not found", not as a compile error.
- Keeping `spring-boot-properties-migrator` in production.
- Upgrading Boot but not the separately versioned projects: **Spring Cloud** needs its 2025.1 "Oakwood" train for Boot 4 (2025.1.2 or later for Boot 4.1) (Chapter 16), **Spring AI** needs 2.0, **springdoc-openapi** needs its Boot 4 line (5.12).

17.17 In the interview — quick round

Question	30-second answer
What is Spring AI?	A Spring project giving portable APIs (`ChatModel`, `ChatClient`, `EmbeddingModel`, `VectorStore`) and Boot starters for AI providers, plus advisors for memory, RAG and tool calling. (§17.1)
`ChatModel` vs `ChatClient`?	Provider-level interface vs fluent API with templates, advisors, tools, structured output. (§17.3)
How does chat memory work?	The model is stateless; an advisor stores messages per conversation id and re-sends a window of them. (§17.6)
How do you get a Java object from an LLM?	`.entity(MyRecord.class)`: JSON Schema instructions + Jackson parsing; native structured output where supported. (§17.7)
Who executes a tool?	The application. The model only requests the call; `ToolCallingAdvisor` runs it and loops. (§17.8)
How do you test Spring AI code without paying?	A fake `ChatModel` bean with canned answers; real-model evaluations separately. (§17.2)
Main Boot 4 changes?	Modular starters, Jackson 3, JSpecify, Jakarta EE 11, API versioning, HTTP service clients, `@MockitoBean`, Undertow removed. (§17.13)
Main Framework 7 changes?	JSpecify, API versioning, `@ImportHttpServices`, `@Retryable` / `@ConcurrencyLimit`, `BeanRegistrar`, `RestTestClient`, Jackson 3. (§17.14)
Main Security 7 changes?	Lambda DSL only, `PathPatternRequestMatcher`, MFA factors, Authorization Server included, password grant removed. (§17.15)

Chapter TL;DR

- **Spring AI 2.0.x** is the version for Boot 4.x: BOM + one model starter + `spring.ai.<provider>.api-key` / `.chat.model` (e.g. `gpt-5-mini`, `claude-sonnet-5-5`, `claude-opus-5-5`). Set the model explicitly.
- `ChatClient` over `ChatModel`; prompts are messages (system, user, assistant, tool); `PromptTemplate` fills `{placeholders}` and fails on missing values.
- **Advisors** are the extension point: logging, memory (pass `ChatMemory.CONVERSATION_ID`), RAG (`QuestionAnswerAdvisor` + threshold + filters), tool calling (auto-registered in 2.0), your own (redaction). Order matters.
- **Structured output** with `.entity(...)`; **tools** with `@Tool` (authorize inside, limits 40/150); **MCP** to share tools across processes (`@McpTool`, Streamable HTTP).
- Everything above the model runs offline against a **fake** `ChatModel` — 14 tests in this chapter prove it.
- **Boot 3 → 4**: go via 3.5, run OpenRewrite, rename starters (webmvc, aspectj, security-oauth2-*), add test and Flyway starters, move to Jackson 3 (`tools.jackson`), `@MockitoBean`, JSpecify, Jakarta EE 11; Undertow gone.
- **Framework 7**: API versioning, HTTP service groups, core resilience, `BeanRegistrar`, `RestTestClient`, JSpecify; `RestTemplate` on its way out.
- **Security 7**: lambda DSL only, `authorizeHttpRequests`, `PathPatternRequestMatcher`, MFA, Authorization Server inside.

Next: Chapter 18 — Spring interview questions turns the whole book into questions and answers for revision.

Chapter 18 — Spring Interview Questions

This chapter is the review round for the whole book. It collects **168 interview questions** about Spring Framework 7.0, Spring Boot 4.1 and Spring Security 7, each with a short, correct answer and a link to the section that teaches it. After the questions come a template for the most common open question ("explain a Spring project you worked on"), and ten **debugging scenarios** in the form interviewers like: *symptom → cause → fix → how to prove it*. Five of the trickiest answers are proved by a small runnable Maven project (§18.15) — **Maven** is Java's standard build tool: it reads a `pom.xml` file, downloads the libraries (dependencies) the project needs, compiles the code and runs the tests.

What you'll learn

- Short answers for the core container, configuration, AOP, MVC/REST, clients, data/JPA/transactions, caching, security, testing, reactive, messaging, production, Spring Cloud, Spring AI and the Boot 4 / Framework 7 changes.
- Which details interviewers use to tell "has used Spring" from "understands Spring": proxies, scopes, rollback rules, persistence context, filter chain, context caching.
- How to structure a project walkthrough so that it survives follow-up questions.
- A repeatable way to debug the ten failures every Spring developer meets.

Versions used in answers: Spring Boot 4.1.1 → Spring Framework 7.0, Spring Security 7.1, Spring Data 2026.0, Hibernate 7.4 (the library that maps Java objects to database tables), Jackson 3.1 (the JSON library), JUnit 6 (the test framework), Java 25 (17 minimum). When an answer depends on a version, the version is named.

18.1 How to use this chapter

How to answer a Spring question

Interviewers rarely want a definition only. A strong answer has three layers, and you can stop after any of them:

1. **One sentence of what it is** — "`@Transactional` starts and ends a database transaction around a method." (A **transaction** is a group of database changes that either all succeed — *commit* — or are all undone — *rollback*.)
2. **How it works underneath** — "A proxy's interceptor asks the transaction manager to begin, calls the method, then commits or rolls back." (A **proxy** is a stand-in object Spring generates around your bean; callers talk to it, and it runs extra code — the *interceptor* — before and after passing the call on.)
3. **The consequence that bites people** — "So self-invocation skips it, and a checked exception commits by default." (**Self-invocation** is a method calling another method of the same object through `this`, which bypasses the proxy; a **checked exception** is one Java forces you to declare or catch — any `Exception` that is not a `RuntimeException`.)

The third layer is what makes you sound experienced. Most answers below are written in that shape.

How to study with it

- Cover the answer, say yours out loud, then compare. If you miss the third layer, follow the link.
- The **→ §N.M** link after each answer goes to the teaching section. Links to a chapter without a section number point to the chapter as a whole.
- Questions marked **(Boot 4)** or **(Framework 7)** are about new features; interviewers use them to check that your knowledge is current.

Self-check

1. What are the three layers of a good answer? → What it is, how it works, the consequence that bites.
2. Why is layer 3 the most valuable? → It shows you have debugged the feature, not just read about it.

18.2 Core container and dependency injection

Q1. What are Inversion of Control and Dependency Injection? IoC means the framework, not your code, controls object creation and the flow of calls into your code. DI is the form of IoC Spring uses: an object declares what it needs (constructor parameters) and the container supplies it. The result is loose coupling — you can swap a real `PaymentGateway` for a fake in tests without changing the class that uses it. → §2.1

Q2. What is a bean? An object whose creation, wiring and lifecycle are managed by the Spring container (the `ApplicationContext` — the object that creates, connects and holds all beans). It is described by a `BeanDefinition` (class or factory method, scope, dependencies, init/destroy callbacks), which the container turns into an instance. → §2.3

Q3. `BeanFactory` vs `ApplicationContext`? `BeanFactory` is the minimal container: it creates beans lazily on request. `ApplicationContext` extends it and adds eager creation of singletons at startup (so wiring errors fail fast), automatic registration of post-processors, events, the `Environment` with profiles and properties, message sources (i18n — internationalization, i.e. messages translated per language) and resource loading. Applications always use an `ApplicationContext`. → §2.2

Q4. `@Component` vs `@Bean`? `@Component` (and `@Service`, `@Repository`, `@Controller`) goes on *your* class and is found by component scanning (Spring searching your packages at startup for annotated classes). `@Bean` goes on a factory method in a `@Configuration`

class; use it for third-party classes you cannot annotate or when creation needs logic. `@Repository` also enables exception translation to `DataAccessException`. → §2.3

Q5. What is the difference between "full" and "lite" `@Configuration`? With `@Configuration` (default `proxyBeanMethods = true`) Spring subclasses the class with CGLIB (a library that generates subclasses at runtime) so that calling one `@Bean` method from another returns the existing singleton instead of a new object. With `proxyBeanMethods = false` (or `@Bean` methods in a plain `@Component`) — "lite mode" — such a call is a normal Java call and creates a second object. Boot's own auto-configurations use lite mode and pass dependencies as method parameters, which is faster and works the same in native images (apps compiled ahead of time by GraalVM into a standalone executable that needs no JVM). → §2.3

Q6. Constructor, setter or field injection — which and why? Constructor injection: dependencies can be `final`, the object is complete after construction, the required dependencies are visible in the signature, tests can call `new` without Spring, and cycles fail loudly. A class with one constructor needs no `@Autowired`. Setter injection suits truly optional dependencies. Field injection hides dependencies and needs reflection (Java code reading or changing classes and private fields at runtime) to test — avoid it outside test classes. → §2.4

Q7. Two beans implement the same interface. How does Spring choose? By type first. With several candidates: `@Qualifier` on the injection point wins; otherwise a `@Primary` bean; otherwise the one non-`@Fallback` bean (Framework 6.2); otherwise a bean whose name equals the parameter name; otherwise startup fails with `NoUniqueBeanDefinitionException`. To get all of them, inject `List<PaymentGateway>` (ordered by `@Order`) or `Map<String, PaymentGateway>` (keyed by bean name). Proved in §18.15. → §2.5

Q8. `@Primary` vs `@Qualifier`? `@Primary` is a default chosen by the *provider* of the beans; `@Qualifier` is an explicit choice made by the *consumer* at one injection point. When both apply, the qualifier wins. Use `@Primary` for "the usual one", `@Qualifier` (or a custom qualifier annotation) where a specific one is required. → §2.5

Q9. How do you inject an optional dependency? `ObjectProvider<T>` (`getIfAvailable()`, `ifAvailable(...)`, also lazy), `Optional<T>`, a `@Nullable` parameter (JSpecify — a standard set of null-safety annotations — in Framework 7), or `@Autowired(required = false)` on a setter. → §2.5

Q10. What bean scopes exist, and what is the default? `singleton` (default — one instance per container), `prototype` (a new instance every time the bean is requested), and in web apps `request`, `session`, `application` and `websocket`. Singleton does not mean thread-safe (safe to use from many threads at the same time): one instance is shared by all request threads, so keep singletons stateless (no fields that change per request) or use thread-safe state. → §2.6

Q11. You inject a prototype bean into a singleton. What happens? The prototype is created **once**, when the singleton is created, and the singleton keeps that one instance forever — "prototype" only means new-per-*request-to-the-container*. To get a fresh one each time, inject `ObjectProvider<T>` and call `getObject()`, use a `@Lookup` method, or use a scoped proxy. Proved in §18.15. → §2.6

Q12. How can a singleton use a request-scoped bean? Through a **scoped proxy**: `@RequestScope` (which is `@Scope(value = "request", proxyMode = TARGET_CLASS)`) injects a proxy that, on every call, looks up the instance bound to the current request. Using it outside a request thread throws `ScopeNotActiveException`. → §2.6

Q13. Describe the bean lifecycle. Instantiate → inject dependencies → `*Aware` callbacks → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` → `InitializingBean.afterPropertiesSet` → custom init method → `postProcessAfterInitialization` (where AOP proxies are created) → bean in use → on shutdown `@PreDestroy` → `DisposableBean.destroy` → custom destroy method. Spring does **not** call destroy callbacks for prototypes. For start/stop behaviour tied to the context (consumers, schedulers), implement `SmartLifecycle`. → §2.7

Q14. `BeanPostProcessor` vs `BeanFactoryPostProcessor`? A `BeanFactoryPostProcessor` runs **before any bean is created** and changes bean *definitions* (for example `PropertySourcesPlaceholderConfigurer` resolves `${...}`). A `BeanPostProcessor` runs for **each bean instance** and can modify or wrap it (for example `AutowiredAnnotationBeanPostProcessor` does injection; the auto-proxy creator wraps beans for `@Transactional`). Declare BFPP `@Bean` methods `static` so they don't force their configuration class to be created early. → §2.8

Q15. How does Spring handle circular dependencies? Constructor-injection cycles can never be resolved: startup fails with `BeanCurrentlyInCreationException`. Field/setter cycles *can* be resolved through early references, but Spring Boot disallows them by default since 2.6 (`spring.main.allow-circular-references=false`). The real fix is design — extract the shared logic into a third bean, or decouple with an event. `@Lazy` on one constructor parameter works as a stop-gap by injecting a lazy proxy. Proved in §18.15. → §2.9

Q16. What does lazy initialization do, and what does it cost? `@Lazy` (or `spring.main.lazy-initialization=true`) creates a bean on first use instead of at startup. Startup gets faster, but configuration errors appear at the first request instead of at deploy time, and the first request is slower. Good for development and some CLI (command-line) tools; risky for services. → §2.10

Q17. `@Profile` vs `@Conditional`? `@Profile("prod")` is a ready-made condition that registers a bean only when that profile is active. `@Conditional(MyCondition.class)` takes any `Condition` implementation (a class present, a property set, an OS...). Boot's `@ConditionalOnClass`, `@ConditionalOnProperty`, `@ConditionalOnMissingBean` are built on it. → §2.11

Q18. How do application events work? Are they asynchronous? Publish any object with `ApplicationEventPublisher.publishEvent(...)`; methods annotated `@EventListener` receive it. By default delivery is **synchronous** (the publisher waits until every listener has finished), on the publisher's thread and inside its transaction — an exception in a listener reaches the publisher. Add `@Async` for asynchronous delivery, or `@TransactionalEventListener` to run after commit. → §2.12

Q19. (Framework 7) What is JSpecify null-safety in Spring? Framework 7 annotates its APIs with JSpecify (`@NullMarked` packages, `@Nullable` for the exceptions), replacing the deprecated `org.springframework.lang` annotations. Tools like NullAway, IntelliJ and Kotlin read them, so null bugs are reported at compile time (when the code is built, before it ever runs). You can do the same in your code. → §2.13

Q20. (Framework 7) What is a `BeanRegistrar`? A programmatic way to register beans with a loop or condition in code (for example one bean per configured tenant — one customer organisation in a shared system), imported with `@Import`. It is friendlier to AOT (ahead-of-time: work done at build time instead of at startup)/native images than a `BeanDefinitionRegistryPostProcessor`. → §2.14

Self-check

1. Default scope, and is it thread-safe? → Singleton; no — shared across threads.
2. What exception do you get with two candidates and no tie-breaker? → `NoUniqueBeanDefinitionException`.
3. Which lifecycle step creates AOP proxies? → `BeanPostProcessor.postProcessAfterInitialization`.

18.3 Configuration and auto-configuration

Q21. What does `@SpringBootApplication` do? It combines `@SpringBootConfiguration` (a `@Configuration`), `@EnableAutoConfiguration` and `@ComponentScan` of the main class's package and sub-packages. That is why the main class belongs in the root package: beans in sibling packages are not scanned. → §1.8

Q22. What happens when you run `java -jar app.jar`? The manifest (`META-INF/MANIFEST.MF`, a file inside the jar — the zip of compiled classes — that names the class to start) starts Boot's `JarLauncher`, which builds a class loader (the JVM component that finds and loads `.class` files) over `BOOT-INF/classes` and `BOOT-INF/lib` and calls your main class. `SpringApplication.run` prepares the `Environment`, creates the right `ApplicationContext` (servlet — classic blocking web, reactive — non-blocking web, see §18.10, or none), loads bean definitions and matching auto-configurations, refreshes the context (creating singletons and starting the embedded server — a web server such as Tomcat running inside your own process), then calls `ApplicationRunner`/`CommandLineRunner` beans. → §1.9

Q23. What is a starter? A dependency (a library your build declares and downloads) with (almost) no code that pulls in a tested set of libraries for one capability plus the Boot module holding its auto-configuration. In Boot 4 they are modular and some were renamed: `spring-boot-starter-webmvc` (was `-web`), `spring-boot-starter-aspectj` (was `-aop`), and each technology has a matching test starter such as `spring-boot-starter-webmvc-test`. → §1.7

Q24. How does auto-configuration work? Each Boot module lists its auto-configuration classes in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. `@EnableAutoConfiguration` loads that list, filters it early by class presence, sorts it (`before`/`after`), and processes it **after** your own configuration. Each class is guarded by conditions — `@ConditionalOnClass`, `@ConditionalOnProperty`, `@ConditionalOnMissingBean`… — so a `DataSource` is only created if a driver is present and you haven't defined your own. → §3.10

Q25. How do you override an auto-configured bean? Define your own bean of that type; `@ConditionalOnMissingBean` makes Boot back off. Often you don't need to replace it at all — set properties or declare a customizer bean (`JsonMapperBuilderCustomizer`, `RestClientCustomizer`, …). To switch one auto-configuration off: `@SpringBootApplication(exclude = …)` or `spring.autoconfigure.exclude`. → §3.12

Q26. How do you find out why a bean was (or was not) auto-configured? Start with `--debug` (or `debug=true`) to print the **condition evaluation report** — positive and negative matches with the reason — or call `/actuator/conditions`. `/actuator/beans` lists what exists. → §3.11

Q27. Name the main property sources in priority order. Highest first: test properties (in tests), command-line arguments, `SPRING_APPLICATION_JSON`, Java system properties (`-Dkey=value` flags on the `java` command), OS environment variables (named values set outside the app by the shell, OS or container), `random.*`, profile-specific files (outside the jar before inside), `application.properties/yaml` (outside before inside), `@PropertySource`, defaults. The rule of thumb: the more "outside" and "later", the stronger. → §3.3

Q28. How does an environment variable map to a property? Relaxed binding: replace `.` with `_`, remove `-`, upper-case. `SHOP_CHECKOUTTIMEOUT` sets `shop.checkout-timeout`; list indexes use `_0_`. → §3.4

Q29. `@Value` vs `@ConfigurationProperties`? `@Value("${x}")` injects one value (and supports SpEL — Spring Expression Language, small expressions inside annotation strings). `@ConfigurationProperties(prefix = "shop")` binds a whole group into a typed object (a record — Java's compact, immutable data class — is ideal), supports relaxed binding, nested objects, `Duration`/`DataSize` conversion, validation with `@Validated`, and IDE metadata (so your editor can auto-complete property names). Use `@ConfigurationProperties` for anything more than one or two values. → §3.6

Q30. How do you make the app refuse to start with bad configuration? Annotate the `@ConfigurationProperties` class with `@Validated` and add Jakarta constraints (Bean Validation annotations such as `@NotBlank`, `@Min`, `@Valid` for nested groups). Binding failures stop startup and the report names each property and where it came from. → §3.6

Q31. What are profiles and profile groups? A profile is a named set of beans and config (`application-prod.yaml`, `@Profile("prod")`) activated with `spring.profiles.active`. A group activates several at once: `spring.profiles.group.prod=postgres,kafka,json-logs`. Keep profiles for *environments or capabilities*, and keep secrets out of files. → §3.7

Q32. How do you load Kubernetes secrets or extra config files? (**Kubernetes** is a platform that runs and restarts containers across many machines; a *secret* is a value such as a password that it stores and mounts into the container.) `spring.config.import=optional:configtree:/etc/secrets/` reads mounted files as properties (file name = key). `spring.config.import` also imports other files, or Config Server/Vault with the matching library. `optional:` prevents failure when absent. → §3.8

Q33. How do you write your own starter? Two artifacts (separately published Maven modules, i.e. jars): `acme-spring-boot` with `@AutoConfiguration` classes (conditions, `@ConfigurationProperties`, `@ConditionalOnMissingBean` on every bean) listed in the `.imports`

file, and `acme-spring-boot-starter`, a pom (a Maven project file with no code of its own) that depends on it and the library. Test it with `ApplicationContextRunner`. → §3.13

Q34. (Boot 4) What does "Boot 4 modularization" mean? Boot 3 kept all auto-configuration in one big `spring-boot-autoconfigure` jar. Boot 4 splits it into per-technology modules (`spring-boot-webmvc`, `spring-boot-jdbc`, `spring-boot-kafka` …) with matching starters and test starters, so the classpath (the list of jars and folders the JVM loads classes from) contains only what you use. `spring-boot-starter-classic` / `-test-classic` bring the old "everything" layout as a migration aid. → §3.14

Q35. What does DevTools do, and is it in production? Automatic restart on class changes (two class loaders), development property defaults (for example template caching off). It is excluded from the packaged jar automatically and disabled when run with `java -jar`. LiveReload is off by default in Boot 4. → §1.10

18.4 AOP and proxies

Q36. What is AOP? Define aspect, advice, pointcut and join point. Aspect-Oriented Programming moves *cross-cutting* concerns (logging, timing, transactions, security) out of business code. A **join point** is a point in execution (in Spring AOP: a method execution on a bean); a **pointcut** is an expression selecting join points; **advice** is the code that runs there (`@Before`, `@AfterReturning`, `@AfterThrowing`, `@After`, `@Around`); an **aspect** is a class bundling pointcuts and advice. → §4.1

Q37. How does Spring AOP work at runtime? With proxies. A `BeanPostProcessor` checks each bean against all pointcuts; if any match, it returns a **proxy** instead of the bean. Callers hold the proxy; the proxy runs the advice chain and then calls the real object. Nothing is changed in your bytecode (the compiled instructions in `.class` files that the JVM runs). → §4.4

Q38. JDK dynamic proxy vs CGLIB? A JDK (Java Development Kit) dynamic proxy implements the bean's interfaces (so you can only inject by interface). A CGLIB proxy is a generated subclass (so you can inject by class) but cannot proxy `final` classes or advise `final`/`private` methods. Spring Boot uses CGLIB for everything by default (`spring.aop.proxy-target-class=true`); plain Spring uses JDK proxies when the bean has interfaces. Framework 7 adds `@Proxyable` to choose per bean. → §4.4

Q39. What is the self-invocation problem? A call `this.saveBatch()` inside the bean goes to the real object, not the proxy, so `@Transactional`, `@Cacheable`, `@Async`, `@Retryable` and custom aspects on the called method are silently skipped. Fixes: move the method to another bean (best), inject the bean into itself lazily (`@Lazy` self-reference / `ObjectProvider`), use `TransactionTemplate`, or switch to AspectJ weaving. Proved in §18.15. → §4.5

Q40. Which Spring features are built on proxies? `@Transactional`, `@Cacheable`/`@CacheEvict`, `@Async`, `@Retryable`/`@ConcurrencyLimit` (Framework 7), `@PreAuthorize` and other method security, `@Validated` method validation, `@Observed`, scoped proxies, `@Lazy` injection points, full `@Configuration` classes (CGLIB), Spring Data repositories and HTTP interface clients (JDK proxies). All share the same limitations. → §4.6

Q41. `@Around` vs the other advice types? Only `@Around` receives a `ProceedingJoinPoint` and decides whether and how the method runs (`proceed()`), can change arguments and the return value, and can catch exceptions. Use the narrowest advice that works — `@Before` cannot forget to call `proceed()`. → §4.2

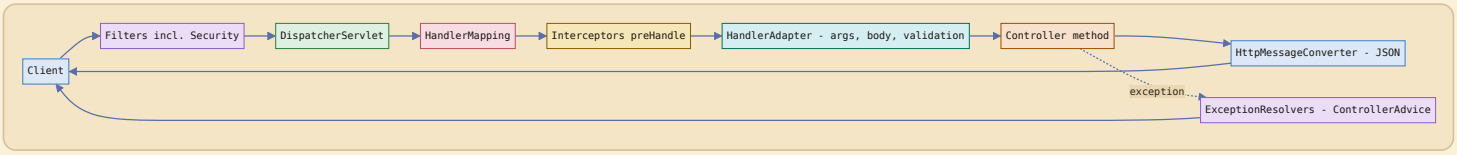
Q42. How do you control the order of two aspects? `@Order` (or `Ordered`) on the aspect class: the lower value is the *outer* layer — it runs first on the way in and last on the way out. Example: put a retry aspect outside the transaction aspect so each attempt gets a fresh transaction. → §4.2

Q43. Spring AOP vs AspectJ? Spring AOP: runtime proxies, method execution on Spring beans only, no build changes. AspectJ: weaves bytecode (inserts the aspect's code directly into your compiled classes) at compile or load time, can advise any join point (constructors, field access, private methods, self-calls), but needs the AspectJ compiler or a weaving agent. Most apps need only Spring AOP. → §4.8

Q44. (Framework 7) What are `@Retryable` and `@ConcurrencyLimit`? Built-in resilience annotations in `spring-context`: `@Retryable` retries a method on exceptions with `maxRetries`, `delay`, `multiplier`, `jitter`, `maxDelay` (jitter = a random amount added to each wait so many clients don't retry in lockstep) and include/exclude filters (it also works for reactive return types); `@ConcurrencyLimit` caps concurrent calls (a bulkhead). They are proxy-based and need `@EnableResilientMethods` — there is no auto-configuration. Programmatic API: `RetryTemplate`/`RetryPolicy` in `spring-core`. There is no circuit breaker (a guard that stops calling a failing dependency for a while and fails fast instead); use Resilience4j, a resilience library, for that. → §4.7

18.5 Spring MVC, REST and HTTP clients

Q45. Walk through a request in Spring MVC. Servlet filters (including Spring Security; a *servlet* is the Java standard API for handling HTTP requests, and *filters* run before and after it on every request) → `DispatcherServlet` (Spring MVC's single front servlet) → `HandlerMapping` finds the controller method (path, HTTP method, consumes/produces, API version) → interceptors' `preHandle` → `HandlerAdapter` resolves arguments (path variables, query params, body via an `HttpMessageConverter`, validation) → method runs → the return value is written by a converter chosen from the `Accept` header → exceptions go to `HandlerExceptionResolver`s (`@ExceptionHandler`, `@ControllerAdvice`) → `afterCompletion` → back through the filters. → §5.2



Q46. `@Controller` vs `@RestController`? `@RestController` = `@Controller` + `@ResponseBody`: return values are written to the response body (JSON) instead of being treated as view names. → §5.3

Q47. `@PathVariable` vs `@RequestParam` vs `@RequestBody`? Path variable: part of the URL identifying a resource (`/orders/{id}`). Request param: query string (the `?key=value&...` part of the URL), usually filters/paging (`?status=NEW&page=2`), optional with `required = false` or a default. Request body: the deserialized payload (the request data turned from JSON text into a Java object, e.g. a record) of POST/PUT/PATCH. → §5.4

Q48. Why return `ResponseEntity`? To control the status code and headers as well as the body — for example `201 Created` with a `Location` header, `204 No Content`, or `304` with an `ETag` (a version fingerprint of the response that lets clients skip re-downloading unchanged data). For plain 200 responses, returning the object is enough. → §5.4

Q49. How do you validate a request? Put Jakarta Bean Validation constraints on the DTO (Data Transfer Object — a simple class or record shaped for the API, not for the database) (`@NotBlank`, `@Positive`, `@Email`, `@Valid` for nested objects) and `@Valid` on the `@RequestBody`. A failure raises `MethodArgumentNotValidException` → 400. Since Framework 6.1, constraints directly on method parameters (`@RequestParam @Min(1) int page`) are validated built-in and raise `HandlerMethodValidationException`. Use `@Validated(Group.class)` for validation groups. → §5.6

Q50. How do you handle errors consistently? One `@RestControllerAdvice` (extending `ResponseEntityExceptionHandler` to cover Spring's own exceptions) with `@ExceptionHandler` methods that map domain exceptions to statuses and return **`ProblemDetail`** — the RFC 9457 (an RFC is a published Internet standard) `application/problem+json` format with `type`, `title`, `status`, `detail`, `instance` and your extra fields. Never leak stack traces (the list of method calls printed with an exception — it reveals internals to attackers). → §5.7

Q51. (Boot 4) What changed with JSON? Boot 4 uses **Jackson 3**: packages `tools.jackson.*` (the annotations stay in `com.fasterxml.jackson.annotation`), an immutable `JsonMapper` built with a builder, unchecked exceptions (subclasses of `RuntimeException`, which callers need not catch), and different defaults (for example dates written as ISO-8601 strings such as `2026-10-07T09:30:00Z`). Customize with `JsonMapperBuilderCustomizer`; `@JsonComponent` became `@JacksonComponent`. Jackson 2 support remains only in deprecated form. → §5.5

Q52. How does content negotiation work? The client's `Accept` header (and `consumes`/`produces` on the mapping) decides which `HttpMessageConverter` writes the response. No converter for the requested type → 406; unsupported request `Content-Type` → 415. → §5.8

Q53. (Framework 7) How do you version a REST API? Framework 7 has first-class API versioning: configure how the version is read (header, query param, path segment or media-type parameter — in Boot via `spring.mvc.apiversion.*`, e.g. `spring.mvc.apiversion.use.header=X-API-Version`), a default and the supported versions, then map with `@GetMapping(path = "/orders", version = "1.1+")` ("1.1 and later"). A deprecation handler can add `Deprecation`, `Sunset` and `Link` headers. → §5.9

Q54. What is CORS and where do you configure it? Cross-Origin Resource Sharing is a browser rule: JavaScript from origin A (an *origin* is scheme + host + port, e.g. `https://shop.example.com:443`) may read a response from origin B only if B's response allows it (`Access-Control-Allow-Origin`); non-simple requests are preceded by an `OPTIONS` preflight. Configure it with `@CrossOrigin`/`WebMvcConfigurer.addCorsMappings` in plain MVC, or — when Spring Security is present — a `CorsConfigurationSource` bean plus `http.cors(...)`, so the preflight is answered before authentication. CORS does not protect your API from non-browser clients. → §5.10, §10.9

Q55. Filter vs interceptor vs aspect? A servlet **filter** sees every request (even ones that never reach Spring MVC), can wrap request/response, and knows nothing about controllers — good for security, logging, compression. A `HandlerInterceptor` runs inside the `DispatcherServlet`, knows the chosen `HandlerMethod`, and only for mapped requests — good for per-handler checks. An **aspect** works on any bean method, not HTTP. → §5.11

Q56. PUT vs PATCH vs POST — which are idempotent? GET, PUT and DELETE are idempotent (repeating them leaves the same state — see System Design Fundamentals); POST is not; PATCH is not guaranteed to be. Use POST to create (`201` + `Location`), PUT to replace, PATCH for partial update. For retry-safe POSTs, accept an `Idempotency-Key` header. → §5.1

Q57. How do you paginate and document an API? Accept `page`/`size`/`sort` (Spring Data's `Pageable` resolves them) and return a page DTO with the content and paging metadata rather than exposing internal classes; for very large or changing data prefer keyset (cursor) pagination ("give me the rows after the last id I saw" instead of skipping N rows). Document with OpenAPI (a standard, machine-readable description of a REST API) generated by springdoc (`/v3/api-docs`, Swagger UI — a web page that renders that description and lets you try the calls). → §5.13, §5.12

Q58. Which HTTP client should a new Boot 4 service use? `RestClient` (Framework 6.1+) for synchronous calls — fluent (calls chained one after another, `a().b().c()`), uses Boot's converters, timeouts and observability when you inject `RestClient.Builder` — or a declarative `@HttpExchange` interface on top of it. `WebClient` is for reactive apps. `RestTemplate` is legacy: its deprecation (official marking as "don't use, will be removed") is planned for Framework 7.1 with removal in 8.0. → §6.1

Q59. What are HTTP service interfaces, and what does Boot 4 add? You declare a Java interface with `@GetExchange`/`@PostExchange` methods; `HttpServiceProxyFactory` creates a JDK-proxy implementation that makes the calls. Boot 4 adds `@ImportHttpServices(group = "catalog", types = CatalogApi.class)` and per-group properties `spring.http.serviceclient.<group>.*` (base URL, timeouts), so you don't write factory code. → §6.6

Q60. How do you handle errors, timeouts and retries on a client? `retrieve()` throws `HttpClientErrorException` (4xx) / `HttpServerErrorException` (5xx), and `ResourceAccessException` for I/O (input/output — network or disk) errors and timeouts; translate them to domain exceptions at the client edge with `onStatus`. Always set connect and read timeouts (`spring.http.clients.*`). Retry only idempotent calls and transient failures (short-lived ones that may succeed a moment later), with exponential back-off (waiting longer after each failure, e.g. 1 s, 2 s, 4 s) and jitter. → §6.3, §6.4

Q61. How do you test a REST client? `@RestClientTest` + `MockRestServiceServer` to check the request your client sends and how it maps responses, plus at least one test against a real stub server (a fake HTTP server returning canned responses, e.g.

WireMock) to catch HTTP-level problems. → §6.8

18.6 Data access, JPA and transactions

Q62. What does Spring add on top of plain JDBC? (JDBC — Java Database Connectivity — is Java's standard low-level API for talking to SQL databases.) A pooled `DataSource` (a *connection pool* keeps a few open database connections and lends them out, because opening a new one per query is slow; HikariCP is Boot's pool), `JdbcClient` / `JdbcTemplate` that handle connections, statements, result-set mapping and cleanup, translation of vendor (database-specific) `SQLException`s into the unchecked `DataAccessException` hierarchy (`DuplicateKeyException`, `DataIntegrityViolationException`…), and declarative transactions. → §7.1, §7.5

Q63. `JdbcClient` vs `JdbcTemplate`? Same engine underneath. `JdbcClient` (Framework 6.1) is a fluent API — `sql(...).param(...).query(Order.class).list()` — with named or positional parameters and automatic mapping to records. `JdbcTemplate` remains for batch updates and existing code. → §7.3

Q64. How big should the connection pool be? Small. HikariCP defaults to 10. A database can only run as many queries in parallel as it has CPU cores and disk bandwidth; more connections mean queueing inside the database instead of in the pool. Size from measurements, remember that *pool* × *instances* must stay under the database's `max_connections`, and watch `hikaricp.connections.pending`. → §7.2

Q65. How does `@Transactional` work internally? The bean is proxied; the proxy's `TransactionInterceptor` asks the `PlatformTransactionManager` (`DataSourceTransactionManager`, `JpaTransactionManager`) to start a transaction, which takes a connection, sets auto-commit (every SQL statement committing on its own) off and binds it to the current **thread**. Repositories/`JdbcClient` on that thread reuse the bound connection. When the method returns, the interceptor commits; when it throws, it applies the rollback rules. → §7.7

Q66. What are the default propagation and isolation? Propagation (what happens when a transactional method is called while a transaction is already running) `REQUIRED` (join the current transaction or start one); isolation (how much concurrent transactions see of each other's changes) `DEFAULT`, meaning the database's own default — READ COMMITTED on PostgreSQL, REPEATABLE READ on MySQL InnoDB. → §7.8

Q67. A `@Transactional` method throws a checked exception. Commit or rollback? Commit. By default Spring rolls back only on `RuntimeException` and `Error` (the EJB — Enterprise JavaBeans, the older Java EE component model — convention: checked exceptions are "expected business outcomes"). Change it per method with `rollbackFor = PaymentDeclinedException.class`, or globally since Framework 6.2 with `@EnableTransactionManagement(rollbackOn = RollbackOn.ALL_EXCEPTIONS)` (which the reference docs recommend). An exception caught inside the method never causes a rollback — the proxy never sees it. Proved in §18.15. → §7.10

Q68. Explain the propagation types.

Propagation	Existing transaction?	None?
`REQUIRED` (default)	join it	start one
`REQUIRES_NEW`	suspend it, start a new one (new connection)	start one
`NESTED`	savepoint inside it	start one
`SUPPORTS`	join it	run without
`NOT_SUPPORTED`	suspend it, run without	run without
`MANDATORY`	join it	throw
`NEVER`	throw	run without

Typical uses: `REQUIRES_NEW` for an audit record that must survive a rollback; `NESTED` for "try this part, roll back only this part" (JDBC savepoints — markers inside a transaction you can roll back to; avoid it with JPA — the database rolls back to the savepoint but Hibernate's persistence context does not). → §7.9

Q69. What is `UnexpectedRollbackException`? An inner `REQUIRED` method (same physical transaction) threw, so the interceptor marked the shared transaction *rollback-only*. The outer method caught the exception and returned normally, so Spring tried to commit — and throws `UnexpectedRollbackException` instead of silently committing. Fix: let the exception propagate, or give the inner work its own transaction (`REQUIRES_NEW` / `NESTED`) if partial success is really wanted. → §7.9

Q70. What does `readOnly = true` do? It is a hint passed down the stack: JDBC connections are set read-only (PostgreSQL then rejects writes), Hibernate skips dirty checking and flushing (less memory and CPU; *dirty checking* compares loaded objects with their original state to find changes, *flushing* sends pending changes to the database), and a routing `DataSource` can send the call to a replica (a read-only copy of the database). It is not a security feature. → §7.12

Q71. Why might `@Transactional` be silently ignored? Self-invocation; the method is `private` (with CGLIB, `protected` and package-private methods work since Framework 6.0) or `final`; the object is not a Spring bean (`new`); the exception is caught inside the method; a checked exception without a rollback rule; a different transaction manager than the one your data access uses; or work done on another thread (`@Async`, `CompletableFuture`), which does not see the thread-bound transaction. → §7.13

Q72. On which layer does `@Transactional` belong? On the service method that represents one business operation (one unit of work), not on controllers (keeps HTTP/serialization out of the transaction) and usually not only on each repository call (then two calls are two transactions). Keep remote HTTP calls out of transactions — they hold a connection while waiting. → §7.13

Q73. When do you use `TransactionTemplate`? When the transaction boundary is smaller than a method, must be decided at runtime, or you want to avoid proxy pitfalls (self-calls, retries around a transaction): `txTemplate.execute(status -> { ... })`. → §7.14

Q74. How do you run code only after the transaction commits? `@TransactionalEventListener` (default phase `AFTER_COMMIT`) on a listener for an event published inside the transaction — e.g. send the confirmation e-mail only if the order was really saved. It is not guaranteed delivery: if the process dies after commit, the event is lost; use a transactional outbox when delivery must be guaranteed. → §7.15

Q75. How do you handle a serialization failure or deadlock? (A *serialization failure* is the database aborting a transaction because it conflicts with a concurrent one; a *deadlock* is two transactions each waiting for a lock the other holds.) They surface as `ConcurrencyFailureException` subtypes (for example `CannotAcquireLockException`, `PessimisticLockingFailureException`). Retry the **whole** transaction from outside it (a retry around the transactional method, not inside), a few times with back-off. → §7.11, SQL book 13

Q76. JPA vs Hibernate vs Spring Data JPA? JPA (Jakarta Persistence 3.2) is the specification (a written standard that several libraries can implement) — annotations and the `EntityManager` API. Hibernate (7.x in Boot 4) is the implementation. Spring Data JPA adds repositories (objects that load and save one type of entity) on top: interfaces whose implementation is generated, with derived queries, paging and auditing. → §8.1

Q77. What is the persistence context? What is dirty checking? The persistence context is the `EntityManager`'s first-level cache and unit of work (the set of changes that will be written together): it holds exactly one managed instance per row and a snapshot of its loaded state. At flush (before commit, or before a query that needs it) Hibernate compares each managed entity with its snapshot and issues `UPDATE`s for the changed ones — so you don't call `save()` after changing a loaded entity inside a transaction. → §8.4

Q78. What are the entity states? Transient (new, not known to the context), managed (in the context; changes are tracked), detached (was managed, context closed — changes not tracked), removed (scheduled for delete at flush). → §8.4

Q79. How does `save()` decide between insert and update? `SimpleJpaRepository.save()` calls `persist` if the entity is **new** — by default a `null` id (or a `null` wrapper-type `@Version`) — otherwise `merge`, which may `SELECT` first. Assigned ids therefore cause a merge; implement `Persistable.isNew()` or use a version field to avoid it. → §8.8

Q80. What is the owning side of a relationship? The side that holds the foreign key (a column pointing to the primary key of another table) — the `@ManyToOne` side in a 1:N (one-to-many) relationship. JPA writes the FK only from the owning side; the inverse `@OneToMany(mappedBy = "order")` is ignored for writes, so update both sides in a helper method (`addItem`). → §8.5

Q81. `CascadeType.REMOVE` vs `orphanRemoval = true`? Cascade REMOVE deletes children when the *parent* is deleted. `orphanRemoval` also deletes a child when it is *removed from the parent's collection*. Use both only for true composition (order → order lines), never on `@ManyToMany`. → §8.5

Q82. What are the default fetch types, and what causes `LazyInitializationException`? To-one (`@ManyToOne`, `@OneToOne`): EAGER; to-many: LAZY — set everything LAZY and fetch per use case. The exception happens when you touch an uninitialized lazy proxy after its persistence context has closed (e.g. in a controller or during JSON serialization). Fix by fetching what you need in the query (`JOIN FETCH`, `@EntityGraph`) inside the transaction and returning DTOs — not by switching to EAGER or relying on Open Session in View. → §8.6

Q83. What is the N+1 problem and how do you fix it? One query loads N parents, then touching each parent's lazy association runs one more query per parent: 1 + N round trips. Fixes: `JOIN FETCH` in JPQL (JPA's SQL-like query language written against entities instead of tables), `@EntityGraph(attributePaths = "books")` on the repository method, batch fetching (`hibernate.default_batch_fetch_size` / `@BatchSize` → 1 + N/size), or a DTO projection that selects only needed columns. Detect it by counting statements in tests (Hibernate statistics or a datasource proxy). → §8.7

Q84. Derived query vs `@Query` vs native query? Derived (`findByStatusAndCreatedAtAfter`) for short conditions; `@Query` JPQL when the name gets long or you need joins/fetches; `nativeQuery = true` for database-specific SQL (window functions — calculations across related rows such as running totals; JSONB — PostgreSQL's binary JSON column type). Updates need `@Modifying` (plus `clearAutomatically = true` if managed entities may be stale). → §8.9, §8.10

Q85. `Page` vs `Slice`? `Page` runs an extra `COUNT` query and knows the total; `Slice` fetches `size + 1` rows to know only whether a next slice exists — cheaper for infinite scrolling. Never combine `JOIN FETCH` of a collection with paging without checking the SQL (Hibernate may page in memory). → §8.11

Q86. What are projections? Reading only some columns into a smaller type: an interface projection (Spring creates a proxy), a record/class DTO projection (constructor expression), or dynamic projections (`<T> List<T> findBy(..., Class<T> type)`). They avoid loading and tracking full entities for read-only screens. → §8.12

Q87. How does optimistic locking work? Add a `@Version` column. Every update runs `... WHERE id = ? AND version = ?` and increments it; if 0 rows match, someone else changed the row and Spring throws `ObjectOptimisticLockingFailureException`. Return 409 (Conflict) or retry the read-modify-write. Pessimistic locking (`@Lock(PESSIMISTIC_WRITE)` → `SELECT ... FOR UPDATE`) blocks instead; use it for short, highly contended updates. → §8.14

Q88. What is Open Session in View and should you disable it? `spring.jpa.open-in-view` (default `true`, with a startup warning) keeps the persistence context open until the web response is written, so lazy loading works in controllers and serialization. It hides N+1 queries, holds a connection longer, and moves database access outside your service layer. Disable it for APIs and fetch what you need in the service. → §8.15

Q89. How is the schema managed in production? With migrations (numbered SQL scripts that change the schema step by step; Flyway or Liquibase run them) versioned in the repository and applied at startup or in the pipeline (the automated build-and-deploy process); `spring.jpa.hibernate.ddl-auto=validate` (or `none`). In Boot 4 add `spring-boot-starter-flyway` — `flyway-core` alone is not auto-configured. → §8.16

Q90. How should an entity implement `equals`/`hashCode`? Not with Lombok (a library that generates getters, `equals` and similar boilerplate at compile time) `@Data` over all fields (lazy fields, changing values, recursion through relationships). Either use

a natural business key that never changes, or compare by id when it is non-null with a constant `hashCode` (e.g. the class's hash) so the value doesn't change when the id is assigned on persist. → §8.3

Q91. Spring Data JDBC vs Spring Data JPA? Spring Data JDBC maps aggregates (DDD — Domain-Driven Design — style: a cluster of objects always loaded and saved as one unit) with plain SQL: no persistence context, no lazy loading, no dirty checking — `save()` writes the whole aggregate. Simpler and more predictable; less automatic. JPA offers caching, lazy loading and rich mapping at the cost of more hidden behaviour. → §9.2

18.7 Caching

Q92. How does Spring's cache abstraction work? `@EnableCaching` activates a proxy-based interceptor. (A **cache** keeps recent results in fast storage so repeated requests skip the slow work — see System Design Fundamentals.) `@Cacheable("products")` checks the cache by key before running the method and stores the result after; `@CachePut` always runs and updates; `@CacheEvict` removes entries (`allEntries = true` to clear). The storage is pluggable through a `CacheManager` (Caffeine — an in-process Java cache library; Redis — an in-memory key-value store running as a separate server; …); Boot auto-configures one from the classpath, falling back to a simple `ConcurrentHashMap`-based cache. → §9.8

Q93. How is the cache key built? By default from all method arguments (`SimpleKeyGenerator`: no args → `SimpleKey.EMPTY`, one arg → the arg, several → a `SimpleKey` of all). Customize with SpEL (`key = "#id"`, `key = "#req.customerId"`) or a `KeyGenerator` bean. Arguments must have proper `equals`/`hashCode`. → §9.8

Q94. `condition` vs `unless`? `condition` is evaluated *before* the call on the arguments (don't even look in the cache if false); `unless` is evaluated *after* on `#result` (don't store it if true — e.g. `unless = "#result == null"`). → §9.8

Q95. Why doesn't my `@Cacheable` work? Missing `@EnableCaching`, self-invocation, a method that isn't public on a proxied bean, keys that don't match (different argument objects without `equals`), or the method is called in a way that bypasses the proxy. → §4.6, §9.8

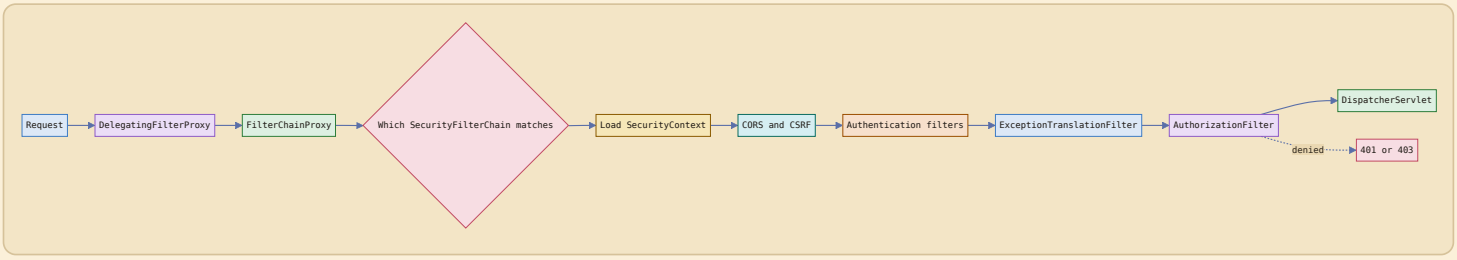
Q96. Local cache (Caffeine) vs distributed cache (Redis)? Local: nanosecond reads, no network, but each instance has its own copy — stale data across instances and memory per instance. Distributed: one shared copy and survives restarts, but a network hop, serialization, and another system to operate. Common choice: Caffeine for small, rarely changing reference data with a short TTL (time to live — how long an entry stays before it expires); Redis for shared session-like or expensive data. → §9.8

Q97. What cache problems should you plan for? Staleness (set TTLs, evict on write), stampede (many requests recomputing the same value at once) when a hot key expires (`@Cacheable(sync = true)` lets one caller compute per key — but only per instance, and it cannot be combined with `unless`), unbounded growth (set a maximum size), caching mutable objects that callers then modify, caching per-user data under a shared key, and forgetting to measure the hit rate — the share of lookups answered from the cache (Micrometer — Spring's metrics library — `cache.gets` metrics; for Caffeine only with `recordStats()`). → §9.8

18.8 Security

Q98. What happens when you add `spring-boot-starter-security` and nothing else? Every endpoint requires authentication (proving who the caller is); form login and HTTP Basic (username and password sent in a header on every request) are enabled; one in-memory user `user` gets a random password printed in the log ("Using generated security password"); CSRF protection (see Q106), security headers and session fixation protection (a new session id after login, so an attacker can't plant one beforehand) are on. As soon as you declare your own `SecurityFilterChain` bean, Boot's default chain backs off. → §10.1

Q99. Explain the filter chain architecture. The servlet container (the web server, e.g. Tomcat, that runs servlets) calls `DelegatingFilterProxy`, which delegates to the `FilterChainProxy` bean. It chooses the **first** `SecurityFilterChain` whose `securityMatcher` matches the request and runs its filters in a fixed order: security context loading, headers, CORS, CSRF, the authentication filters (form, Basic, bearer token), anonymous authentication, `ExceptionTranslationFilter`, and finally `AuthorizationFilter`. Several chains (e.g. `/api/**` stateless — no server-side session — with JWT, a signed JSON token the client sends on every call, see Q107; the rest with form login) are ordered with `@Order`. → §10.2



Q100. (Security 7) How do you configure a chain today? Declare a `SecurityFilterChain` `@Bean` using the lambda DSL (domain-specific language — here, configuration written as chained method calls taking lambdas, i.e. short inline functions `a -> ...`) only: `http.authorizeHttpRequests(a -> a.requestMatchers("/admin/**").hasRole("ADMIN").anyRequest().authenticated()).httpBasic(withDefaults())`. Security 7 removed `and()` chaining, `authorizeRequests` and the Ant/MVC request matchers (paths use `PathPatternRequestMatcher`); `WebSecurityConfigurerAdapter` has been gone since 6.0. Rules are checked top-down — put specific ones first and end with `anyRequest()`. → §10.3

Q101. How does authentication work internally? An authentication filter builds an unauthenticated `Authentication` (e.g. username + password) and passes it to the `AuthenticationManager` (a `ProviderManager`), which asks each `AuthenticationProvider`.

`DaoAuthenticationProvider` loads the user via your `UserDetailsService` and checks the password with the `PasswordEncoder`. On success, the authenticated `Authentication` (principal — the user's identity — plus authorities — the permissions granted) is stored in the `SecurityContextHolder` (thread-local: a variable holding a separate value for each thread) and, for session-based apps, in the HTTP session. → §10.4

Q102. How should passwords be stored? Hashed (turned by a one-way function into a fingerprint that can't be reversed) with a slow, salted (a random value mixed in per user, so equal passwords give different hashes), adaptive (its cost can be raised as hardware gets faster) function via `PasswordEncoderFactories.createDelegatingPasswordEncoder()`: bcrypt by default, Argon2id/scrypt/PBKDF2 also available; the `{bcrypt}` prefix records the algorithm so you can upgrade later (re-hash on login via `UserDetailsPasswordService`). Never encrypt passwords reversibly, never use MD5/SHA-256 alone. → §10.5

Q103. Role vs authority? `hasRole` vs `hasAuthority`? An authority is any granted permission string (`orders:write`, `SCOPE_orders.read`). A role is an authority with the `ROLE_` prefix. `hasRole("ADMIN")` checks the authority `ROLE_ADMIN`; `hasAuthority("ROLE_ADMIN")` is the same check spelled out. Mixing them up (`hasRole("ROLE_ADMIN")` or granting `ADMIN` without prefix) is a classic 403. → §10.6

Q104. 401 vs 403? 401 Unauthorized really means *unauthenticated*: no or invalid credentials — "who are you?" (sent with `WWW-Authenticate`, or a redirect to the login page for browsers). 403 Forbidden means the caller is known but lacks the permission — "I know you; no." `ExceptionTranslationFilter` decides: denied + anonymous → the `AuthenticationEntryPoint` (401/redirect); denied + authenticated → the `AccessDeniedHandler` (403). A missing CSRF token also gives 403. → §10.6

Q105. What is method security? Authorization on service methods with `@PreAuthorize("hasRole('ADMIN')")`, `@PostAuthorize("returnObject.owner == authentication.name")`, `@PreFilter`/`@PostFilter`, enabled with `@EnableMethodSecurity`. It is proxy-based (self-invocation again) and lets you express ownership rules that URL patterns can't. → §10.7

Q106. What is CSRF and when can you disable protection? Cross-Site Request Forgery: a malicious site makes the victim's browser send a state-changing request, and the browser adds the victim's cookies automatically. Spring requires a secret token on unsafe methods (POST, PUT, PATCH, DELETE). Disable it only for chains that never authenticate with browser-automatic credentials — e.g. a stateless API using `Authorization: Bearer` headers. SPAs using cookies should keep it (`csrf.spa()` in Security 7). → §10.8

Q107. Sessions or tokens? Sessions: the server stores the login and the browser holds a `JSESSIONID` cookie (a small value the browser stores and sends back automatically on every request to that site) — simple, instant logout, needs CSRF protection and shared session storage when scaled (Spring Session with Redis/JDBC). Tokens (JWT): the client sends a signed token on every call; services validate it locally without shared state — good for APIs and service-to-service, but revocation is harder, so lifetimes are short. → §10.10, §11.1

Q108. How does a Spring resource server validate a JWT? (A *resource server* is an API that accepts access tokens.) `BearerTokenAuthenticationFilter` extracts the token; the `JwtDecoder` (Nimbus) fetches the issuer's public keys (JWK Set — the JSON list of keys the token issuer publishes, found via `spring.security.oauth2.resourceserver.jwt.issuer-uri`), selects the key by `kid`, verifies the signature, then validators check the claims (fields inside the token) `exp`/`nbf` (expiry / not-before time, with 60 s clock skew allowed), `iss` (issuer) and — if you configure it — `aud` (audience: who the token is for). `JwtAuthenticationConverter` maps the `scope` claim to `SCOPE_*` authorities. Invalid token → 401; missing scope → 403. → §11.3, §11.5

Q109. OAuth 2.0 vs OpenID Connect? OAuth 2.0 is *delegated authorization*: a client gets an access token to call an API on the user's behalf. OIDC is an identity layer on top: it adds the ID token (a JWT about the user) and the UserInfo endpoint, so it answers "who logged in". "Sign in with Google" is OIDC using the authorization code flow with PKCE (Proof Key for Code Exchange — a one-time secret that stops a stolen authorization code from being used). → §11.2

Q110. JWT vs opaque tokens? A JWT is self-contained and validated locally (fast, no call per request) but cannot be revoked before expiry and exposes its claims. An opaque token is a random reference checked by calling the authorization server's introspection endpoint — instant revocation, at the cost of a network call (cache it briefly). → §11.7

Q111. How does one service call another securely? With the client credentials grant: `OAuth2AuthorizedClientManager` obtains and caches a token for the service itself, and an `OAuth2ClientHttpRequestInterceptor` adds it to `RestClient` calls. When the *user's* identity matters downstream, relay or exchange the user's token. Inside a service mesh (an infrastructure layer of proxies that handles traffic between services), mTLS (mutual TLS — both sides prove their identity with certificates) may also be used. → §11.10

Q112. Where should a browser SPA keep tokens? (An SPA — single-page application — is a browser app built with a framework such as React that calls APIs from JavaScript.) Preferably nowhere: the **BFF (backend-for-frontend)** pattern keeps tokens server-side and gives the SPA an `HttpOnly`, `Secure`, `SameSite` session cookie (plus CSRF protection). Tokens in `localStorage` (browser storage readable by any script on the page) can be stolen by any XSS (cross-site scripting: attacker-injected JavaScript running in your page). → §11.12

Q113. Is the security context available in `@Async` methods or new threads? Not by default — `SecurityContextHolder` uses a `ThreadLocal`. Wrap the executor (the thread pool that runs submitted tasks) in `DelegatingSecurityContextAsyncTaskExecutor` (or use `DelegatingSecurityContextRunnable`) to copy the context. In WebFlux, the context lives in the Reactor `Context` (`ReactiveSecurityContextHolder`) instead. → §10.10

Q114. (Security 7) What is new? Lambda DSL only, `authorizeRequests` and Ant/MVC matchers removed, built-in multi-factor authentication (login that needs two proofs, e.g. password plus a one-time code) (`FACTOR_*` authorities), `csrf.spa()`, Password4j-based encoders, PKCE sent by default by OAuth2 clients, the password grant removed, and Spring Authorization Server moved into the Spring Security project. → §10.14, §11.14

18.9 Testing

Q115. How do you structure tests in a Spring Boot app? A pyramid: many plain unit tests (tests of one class in isolation; no Spring — `new` the class with fakes/mocks, i.e. stand-in objects that return programmed answers and record calls), a layer of **slice tests** (`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest`) that start only the beans for one layer, and a few `@SpringBootTest` integration tests (tests of several real parts working together) of whole flows, ideally against real infrastructure via Testcontainers (a library that starts real databases and brokers in Docker containers for the test run). → §12.2

Q116. `@SpringBootTest` vs `@WebMvcTest`? `@SpringBootTest` loads the whole application context (all your beans and auto-configuration); with `webEnvironment = RANDOM_PORT` it starts a real server. `@WebMvcTest(OrderController.class)` loads only the MVC layer — controllers, `@ControllerAdvice`, converters, filters, security — and you mock the services with `@MockitoBean`. It is much faster and pinpoints web-layer bugs. → §1.12, §12.6, §12.7

Q117. (Boot 4) `@MockBean` doesn't compile any more. Why? Boot's `@MockBean`/`@SpyBean` were deprecated in 3.4 and **removed in Boot 4.0**. Use Spring Framework's `@MockitoBean` / `@MockitoSpyBean` (since 6.2), which replace or wrap a bean in the test context. → §12.10

Q118. `MockMvcTester`, `RestTestClient`, `WebTestClient` — which when? `MockMvcTester` (Framework 6.2) drives the `DispatcherServlet` in memory with AssertJ (a fluent assertion library: `assertThat(x).isEqualTo(y)`) assertions — no server; use it in `@WebMvcTest` (with `@SpringBootTest` you must add `@AutoConfigureMockMvc` — in Boot 4 `@SpringBootTest` alone no longer provides MockMvc). `RestTestClient` (Framework 7) is a fluent client for Spring MVC apps that can target MockMvc or a running server with the same test code; Boot 4 injects it only with `@AutoConfigureRestTestClient` (even in `@WebMvcTest`). `WebTestClient` does the same for WebFlux (automatic in `@WebFluxTest`, `@AutoConfigureWebTestClient` in `@SpringBootTest`). `TestRestTemplate` still exists but needs `@AutoConfigureTestRestTemplate` in Boot 4. → §12.8, §12.9

Q119. What is test context caching, and what breaks it? The TestContext framework caches each application context by its configuration "key" (configuration classes, active profiles, properties, context customizers such as `@MockitoBean` definitions) and reuses it across test classes in the same JVM (Java Virtual Machine — the running Java process). Every different combination — a new `@MockitoBean`, a different `@TestPropertySource`, `@ActiveProfiles` — creates a new context, and `@DirtiesContext` throws one away. Many variants make the suite slow; standardize on a few base configurations. The cache holds at most 32 contexts (LRU — least recently used: the context unused for longest is evicted first). In Framework 7, when the tests switch to a different context the inactive one is **paused** by default (its `Lifecycle`/`SmartLifecycle` beans — schedulers, listener containers — stopped) and restarted when reused. → §12.11

Q120. Does `@Transactional` on a test roll back? Yes: the test-managed transaction is rolled back after each test method by default (`@Commit` to keep it); `@DataJpaTest` is transactional out of the box. But with `@SpringBootTest(webEnvironment = RANDOM_PORT)` the request is handled on a server thread, in a different transaction, so data written by the request is **not** rolled back. → §8.17, §12.6

Q121. What does `@DataJpaTest` configure? JPA entities, repositories, a `DataSource` (replaced by an embedded database — by default `Replace.NON_TEST`, so a Testcontainers `@ServiceConnection` database is kept; `replace = NONE` always keeps yours), `TestEntityManager`, transactions rolled back per test — and no web layer or services. → §8.17, §12.7

Q122. How do you test against a real database or broker? Testcontainers starts a Docker container (a lightweight, isolated process packaged with everything it needs, e.g. a PostgreSQL server) per test class or suite; `@ServiceConnection` (Boot 3.1) on the container field registers the connection details automatically, and `@DynamicPropertySource` covers anything else. Prefer this to H2 (a small in-memory Java database) when you rely on database-specific SQL or behaviour. → §12.14

Q123. How do you test secured endpoints? With `spring-security-test`: `@WithMockUser(roles = "ADMIN")` for a fake user; request post-processors `user(...)`, `httpBasic(...)`, `jwt().authorities(...)` for token-based APIs; `csrf()` for unsafe methods on session-based chains. Always test both the allowed path and the 401/403 paths. → §10.12, §11.6

Q124. How do you test a custom auto-configuration or conditional beans quickly? `ApplicationContextRunner` (or `WebApplicationContextRunner`) builds a tiny context in milliseconds with chosen configurations and properties, and AssertJ checks like `assertThat(ctx).hasSingleBean(X.class)` or `hasFailed()`. The proofs in §18.15 use it. → §3.13

Q125. (Framework 7) What changed for testing? JUnit 6 (Jupiter 6 — the JUnit programming model — with a Java 17 baseline, i.e. minimum) via the Boot test starters; JUnit 4 support (`SpringRunner`) deprecated; `RestTestClient`; context pausing; `@MockitoBean` on non-singleton beans; `SpringExtension` uses a test-method-scoped extension context by default, which makes `@Nested` classes behave consistently. Boot 4 adds a test starter per technology (`spring-boot-starter-webmvc-test`, `-data-jpa-test`, …), moves the slice annotations to per-module packages, removes `@MockBean`, and requires explicit `@AutoConfigure…` annotations for test clients in `@SpringBootTest`. → §12.3, §12.17, §17.14

18.10 Reactive and WebFlux

Q126. What is reactive programming, and what is Reactive Streams? A style where you describe a pipeline of asynchronous (arriving later, without the caller waiting) data and the framework pushes items through it without blocking threads. Reactive Streams is the small standard behind it — `Publisher`, `Subscriber`, `Subscription`, `Processor` (also in the JDK as `java.util.concurrent.Flow`) — whose key idea is **backpressure**: the subscriber asks for `request(n)` items, so a fast producer cannot overwhelm a slow consumer. → §13.2

Q127. `Mono` vs `Flux`? Project Reactor's (the reactive library under WebFlux) publishers: `Mono<T>` emits 0 or 1 item then completes (or errors); `Flux<T>` emits 0..N items. Both are lazy — **nothing happens until someone subscribes** (in WebFlux the framework subscribes for you). → §13.3

Q128. `map` vs `flatMap` (vs `concatMap`)? `map` applies a synchronous function to each item. `flatMap` applies a function that returns another publisher (an async call) and merges the results — concurrently, so order is not preserved. `concatMap` does the same one at a time, preserving order; `flatMapSequential` runs concurrently but re-orders the output. → §13.4

Q129. What happens if you block inside a WebFlux handler? WebFlux on Netty (a non-blocking network server library) runs on a few event-loop threads (about one per core) — threads that each juggle many connections and must never wait. A blocking call (JDBC, `Thread.sleep`, `block()`) stalls one of them, and every request on it waits — throughput collapses (in §13.15 a 100 ms `Thread.sleep` cut WebFlux from ~8,000 to ~100 req/s). Reactor refuses `block()` on its non-blocking threads with an `IllegalStateException`. If you must call blocking code, move it to `Schedulers.boundedElastic()` with `subscribeOn`; better, use non-blocking drivers (R2DBC — Reactive Relational Database Connectivity, `WebClient`). → §13.8, §13.15

Q130. `publishOn` vs `subscribeOn`? `subscribeOn` chooses the scheduler on which the subscription — and therefore the source — runs, wherever it appears in the chain. `publishOn` switches the thread for the operators *after* it. → §13.8

Q131. WebFlux or Spring MVC with virtual threads? Since Java 21 and Boot 3.2 (`spring.threads.virtual.enabled=true`), MVC can handle each request on a cheap virtual thread (a lightweight thread managed by the JVM that frees its underlying OS thread while it waits on I/O), so simple blocking code scales to many concurrent I/O-bound requests (requests that spend most time waiting for the network or disk, not computing). Choose WebFlux when you need streaming (SSE — Server-Sent Events, a one-way stream from server to browser; WebSockets — a two-way connection kept open; with backpressure), a fully non-blocking stack end to end, or heavy fan-out composition; choose MVC + virtual threads for typical CRUD (Create, Read, Update, Delete) services with JDBC/JPA — simpler code, stack traces and debugging (measured in §13.15: at 1,000 concurrent clients on a 100 ms I/O endpoint, MVC on virtual threads and WebFlux both reached ~8,000 req/s vs ~1,900 on 200 platform threads — ordinary threads, each backed by an OS thread). Either way the database pool is still the real limit. → §13.15, Java 15.4

Q132. How do you test reactive code, and how do transactions and security work there? `StepVerifier` subscribes and asserts the emitted items, errors and completion (with virtual time — a fake clock the test can fast-forward — for delays); `WebTestClient` tests endpoints. Since there is no one thread per request, reactive transactions (`ReactiveTransactionManager`, R2DBC) and the security context (`ReactiveSecurityContextHolder`) are carried in the Reactor `Context`, not in `ThreadLocal`s. JPA/JDBC are blocking — use R2DBC for a reactive database. Note that `@DataR2dbcTest` does **not** roll back like `@DataJpaTest`. → §13.5, §13.14

18.11 Messaging, async and scheduling

Q133. How does `@Async` work, and what are its pitfalls? With `@EnableAsync`, calls to `@Async` methods go through a proxy that submits them to an executor and returns immediately (`void` or `CompletableFuture<T>` — a handle to a result that will arrive later). Boot auto-configures the `applicationTaskExecutor` (a `ThreadPoolTaskExecutor`, tuned by `spring.task.execution.*`, or virtual threads when enabled). Pitfalls: self-invocation, exceptions from `void` methods only reach an `AsyncUncaughtExceptionHandler`, the default pool's queue is unbounded, and the caller's transaction and security context are not propagated. → §14.4, §14.6

Q134. `fixedRate` vs `fixedDelay` vs `cron` in `@Scheduled`? `fixedRate` starts runs at a fixed interval from start to start; `fixedDelay` waits a fixed time after the previous run *finished*; `cron` uses a calendar expression (the classic Unix scheduling syntax, e.g. `0 0 3 * * *` = 03:00 every day) (Spring's 6 fields including seconds, plus `zone`). Needs `@EnableScheduling`. Boot's scheduler has **one thread** by default (`spring.task.scheduling.pool.size`), so a slow job delays the others. → §14.7

Q135. A `@Scheduled` job runs on every instance. How do you run it once? Scheduling is per JVM. Use a distributed lock (a lock shared by all instances, held in a database or Redis — ShedLock with a database table, or a Redis lock), leader election (the instances agree on one "leader" that does the work), or move the job to a single scheduler (Kubernetes CronJob, Quartz in clustered mode). Make the job idempotent anyway. → §14.7

Q136. How do you produce and consume Kafka messages with Spring? (**Kafka** is a distributed event-streaming platform: producers append messages to named *topics*, each split into ordered *partitions*, and consumers read them.) Boot 4 has `spring-boot-starter-kafka`. Produce with `KafkaTemplate.send(topic, key, value)`, which returns a `CompletableFuture<SendResult>`; consume with `@KafkaListener(topics = "orders", groupId = "billing")`. The key decides the partition, and Kafka guarantees order only **within a partition** — so key by the entity (order id) whose events must stay ordered. Consumers in one group (a consumer group — instances of the same service that split the work) share the partitions. → §14.11, §14.12

Q137. What delivery guarantee does a Spring Kafka listener give, and how do you handle duplicates? At-least-once (every message is processed, but possibly more than once) by default: spring-kafka turns off Kafka's auto-commit and commits offsets (the position number of the last processed record in a partition) after the listener succeeds (ack mode `BATCH`: after the whole polled batch), so a crash before the commit redelivers the record. Make consumers **idempotent** (store processed message ids, use upserts or natural keys). The producer side has idempotence enabled by default in modern Kafka clients, which prevents duplicates caused by producer retries — not duplicates caused by consumer redelivery. → §14.8, §14.16

Q138. How do you handle a message that keeps failing? Configure the listener container's `DefaultErrorHandler` with a back-off (limited retries) and a `DeadLetterPublishingRecoverer` that sends the record to a dead-letter topic (a side topic where messages that keep failing are parked for later inspection) after the last attempt (default name `<topic>-dlt` since spring-kafka 3.3; older articles show `<topic>.DLT`), on the same partition number; mark non-retryable exceptions (validation, deserialization — turning the received bytes back into an object — use `ErrorHandlingDeserializer`) so they go straight to the DLT. For long waits without blocking the partition, use non-blocking retry topics (`@RetryableTopic`). → §14.14, §14.15

Q139. How do you update the database and publish an event reliably? You can't make a database commit and a Kafka send atomic (all-or-nothing) with a normal transaction (the dual-write problem). Use the **transactional outbox**: write the event to an `outbox` table in the same database transaction as the business change, and let a relay (a poller or CDC — Change Data Capture, reading the database's own change log — such as Debezium) publish it; consumers deduplicate. → §14.17, event-driven notes

Q140. Kafka vs RabbitMQ? Kafka is a partitioned, replicated (copied to several servers) **log**: messages are kept for a retention period, consumers track offsets and can replay, very high throughput, ordering per partition — good for event streaming. RabbitMQ is a **broker with queues** (a message broker is a server that receives messages and hands them to consumers): flexible routing (exchanges — routers that receive messages — and bindings — rules that connect them to queues), per-message acknowledgement, messages removed when consumed — good for task queues and complex routing. With Spring AMQP (Advanced Message Queuing Protocol, RabbitMQ's protocol), blocking listener retries are `spring.rabbitmq.listener.simple.retry.*` (Boot 4 replaced `max-attempts`, which counted the first attempt, with `max-retries`, which does not), and rejected messages go to the queue's dead-letter exchange when `default-requeue-rejected=false`. → §14.18

18.12 Production and observability

Q141. What is Spring Boot Actuator, and how do you expose it safely? Production endpoints under `/actuator`: `health`, `info`, `metrics`, `prometheus` (metrics in the format of Prometheus, a monitoring system that scrapes and stores them), `loggers` (change log levels at runtime), `env`, `configprops`, `beans`, `conditions`, `threaddump`, `heapdump`… Over HTTP only `health` is exposed by default; add more with `management.endpoints.web.exposure.include`. Protect everything except health with Spring Security, or move it to a separate `management.server.port` not reachable from outside. `env`/`configprops` values are masked by default; `heapdump` (a file with every object in JVM memory) can contain secrets. → §15.2

Q142. Liveness vs readiness probes? Liveness answers "is the process broken beyond repair?" — if it fails, Kubernetes restarts the pod (the smallest unit Kubernetes runs: one or more containers). Readiness answers "can it take traffic now?" — if it fails, the pod is removed from load balancing (spreading requests over instances — see System Design Fundamentals) but not restarted (during startup, shutdown or a lost dependency). Boot exposes `/actuator/health/liveness` and `/readiness`; in Boot 4 these probes are enabled by default. Never put the database check into liveness: a database outage would restart every pod and make things worse. → §15.3

Q143. What is Micrometer? Name the meter types. A vendor-neutral metrics facade (one API in front of many back ends — "SLF4J for metrics", SLF4J being the standard Java logging facade): code records to a `MeterRegistry`, and registries export to Prometheus, OTLP (OpenTelemetry Protocol), Datadog, etc. Types: `Counter` (only increases), `Gauge` (current value), `Timer` (count + total + max + optional percentiles/histograms of durations — e.g. p99, the time 99% of calls beat), `DistributionSummary` (same for sizes), `LongTaskTimer`. Keep **tag cardinality** (the number of distinct tag values — each one creates a separate time series) low — never tag with user id or raw URL. → §15.5

Q144. What is the Observation API? Micrometer's "instrument once" API: you wrap an operation in an `Observation` (or annotate with `@Observed`), and registered handlers turn it into a timer metric, a trace span (one timed step of a request, see Q145) and log correlation at the same time. Spring MVC, `RestClient`, JDBC, Kafka and others are instrumented this way already. → §15.6

Q145. How does distributed tracing work in Boot? Micrometer Tracing (the successor of Spring Cloud Sleuth since Boot 3) with an OpenTelemetry (the vendor-neutral standard for traces, metrics and logs) or Brave bridge creates spans for incoming and outgoing calls and propagates the context (the trace id) in W3C `traceparent` headers (and in Kafka headers). Spans are exported (OTLP → Tempo/Jaeger/Zipkin, tools that store and display traces); `traceId`/`spanId` appear in log lines for correlation. The default sampling probability (share of requests that are traced) is 10% (`management.tracing.sampling.probability`). Boot 4 adds `spring-boot-starter-opentelemetry` for OTLP export of metrics and traces (Brave/Zipkin: `spring-boot-starter-zipkin`). → §15.7

Q146. What is structured logging in Boot? Since Boot 3.4, `logging.structured.format.console=ecs` (or `logstash`, `gelf`) writes each log event as one JSON object — machine-parseable, with MDC (Mapped Diagnostic Context — per-thread key-values added to every log line) fields and trace ids — instead of formatting text you later have to parse with regexes (regular expressions, text-matching patterns). → §15.8

Q147. What does graceful shutdown do? On `SIGTERM` (the signal the OS or Kubernetes sends to ask a process to stop politely), the web server stops accepting new requests, in-flight requests (already being processed) may finish within `spring.lifecycle.timeout-per-shutdown-phase` (30 s default), then the context closes and `@PreDestroy`/`SmartLifecycle.stop` run. It is the default since Boot 3.4. In Kubernetes, readiness turns to "refusing traffic" first; add a short `preStop` sleep (a Kubernetes hook that runs before the stop signal) so the load balancer stops routing before the app stops. → §15.9

Q148. What does `spring.threads.virtual.enabled=true` change? (Boot 3.2+, Java 21+.) Tomcat handles requests on virtual threads, `@Async` and the task scheduler use virtual-thread executors, and some clients and listener containers use them too. Blocking I/O no longer ties up a scarce platform thread. It does not make CPU-bound work (work limited by computation, not waiting) faster, and the connection pool still limits database concurrency — protect downstream systems with limits such as `@ConcurrencyLimit`. Java 24 (JEP 491 — a JDK Enhancement Proposal) removed most pinning (a virtual thread stuck to its OS thread, losing the benefit) caused by `synchronized`. → §15.10, §14.5, §4.7

Q149. What are AOT processing and GraalVM native images? Trade-offs? Spring AOT runs at build time: it evaluates the configuration and generates source code for the bean definitions plus reflection/proxy "hints". GraalVM then compiles the app into a native executable under a closed-world assumption (every class the program can ever use must be known at build time). Result: startup in tens of milliseconds and less memory — good for scale-to-zero (running no instances while there is no traffic) and CLIs. Costs: long builds, conditions and profiles fixed at build time, reflection-heavy libraries need hints, and peak throughput may be lower than a warmed-up JIT (Just-In-Time compiler — turns frequently run code into machine code while the app runs). → §15.11

Q150. How can a normal JVM app start faster without native images? Class Data Sharing (CDS — a file of pre-parsed classes the JVM maps at startup) and the JDK's **AOT cache** (JEP 483, Java 24; simplified by JEP 514 in Java 25): do a training run that loads and links classes (Boot supports `-Dspring.context.exit=onRefresh` for this), store the result, then start with it — often a

large reduction in startup time, with the full JIT still available. Also: lazy init where acceptable, fewer auto-configurations, and not scanning huge packages. → §15.11

Q151. How do you package a Boot app as a container image? `mvn spring-boot:build-image` builds an OCI image (the standard container image format) with Cloud Native Buildpacks (tools that turn source or a jar into an image automatically — no Dockerfile, the hand-written image recipe); or a Dockerfile that extracts the jar into layers (`java -Djarmode=tools -jar app.jar extract --layers`) so dependencies are cached separately from your code. Run as non-root (not the all-powerful admin user), and size the heap (the JVM memory area for objects) relative to the container limit (`-XX:MaxRAMPercentage`). → §15.11

Q152. What is your production configuration checklist? Secrets from the environment/vault (a secret store such as HashiCorp Vault), not files in the jar; `ddl-auto=validate` + migrations; `spring.jpa.open-in-view=false`; timeouts on every client; right-sized pools; actuator exposure limited and secured; structured logs; tracing and metrics exported; graceful shutdown with probes; JVM memory flags for containers; `server.forward-headers-strategy` set when behind a proxy (here a reverse proxy or load balancer in front of the app that adds `X-Forwarded-*` headers). → §15.12

18.13 Microservices and Spring Cloud

Q153. Which Spring Cloud version works with Boot 4? The **2025.1.x "Oakwood"** release train (a set of project versions released and tested together; GA — general availability, the first production-ready release — in November 2025) is built on Framework 7 / Boot 4; its projects moved to 5.0.x. Boot 4.1 is supported from 2025.1.2 (2025.1.3 was the latest service release when this book was written). Always import the `spring-cloud-dependencies` BOM (Bill of Materials — a Maven file that pins compatible versions of many libraries) matching your Boot version. → §16.2

Q154. What does Spring Cloud Config do, and how are changes picked up? A Config Server serves configuration from Git (the version-control system; or Vault, a database…) to all services; clients load it with `spring.config.import=configserver:`. Beans in `@RefreshScope` (and `@ConfigurationProperties` beans) are rebuilt on `POST /actuator/refresh`, or for all instances via Spring Cloud Bus. On Kubernetes, ConfigMaps/Secrets (Kubernetes objects holding configuration and secret values) mounted as files (`configtree:`) often replace it. → §16.3, config notes

Q155. Eureka or Kubernetes for service discovery? Eureka is a registry (a directory of which instances of each service are running and where) that services register with and query, used with client-side load balancing (the caller itself picks one instance from the list) (Spring Cloud LoadBalancer + a `@LoadBalanced` `RestClient.Builder`). On Kubernetes, the platform already provides discovery (Service DNS names — DNS is the system that turns names into network addresses) and load balancing, so Eureka is usually unnecessary. → §16.4, discovery notes

Q156. What does an API gateway do, and how does Spring Cloud Gateway work? A single entry point that routes requests to services and handles cross-cutting edge concerns: authentication/token relay, rate limiting (capping how many requests a client may make per time window), CORS, retries, request/response rewriting. Spring Cloud Gateway defines **routes** (id, URI), **predicates** (conditions a request must match: path, host, header…) and **filters**; it exists in a reactive variant (`spring-cloud-starter-gateway-server-webflux`, Netty) and a servlet variant (`spring-cloud-starter-gateway-server-webmvc`, pairs well with virtual threads). In Gateway 5.0 (Spring Cloud 2025.1) the MVC `http(String)`/`http(URI)` handler overloads are gone — use `http()` with `before(uri(...))`. → §16.7, gateway notes

Q157. How do you make service-to-service calls resilient? Timeouts first; retries with back-off and jitter for idempotent calls (Framework 7 `@Retryable` or Resilience4j); a **circuit breaker** (Resilience4j via Spring Cloud CircuitBreaker) that fails fast when a dependency is unhealthy; a **bulkhead** (`@ConcurrencyLimit` or Resilience4j) so one slow dependency cannot use all threads; fallbacks (a default answer returned when the call fails) where acceptable. Order matters: in Resilience4j's default aspect order, retry wraps the circuit breaker. → §16.8, circuit breaker notes

Q158. How do you keep data consistent across services? Not with distributed (2PC/XA — two-phase commit, a protocol that coordinates one commit across several databases) transactions in practice. Each service owns its database; use a **saga** — a sequence of local transactions with compensating actions (steps that undo earlier ones), orchestrated (one coordinator tells each service what to do) or choreographed (each service reacts to the others' events) through events — plus the transactional outbox and idempotent consumers. Accept eventual consistency (all services agree after a short delay, not instantly) and design the UI for it. → saga notes

Q159. How do you test contracts between services? Consumer-driven contract tests: the consumer's expectations are written as contracts, the provider's build verifies them, and the consumer tests against generated stubs (fake versions of the provider built from the contracts). This catches breaking API changes before deployment without full end-to-end environments (a copy of the whole system with every service running). Today the usual tool is **Pact** (polyglot, with a broker and a `can-i-deploy` gate). **Spring Cloud Contract** is no longer in the 2025.1.3 BOM and its repository is archived, so don't start new work on it. → §16.10

Q160. Feign or HTTP interfaces? Spring Cloud OpenFeign (an older declarative HTTP client library) is feature-complete (maintenance only). New code should use Spring's own `@HttpExchange` interfaces, which Boot 4 auto-configures, with Spring Cloud LoadBalancer when you need client-side discovery. → §6.6, §16.6

18.14 Spring AI and the Boot 4 / Framework 7 changes

Q161. What is Spring AI? Which version goes with Boot 4? A Spring project that gives portable APIs over AI models and vector stores (databases that store embeddings and find the most similar ones): `ChatClient`/`ChatModel`, `EmbeddingModel`, `VectorStore`, tool calling, chat memory (earlier messages of a conversation kept and re-sent) and MCP support, with Boot starters per

provider (OpenAI, Anthropic, Ollama — a tool for running models locally, …). **Spring AI 2.0** (GA June 2026; 2.0.1 current) requires Boot 4 (4.1.x is the smoothest match); the 1.x line (1.1.x) is for Boot 3.4/3.5. → §17.1, AI book

Q162. How do you call a model and get typed output? Inject `ChatClient.Builder`, build a client with defaults (system prompt — fixed instructions sent to the model before the user's message; advisors — interceptors that add to or change each request), then `chatClient.prompt().user("...").call().content()` for text or `.entity(Invoice.class)` for **structured output** — Spring AI adds format instructions and converts the JSON reply into your record. Validate the result like any untrusted input. → §17.3, §17.7

Q163. How does RAG work in Spring AI? (RAG — Retrieval-Augmented Generation — means looking up relevant text first and giving it to the model with the question.) Ingest: split documents into chunks, embed them (turn each chunk into an *embedding*, a list of numbers where similar meanings are close together) with an `EmbeddingModel`, store them in a `VectorStore` (PGVector, Redis, …). Query: a RAG advisor (for example `QuestionAnswerAdvisor`) runs a similarity search for the user's question and adds the best chunks to the prompt (the text sent to the model), so the model answers from your data. Quality depends on chunking, metadata filters and evaluation. → §17.9, AI book 7

Q164. What are tools and MCP in Spring AI? Tools are your methods annotated with `@Tool` that the model may ask to call (look up an order, check stock); Spring AI executes the call and returns the result to the model. MCP (Model Context Protocol) is a standard for exposing tools and resources across processes; Spring AI has MCP client and server starters (Streamable HTTP is the default transport for its web servers in 2.0; the older SSE transport is deprecated). Treat tool calls as untrusted input: authorize them and never expose destructive operations without checks. → §17.8, §17.10

Q165. How do you test code that uses an LLM? (An LLM — large language model — is the text-generating AI model behind a chat API.) Unit-test your logic with a stub `ChatModel` that returns fixed responses (fast, deterministic — same result every run, no API key — the secret that authorizes calls to the provider); keep a smaller set of evaluation tests against the real model that check properties of the answer (relevance, grounding, format) rather than exact strings (Spring AI ships `RelevancyEvaluator` and `FactCheckingEvaluator`). → §17.2

Q166. What changed from Boot 3 to Boot 4? Framework 7 / Jakarta EE 11 (the enterprise Java standards, formerly Java EE: Servlet 6.1, Tomcat 11, Jetty 12.1 — Tomcat, Jetty and Undertow are embedded web servers; **Undertow removed**); modular auto-configuration with renamed starters and per-technology test starters; Jackson 3; JUnit 6; JSpecify null-safety; auto-configuration for API versioning and HTTP service clients; `RestTestClient`; OpenTelemetry starter; liveness/readiness probes on by default; `@MockBean`/`@SpyBean` and embedded launch scripts removed (Spock — a Groovy-based test framework — support was dropped in 4.0 and is back in 4.1); `@SpringBootTest` no longer sets up MockMvc, `RestTestClient`, `WebTestClient` or `TestRestTemplate` without the matching `@AutoConfigure…` annotation; `WebClient.Builder` needs `spring-boot-starter-webclient`. Java 17 is still the minimum. → §3.14, §17.13

Q167. What changed from Framework 6 to 7? JSpecify annotations (the `org.springframework.lang` ones deprecated); API versioning; HTTP service groups; `BeanRegistrar`; core resilience (`@Retryable`, `@ConcurrencyLimit`, `RetryTemplate`); `@Proxyable`; `RestTestClient` and test context pausing; Jackson 3 support (Jackson 2 deprecated); JUnit 4 support deprecated; `javax.annotation`/`javax.inject` support removed; `spring-jcl` replaced by Apache Commons Logging 1.3; `RestTemplate` deprecation planned for 7.1. → §17.14

Q168. How would you migrate a Boot 3 app to Boot 4? First upgrade to the latest Boot 3.5.x and remove every deprecation warning; make sure you are on Java 17+ and Jakarta (no `javax`). Then switch the parent to 4.x, rename starters (or use the `-classic` starters temporarily), fix Jackson 3 imports and customizers, replace `@MockBean`, add the per-technology test starters and `@AutoConfigureMockMvc` where needed, update Spring Cloud to 2025.1.x (2025.1.2+ for Boot 4.1) and Spring AI to 2.0.x, rename changed properties (e.g. RabbitMQ `retry.max-attempts` → `retry.max-retries`), run the test suite and compare the condition report and `/actuator` output. OpenRewrite recipes (scripted, automated source-code refactorings) automate much of the mechanical work. → §17.13

18.15 Proofs: five tricky answers, run

What it is

Five answers that candidates often get wrong, each proved by a test in `code/interview-proofs/` (Boot 4.1.1 parent, Java 25, `spring-boot-starter-jdbc` + H2 + `spring-boot-starter-test`). Every test builds a tiny context with Boot's `ApplicationContextRunner`, so each scenario — including the ones that *fail* to start — is isolated. Run it with `mvn -q -B -ntp verify` (or `bash tools_run_spring.sh spring '18-*'` from the repository root).

Test class	Proves the answer to
`ScopeProofTests`	Q11 — prototype injected into a singleton
`BeanResolutionTests`	Q7/Q8 — `@Qualifier` vs `@Primary` vs parameter name
`CircularDependencyTests`	Q15 — constructor cycles, `@Lazy`, Boot's field-cycle default
`RollbackTests`	Q67 — checked exception commits
`SelfInvocationTests`	Q39/Q71 — self-invocation skips `@Transactional`

Proof 1 — a prototype inside a singleton is created once

```
@Bean @Scope("prototype") ShoppingCart cart() { return new ShoppingCart(); }
@Bean CheckoutService checkout(ShoppingCart cart, ObjectProvider<ShoppingCart> carts) {
    return new CheckoutService(cart, carts);   // cart is resolved ONCE, here
}
// test
CheckoutService a = ctx.getBean(CheckoutService.class), b = ctx.getBean(CheckoutService.class);
System.out.println("same injected cart both times?        " + (a.injectedOnce == b.injectedOnce));
System.out.println("ObjectProvider.getObject() twice same? " + (a.carts.getObject() == a.carts.getObject()));
```

Output:

```
same CheckoutService both times?      true
same injected cart both times?        true
ObjectProvider.getObject() twice same? false
```

The singleton got one cart at construction and keeps it; asking the container through `ObjectProvider` returns a new cart each time.

Proof 2 — `@Qualifier` beats `@Primary`; the parameter name is the last tie-breaker

```
@Bean @Primary PaymentGateway stripe() { return new Gateway("stripe"); }
@Bean PaymentGateway paypal() { return new Gateway("paypal"); }
@Bean Checkout checkout(PaymentGateway gateway) { ... }                      // -> stripe
@Bean Refunds refunds(@Qualifier("paypal") PaymentGateway gateway) { ... }   // -> paypal
// second config: no @Primary, parameter named "paypal"                      // -> paypal
// third config:  no @Primary, parameter named "gateway"                     // -> fails
```

Output:

```
no qualifier       -> stripe  (the @Primary bean)
@Qualifier(paypal) -> paypal  (qualifier wins over @Primary)
no @Primary, parameter named 'paypal' -> paypal
context failed: NoUniqueBeanDefinitionException
  No qualifying bean of type 'ch18.proofs.BeanResolutionTests$PaymentGateway' available: expected single matching bean but found 2:
stripe,paypal
```

Parameter-name matching works because the Boot parent compiles with `-parameters` (a compiler flag that keeps parameter names in the bytecode); do not rely on it for important choices — a rename silently changes which bean is used.

Proof 3 — circular dependencies

```
static class OrderService   { OrderService(InvoiceService invoices) { } }
static class InvoiceService { InvoiceService(OrderService orders) { } }        // constructor cycle
static class LazyOrderService { LazyOrderService(@Lazy LazyInvoiceService invoices) { } }
static class FieldA { @Autowired FieldB b; }  static class FieldB { @Autowired FieldA a; }
// ApplicationContextRunner follows Boot's default: circular references not allowed
new ApplicationContextRunner().withAllowCircularReferences(true) ...           // = spring.main.allow-circular-references=true
```

Output:

```
constructor cycle      -> BeanCurrentlyInCreationException
field cycle, Boot default          -> BeanCurrentlyInCreationException
field cycle, circular refs allowed -> started: true
cycle with @Lazy       -> started: true
```

The log line for the constructor case shows the path Spring followed — read it when debugging: `Error creating bean with name '...OrderService': ... '...InvoiceService': ... '...OrderService': Requested bean is currently in creation: Is there an unresolvable circular reference ...`

Proof 4 — a checked exception commits

```
@Transactional
public void checkedDefault(int id) throws PaymentDeclinedException {       // checked
    insert(id); throw new PaymentDeclinedException("card declined");
}
@Transactional(rollbackFor = PaymentDeclinedException.class)
public void checkedWithRollbackFor(int id) throws PaymentDeclinedException { ...same... }
@Transactional
public void unchecked(int id) { insert(id); throw new IllegalStateException("bug"); }
```

Output:

```
checked exception, default rules  -> row 1 kept?  true
checked exception, rollbackFor    -> row 2 kept?  false
RuntimeException, default rules   -> row 3 kept?  false
```

All three methods threw, yet row 1 was committed: that is the default rollback rule, not a bug in Spring.

Proof 5 — self-invocation skips the transaction

```
public void importAll() { saveBatch(); }                // this.saveBatch(): no proxy
@Transactional
public void saveBatch() {
    System.out.println("  inside saveBatch, transaction active? "
            + TransactionSynchronizationManager.isActualTransactionActive());
    jdbc.sql("INSERT INTO orders VALUES (10, 'a')").update();
    throw new IllegalStateException("second insert failed");
}
```

Output:

```
bean class: SelfInvocationTests$ImportService$$SpringCGLIB$$0
call through the proxy (svc.saveBatch()):
  inside saveBatch, transaction active? true
  rows after failure: 0  (rolled back)
self-invocation (svc.importAll() -> this.saveBatch()):
  inside saveBatch, transaction active? false
  rows after failure: 1  (NOT rolled back - no transaction)
```

The bean class name shows the CGLIB proxy. `TransactionSynchronizationManager.isActualTransactionActive()` is the quickest way to *prove* whether a transaction exists at a given line — use it in any debugging session.

Self-check

1. Why does the test use `ApplicationContextRunner` instead of `@SpringBootTest`? → Each scenario needs its own small context, and some must fail to start; the runner captures the failure instead of failing the test class.
2. What one-liner proves a transaction is active? → `TransactionSynchronizationManager.isActualTransactionActive()`.

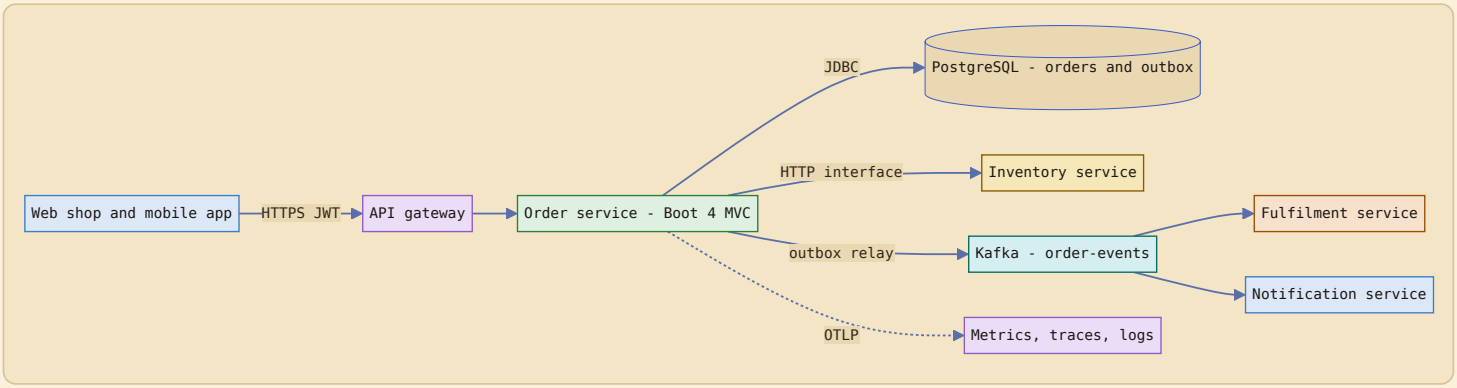
18.16 "Explain your Spring project" — a walkthrough template

Why it matters

"Tell me about a project you built with Spring" is asked in almost every interview. The interviewer is checking whether you understand *why* the pieces are there, and they will dig into whatever you mention. A structured five-minute answer lets you steer that digging toward topics you know.

The template (about 5 minutes)

1. **Purpose in one sentence** — who uses it and what business problem it solves. "An order service that takes orders from the web shop and mobile app, reserves stock and hands paid orders to fulfilment."
2. **Scale and constraints** — requests per second, data size, latency target (latency = how long one request takes), team size. Numbers make it real; round honestly.
3. **Architecture picture** — draw it (below). Name each box and the protocol on each arrow.
4. **Stack and why** — Boot version, MVC or WebFlux (and why), database and access style, messaging, security. For each, one reason.
5. **Follow one request end to end** — through the filter chain, controller, validation, service transaction, repository, event, response.
6. **Data and consistency** — transactions, locking, how events leave the service reliably, how duplicates are handled.
7. **Security** — how users and other services authenticate, where authorization lives.
8. **Quality** — test strategy (unit, slices, Testcontainers, contract tests), CI (Continuous Integration — every change built and tested automatically).
9. **Running it** — deployment, observability (metrics, traces, logs, alerts), how you handle failures of dependencies.
10. **One hard problem** (Situation → Task → Action → Result — the STAR story format) and **what you would improve**.



A filled-in example (say it in your own words)

> "The order service is a Spring Boot 4 MVC application on Java 25 with virtual threads enabled. We chose MVC over WebFlux because the work is CRUD plus a few downstream calls, and virtual threads gave us the concurrency we needed with simple blocking code.
>
> A `POST /orders` passes the Spring Security chain — it's a stateless resource server validating JWTs from our identity provider (the service that logs users in and issues tokens), with `orders:write` scope required. The controller validates the request DTO with Bean Validation and calls `OrderService.place()`, which is the one `@Transactional` boundary. Inside it we check stock through an `@HttpExchange` client to the inventory service — with 2-second timeouts and a `@Retryable` on the idempotent reservation call, which carries an idempotency key — then save the order with Spring Data JPA and write an `OrderPlaced` row to an outbox table in the same transaction. A relay publishes outbox rows to Kafka; fulfilment and notification consume them idempotently using the event id.
>
> We use optimistic locking with `@Version` on orders because concurrent edits are rare, and turned off Open Session in View; read endpoints use DTO projections and `@EntityGraph` to avoid N+1 queries, which we check with statement-count assertions in `@DataJpaTest`s.
>
> Tests: plain unit tests for pricing rules, `@WebMvcTest` with `MockMvcTester` and `jwt()` for the API including 401/403 cases, `@DataJpaTest` against PostgreSQL in Testcontainers, and a contract test with inventory. In production we export Micrometer metrics and OpenTelemetry traces; alerts are on p99 latency, error rate and consumer lag (how far consumers are behind the newest messages).
>
> The hardest issue was duplicate orders when clients retried after timeouts; we added an `Idempotency-Key` header stored with a unique constraint (a database rule that rejects a second row with the same value), returning the original response on a repeat. Next I'd split the read side into a separate projection, because reporting queries compete with order writes."

Follow-ups to prepare for (and where to revise)

They ask	Be ready to explain	Revise
"Why not WebFlux?"	virtual threads vs reactive trade-off	Q131
"What if Kafka is down?"	outbox keeps events; relay retries; consumer lag alerts	Q139
"How do you avoid double charging?"	idempotency keys, unique constraints, idempotent consumers	Q56, Q137
"Show me the transaction boundary"	service method, propagation, what's outside it	Q65–Q72
"How do you know it's slow?"	metrics, traces, SQL logging, statement counts	Q143–Q145, Q83
"What would break at 10× traffic?"	DB connections, hot partitions, cache, N+1	Q64, Q136
"How is it secured?"	JWT validation, scopes, method security, service auth	Q108, Q111

Common mistakes

- Listing technologies without reasons ("we used Kafka, Redis, Kubernetes…").
- Saying "we" for everything — say what **you** designed, built or fixed.
- Mentioning something you can't explain two levels deeper; the interviewer *will* ask about it.
- Inventing numbers — approximate honestly ("a few hundred requests per second at peak").

TL;DR

Purpose → scale → picture → stack with reasons → one request end to end → data, security, tests, operations → one hard problem and one improvement.

18.17 Debugging scenarios

Interviewers increasingly ask "this is happening in production — what do you do?" A good answer follows the same four steps every time: **symptom** (what exactly you see), **cause** (the mechanism, not a guess), **fix** (the smallest correct change) and **proof** (how you show the cause and the fix with evidence — a log line, a test, a metric). Always read the *root cause* (the original exception, shown as the last "Caused by") at the bottom of a Spring stack trace; the top lines are just the chain of beans that were being created.

Scenario 1 — "No qualifying bean" / `NoUniqueBeanDefinitionException`

- **Symptom.** Startup fails with the Boot failure analyzer's report (Boot's human-readable explanation printed for common startup errors): *"Parameter 0 of constructor in OrderService required a bean of type 'PaymentGateway' that could not be found"* (`NoSuchBeanDefinitionException`), or *"...required a single bean, but 2 were found"* (`NoUniqueBeanDefinitionException`).
- **Cause (not found).** The class is outside the scanned packages (main class not in the root package); it lacks a stereotype annotation (`@Component`, `@Service`, `@Repository`, `@Controller`); its `@Profile`/`@Conditional` didn't match; the auto-configuration that should create it backed off or its starter is missing (common after the Boot 4 starter renames); or you inject the concrete class while a JDK proxy only implements the interface.

- **Cause (not unique).** Two beans of the type — often your own bean plus one from a library or auto-configuration — and no `@Primary`, `@Qualifier` or matching parameter name.
- **Fix.** Move the main class up / add `@ComponentScan` or `@Import`; add the missing starter; activate the profile; for ambiguity choose with `@Qualifier` at the injection point or mark the usual one `@Primary`; inject `List<T>` if you want all.
- **Prove it.** `--debug` → condition evaluation report shows why an auto-configuration did *not* match; `/actuator/beans` (or `ctx.getBeanNamesForType(PaymentGateway.class)` in a test) lists the candidates; an `ApplicationContextRunner` test reproduces the failure in milliseconds (see `BeanResolutionTests`, §18.15). → Q7, §2.5, §3.11

Scenario 2 — circular dependency at startup

- **Symptom.** *"The dependencies of some of the beans in the application context form a cycle"* with an ASCII diagram of the loop, or `BeanCurrentlyInCreationException: Requested bean is currently in creation`.
- **Cause.** A → B → A through constructors (never resolvable), or through fields/setters, which Boot rejects by default since 2.6. It usually means responsibilities are split wrongly — e.g. `OrderService` calls `InvoiceService` and `InvoiceService` calls back into `OrderService`.
- **Fix.** Extract the shared logic into a third bean both can depend on; or turn the back-call into an application event; or depend on a narrower interface. `@Lazy` on one constructor parameter is an acceptable short-term patch. Setting `spring.main.allow-circular-references=true` hides the design problem — avoid it.
- **Prove it.** The failure report lists the cycle; after the fix, a context test (or `CircularDependencyTests`) starts cleanly. → Q15, §2.9

Scenario 3 — `@Transactional` does not roll back

- **Symptom.** A method threw, but its first insert is still in the database.
- **Cause (in order of likelihood).** (1) The exception is **checked** and there is no rollback rule. (2) The exception was **caught** inside the method (or in a lower method) and never reached the proxy. (3) **Self-invocation** or a `private`/`final` method — there was no transaction at all. (4) The write went through a different `DataSource`/transaction manager than the one `@Transactional` uses. (5) The work ran on another thread (`@Async`, `parallelStream`, `CompletableFuture`). (6) MySQL MyISAM tables (an old MySQL storage engine), which don't support transactions.
- **Fix.** `rollbackFor`, or the global `rollbackOn = ALL_EXCEPTIONS`; rethrow (or call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` if you must swallow); move the method to another bean; name the right transaction manager (`@Transactional("ordersTx")`); keep the unit of work on one thread.
- **Prove it.** Print `TransactionSynchronizationManager.isActualTransactionActive()` inside the method; turn on `logging.level.org.springframework.transaction.interceptor=TRACE` (logs "Getting transaction for [...]" / "Completing transaction for [...]") and `logging.level.org.springframework.jdbc.support.JdbcTransactionManager=DEBUG` (or `...orm.jpa.JpaTransactionManager=DEBUG`), which logs "Initiating transaction rollback" or "Initiating transaction commit". Then write a test that asserts the row count after the failure (`RollbackTests`, `SelfInvocationTests`). → Q67, Q71, §7.10, §7.13

Scenario 4 — `LazyInitializationException`

- **Symptom.** *"failed to lazily initialize a collection of role: Order.items: could not initialize proxy - no Session"*, typically in a controller, a mapper or during JSON serialization.
- **Cause.** A lazy association was touched after the transaction — and with it the persistence context — had ended. It often appears after someone turns off Open Session in View, which had been hiding it.
- **Fix.** Load what the use case needs inside the service transaction: `JOIN FETCH` or `@EntityGraph` for that query, or a DTO projection; map entities to DTOs inside the transaction and return DTOs from controllers. Do **not** switch the association to EAGER (causes over-fetching and N+1 elsewhere) and don't re-enable OSIV or `hibernate.enable_lazy_load_no_trans`.
- **Prove it.** Reproduce it with a `@DataJpaTest` that loads the entity in one transaction and touches the collection outside it; after the fix the same test passes and the SQL log shows the association loaded by the one query. → Q82, §8.6, §8.15

Scenario 5 — N+1 queries make a list endpoint slow

- **Symptom.** `GET /authors` takes 2 s in production but 20 ms locally; the database shows hundreds of identical small `SELECT`s per request.
- **Cause.** One query for the list, then one query per row when the code (or Jackson) touches a lazy association — 1 + N round trips. Local data had 3 rows, so nobody noticed.
- **Fix.** For this use case, fetch with `JOIN FETCH` / `@EntityGraph` or query a DTO projection; as a global safety net set `spring.jpa.properties.hibernate.default_batch_fetch_size=16` (1 + N/16 queries). Beware fetching a collection together with pagination.
- **Prove it.** Count statements: enable Hibernate statistics (`hibernate.generate_statistics=true`) or a datasource proxy and assert the count in a `@DataJpaTest` (e.g. "exactly 1 statement for 50 authors"); in production, traces show one span per SQL statement under the request span. → Q83, §8.7

Scenario 6 — "I'm logged in but get 401", or 403 where 401 was expected

- **Symptom.** A client gets 401 or 403 and nobody knows which rule denied it.
- **Cause.** Remember: **401 = not authenticated** (no or invalid credentials — expired JWT, wrong issuer/audience, missing `Authorization` header, a chain that doesn't have the authentication mechanism you think), **403 = authenticated but not allowed**. Classic 403 causes: `hasRole("ADMIN")` while the user has authority `ADMIN` without the `ROLE_` prefix (or JWT scopes that became `SCOPE_x` authorities); a missing **CSRF token** on POST/PUT/DELETE in a session-based chain; a broader rule

(`anyRequest()`, `/api/**`) placed before the specific one; the request matched a different `SecurityFilterChain` than expected. An exception forwarded to `/error` that your rules don't permit can also turn a real 500 into a 401/403 — permit `/error`.

- **Fix.** Correct the authority mapping (`JwtAuthenticationConverter`, role prefix), send the CSRF token or disable CSRF only on the stateless token chain, reorder rules from specific to general, check each chain's `securityMatcher` and `@Order`.
- **Prove it.** `logging.level.org.springframework.security=TRACE` prints every filter the request passes and the authorization decision; print the current `Authentication` and its authorities in a debug endpoint or test; write `spring-security-test` tests for the allowed path and both denial paths (`jwt().authorities(...)`, `with(csrf())`). → Q103, Q104, §10.6, §10.12

Scenario 7 — CORS "blocked by CORS policy" in the browser

- **Symptom.** The browser console (the developer-tools panel that shows JavaScript errors) says *"has been blocked by CORS policy: Response to preflight request doesn't pass access control check"*; the same call works in Postman or curl (a GUI and a command-line tool for sending HTTP requests by hand).
- **Cause.** Curl ignores CORS; only browsers enforce it. Typical server-side causes: Spring Security runs before CORS handling and answers the credential-less `OPTIONS` preflight with 401 (CORS configured only in MVC, without `http.cors(...)`); the origin doesn't match exactly (scheme, host and port — `http://localhost:3000` ≠ `http://localhost:5173`); a requested header such as `Authorization` or a method such as `PATCH` isn't allowed; `allowCredentials(true)` combined with origin `*` (forbidden); both the gateway and the service add `Access-Control-Allow-Origin`, producing two values.
- **Fix.** One `CorsConfigurationSource` bean with the exact origins, methods and headers, plus `http.cors(withDefaults())` in the chain; CORS in only one layer (gateway *or* service); `allowedOriginPatterns` if you need patterns with credentials.
- **Prove it.** Replay the preflight: `curl -i -X OPTIONS https://api.example.com/api/notes -H "Origin: https://app.example.com" -H "Access-Control-Request-Method: POST" -H "Access-Control-Request-Headers: authorization,content-type"` — expect 200 and the `Access-Control-Allow-*` headers; the browser's Network tab shows the preflight's status. Lock it in with a `MockMvcTester` preflight test for an allowed and a refused origin. → Q54, §10.9, §5.10

Scenario 8 — `@Async` / `@Cacheable` / `@Transactional` "does nothing"

- **Symptom.** An `@Async` method runs on the request thread (the caller waits), a `@Cacheable` method runs every time, or a transaction is missing — with no error.
- **Cause.** The call never passed through the proxy: self-invocation (`this.method()`), a `private`/`final` method, an object created with `new`, or the `@Enable...` annotation is missing (`@EnableAsync`, `@EnableCaching`; `@EnableResilientMethods` for `@Retryable`).
- **Fix.** Move the annotated method to a separate bean and inject it; make it public and non-final; add the enabling annotation; or use a programmatic API (`TransactionTemplate`, an executor, `CacheManager`).
- **Prove it.** Print `AopUtils.isAopProxy(bean)` and `bean.getClass()` (look for `$$SpringCGLIB$$`); print `Thread.currentThread()` inside the `@Async` method; count calls of the cached method in a test; `isActualTransactionActive()` for transactions (Proof 5). → Q39, Q40, §4.5

Scenario 9 — the context fails to start because of auto-configuration

- **Symptom.** `APPLICATION FAILED TO START` with a failure-analyzer message such as *"Failed to configure a DataSource: 'url' attribute is not specified and no embedded datasource could be configured"* — right after adding `spring-boot-starter-data-jpa`; or a bean definition override error; or a `ClassNotFoundException`/`NoSuchMethodError` (the JVM could not find a class / a method that the compiled code expects — a sign of mismatched library versions) from inside an auto-configuration.
- **Cause.** Auto-configuration reacts to the **classpath**: adding a starter activates its configuration, which now needs settings (a JDBC URL) or a driver. Other causes: two libraries define beans with the same name (overriding is disabled by default); mixed versions because a dependency has a hard-coded `<version>` that fights Boot's BOM, or a Boot 3 library on Boot 4 (old auto-configuration packages, Jackson 2 vs 3, `javax` vs `jakarta`).
- **Fix.** Provide the missing configuration (URL, driver, or an embedded database for tests); if the feature isn't wanted, remove the starter or exclude that auto-configuration; remove explicit versions and let the BOM manage them; upgrade libraries to Boot 4-compatible versions (Spring Cloud 2025.1, Spring AI 2.0, springdoc for Boot 4).
- **Prove it.** Read the *Description* and *Action* sections of the failure report; run with `--debug` for the condition report; `mvn dependency:tree -Dincludes=<group>` shows which artifact pulled in the conflicting version; `ApplicationContextRunner` with `AutoConfigurations.of(...)` reproduces a single auto-configuration in a test. → Q24–Q26, §3.11, §3.12

Scenario 10 — slow startup

- **Symptom.** "Started OrderApplication in 48 seconds"; pods fail their startup probe (the Kubernetes check that gives a slow-starting app time before liveness checks begin); the test suite takes minutes.
- **Cause.** Usually a few expensive beans, not Spring itself: work in `@PostConstruct` or constructors (warming caches, calling remote services, loading files), remote calls that hang until a timeout because a dependency isn't reachable, large Flyway migrations or Hibernate schema validation, very broad component scanning; in tests, many different context configurations defeating the context cache.
- **Fix.** Move slow warm-up out of bean creation (an `ApplicationReadyEvent` listener or a background task, with readiness reporting "not ready" until done); set connect timeouts; run migrations as a separate job for very large schemas; narrow scanning; standardize test configurations. Then consider lazy initialization (with care), and the JDK AOT cache/CDS or a native image for the remaining JVM start cost.
- **Prove it.** Record startup steps: `SpringApplication app = ...; app.setApplicationStartup(new BufferingApplicationStartup(2048));` and call `/actuator/startup`, then sort steps by duration — the slow bean is named; or use `FlightRecorderApplicationStartup` with JFR (Java Flight Recorder, the JVM's built-in low-overhead profiler). Compare the "Started ... in X seconds" line before and

after. For tests, enable `logging.level.org.springframework.test.context.cache=DEBUG` to see context-cache hits and misses. → Q16, Q119, Q150, §2.10, §15.11, §12.11

Self-check

1. Postman works, the browser doesn't. First suspicion? → CORS — most often the preflight hitting security (401) or an origin mismatch.
2. Which log category tells you why a request got 403? → `org.springframework.security` at TRACE.
3. What is the quickest proof that N+1 is fixed? → A test asserting the SQL statement count.
4. A new starter made startup fail. Where do you look first? → The failure analyzer's Description/Action, then the condition report (`--debug`).

Chapter TL;DR

- **Container:** beans are singletons by default and shared across threads; type → `@Qualifier` → `@Primary` → `@Fallback` → parameter name decides injection; a prototype inside a singleton is created once (use `ObjectProvider`); constructor cycles never work.
- **Boot:** auto-configuration = `.imports` list + conditions + `@ConditionalOnMissingBean`; `--debug` explains every decision; outside/later property sources win; Boot 4 is modular with renamed starters.
- **Proxies:** `@Transactional`, `@Cacheable`, `@Async`, `@Retryable`, `@PreAuthorize` all work only through the proxy — self-invocation and `private` methods skip them.
- **Transactions:** REQUIRED + database-default isolation; checked exceptions commit unless told otherwise; caught exceptions never roll back; keep one business operation per transaction and no remote calls inside it.
- **JPA:** persistence context + dirty checking; everything LAZY, fetch per use case; fix N+1 with fetch joins, entity graphs or batch size; OSIV off; `@Version` for concurrent edits; migrations own the schema.
- **Security 7:** filter chain → authentication → `SecurityContextHolder` → authorization; 401 = who are you, 403 = not allowed; lambda DSL only; CSRF matters whenever the browser sends credentials automatically; JWT resource servers validate locally.
- **Testing:** unit → slices → a few `@SpringBootTest`s; `@MockitoBean` (not `@MockBean`) in Boot 4; keep context configurations few so caching works.
- **Production:** Actuator with probes, Micrometer metrics and traces, structured logs, graceful shutdown, virtual threads for blocking I/O; outbox + idempotent consumers for messaging.
- **Debugging:** symptom → mechanism → smallest fix → proof with a log, a test or a metric.

Next: Chapter 19 — Cheat sheet and glossary collects the annotations, properties, starters and terms on a few pages.

Chapter 19 — Cheat Sheet and Glossary

This chapter is the book on one table. Chapters 1–18 teach each topic with runnable projects; this chapter **looks things up**. Any word you meet here and don't know — starter, BOM, proxy, propagation, slice — has a plain-English entry in the A–Z glossary at the end (§19.16). You come here when you remember that something exists but not its exact name, default or package: "which starter brings `@WebMvcTest` in Boot 4?", "what does `REQUIRES_NEW` do when the caller rolls back?", "which property turns on virtual threads?", "is it `hasRole("ADMIN")` or `hasRole("ROLE_ADMIN")`?".

Every row links to the section that explains it, so a sheet is never the end of the road: if a line is not enough, follow the **§** link. Nothing here is new material; where a sheet quotes a default or a behaviour, a test in `code/cheatsheet-checks/` proves it (§19.15).

What you'll find

1. Versions of the Boot 4.1.1 stack at a glance (§19.2).
2. **Annotations**: every common one → module / starter → purpose → section (§19.3).
3. **Starters** in Boot 4.1, including renamed, deprecated and removed ones (§19.4).
4. **Properties** with their defaults: web, data, cache, security, actuator, observability, Kafka, threads, HTTP clients, API versioning (§19.5).
5. `@Transactional`: attributes, **propagation**, isolation and **rollback** tables (§19.6).
6. **Bean scopes** and the **lifecycle order** (§19.7).
7. **Spring Security 7 DSL** snippets that compile: form login, JWT resource server, method security, CORS/CSRF (§19.8).
8. **Testing** annotations and slices (§19.9), **Reactor** operators (§19.10), **Actuator** endpoints (§19.11).
9. **Which tool for which job** decision tables (§19.12) and a **Boot 3 → 4 checklist** (§19.13).
10. An **interview index** into Chapter 18 (§19.14), the **proofs** (§19.15) and an **A–Z glossary** of more than 500 terms (§19.16).

Versions: Spring Boot **4.1.1** → Spring Framework **7.0.9**, Spring Security **7.1.1**, Java **25** (17 minimum). "(Boot 4)", "(F7)" and "(S7)" mark things that are new or changed in Boot 4, Framework 7 or Security 7.

19.1 How to use this chapter

How the sheets are organised

You want to...	Go to
find the name, jar or purpose of an annotation	§19.3
add a dependency	§19.4
set or look up a property and its default	§19.5
pick a propagation / isolation / rollback rule	§19.6
remember the bean lifecycle order	§19.7
copy a security configuration	§19.8
choose a test annotation	§19.9
find a Reactor operator	§19.10
know what an Actuator endpoint shows	§19.11
choose between two technologies	§19.12
upgrade a Boot 3 app	§19.13
revise for an interview	§19.14
look up a word	§19.16

Conventions

- **Module / starter**: the jar (Java archive — a zip file of compiled classes) that contains the class, and the Boot starter that brings it. "core" means `spring-context`/`spring-beans`, which every starter includes.
- **§N.M** links go to the teaching section of Chapters 1–18. Follow them for the explanation, the trade-offs and the proof.
- Defaults quoted in these sheets come from the Boot 4.1.1 configuration metadata (the `spring-configuration-metadata.json` files inside the Boot jars) or from the chapter that measured them. The ones marked ✔ **proved** are asserted by a test in §19.15.
- Properties are written in `.properties` form. The same keys work in YAML (an indentation-based configuration format) and as environment variables (named values set outside the app by the shell, OS or container) (§3.4).

19.2 Versions at a glance

What `spring-boot-starter-parent` **4.1.1** puts on your classpath — the list of jars the JVM loads classes from — (from the `spring-boot-dependencies-4.1.1` BOM — bill of materials, a list that pins compatible versions of many libraries). You choose one number, the Boot version, and Boot chooses the rest (§1.4).

Component	Version	Notes
Java	17 minimum, 25 recommended	this book: Java 25
Spring Framework	7.0.9	Jakarta EE 11 (Servlet 6.1, JPA 3.2, Bean Validation 3.1)
Spring Security	7.1.1	Authorization Server now inside Spring Security (§11.14)
Spring Data	2026.0.1 release train	JPA, JDBC, MongoDB, Redis, R2DBC, REST
Hibernate ORM / Validator	7.4.5 / 9.1.3	
Jackson	3.1.5 (`tools.jackson.*`)	annotations still `com.fasterxml.jackson.annotation` (§5.5)
Tomcat / Jetty	11.0.24 / 12.1.12	Undertow removed
HikariCP / H2 / PostgreSQL JDBC	7.0.2 / 2.4.240 / 42.7.13	
Reactor	2025.0.7 train (Reactor Core 3.8)	
Micrometer / Micrometer Tracing / OpenTelemetry	1.17.1 / 1.7.1 / 1.62.0	
Spring for Apache Kafka / kafka-clients	4.1.1 / 4.2.1	
Spring AMQP	4.1.1	
Flyway / Liquibase	12.4.0 / 5.0.3	Liquibase 5 Community is FSL-licensed (§9.4)
JUnit / Mockito / AssertJ / Awaitility	6.0.3 / 5.23.0 / 3.27.7 / 4.3.0	
Testcontainers	2.0.5	new artifact names `testcontainers-<module>` (§12.14)
Caffeine / Lettuce	3.2.4 / 7.5.2	
Not managed by Boot		import their own BOM
Spring Cloud	2025.1.x "Oakwood" (2025.1.2+ for Boot 4.1)	§16.2
Spring AI	2.0.x (2.0.1)	§17.1
springdoc-openapi	3.x (3.1.1)	§5.12
Resilience4j	`resilience4j-spring-boot4` 2.4.0	§16.8

19.3 Annotations cheat sheet

The annotations of this book, grouped by area. "Needs" names the switch without which the annotation silently does nothing — the most common reason an annotation "doesn't work".

Core container and beans

Annotation	Module / starter	Purpose	§
`@Component`	core	generic bean found by component scanning	§2.3
`@Service`	core	`@Component` for business logic (no extra behaviour)	§2.3
`@Repository`	core	`@Component` for data access + exception translation to `DataAccessException`	§2.3, §7.5
`@Controller` / `@RestController`	spring-web / `-webmvc`, `-webflux`	web handler; `@RestController` = `@Controller` + `@ResponseBody`	§5.3
`@Configuration`	core	class of `@Bean` methods; full mode = CGLIB subclass (`proxyBeanMethods = true`)	§2.3
`@Bean`	core	factory method registering its return value as a bean; name = method name	§2.3
`@ComponentScan`	core	scan a package tree for components (inside `@SpringBootApplication`)	§1.8
`@Import`	core	pull in another configuration class, selector, registrar or `BeanRegistrar`	§2.3
`@Autowired`	core	inject here; optional on a single constructor	§2.4
`@Qualifier`	core	pick a bean by name/qualifier at the injection point (beats `@Primary`) ✔	§2.5
`@Primary`	core	default bean when several match ✔	§2.5
`@Fallback`	core (Framework 6.2)	use this bean only if no other candidate qualifies	§2.5
`@Order`	core	order in `List<T>` injection, aspects, filters, listeners (lower first) ✔	§2.5
`@Value`	core	inject one value: `${key:default}`, SpEL `#{...}` ✔	§3.5
`@Scope`	core	`singleton` (default), `prototype`, web scopes; `proxyMode` for scoped proxies	§2.6
`@RequestScope` / `@SessionScope` / `@ApplicationScope`	spring-web	web scopes with a CGLIB scoped proxy	§2.6
`@Lookup`	spring-beans	method injection: returns a fresh prototype on each call	§2.6
`@Lazy`	core	create on first use; on an injection point, inject a lazy proxy (breaks cycles)	§2.10, §2.9
`@PostConstruct` / `@PreDestroy`	`jakarta.annotation`	init / destroy callbacks (not for prototypes' destroy) ✔	§2.7
`@Profile`	core	register only when a profile expression matches (`"!dev"`, `"prod & eu"`)	§2.11
`@Conditional`	core	register only if a `Condition` matches	§2.11
`@EventListener`	core	receive application events (synchronous by default)	§2.12
`@Proxyable`	core (F7)	choose JDK or CGLIB proxy for one bean	§4.4
`@NullMarked` / `@Nullable`	`org.jspecify.annotations` (F7)	null-safety contract; `@Nullable` injection point = optional	§2.13
`@Inject`	`jakarta.inject` (optional jar)	standard alternative to `@Autowired`; `javax.inject` removed in F7	§2.4

Spring Boot and configuration

Annotation	Module / starter	Purpose	§
`@SpringBootApplication`	spring-boot-autoconfigure / `spring-boot-starter`	`@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`	§1.8
`@EnableAutoConfiguration`	spring-boot-autoconfigure	load `AutoConfiguration.imports` candidates and apply them after your beans	§3.10
`@AutoConfiguration`	spring-boot-autoconfigure	marks an auto-configuration class (`before` / `after` ordering)	§3.13
`@ConfigurationProperties("prefix")`	spring-boot	bind a group of properties to a typed object (record ideal)	§3.6
`@ConfigurationPropertiesScan` / `@EnableConfigurationProperties`	spring-boot	register `@ConfigurationProperties` classes	§3.6
`@DefaultValue`	spring-boot	default for a constructor-bound property ✔	§3.6
`@ConstructorBinding`	spring-boot	pick the binding constructor when there are several	§3.6
`@DurationUnit` / `@DataSizeUnit`	spring-boot	unit for plain numbers (default ms / bytes) ✔	§3.6
`@Validated` (on properties)	core + `spring-boot-starter-validation`	validate bound properties at startup	§3.6
`@ConditionalOnClass` / `OnMissingClass`	spring-boot-autoconfigure	class present / absent on the classpath	§3.10
`@ConditionalOnBean` / `OnMissingBean`	spring-boot-autoconfigure	bean present / absent (reliable in auto-configurations only)	§3.10
`@ConditionalOnProperty` / `OnBooleanProperty`	spring-boot-autoconfigure	property value / boolean switch (`OnBooleanProperty` since Boot 3.5)	§3.10
`@ConditionalOnWebApplication` / `OnThreading` / `OnCloudPlatform` / ...	spring-boot-autoconfigure	web type, virtual vs platform threads, platform	§3.10
`@PropertySource`	core	add a `.properties` file (low precedence, no YAML)	§3.9
`@EntityScan`	spring-boot-persistence (Boot 4 package)	where to find JPA entities outside the main package	§17.13

AOP and resilience

Annotation	Module / starter	Purpose	§
`@Aspect` (+ `@Component`)	aspectjweaver / `spring-boot-starter-aspectj` (Boot 4)	a class of pointcuts and advice; must also be a bean	§4.2
`@Pointcut`	aspectjweaver	named, reusable pointcut expression	§4.3
`@Before` / `@AfterReturning` / `@AfterThrowing` / `@After` / `@Around`	aspectjweaver	the five advice types; only `@Around` controls `proceed()`	§4.2
`@EnableAspectJAutoProxy`	core	switch on `@Aspect` support (Boot does it: `spring.aop.auto=true`)	§4.2
`@Retryable`	core `org.springframework.resilience` (F7)	retry on exception; `maxRetries` 3, `delay` 1000 ms, `multiplier` 1.0 ✔ — **needs `@EnableResilientMethods`**	§4.7
`@ConcurrencyLimit`	core (F7)	bulkhead: max concurrent calls, policy `BLOCK` or `REJECT` — needs `@EnableResilientMethods`	§4.7
`@EnableResilientMethods`	core (F7)	turns on `@Retryable` and `@ConcurrencyLimit` (no auto-configuration)	§4.7
`@CircuitBreaker` / `@Retry` / `@RateLimiter` / `@Bulkhead` / `@TimeLimiter`	`resilience4j-spring-boot4`	Resilience4j patterns with `fallbackMethod`	§16.8

Web: Spring MVC, WebFlux, REST

Annotation	Module / starter	Purpose	§
`@RequestMapping`	spring-web	class-level prefix / shared mapping attributes	§5.3
`@GetMapping` / `@PostMapping` / `@PutMapping` / `@PatchMapping` / `@DeleteMapping`	spring-web	map an HTTP method + path; `version = "1.1+"` (F7)	§5.3, §5.9
`@PathVariable`	spring-web	URL segment `{id}`	§5.4
`@RequestParam`	spring-web	query parameter / form field (`required = true` by default)	§5.4
`@RequestHeader` / `@CookieValue`	spring-web	header / cookie value	§5.4
`@RequestBody`	spring-web	body converted by an `HttpMessageConverter` (Jackson 3)	§5.4
`@RequestPart`	spring-web	part of a `multipart/form-data` request (`MultipartFile`)	§5.10
`@ModelAttribute`	spring-web	bind query params/form fields to an object	§5.4
`@ResponseStatus`	spring-web	fixed status for a handler or an exception class	§5.4, §5.7
`@ResponseBody`	spring-web	write the return value as the body	§5.3
`@ExceptionHandler`	spring-web	handle an exception type (in a controller or an advice)	§5.7
`@ControllerAdvice` / `@RestControllerAdvice`	spring-web	global exception handling (return `ProblemDetail`)	§5.7
`@Valid`	`jakarta.validation` + `-validation` starter	cascade validation into a body / nested object	§5.6
`@Validated`	core	validation **groups**; on a class: method validation via proxy	§5.6
`@CrossOrigin`	spring-web	per-controller CORS rules	§5.10
`@JsonProperty` / `@JsonIgnore`	`com.fasterxml.jackson.annotation`	JSON names / hidden fields (package unchanged in Jackson 3)	§5.5
`@JacksonComponent` / `@JacksonMixin`	spring-boot-jackson (Boot 4)	register (de)serializers / mix-ins (were `@JsonComponent` / `@JsonMixin`)	§5.5
`@EnableWebMvc`	spring-webmvc	full manual MVC control — **switches Boot's MVC auto-configuration off**	§3.12

HTTP clients

Annotation	Module / starter	Purpose	§
`@HttpExchange`	spring-web	HTTP service interface: shared URL prefix, headers, version	§6.6
`@GetExchange` / `@PostExchange` / `@PutExchange` / `@PatchExchange` / `@DeleteExchange`	spring-web	one HTTP call per interface method	§6.6
`@ImportHttpServices(group, types)`	spring-web (F7) + `spring-boot-starter-restclient`	register HTTP interface proxies per group, configured by `spring.http.serviceclient.<group>.*`	§6.6
`@ClientRegistrationId`	spring-security-oauth2-client	which OAuth2 registration's token to add (type-level since S7)	§11.10
`@LoadBalanced`	spring-cloud-commons	resolve `http://service-id` through Spring Cloud LoadBalancer	§16.5
`@FeignClient` / `@EnableFeignClients`	spring-cloud-openfeign (maintenance mode)	declarative client, Feign style	§16.6

Data access, transactions, JPA and Spring Data

Annotation	Module / starter	Purpose	§
`@Transactional`	spring-tx	declarative transaction (see §19.6) ✔	§7.8
`@EnableTransactionManagement`	spring-tx	switch on `@Transactional` (Boot does it); `rollbackOn = ALL_EXCEPTIONS` (F6.2)	§7.10
`@TransactionalEventListener`	spring-tx	run a listener at `AFTER_COMMIT` (default), `BEFORE_COMMIT`, `AFTER_ROLLBACK`, `AFTER_COMPLETION`	§7.15
`@Entity` / `@Table` / `@Column`	`jakarta.persistence` / `-data-jpa`	map a class / table / column	§8.3
`@Id` / `@GeneratedValue`	`jakarta.persistence`	primary key and strategy (`IDENTITY`, `SEQUENCE`, `UUID`, `AUTO`)	§8.3
`@ManyToOne` / `@OneToMany` / `@OneToOne` / `@ManyToMany`	`jakarta.persistence`	relationships; to-one EAGER, to-many LAZY by default	§8.5
`@JoinColumn` / `mappedBy`	`jakarta.persistence`	owning side (FK) / inverse side	§8.5
`@Enumerated(EnumType.STRING)` / `@Embedded` / `@Embeddable` / `@Transient`	`jakarta.persistence`	enum as text / value object / not stored	§8.3
`@Version`	`jakarta.persistence` (JPA) or `org.springframework.data.annotation` (Data JDBC, Mongo)	optimistic locking column	§8.14, §9.2
`@Query`	spring-data-jpa / -jdbc / ...	JPQL (JPA), SQL (JDBC), `nativeQuery = true`	§8.10
`@Modifying`	spring-data-jpa / -jdbc	UPDATE/DELETE query; `clearAutomatically`	§8.10
`@Param`	spring-data-commons	name a query parameter (optional with `-parameters`)	§8.10
`@EntityGraph(attributePaths = ...)`	spring-data-jpa	fetch associations in the same query (left join)	§8.7
`@Lock(LockModeType.PESSIMISTIC_WRITE)`	spring-data-jpa	`SELECT ... FOR UPDATE`	§8.14
`@BatchSize`	Hibernate	batch-load lazy collections (N+1 fix)	§8.7
`@DynamicUpdate`	Hibernate	UPDATE only changed columns	§8.4
`@EnableJpaAuditing` / `@EnableJdbcAuditing`	spring-data-jpa / -jdbc	switch on auditing	§8.13, §9.2
`@CreatedDate` / `@LastModifiedDate` / `@CreatedBy` / `@LastModifiedBy`	spring-data-commons	audit columns	§8.13
`@MappedCollection(idColumn, keyColumn)`	spring-data-relational	child collection of a Data JDBC aggregate	§9.2
`@RepositoryRestResource`	spring-data-rest	export (or hide) a repository as HAL endpoints	§9.3
`@Document` / `@Indexed`	spring-data-mongodb	MongoDB document / index	§9.5
`@RedisHash(timeToLive = ...)`	spring-data-redis	Redis repository entity with TTL	§9.6

Caching

Annotation	Module / starter	Purpose	§
`@EnableCaching`	core / `spring-boot-starter-cache`	switch on the cache proxy — without it nothing is cached	§9.8
`@Cacheable`	core	return the cached value; run the method only on a miss (`sync`, `condition`, `unless`)	§9.8
`@CachePut`	core	always run, then store the result	§9.8
`@CacheEvict`	core	remove a key (`allEntries = true` to clear)	§9.8
`@Caching` / `@CacheConfig`	core	several cache ops on one method / class defaults	§9.8

Security

Annotation	Module / starter	Purpose	§
`@EnableWebSecurity`	spring-security-config	web security (Boot applies it for you)	§10.3
`@EnableMethodSecurity`	spring-security-config	switch on method security annotations	§10.7
`@PreAuthorize` / `@PostAuthorize`	spring-security-core	SpEL rule before / after the call (`returnObject`)	§10.7
`@PreFilter` / `@PostFilter`	spring-security-core	filter a collection argument / result	§10.7
`@Secured` / `@RolesAllowed`	spring-security-core / `jakarta.annotation.security`	legacy role checks (`securedEnabled`, `jsr250Enabled`)	§10.7
`@AuthenticationPrincipal`	spring-security-core	inject the principal (`UserDetails`, `Jwt`, `OidcUser`)	§10.4
`@EnableMultiFactorAuthentication`	spring-security-config (S7)	require several `FACTOR_*` authorities	§10.4

Messaging, async and scheduling

Annotation	Module / starter	Purpose	§
`@EnableAsync`	core	switch on `@Async` (Boot configures the executor but does **not** enable it)	§14.4
`@Async("executor")`	core	run on an executor; return `void` or `CompletableFuture`	§14.4
`@EnableScheduling`	core	switch on `@Scheduled`	§14.7
`@Scheduled(fixedRate / fixedDelay / cron)`	core	timer jobs; cron has **six** fields ✔	§14.7
`@KafkaListener(topics, groupId, concurrency)`	spring-kafka / `spring-boot-starter-kafka`	Kafka consumer method	§14.12
`@RetryableTopic` / `@DltHandler`	spring-kafka	non-blocking retries via retry topics; dead-letter handler	§14.15
`@RabbitListener`	spring-rabbit / `spring-boot-starter-amqp`	RabbitMQ consumer method	§14.18
`@JmsListener`	spring-jms	JMS consumer method	§14.18
`@SchedulerLock`	ShedLock (third party)	run a `@Scheduled` job on one instance only	§14.7

Observability, cloud and AI

Annotation	Module / starter	Purpose	§
`@Observed`	micrometer-observation (+ AspectJ starter)	timer + span around a method; needs `management.observations.annotations.enabled=true`	§15.6
`@Timed` / `@Counted` / `@NewSpan`	Micrometer	metric / span annotations (same switch as `@Observed`)	§15.6
`@EnableConfigServer`	spring-cloud-config-server	make the app a Config Server	§16.3
`@RefreshScope`	spring-cloud-context	re-create the bean after a config refresh	§16.3
`@EnableEurekaServer`	spring-cloud-netflix-eureka-server	run a Eureka registry	§16.4
`@Tool` / `@ToolParam`	spring-ai-model (Spring AI 2.0)	expose a method to the model for tool calling	§17.8
`@McpTool` / `@McpToolParam`	Spring AI 2.0 MCP (`org.springframework.ai.mcp.annotation`)	expose a tool from an MCP server	§17.10

Testing annotations have their own sheet: §19.9.

19.4 Starters in Boot 4.1

A **starter** is a code-free dependency that brings one capability's libraries plus the Boot module holding its auto-configuration (§1.7). Boot 4 made them **modular**: one starter per technology, and a matching `-test` starter that brings the test slice (a test that starts only one layer of the app, such as the web layer) for it and already includes `spring-boot-starter-test` (§3.14, §12.3). All names below are managed by the 4.1.1 BOM — never add a `<version>`.

The starters you will use most

Capability	Starter	Test starter (slice, test clients)	§
Core only (no web)	`spring-boot-starter`	`spring-boot-starter-test`	§1.7
Servlet web / REST (Spring MVC + Tomcat + Jackson 3)	`spring-boot-starter-webmvc`	`spring-boot-starter-webmvc-test` (`@WebMvcTest`, MockMvc, `RestTestClient`)	§5.2
Reactive web (WebFlux + Reactor Netty)	`spring-boot-starter-webflux`	`spring-boot-starter-webflux-test` (`@WebFluxTest`, `WebTestClient`)	§13.10
Blocking HTTP client (`RestClient.Builder`, HTTP service groups)	`spring-boot-starter-restclient`	`spring-boot-starter-restclient-test` (`@RestClientTest`)	§6.1
Reactive HTTP client (`WebClient.Builder`)	`spring-boot-starter-webclient`	`spring-boot-starter-webclient-test`	§13.13
JSON (Jackson 3)	`spring-boot-starter-jackson`	`spring-boot-starter-jackson-test` (`@JsonTest`)	§5.5
Bean Validation (Hibernate Validator)	`spring-boot-starter-validation`	`spring-boot-starter-validation-test`	§5.6
AOP with `@Aspect`	`spring-boot-starter-aspectj`	`spring-boot-starter-aspectj-test`	§4.2
JDBC (`DataSource`, HikariCP, `JdbcClient`)	`spring-boot-starter-jdbc`	`spring-boot-starter-jdbc-test` (`@JdbcTest`)	§7.2
JPA (Hibernate + Spring Data JPA)	`spring-boot-starter-data-jpa`	`spring-boot-starter-data-jpa-test` (`@DataJpaTest`)	§8.2
Spring Data JDBC	`spring-boot-starter-data-jdbc`	`spring-boot-starter-data-jdbc-test` (`@DataJdbcTest`)	§9.2
R2DBC	`spring-boot-starter-data-r2dbc`	`spring-boot-starter-data-r2dbc-test` (`@DataR2dbcTest`)	§13.14
MongoDB / Redis	`spring-boot-starter-data-mongodb` / `-data-redis`	`...-data-mongodb-test` / `...-data-redis-test`	§9.5, §9.6
Spring Data REST	`spring-boot-starter-data-rest`	`spring-boot-starter-data-rest-test`	§9.3
Schema migrations	`spring-boot-starter-flyway` / `-liquibase`	`...-flyway-test` / `...-liquibase-test`	§8.16, §9.4
Cache abstraction	`spring-boot-starter-cache` (+ `caffeine`)	`spring-boot-starter-cache-test`	§9.8
Security	`spring-boot-starter-security`	`spring-boot-starter-security-test`	§10.1
OAuth2 resource server / client / authorization server	`spring-boot-starter-security-oauth2-resource-server` / `-client` / `-authorization-server`	`...-test` for each	§11.3, §11.9, §11.14
Kafka	`spring-boot-starter-kafka` (+ `-jackson` for JSON)	`spring-boot-starter-kafka-test` (`@EmbeddedKafka`)	§14.10
RabbitMQ	`spring-boot-starter-amqp`	`spring-boot-starter-amqp-test`	§14.18
Actuator	`spring-boot-starter-actuator`	`spring-boot-starter-actuator-test`	§15.2
Tracing + OTLP export	`spring-boot-starter-opentelemetry` (or `-zipkin` for Brave)	`spring-boot-starter-opentelemetry-test` (`@AutoConfigureTracing`)	§15.7
Metrics only	`spring-boot-starter-micrometer-metrics` (+ a registry, e.g. `micrometer-registry-prometheus`)	`...-micrometer-metrics-test` (`@AutoConfigureMetrics`)	§15.5
HATEOAS	`spring-boot-starter-hateoas`	`...-hateoas-test`	§5.12
Testcontainers support	`spring-boot-testcontainers` (module, test scope)	—	§12.14
Also available	`-websocket`, `-graphql`, `-grpc-server` / `-grpc-client` (Boot 4.1), `-batch`, `-batch-jdbc`, `-integration`, `-quartz`, `-mail`, `-jms`, `-artemis`, `-pulsar`, `-thymeleaf`, `-session-data-redis`, `-session-jdbc`, `-jooq`, `-jetty`	most have a `-test` companion	§1.3

Renamed, deprecated and removed starters

Boot 3 name	Boot 4.1 status	Use instead	§
`spring-boot-starter-web`	**deprecated** alias (still in the BOM)	`spring-boot-starter-webmvc`	§1.7
`spring-boot-starter-web-services`	**deprecated** alias	`spring-boot-starter-webservices`	§17.13
`spring-boot-starter-oauth2-resource-server` / `-oauth2-client` / `-oauth2-authorization-server`	**deprecated** aliases	`spring-boot-starter-security-oauth2-...`	§11.3
`spring-boot-starter-aop`	**removed**	`spring-boot-starter-aspectj`	§4.2
`spring-boot-starter-undertow`	**removed** (Undertow lacks Servlet 6.1)	Tomcat (default) or `spring-boot-starter-jetty`	§17.13
`spring-boot-starter-json`	still available	`spring-boot-starter-jackson` is the Boot 4 name	§1.7
`flyway-core` / `liquibase-core` alone	no longer auto-configured	`spring-boot-starter-flyway` / `-liquibase`	§8.16
`spring-kafka` alone	still works as a library, but use the starter	`spring-boot-starter-kafka`	§14.10
part of `-web` / `-webflux` in Boot 3	split out	`spring-boot-starter-restclient`, `spring-boot-starter-webclient`	§6.1, §13.10
`spring-boot-starter-batch` (with JDBC metadata)	batch now runs without a database by default	`spring-boot-starter-batch-jdbc`	§17.13
(migration aid)	new	`spring-boot-starter-classic`, `spring-boot-starter-test-classic`: the old "everything" layout while you migrate	§3.14

Third-party starters are named `<name>-spring-boot-starter` (`mybatis-spring-boot-starter`); the `spring-boot-*` prefix is reserved (§3.13).

19.5 Key properties cheat sheet

Defaults are the Boot 4.1.1 metadata values; "—" means *no default: the feature is off, or the value comes from somewhere else*. Precedence (which source wins) and relaxed binding (several spellings of one property name, e.g. `SHOP_NAME` for `shop.name`, are accepted) are at the end of this section.

Core, profiles and startup

Property	Default	What it does	§
`spring.application.name`	—	name in logs, traces, Config Server, metrics	§1.9
`spring.profiles.active` / `.default` / `.include`	— / `default` / —	activate profiles (not allowed inside profile-specific files)	§3.7
`spring.profiles.group.<name>`	—	one name activates several profiles	§3.7
`spring.config.import`	—	extra files, `optional:`, `configtree:`, `configserver:`	§3.8
`spring.config.activate.on-profile`	—	make a document of a multi-document file conditional	§3.2
`spring.autoconfigure.exclude`	—	switch one auto-configuration off	§3.12
`debug`	`false`	print the condition evaluation report	§3.11
`spring.main.lazy-initialization`	`false`	make every bean lazy	§2.10
`spring.main.allow-circular-references`	`false`	Boot rejects field/setter cycles (since 2.6)	§2.9
`spring.main.allow-bean-definition-overriding`	`false`	two beans with one name → `BeanDefinitionOverrideException`	§2.3
`spring.main.web-application-type`	detected	`servlet`, `reactive`, `none` (MVC wins if both starters present)	§13.10
`spring.main.keep-alive`	`false`	keep the JVM alive when only virtual (daemon) threads run	§14.5
`spring.aop.proxy-target-class`	`true`	Boot uses CGLIB proxies for everything	§4.4
`spring.aop.auto`	`true`	`@EnableAspectJAutoProxy` applied by Boot	§4.2
`spring.lifecycle.timeout-per-shutdown-phase`	`30s` ✔	how long graceful shutdown waits	§15.9
`spring.devtools.restart.enabled`	`true`	DevTools restart (off for `java -jar` and tests)	§1.10
`spring.devtools.livereload.enabled`	`false`	LiveReload — off in Boot 4.0, deprecated in 4.1	§1.10

Web server, MVC and JSON

Property	Default	What it does	§
`server.port`	`8080`	HTTP port; `0` = random free port	§1.9
`server.shutdown`	`graceful`	`immediate` turns graceful shutdown off (default since Boot 3.4)	§15.9
`server.servlet.context-path`	—	URL prefix of the whole app	§5.2
`server.forward-headers-strategy`	—	`framework` honours `X-Forwarded-*` behind a proxy	§5.4
`server.servlet.session.timeout`	`30m`	HTTP session timeout	§10.10
`server.servlet.session.cookie.secure` / `.http-only` / `.same-site`	—	harden the session cookie	§10.10
`spring.web.error.include-stacktrace`	`never`	stack traces in Boot's error body (Boot 4 name; `server.error.*` no longer bound)	§15.12
`spring.mvc.problemdetails.enabled`	`false`	RFC 9457 bodies for MVC exceptions without your own advice	§5.7
`spring.webflux.problemdetails.enabled`	`false`	the same for WebFlux	§13.10
`spring.mvc.contentnegotiation.favor-parameter`	`false`	allow `?format=csv`	§5.8
`spring.servlet.multipart.max-file-size` / `max-request-size`	`1MB` / `10MB`	upload limits (413 when exceeded)	§5.10
`spring.jackson.default-property-inclusion`	—	e.g. `non_null`	§5.5
`spring.jackson.serialization.*` / `deserialization.*`	—	Jackson feature switches (`indent-output`...)	§5.5
`spring.jackson.json.read.*` / `spring.jackson.json.write.*`	—	JSON parser/generator features (Boot 4; were `spring.jackson.parser/generator.*`)	§5.5
`spring.jackson.use-jackson2-defaults`	`false`	restore Jackson 2 defaults while migrating	§17.13
`spring.threads.virtual.enabled`	`false`	virtual threads for Tomcat, `@Async`, `@Scheduled`, listeners (Java 21+) ✔	§15.10

Jackson 3 defaults under Boot 4 (✔ proved): `LocalDate` written as `"2026-09-28"`, unknown JSON properties ignored, `JacksonException` is unchecked (a `RuntimeException`, so callers need not catch it) (§5.5).

API versioning (F7 / Boot 4)

Property	Meaning	§
`spring.mvc.apiversion.use.header`	read the version from this header (e.g. `X-API-Version`)	§5.9
`spring.mvc.apiversion.use.query-parameter` / `.use.path-segment` / `.use.media-type-parameter`	other places to read it	§5.9
`spring.mvc.apiversion.default`	version assumed when the request has none	§5.9
`spring.mvc.apiversion.required`	reject requests without a version (400)	§5.9
`spring.mvc.apiversion.supported` / `.detect-supported`	extra supported versions / detect them from mappings	§5.9
`spring.webflux.apiversion.*`	the same for WebFlux	§17.13
`spring.http.serviceclient.<group>.apiversion.insert.header`	client side: send the version in this header	§6.7

HTTP clients

Property	Default	What it does	§
`spring.http.clients.connect-timeout` / `read-timeout`	— (library default)	global timeouts for auto-configured clients (Boot 4 name; `spring.http.client.*` deprecated)	§6.4
`spring.http.clients.imperative.factory`	detected	`http-components`, `jetty`, `reactor`, `jdk`, `simple`	§6.4
`spring.http.clients.reactive.connector`	detected	the same for `WebClient`	§6.4
`spring.http.clients.redirects`	—	`follow`, `dont-follow`, `follow-when-possible`	§6.4
`spring.http.serviceclient.<group>.base-url`	—	base URL of an `@ImportHttpServices` group (none + Spring Cloud = load-balanced)	§6.6, §16.6
`spring.http.serviceclient.<group>.read-timeout` / `.default-header.<name>`	—	per-group timeout / header	§6.6

DataSource, JDBC, transactions

Property	Default	What it does	§
`spring.datasource.url` / `.username` / `.password`	—	connection; no URL + embedded DB on the classpath → in-memory DB ✔	§7.2
`spring.datasource.generate-unique-name`	`true`	random in-memory DB name per context ✔	§7.2
`spring.datasource.hikari.maximum-pool-size`	`10` (HikariCP) ✔	most connections	§7.2
`spring.datasource.hikari.minimum-idle`	= max	leave unset: fixed-size pool	§7.2
`spring.datasource.hikari.connection-timeout`	`30000` ms ✔	wait for a free connection, then fail	§7.2
`spring.datasource.hikari.max-lifetime` / `idle-timeout` / `keepalive-time`	30 min / 10 min / 2 min	connection recycling	§7.2
`spring.datasource.hikari.leak-detection-threshold`	`0` (off)	log connections held too long	§7.2
`spring.datasource.connection-fetch`	`eager`	`lazy` = no physical connection until the first statement (Boot 4.1)	§7.2
`spring.sql.init.mode`	`embedded`	run `schema.sql` / `data.sql`: `embedded`, `always`, `never`	§7.4
`spring.transaction.default-timeout`	—	default transaction timeout	§7.12
`spring.persistence.exceptiontranslation.enabled`	`true`	`@Repository` exception translation (was `spring.dao.*`)	§7.5

JPA, Hibernate, migrations

Property	Default	What it does	§
`spring.jpa.hibernate.ddl-auto`	`create-drop` for an embedded DB without a migration tool, else `none`	`validate` in production	§8.2
`spring.jpa.open-in-view`	`true` (+ startup warning)	set `false` for APIs	§8.15
`spring.jpa.show-sql`	`false`	print SQL (`logging.level.org.hibernate.SQL=DEBUG` is better)	§8.2
`spring.jpa.defer-datasource-initialization`	`false`	run `data.sql` after Hibernate created the tables	§7.4
`spring.jpa.properties.hibernate.default_batch_fetch_size`	—	N+1 safety net (e.g. `16`)	§8.7
`spring.jpa.properties.hibernate.jdbc.batch_size` / `order_inserts`	—	batched inserts (needs SEQUENCE ids)	§8.18
`spring.jpa.properties.*`	—	passed to Hibernate unchanged	§8.2
`spring.flyway.enabled` / `.locations`	`true` / `classpath:db/migration`	Flyway (needs `spring-boot-starter-flyway`)	§9.4
`spring.flyway.baseline-on-migrate` / `.clean-disabled`	`false` / `true`	adopt an existing DB / keep `clean` disabled	§9.4
`spring.liquibase.change-log`	`classpath:/db/changelog/db.changelog-master.yaml`	Liquibase master changelog	§9.4
`spring.data.rest.base-path` / `.detection-strategy`	— / `default`	Data REST prefix / `annotated` = only `@RepositoryRestResource`	§9.3
`spring.mongodb.uri`	`mongodb://localhost/test`	MongoDB connection (Boot 4; `spring.data.mongodb.uri` is no longer bound)	§9.5
`spring.data.redis.host` / `.port`	`localhost` / `6379`	Redis connection	§9.6
`spring.r2dbc.url`	—	reactive SQL connection	§13.14

Cache

Property	Default	What it does	§
`spring.cache.type`	detected (Redis before Caffeine before Simple)	force a provider, or `none` in tests	§9.8
`spring.cache.cache-names`	—	fixed set of caches (typos fail)	§9.8
`spring.cache.caffeine.spec`	—	e.g. `maximumSize=500,expireAfterWrite=10m`	§9.8
`spring.cache.redis.time-to-live`	— (no expiry)	TTL for Redis caches	§9.8
`spring.cache.redis.cache-null-values` / `.key-prefix`	`true` / —	cache `null`s / key prefix (version it)	§9.8

Security

Property	Default	What it does	§
`spring.security.user.name` / `.password` / `.roles`	`user` / random UUID (logged) / —	the dev-only generated user	§10.1
`spring.security.oauth2.resourceserver.jwt.issuer-uri`	—	discover JWKS, check `iss`	§11.3
`spring.security.oauth2.resourceserver.jwt.jwk-set-uri` / `.public-key-location`	—	keys without discovery	§11.3
`spring.security.oauth2.resourceserver.jwt.audiences`	—	check `aud` (do it!)	§11.3
`spring.security.oauth2.resourceserver.jwt.authorities-claim-name` / `.authority-prefix` / `.principal-claim-name`	— (`scope`/`scp` → `SCOPE_`)	map roles from a claim without code	§11.5
`spring.security.oauth2.resourceserver.opaquetoken.introspection-uri` / `.client-id` / `.client-secret`	—	opaque tokens	§11.7
`spring.security.oauth2.client.registration.<id>.*` / `.provider.<id>.issuer-uri`	—	OAuth2 login / client credentials	§11.9, §11.10

Actuator, health, metrics, tracing, logging

Property	Default	What it does	§
`management.endpoints.web.exposure.include` / `.exclude`	`health`	which endpoints are reachable over HTTP (`*` = all)	§15.2
`management.endpoints.web.base-path`	`/actuator`	URL prefix	§15.2
`management.endpoints.access.default` / `management.endpoint.<id>.access`	per endpoint (`shutdown` = `none`)	`none`, `read-only`, `unrestricted` (since 3.4)	§15.2
`management.server.port`	— (same port)	separate management port	§15.2
`management.endpoint.health.show-details`	`never`	`when-authorized`, `always`	§15.3
`management.endpoint.health.probes.enabled`	`true` (Boot 4)	`/actuator/health/liveness` and `/readiness`	§15.3
`management.endpoint.health.group.<name>.include`	—	custom health group	§15.3
`management.endpoint.env.show-values`	`never`	mask values in `env` / `configprops`	§15.13
`management.info.env.enabled` / `.java.enabled`	`false` / `false`	info contributors	§15.4
`management.metrics.tags.<key>`	—	common tag on every meter	§15.5
`management.metrics.distribution.percentiles-histogram.<meter>`	—	histogram buckets for p95/p99	§15.5
`management.observations.annotations.enabled`	`false`	activate `@Observed`, `@Timed`, `@Counted`, `@NewSpan`	§15.6
`management.observations.enable.<prefix>`	—	switch observation families off	§15.6
`management.tracing.sampling.probability`	`0.1`	share of traces recorded	§15.7
`management.tracing.export.enabled`	`true`	export spans (was `management.tracing.enabled`)	§17.13
`management.opentelemetry.tracing.export.otlp.endpoint`	—	OTLP collector URL (was `management.otlp.tracing.endpoint`)	§15.7
`management.tracing.propagation.type`	W3C	`b3` for legacy Zipkin services	§16.9
`management.tracing.baggage.remote-fields` / `.correlation.fields`	—	baggage in headers / in the MDC	§15.7
`logging.level.<logger>`	`INFO` root	per-logger level (`POST /actuator/loggers/...` at runtime)	§15.2
`logging.structured.format.console` / `.file`	—	`ecs`, `logstash`, `gelf` JSON logs (Boot 3.4+)	§15.8
`logging.pattern.correlation`	Boot's `[traceId-spanId]`	how trace ids appear in text logs	§15.7

Threads, async, scheduling, Kafka, RabbitMQ

Property	Default	What it does	§
`spring.task.execution.pool.core-size`	`8` ✔	`applicationTaskExecutor` threads (`@Async`)	§14.4
`spring.task.execution.pool.queue-capacity` / `.max-size`	unbounded ✔ / unlimited	bound the queue!	§14.4
`spring.task.execution.thread-name-prefix`	`task-` ✔	thread names	§14.4
`spring.task.execution.mode`	`auto`	`force` keeps Boot's executor next to your own `Executor` beans	§14.4
`spring.task.execution.propagate-context`	`false`	copy tracing/MDC context into `@Async` threads	§14.4
`spring.task.execution.simple.concurrency-limit`	—	cap for the virtual-thread executor	§14.5
`spring.task.execution.shutdown.await-termination`	`false`	finish queued tasks on shutdown	§14.4
`spring.task.scheduling.pool.size`	`1` ✔	scheduler threads for all `@Scheduled` methods	§14.7
`spring.reactor.context-propagation`	`limited`	`auto` bridges ThreadLocals and the Reactor `Context`	§13.9
`spring.kafka.bootstrap-servers`	`localhost:9092`	brokers (tests: `${spring.embedded.kafka.brokers}`)	§14.10
`spring.kafka.consumer.group-id` / `.auto-offset-reset`	— / Kafka's `latest`	consumer group / where a new group starts	§14.10
`spring.kafka.consumer.isolation-level`	`read-uncommitted`	`read-committed` to hide aborted transactions	§14.16
`spring.kafka.listener.ack-mode` / `.concurrency`	`BATCH` (container default) / 1	offset commits / consumer threads	§14.12
`spring.kafka.producer.acks` / `.transaction-id-prefix`	Kafka's `all` / —	durability / transactional template	§14.16
`spring.kafka.template.observation-enabled` / `listener.observation-enabled`	`false` / `false`	tracing through Kafka headers	§16.9
`spring.rabbitmq.host` / `.listener.simple.prefetch`	`localhost` / —	broker / unacknowledged messages per consumer	§14.18
`spring.rabbitmq.listener.simple.retry.max-retries`	`3`	retries **after** the first attempt (Boot 4; replaces `max-attempts`)	§14.18

Precedence and relaxed binding (✔ proved)

Highest wins (§3.3, §12.15):

1. In tests: `@DynamicPropertySource` > `@TestPropertySource` > `@SpringBootTest(properties = ...)`.
2. Command-line arguments `--shop.name=...`
3. `SPRING_APPLICATION_JSON`
4. Java system properties `-Dshop.name=...`
5. OS environment variables `SHOP_NAME=...`
6. `random.*`
7. Profile-specific files `application-{profile}.*` (outside the jar > inside)
8. `application.properties` / `.yaml` (outside the jar > inside; an imported file > its importer)
9. `@PropertySource` files
10. `SpringApplication.setDefaultProperties(...)`

Rule	Example	§
env var name = upper case, `.` → `_`, drop `-`	`SHOP_CHECKOUTTIMEOUT=2m` → `shop.checkout-timeout` ✔	§3.4
list index	`SHOP_CURRENCIES_0=USD` → `shop.currencies[0]` ✔	§3.4
legacy form (`-` → `_`) is also accepted by the binder	`SHOP_CHECKOUT_TIMEOUT=3m` also binds ✔ — prefer the documented form	§3.4
`Duration`	`45s`, `5m`, `PT1H`; plain `500` = milliseconds ✔	§3.6
`DataSize`	`10MB` = 10 485 760 bytes; plain `1024` = bytes ✔	§3.6
`@Value`	`${key}` fails if missing; `${key:default}`; `#{${key:3} + 1}` = SpEL ✔	§3.5

19.6 Transactions: `@Transactional` at a glance

A Spring transaction = a **proxy** + a `PlatformTransactionManager` + a connection **bound to the current thread** (stored per thread, so every data-access call on that thread reuses the same connection) (§7.7). Everything below applies only to calls that go **through the proxy** (§7.13).

Attributes and their defaults (✔ proved)

Attribute	Default	Meaning	§
`propagation`	`REQUIRED`	what to do when a transaction already exists (table below)	§7.9
`isolation`	`DEFAULT` = the database's default	only applied when this method **starts** the transaction	§7.11
`readOnly`	`false`	JDBC: read-only connection (PostgreSQL rejects writes); JPA: no dirty checking/flush; can route to a replica	§7.12
`timeout`	`-1` (none)	seconds; enforced on the next SQL statement	§7.12
`rollbackFor` / `noRollbackFor`	–	add / remove exception types from the rollback rule	§7.10
`transactionManager` (`value`)	the primary manager	pick one of several managers	§7.8
`label`	–	free-form labels for custom managers	§7.8

Where: on the **service method** that is one use case. Not on controllers (too wide), not only on repositories (too narrow). A class-level `@Transactional(readOnly = true)` + method-level `@Transactional` on writes is the Spring Data pattern (§7.8).

Propagation (✔ all seven proved)

Propagation	A transaction exists	None exists	Typical use
`REQUIRED` (default)	**join** it (same connection)	start one	almost everything
`REQUIRES_NEW`	**suspend** it, start an independent one on a **second connection**; commits even if the caller later rolls back	start one	audit log that must survive a rollback
`NESTED`	**savepoint** inside it; an inner failure rolls back to the savepoint only	start one	"try this part" (JDBC; avoid with JPA)
`MANDATORY`	join it	**throw** `IllegalTransactionStateException`	must be called inside a transaction
`SUPPORTS`	join it	run **without** a transaction	read helpers that don't care
`NOT_SUPPORTED`	**suspend** it, run without	run without	slow non-transactional work inside a flow
`NEVER`	**throw** `IllegalTransactionStateException`	run without	guard: never inside a transaction

The trap ✔: an inner `REQUIRED` method throws, the outer method **catches** the exception and returns normally → the shared transaction was marked **rollback-only** → Spring rolls back at commit and throws `UnexpectedRollbackException` (§7.9).

Rollback rules (✔ proved)

Thrown out of the method	Default	`rollbackFor = X`	`noRollbackFor = X`	`rollbackOn = ALL_EXCEPTIONS` (F6.2)
`RuntimeException` / `Error`	**rollback**	rollback	commit if it is an X	rollback
checked `Exception`	**commit** (!)	rollback if it is an X	commit	rollback
caught **inside** the method	commit – the proxy never sees it	commit	commit	commit

Global switch: `@EnableTransactionManagement(proxyTargetClass = true, rollbackOn = RollbackOn.ALL_EXCEPTIONS)` (§7.10). To mark a transaction without throwing: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` or `status.setRollbackOnly()` in a `TransactionTemplate`.

Isolation levels

Isolation	Prevents	Notes
`DEFAULT`	whatever the database default prevents	PostgreSQL, Oracle, SQL Server: READ COMMITTED; MySQL InnoDB: REPEATABLE READ
`READ_UNCOMMITTED`	nothing guaranteed (dirty reads possible)	PostgreSQL treats it as READ COMMITTED
`READ_COMMITTED`	dirty reads	non-repeatable reads and lost updates still possible (proved in §7.11)
`REPEATABLE_READ`	+ non-repeatable reads	PostgreSQL: one snapshot; concurrent update → serialization failure (`40001`) → retry the whole transaction
`SERIALIZABLE`	+ phantoms, write skew	expect serialization failures; always retry

The anomalies themselves (wrong results that concurrent transactions can produce, such as dirty reads, non-repeatable reads and lost updates): SQL 13.4.

Transaction managers and helpers

You use	Manager Boot creates	§
JDBC (`spring-boot-starter-jdbc`)	`JdbcTransactionManager` (a `DataSourceTransactionManager`)	§7.7
JPA (`spring-boot-starter-data-jpa`)	`JpaTransactionManager` (JDBC on the same `DataSource` joins it)	§7.7
R2DBC	`R2dbcTransactionManager` (context-bound, not thread-bound)	§13.14
Kafka with `spring.kafka.producer.transaction-id-prefix`	`KafkaTransactionManager`	§14.16
Programmatic	`TransactionTemplate.execute(status -> ...)` — no proxy, no self-invocation problem	§7.14
After-commit side effects	`@TransactionalEventListener` (`AFTER_COMMIT` default; skipped without a transaction unless `fallbackExecution = true`; a transactional listener must use `REQUIRES_NEW`)	§7.15

"Why is my `@Transactional` not working?"

Symptom	Cause	§
partial writes after an exception	self-invocation (`this.save()`), `private` method, bean made with `new`	§7.13, §4.5
`NullPointerException` inside a `final` `@Transactional` method	the method ran on the CGLIB proxy instance, whose fields are `null`	§4.5
data committed although the method threw	checked exception without `rollbackFor`, or exception caught inside	§7.10
`UnexpectedRollbackException`	inner `REQUIRED` failure was caught	§7.9
`isolation` / `timeout` ignored	the method joined an existing transaction	§7.11
works in tests, fails in production	the test was `@Transactional` and supplied the transaction	§12.7
`@Async` code doesn't see uncommitted data	transactions are bound to one thread	§7.13
pool exhausted under load	`REQUIRES_NEW` holds two connections; remote calls inside transactions	§7.9, §7.13

19.7 Bean scopes and the lifecycle

Scopes (✔ singleton and prototype proved)

Scope	Instances	Destroy callbacks?	Declare with	§
`singleton` (default)	one per container, shared by all threads — keep it **stateless**	yes	nothing	§2.6
`prototype`	a new one per `getBean`/injection	**no** — Spring forgets it	`@Scope("prototype")`	§2.6
`request`	one per HTTP request	yes	`@RequestScope` (scoped proxy)	§2.6
`session`	one per HTTP session	yes	`@SessionScope`	§2.6
`application`	one per `ServletContext`	yes	`@ApplicationScope`	§2.6
`websocket`	one per WebSocket session	yes	`@Scope("websocket")`	§2.6
refresh (Spring Cloud)	re-created after a config refresh	yes	`@RefreshScope`	§16.3

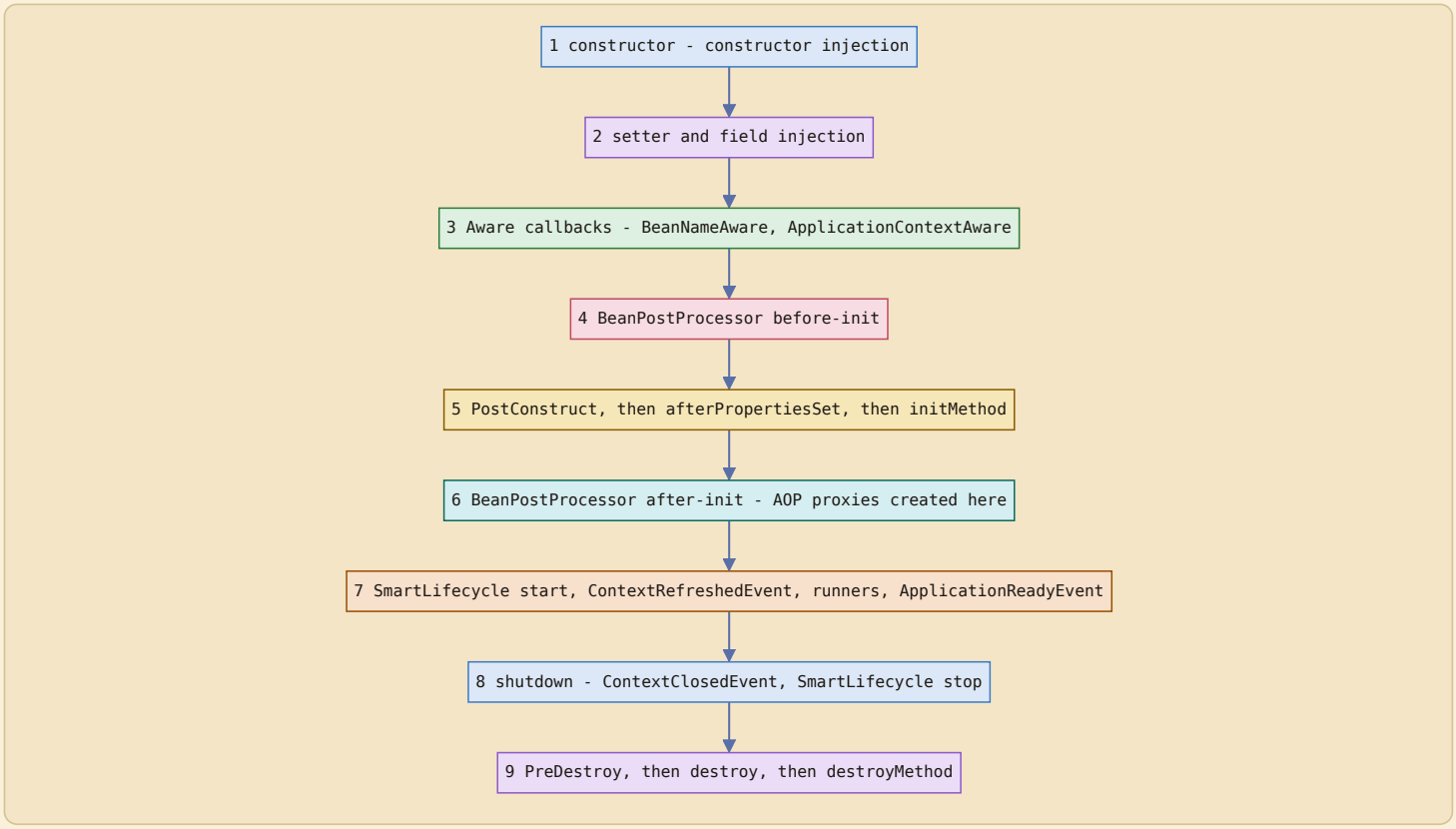
A prototype injected into a singleton is created **once** and stays ✔. For a fresh one each time: `ObjectProvider<T>.getObject()`, a `@Lookup` method, or a scoped proxy (a stand-in that looks up the right instance on every call) (§2.6).

How Spring picks one bean among several (✔ proved)

1. `@Qualifier("name")` on the injection point
2. `@Primary` bean
3. the only non-`@Fallback` bean (F6.2)
4. a bean whose name equals the parameter name (needs `-parameters` — a compiler flag that keeps parameter names in the bytecode — which the Boot parent sets)
5. otherwise `NoUniqueBeanDefinitionException`

`List<T>` gets all beans ordered by `@Order` (lowest first); `Map<String, T>` is keyed by bean name. Optional dependencies: `Optional<T>`, `ObjectProvider<T>`, `@Nullable`, `@Autowired(required = false)` (§2.5).

Lifecycle order (✔ proved)



Proved order: init `[constructor, @PostConstruct, afterPropertiesSet, initMethod]`, destroy `[@PreDestroy, destroy(), destroyMethod]`, and no `@PreDestroy` for the prototype.

Hook	Use for	§
`@PostConstruct` / `@PreDestroy`	default choice: validate, warm up, release	§2.7
`InitializingBean` / `DisposableBean`	framework code (couples you to Spring)	§2.7
`@Bean(initMethod, destroyMethod)`	third-party classes; a public `close()` / `shutdown()` is inferred	§2.7
`SmartLifecycle` (`getPhase()`)	things that *run*: pollers, listeners (start low phase first, stop it last)	§2.7
`CommandLineRunner` / `ApplicationRunner`	one-off startup work with the arguments	§1.8
`@EventListener(ApplicationReadyEvent.class)`	"after the app is fully started"	§2.7
`BeanFactoryPostProcessor` (declare `static`)	change bean **definitions** before any bean exists	§2.8
`BeanPostProcessor` (declare `static`)	change or wrap every bean **instance**	§2.8

Pitfall: a `@Transactional` call from `@PostConstruct` has no transaction — the proxy is created in step 6, after step 5, and the call is on `this` anyway (§2.7).

19.8 Spring Security 7 DSL snippets

Security 7 accepts only the **lambda DSL** (domain-specific language: configuration written as method calls that take lambdas, i.e. short inline functions `x -> ...`): one lambda per feature, no `.and()`, `authorizeHttpRequests` (not `authorizeRequests`), path matchers built on `PathPattern` (§10.3). The four snippets below are compiled by every build of `SecuritySnippets.java`, so each DSL call exists in Security 7.1.1. Rules are checked **top to bottom, first match wins** (§10.6).

Form login + HTTP Basic (browser app)

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth                    // first matching rule wins
            .requestMatchers("/", "/public/**", "/login").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")        // checks authority ROLE_ADMIN
            .requestMatchers(HttpMethod.GET, "/reports/**").hasAuthority("report:read")
            .anyRequest().authenticated())                        // or denyAll() to fail closed
        .formLogin(form -> form.loginPage("/login").permitAll()) // omit loginPage for the generated page
        .httpBasic(Customizer.withDefaults())
        .logout(Customizer.withDefaults());                       // POST /logout (+ CSRF token)
    return http.build();
}

@Bean
PasswordEncoder passwordEncoder() {                               // {bcrypt} by default
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

@Bean
static RoleHierarchy roleHierarchy() {                            // admins also pass hasRole("USER")
    return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER");
}
```

Plus a `UserDetailsService` bean (in-memory in the snippet; JPA/JDBC-backed in real apps). Defining it (or any `SecurityFilterChain`) makes Boot's generated user and default chain back off (§10.1).

JWT resource server (stateless API, first of two chains)

```
@Bean
@Order(1)                                                         // specific chain first
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")                               // which requests this chain owns
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/**").hasAuthority("SCOPE_orders.read")
            .requestMatchers(HttpMethod.POST, "/api/**").hasAuthority("SCOPE_orders.write")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))   // issuer-uri from properties
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(csrf -> csrf.disable());                            // no cookies on this chain
    return http.build();
}
```

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://idp.example.com/realms/shop
spring.security.oauth2.resourceserver.jwt.audiences=orders-api
# optional: roles from a claim, without writing a converter
spring.security.oauth2.resourceserver.jwt.authorities-claim-name=roles
spring.security.oauth2.resourceserver.jwt.authority-prefix=ROLE_
```

Note the last two lines replace the default `scope` → `SCOPE_` mapping; to keep both, write a converter that combines two `JwtGrantedAuthoritiesConverter`s (§11.5). The browser chain (no `securityMatcher`) gets `@Order(2)` — catch-all chains go last (§10.3).

Method security

```
@Configuration
@EnableMethodSecurity                                          // @PreAuthorize & co. (proxy-based)
class MethodSecurityConfig { }

public class OrderService {
    @PreAuthorize("hasRole('ADMIN') or #owner == authentication.name")
    public List<Order> ordersOf(String owner) { ... }

    @PostAuthorize("returnObject.owner() == authentication.name")
    public Order find(long id) { ... }

    @PreAuthorize("hasAuthority('order:refund')")
    public String refund(long id) { ... }
}
```

Proxy rules apply: self-invocation and `private` methods are **not** protected (§10.7).

CORS and CSRF for a cookie-based single-page app

```
@Bean
SecurityFilterChain spa(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .cors(Customizer.withDefaults())         // uses the CorsConfigurationSource bean below
        .csrf(csrf -> csrf.spa())                 // Security 7: XSRF-TOKEN cookie + X-XSRF-TOKEN header
        .formLogin(Customizer.withDefaults());
    return http.build();
}

@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration api = new CorsConfiguration();
    api.setAllowedOrigins(List.of("https://app.example.com"));   // exact origins, never "*" with credentials
    api.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    api.setAllowedHeaders(List.of("Content-Type", "X-XSRF-TOKEN"));
    api.setAllowCredentials(true);
    api.setMaxAge(3600L);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
}
```

The rules in one table

Fact	Detail	§
`hasRole("ADMIN")`	checks the authority `ROLE_ADMIN` ✔; `hasRole("ROLE_ADMIN")` throws `IllegalArgumentException` ✔	§10.6
`hasAuthority("x")`	checks the literal string (`report:read`, `SCOPE_orders.read`)	§10.6
JWT scopes	`scope`/`scp` claim → `SCOPE_x` authorities	§11.5
401 vs 403	anonymous + denied → 401 (or login redirect); authenticated + denied → 403	§10.6
JWT failures	bad/expired token → 401 `invalid_token`; missing scope → 403 `insufficient_scope`; 60 s clock skew	§11.3
CSRF	on for unsafe methods; disable only on chains without cookies/cached Basic	§10.8
CORS	`http.cors(...)` so preflights are answered before authentication	§10.9
sessions	`IF_REQUIRED` (default), `ALWAYS`, `NEVER`, `STATELESS`; session id changes at login	§10.10
passwords	`DelegatingPasswordEncoder`: `{bcrypt}` by default ✔, reads `{noop}`, `{argon2...}`; `upgradeEncoding` → re-hash on login ✔	§10.5
MFA (S7)	every login adds a `FACTOR_*` authority (`FACTOR_PASSWORD`, `FACTOR_OTT`, `FACTOR_WEBAUTHN`)	§10.4
removed in S7	`.and()`, `authorizeRequests`, `AntPathRequestMatcher`/`MvcRequestMatcher`, OAuth2 password grant client support	§17.15
security context	`SecurityContextHolder` is a `ThreadLocal`: use `DelegatingSecurityContext*` executors for `@Async`	§10.2

19.9 Testing annotations and slices

Whole-application tests

Annotation / type	Module / test starter	Purpose	§
`@SpringBootTest`	spring-boot-test (`spring-boot-starter-test`)	full context; `webEnvironment = MOCK` (default), `RANDOM_PORT`, `DEFINED_PORT`, `NONE`	§12.6
`@AutoConfigureMockMvc`	`-webmvc-test`	`MockMvc` + `MockMvcTester` in a `@SpringBootTest` (no longer automatic in Boot 4)	§12.6
`@AutoConfigureRestTestClient`	spring-boot-resttestclient (in `-webmvc-test`)	inject `RestTestClient` (F7), bound to MockMvc or the running server	§12.9
`@AutoConfigureWebTestClient`	`-webflux-test`	`WebTestClient` in a `@SpringBootTest`	§13.12
`@AutoConfigureTestRestTemplate`	spring-boot-resttestclient	legacy `TestRestTemplate`	§12.9
`@LocalServerPort`	spring-boot-test	the random port	§1.12
`@TestConfiguration`	spring-boot-test	extra beans; nested static = added, top-level = `@Import` it	§12.6
`@ActiveProfiles`	spring-test	activate profiles (part of the cache key)	§12.15
`@TestPropertySource` / `properties = ...`	spring-test / spring-boot-test	test-only properties	§12.15
`@DynamicPropertySource`	spring-test	runtime values via `Supplier`s; highest precedence	§12.12
`@DirtiesContext`	spring-test	close and evict the cached context (avoid)	§12.11
`@Sql`	spring-test	run SQL scripts before/after a test	§12.16
`@Transactional` (on a test)	spring-test	rolled back after each test — can hide transaction bugs	§12.7, §12.17

Slices

Slice	Test starter	Loads	Rolled back?	Extras	§
`@WebMvcTest(Controller.class)`	`spring-boot-starter-webmvc-test`	MVC, Jackson, validation, security filters, your controllers/advice/filters	no	`MockMvcTester`, `MockMvc`	§12.7
`@WebFluxTest`	`spring-boot-starter-webflux-test`	WebFlux, codecs, controllers	no	`WebTestClient`	§13.10
`@DataJpaTest`	`spring-boot-starter-data-jpa-test`	JPA, repositories, embedded DB, Flyway/Liquibase	**yes**	`TestEntityManager`, SQL shown	§12.7
`@JdbcTest`	`spring-boot-starter-jdbc-test`	`DataSource`, `JdbcClient`, `JdbcTemplate`	**yes**	—	§7.6
`@DataJdbcTest`	`spring-boot-starter-data-jdbc-test`	Spring Data JDBC + Flyway	**yes**	—	§9.2
`@DataR2dbcTest`	`spring-boot-starter-data-r2dbc-test`	R2DBC repositories	**no**	—	§13.14
`@JsonTest`	`spring-boot-starter-jackson-test`	Jackson `JsonMapper` with your customisations	no	`JacksonTester`	§12.7
`@RestClientTest(Client.class)`	`spring-boot-starter-restclient-test`	`RestClient.Builder`, Jackson, the named client	no	`MockRestServiceServer`	§6.8
`@DataMongoTest` / `@DataRedisTest`	`...-data-mongodb-test` / `...-data-redis-test`	the store's repositories	no	—	§9.7
`@AutoConfigureTestDatabase(replace = ...)`	`spring-boot-starter-jdbc-test`	`NON_TEST` (default since 3.4) replaces your DB unless it is already a test DB; `NONE` keeps it	—	—	§12.7

Mocks, containers, security helpers

Annotation / helper	Module	Purpose	§
`@MockitoBean`	spring-test (F6.2)	Mockito mock **in the context** (replaced removed `@MockBean`); reset after each test	§12.10
`@MockitoSpyBean`	spring-test	spy around the real bean (replaced `@SpyBean`)	§12.10
`@TestBean`	spring-test	override a bean with a static factory method's result	§12.10
`@Mock` + `@ExtendWith(MockitoExtension.class)`	Mockito	plain unit test, no Spring	§12.4
`@Testcontainers` / `@Container`	Testcontainers 2	start Docker containers for the test	§12.14
`@ServiceConnection`	spring-boot-testcontainers	take connection details from the container (no URL properties)	§12.14
`@EmbeddedKafka`	spring-kafka-test (`-kafka-test`)	in-JVM KRaft broker; sets `spring.embedded.kafka.brokers`	§14.13
`@AutoConfigureMetrics` / `@AutoConfigureTracing`	`-micrometer-metrics-test` / `-opentelemetry-test`	keep metric export / span export on in tests	§15.5, §15.7
`@WithMockUser` / `@WithUserDetails` / `@WithAnonymousUser`	spring-security-test	identity for a test method	§10.12
`user()`, `httpBasic()`, `csrf()`, `formLogin()`, `logout()`	spring-security-test	request post-processors / builders	§10.12
`jwt()`, `opaqueToken()`, `oauth2Login()`, `oidcLogin()`	spring-security-test	token identities — pass your converter to `jwt()`	§11.6
`ApplicationContextRunner`	spring-boot-test	tiny context per test, for auto-configuration and conditions	§3.13
`StepVerifier` (+ `withVirtualTime`)	reactor-test	verify `Mono`/`Flux` signals	§13.5
Awaitility `await().atMost(...)`	awaitility (in `spring-boot-starter-test`)	wait for asynchronous results, never `Thread.sleep`	§14.2

Test clients and the context cache

Client	Stack	Needs a server?	§
`MockMvcTester` (F6.2, AssertJ style)	MVC	no	§12.8
`RestTestClient` (F7)	MVC (or any server with `bindToServer`)	optional	§12.9
`WebTestClient`	WebFlux (also MockMvc)	optional	§13.12
`TestRestTemplate`	any	yes	§12.9

Context cache: key = the merged configuration (classes, profiles, properties, `@MockitoBean`s, `@DynamicPropertySource`, `@AutoConfigure...`, `webEnvironment`); **max 32** contexts, LRU (least recently used — the context unused for longest is evicted first) (`spring.test.context.cache.maxSize`); Framework 7 **pauses** inactive contexts (`spring.test.context.cache.pause`). Keep one shared base class (§12.11).

19.10 Reactor operators quick table

`Mono` = 0..1 item, `Flux` = 0..N items; both end with complete **or** error, and **nothing runs until someone subscribes** (in WebFlux, the framework does) (§13.3).

Goal	Operator(s)	Remember	§
create from values	`Mono.just(v)`, `Flux.just(...)`, `Flux.fromIterable(list)`, `Flux.range(1, n)`	`just(expr)` evaluates `expr` **immediately**	§13.3
create lazily	`Mono.fromCallable(...)`, `Mono.defer(() -> ...)`, `Mono.fromFuture(...)`	runs per subscription — retries repeat it	§13.3
empty / error	`Mono.empty()`, `Mono.error(e)`		§13.3
timers	`Flux.interval(d)`, `Mono.delay(d)`, `delayElements(d)`	run on `Schedulers.parallel()`	§13.3
bridge callbacks	`Flux.create(sink -> ...)`, `Sinks.many()`, `Sinks.one()`	choose an overflow strategy	§13.9
transform (sync)	`map`	must not block or return a publisher	§13.4
async call per item	`flatMap(fn[, concurrency])`	concurrent, **interleaved** output; default concurrency 256	§13.4
async, one at a time, in order	`concatMap`	sequential (sum of times)	§13.4
async, concurrent, source order	`flatMapSequential`	buffers early results	§13.4
filter	`filter`, `take(n)`, `skip(n)`, `distinct`		§13.4
combine by position	`Mono.zip(a, b, c)`, `zipWith`	"call 3 services in parallel" (like `Promise.all`)	§13.4
combine in sequence / as they come	`concat`, `concatWith` / `merge`, `mergeWith`		§13.4
fallback when empty	`switchIfEmpty(...)`, `defaultIfEmpty(v)`	the usual 404 path	§13.4
aggregate	`collectList`, `collectMap`, `reduce`, `count`	return a `Mono`	§13.4
group	`buffer(n)`, `window(n)`, `groupBy`		§13.4
then	`then()`, `then(mono)`	"do A, then B", ignoring A's values	§13.4
side effects	`doOnNext`, `doOnError`, `doOnSubscribe`, `doFinally`, `log()`	not for business logic	§13.4
errors: fallback	`onErrorReturn(v)`, `onErrorResume(e -> pub)`, `onErrorMap(e -> ex)`	an error **ends** the sequence	§13.6
errors: retry	`retry(n)`, `retryWhen(Retry.backoff(3, Duration.ofSeconds(1)))`	re-subscribes; attempts at 0/1/3/7 s with `jitter(0)`	§13.6
timeout	`timeout(d[, fallback])`	before `retryWhen` = per attempt	§13.6, §13.13
backpressure	`limitRate(n)` (re-requests at 75 %), `onBackpressureDrop/Latest/Buffer(n)/Error`	unbounded buffers risk `OutOfMemoryError`	§13.7
threads	`publishOn(s)` (everything **after** it), `subscribeOn(s)` (the **source**; position irrelevant)		§13.8
schedulers	`Schedulers.parallel()`, `boundedElastic()` (wrap blocking calls), `single()`, `immediate()`	never block Netty event loops	§13.8
hot / shared	`share()`, `publish().autoConnect()`, `cache()`, `Sinks.many().multicast()`	late subscribers miss earlier items	§13.9
context	`contextWrite(c -> c.put(k, v))`, `Mono.deferContextual(ctx -> ...)`	the reactive `ThreadLocal`	§13.9
leave the reactive world	`block()`, `blockFirst()`, `blockLast()`	tests and `main` only — throws on event-loop threads	§13.3
test	`StepVerifier.create(pub).expectNext(...).verifyComplete()`, `withVirtualTime(() -> ...)` + `thenAwait(d)`	build the publisher **inside** the supplier	§13.5

19.11 Actuator endpoints

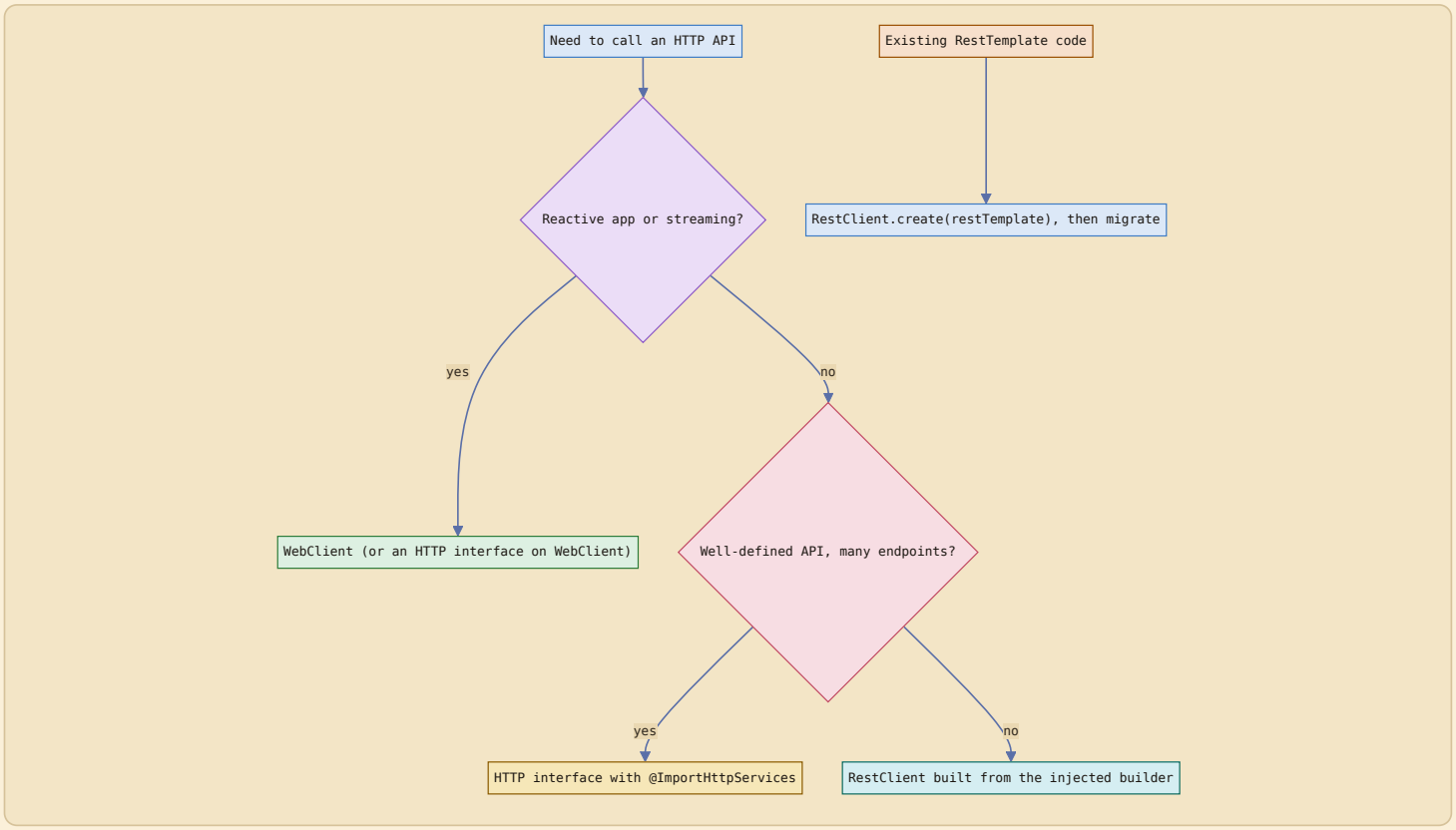
Add `spring-boot-starter-actuator`. Two switches decide availability: **access** (`none` / `read-only` / `unrestricted`) and **exposure** (HTTP / JMX — Java Management Extensions, the JVM's built-in management channel). By default only `health` is exposed over HTTP (§15.2).

Endpoint (`/actuator/<id>`)	Shows / does	Notes	§
`health`	status `UP` / `DOWN` / `OUT_OF_SERVICE` / `UNKNOWN`; groups `liveness`, `readiness` (on by default in Boot 4)	`DOWN` / `OUT_OF_SERVICE` → HTTP 503; details `never` by default	§15.3
`info`	build, git, java, os, process, `info.*`	contributors switched on by properties	§15.4
`metrics`, `metrics/{name}`	meters as JSON, drill down with `?tag=k:v`	for humans	§15.5
`prometheus`	all meters in Prometheus text format	needs `micrometer-registry-prometheus`	§15.5
`loggers`	view and change log levels at runtime	`POST` with `{"configuredLevel":"DEBUG"}`	§15.2
`env`, `configprops`	property sources, bound `@ConfigurationProperties`	values masked (`show-values=never`)	§3.11, §15.2
`conditions`	the auto-configuration report	"why was bean X (not) created?"	§3.11
`beans`, `mappings`	all beans / all request mappings	debugging	§15.2
`threaddump`, `heapdump`	JVM threads / heap dump file	heap dumps can contain secrets	§15.2
`scheduledtasks`, `caches`, `flyway`, `liquibase`, `sessions`, `quartz`	feature-specific views (`caches` can also clear)	appear when the feature is present	§15.2
`httpexchanges`	recent requests/responses	needs an `HttpExchangeRepository` bean	§15.2
`startup`	startup steps and durations	needs `BufferingApplicationStartup`	§15.2
`sbom`, `auditevents`, `integrationgraph`, `logfile`	software bill of materials, audit events, Spring Integration graph, the log file	when configured	§15.2
`shutdown`	stops the app	access `none` by default	§15.2
`refresh`, `busrefresh` (Spring Cloud)	re-bind config / broadcast a refresh	Spring Cloud Context / Bus	§16.3

Secure them with `EndpointRequest.to("health", "info")` / `EndpointRequest.toAnyEndpoint()` (Boot 4 package `org.springframework.boot.security.autoconfigure.actuate.web.servlet`), ideally on a separate `management.server.port` (§15.2).

19.12 Which tool for which job

HTTP client: `RestClient` vs `WebClient` vs HTTP interface



	`RestClient`	`WebClient`	**HTTP interface (`@HttpExchange`)**	`RestTemplate`
Style	fluent, blocking	fluent, non-blocking (`Mono`/`Flux`)	declarative Java interface → proxy	template methods
Since / status	F6.1, recommended for blocking code	F5.0, recommended for reactive	F6.0; Boot 4 groups via `@ImportHttpServices`	deprecation announced (F7.1 `@Deprecated`, removal in 8.0)
Starter (Boot 4)	`spring-boot-starter-restclient`	`spring-boot-starter-webclient`	`-restclient` (or `-webclient` with `clientType = WEB_CLIENT`)	`-restclient`
Pick when	hand-written calls, custom per-call logic	WebFlux apps, streaming, heavy async composition	typed clients to other services, least boilerplate	never for new code
Watch out	inject `RestClient.Builder` (Boot converters, timeouts, tracing); `exchange()` skips status handlers	don't `block()` on event loops	wrap for `Optional`/domain exceptions	—
§	§6.2	§13.13	§6.6	§6.9

Web stack: Spring MVC + virtual threads vs WebFlux

	MVC + `spring.threads.virtual.enabled=true`	WebFlux
Code style	plain imperative, blocking libraries OK	`Mono` / `Flux` pipelines, non-blocking everything
Measured (1,000 clients, 100 ms I/O)	**7,942 req/s**, p50 108 ms, 33 platform threads	**8,058 req/s**, p50 110 ms
Same test, MVC on 200 platform threads	1,909 req/s, p50 517 ms	WebFlux + one `Thread.sleep`: **97 req/s**
Data access	JDBC, JPA	R2DBC, reactive Mongo/Redis
Streaming, backpressure end to end (SSE, WebSockets)	possible (`SseEmitter`), each stream holds a request	natural (`Flux`)
Debugging, `ThreadLocal` libraries	easy	needs Reactor `Context`, context propagation
Choose for	typical CRUD/REST services (the default)	streaming, gateways, all-non-blocking pipelines
§	§13.15, §15.10	§13.15

Data access: JPA vs `JdbcClient` vs Spring Data JDBC

	Spring Data JPA (Hibernate)	`JdbcClient` / `JdbcTemplate`	Spring Data JDBC
Model	rich object graph, relationships both ways	your own SQL, rows → records	DDD aggregates: root + owned children
Magic	persistence context, lazy loading, dirty checking, first-level cache	none	none: `save` writes, loading reads the whole aggregate
SQL you can predict	not always (N+1, flush timing)	yes	yes (children are deleted + re-inserted on save)
Schema	`ddl-auto` possible (use Flyway)	you write it	you write it (Flyway)
Typical pitfalls	N+1, `LazyInitializationException`, OSIV	SQL injection by concatenation, `.single()` on 0 rows	assigned ids without `@Version` → silent `UPDATE` of 0 rows
Choose for	domain models with relationships, team knows JPA	reports, bulk work, full SQL control	clear aggregates, predictable SQL
§	§8.1	§7.3	§9.2

Also: R2DBC for non-blocking SQL (§13.14), MongoDB for flexible documents (§9.5), Redis for counters/sessions/shared cache (§9.6); the overview table is §9.9.

Cache: Caffeine vs Redis

	Caffeine (local)	Redis (shared)
Read cost	nanoseconds, no serialization	network hop + (de)serialization
Consistency across instances	each instance has its own copy (stale until TTL)	one copy for all
Survives restarts	no	yes (with Redis persistence)
Stores	object **references** — cache immutable values	serialized copies (choose a JSON serializer)
Stampede protection	`@Cacheable(sync = true)` per key per JVM	same annotation, still per JVM; distributed lock for cluster-wide
TTL / size	`expireAfterWrite`, `maximumSize` (`spring.cache.caffeine.spec`)	`spring.cache.redis.time-to-live` / `RedisCacheConfiguration`
Choose for	small, hot, rarely changing reference data	data shared between instances, sessions, expensive results
§	§9.8	§9.8

Messaging: Kafka vs RabbitMQ

	Kafka	RabbitMQ
Model	partitioned, replicated **log**; consumers track offsets	**queue** broker: exchanges → bindings → queues; acked messages are deleted
Replay / many independent readers	yes: each consumer group has its own offset	no: each reader needs its own queue
Ordering	per partition (use the aggregate id as key)	per queue, weakened by competing consumers
Routing	topic + partition only	rich: direct, topic, fanout, headers
Per-message TTL, priority, delay	no	yes (delay via plugin)
Spring	`KafkaTemplate`, `@KafkaListener`, `DefaultErrorHandler` → `<topic>-dlt`, `@RetryableTopic`	`RabbitTemplate`, `@RabbitListener`, DLX, publisher confirms
Test without Docker	`@EmbeddedKafka`	no embedded broker (Testcontainers)
Choose for	event streaming, event sourcing, high throughput	task distribution, complex routing, request/reply
§	§14.9	§14.18

Resilience: Resilience4j vs Framework 7 `@Retryable`

Need	Framework 7 core	Resilience4j	Spring Retry
Retry with back-off + jitter	`@Retryable` (`maxRetries` = retries **after** the first call ✔), `RetryTemplate`	`@Retry` (`max-attempts` counts the first call)	yes (no longer managed by Boot 4)
Bulkhead	`@ConcurrencyLimit` (`BLOCK` / `REJECT`)	`@Bulkhead` (semaphore / thread pool)	no
Circuit breaker	**no**	`@CircuitBreaker` (CLOSED → OPEN → HALF_OPEN)	basic
Rate limiter / time limiter	no	`@RateLimiter`, `@TimeLimiter`	no
Fallback methods	no (catch at the caller)	`fallbackMethod`	`@Recover`
Metrics, health	`MethodRetryEvent`, `RetryListener`	Micrometer meters, health indicators	listeners
Switch	`@EnableResilientMethods`	`resilience4j-spring-boot4` + properties	`@EnableRetry`
Choose for	simple retries and concurrency caps, no extra dependency	circuit breaking, rate limits, fallbacks	legacy code only
§	§4.7	§16.8	§4.7

Rules for any retry: only transient failures (short-lived ones that may succeed a moment later) and idempotent operations (safe to repeat), exponential back-off (longer waits after each failure) with jitter (a random extra delay so clients don't retry in lockstep), a total time budget, retry in **one** layer (§6.7).

More decisions in one line each

Question	Answer	§
`@Value` or `@ConfigurationProperties`?	one value / SpEL → `@Value`; a group of settings → a `@ConfigurationProperties` record	§3.6
`@Component` or `@Bean`?	your class → stereotype; third-party class or creation logic → `@Bean`	§2.3
Unit test, slice or `@SpringBootTest`?	logic → unit; Spring-produced behaviour of one layer → slice; wiring → a few `@SpringBootTest`s	§12.2
`Page` or `Slice` (or `Window`)?	page numbers → `Page` (extra COUNT); "load more" → `Slice`; big tables → keyset `Window`	§8.11
`@EventListener`, `@TransactionalEventListener` or outbox?	in-process now → `@EventListener`; only after commit → `@TransactionalEventListener`; must not be lost → outbox	§2.12, §7.15, §14.17
JWT or opaque token?	local fast validation → JWT; instant revocation / hidden claims → opaque + introspection	§11.7
Session cookie or tokens?	browser app / BFF → session (+ CSRF); service-to-service, mobile → bearer tokens	§10.10, §11.12
Blocking or non-blocking retries (Kafka)?	order matters → `DefaultErrorHandler` (blocking); independent records → `@RetryableTopic`	§14.15
Flyway or Liquibase?	SQL-first, one database → Flyway; rollbacks, preconditions, many databases → Liquibase	§9.4
Eureka or Kubernetes Services?	on Kubernetes, Services + DNS usually suffice	§16.4
OpenFeign or HTTP interfaces?	new code → HTTP interfaces (OpenFeign is in maintenance mode)	§16.6
JVM + AOT cache or native image?	long-running, throughput-heavy → JVM (AOT cache halves startup); CLIs, scale-to-zero → native	§15.11

19.13 Boot 3 → 4 quick migration checklist

Work top to bottom; each step names the symptom you see if you skip it. Details: §17.12, §17.13, §17.14, §17.15.

Before you start

- [ ] Upgrade to **Boot 3.5.x** first and remove every deprecation warning (4.0 deletes what 3.x deprecated). From 2.7, go 2.7 → 3.5 (`javax` → `jakarta`) first.
- [ ] Java 17+ (25 recommended); Kotlin 2.2+, GraalVM 25+, Gradle 8.14+ if you use them.
- [ ] Run OpenRewrite's (an automated code-refactoring tool) `org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0` recipe and review the diff.
- [ ] Add `spring-boot-properties-migrator` (runtime, temporarily) to report renamed properties; remove it afterwards.
- [ ] Stuck? `spring-boot-starter-classic` / `spring-boot-starter-test-classic` restore the old all-in-one layout while you fix imports.

Dependencies

- [ ] `spring-boot-starter-web` → `spring-boot-starter-webmvc`; `-aop` → `-aspectj`; `-oauth2-*` → `-security-oauth2-*`; `-web-services` → `-webservices`.
- [ ] Add `spring-boot-starter-restclient` (client code) and `spring-boot-starter-webclient` (`WebClient.Builder`, else "No qualifying bean of type WebClient$Builder").
- [ ] Add a **test starter** per technology (`-webmvc-test`, `-data-jpa-test`, ...) — else `@WebMvcTest` does not resolve.
- [ ] Add `spring-boot-starter-flyway` / `-liquibase` — else migrations silently stop running.
- [ ] Undertow → Tomcat or Jetty. Spring Retry → declare its version or move to `@Retryable` (F7).
- [ ] Separately versioned projects: Spring Cloud **2025.1.x** (2025.1.2+ for Boot 4.1), Spring AI **2.0**, springdoc **3.x**, Testcontainers **2.x** artifact names.

Code

- [ ] Jackson 3: `com.fasterxml.jackson.databind` → `tools.jackson.databind` (annotations unchanged); `ObjectMapper` bean → `JsonMapper`; `Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`.
- [ ] Auto-configuration class imports moved to per-module packages (check `exclude = ...`, `@ImportAutoConfiguration`, `@AutoConfiguration(after = ...)`).
- [ ] `@EntityScan` → `org.springframework.boot.persistence.autoconfigure`; `hibernate-jpamodelgen` → `hibernate-processor`.

- [ ] Null-safety: `org.springframework.lang.Nullable` → `org.jspecify.annotations.Nullable`; `javax.annotation`/`javax.inject` (the old Java EE package names, now `jakarta.*`) are gone.
- [ ] Security 7: rewrite `.and()` chains as one lambda per feature, `authorizeRequests` → `authorizeHttpRequests`, Ant/MVC matchers → `PathPatternRequestMatcher` (plain `requestMatchers("/x/**")`).
- [ ] `RestTemplate` → `RestClient` (start with `RestClient.create(restTemplate)`).

Tests

- [ ] `@MockBean` / `@SpyBean` → `@MockitoBean` / `@MockitoSpyBean` (removed in Boot 4; not allowed inside `@Configuration` classes).
- [ ] `@SpringBootTest` + `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureWebTestClient` / `@AutoConfigureTestRestTemplate` — no longer automatic.
- [ ] Slice annotation imports moved (e.g. `org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest`); JUnit 4 runners → JUnit 6 (Jupiter).

Properties (most common renames)

Boot 3	Boot 4
`spring.jackson.parser.*` / `generator.*`	`spring.jackson.json.read.*` / `json.write.*`
`spring.data.mongodb.uri` / `.host` / ...	`spring.mongodb.uri` / `.host` / ...
`spring.http.client.*`	`spring.http.clients.*`
`server.error.*` (e.g. `include-stacktrace`, `path`)	`spring.web.error.*`
`spring.dao.exceptiontranslation.enabled`	`spring.persistence.exceptiontranslation.enabled`
`management.tracing.enabled`	`management.tracing.export.enabled`
`management.otlp.tracing.endpoint`	`management.opentelemetry.tracing.export.otlp.endpoint`
`management.health.mongo.enabled`	`management.health.mongodb.enabled`
`spring.session.redis.*`	`spring.session.data.redis.*`
`spring.kafka.retry.topic.backoff.random`	`spring.kafka.retry.topic.backoff.jitter`
`spring.rabbitmq.listener.simple.retry.max-attempts` (counted the first call)	`...retry.max-retries` (does not)

Behaviour changes to re-test

- [ ] Health probes (`/actuator/health/liveness`, `/readiness`) are **on by default** — check what your platform calls.
- [ ] DevTools LiveReload is off (and deprecated in 4.1).
- [ ] `server.forward-headers-strategy` no longer applies to WAR deployments (apps packaged as a `.war` file and deployed into an external servlet container).
- [ ] Jackson 3 defaults: ISO dates, unknown properties ignored, properties sorted alphabetically after creator properties.

19.14 Interview index

Chapter 18 holds 168 questions with short answers, five runnable proofs (small tests that demonstrate an answer by running it), a project walkthrough and ten debugging scenarios. This index maps each topic to its Chapter 18 section and to the chapter that teaches it.

Topic	Chapter 18 section	Questions	Teaching chapter
IoC, beans, scopes, lifecycle, events	§18.2	Q1–Q20	Chapter 2
Boot, starters, configuration, auto-configuration	§18.3	Q21–Q35	Chapter 1, Chapter 3
AOP, proxies, Framework 7 resilience	§18.4	Q36–Q44	Chapter 4
MVC, REST, validation, errors, versioning, clients	§18.5	Q45–Q61	Chapter 5, Chapter 6
JDBC, transactions, JPA, Spring Data	§18.6	Q62–Q91	Chapter 7, Chapter 8, Chapter 9
Caching	§18.7	Q92–Q97	Chapter 9
Security, JWT, OAuth2	§18.8	Q98–Q114	Chapter 10, Chapter 11
Testing	§18.9	Q115–Q125	Chapter 12
Reactive and WebFlux	§18.10	Q126–Q132	Chapter 13
Messaging, async, scheduling	§18.11	Q133–Q140	Chapter 14
Production and observability	§18.12	Q141–Q152	Chapter 15
Microservices and Spring Cloud	§18.13	Q153–Q160	Chapter 16
Spring AI and Boot 4 / Framework 7 changes	§18.14	Q161–Q168	Chapter 17
Proofs of five tricky answers	§18.15	prototype-in-singleton, `@Qualifier` vs `@Primary`, cycles, checked exceptions, self-invocation	this chapter's §19.15 adds more
"Explain your Spring project" template	§18.16	—	—
Ten debugging scenarios	§18.17	bean not found, cycles, no rollback, lazy loading, N+1, 401/403, CORS, "annotation does nothing", auto-configuration failures, slow startup	—

The twenty questions that come up most

#	Question	Short answer	Study
1	IoC vs DI?	framework creates objects / objects receive dependencies	§2.1
2	`BeanFactory` vs `ApplicationContext`?	lazy minimal container / eager, events, environment, post-processors	§2.2
3	Why constructor injection?	`final`, complete objects, testable with `new`, cycles fail fast	§2.4
4	Two beans of one type?	`@Qualifier` > `@Primary` > `@Fallback` > parameter name	§2.5
5	Singleton thread-safe? prototype in singleton?	shared → keep stateless / created once, use `ObjectProvider`	§2.6
6	Bean lifecycle?	§19.7 order	§2.7
7	How does auto-configuration work?	`.imports` files, conditions, applied after your beans, `@ConditionalOnMissingBean`	§3.10
8	Property precedence?	command line > system props > env vars > profile files > `application.*`	§3.3
9	JDK vs CGLIB proxy; self-invocation?	interfaces vs subclass (Boot default CGLIB) / `this.x()` skips the proxy	§4.4, §4.5
10	Request flow in MVC?	filters → `DispatcherServlet` → mapping → interceptors → arguments → controller → converter → resolvers	§5.2
11	Global error handling?	`@RestControllerAdvice` + `ProblemDetail` (RFC 9457)	§5.7
12	How does `@Transactional` work?	proxy + manager + thread-bound connection	§7.7
13	Propagation types; checked exception?	§19.6 / commits by default	§7.9, §7.10
14	N+1 and `LazyInitializationException`?	JOIN FETCH / `@EntityGraph` / batch size; fetch per use case, DTOs	§8.7, §8.6
15	Security filter chain; 401 vs 403?	`DelegatingFilterProxy` → `FilterChainProxy` → first matching chain	§10.2, §10.6
16	JWT validation in a resource server?	signature via JWKS, `exp` / `nbf` (60 s skew), `iss`, `aud`, `scope` → `SCOPE_`	§11.3
17	Slices vs `@SpringBootTest`; context caching?	partial vs full context / cache keyed by configuration, max 32	§12.7, §12.11
18	WebFlux or virtual threads?	MVC + virtual threads for CRUD; WebFlux for streaming	§13.15
19	Reliable DB write + event?	transactional outbox + idempotent consumers	§14.17
20	What's new in Boot 4 / Framework 7?	modular starters, Jackson 3, JSpecify, API versioning, HTTP service clients, `@Retryable`, `RestTestClient`	§17.13, §17.14

19.15 Proofs: the facts on these sheets, run

What it is

`code/cheatsheet-checks/` is a small Maven (Java's standard build tool) project (Boot 4.1.1, Java 25, no web server, no Docker) whose tests assert the facts marked ✔ in this chapter. If a future Boot version changes a default, a test fails, and the sheet must be updated. Run it from the repository root with `bash tools_run_spring.sh spring '19-*'`.

Test class	Proves	Sheet
`ScopeAndLifecycleTests`	singleton shared, prototype fresh, prototype-in-singleton stuck, init/destroy order, no destroy callback for prototypes	§19.7
`BeanSelectionTests`	`@Qualifier` beats `@Primary`, `@Primary` beats the parameter name, `List` by `@Order`, `Map` by bean name	§19.3, §19.7
`TransactionTests`	`@Transactional` defaults, all seven propagation types, rollback rules, `UnexpectedRollbackException`	§19.6
`PropertyBindingTests`	env-var mapping (canonical and legacy), list index, `Duration` / `DataSize` units, record + `@DefaultValue`	§19.5
`PropertyPrecedenceTests`	defaults < `application.properties` < env var < `-D` < `--arg`; `@Value` defaults and SpEL	§19.5
`BootDefaultsTests`	HikariCP pool 10 / timeout 30 s, unique embedded DB name, `@Async` executor 8 threads `task-` (virtual threads → `SimpleAsyncTaskExecutor`), scheduler 1 thread, shutdown 30 s	§19.5
`CoreUtilitiesTests`	default cache keys, six-field cron, `@Retryable` defaults and 1 + `maxRetries` attempts	§19.3, §19.12
`JsonAndSecurityDefaultsTests`	Jackson 3 defaults, `hasRole` prefix rules, `DelegatingPasswordEncoder`	§19.5, §19.8

The transaction checks use plain Spring on an in-memory H2 database (a small Java SQL database that lives only in the test's memory), the same technique as §18.15:

```
@Transactional
void catchInnerRequired() {
    inner.insert("outer");
    try { inner.requiredThenFail(); } catch (IllegalStateException swallowed) { }
}   // commit attempt -> UnexpectedRollbackException, nothing is saved
```

The environment-variable checks bind through the same `Binder` that `@ConfigurationProperties` uses, over a fake `SystemEnvironmentPropertySource`:

```
Binder canonical = envBinder(Map.of("SHOP_CHECKOUTTIMEOUT", "2m", "SHOP_PAYMENT_RETRIES", "4", "SHOP_CURRENCIES_0", "USD"));
Duration timeout = canonical.bind("shop.checkout-timeout", Duration.class).get();   // PT2M
```

Output

`mvn -q -B -ntp verify` (20 tests, all passing; JUnit does not fix the order of test classes, so blocks may appear in another order):

```
spring.task.scheduling.pool.size default = 1, spring.lifecycle.timeout-per-shutdown-phase default = PT30S
applicationTaskExecutor = ThreadPoolTaskExecutor, core = 8, prefix = task-, queue = unbounded
with spring.threads.virtual.enabled=true -> SimpleAsyncTaskExecutor
DataSource = HikariDataSource, maximumPoolSize = 10, connectionTimeout = 30000 ms
no spring.datasource.url + H2 on the classpath -> jdbc:h2:mem:<random name>  (generate-unique-name=true)
'0 30 2 * * MON-FRI' after 2026-09-28T10:00 -> 2026-09-29T02:30
five-field Unix cron '0 2 * * *' -> IllegalArgumentException
SimpleKeyGenerator: () -> SimpleKey [], ("BOOK-1") -> BOOK-1, ("BOOK-1","EUR") -> SimpleKey [BOOK-1, EUR]
@Retryable defaults: maxRetries=3, delay=1000 ms, multiplier=1.0
maxRetries = 2 -> [attempt 1, attempt 2, attempt 3], then the last exception is rethrown
SHOP_CHECKOUTTIMEOUT=2m   -> shop.checkout-timeout = PT2M
SHOP_PAYMENT_RETRIES=4    -> shop.payment.retries  = 4
SHOP_CURRENCIES_0=USD     -> shop.currencies[0]    = USD
SHOP_CHECKOUT_TIMEOUT=3m  -> shop.checkout-timeout = PT3M (legacy '-' -> '_' form, also accepted)
Duration '45s' -> PT45S, '500' -> 500 ms, 'PT1H' -> PT1H
DataSize '10MB' -> 10485760 bytes, '1024' -> 1024 bytes
record bound with @DefaultValue: ShopProperties[name=Corner Shop, checkoutTimeout=PT30S, maxUpload=5242880B]
init order    : [constructor, @PostConstruct, afterPropertiesSet, initMethod]
destroy order : [@PreDestroy, destroy(), destroyMethod]  (no Proto @PreDestroy)
singleton: getBean twice -> same instance?        true
prototype: getBean twice -> new instance?         true
prototype injected into a singleton -> same one?  true
@Value ${shop.missing:fallback} -> fallback, #{${shop.retries:3} + 1} with retries=2 -> 3, key set only in defaultProperties -> eu (default)
defaultProperties + application.properties -> from application.properties
+ env SHOP_NAME                           -> from env var
+ -Dshop.name                             -> from -D system property
+ --shop.name                             -> from --command-line
by type (@Primary)        -> sms
@Qualifier("email")        -> email
parameter named 'push'    -> sms  (@Primary wins over the name)
List<Notifier> by @Order  -> [email, sms, push]
Map keys (bean names)     -> [email, push, sms]
encode -> {bcrypt}$2a$10$...  matches {noop}s3cret? true  upgradeEncoding({noop}) = true
hasRole("ADMIN"): authority ROLE_ADMIN -> true, authority ADMIN -> false
hasRole("ROLE_ADMIN") -> IllegalArgumentException
JsonMapper (tools.jackson.databind.json) writes {"item":"book","on":"2026-09-28"}
unknown property ignored -> Delivery[item=pen, on=2026-10-01]
broken JSON -> JacksonException (unchecked)
committed despite an exception: [checked, default rules, caught inside the method]
inner REQUIRED failed, outer caught it -> UnexpectedRollbackException, rows = []
@Transactional defaults: propagation=REQUIRED isolation=DEFAULT readOnly=false timeout=-1
REQUIRED      : inner runs in the outer transaction? true (Outer.requiredJoins)
REQUIRES_NEW  : outer rolled back, rows left = [audit (REQUIRES_NEW)]
NESTED        : inner failed + caught, rows left = [outer] (savepoint)
MANDATORY     : no transaction -> IllegalTransactionStateException
NEVER         : inside a transaction -> IllegalTransactionStateException
SUPPORTS      : active alone? false, active inside a transaction? true
NOT_SUPPORTED : active inside a transaction? false (outer suspended)
```

What the output teaches

- A **legacy** environment-variable spelling (`SHOP_CHECKOUT_TIMEOUT`, dash turned into an underscore) also binds to `shop.checkout-timeout`. The documented form without the dash separator (`SHOP_CHECKOUTTIMEOUT`) is the one to use and to search for; know that both work when you hunt for "who set this value?".
- `@Primary` beats a matching parameter name — the name is only the **last** tie-breaker.
- HikariCP reports its pool size only after the pool has started; the test takes one connection first.
- The `@Retryable` bean is a CGLIB proxy, so the test reads its attempt list through a method: a field read on a proxy returns the proxy's own, never-initialised field (§4.5).

19.16 Glossary A–Z

Short definitions of the terms used in this book. Annotations are listed without `@` in the sort order. Follow the link for the full explanation.

A

Term	Meaning	See
Access (Actuator)	per-endpoint permission: `none`, `read-only`, `unrestricted` (Boot 3.4+); separate from exposure	§15.2
Access token	short-lived token a client sends to an API to prove it may call it	§11.1
`acks` (Kafka)	how many replicas must confirm a write; `all` (`-1`) is the default since Kafka 3.0	§14.16
Actuator	Boot's management endpoints: health, metrics, info, loggers, env...	§15.2
Advice (AOP)	the code an aspect runs at a join point: before, after returning, after throwing, after, around	§4.1
Advisor (AOP)	Spring's internal pair of one advice and one pointcut	§4.1
Advisor (Spring AI)	interceptor around every `ChatClient` call (memory, RAG, tool calling, logging), ordered by `getOrder()`	§17.5
AES-GCM	standard symmetric encryption (one shared key) with built-in tamper detection; used for encrypting secrets and data	§10.5
Aggregate	DDD cluster of objects loaded and saved as one unit through its root; the unit of Spring Data JDBC	§9.2
`AggregateReference`	Spring Data JDBC reference to another aggregate by id only	§9.2
Annotation (Java)	metadata written as `@Name` on a class, method or field; Spring reads it to add behaviour	§1.1
Anthropic / OpenAI / Ollama	AI model providers Spring AI has starters for; Ollama runs open models on your own machine	§17.2
AOP	Aspect-Oriented Programming: write a cross-cutting concern once, apply it where a pointcut matches	§4.1
AOT cache (JDK)	JDK 24+ file of classes loaded and linked in a training run; `-XX:AOTCacheOutput` / `-XX:AOTCache`	§15.11
AOT processing (Spring)	build-time generation of bean-definition code, used by native images and optionally on the JVM	§15.11
API	Application Programming Interface: the operations one piece of software offers to another (Java methods or HTTP endpoints)	§1.1
API gateway	single front door that routes requests and applies edge concerns (Spring Cloud Gateway)	§16.7
API versioning	Framework 7 feature: read a version from the request and match `version = "1.1+"` mappings	§5.9
`ApiVersionInserter`	client-side strategy that puts the API version into requests	§6.7
`ApplicationContext`	Spring's full container: eager singletons, post-processors, events, `Environment`, i18n	§2.2
`ApplicationContextRunner`	test helper that builds a tiny context per test, ideal for auto-configuration	§3.13
`ApplicationEventPublisher`	bean used to publish application events	§2.12
`ApplicationReadyEvent`	Boot event after startup and runners have finished	§2.7
`ApplicationRunner` / `CommandLineRunner`	beans run once at startup with the program arguments	§1.8
Argon2 / bcrypt / PBKDF2 / scrypt	slow, salted password-hashing algorithms; bcrypt is Spring Security's default	§10.5
Aspect	class bundling pointcuts and advice for one concern (`@Aspect` + `@Component`)	§4.2
AspectJ	full AOP language with compile/load-time weaving; Spring borrows its annotations and pointcut syntax	§4.8
AssertJ	fluent assertion library: `assertThat(x).isEqualTo(y)`; included in the Boot test starter	§12.8
Asynchronous	the caller continues without waiting; the result arrives later (callback, future, message)	§14.1
At-least-once	delivery guarantee: never lost, possibly duplicated — consumers must be idempotent	§14.8
Audience (`aud`)	JWT claim naming who the token is meant for; an API should reject tokens for another audience	§11.1
Auditing (Spring Data)	automatically filling created/modified date and user columns	§8.13
`Authentication`	object holding the principal, credentials and authorities of the current caller	§10.1
Authentication	proving who the caller is	§10.1
`AuthenticationEntryPoint`	starts authentication for an anonymous, denied request: 401 challenge or login redirect	§10.2
`AuthenticationManager`	entry point of authentication; usually a `ProviderManager` asking `AuthenticationProvider`s	§10.4

Term	Meaning	See
Authority (`GrantedAuthority`)	one permission string held by the caller (`report:read`, `ROLE_ADMIN`, `SCOPE_x`)	§10.1
Authorization	deciding whether a known caller may perform an action	§10.1
Authorization code flow	OAuth2 flow for apps with a user: code in the browser, token exchange on the back channel (with PKCE)	§11.9
Authorization server	issues tokens (Keycloak, Okta, Entra ID, Spring Authorization Server)	§11.2
`AuthorizationFilter`	last filter of a security chain; applies `authorizeHttpRequests` rules	§10.2
Auto-configuration	conditional `@Configuration` classes that Boot applies after yours to fill gaps	§3.10
`AutoConfiguration.imports`	file `META-INF/spring/...AutoConfiguration.imports` listing a jar's auto-configurations	§3.10
Availability state	Boot's `LivenessState` and `ReadinessState`, exposed as health groups	§15.3
Awaitility	test library that polls a condition until true or timeout	§14.2
`Aware` callbacks	interfaces (`BeanNameAware`, `ApplicationContextAware`) through which a bean receives container objects	§2.7

B

Term	Meaning	See
B3 headers	Zipkin's older trace-propagation headers, an alternative to W3C `traceparent`	§15.7
Backpressure	the consumer limits the producer via `request(n)`	§13.7
Baggage	extra key/value context that travels with a trace	§15.7
Bean	object created, wired and managed by the Spring container	§2.1
Bean definition	the recipe for a bean: class or factory method, scope, dependencies, callbacks	§2.2
Bean Validation	Jakarta standard for constraint annotations (`@NotBlank`, `@Email`); Hibernate Validator implements it	§5.6
`BeanFactory`	minimal container: bean definitions + lazy creation	§2.2
`BeanFactoryPostProcessor`	changes bean **definitions** before any bean is created (e.g. `${}` resolution)	§2.8
`BeanPostProcessor`	sees every bean **instance** before/after init and may wrap it in a proxy	§2.8
`BeanRegistrar`	Framework 7 contract for registering beans programmatically (loops, conditions)	§2.14
Bearer token	token that grants access to whoever holds it, sent as `Authorization: Bearer ...`	§11.1
BFF (Backend for Frontend)	server-side OAuth2 client that keeps tokens away from the browser and gives it a session cookie	§11.12
BlockHound	Java agent that detects blocking calls on non-blocking threads in tests	§13.8
Blocking call	a call that holds its thread while it waits (JDBC, `Thread.sleep`, a blocking HTTP client)	§13.1
BOM	"bill of materials" POM pinning versions of many artifacts (`spring-boot-dependencies`)	§1.6
`boundedElastic`	Reactor scheduler for wrapping unavoidable blocking calls	§13.8
BPP / BFPP	short for `BeanPostProcessor` / `BeanFactoryPostProcessor`	§2.8
Brave / Zipkin / Jaeger / Tempo	tracing libraries and back ends: Brave records spans, Zipkin, Jaeger and Tempo store and display traces	§15.7
Broker (message broker)	server that receives messages from producers and delivers them to consumers (Kafka, RabbitMQ)	§14.8
Brute-force protection	slowing or locking out repeated failed logins so passwords can't simply be guessed	§11.15
Bucket4j	Java rate-limiting library based on the token-bucket algorithm	§11.15
Build tool	program that downloads dependencies, compiles, tests and packages the code (Maven, Gradle)	§1.6
Bulkhead	cap on concurrent calls to one dependency (`@ConcurrencyLimit`, Resilience4j `@Bulkhead`)	§4.7, §16.8
Buildpacks	Cloud Native Buildpacks: build an OCI image from a jar without a Dockerfile (`spring-boot:build-image`)	§15.11
Bytecode	the compiled instructions in `.class` files that the JVM executes	§4.4

C

Term	Meaning	See
Cache-aside	look in the cache; on a miss load, store and return — what `@Cacheable` does	§9.8
Cache stampede	many concurrent misses for one hot key all compute it; `sync = true` mitigates per JVM	§9.8
`CacheManager`	provider-neutral cache interface behind the cache annotations	§9.8
Caffeine	high-performance local Java cache (W-TinyLFU eviction)	§9.8
Cardinality	number of distinct tag-value combinations of a metric; keep it small	§15.5
Cascade	passing a JPA operation (persist, merge, remove...) from parent to children	§8.5
Cascading failure	one slow or failed service makes its callers fail too, spreading through the system	§16.8
CDC (Change Data Capture)	streaming every insert/update/delete from the database's change log (e.g. Debezium); one way to relay an outbox	§14.17
CDS	Class Data Sharing: JVM archive of parsed classes for faster startup	§15.11
CGLIB proxy	runtime subclass used as a proxy; Boot's default; cannot handle `final`	§4.4
`ChatClient`	Spring AI fluent API over a `ChatModel`, with advisors, templates, tools	§17.3
`ChatMemory`	Spring AI store of past messages per conversation id (window of 20 by default)	§17.6
`ChatModel`	Spring AI provider-level interface: `Prompt` in, `ChatResponse` out	§17.3
Checked exception	exception the compiler forces you to declare or catch (not a `RuntimeException`); commits a transaction by default	§7.10
Chunking (RAG)	splitting documents into small pieces before embedding them, so retrieval returns focused passages	§17.9
Circuit breaker	stops calling a failing dependency (CLOSED → OPEN → HALF_OPEN)	§16.8
Circular dependency	dependency cycle between beans; constructor cycles always fail, Boot rejects field cycles	§2.9
Claim	statement in a JWT payload (`sub`, `iss`, `aud`, `exp`, `scope`)	§11.1
Class loader	JVM component that finds and loads `.class` files; DevTools uses two of them for fast restart	§1.10
Classpath	the list of jars and folders the JVM loads classes from; auto-configuration reacts to what is on it	§1.2
CLI	command-line interface: a program you run from a terminal	§1.9
Client credentials	OAuth2 grant for service-to-service calls without a user	§11.10
Cold publisher	replays the whole pipeline for every subscriber	§13.9
Commit / rollback	finish a transaction by making its changes permanent / by undoing all of them	§7.7
Component scanning	finding `@Component` classes in a package tree	§2.3
`ConcurrencyLimit`	Framework 7 annotation limiting concurrent calls of a method (`BLOCK` or `REJECT`)	§4.7
Condition evaluation report	Boot's list of which auto-configurations matched and why (`--debug`, `/actuator/conditions`)	§3.11
`Conditional`	register a bean only if a `Condition` matches; base of all `@ConditionalOn...`	§2.11
Config data	Boot's files and imports: `application.*`, profile files, `spring.config.import`	§3.3
Config Server	Spring Cloud service serving configuration from Git or other backends	§16.3
`configtree:`	import where each file in a directory is a property (Kubernetes secrets)	§3.8
`ConfigurationProperties`	binds a prefix of properties into a typed, validated object	§3.6
Connection pool	set of open database connections reused across requests because opening one is slow (HikariCP)	§7.2
Constructor injection	dependencies passed as constructor parameters; the recommended injection style	§2.4
Consul	HashiCorp service registry and key/value store, an alternative to Eureka	§16.4
Consumer group	Kafka consumers sharing a `group.id`; each partition goes to one member, each group gets every record	§14.9
Consumer lag	how many messages a consumer group is behind the newest ones in a partition	§14.12
Container (Docker)	isolated process packaged with everything it needs to run; started from a container image	§12.14

Term	Meaning	See
Container image	packaged file system plus start command from which containers are run (OCI format)	§15.11
Content negotiation	choosing the representation from `Accept` / `produces` (406 if impossible)	§5.8
Context caching (tests)	the TestContext Framework reuses started contexts with the same configuration	§12.11
Context pausing	Framework 7: inactive cached test contexts have their lifecycle beans stopped	§12.11
Context propagation	copying `ThreadLocal` values (trace, MDC) across threads or into the Reactor `Context`	§15.6, §13.9
Contract testing	consumer and provider tested against a shared contract (Pact, Spring Cloud Contract)	§16.10
Cookie	small value a site stores in the browser, sent back automatically on every request to that site	§10.10
CORS	browser rules letting a page from one origin read responses from another (preflight `OPTIONS`)	§10.9
Counter / Timer / DistributionSummary	Micrometer meters: a count that only rises / durations / sizes	§15.5
CPU-bound vs I/O-bound	work limited by computation vs work that mostly waits for network or disk	§13.8
CQRS	Command Query Responsibility Segregation: separate models for writes and for reads	§16.1
Cron expression	Spring's six-field schedule, starting with seconds (`0 0 2 * * *`)	§14.7
Cross-cutting concern	requirement spread over many classes (logging, transactions, security)	§4.1
CRUD	Create, Read, Update, Delete: the four basic data operations	§8.8
CSRF	attack that makes a logged-in browser send a forged state-changing request; blocked by tokens	§10.8
curl / Postman	command-line and graphical tools for sending HTTP requests by hand; they ignore CORS	§1.5
Customizer	bean that adjusts an auto-configured object instead of replacing it	§3.12

D

Term	Meaning	See
DAO	Data Access Object: a class that hides the SQL for one kind of data	§7.1
`DataAccessException`	Spring's unchecked, database-neutral exception hierarchy	§7.5
`DataSize`	Boot type for sizes (`10MB`); plain numbers are bytes	§3.6
`DataSource`	factory of JDBC connections; in Boot a HikariCP pool	§7.2
DDD	Domain-Driven Design: modelling software around business concepts (entities, aggregates, bounded contexts)	§9.2
DDL / `ddl-auto`	Data Definition Language (`CREATE TABLE`, `ALTER`); `ddl-auto` lets Hibernate create or validate the schema	§8.16
Dead-letter exchange (DLX)	RabbitMQ exchange that receives rejected or expired messages from a queue	§14.18
Dead-letter topic (DLT)	where records go after the last failed attempt; default name `<topic>-dlt` (spring-kafka 3.3+)	§14.14
`DeadLetterPublishingRecoverer`	spring-kafka recoverer that publishes failed records to the DLT	§14.14
Deadlock	two transactions each wait for a lock the other holds; the database aborts one of them	§7.11
`DefaultErrorHandler`	spring-kafka container error handler: blocking retries, then a recoverer	§14.14
`DelegatingFilterProxy`	servlet filter that hands requests to the `springSecurityFilterChain` bean	§10.2
`DelegatingPasswordEncoder`	encoder storing `{id}hash`; verifies old formats, upgrades on login	§10.5
Dependency	a library your project uses, declared in the build file; or an object a bean needs	§1.6
Dependency Injection (DI)	an object receives its dependencies instead of creating them	§2.1
Deprecated	still works but is marked for removal in a later version; move off it	§1.4
Derived query	Spring Data method whose name is the query (`findByTitleContaining`)	§8.9
Deserialization / serialization	turning bytes or JSON into objects / objects into bytes or JSON	§5.5
Detached entity	JPA entity no longer in a persistence context; changes are not tracked	§8.4
DevTools	development module: fast restart, dev defaults; off for `java -jar`	§1.10
Dirty checking	Hibernate compares managed entities with their snapshots at flush and writes `UPDATE`s	§8.4
`DirtiesContext`	test annotation that closes and evicts the cached context	§12.11
`DiscoveryClient`	Spring Cloud abstraction returning the instances of a service id	§16.4
`DispatcherServlet`	Spring MVC's front controller	§5.2
`DispatcherHandler`	WebFlux's front controller	§13.10
DNS	Domain Name System: turns host names into network addresses; Kubernetes uses it for service discovery	§16.4
Docker	tool that builds and runs containers; Testcontainers needs it	§12.14
DTO	Data Transfer Object: a simple class or record shaped for an API instead of the database	§5.4
Dual-write problem	a database write and a broker publish cannot be atomic; solved by the outbox	§14.17
`Duration` binding	`45s`, `5m`, `PT1H`; plain numbers are milliseconds	§3.6
`DynamicPropertySource`	test method registering properties with lazy suppliers (random ports, containers)	§12.12

E

Term	Meaning	See
EAGER / LAZY	load an association with its owner / on first access; make everything LAZY	§8.6
EJB	Enterprise JavaBeans: the older Java EE component model Spring was created as an alternative to	§1.4
Embedded database	database that runs inside the application's JVM (H2, HSQLDB); handy for tests and demos	§7.2
Embedded server	Tomcat/Jetty/Netty running inside the application jar	§1.2
`EmbeddedKafka`	in-JVM Kafka broker for tests (spring-kafka-test)	§14.13
Embedding	vector of numbers representing text so similar texts are close	§17.9
`EnableAutoConfiguration`	imports Boot's auto-configuration candidates	§3.10
`EnableResilientMethods`	Framework 7 switch for `@Retryable` and `@ConcurrencyLimit`	§4.7
Entity	JPA class whose instances are table rows (`@Entity`, `@Id`, no-arg constructor, not `final`)	§8.3
Entity graph	declares associations to fetch with a query (`@EntityGraph`, left join)	§8.7
`EntityManager`	JPA API over the persistence context (Hibernate `Session`)	§8.4
`Environment`	Spring's view of all property sources and active profiles	§3.1
ETag	response header holding a version fingerprint so clients can ask "has it changed?" (304 Not Modified)	§8.14
Eureka	Netflix service registry supported by Spring Cloud (AP design)	§16.4
Event loop	few threads that run pipeline steps when I/O completes, never waiting	§13.1
Event-driven	services communicate by publishing and reacting to events instead of calling each other directly	§14.8
`EventListener`	method receiving application events, synchronously by default	§2.12
Eventual consistency	copies or services agree after a short delay instead of instantly	§14.17
Exactly-once	each message's effect happens once; in practice at-least-once delivery plus idempotent processing	§14.16
Exception translation	turning `SQLException`s into `DataAccessException`s	§7.5
`ExceptionTranslationFilter`	turns security exceptions into 401/redirect (anonymous) or 403 (authenticated)	§10.2
Exchange / binding (RabbitMQ)	an exchange receives messages and routes them to queues according to bindings	§14.18
Executor / `TaskExecutor`	owner of worker threads and a task queue; runs `@Async` work	§14.4
Exponential back-off	waiting longer after each failed attempt (1 s, 2 s, 4 s...) before retrying	§4.7
Exposure (Actuator)	whether an endpoint is reachable over HTTP or JMX (`health` only, by default)	§15.2
Externalized configuration	settings outside the build artifact (files, env vars, args)	§3.1

F

Term	Meaning	See
`FACTOR_*` authority	Security 7 authority recording a completed login factor (`FACTOR_PASSWORD`)	§10.4
`Fallback`	Framework 6.2: bean used only if no other candidate qualifies	§2.5
Fallback (resilience)	default answer returned when a protected call fails or a circuit is open	§16.8
Fat (executable) jar	Boot jar with `BOOT-INF/lib` dependencies, started by `JarLauncher`	§1.9
Filter (servlet)	code around every request before the `DispatcherServlet`; can wrap request/response	§5.11
`FilterChainProxy`	the `springSecurityFilterChain` bean that picks the first matching `SecurityFilterChain`	§10.2
`fixedDelay` / `fixedRate`	`@Scheduled` timing: end-to-start / start-to-start	§14.7
`flatMap` / `concatMap`	async per item: concurrent and interleaved / sequential and ordered	§13.4
Flush	Hibernate sends queued SQL: before commit, before affected queries, on `flush()`	§8.4
`Flux`	Reactor publisher of 0..N items	§13.3
Flyway	SQL-file schema migration tool (`V1__x.sql`, checksums)	§8.16
Foreign key	column that points to the primary key of a row in another table	§8.5
Form login	username/password via an HTML form; session cookie afterwards	§10.4
Full vs lite configuration	`@Configuration` CGLIB subclass (inter-bean calls return singletons) vs plain calls	§2.3
Functional endpoints	routes as `RouterFunction` beans and handler functions instead of annotations	§13.11

G – H

Term	Meaning	See
GA / milestone / SNAPSHOT	release stages: general availability (production-ready) / preview builds / unreleased development builds	§1.4
Garbage collection (GC)	the JVM automatically freeing memory of objects no longer used; pauses can add latency	§13.15
Gateway (Spring Cloud)	API gateway: routes = predicates + filters; Web MVC or WebFlux flavour	§16.7
Gauge	Micrometer meter sampling a current value	§15.5
GPU	graphics processor; runs AI models much faster than a CPU	§10.5
Graceful shutdown	stop accepting requests, finish in-flight ones, then close (default since Boot 3.4)	§15.9
GraalVM native image	whole app compiled ahead of time to a native executable (closed world)	§15.11
Gradle	build tool alternative to Maven, with Groovy or Kotlin build scripts	§1.5
Grant type (OAuth2)	way a client obtains a token: authorization code, client credentials, refresh token, token exchange	§11.2
H2	small Java SQL database that can run in memory; used for demos and quick tests	§7.2
HAL / HATEOAS	REST style where responses include links to related actions; HAL is a JSON format for those links	§5.12
`HandlerInterceptor`	MVC hook around handler methods (`preHandle`, `postHandle`, `afterCompletion`)	§5.11
`HandlerMapping`	finds the handler method for a request	§5.2
Hash (password)	one-way fingerprint of a value; passwords are stored as slow, salted hashes, never encrypted	§10.5
Health group	named subset of health indicators at `/actuator/health/<group>`	§15.3
`HealthIndicator`	bean contributing one check to `/actuator/health`	§15.3
Heap	JVM memory area where objects live; sized with `-Xmx` or `-XX:MaxRAMPercentage`	§15.11
Hibernate	the JPA implementation (ORM) Boot uses; maps entities to tables and generates SQL	§8.1
HikariCP	Boot's default JDBC connection pool (default 10 connections)	§7.2
HMAC / RSA / EC (JWT signing)	signing families: HMAC (HS256) shares one secret; RSA (RS256) and EC (ES256) sign privately and verify with a public key	§11.1
Hot publisher	produces whether or not anyone listens; late subscribers miss items	§13.9
HOTP / TOTP	one-time password algorithms based on a counter / on the current time	§11.18
HSQLDB / Derby	other embedded Java databases Boot can auto-configure, like H2	§7.2
HTTP	HyperText Transfer Protocol: request/response protocol of the web (method, path, headers, body, status)	§5.1
HTTP Basic	credentials in an `Authorization: Basic` header on every request	§10.4
HTTP methods	GET reads, POST creates, PUT replaces, PATCH partly updates, DELETE removes	§5.1
HTTP service interface	Java interface with `@GetExchange`... implemented by a generated proxy	§6.6
HTTP status 401 vs 403	401 = not authenticated (who are you?), 403 = authenticated but not allowed	§10.6
`HttpMessageConverter`	converts bodies to and from Java objects (Jackson for JSON)	§5.2
`HttpOnly` / `Secure` / `SameSite`	cookie flags: hidden from JavaScript / sent only over HTTPS / not sent on cross-site requests	§11.12
HTTPS / TLS	HTTP encrypted with TLS (Transport Layer Security), which also proves the server's identity	§11.18

I – J

Term	Meaning	See
ID token	OIDC JWT that tells the client who logged in; not for calling APIs	§11.2
IDE	Integrated Development Environment such as IntelliJ IDEA, Eclipse or VS Code	§1.5
Idempotency key	unique id a client sends with a request so a retried request is processed only once	§5.1
Idempotent	repeating the operation gives the same result (GET, PUT, DELETE; not POST)	§5.1
Idempotent consumer	de-duplicates messages by id so at-least-once delivery is safe	§14.17
Idempotent producer	Kafka producer whose retries cannot duplicate records (on by default)	§14.16
Identity provider (IdP)	system that owns user accounts and issues tokens	§11.2
`IDENTITY` / `SEQUENCE`	JPA id strategies: auto-increment (no insert batching) / database sequence	§8.3
`ImportHttpServices`	Framework 7 registration of HTTP interface groups; Boot configures them by properties	§6.6
Integration test	test of several real parts working together (often a whole context plus a real database)	§12.2
Interceptor	code that runs around a call (method, handler or HTTP request) and can change or stop it	§5.11
Introspection	asking the authorization server whether an opaque token is active (RFC 7662)	§11.7
Inversion of Control (IoC)	the framework, not your code, creates objects and calls you	§2.1
Isolation level	how much a transaction sees of concurrent ones (READ COMMITTED...)	§7.11
Issuer (`iss`)	JWT claim naming the authorization server that created the token	§11.1
J2EE / Java EE	old names of the enterprise Java standards, now Jakarta EE	§1.4
Jackson 3	Boot 4's JSON library, packages `tools.jackson.*`	§5.5
Jakarta EE	successor of Java EE; `jakarta.*` packages (EE 11 in Boot 4)	§1.4
Jar / WAR	Java archive of compiled classes / web archive deployed into an external servlet container	§1.9
`JarLauncher`	`Main-Class` of a Boot fat jar that loads nested jars and calls your `Start-Class`	§1.9
Java record	compact immutable data class (`record Order(long id, String status)`); ideal for DTOs and properties	§1.11
JDBC	Java Database Connectivity: Java's standard low-level API for SQL databases	§7.1
`JdbcClient`	Framework 6.1 fluent API for SQL with automatic record mapping	§7.3
`JdbcTemplate`	classic SQL template; still used for batch updates	§7.3
JDK	Java Development Kit: compiler, tools and the Java runtime	§1.4
JDK dynamic proxy	`java.lang.reflect.Proxy` implementing the bean's interfaces only	§4.4
JEP	JDK Enhancement Proposal: the numbered design document for a new Java feature	§13.15
JIT compiler	Just-In-Time compiler: turns frequently run bytecode into machine code while the app runs	§15.11
Jitter	random variation added to retry delays so clients don't retry in lockstep	§4.7
JMS	Jakarta Messaging: the standard Java API for message brokers such as ActiveMQ Artemis	§14.18
JMX	Java Management Extensions: built-in JVM channel for monitoring and management tools	§15.2
Join point	a point where advice can run; in Spring AOP always a bean method execution	§4.1
`JOIN FETCH`	JPQL that loads an association in the same query (N+1 fix)	§8.7
JPA	Jakarta Persistence specification; Hibernate implements it	§8.1
JPQL	query language over entities and fields, not tables	§8.10
JSON	JavaScript Object Notation: the text format of most REST APIs (`{"id": 1}`)	§5.5
`JsonMapper`	Jackson 3's immutable JSON mapper, built with `JsonMapper.builder()`	§5.5
JSpecify	standard null-safety annotations (`@NullMarked`, `@Nullable`) used by Framework 7	§2.13
JUnit	the standard Java test framework (JUnit 6 / Jupiter in Boot 4)	§12.3
JVM	Java Virtual Machine: the runtime that loads and executes bytecode	§1.2
JWK Set (JWKS)	JSON document of an issuer's public keys, selected by `kid`	§11.1
JWS / JWE	a signed JWT (contents readable, tamper-proof) / an encrypted JWT (contents hidden)	§11.1
JWT	signed token `header.payload.signature`; claims readable by anyone	§11.1

Term	Meaning	See
`JwtAuthenticationConverter`	turns JWT claims into authorities (`scope` → `SCOPE_x`)	§11.5
`JwtDecoder`	parses and validates a JWT (signature, `exp`, `iss`, custom validators)	§11.3

K – L

Term	Meaning	See
Kafka	distributed, replicated commit log for event streaming	§14.9
`KafkaListener`	spring-kafka consumer method	§14.12
`KafkaTemplate`	spring-kafka producer API; `send` returns a `CompletableFuture`	§14.11
Key (Kafka)	decides the partition; same key → same partition → ordered	§14.9
Keycloak	open-source identity provider and authorization server	§11.2
KMS	Key Management Service: cloud service that stores encryption keys and performs crypto without revealing them	§11.13
KRaft	Kafka's built-in consensus replacing ZooKeeper (Kafka 4.0+)	§14.9
Kubernetes (K8s)	platform that runs, restarts, scales and connects containers across machines	§15.3
Lambda (Java)	short inline function, `x -> x * 2`; the Security DSL is configured with lambdas	§10.3
Latency	how long one request takes from start to finish (often reported as p50/p99)	§15.5
Layered jar	Boot jar split into layers (dependencies, application code) so container images rebuild only what changed	§15.11
Lazy initialization	create beans on first use (`@Lazy`, `spring.main.lazy-initialization`)	§2.10
`LazyInitializationException`	touching a lazy association after its persistence context closed	§8.6
LDAP	directory protocol used by companies to store users and groups; Spring Security can authenticate against it	§9.1
Lifecycle callbacks	init/destroy hooks: `@PostConstruct`, `afterPropertiesSet`, `@PreDestroy`...	§2.7
Liquibase	changelog-based migration tool with rollbacks and preconditions	§9.4
Liveness / readiness / startup probes	Kubernetes health checks: restart me? / send me traffic? / am I still starting?	§15.3
Liveness probe	"restart me?" — must depend only on internal state	§15.3
LLM	large language model: the text-generating AI behind chat APIs	§17.1
Load balancer	component that spreads requests over several instances of a service	§16.5
Load balancing (client-side)	the caller picks an instance per request (Spring Cloud LoadBalancer, round robin)	§16.5
Load testing (Gatling, k6, JMeter)	tools that fire many concurrent requests to measure throughput and latency under load	§12.18
Logback / SLF4J	SLF4J is the logging API your code calls; Logback is Boot's default implementation behind it	§15.8
Lombok	library that generates getters, constructors, `equals` and similar code at compile time	§8.3
`Lookup`	method injection returning a fresh prototype per call	§2.6
Lost update	two read-modify-write cycles overwrite each other; prevent with SQL math, locks or `@Version`	§7.11
LRU	least recently used: an eviction policy that removes the entry unused for longest	§9.8

M

Term	Meaning	See
Managed entity	entity tracked by the persistence context; changes are flushed automatically	§8.4
`mappedBy`	marks the inverse side of a JPA relationship; only the owning side is written	§8.5
`MappedCollection`	Spring Data JDBC child collection mapping (`idColumn`, `keyColumn`)	§9.2
Maven	Java build tool: `pom.xml` declares dependencies and plugins; `mvn verify` builds and tests	§1.6
MCP	Model Context Protocol: standard way to expose and call AI tools across processes	§17.10
MD5 / SHA-256	fast hash functions; fine for checksums, unsafe alone for passwords because they are fast to guess	§10.5
MDC	Mapped Diagnostic Context: per-thread log fields (request id, trace id)	§14.4, §15.1
`merge` vs `persist`	copy a detached entity's state onto a managed copy / make a new entity managed	§8.4
`MeterRegistry`	Micrometer registry where meters are recorded and exported	§15.5
Method security	authorization on bean methods (`@PreAuthorize...`), proxy-based	§10.7
Method validation	constraints on method parameters; built into MVC since Framework 6.1	§5.6
Metrics / logs / traces	the three kinds of observability data: numbers over time / event records / request paths	§15.1
Micrometer	vendor-neutral metrics facade ("SLF4J for metrics")	§15.1
Micrometer Tracing	tracing facade over OpenTelemetry or Brave	§15.7
MFA	multi-factor authentication; Security 7 tracks factors as `FACTOR_*` authorities	§10.4
Microservice	small, independently deployed service owning one business capability and its data	§16.1
Mock / stub / spy	test doubles: a mock records and checks calls, a stub returns canned answers, a spy wraps a real object	§12.4
Mockito	the Java mocking library behind `@MockitoBean`	§12.10
`MockitoBean` / `MockitoSpyBean`	mock / spy registered in the test context (replaced `@MockBean` / `@SpyBean`)	§12.10
`MockMvc` / `MockMvcTester`	in-process MVC test client; `MockMvcTester` adds AssertJ assertions (F6.2)	§12.8
`MockRestServiceServer`	fake server bound to a `RestClient.Builder` in `@RestClientTest`	§6.8
Modular starters	Boot 4: one starter, one auto-configuration module and one test starter per technology	§3.14
MongoDB	document database storing JSON-like documents (BSON)	§9.5
`Mono`	Reactor publisher of 0..1 item	§13.3
Monolith / modular monolith	one deployable application; modular when its internal modules have strict boundaries (Spring Modulith)	§16.1
mTLS	mutual TLS: both client and server present certificates, so each proves its identity	§11.10
Multi-tenancy (JWT)	several issuers in one API; `JwtIssuerAuthenticationManagerResolver` with an allow-list	§11.8
MVCC	Multi-Version Concurrency Control: the database keeps row versions so readers don't block writers	§7.11

N – O

Term	Meaning	See
N+1 problem	1 query for parents + 1 per parent for a lazy association	§8.7
Native query	plain database-specific SQL in a Spring Data `@Query` instead of JPQL	§8.10
NDJSON	newline-delimited JSON: one JSON object per line, used for streaming responses	§13.10
`NESTED`	propagation using a savepoint inside the current transaction	§7.9
Netty	non-blocking network library; Reactor Netty is WebFlux's default server	§13.10
`NimbusJwtEncoder`	Spring Security's JWT signer; Security 7 adds `withKeyPair` / `withSecretKey` builders	§11.4
Non-blocking	a call that returns immediately and frees its thread; the result is delivered later	§13.1
Nonce	OIDC random value bound into the ID token against replay	§11.9
`NullMarked` / `Nullable`	JSpecify: non-null by default in a scope / this may be null	§2.13
OAuth 2.0	framework for delegated authorization with access tokens	§11.2
`ObjectProvider`	lazy/optional lookup of a bean (`getIfAvailable`, `getObject` for prototypes)	§2.5
Observation API	Micrometer API: instrument once, get a metric and a span	§15.6
`Observed`	annotation creating an observation around a method	§15.6
Offset (Kafka)	a record's position in its partition; committed per consumer group	§14.9
OIDC	OpenID Connect: identity layer on OAuth2 adding the ID token	§11.2
ONNX	portable file format for machine-learning models, e.g. local embedding models	§17.1
Opaque token	random token string checked by introspection; instantly revocable	§11.7
Open Session in View	persistence context kept open for the whole web request (`spring.jpa.open-in-view`)	§8.15
OpenAPI	machine-readable API description; generated by springdoc	§5.12
OpenFeign	Spring Cloud declarative client, in maintenance mode	§16.6
OpenRewrite	automated refactoring recipes used for Boot upgrades	§17.12
OpenTelemetry	open standard and SDK for traces/metrics; OTLP is its protocol	§15.7
Optimistic locking	`@Version` column checked in `UPDATE ... WHERE version = ?`	§8.14
ORM	Object-Relational Mapping: mapping classes to tables so you work with objects instead of SQL rows	§8.1
`orphanRemoval`	delete a child removed from its parent's collection	§8.5
OSIV	short for Open Session in View	§8.15
OTLP	OpenTelemetry Protocol: the standard wire format for exporting traces, metrics and logs	§15.7
Outbox (transactional)	write the event into an outbox table in the same transaction; a relay publishes it	§14.17
OWASP	Open Worldwide Application Security Project: publishes the Top 10 web security risks	§10.5
Owning side	the relationship side holding the foreign key; the only one JPA writes	§8.5

P

Term	Meaning	See
p99 / percentile	the value that 99% of measurements are below; p99 latency shows the slow tail	§15.5
Pact	polyglot consumer-driven contract-testing tool with a broker and `can-i-deploy` check	§16.10
`Page` / `Slice` / `Window`	paged result with total (extra COUNT) / "has next" only / keyset scrolling	§8.11
Partition	ordered, append-only log inside a Kafka topic; unit of parallelism	§14.9
Passkey / WebAuthn	password-less login with a key pair stored on the user's device; WebAuthn is the browser standard	§10.4
Password4j	hashing library behind the new password encoders of Spring Security 7	§10.5
`PathPattern`	Spring's URL pattern matcher (`{id}`, `**`, `{*path}`); no trailing-slash match	§5.3
`PathPatternRequestMatcher`	Security 7 request matcher using `PathPattern` syntax	§10.3
Persistence context	the `EntityManager`'s first-level cache and unit of work	§8.4
Pessimistic locking	`SELECT ... FOR UPDATE` via `@Lock(PESSIMISTIC_WRITE)`	§8.14
PKCE	proof key that binds an authorization code to the client that started the flow	§11.9
Platform thread	ordinary Java thread backed one-to-one by an operating-system thread	§13.1
`PlatformTransactionManager`	Spring interface that begins, commits and rolls back transactions	§7.7
Pod	smallest unit Kubernetes runs: one or more containers sharing a network address	§15.3
Pointcut	predicate selecting join points (`execution(...)`, `within(...)`, `@annotation(...)`)	§4.3
Poison pill	record that can never be deserialized; `ErrorHandlingDeserializer` sends it to the DLT	§14.10
POM	Project Object Model: Maven's `pom.xml` build file	§1.6
PostgreSQL	open-source relational database used for most examples in this book	§8.2
Precedence	which property source wins (command line > ... > defaults)	§3.3
Preflight	the browser's `OPTIONS` request asking CORS permission	§10.9
`Primary`	default bean when several candidates match	§2.5
Primary key	column (or columns) that uniquely identifies each row of a table	§8.3
Principal	the "who" of a request: user, service or device	§10.1
`ProblemDetail`	RFC 9457 error body: `type`, `title`, `status`, `detail`, `instance`	§5.7
Profile	named environment selecting beans (`@Profile`) and files (`application-dev.yaml`)	§3.7
Profile group	one profile name that activates several	§3.7
Projection	query result with only the needed fields (interface or record)	§8.12
Prometheus	monitoring system that scrapes metrics from `/actuator/prometheus` and stores them as time series	§15.5
Prompt template	text with `{placeholders}` filled at run time (Spring AI `PromptTemplate`)	§17.4
Propagation	what a transactional method does when a transaction already exists	§7.9
Property source	one named set of key/values inside the `Environment`	§3.1
Prototype scope	new bean instance per lookup; no destroy callbacks	§2.6
Proxy	stand-in object that runs extra behaviour and then calls the real bean	§4.4
`Proxyable`	Framework 7 per-bean choice of JDK or CGLIB proxy	§4.4
Publish/subscribe	messaging style where every subscriber gets its own copy of each message	§14.8
`publishOn` / `subscribeOn`	move downstream operators / the subscription to a scheduler	§13.8

Q – R

Term	Meaning	See
`Qualifier`	selects a specific bean at an injection point	§2.5
`QuestionAnswerAdvisor`	Spring AI advisor doing simple RAG over a `VectorStore`	§17.9
R2DBC	reactive (non-blocking) SQL API; not an ORM	§13.14
RabbitMQ	queue broker: exchanges route messages to queues via bindings	§14.18
RAG	retrieval-augmented generation: add retrieved documents to the prompt	§17.9
Rate limiter	cap on calls per time period (Resilience4j, gateway filter)	§16.8
Reactive programming	describing asynchronous data pipelines that are pushed through without blocking threads	§13.1
Reactive Streams	spec: `Publisher`, `Subscriber`, `Subscription`, `Processor` with demand	§13.2
Readiness probe	"send me traffic?" — may include essential dependencies	§15.3
Redis	in-memory key/value store used as a shared cache, session store or lock	§9.6
Reflection	Java code inspecting or calling classes, methods and fields at runtime	§2.4
Refresh (context)	`refresh()` creates all non-lazy singletons and starts the context	§2.2
Refresh token	long-lived token used only to get a new access token without logging in again	§11.1
`RefreshScope`	Spring Cloud scope re-creating beans after a configuration refresh	§16.3
Relaxed binding	several spellings map to one property (`checkout-timeout`, `checkoutTimeout`, `SHOP_CHECKOUTTIMEOUT`)	§3.4
Release train	set of versions released together (Spring Data 2026.0, Spring Cloud 2025.1)	§16.2
Replica / replication	a copy of data kept on another server; replication keeps the copies in sync	§7.12
Repository	Spring Data interface implemented at startup (`JpaRepository`, `CrudRepository`)	§8.8
`REQUIRES_NEW`	propagation that suspends the caller's transaction and runs its own	§7.9
Resilience4j	library for circuit breaker, retry, rate limiter, bulkhead, time limiter	§16.8
Resource server	API that accepts access tokens (`oauth2ResourceServer`)	§11.3
`ResponseEntity`	full control of status, headers and body in MVC	§5.4
`ResponseEntityExceptionHandler`	advice base class mapping Spring MVC exceptions to `ProblemDetail`	§5.7
REST	style of HTTP API built around resources (URLs) and standard methods and status codes	§5.1
`RestClient`	Framework 6.1 fluent blocking HTTP client	§6.2
`RestTemplate`	legacy blocking HTTP client; deprecation announced	§6.9
`RestTestClient`	Framework 7 fluent test client for MockMvc or a real server	§12.9
Retry	calling an operation again after a transient failure, usually with back-off and jitter	§4.7
`Retryable`	Framework 7 retry annotation (1 + `maxRetries` attempts)	§4.7
`RetryableTopic`	spring-kafka non-blocking retries via retry topics	§14.15
`RetryTemplate`	programmatic retry (Framework 7 `org.springframework.core.retry`)	§4.7
Role	authority whose name starts with `ROLE_`	§10.6
Role hierarchy	`ROLE_ADMIN > ROLE_USER`: higher roles include lower ones	§10.6
Rollback-only	flag on a shared transaction that forces a rollback at commit	§7.9
Rollback rules	unchecked → rollback, checked → commit by default	§7.10

S

Term	Meaning	See
Saga	sequence of local transactions across services, with compensating steps that undo earlier ones on failure	§14.17
Salt	random value mixed into each password before hashing so equal passwords give different hashes	§10.5
SAML	older XML-based single sign-on standard used by enterprises; Spring Security supports it	§11.18
Sampling (tracing)	share of traces recorded; Boot default `0.1`	§15.7
Savepoint	point inside a transaction that can be rolled back to (`NESTED`)	§7.9
`Scheduled`	method run on a timer (`fixedRate`, `fixedDelay`, `cron`)	§14.7
Scheduler (Reactor)	thread pool abstraction: `parallel`, `boundedElastic`, `single`	§13.8
Scope	how many bean instances exist and how long they live	§2.6
Scope (OAuth2)	permission requested for an access token (`orders.read`); mapped to `SCOPE_x` authorities	§11.1
Scoped proxy	stand-in that finds the request/session instance on every call	§2.6
Security headers (HSTS, CSP...)	response headers that make browsers safer: force HTTPS, restrict scripts, block framing	§10.11
`securityMatcher`	which requests a whole `SecurityFilterChain` handles	§10.3
`SecurityContextHolder`	`ThreadLocal` holder of the current `Authentication`	§10.2
`SecurityFilterChain`	ordered list of security filters for matching requests	§10.3
Self-invocation	`this.method()` inside a bean bypasses its proxy and all proxy features	§4.5
Service discovery	how a service finds the current addresses of another (Eureka, Consul, Kubernetes DNS)	§16.4
Service mesh	infrastructure layer of sidecar proxies handling traffic, mTLS and retries between services	§16.1
`ServiceConnection`	Boot takes connection details from a Testcontainers container	§12.14
Servlet / servlet container	Java standard API for HTTP request handling / the server that runs servlets (Tomcat, Jetty)	§5.2
Session (HTTP)	server-side state for one browser, found through the `JSESSIONID` cookie	§10.10
Session fixation	attack reusing a pre-login session id; Spring changes the id at login	§10.10
`SessionCreationPolicy`	`IF_REQUIRED`, `ALWAYS`, `NEVER`, `STATELESS`	§10.10
ShedLock	library that lets a `@Scheduled` job run on one instance only	§14.7
SIGTERM / SIGKILL	operating-system signals: "please stop" (allows graceful shutdown) / "stop now" (cannot be handled)	§15.9
Singleton scope	one bean instance per container, shared by all threads (default)	§2.6
Sinks	Reactor API to push values into a `Flux`/`Mono` from imperative code	§13.9
Slice (test)	partial context for one layer (`@WebMvcTest`, `@DataJpaTest`...)	§12.7
`SmartLifecycle`	beans started/stopped with the context, ordered by phase	§2.7
SMTP	Simple Mail Transfer Protocol: how applications send e-mail	§14.4
SPA	single-page application: a browser app (React, Angular...) that calls APIs from JavaScript	§10.8
Span	one timed operation in a trace, with trace id, span id and parent	§15.7
SpEL	Spring Expression Language: `#{...}` in `@Value`, security and cache expressions	§3.5
Spock	Groovy-based test framework; supported again in Boot 4.1	§17.13
Spring AI	Spring project for LLM chat, embeddings, RAG, tools and MCP	§17.1
Spring Authorization Server	Spring's own OAuth2/OIDC authorization server, now part of Spring Security	§11.14
Spring Batch	framework for large, restartable batch jobs (read-process-write in chunks)	§1.3
Spring Boot	Spring + starters + managed versions + auto-configuration + embedded server + Actuator	§1.2
Spring Cloud	projects for distributed systems: config, discovery, gateway, load balancing	§16.1
Spring Cloud Stream	binder-based messaging abstraction over Kafka, RabbitMQ and others	§14.18
Spring Data	family of repository modules sharing one programming model	§9.1
Spring Data JDBC	aggregate-oriented mapping over plain JDBC, no persistence context	§9.2

Term	Meaning	See
Spring Data REST	exports repositories as HAL REST endpoints	§9.3
Spring Framework	the core project: container, AOP, MVC, WebFlux, transactions; Boot builds on it	§1.1
Spring Initializr	start.spring.io project generator	§1.5
Spring Modulith	project for structuring and verifying a modular monolith	§1.3
Spring Security	Spring's authentication and authorization framework, built from servlet filters	§10.1
Spring Session	stores HTTP sessions in Redis or JDBC so all instances share them	§10.10
SQL	Structured Query Language for relational databases	§7.1
SSE	Server-Sent Events: one-way server push over `text/event-stream`	§13.12
SSL	the old name of TLS; still used in property names (`server.ssl.*`)	§3.14
SSO	single sign-on: log in once and reach several applications	§11.2
Stack trace	list of method calls printed with an exception; read the last "Caused by" first	§5.7
Starter	code-free dependency bringing one capability's libraries and auto-configuration	§1.7
Stateless	keeps no per-client data between requests (no HTTP session); every request carries what it needs	§11.1
`StepVerifier`	reactor-test tool that asserts the signals of a publisher	§13.5
Stereotype annotation	`@Component` and its specialisations `@Service`, `@Repository`, `@Controller`	§2.3
Structured logging	JSON log lines (`ecs`, `logstash`, `gelf`), Boot 3.4+	§15.8
Structured output	converting an LLM answer into a Java type (`.entity(...)`)	§17.7
`SpringBootApplication`	`@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`	§1.8
`SpringBootTest`	test that starts the whole application context	§12.6
Swagger UI	web page that renders an OpenAPI description and lets you try the endpoints	§5.12
Synchronous	the caller waits until the operation finishes and has its result	§14.2

T

Term	Meaning	See
`TaskDecorator`	wraps submitted tasks to copy `ThreadLocal`s (MDC) to worker threads	§14.4
TCP	Transmission Control Protocol: reliable, ordered byte stream under HTTP, JDBC and Kafka connections	§6.4
Test pyramid	many fast unit tests, fewer slice tests, few full-context tests	§12.2
`TestConfiguration`	configuration for tests that adds to (not replaces) the app's configuration	§12.6
TestContext Framework	spring-test engine: `SpringExtension`, listeners, context cache	§12.5
Testcontainers	library starting real services in Docker for tests	§12.14
Thread / thread pool	a thread runs code concurrently; a pool keeps a fixed set of threads that take tasks from a queue	§14.4
Thread-per-request	classic servlet model: one thread per request from start to end	§13.1
`ThreadLocal`	variable with a separate value per thread; holds the security context and the transaction	§7.7
Throughput	how much work is done per unit of time (requests or messages per second)	§13.15
Time limiter	Resilience4j pattern that cancels a call taking longer than a set time	§16.8
Timeout (connect / read)	time to establish a connection / to wait for response data	§6.4
TL;DR	"too long; didn't read": the short summary at the end of a section	§1.1
Token exchange	OAuth2 grant (RFC 8693) that swaps one token for another, e.g. for a downstream service	§11.10
Token relay	forwarding the user's access token to downstream services	§11.10
Tool calling	the model requests a function; the application runs it (`@Tool`)	§17.8
Topic (Kafka)	named stream of records in Kafka, split into partitions	§14.8
TOTP	time-based one-time password: the 6-digit code from an authenticator app (a second factor)	§10.4
Trace / `traceparent`	a request's tree of spans / the W3C header that propagates it	§15.7
Transaction	group of database operations that succeed or fail together (ACID)	§7.7
`Transactional`	declarative transaction around a method (proxy-based)	§7.7
`TransactionalEventListener`	listener bound to a transaction phase (`AFTER_COMMIT` default)	§7.15
`TransactionTemplate`	programmatic transaction around a lambda	§7.14
Transient entity	new object unknown to the persistence context	§8.4
TTL	time to live of a cache entry; a provider setting, not an annotation attribute	§9.8

U – W

Term	Meaning	See
Unchecked exception	`RuntimeException` or `Error`; need not be declared; rolls a transaction back by default	§7.10
Undertow	embedded web server removed in Boot 4 (no Servlet 6.1 support)	§17.13
`UnexpectedRollbackException`	commit of a transaction already marked rollback-only	§7.9
Unit of work	set of changes tracked and written together; the persistence context is one	§8.4
Unit test	fast test of one class in isolation, usually without Spring	§12.4
URI / URL	address of a resource; a URL is a URI that also says how to reach it (`https://...`)	§5.1
`UserDetailsService`	loads a user by name for password authentication	§10.4
UUID	128-bit random-looking unique id, often used as a primary key or message id	§8.3
Validation groups	subsets of constraints selected with `@Validated(Group.class)`	§5.6
`Validated`	Spring annotation: validation groups; on a class, method validation via proxy	§5.6
Vault	HashiCorp secret store; Spring Cloud Vault loads secrets from it	§3.8
`Version`	optimistic-locking column (JPA and Spring Data)	§8.14
Vector store	database of embeddings for similarity search (`SimpleVectorStore`, PgVector)	§17.9
Virtual threads	cheap JVM-managed threads (Java 21); `spring.threads.virtual.enabled=true`	§13.15, §15.10
Virtual time	`StepVerifier` clock that advances only when the test says so	§13.5
W3C Trace Context	web standard defining the `traceparent` header used to propagate traces	§15.7
Weaving	linking aspects with targets; Spring does it at runtime with proxies	§4.1
`WebClient`	reactive, non-blocking HTTP client	§13.13
WebFlux	Spring's reactive web framework on Reactor Netty	§13.10
`WebMvcTest`	web-layer test slice	§12.7
WebSocket	long-lived, two-way connection between browser and server	§13.12
`WebTestClient`	fluent test client for WebFlux (and MockMvc)	§13.12
WireMock	stub HTTP server for testing clients against canned responses	§6.8

X – Z

Term	Meaning	See
X25519 / ECDHE	elliptic-curve key-exchange methods TLS uses to agree on a fresh session key	§11.18
XSRF-TOKEN	cookie name in which Spring sends the CSRF token for JavaScript clients to echo back in a header	§10.8
XSS	cross-site scripting: attacker JavaScript running in your page, able to read anything scripts can	§10.11
YAML	indentation-based configuration format (`application.yaml`), an alternative to `.properties`	§3.2

Chapter TL;DR

- **Versions**: Boot 4.1.1 → Framework 7.0.9, Security 7.1.1, Hibernate 7.4, Jackson 3.1, JUnit 6, Java 17+ (25 used); Spring Cloud 2025.1, Spring AI 2.0 and springdoc 3 are versioned separately (§19.2).
- **Annotations** only work with their switch: `@EnableAsync`, `@EnableScheduling`, `@EnableCaching`, `@EnableMethodSecurity`, `@EnableResilientMethods`, `management.observations.annotations.enabled` — and only through the proxy (§19.3).
- **Starters** are modular in Boot 4: `-webmvc` (not `-web`), `-aspectj` (not `-aop`), `-security-oauth2-*`, `-restclient`, `-webclient`, `-flyway`, plus a `-test` starter per technology (§19.4).
- **Defaults worth remembering** (✔): Hikari pool 10 / 30 s wait, `@Async` pool 8 threads `task-`, one scheduler thread, 30 s shutdown phase, only `health` exposed, tracing sampling 0.1, OSIV on, graceful shutdown on (§19.5).
- **Transactions** (✔): `REQUIRED` joins, `REQUIRES_NEW` survives the caller's rollback, `NESTED` = savepoint, `MANDATORY`/`NEVER` throw; checked exceptions **commit**; a caught inner failure → `UnexpectedRollbackException` (§19.6).
- **Beans** (✔): singleton shared, prototype new per lookup and never destroyed; `@Qualifier` > `@Primary` > `@Fallback` > name; init `@PostConstruct` → `afterPropertiesSet` → `initMethod` (§19.7).
- **Security 7**: lambda DSL only; `hasRole("X")` = `ROLE_X`; 401 unknown vs 403 forbidden; CSRF on for cookies, `csrf.spa()` for SPAs; `http.cors()` before authentication (§19.8).

- **Tests**: unit → slice → a few `@SpringBootTest`s; `@MockitoBean` replaced `@MockBean`; explicit `@AutoConfigure...` for test clients in Boot 4; context cache max 32 (§19.9).
- **Choosing**: `RestClient`/HTTP interfaces for blocking calls, MVC + virtual threads for CRUD, WebFlux for streaming, JPA vs `JdbcClient` vs Data JDBC by model, Caffeine local vs Redis shared, Kafka log vs RabbitMQ queues, Resilience4j for circuit breakers (§19.12).
- **Upgrading**: 3.5 first, OpenRewrite, rename starters and properties, Jackson 3 imports, test starters, `@MockitoBean`, lambda-only security (§19.13).

This is the last chapter. Where to go from here:

- Spring Handbook front page — the chapter list, reading paths and how to run the code.
- Back to the start: Chapter 1 — The Spring Ecosystem and Your First App; revision questions: Chapter 18.
- Java Handbook — the language underneath Spring (records, generics, lambdas, concurrency and virtual threads).
- SQL Handbook — the queries, indexes and transaction isolation behind Chapters 7–9.
- Microservices patterns notes — gateway, discovery, circuit breaker, saga, CQRS and outbox, the designs Chapter 16 implements.